

Intel® oneAPI DPC++/C++ Compiler Handbook for FPGAs

Contents

Chapter 1: Intel® oneAPI DPC++/C++ Compiler Handbook for FPGAs Overview

Chapter 2: Introduction To FPGA Design Concepts

FPGA Architecture Overview.....	10
Concepts of FPGA Hardware Design	14
How Source Code Becomes a Custom Hardware Datapath.....	17
Mapping Source Code Instructions to Hardware	18
Scheduling	21
Mapping Parallelism Models to FPGA Hardware	27
Memory Types	36

Chapter 3: Intel® oneAPI FPGA Development

FPGA Flow Terminology	42
Intel® oneAPI FPGA Development Flow.....	43
Types of SYCL* FPGA Compilation.....	45
Separating Device and Host Code Compilation.....	47
FPGA Compilation Flags.....	50
FPGA Workflows in IDEs	54

Chapter 4: Getting Started with the Intel® oneAPI DPC++/C++ Compiler for Intel® FPGA Development

Installing the Intel® oneAPI FPGA Development Environment	55
Simulation Prerequisites.....	55
Installing the Questa*-Intel FPGA Edition Software	55
Set Up the Simulation Environment	56
FPGA Development for Intel® oneAPI Toolkits with Visual Studio* Code	57

Chapter 5: Defining a Kernel for FPGAs

Suggested Kernel Coding Styles	58
Single Work-Item Kernels	62
Single Work-item Kernel Design Guidelines	62
Single-Cycle Floating-Point Accumulator for Single Work-Item Kernels....	65
Strategies for Inferring the Accumulator.....	65

Chapter 6: Debugging and Verifying Your Design

Device Selectors for FPGA.....	67
<code>printf</code> Command Restrictions	68
Emulate and Debug Your Design.....	68
Emulator Environment Variables.....	69
Compile and Emulate Your Design	70
Limitations of the Emulator.....	71
Troubleshooting Discrepancies in Hardware and Emulator Results	72
Emulator Known Issues.....	73
Evaluate Your Kernel Through Simulation.....	74
Debug During Verification.....	74
Compile a Kernel for Simulation	75
Simulate Your Kernel	75

Viewing Simulation Waveforms	77
Troubleshooting Simulator Issues	77
Chapter 7: Analyzing Your Design	
Review the FPGA Optimization Report	79
Loop Analysis	84
Bottlenecks Viewer	87
Area Estimates	87
System Viewer.....	91
Kernel Memory Viewer	97
Schedule Viewer	103
Access FPGA Optimization Reports in JSON Format	104
Quartus (Static) Summary.....	105
Timing Failures	106
Intel® FPGA Dynamic Profiler for DPC++	107
Measure Kernel Performance.....	107
Instrument the Kernel Pipeline with Performance Counters (-Xsprofile) .	108
Obtain Profiling Data During Run Time	108
Invoke the Profiler Runtime Wrapper to Obtain Profiling Data	108
Use Intel® VTune™ Profiler.....	109
Reduce Area Resource Use While Profiling.....	113
Profiler Analyses of Example SYCL* Design Scenarios	113
Limitations	116
Chapter 8: Optimizing Your Kernel	
Chapter 9: Optimizing Your Host Application	
Throughput	120
Resource Use.....	120
System-level Profiling Using the Intercept Layer for OpenCL™ Applications	120
Set Up the Intercept Layer for OpenCL™ Applications	121
Multithreaded Host Application	123
Utilizing Hardware Kernel Invocation Queue	124
Double Buffering Host Utilizing Kernel Invocation Queue	125
Analyzing Buffering Using the Intercept Layer for OpenCL™ Applications	127
N-Way Buffering to Overlap Kernel Execution	128
Prepinning Memory.....	130
Simple Host-Device Streaming	131
Buffered Host-Device Streaming.....	136
Chapter 10: Integrating Your Kernel into DSP Builder for Intel® FPGAs	
Chapter 11: Integrating Your RTL IP Core Into a System	
Synthesize Your RTL IP Core with Quartus® Prime Software	143
Encrypting RTL IP Cores	143
Encrypting RTL IP Cores Without Licensing	144
Encrypting RTL IP Core With Licensing	144
Add an RTL IP Core into a Platform Designer System.....	145
Add an RTL IP Core into a Quartus® Prime Project	145
Chapter 12: RTL IP Core Kernel Interfaces	
Kernel Argument Interfaces	151
Memory-Mapped (MM) Agent Kernel Invocation Interface.....	151
Ready/Valid Handshaking Kernel Invocation Interface.....	156
Memory-Mapped Host Interfaces	158

Memory-Mapped Host Interfaces Using Unified Shared Memory	
Pointers and the <code>annotated_arg</code> Class	158
Buffer Locations in Memory-Mapped Host Interfaces	160
The <code>annotated_arg</code> Template Class	164
Memory-Mapped Host Interfaces Using Accessors	170
Structs in RTL IP Core Interfaces	172
IP core Reset Behavior	172
Chapter 13: Loops	
Refactor the Loop-Carried Data Dependency	174
Relax Loop-Carried Dependency	175
Transfer Loop-Carried Dependency to Local Memory	178
Minimize the Memory Dependencies for Loop Pipelining	178
Unroll Loops	179
Fuse Loops to Reduce Overhead and Improve Performance	181
Optimize Loops With Loop Speculation	182
Remove Loop Bottlenecks	183
Improve f_{MAX}/II with Shannonization	186
Optimize Inner Loop Throughput	187
Improve Loop Performance by Caching Data in On-Chip Memory	190
Chapter 14: Pipes	
Host Pipes	192
Host Pipe Declaration	192
Host Pipe API	196
Host Pipes RTL Interfaces	198
Pipes Extension	201
Key Properties of a Pipe	201
Accessing Pipes	202
The <code>pipe</code> Class and its Use	202
I/O Pipes	205
Emulate Applications with a Pipe That Reads or Writes to an I/O	
Pipe	211
Characteristics of Pipes	212
Restrictions of Pipes	212
Guidelines for Designing Pipes	213
The <code>atomic_fence</code> Function and Pipes	214
Chapter 15: Data Types and Arithmetic Operations	
Optimize Floating-point Operation	216
Minimize the Use of Expensive Functions	217
Variable-Precision Data Type Support	218
Advantages and Limitations of Variable Precision Data Types	219
Declare and Use the AC Data Types	221
Declare the <code>ac_int</code> Data Type	222
Declare the <code>ac_fixed</code> Data Type	223
Declare the <code>ac_complex</code> Data Type	225
Declare the <code>ap_float</code> Data Type	225
Chapter 16: Parallelism	
Pipelined Kernels	234
Stable Arguments	236
NDRange Kernels	236
Asynchronous Parallelism Within Kernels (<code>task_sequence</code>)	237
Task Functions	238

task_sequence Use Cases.....	240
Chapter 17: Memories and Memory Operations	
The device_global Extension (Beta)	242
The annotated_ptr Template Class (Beta).....	244
Memory Accesses	245
Load-Store Units	246
Load-Store Unit Styles.....	246
Load-Store Unit Modifiers.....	247
Load-Store Unit Controls.....	248
Global Memory Accesses Optimization.....	250
Global Memory Bandwidth Use Calculation.....	251
Manual Partition of Global Memory	251
Partitioning Buffers Across Different Memory Types (Heterogeneous Memory).....	252
Partitioning Buffers Across Memory Channels of the Same Memory Type.....	253
Ignoring Dependencies Between Accessor Arguments	254
Contiguous Memory Accesses	254
Static Memory Coalescing	255
Perform Kernel Computations Using Local or Private Memory	257
Local and Private Memory Accesses Optimization.....	257
Annotating Unified Shared Memory Pointers.....	262
Zero-Copy Memory Access	262
Additional Recommendations	263
Kernel Variable Accesses	263
Chapter 18: Libraries	
Use SYCL Shared Library With Third-Party Applications.....	265
Use of RTL Libraries for FPGA	268
Object Manifest File Syntax of an RTL Library	271
Restrictions and Limitations in RTL Support.....	278
Agilex™ 7 and Stratix® 10 Design-Specific Reset Requirements for Stall-Free and Stallable RTL Libraries.....	279
Stall-Free RTL	279
Chapter 19: Additional FPGA Acceleration Flow Considerations	
FPGA-CPU Interaction	281
FPGA Boards and Board Support Packages (BSPs).....	282
The FPGA Board Utility Commands	284
Managing an FPGA Board	285
Advanced Board Management.....	287
Extracting the FPGA Device Image (.aocx) File from a Multiarchitecture Binary File	288
The OpenCL™ Installable Client Driver (ICD)	288
Targeting Multiple Identical FPGA Devices	290
Targeting Multiple Platforms.....	291
Split Kernel into Multiple FPGA Images (Linux only).....	292
Chapter 20: FPGA Optimization Flags, Attributes, Pragmas, and Extensions	
Optimization Flags.....	295
Specify Schedule f _{MAX} Target for Kernels (-Xsclock=<clock target>)	295
Create a 2xclock Interface (-Xsuse-2xclock).....	295
Disable Burst-Interleaving of Global Memory (-Xsno-interleaving)	296

Force Ring Interconnect for Global Memory (-Xsglobal-ring)	296
Force a Single Store Ring to Reduce Area (-Xsforce-single-store-ring) ..	297
Force Fewer Read Data Reorder Units to Reduce Area (-Xsnum-reorder)	297
Disable Hardware Kernel Invocation Queue (-Xsno-hardware-kernel- invocation-queue)	297
Modify the Handshaking Protocol Between Clusters (-Xshyper- optimized-handshaking)	298
Disable Automatic Fusion of Loops (-Xsdisable-auto-loop-fusion)	298
Fuse Adjacent Loops With Unequal Trip Counts (-Xsenable-unequal-tc- fusion)	299
Pipeline Loops in Non-task Kernels (-Xsauto-pipeline)	299
Control Semantics of Floating-Point Operations (-fp-model=<value>) ..	299
Modify the Rounding Mode of Floating-point Operations (- Xsrounding=<rounding_type>)	300
Global Control of Exit FIFO Latency of Stall-free Clusters (-Xssfc-exit- fifo-type=<value>)	301
Enable the Read-Only Cache for Read-Only Accessors (-Xsread-only- cache-size=<N>)	301
Control Hardware Implementation of the Supported Data Types and Math Operations (-Xsdsp-mode=<option>)	302
Generate Register Map Wrapper (-Xsregister-map-wrapper-type)	305
Allow Wide Memory Initialization (-Xsallow-wide-device-globals)	305
Optimization Targets	306
Minimum Latency Flow	306
Minimum Area Flow	306
Balanced Throughput-Area Trade-Offs Flow	307
Kernel Variables	307
Kernel Attributes	308
Specify Schedule f_{MAX} Target for Kernels (scheduler_target_fmax_mhz)	309
Specify a Workgroup Size (max_work_group_size/ reqd_work_group_size)	309
Specify Number of SIMD Work Items (num_simd_work_items)	310
Omit Hardware that Generates and Dispatches Kernel IDs (max_global_work_dim)	311
Omit Hardware that Supports Global Work Offsets (no_global_work_offset)	312
Reduce Kernel Area and Latency (use_stall_enable_clusters)	312
Memory Attributes	313
Loop Directives	315
disable_loop_pipelining Attribute	315
initiation_interval Attribute	316
ivdep Attribute	317
loop_coalesce Attribute	321
max_concurrency Attribute	323
max_interleaving Attribute	324
speculated_iterations Attribute	326
unroll Pragma	326
Loop Fuse Functions and nofusion Attribute	328
max_reinvocation_delay Attribute (Beta)	331
Floating-Point Pragmas	331
Latency Controls (Beta)	333

Appendix A: Quick Reference

Algorithmic C Data Types	337
--------------------------------	-----

Floating Point Pragmas	339
FPGA Accessor Properties	339
FPGA Extensions	340
FPGA Kernel Attributes	342
FPGA Local Memory Function	343
Latency Control Properties (Beta)	343
FPGA LSU Controls	344
FPGA Loop Directives	345
FPGA Memory Attributes.....	348
FPGA Optimization Flags.....	350
Pipe API	351

Appendix B: Additional Information

Appendix C: Document Revision History for the Intel® oneAPI DPC+ +/C++ Compiler Handbook for Intel FPGAs

Notices and Disclaimers	362
--------------------------------------	------------

Intel® oneAPI DPC++/C++ Compiler Handbook for FPGAs Overview

1

The *Intel® oneAPI DPC++/C++ Compiler Handbook for FPGAs* provides guidance on leveraging the functionalities of data parallel C++ (DPC++) and the SYCL* cross-platform abstraction layer to optimize your design.

A good way to learn about SYCL* programming for Altera® FPGA devices is to work through the oneAPI samples for FPGAs available on GitHub: [oneAPI Samples for FPGA](#).

For more information, about SYCL and programming in DPC++, refer to the following publications:

- SYCL* 2020 Specification by the Khronos* Group. (<https://registry.khronos.org/SYCL/specs/sycl-2020/html/sycl-2020.html>)

The SYCL* specification provides detailed descriptions of SYCL concepts and application programming interfaces (APIs).

- *Data Parallel C++: Programming Accelerated Systems Using C++ and SYCL** (<https://link.springer.com/book/10.1007/978-1-4842-9691-2>)

This free book teaches you how to accelerate C++ programs using data parallel programming that targets various device types, including FPGA devices.

Intel® oneAPI DPC++/C++ Compiler Handbook for FPGAs Organization

This guide is organized into the following parts:

- [Introduction To FPGA Design Concepts](#)

A description of FPGA hardware concepts and how the Intel® oneAPI DPC++/C++ Compiler converts your code to an FPGA application.

- Intel® oneAPI FPGA development flow:
 - [Intel oneAPI FPGA Development](#) provides an overview of the FPGA development flow.
 - [Defining a Kernel for FPGAs](#) provides coding guidelines for your kernel
 - [Optimizing Your Kernel](#) provides an overview of methods to optimize your kernel performance and the parts of your kernel code that you can examine to improve performance.
 - [Optimizing Your Host Application](#) provides techniques to optimize the host application part of your multiarchitecture binary. This chapter does not apply for developing RTL IP cores.
 - [Analyzing Your Design](#) provides information about tools and techniques that you can use analyze your kernel design without running the kernel on hardware. Use these tools and techniques to determine what parts of your kernel you can optimize or adjust before you compile your kernel for FPGA hardware.
 - [Debugging and Verifying Your Design](#) provides information about tools and techniques that you can use to verify the functional correctness of your kernel.
 - [Integrating Your RTL IP Core Into a System](#) provides information about how to integrate an RTL IP core generated by the Intel® oneAPI DPC++/C++ Compiler into a larger FPGA design.
- Optimization areas:
 - [RTL IP Core Kernel Interfaces](#)
 - [Loops](#)
 - [Pipes](#)
 - [Data Types and Arithmetic Operations](#)
 - [Parallelism](#)

- [Memories and Memory Operations](#)
- [Libraries](#) covers using libraries in your kernel and generating libraries from your kernel.
- Additional considerations when developing your kernel:
 - [Additional FPGA Acceleration Flow Considerations](#)
- Reference material:
 - [FPGA Optimization Flags, Attributes, Pragmas, and Extensions:](#)

Describes a list of compiler optimization flags, attributes, pragma, and extensions that allow you to customize the kernel compilation process.
 - [Quick Reference:](#)

A cheat sheet of all FPGA-specific attributes, pragmas, and variables.

Introduction To FPGA Design Concepts

2

To maximize the performance of your SYCL® application for FPGA devices, it helps to understand the details of the underlying FPGA hardware and how the Intel® oneAPI DPC++/C++ Compiler converts your code to an FPGA applications.

Later sections in this publication help you to understand the compiler optimizations that convert and map your SYCL® application to FPGAs.

FPGA Architecture Overview

A field-programmable gate array (FPGA) is a reconfigurable semiconductor integrated circuit (IC).

FPGAs occupy a unique computational niche relative to other computing devices, such as central and graphics processing units (CPUs and GPUs), and custom accelerators, such as application-specific integrated circuits (ASICs). CPUs and GPUs have a fixed hardware structure to which a program maps. Conversely, ASICs and FPGAs can build custom hardware to implement a program.

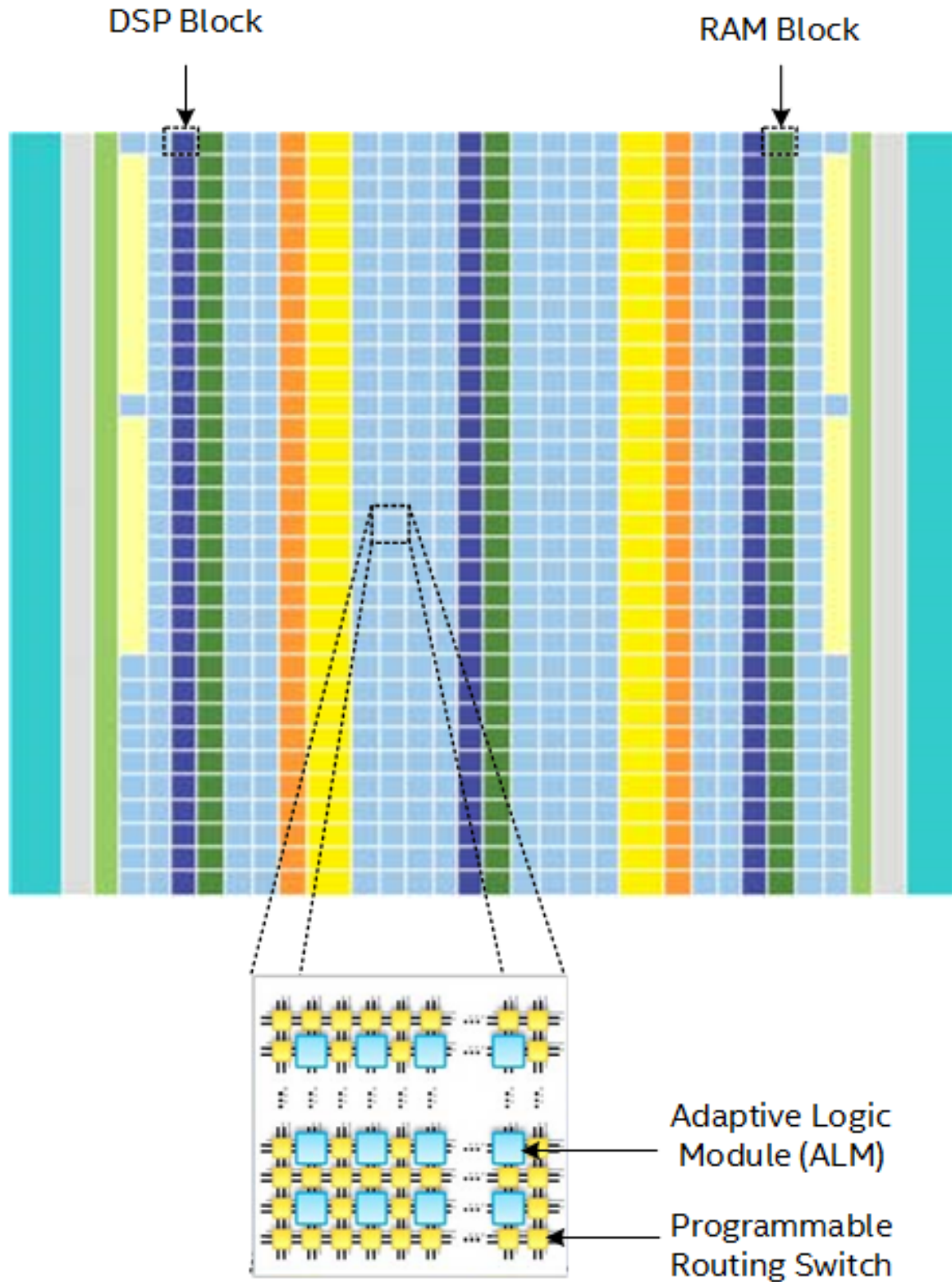
While a custom ASIC generally outperforms an FPGA on a specific task, they take significant time and money to develop. However, FPGAs are a cheaper off-the-shelf alternative that you can reprogram for each new application.

An FPGA is made up of a grid of programmable logic blocks, which are called adaptive logic modules (ALMs) in FPGA devices, and specialized blocks, such as digital signal processing (DSP) blocks and random-access memory (RAM) blocks. These programmable blocks are connected via configurable routing interconnects to implement complete digital circuits.

The total number of ALMs, DSP blocks, and RAM blocks used by a design is often referred to as the *FPGA area* or *area* that the design uses.

The following image illustrates a high-level architectural view of an FPGA:

FPGA Architecture



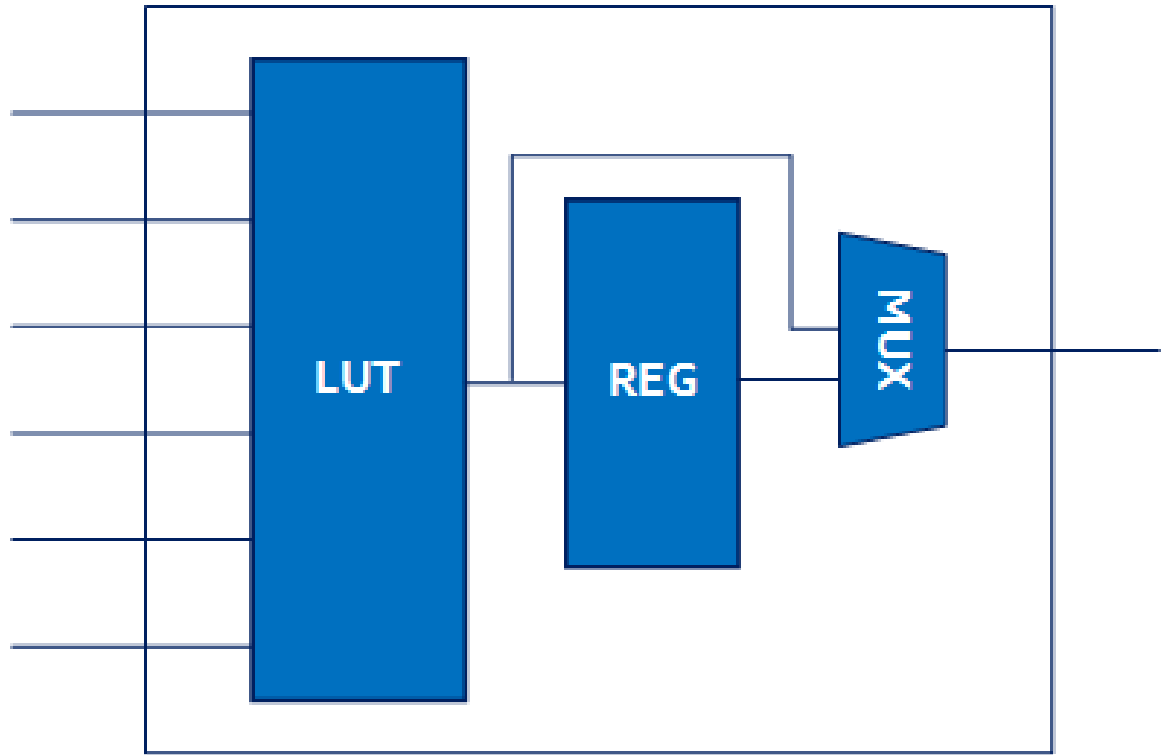
Adaptive Logic Module (ALM)

The basic building block in an FPGA is an adaptive logic module (ALM).

A simplified ALM consists of a [lookup table \(LUT\)](#) and an output register from which the compiler can build any arbitrary Boolean logic circuit.

The following figure illustrates a simplified ALM:

Adaptive Logic Module



Lookup Table (LUT)

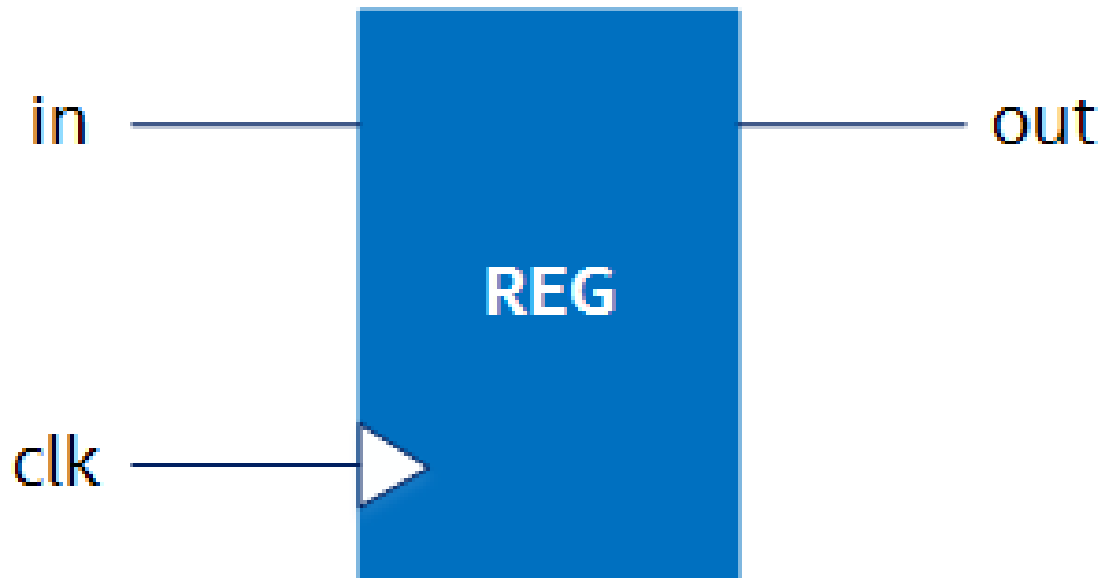
A *lookup table (LUT)* that implements an arbitrary Boolean function of N inputs is often referred to as an N -LUT.

Register

A register is the most basic storage element in an FPGA. It has an input (`in`), an output (`out`), and a clock signal (`clk`). It is synchronous, that is, it synchronizes output changes to a clock. In an ALM, a register may store the output of the LUT.

The following figure illustrates a register:

Register

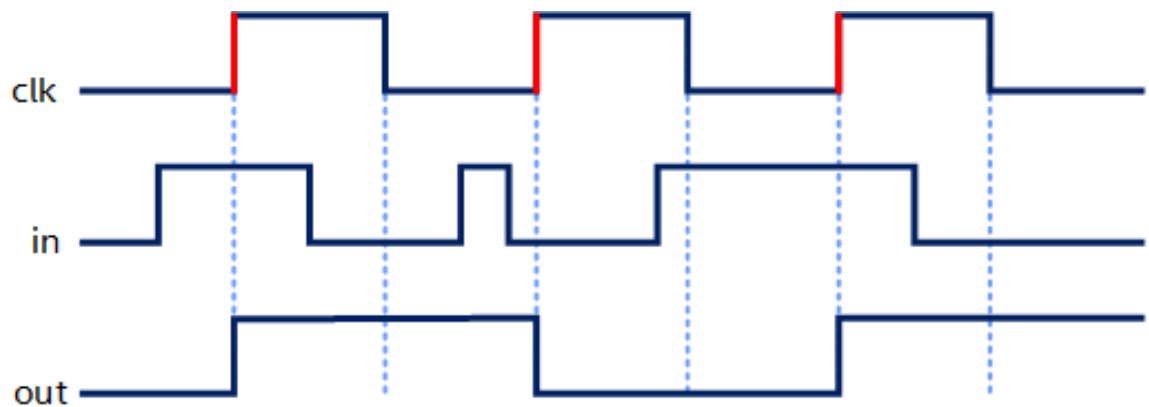


NOTE

The clock signal is implied and not shown in some figures.

The following figure illustrates the waveform of register signals:

Waveform of Register Signals



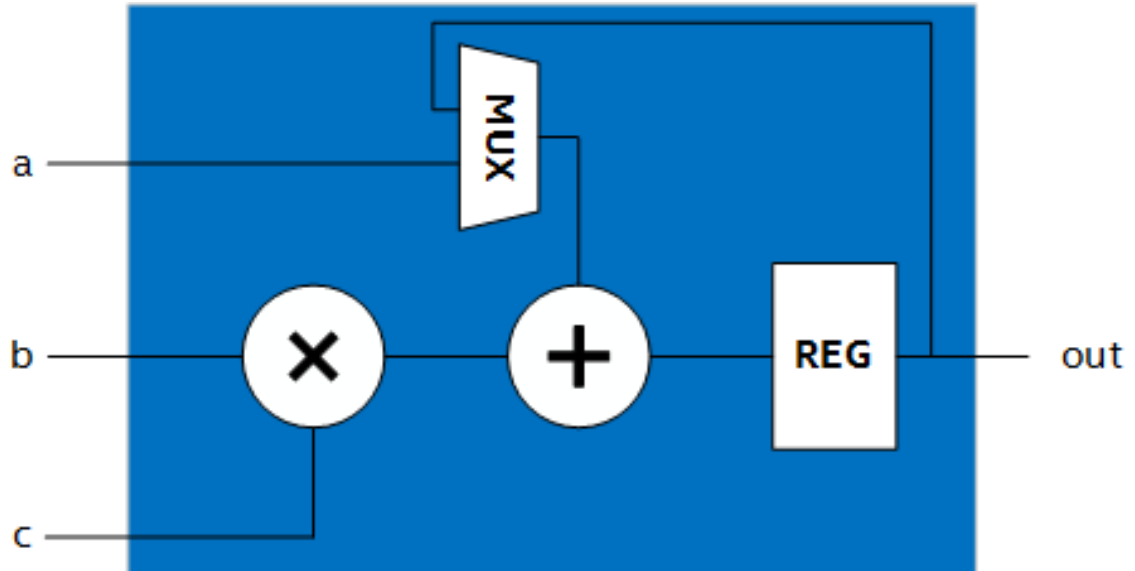
The input data propagates to the output on every clock cycle. The output remains unchanged between clock cycles.

Digital Signal Processing (DSP) Block

A digital signal processing (DSP) block implements specific arithmetic operations (addition and multiplication) that reduce the need to build equivalent logic from general-purpose ALMs. Some FPGAs support floating-point arithmetic in DSP blocks in addition to integer/fixed-point arithmetic. For more information, refer to [Digital Signal Processing](#).

The following figure illustrates a simplified three-input DSP block consisting of a multiplier (×) and an adder (+):

DSP Block



Random Access Memory (RAM) Blocks

A random access memory (RAM) block provides more efficient storage of data than registers, making it suitable for collections of data that need not be accessed all at the same time. RAM blocks may either be implemented with dedicated block RAM modules (sometimes called M10K or M20K modules) or with specially configured ALMs called Memory-Logic Array Blocks (MLAB)s.

For more information, refer to [Memory Types](#).

Concepts of FPGA Hardware Design

FPGA designs are subject to constraints and tradeoffs in the following areas:

- [Maximum Frequency \(\$f_{MAX}\$ \)](#)
- [Latency](#)
- [Pipelining](#)
- [Throughput](#)
- [Datapath](#)
- [Control Path](#)
- [Occupancy](#)

Maximum Frequency (f_{MAX})

The maximum clock frequency at which a digital circuit can operate is called its f_{MAX} . This is the maximum rate at which the outputs of registers are updated.

The physical propagation delay of the signal across combinational logic between two consecutive register stages limits the clock speed. This propagation delay is a function of the complexity of the combinational logic in the path. The path with the most combinational logic elements (and the highest delay) limits the speed of the entire circuit. This speed-limiting path is often referred to as the *critical path* (see [Figure 6: Unpipelined Logic Block with the \$f_{MAX}\$ of 50 MHz and Latency of Two Clock Cycles](#)).

The f_{MAX} is calculated as the inverse of the critical path delay. You may want to have high f_{MAX} since it results in high performance in the absence of other bottlenecks.

Latency

Latency is the measure of how long it takes to complete one or more operations in a digital circuit. You can measure latency at different granularities. For example, you can measure the latency of a single operation or the latency of the entire circuit.

You can measure latency in time (for example, microseconds) or clock cycles. Typically, clock cycles are the preferred way to express latency because measuring latency in clock cycles disconnects latency from your circuit clock frequency. By expressing latency independent of circuit clock frequency, it is easier to discern the true impact of circuit changes to the circuit's performance.

For more information and an example, refer to [Pipelining](#).

Pipelining

Pipelining is a design technique used in synchronous digital circuits to increase f_{MAX} . Pipelining involves adding registers to the critical path, which decreases the amount of logic between each register. Less logic takes less time to execute, which enables an increase in f_{MAX} .

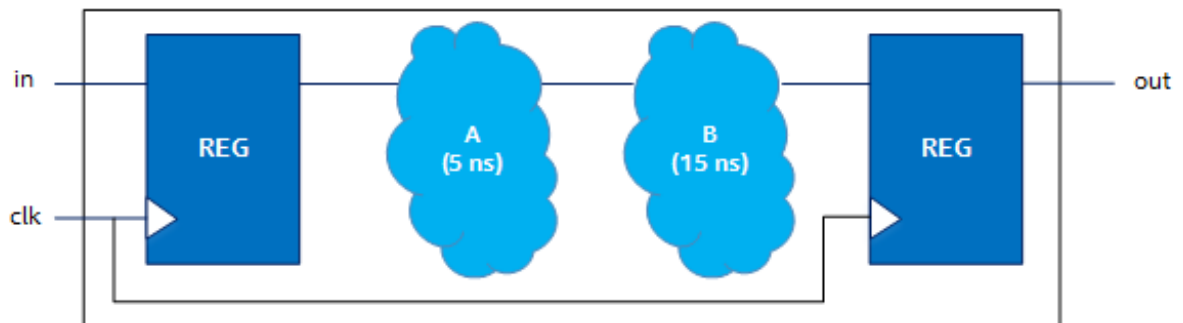
The critical path in a circuit is the path between any two consecutive registers with the highest latency. That is, the path between two consecutive registers where the operations take the longest to complete.

Pipelining is especially useful when processing a stream of data. A pipelined circuit can have different stages of the pipeline operating on different input stream data in the same clock cycle, which leads to better data processing [throughput](#).

Pipelining Example

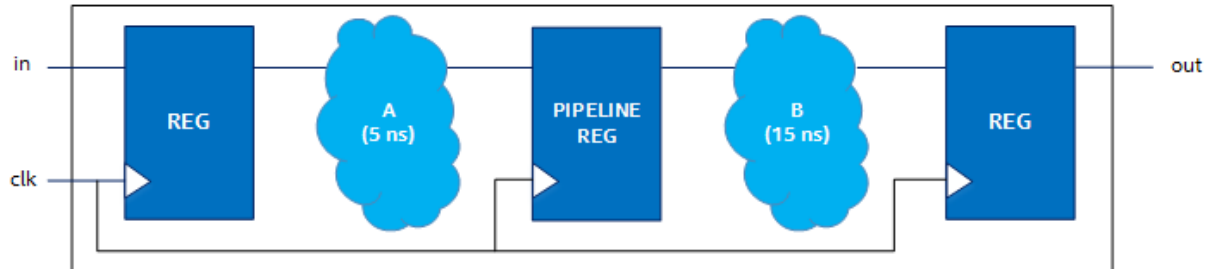
Consider a simple circuit with operations A and B on the critical path. If operation A takes 5 ns to complete and operation B takes 15ns to complete, then the time delay on the critical path is 20 ns. This results in an f_{MAX} of 50 MHz ($1/\text{max_delay}$).

Unpipelined Logic Block with the f_{MAX} of 50 MHz and Latency of Two Clock Cycles



If a pipeline register is added between A and B, the critical path changes. The delay on the critical path is now 15ns. Pipelining this block results in an f_{MAX} of 66.67 MHz, and the maximum delay between two consecutive registers is 15 ns.

Pipelined Logic Block with an f_{MAX} of 66.67 MHz and Latency of Three Clock Cycles



While pipelining generally results in a higher f_{MAX} , it increases latency. In the previous example, the latency of the block containing A and B increases from two to three clock cycles after pipelining.

Throughput

Throughput of a digital circuit is the rate at which data is processed. In the absence of other bottlenecks, higher f_{MAX} results in higher throughput (for example, samples/second).

Throughput is a good measure of the performance of a circuit, and throughput and performance are often used interchangeably when discussing a circuit.

Datapath

A datapath is a chain of registers and combinational logic in a digital circuit that performs computations.

For example, the datapath in [Figure 7: Pipelined Logic Block with an \$f_{MAX}\$ of 66.67 MHz and Latency of Three Clock Cycles](#) consists of all elements shown, from the input register to the last output register.

In contrast, memory blocks are outside the datapath and reads and writes to memory are also considered to be outside of the datapath.

Control Path

While the datapath is the path on which computations occur, the control path is the path of signals that control the datapath circuitry.

The control path is the logic added by the compiler to manage the flow of data through your design. Control paths include controls such as the following:

Control	Description
Handshaking flow control	Handshaking ensures that one part of your design is ready and able to accept data from another part of your design.
Loop control	Loop controls control the data flow through the hardware generated for loops in your code, including any loop carried dependencies.
Branch control	Branch controls implement conditional statements in your code. Branch control can include parallelizing parts of conditional statements to improve performance.

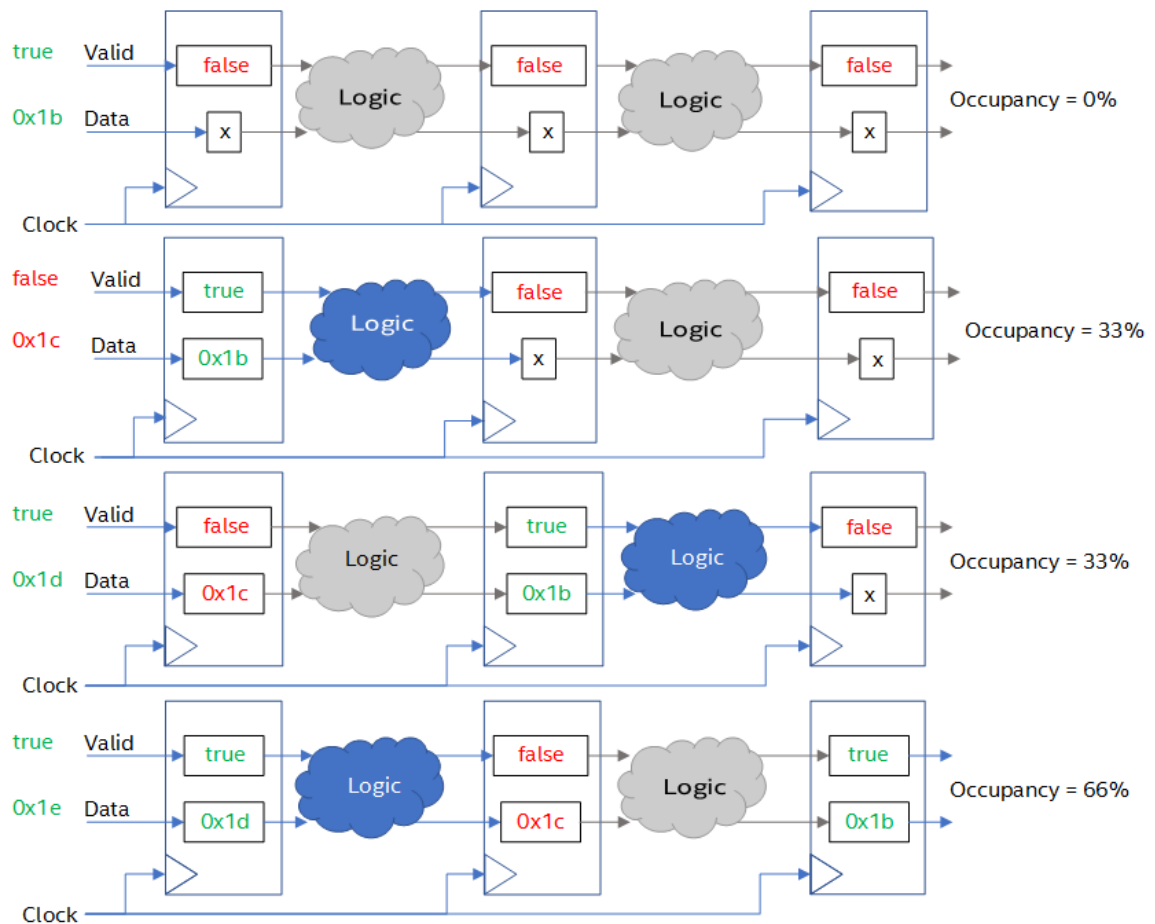
Occupancy

The occupancy of a datapath at a point in time refers to the proportion of the datapath that contains valid data. The occupancy of a circuit over the execution of a program is the average occupancy over time from the moment the program starts to run until it has been completed.

Unoccupied portions of the datapath are often referred to as *bubbles*. Bubbles are analogous to *no-operation (no-ops)* instructions for a CPU that have no effect on the final output.

Decreasing bubbles increase occupancy. In the absence of other bottlenecks, maximizing occupancy of the datapath results in higher throughput.

A Datapath Through Four Iterations Showing A Bubble Traveling Through



How Source Code Becomes a Custom Hardware Datapath

Traditionally, you program an FPGA using a hardware description language (HDL) such as Verilog or VHDL. However, a recent trend is to use higher-level languages. Higher levels of abstraction can reduce the design time and increase the portability of the resulting design.

The rest of this chapter discusses how the compiler maps high-level languages to a hardware datapath.

Mapping Source Code Instructions to Hardware

Based on your source code and the following principles, the Intel® oneAPI DPC++/C++ Compiler builds a custom hardware datapath in the FPGA's logic:

- The datapath is functionally equivalent to the C++ program described by the source. Once the datapath is constructed, the compiler orchestrates your code such that the hardware is effectively occupied.
- The compiler builds the custom hardware datapath while minimizing the area of the FPGA resources (ALMs, DSPs, and so on) used by the design.

Mapping Operations to Hardware

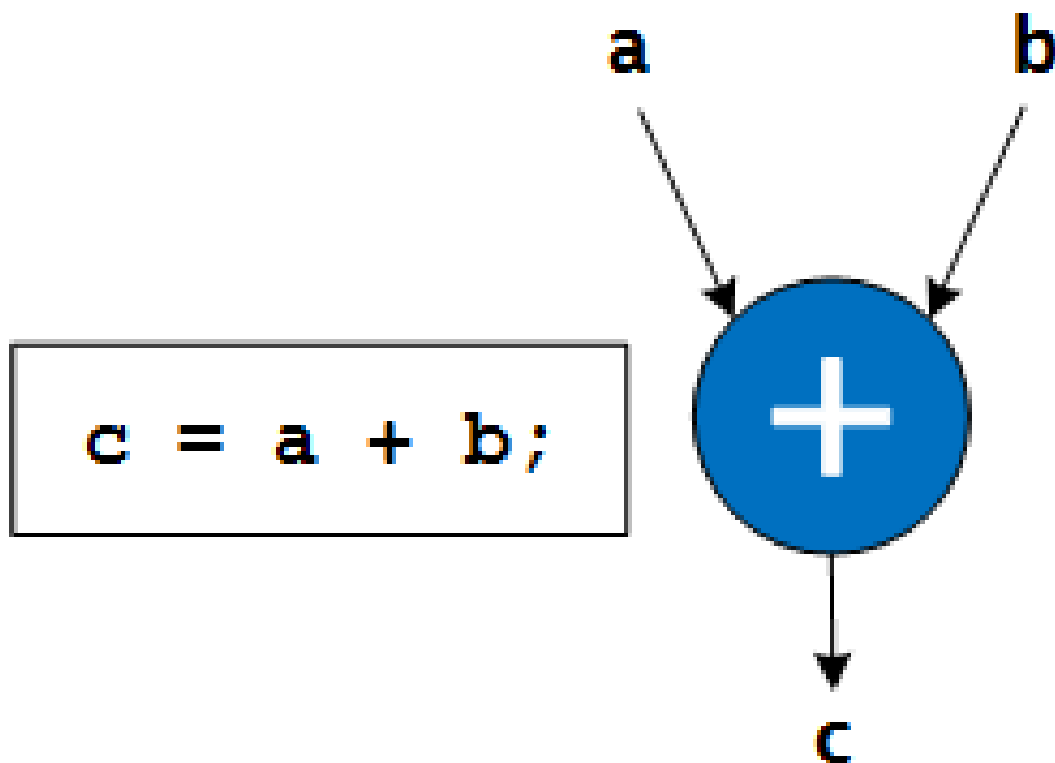
For fixed architectures such as CPUs and GPUs, the source code is compiled by the compiler into a set of instructions that run on functional units with a fixed functionality. For these fixed architectures to be useful in a broad range of applications, some of their available functional units are not useful to every program. Unused functional units mean that your program does not fully occupy the fixed architecture hardware.

FPGAs are not subject to these restrictions of fixed functional units. On an FPGA, you can synthesize a specialized hardware datapath that can be fully occupied for an arbitrary set of instructions, which means you can be more efficient with the chip's silicon area.

By implementing your algorithm in hardware, you can fill your chip with custom hardware that is always (or almost always) working on your problem instead of having idle functional units.

The Intel® oneAPI DPC++/C++ Compiler maps statements from the source code to individual specialized hardware operations, as shown in the example in the following image:

Mapping Source Code Instructions to Hardware



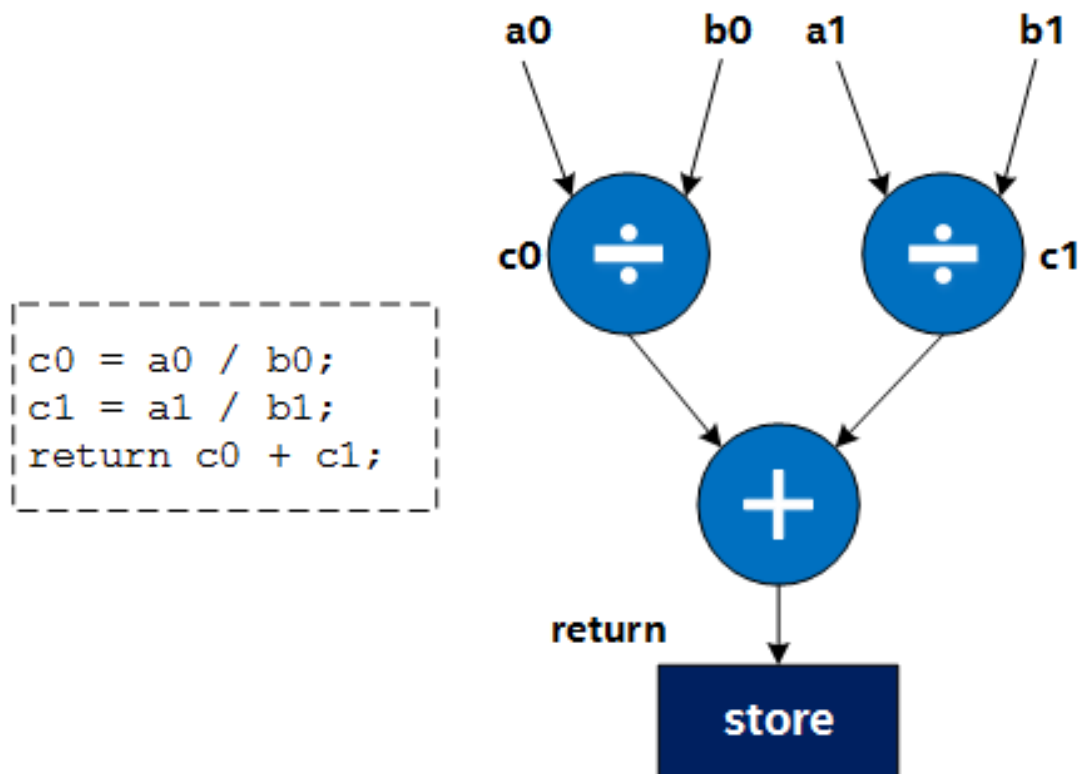
In general, each instruction maps to its own unique instance of a hardware operation. However, a single statement may map to more than one hardware operation, or multiple statements may combine into a single hardware operation when the compiler finds that it can generate more efficient hardware.

The latency of hardware operations is dependent on the complexity of the operation and the target f_{MAX} .

The compiler then takes these hardware operations and connects them into a graph based on their dependencies. When operations are independent, the compiler automatically infers parallelism by executing those operations simultaneously in time.

The following figure illustrates a dependency graph created for the hardware datapath:

Dependency Graph



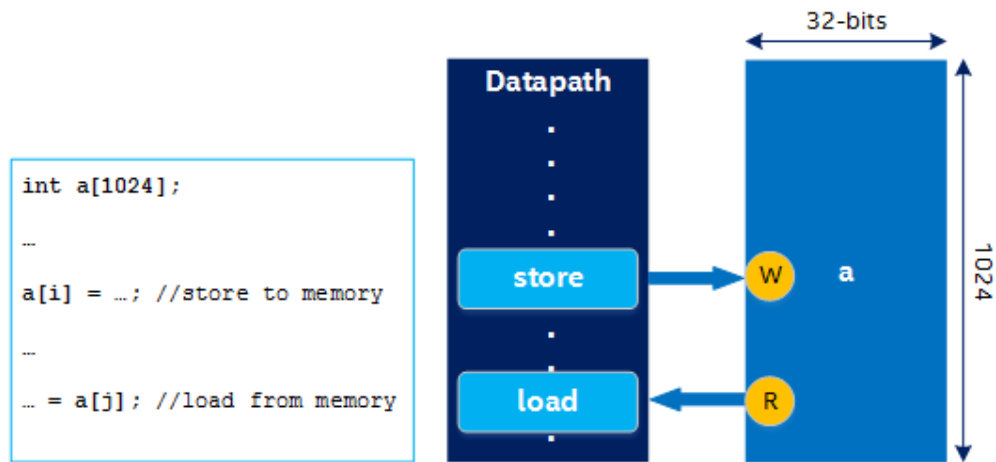
The dependency graph illustrates how the instruction is mapped to hardware operations and how the hardware operations are connected based on their dependencies. The loads in this example instruction are independent of each other and can therefore run simultaneously.

Mapping Arrays and Their Accesses to Hardware

Similar to mapping statements to specialized hardware operations, the compiler maps arrays to hardware memories based on memory access patterns and variable sizes. The datapath interacts with this memory through load/store units (LSUs), which are inferred from array accesses in the source code.

The following figure illustrates a simple example of mapping arrays and their accesses to hardware:

Mapping Arrays and Their Accesses to Hardware



A RAM can have a limited number of read ports and write ports, but a datapath can have many LSUs. When the number of LSUs does not match the available number of read and write ports, the compiler uses techniques like [replication](#), [double-pumping](#), sharing, and arbitration. For more information, refer to [Kernel Memory](#).

NOTE

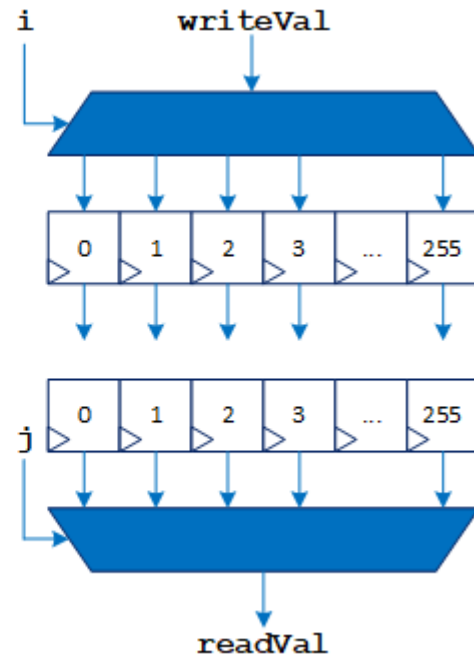
FPGAs provide specialized hardware block RAMs that you can configure and combine to match the size of your arrays. Doing so can provide many terabytes per second of on-chip memory bandwidth because each of these memories can interact with the datapath simultaneously.

Arrays might also be implemented in your kernel datapath. In this case, the array contents are stored as registers in the datapath when your algorithm is pipelined (as discussed in [Pipelining](#)). Storing array contents as registers in the datapath can improve performance in some cases, but it is a design decision whether to implement an array as registers or as memories.

When you access an array that is implemented as registers, LSUs are not used. The compiler might choose to use a select or a barrel shifter instead.

Memory Access via Select

```
int a[256];
a[i] = writeVal; //store to array
readVal = a[j]; // load from array
```



Scheduling

Scheduling refers to the process of determining clock cycles at which each operation in the datapath executes. Pipelining is the outcome of scheduling.

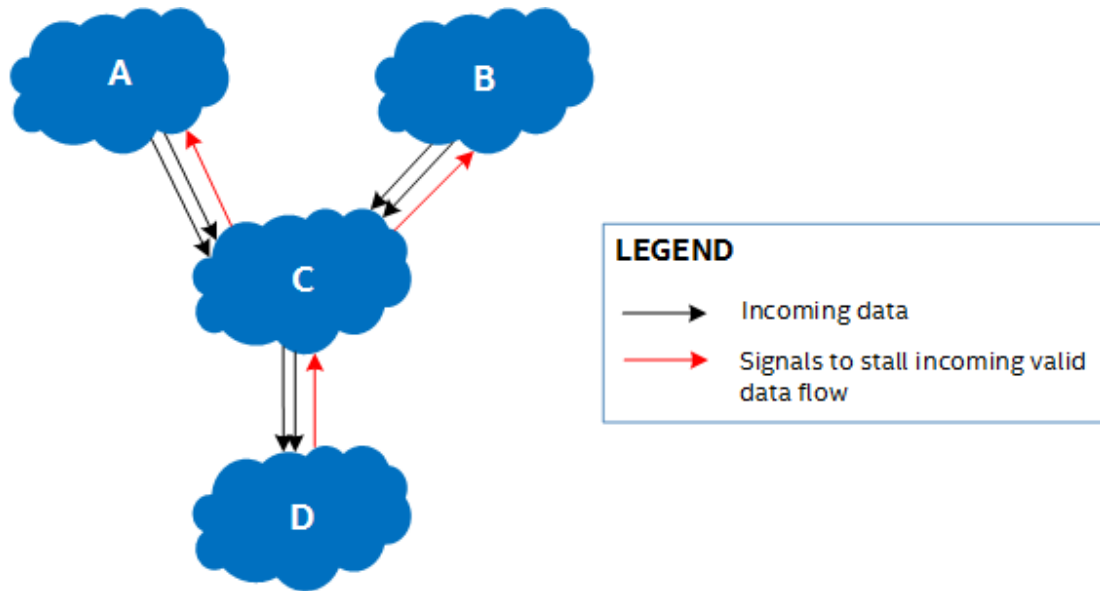
Dynamic Scheduling

The Intel® oneAPI DPC++/C++ Compiler generates pipelined datapath that is dynamically scheduled. A dynamically scheduled portion of the datapath does not pass data to its successor until its successor signals that it is ready to receive it. This signaling is accomplished using handshaking control logic. For example, a variable latency load from memory may refuse to accept its predecessors' data until the load is complete.

Handshaking helps in removing bubbles in the pipeline, thereby increasing occupancy. For more information about bubbles, refer to [Occupancy](#).

The following figure illustrates four regions of dynamically scheduled logic:

Dynamically Scheduled Logic

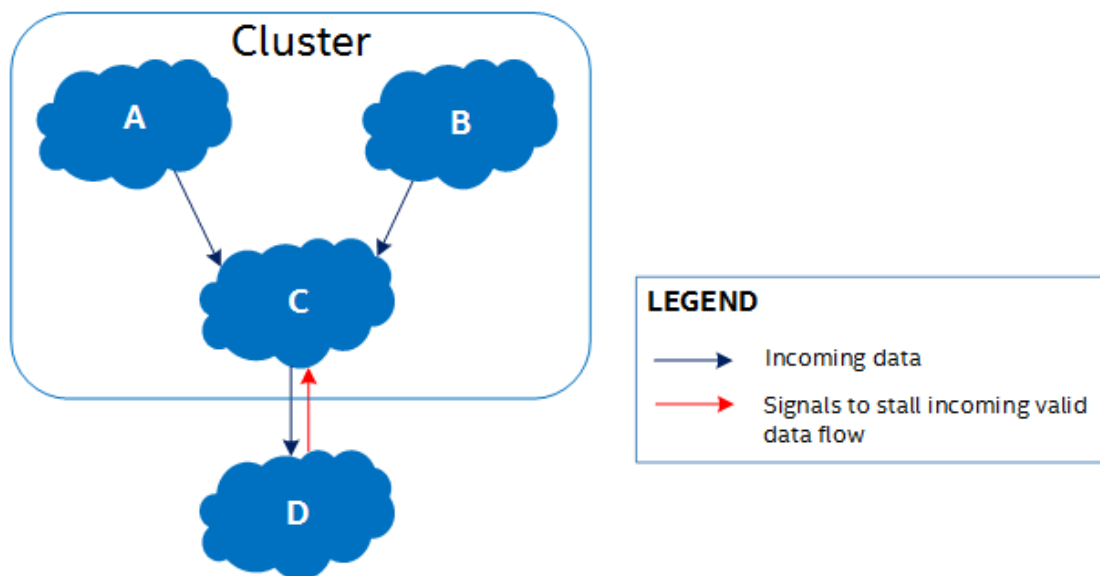


Clustering the Datapath

Dynamically scheduling all operations adds overhead in the form of additional FPGA areas required to implement the required handshaking control logic.

To reduce this overhead, the compiler groups fixed latency operations into clusters. A cluster of fixed latency operations, such as arithmetic operations, requires fewer handshaking interfaces, thereby reducing the area overhead.

Clustered Logic



If A, B, and C from [Figure 13: Dynamically Scheduled Logic](#) do not contain variable latency operations, the compiler can cluster them together, as illustrated in [Figure 14: Clustered Logic](#). Clustering the logic reduces area by removing the need for signals to stall data flow in addition to other handshaking logic within the cluster.

Cluster Types

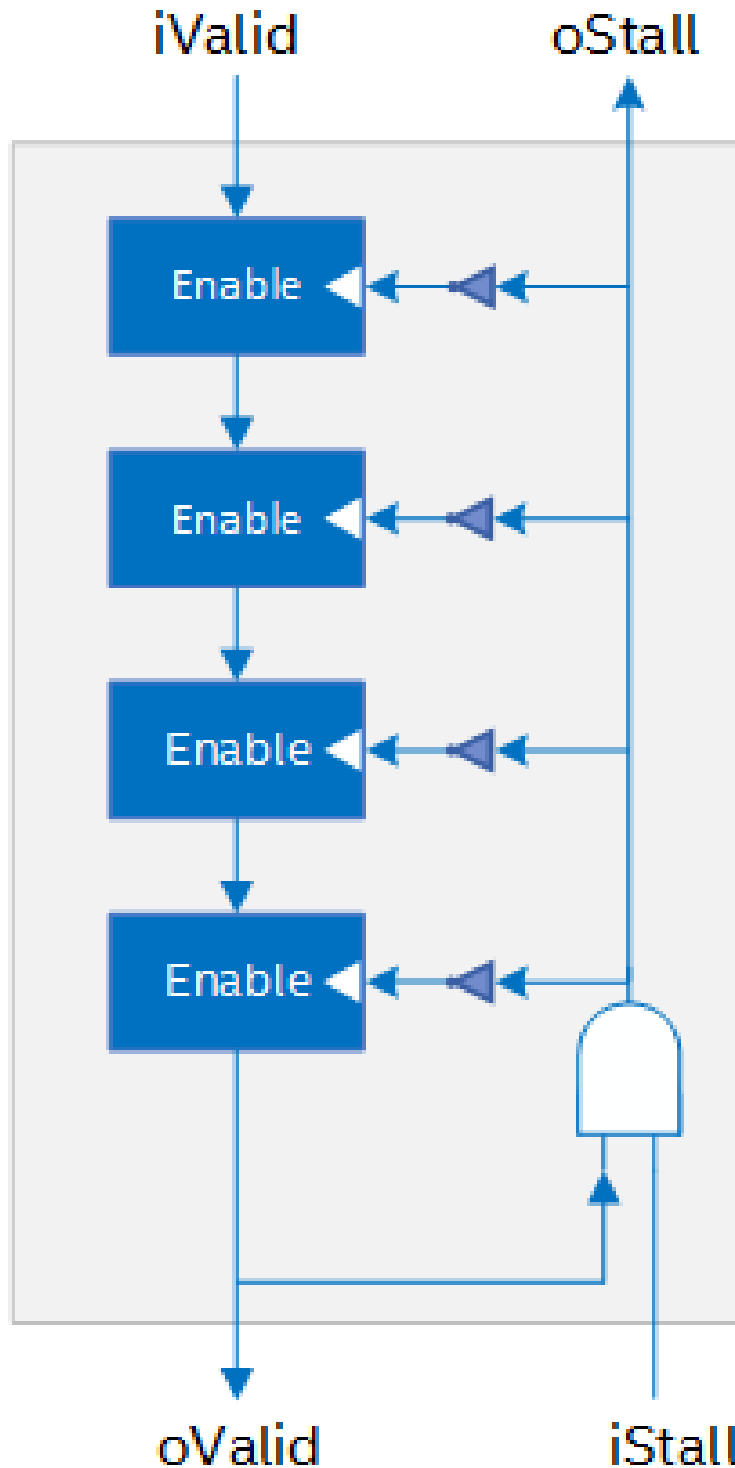
The Intel® oneAPI DPC++/C++ Compiler can create the following types of clusters:

- [Stall-Enable Cluster \(SEC\)](#)

- Stall-Free Cluster (SFC)

Stall-Enable Cluster (SEC)

This cluster type passes the handshaking logic to every pipeline stage in the cluster in parallel. This means that if the cluster is stalled by logic from further down in the datapath, all logic in the SEC stalls simultaneously.



Stall-Free Cluster (SFC)

This cluster type adds a first in, first out (FIFO) buffer to the end of the cluster that can accommodate the entire latency of the pipeline in the cluster. This FIFO is often called an *exit FIFO* because it is attached to the exit of the cluster datapath.

Because of this FIFO, the input to the cluster is not backlogged by the cluster logic. The input to the cluster is always valid, and the output is always valid.

Cluster Characteristics

The exit FIFO of the stall-free cluster results in some of the following tradeoffs:

- **Area:** Because an SEC does not use an exit FIFO, it can save FPGA area compared to an SFC. If you have a design with many small, low-latency clusters, you can save a substantial amount of area by asking the compiler to use SECs instead of SFCs.
- **Latency:** Logic that uses SFCs might have a larger latency than logic that uses SECs because of the write-read latency of the exit FIFO. If you use a zero-latency FIFO for the exit FIFO, you can mitigate the latency, but f_{MAX} or FPGA area use might be negatively impacted. For additional information, refer to [Global Control of Exit FIFO Latency of Stall-free Clusters \(-Xssfc-exit-fifo-type=<value>\)](#).
- **F_{MAX} :** In an SFC, the `oStall` signal has less fanout than in an SEC. For a cluster with many pipeline stages, you can improve your design f_{MAX} by using an SFC.
- **Handshaking:** The exit FIFO in SFCs allow them to take advantage of hyper-optimized handshaking between clusters. For more information, refer to [Hyper Optimized Handshaking](#). SECs do not support this capability.
- **Bubble Handling:** SECs remove only leading bubbles in the pipeline under limited circumstances. A leading bubble is a bubble that arrives before the first piece of valid data arrives in the cluster. SECs do not remove any arriving afterward.

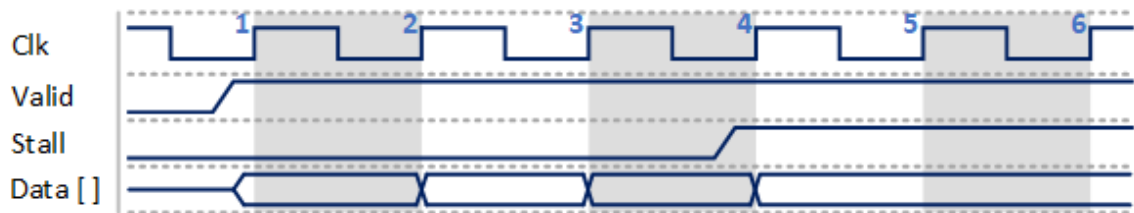
SFCs can use the capacity FIFO to remove all bubbles from the pipeline if the SFC gets a downstream `stall` signal.

- **Stall Behavior:** When an SEC receives a downstream stall, it stalls any logic upstream of it within one clock cycle. When an SFC receives a downstream stall, the exit FIFO allows it to consume additional valid data depending on how deep the exit FIFO is and how many bubbles are in the cluster datapath.

Handshaking Between Clusters

By default, the handshaking protocol between clusters is a simple stall/valid protocol. Data from the upstream cluster is consumed when the `stall` signal is low, and the `valid` signal is high.

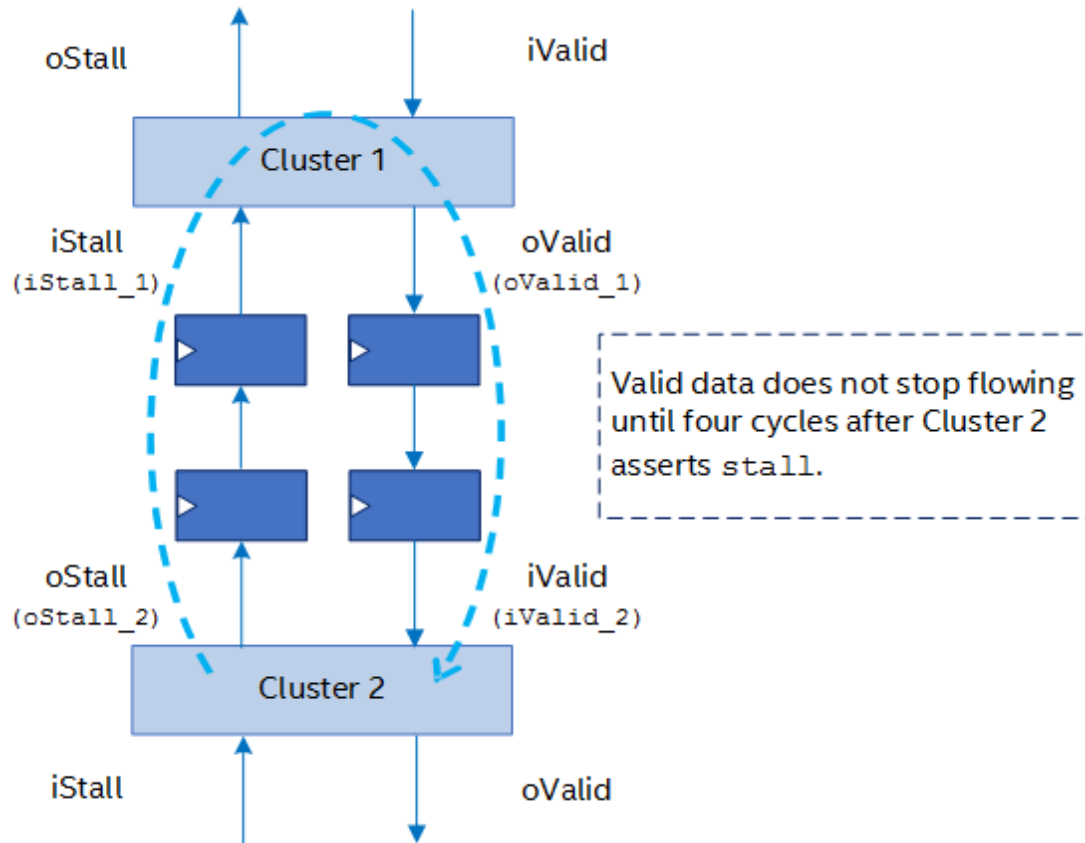
Handshaking Between Clusters



Hyper Optimized Handshaking

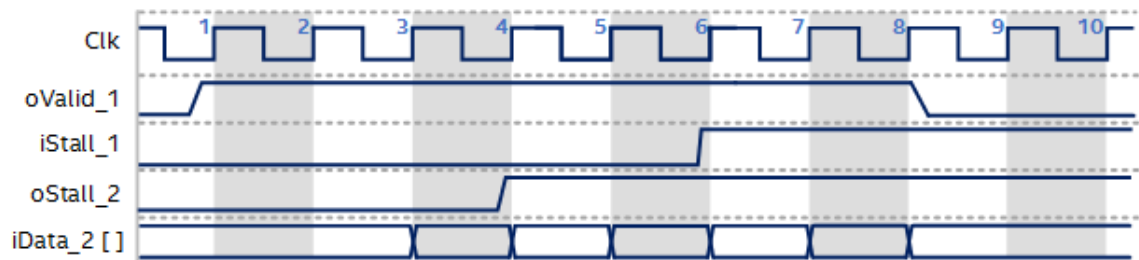
If the distance across the FPGA between these two clusters is large, handshaking may become the critical path that affects peak f_{MAX} in the design. To improve these cases, the Intel® oneAPI DPC++/C++ Compiler can add pipelining registers to the stall/valid protocol to ease the critical path and improve f_{MAX} . This enhanced handshaking protocol is called *hyper-optimized handshaking*.

Hyper-Optimized Handshaking Data Flow



The following timing diagram illustrates an example of upstream cluster 1 and downstream cluster 2 with two pipelining registers inserted in-between:

Hyper-Optimized Handshaking



Restriction

Hyper-optimized handshaking is currently available only for the Agilex™ 7 and Stratix® 10 device families.

Mapping Parallelism Models to FPGA Hardware

This section describes various mechanisms for mapping parallelism models to FPGA hardware:

- [Data Parallelism](#)
- [Task Parallelism](#)

Data Parallelism

Traditional instruction set architecture (ISA)-based accelerators, such as GPUs, derive data parallelism from vectorized instructions and execute the same operation on multiple processing units. In comparison, FPGAs derive their performance by taking advantage of their spatial architecture. FPGA compilers do not require you to vectorize your code. The compiler vectorizes your code automatically whenever it can.

The generated hardware implements data parallelism in the following ways:

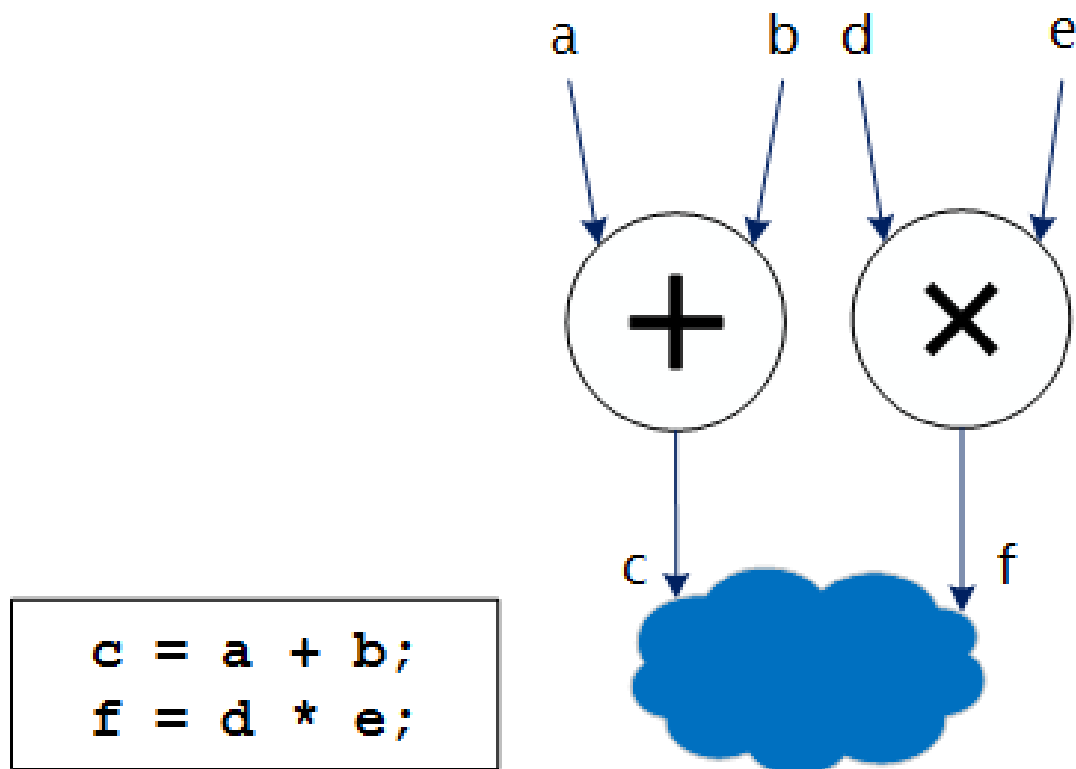
- [Executing Independent Operations Simultaneously](#)
- [Pipelining](#)

Executing Independent Operations Simultaneously

As described in [Mapping Operations to Hardware](#), the compiler can automatically identify independent operations and execute them simultaneously in hardware. This, when combined with pipelining (explained below), is how performance through data parallelism is achieved on the FPGA.

The following image illustrates an example of an adder and a multiplier, which are scheduled to execute simultaneously while operating on separate inputs:

Automatic Vectorization in the Generated Hardware Datapath



This automatic [vectorization](#) is analogous to how a [superscalar processor](#) takes advantage of instruction-level parallelism, but this vectorization happens statically at compile time instead of dynamically, at runtime. This means that there is no hardware or runtime cost of dependency checking for the generated hardware datapath. Additionally, the flexible logic and routing of an FPGA mean that only the available resources (ALMs, DSPs, and so on) of the FPGA bound the number of independent operations operating simultaneously.

Unrolling Loops

You can unroll loops in the design by using loop attributes. Loop unrolling decreases the number of iterations executed at the expense of increasing hardware resource consumption corresponding to executing multiple iterations of the loop simultaneously. Once unrolled, the hardware resources are scheduled as described in [Scheduling](#).

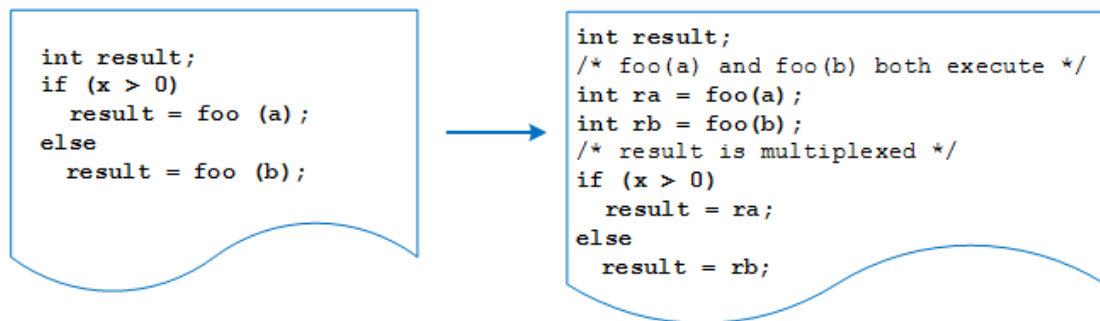
NOTE The Intel® oneAPI DPC++/C++ Compiler never attempts to unroll any loops in your source code automatically. You must always control loop unrolling by using the corresponding pragma. For more information, refer to [Unroll Loops](#), [Loop Directives](#), and [Unrolling Loops](#) tutorial.

Conditional Statements

The Intel® oneAPI DPC++/C++ Compiler attempts to eliminate conditional or branch statements as much as possible. Conditionally executed code becomes predicated in the hardware. Predication increases the possibilities for executing operations simultaneously and achieving better performance. Additionally, removing branches allows the compiler to apply other optimizations to the design.

In the following example, the function `foo` runs unconditionally. The code that cannot be run unconditionally, like the memory assignments, retains a condition.

Conditional Statements



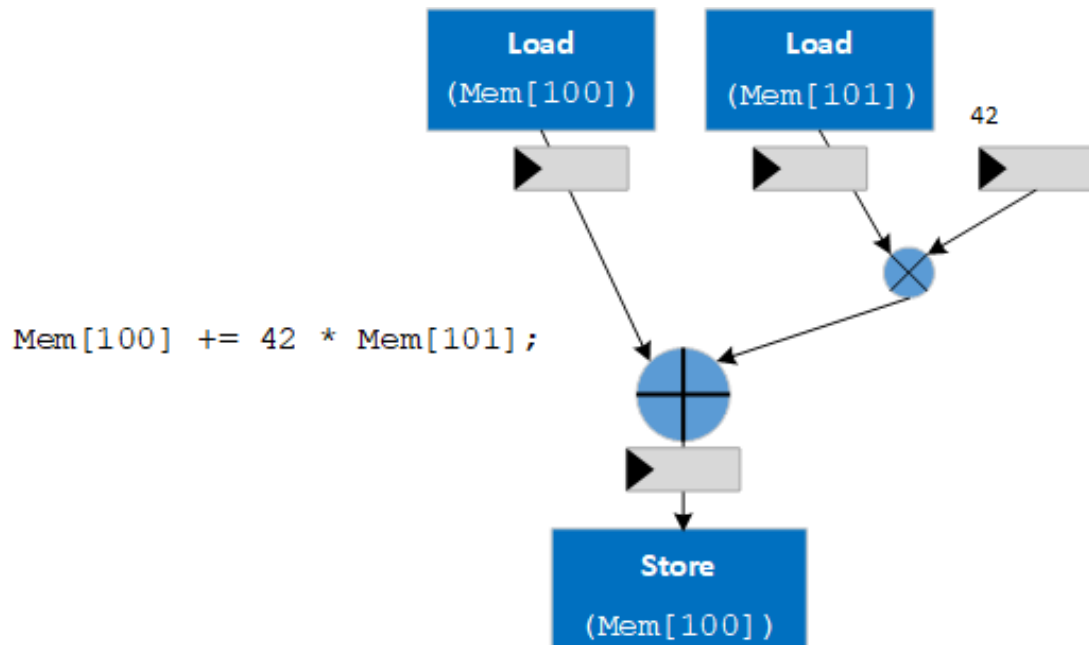
Pipelining

Similar to implementing a CPU with multiple pipeline stages, the compiler generates a deeply pipelined hardware datapath. For more information, refer to [Concepts of FPGA Hardware Design](#) and [Mapping Source Code Instructions to Hardware](#). Pipelining allows for many data items to be processed concurrently while efficiently using the hardware in the datapath by keeping it occupied.

Example of Vectorization of the Datapath vs. Pipelining the Datapath

Consider the following example of code mapping to hardware:

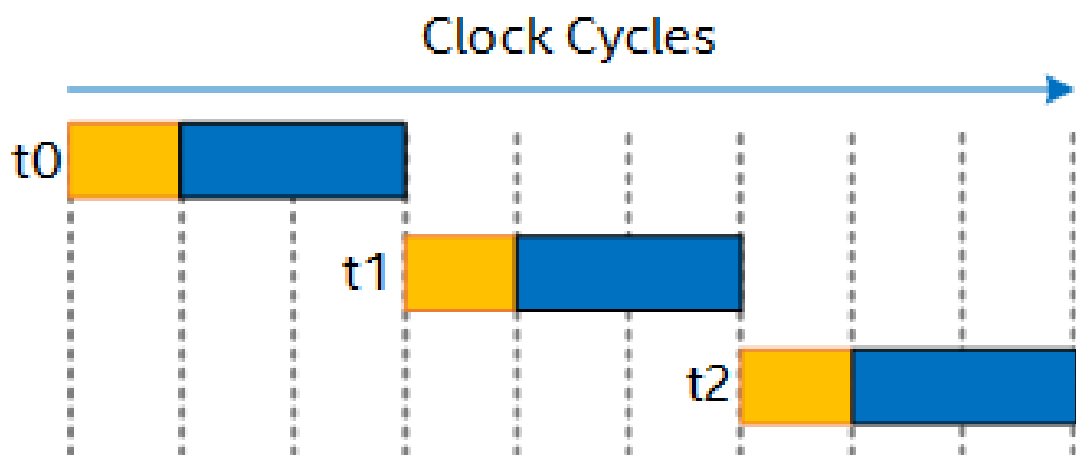
Example Code Mapping to Hardware



Multiple invocations of this code when running on a CPU would not be pipelined. The output of an invocation is completed before inputs are passed to the next invocation of the code. On an FPGA device, this kind of unpipelined invocation results in poor throughput and low occupancy of the datapath because many of the operations are sitting idle while other parts of the datapath are operating on the data.

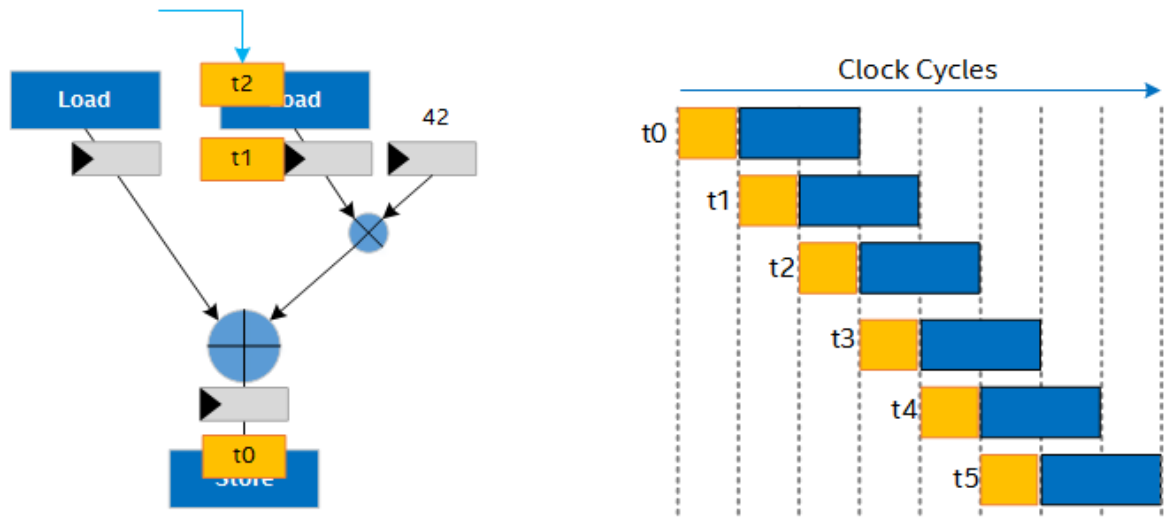
The following figure shows what throughput and occupancy of invocations look like in this scenario:

Unpipelined Execution Resulting in Low Throughput and Low Occupancy



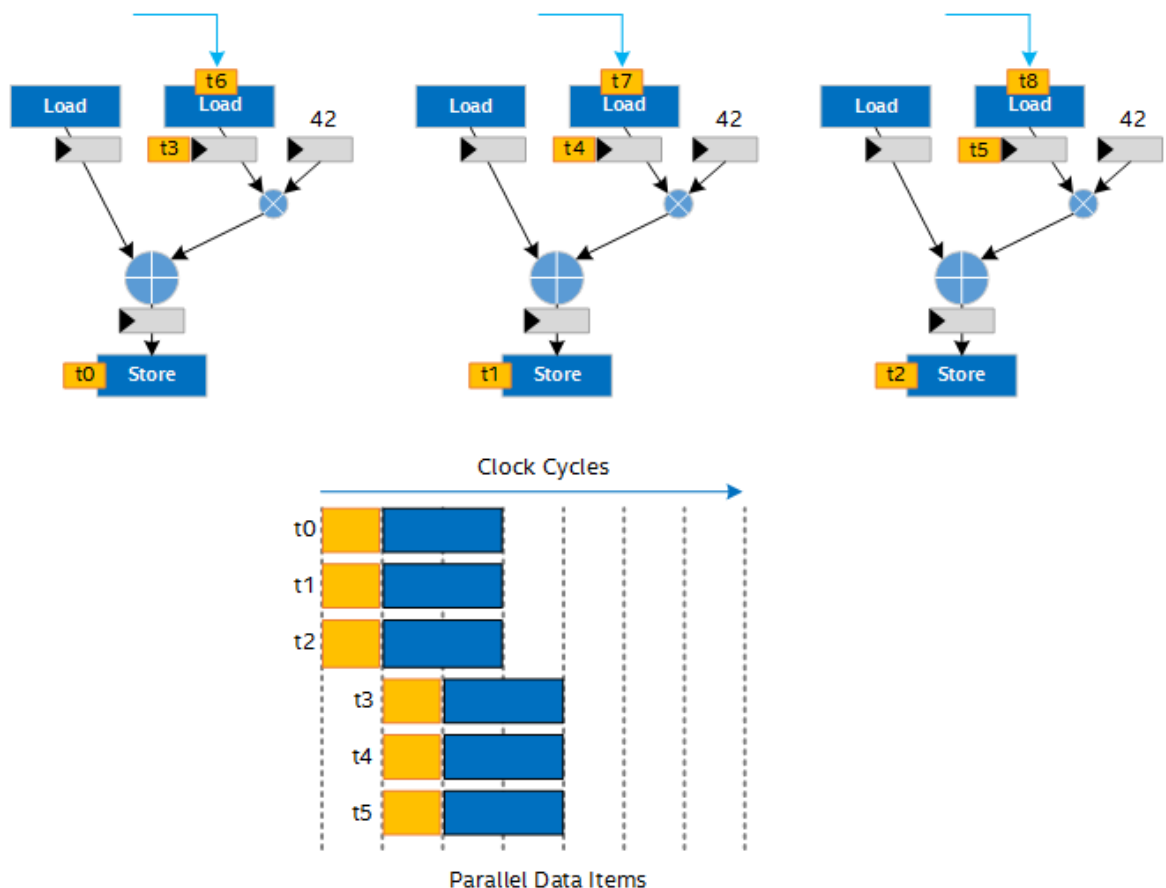
The Intel® oneAPI DPC++/C++ Compiler pipelines your design as much as possible. New inputs can be sent into the datapath each cycle, giving you a fully occupied datapath for higher throughput, as shown in the following figure:

Pipelining the Datapath Results in High Throughput and High Occupancy



You can gain even further throughput by vectorizing the pipelined hardware. Vectorizing the hardware improves throughput but requires more FPGA area for the additional copies of the pipelined hardware:

Vectorizing the Datapath Resulting in High Throughput and High Occupancy



Intel® oneAPI DPC++/C++ Compiler can apply the principles of datapath pipelining across the following items:

- Sequential iterations of a loop within a kernel
- Sequential work item that are processed by the same kernel

Advanced pipelining Multiple invocations of a [task sequence](#) can also be pipelined. For more details, refer to the [Hardware Reuse tutorial on GitHub](#).

Pipelining Loops

The Intel® oneAPI DPC++/C++ Compiler can apply the principles of datapath pipelining to loops in your code. When the Intel® oneAPI DPC++/C++ Compiler pipelines a loop, it attempts to schedule the loop's execution such that the next iteration of the loop enters the pipeline before the previous iteration has completed. This pipelining of loop iterations can lead to higher performance.

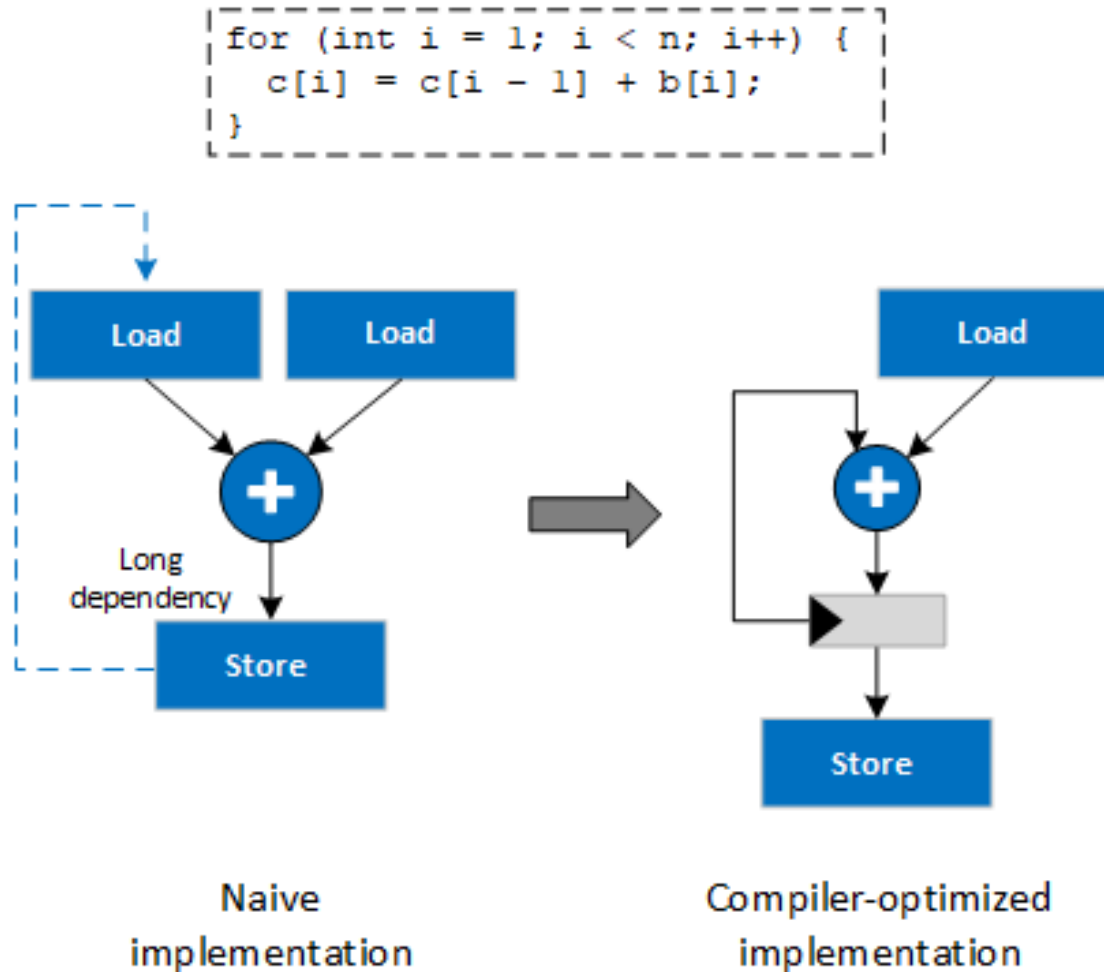
The number of clock cycles between iterations of the loop is called the *Initiation Interval* (II). For the highest performance, a new iteration of the loop would start every clock cycle corresponding to an II of 1. Data dependencies that are carried from one iteration of the loop to another can affect the ability to achieve an II of 1. These dependencies are called *loop carried dependencies*. The II of the loop must be high enough to accommodate all loop carried dependencies.

NOTE

The II required to satisfy this constraint is a function of the f_{MAX} of the design. If the f_{MAX} is lower, the II might also be lower. Conversely, if the f_{MAX} is higher, a higher II might be required. The Intel® oneAPI DPC++/C++ Compiler automatically identifies these dependencies and builds hardware to resolve them while minimizing the II, subject to the target f_{MAX} .

Naively generating hardware for the code in [Figure 26: Pipelining a Datapath with Loop Iteration](#) results in two loads: one from memory b and one from memory c. Because the compiler knows that the access to `c[i-1]` was written to in the previous iteration, the load from `c[i-1]` can be optimized away.

Pipelining a Datapath with Loop Iteration



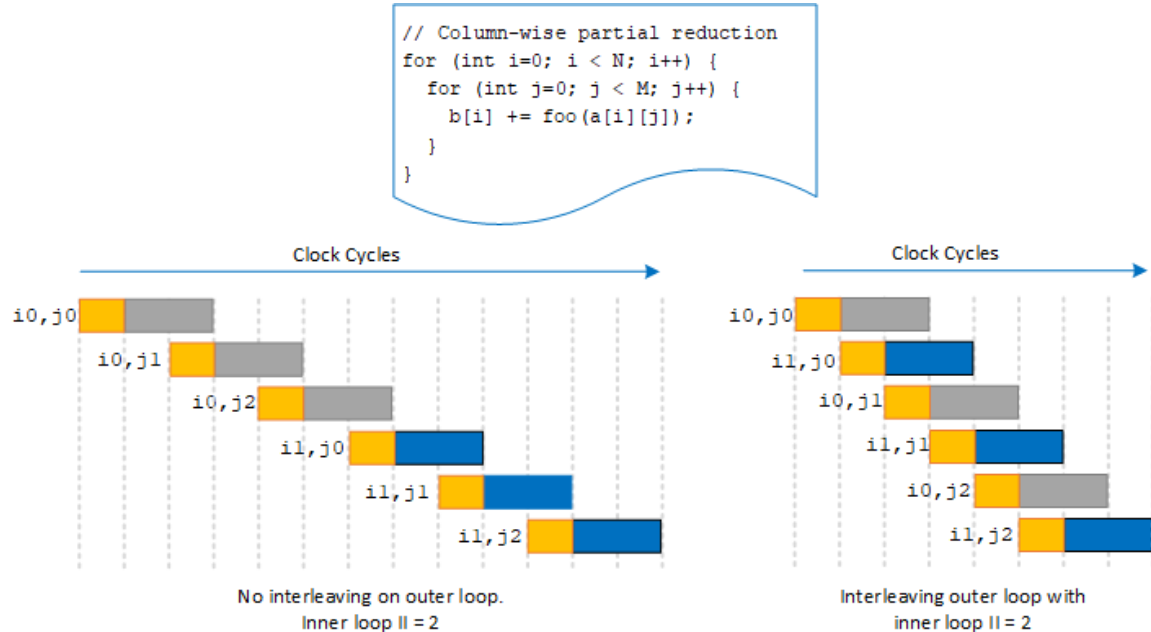
NOTE

The dependency on the value stored to `c` in the previous iteration is resolved in a single clock cycle, so an II of 1 is achieved for the loop even though the iterations are not independent.

In cases where the Intel® oneAPI DPC++/C++ Compiler cannot initially achieve II of 1, you have several optimization strategies to choose from:

- **Interleaving:** When a loop nest with an inner loop II is greater than 1, the Intel® oneAPI DPC++/C++ Compiler can attempt to interleave iterations of the outer loop into iterations of the inner loop to utilize the hardware resources better and achieve higher throughput.

Interleaving



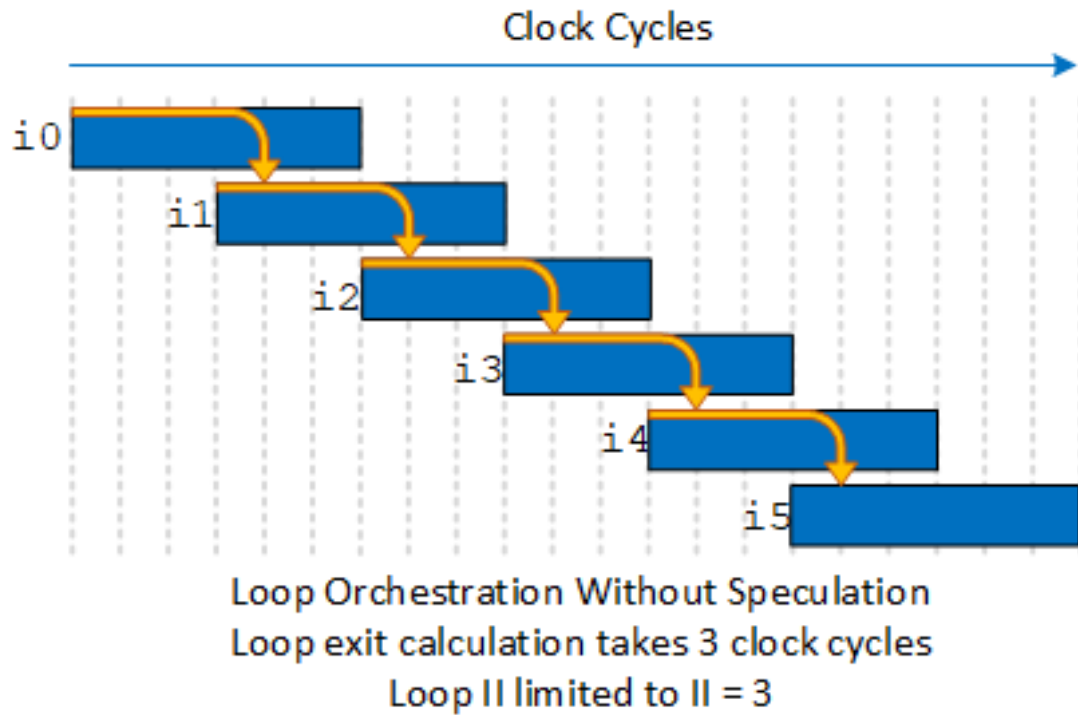
For additional information about controlling interleaving in your kernel, refer to [max_interleaving Attribute](#).

- **Speculative Execution:** When the critical path affecting II is the computation of the loop's exit condition and not a loop carried dependency, the Intel® oneAPI DPC++/C++ Compiler can attempt to relax this scheduling constraint by speculatively continuing to execute iterations of the loop while the exit condition is being computed. If it is determined that the exit condition is satisfied, the effects of these extra iterations are suppressed. This speculative iteration can achieve lower II and higher throughput, but it can incur additional overhead between the loop invocations (equivalent to the number of speculated iterations). A larger loop trip count helps to minimize this overhead.

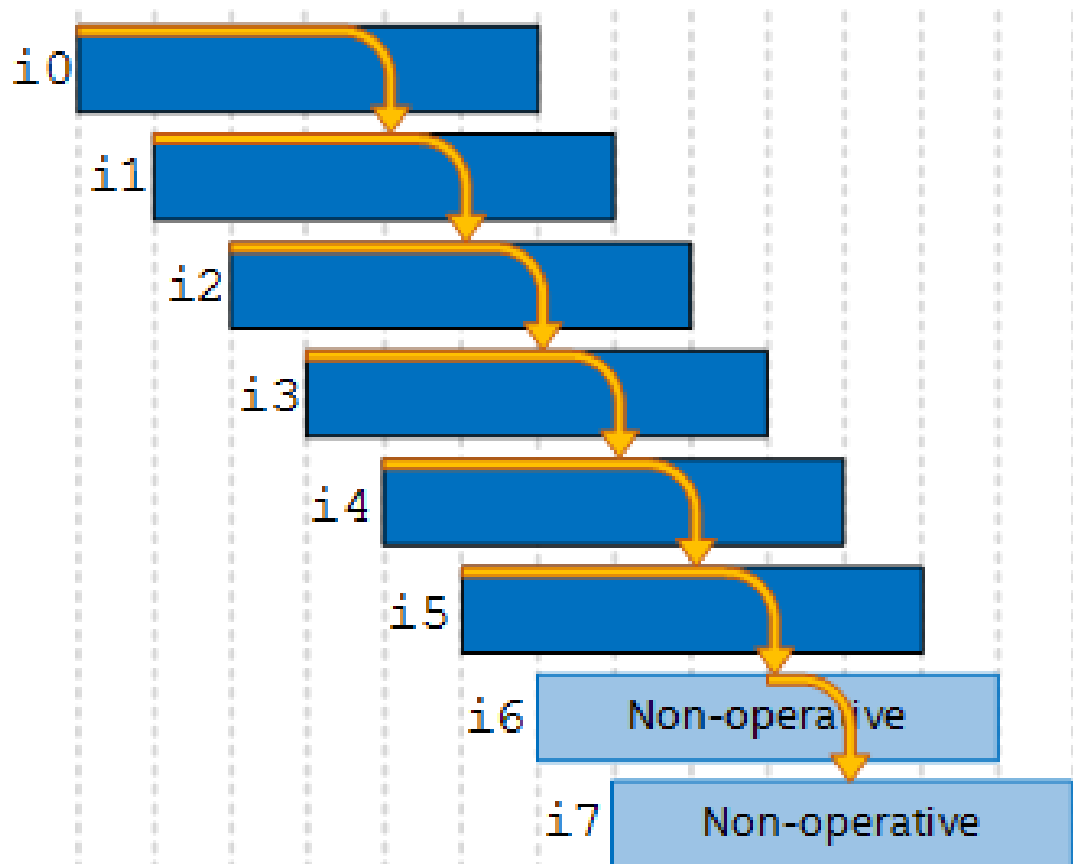
NOTE

A loop invocation is what starts a series of loop iterations. One loop iteration is one execution of the body of a loop.

Loop Orchestration Without Speculative Execution



Loop Orchestration With Speculative Execution



These optimizations are applied automatically by the Intel® oneAPI DPC++/C++ Compiler, and they can also be controlled through `pragma` statements and loop attributes in the design. For additional information, refer to [speculated_iterations Attribute](#)

Pipelining Across Multiple Work Items

The SYCL specification defines a *work-item* as a specific instance/invocation of a kernel (that is, a unique call to `single_task` or an iteration of `parallel_for`). A workgroup is a range of work-items, and represents a large set of independent work that can be executed in parallel. Because there are no implicit dependencies between work items, at each clock cycle, a new work item can always enter a kernel datapath before previous work items have completed, unless there is a dynamic stall in the datapath. The following figure illustrates the pipeline in [Figure 22: Example Code Mapping to Hardware](#) filled with work items:

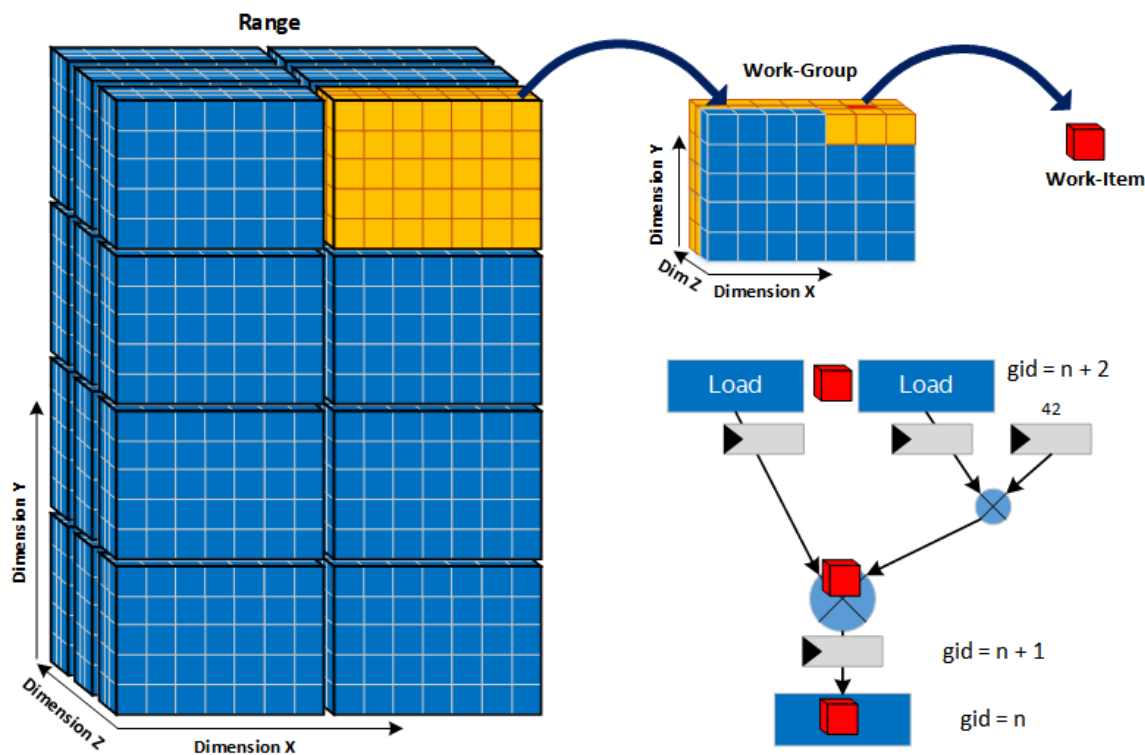
Important

Loops are not pipelined by default for kernels that use more than one work item.

Because there are no implicit dependencies between work items, you must manage accesses to shared resources such as pipes, global memory, and `device_global` variables yourself to avoid race conditions between your work items. For example, you can manage accesses at a coarse-grained level by waiting for a kernel to finish before launching another kernel if the kernels access the same resource. At a fine-grained level, you can use atomic operations.

Another option is to convert your kernel to a single work-item kernel by moving host-code loops that re-invoke a kernel into device code. This implementation avoids race conditions because iterations of a loop within a kernel guarantee serial equivalence, and can be pipelined as described earlier.

Pipelining a Datapath with SYCL* Work Items

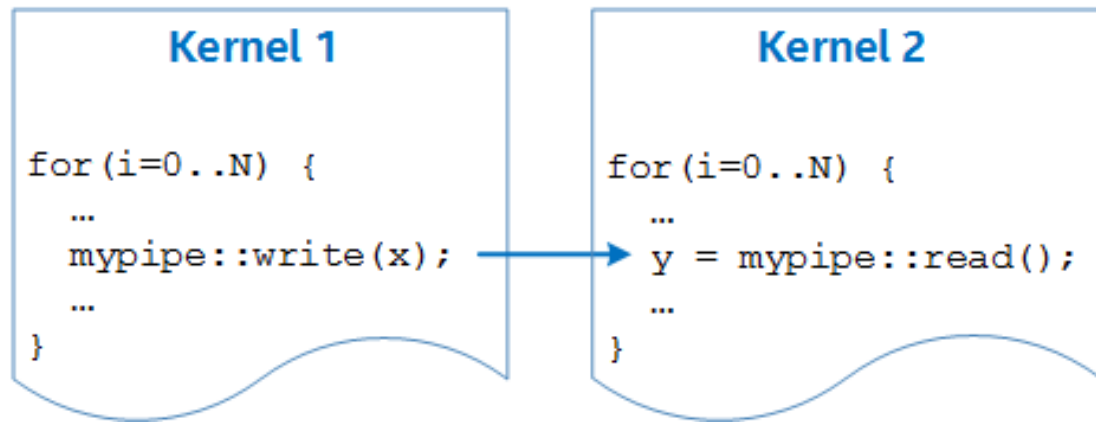


Task Parallelism

While the compiler achieves concurrency by scheduling independent individual operations to execute simultaneously, it does not achieve concurrency at coarser granularities (for example, across loops).

For larger code structures to execute in parallel, you must write them as separate kernels that launch simultaneously. These kernels then run asynchronously with each other, and you can achieve synchronization and communication using pipes, as illustrated in the following figure:

Multiple Kernels Running Asynchronously



This is similar to how a CPU program can leverage threads running on separate cores to achieve simultaneous asynchronous behavior.

Memory Types

The compiler maps the user-defined arrays in the source code to hardware memories. You can classify these hardware memories into [Kernel Memory](#) and [Global Memory](#).

Kernel Memory

If you declare a private array, a [group local memory](#), or a local accessor, then the Intel® oneAPI DPC++/C++ Compiler creates a kernel memory in hardware. Kernel memory is sometimes referred to as *on-chip memory* because it is created from memory sources (such as RAM blocks) available on the FPGA. The following source code snippet illustrates both a kernel and a global memory and their accesses:

```
constexpr int N = 32;
Q.submit([&](handler &cgh) {
    // Create an accessor for device global memory from buffer buff
    accessor acc(buff, cgh, write_only);
    cgh.single_task<class Test>([=]() {
        // Declare a private array
        int T[N];

        // Write to private memory
        for (int i = 0; i < N; i++)
            T[i] = i;

        // Read from private memory and write to global memory through the accessor
        for (int i = 0; i < N; i+=2)
            acc[i] = T[i] + T[i+1];
    });
});
```

To allocate local memory that is accessible to and shared by all work items of a workgroup, define a group-local variable at the function scope of a workgroup using the `group_local_memory_for_overwrite` function, as shown in the following example:

```
Q.submit([&](handler &cgh) {
    cgh.parallel_for<class Test>(
        nd_range<1>(range<1>(128), range<1>(32)), [=](nd_item<1> item) {
            auto ptr = group_local_memory_for_overwrite<int[64]>(item.get_group());
            auto& ref = *ptr;
            ref[2 * item.get_local_linear_id()] = 42;
        });
});
```

The example above creates a kernel with four workgroups, each containing 32 work items. It defines an `int[64]` object as a group-local variable, and each work-item in the workgroup obtains a `multi_ptr` to the same group-local variable.

The compiler performs the following to build a memory system:

- Maps each array access to a load-store unit (LSU) in the datapath that transacts with the kernel memory through its ports.
- Builds the kernel memory and LSUs and retains complete control over their structure.
- Automatically optimizes the kernel memory geometry to maximize the bandwidth available to loads and stores in the datapath.
- Attempts to guarantee that kernel memory accesses never stall.

These are discussed in detail in later sections of this guide.

Stallable and Stall-Free Memory Systems

Accesses to a memory (read or write) can be stall-free or stallable:

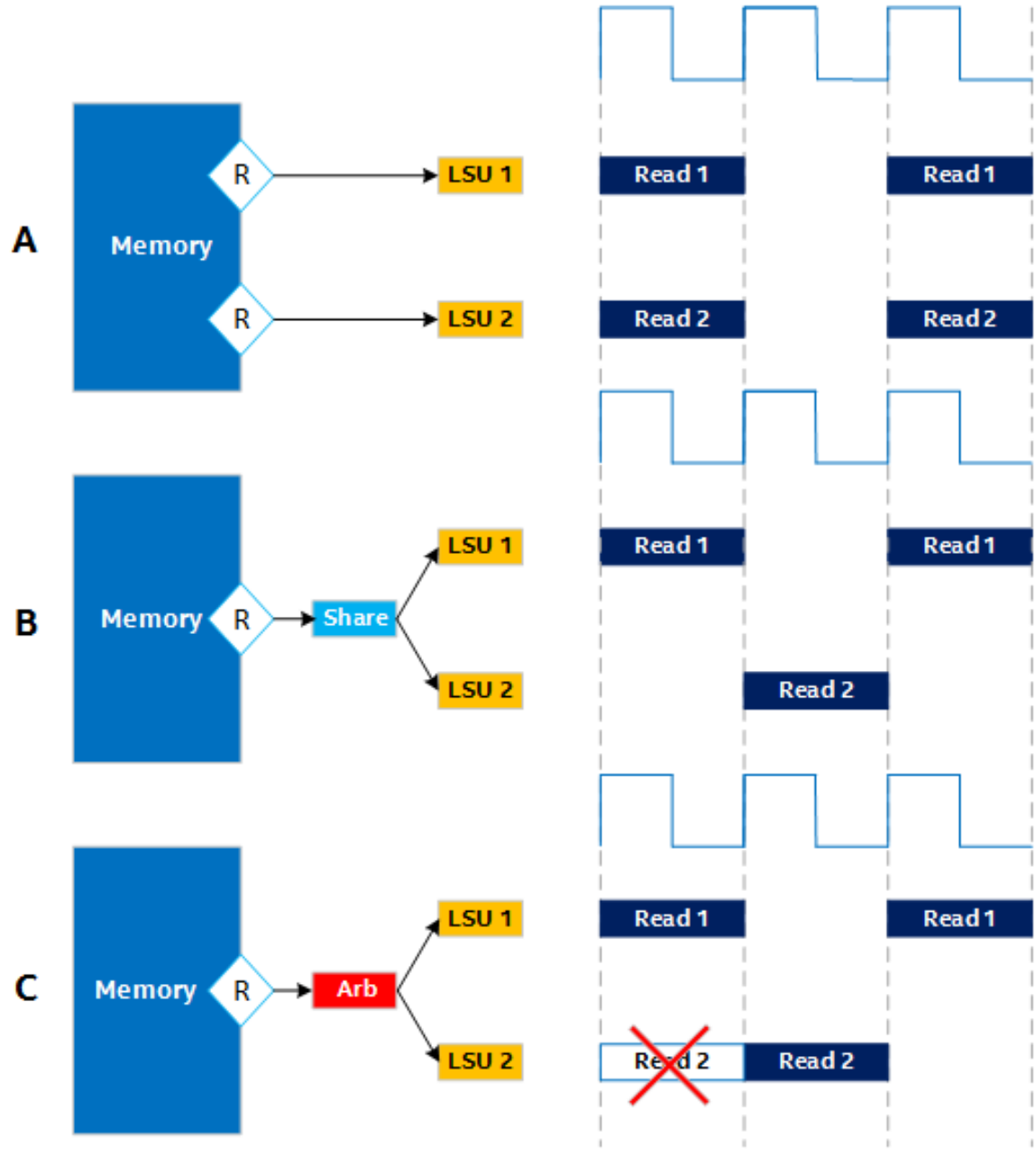
Memory Systems

Memory Access	Description
Stall-free	A memory access is stall-free if it has contention-free access to a memory port. This is illustrated in Figure 32: Examples of Stall-free and Stallable Memory Systems . A memory system is stall-free if each of its memory operations has contention-free access to a memory port.
Stallable	A memory access is stallable if it does not have contention-free access to a memory port. When two datapath LSUs attempt to transact with a memory port in the same clock cycle, one of those memory accesses is delayed (or stalled) until the memory port in contention becomes available.

As much as possible, the Intel® oneAPI DPC++/C++ Compiler attempts to create stall-free memory systems for your kernel.

A read or write is stall-free if it has contention-free access to a memory port, as shown in the following figure:

Examples of Stall-free and Stallable Memory Systems



The [Figure 32: Examples of Stall-free and Stallable Memory Systems](#) shows the following example memory systems:

- **A:** A stall-free memory system

This memory system is stall-free because, even though the reads are scheduled in the same cycle, they are mapped to different ports. There is no contention for accessing the memory systems.

- **B:** A stall-free memory system

This memory system is stall-free because the two reads are statically scheduled to occur in different clock cycles. The two reads can share a memory port without any contention for the read access.

- **C:** A stallable memory system

This memory system is stallable because two reads are mapped to the same port in the same cycle. The two reads happen at the same time. These reads require collision arbitration to manage their port access requests, and arbitration can affect throughput.

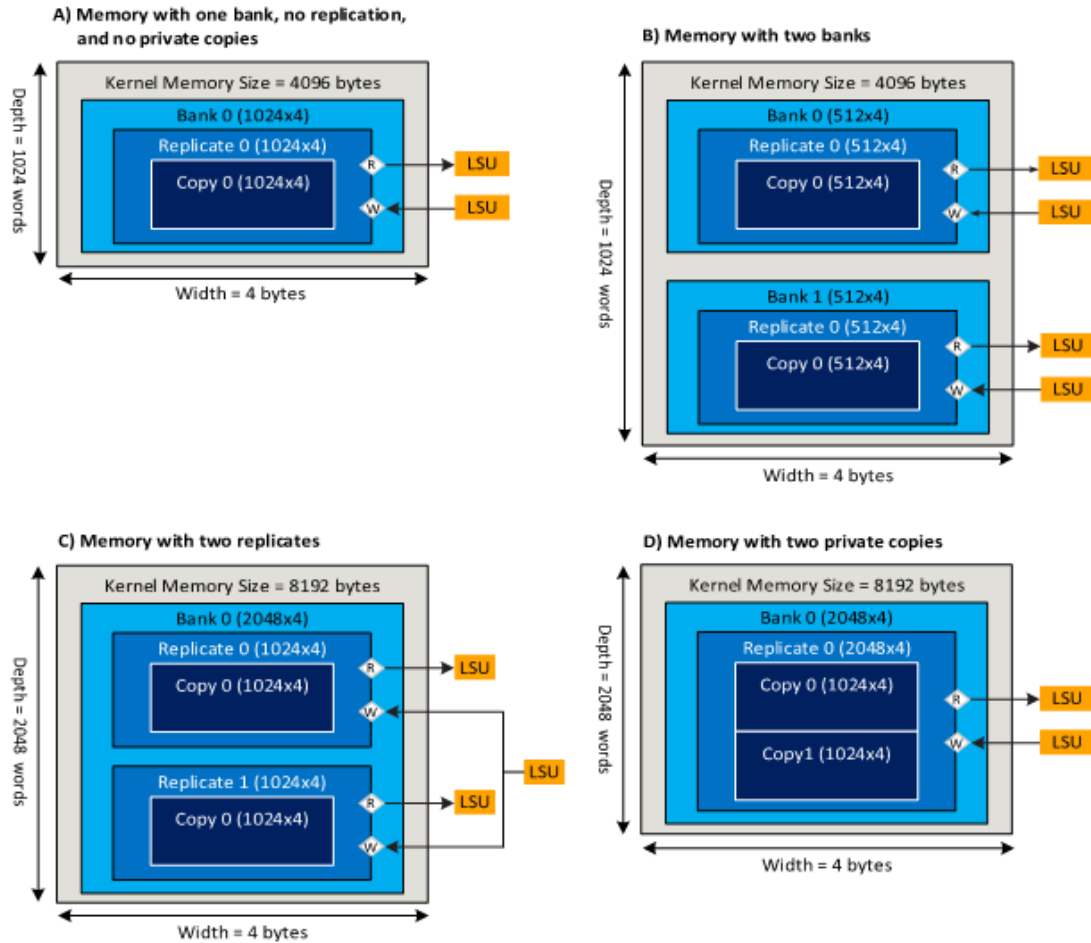
A kernel memory system consists of the following parts:

Parts of a Kernel Memory System

Part	Description
Port	A memory <i>port</i> is a physical access point into a memory. A port is connected to one or more load-store units (LSUs) in the datapath. An LSU can connect to one or more ports. A port can have one or more LSUs connected.
Bank	A memory <i>bank</i> is a division of the kernel memory system that contains a subset of the data stored. That is, all of the data stored for a kernel is split across banks, with each bank containing a unique piece of the stored data. A memory system always contains at least one bank.
Replicate	A memory bank <i>replicate</i> is a copy of the data that exists in a memory bank. All replicates in a bank contain the same data. Each replicate can be accessed independent of the others. A memory bank always contains at least one replicate.
Private Copy	A <i>private copy</i> is a copy of the data within a replicate that is created for nested loops to enable concurrent iterations of the outer loop. A replicate can contain multiple private copies, with each iteration of an outer loop having its own private copy. Because each outer loop iteration has its own private copy, private copies are not expected to contain the same data.

The following figure illustrates the relationship between banks, replicates, ports, and private copies:

Schematic Representation of Kernel Memories Showing the Relationship between Banks, Replicates, Ports, and Private Copies



Strategies That Enable Concurrent Stall-Free Memory Accesses

The compiler uses a variety of strategies to ensure that concurrent accesses are stall-free including:

- Adjusting the number of ports the memory system has. This can be done either by replicating the memory to enable more read ports or by using double pumping to enable four ports instead of two per replicate. All of the physical access ports for a replicate can be accessed concurrently.
- Partitioning memory content into one or more banks, such that each bank contains a subset of the data contained in the original memory (corresponds to the top-right box of [Schematic Representation of Local Memories Showing the Relationship between Banks, Replicates, Ports, and Private Copies](#)). The banks of a kernel memory can be accessed concurrently by the datapath.
- Replicating a bank to create multiple coherent replicates (corresponds to the bottom-left box of [Schematic Representation of Local Memories Showing the Relationship between Banks, Replicates, Ports, and Private Copies](#)). Each replicate in a bank contains identical data. The replicates are loaded concurrently.
- Creating private copies of an array declared inside a loop nest (corresponds to the bottom-right box of [Schematic Representation of Local Memories Showing the Relationship between Banks, Replicates, Ports, and Private Copies](#)). This enables loop pipelining, as each pipeline-parallel loop iteration accesses its own private copy of the array declared within the loop body. It is not expected for the private copies to contain the same data.

Despite the compiler's best efforts, the kernel memory system can still be stallable. This might happen due to [resource constraints](#) or [memory attributes](#) defined in your source code. In that case, the compiler tries to minimize the hardware resources consumed by the arbitrated memory system.

Global Memory

If the kernel code accesses a host-allocated buffer, the compiler creates a hardware interface through which the datapath accesses the buffer in global memory. A host-allocated buffer resides in device global memory off-chip. The code snippet in the [Kernel Memory](#) section shows a device global memory and its accesses within the kernel.

Unlike kernel memory, the compiler does not define the structure of a buffer in global memory. The compiler instantiates a specialized LSU for each access site based on the memory access pattern to maximize the efficiency of data accesses.

All accesses to global memory must go through the hardware interface. The compiler connects every LSU to an existing hardware interface through which it transacts with device global memory. Since the compiler cannot alter that interface or create more such interfaces, it must share the interface between multiple datapath reads or writes, which can limit the throughput of the design. The strategies used by the compiler to maximize efficient use of available memory interface bandwidth include (but are not limited to) the following:

- Eliminating unnecessary accesses.
- Statically coalescing contiguous accesses.
- Generating specialized LSUs that can perform the following:
 - Dynamically coalesced accesses that fall within the same memory word (as defined by the interface).
 - Prefetch and cache memory contents.

Intel® oneAPI FPGA Development

The Intel® oneAPI Base Toolkit can generate the following types of outputs for FPGA devices:

- Multiarchitecture binary (also known as a *fat binary*)

A multiarchitecture binary contains both host code and FPGA device code.

Multiarchitecture binaries require an FPGA acceleration board and some aspects of your FPGA code are constrained by the capabilities provided by the board support package (BSP). For more information about BSPs and boards, refer to [FPGA Boards and Board Support Packages \(BSPs\)](#).

The flow for creating a multiarchitecture binary is referred to as the *FPGA acceleration flow*. It might also occasionally be referred to as the *full-stack flow*.

- RTL IP core

An RTL IP core is ready to integrate into your overall FPGA application. Use Quartus® Prime Platform Designer to integrate the IP core into your design.

In an RTL IP core, the FPGA capabilities you can access are not constrained by a BSP. However, you are responsible for more parts of the IP design than when generating a multiarchitecture binary.

The flow for creating an RTL IP core is called the *SYCL high-level synthesis (HLS) flow* or just *HLS flow*. It might also occasionally be referred to as the *IP authoring flow*.

FPGA Flow Terminology

Field-programmable gate arrays (FPGAs) are configurable integrated circuits that you can program to implement arbitrary circuit topologies. Classified as spatial compute architectures, FPGAs differ significantly from fixed Instruction Set Architecture (ISA) devices such as CPUs and GPUs. FPGAs offer a different set of optimization trade-offs from these traditional accelerator devices.

While you can compile SYCL* code for CPU, GPU or FPGA, the compiling process for FPGA development is somewhat different than that for CPU or GPU development.

The following table summarizes terminologies used in describing the FPGA flow:

FPGA Flow-specific Terminology

Term	Definition
Device code	SYCL source code that executes on a SYCL device rather than the host. Device code is specified via lambda expression, functor, or kernel class.
Host code	SYCL source code that is compiled by the host compiler and executes on the host rather than the device. In the SYCL HLS flow, your host code becomes the testbench for your IP core.
FPGA emulator image	The device image resulting from compiling for the FPGA emulator. See FPGA Emulator .
FPGA Optimization Report	The device image resulting from the FPGA Optimization Report compilation stage. See FPGA Optimization Report .

Term	Definition
FPGA hardware image	Compiling for the FPGA Optimization Report also provides an RTL IP core version of your kernel (device code) that you can integrate into a larger FPGA hardware design. The device image resulting from the hardware image compilation stage. See FPGA Optimization Report and FPGA Hardware. Compiling for the FPGA hardware image also provides an RTL IP core version of your kernel (device code) that you can integrate into a larger FPGA hardware design.

Tip You can also learn about programming for FPGA devices in detail in the “Programming for FPGAs” chapter of *Data Parallel C++: Programming Accelerated Systems Using C++ and SYCL**, a free book available at https://link.springer.com/chapter/10.1007/978-1-4842-9691-2_17.

Intel® oneAPI FPGA Development Flow

The Intel® oneAPI DPC++/C++ Compiler uses your SYCL* code to generate multiarchitecture binaries or RTL IP cores, depending on the compilation target that you specify. For multiarchitecture binaries, set the compilation target to an FPGA acceleration board. For RTL IP cores, set the compilation target to a supported Intel® FPGA device family or part number instead of a specific acceleration platform.

The typical design flow when you author kernels for FPGA devices or boards consists of the following stages:

1. Creating your SYCL kernel and host code.

For information about writing SYCL code, refer to [Data Parallelism in C++ using SYCL*](#) in the *Intel® oneAPI Programming Guide*.

In the SYCL HLS flow, your kernel code becomes your RTL IP core and the host code serves as the testbench for the emulation and simulation flows.

In the FPGA acceleration flow, both kernel and host code are combined to form a multiarchitecture binary.

2. Verify the functionality of your kernel algorithm and testbench through emulation.

Verify the functionality of your kernel and refine the algorithms in your kernel by compiling your design to an x86-64 executable and running the executable. For details, see [Emulate and Debug Your Design](#).

3. Optimize and refine the FPGA performance of your kernel.

Use the FPGA Optimization Report to find areas where you can optimize your kernel to improve its performance and throughput. Compile your design with the `-xshardware` and `-fsycl-link=early` compiler command options to obtain the FPGA Optimization Report.

For more details about analyzing your application, refer to [Analyzing Your Design](#).

After completing some initial optimization based on the contents of the FPGA Optimization Report, you can see where to further refine your component by simulating it.

For details, see [Evaluate Your Kernel Through Simulation](#).

SYCL HLS Flow This step also generates an RTL IP core from your kernel.

4. (Optional) Obtain More Accurate Kernel Performance Estimates

When you are satisfied with the predicted performance of your kernel, use Quartus® Prime software to synthesize your kernel. Synthesis also generates accurate area and performance (f_{MAX}) estimates for your design. However, your design is not expected to cleanly close timing in the Quartus® Prime reports.

To synthesize your kernel, use the `-Xshardware` compiler command option. Compiling without the `-fsycl-link=early` compiler command option provides a more accurate estimate of your kernel's area and f_{MAX} but the compilation takes much longer (hours instead of minutes)

You can expect to see timing closure warnings in the Quartus® Prime logs because the generated project targets a clock speed of 1000 MHz to achieve the best possible placement for your design. The f_{MAX} value presented in the FPGA Optimization Report estimates the maximum clock rate your component can cleanly close timing for.

To synthesize your kernel and generate quality of results (QoR) data, instruct the compiler to run the Quartus® Prime compilation flow automatically after synthesizing the components. Include the `-Xshardware` option in your `icpx` command:

```
icpx -fintel-fpga -Xshardware -Xstarget="<FPGA device family or part number>"...
```

5. (FPGA Acceleration Flow required/SYCL HLS Flow optional) Compile your code to hardware

Use the `-Xshardware` and `-Xstarget=` compiler command options to compile your application for the target FPGA device or FPGA acceleration board.

A hardware compilation can take hours.

For details about the hardware compilation in the FPGA Acceleration flow, refer to [AOT Compilation Flow](#) in the *Intel® oneAPI Programming Guide*.

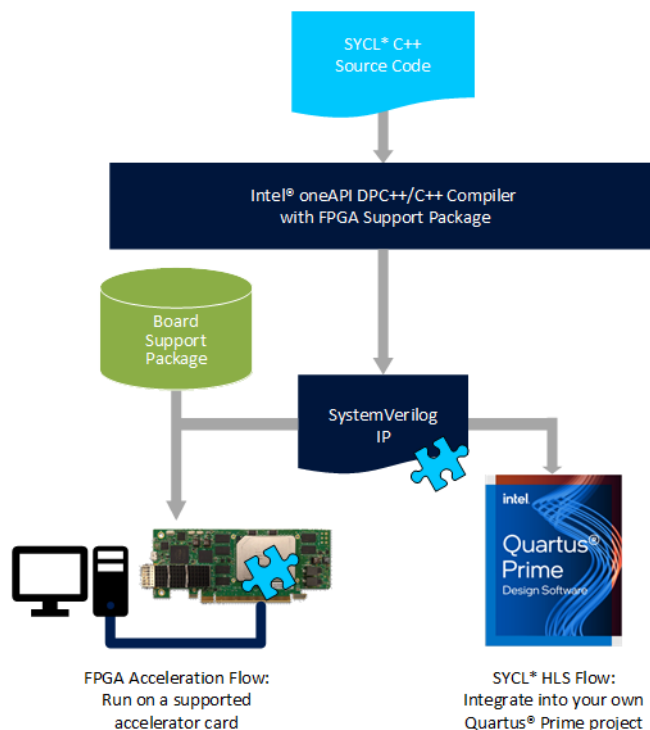
While this step also produces an RTL IP core, you can obtain the same RTL IP core faster without compiling to hardware.

6. (SYCL HLS Flow only) Integrate your IP into a system with Quartus® Prime or Platform Designer.

For details, refer to [Integrating Your RTL IP Core Into a System](#).

The following flowchart shows a coarse-grained progression through the stages of a typical Intel oneAPI FPGA development cycle.

Overview of Intel oneAPI FPGA Development Flow



Types of SYCL* FPGA Compilation

SYCL supports accelerators in general. The Intel® oneAPI DPC++/C++ Compiler implements additional FPGA-specific support to assist FPGA code development. This topic highlights different stages of FPGA compilation that the Intel oneAPI Base Toolkit supports.

For a hands-on lesson in the types of FPGA compilation, review the [FPGA Compile Sample on GitHub](#).

The following table summarizes the types of FPGA compilation:

Types of FPGA Compilation

Device Image Type	Time to Compile	Description
FPGA Emulator	Seconds	Compiles the FPGA device code to the CPU. Use the Intel® FPGA Emulation Platform for OpenCL™ software to verify your SYCL code's functional correctness.
FPGA Optimization Report	Minutes	Partially compiles the FPGA device code for hardware. The compiler generates an optimization report that describes the structures generated on the FPGA, identifies performance bottlenecks, and estimates resource utilization. SYCL HLS Flow When your compilation targets an FPGA device family or part number, this stage also gives you RTL files for the IP core in your code. You can then use Quartus® Prime software to integrate your IP cores into a larger design.
FPGA Simulator	Minutes	Compiles the FPGA device code to the CPU. Use the Questa*-Intel® FPGA Edition simulator to debug your code.
FPGA Hardware Image	Hours	Generates the final output for you design. The output depends on the target: • FPGA acceleration board target When your compilation targets an FPGA acceleration board, this stage generates the real FPGA bitstream to execute on the target FPGA platform. • FPGA device target When your compilation targets an FPGA device family or part number, this stage gives you RTL files for the IP core in your code. You can then use Quartus® Prime software to integrate your IP cores into a larger design.

A typical FPGA development workflow is to iterate in the emulation, optimization report, and simulation stages, refining your code using the feedback provided by each stage. Intel® recommends relying on emulation and the FPGA optimization report whenever possible.

Tip To compile for FPGA emulation, FPGA simulation, or generate the FPGA optimization report, you require only the Intel® oneAPI DPC++/C++ Compiler that is part of the Intel® oneAPI Base Toolkit. An FPGA hardware compile requires installing the Quartus® Prime software separately. Targeting a board also requires that you install the BSP for the board. For more information, refer to the [Intel® oneAPI Toolkits Installation Guide](#) and [Intel® FPGA development flow](#) web page.

Also, generating RTL code for an IP core requires only the Intel® oneAPI DPC++/C++ Compiler that is part of the Intel® oneAPI Base Toolkit. However, for simulating or integrating that IP core into your hardware design requires installing the Quartus® Prime Pro Edition software.

FPGA Emulator

The FPGA emulator (Intel® FPGA Emulation Platform for OpenCL™ software) is the fastest method to verify the correctness of your code. It executes the SYCL device code on the CPU. The emulator is similar to the SYCL host device, but unlike the host device, the FPGA emulator device supports FPGA extensions such as FPGA pipes and `fpga_reg`. For more information, refer to [Pipes Extension](#) and [Kernel Variables](#).

Important Remember the following caveats when you use the FPGA emulator:

- **Performance is not representative.**

Never draw inferences about FPGA performance from the FPGA emulator. The FPGA emulator's timing behavior is not correlated to that of the physical FPGA hardware. For example, an optimization that yields a 100x performance improvement on the FPGA may not impact the emulator performance. The emulator might show an unrelated increase or decrease.

- **Undefined behavior may differ.**

If your code produces different results when compiled for the FPGA emulator versus FPGA hardware (or simulation), your code most likely exercises undefined behavior. By definition, undefined behavior is not specified by the language specification and might manifest differently on different targets.

For detailed information about emulation, refer to [Emulate and Debug Your Design](#).

FPGA Optimization Report

The following compilation stages generate an FPGA Optimization Report:

FPGA Optimization Report

Stages	Optimization Report Information
FPGA Optimization Report (Compilation takes minutes to complete)	Contains important information about how the compiler has transformed your SYCL device code into an FPGA design. The report includes the following information: • Visualizations of structures generated on the FPGA. • Performance and expected performance bottleneck. • Estimated resource utilization. For information about the FPGA optimization report, refer to the Review the FPGA Optimization Report .
FPGA Simulator	After running the simulation binary, the optimization report summarizes the simulated performance results in the Simulation Statistics page.

Stages	Optimization Report Information
FPGA hardware image (Compilation takes hours to complete)	Contains precise information about resource utilization and f_{MAX} numbers. For detailed information about how to analyze reports, refer to Analyzing Your Design . For information about the FPGA hardware image, refer to the Review the FPGA Optimization Report .

When your compilation targets an FPGA device or part number, this stage gives you RTL files for the IP core in your code. You can then use Quartus® Prime software to integrate your IP cores into a larger design.

FPGA Simulator

The simulation flow allows you to use the Questa*-Intel® FPGA Edition simulator software to simulate the exact behavior of the synthesized kernel. FPGA simulation requires Quartus® Prime software (installed separately).

Like emulation, you can run simulation on a system that does not have a target FPGA board installed. The simulator models a kernel much more accurately than the emulator, but it is much slower than the emulator.

The simulation flow is cycle-accurate and bit-accurate. It exactly models the behavior of a kernel's datapath and the results of operations on floating-point data types. However, simulation cannot accurately model variable-latency memories or other external interfaces. Intel recommends that you simulate your design with a small input data set because simulation is much slower than running on FPGA hardware or emulator.

You can use the simulation flow in conjunction with profiling to collect additional information about your design. For more information about profiling, refer to [Intel® FPGA Dynamic Profiler for DPC++](#).

NOTE You cannot debug kernel code compiled for simulation using the GNU Project Debugger (GDB)*, Microsoft* Visual Studio*, or any standard software debugger.

For more information about the simulation flow, refer to [Evaluate Your Kernel Through Simulation](#).

FPGA Hardware

An FPGA hardware compile requires the [Quartus® Prime software](#) (installed separately). This is a full compilation stage through to the FPGA hardware image where you can target one of the following:

- Intel® FPGA device family
- Specific Intel® FPGA device part number
- Custom board with a supported BSP
- Intel® Programmable Acceleration Card (PAC) (deprecated)

For more information about the targets, refer to the [Intel® oneAPI DPC++/C++ Compiler System Requirements](#). For more information about using Intel® PAC or custom boards, refer to the [FPGA BSPs and Boards](#) section and the [Intel® oneAPI Toolkits Installation Guide for Linux* OS Installation Guide](#).

Separating Device and Host Code Compilation

The Intel® oneAPI DPC++/C++ Compiler supports only the ahead-of-time (AoT) compilation for FPGA hardware and simulation, which means that an FPGA device image is generated at compile time. The FPGA device image generation process can take hours to complete.

If you make a change exclusive to the host code, then recompile only your host code by reusing the existing FPGA device image and circumventing the time-consuming device compilation process.

The Intel® oneAPI DPC++/C++ Compiler provides the following mechanisms to separate device code and host code compilation:

- Passing the `-reuse-exe=<exe_name>` flag to instruct the compiler to attempt to reuse the existing FPGA device image.
- Separating the host and device code into separate files. When a code change applies only to host-only files, the FPGA device image is not regenerated.
- Separating the device code using the compiler option `-fsycl-device-code-split`.

Using the `-reuse-exe` Flag

If the device code and options affecting the device have not changed since the previous compilation, passing the `-reuse-exe=<exe_name>` flag instructs the compiler to extract the compiled FPGA hardware or simulation image from the existing executable and package it into the new executable, saving the device compilation time.

Sample use:

```
# Initial compilation
icpx -fintel-fpga -Xshardware <files.cpp> -o out.fpga
```

The initial compilation generates an FPGA device image, which takes several hours. Suppose you now make some changes to the host code.

```
# Subsequent recompilation
icpx -fintel-fpga -Xshardware <files.cpp> -o out.fpga -reuse-exe=out.fpga
```

One of the following actions are taken by the command:

- If the `out.fpga` file does not exist, the `-reuse-exe` flag is ignored, and the FPGA device image is regenerated. This is always the case the first time you compile a project.
- If the `out.fpga` file is found, the compiler verifies no change that affects the FPGA device code is made since the last compilation. If no change is detected in the device code, the compiler then reuses the existing FPGA device image and recompiles only the host code. The recompilation process takes a few minutes to complete.
- If the `out.fpga` file is found, but the compiler cannot prove that the FPGA device code will yield a result identical to the last compilation, a warning is printed, and the FPGA device code is fully recompiled. Since the compiler checks must be conservative, spurious recompilations can sometimes occur when using the `-reuse-exe` flag.

Using the Device Link Method

Suppose the program is separated into two files, `main.cpp` and `kernel.cpp`, where only the `kernel.cpp` file contains the device code.

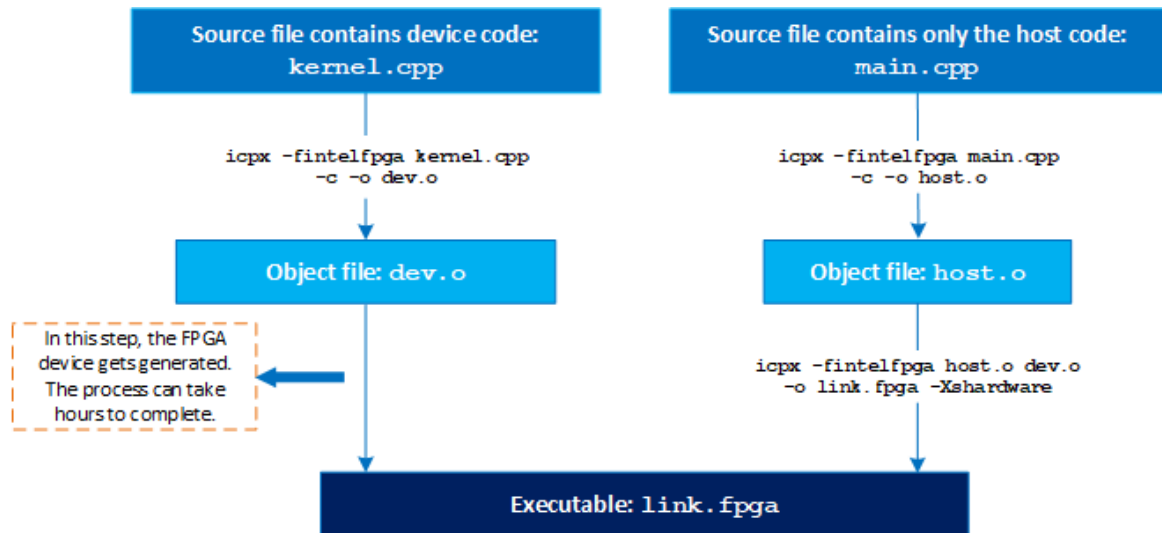
In the normal compilation process, FPGA device image generation happens at link time.

```
# normal compile command
icpx -fintel-fpga -Xshardware main.cpp kernel.cpp -o link.fpga
```

As a result, any change to either the `main.cpp` or `kernel.cpp` triggers the regeneration of an FPGA hardware image.

The following graph depicts this compilation process:

Compilation Process



If you want to iterate on the host code and avoid a long compile time for your FPGA device, consider using a device link to separate the device and host compilation:

```
# device link command
icpx -fintel FPGA -fsycl-link=image <input files> [options]
```

The compilation is a three-step process as listed in the following:

1. Compile the device code.

```
icpx -fintel FPGA -Xshardware -fsycl-link=image kernel.cpp -o dev_image.a
```

Input files must include all files that contain the device code. This step might take several hours to complete.

2. Compile the host code.

```
icpx -fintel FPGA main.cpp -c -o host.o
```

Input files should include all source files that contain only the host code. These files must not contain any source code that executes on the device but may contain setup and tear-down code, for example, parsing command-line options and reporting results. This step takes seconds to complete.

3. Create the device link.

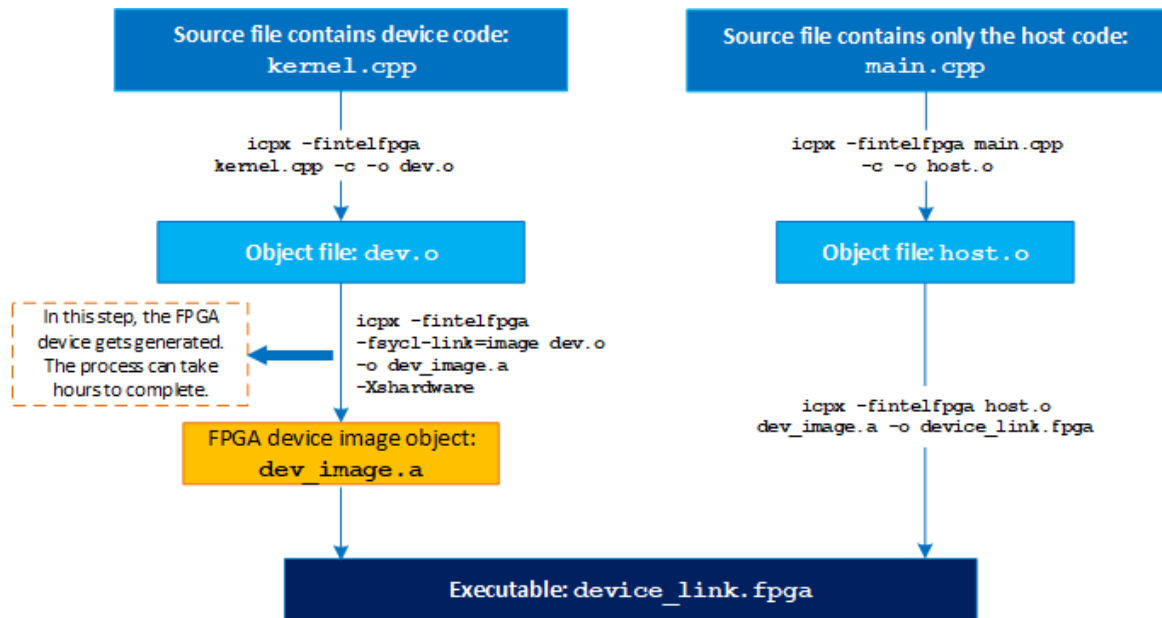
```
icpx -fintel FPGA host.o dev_image.a -o fast_recompile.fpga
```

This step takes seconds to complete. The input should include one or more host object files (`.o`) and exactly one device image file (`.a`). When linking a static library (`.a` file), always include the static library after its use. Otherwise, the library's functions are discarded. For additional information about static library linking, refer to [Library order in static linking](#).

NOTE You only need to perform steps 2 and 3 when modifying host-only files.

The following diagram illustrates the device link process:

FPGA Device Link Process



Refer to the [fast_recompile](#) tutorial on GitHub for an example using the device link method.

Using the `-fsycl-device-code-split[=value]` Option

The `-fsycl-device-code-split[=value]` option informs the compiler how to separate your design into device code modules.

For details about this option, refer to the `-fsycl-device-code-split` option description in the [Other Supported FPGA Flags](#) tables in "FPGA Compilation Flags".

Which Mechanism to Use?

Of the mechanisms described above, the `-reuse-exe` flag mechanism is easier to use than the device link mechanism. The flag also allows you to keep your host and device code as a single source, which is preferred for small programs. For larger and more complex projects, the device link method gives you more control over the compiler's behavior.

However, there are some drawbacks of the `-reuse-exe` flag when compared to compiling separate files. Consider the following when using the `-reuse-exe` flag:

- The compiler must spend time partially recompiling and then analyzing the device code to ensure that it is unchanged. This takes several minutes for larger designs. Compiling separate files does not incur this extra time.
- You might occasionally encounter a false positive where the compiler incorrectly believes it must recompile your device code. In a single source file, the device and host code are coupled, so certain changes to the host code can change the compiler's view of the device code. The compiler always behaves conservatively and triggers a full recompilation if it cannot prove that reusing the previous FPGA binary is safe. Compiling separate files eliminates this possibility.

FPGA Compilation Flags

FPGA compilation flags control the [FPGA image type](#) the Intel® oneAPI DPC++/C++ Compiler targets.

The following table explains the most commonly-used compiler flags:

FPGA Compilation Flags

Flag	Description
-fintel_fpga	Performs ahead-of-time (offline) compilation for FPGA.
-Xshardware	Instructs the compiler to target FPGA hardware. If you omit this flag, the compiler targets the FPGA emulator.
	NOTE Using the prefix <code>-Xs</code> causes an argument to be passed to the FPGA backend.
-Xsemulator	Generates an emulator device image. This is the default behavior.
-Xssimulation	Generates a simulator device image.
-fsycl-link=early	Instructs the compiler to stop after creating the FPGA Optimization Report and RTL IP core.
-Xstarget=<FPGA device family>	[Optional] Instructs the compiler to target an FPGA device family, an FPGA part number, or an FPGA board as follows:
-Xstarget=<FPGA part number>	• <code>-Xstarget=<FPGA device family></code> specifies the target FPGA device family. Use this target to facilitate initial development.
-Xstarget=<bsp>:<variant>	For more accurate reports, use the <code>-Xstarget=<FPGA part number></code> or <code>-Xstarget=<bsp>:<variant></code> to target specific FPGA devices or acceleration platforms.
-	Valid values are Agilex5, Agilex7, Arria10, CycloneV, Cyclone10GX, and Stratix10.
Xstarget=<path_to_asp>:<variant>	These values target the following FPGA device part numbers (OPNs):
	• Agilex5: A5EC065BB32AE4S • Agilex7: AGFB014R24A2E2V • Arria10: 10AX115U1F45I1SG • CycloneV: 5CGXFC7C7F23C8 • Cyclone10GX: 10CX220YF780I5G • Stratix10: 1SG280LU3F50I2VG
	• <code>-Xstarget=<FPGA part number></code> specifies the target FPGA part number (sometimes called an OPN). You can specify any valid Agilex™ 5, Agilex™ 7, Arria® 10, Cyclone® V, Cyclone® 10 GX, or Stratix® 10 part number. • <code>-Xstarget=<bsp>:<variant></code> specifies the FPGA board variant and BSP. Refer to the FPGA BSPs and Boards section for additional details. • <code>-Xstarget=<path_to_asp>:<variant></code> specifies the FPGA board Open FPGA Stack (OFS) acceleration support package (ASP) and its variant. For a list of ASPs and board variants supported, refer to oneapi-asp Hardware Implementation in the oneAPI Accelerator Support Package(ASP) Reference Manual.
	If you omit the <code>-Xstarget</code> flag, the compiler assumes an Agilex™ 7 device target (equivalent to specifying <code>-Xstarget=Agilex7</code>).

The following examples show Intel® oneAPI DPC++/C++ Compiler commands and common compilation flags used for various output targets:

FPGA Optimization Report

- **FPGA device family**

```
icpx -fintel_fpga fpga_compile.cpp -Xshardware -fsycl-link=early -Xstarget=Stratix10 -o
fpga_compile_report.a
```

- **FPGA part number**

```
icpx -fintel_fpga fpga_compile.cpp -Xshardware -fsycl-link=early -Xstarget=1SG280LU3FS0E3VG -o
fpga_compile_report.a
```

- **Explicit board**

```
icpx -fintel_fpga fpga_compile.cpp -Xshardware -fsycl-link=early
-Xstarget=intel_s10sx_pac:pac_s10 -o fpga_compile_report.a
```

FPGA Emulator Image

```
icpx -fintel_fpga fpga_compile.cpp -o fpga_compile.fpga_emu
```

FPGA Simulator Image

- **FPGA device family**

```
icpx -fintel_fpga fpga_compile.cpp -Xssimulation -Xstarget=Agilex7 -Xsghdl -o fpga_compile.fpga_sim
```

- **FPGA part number**

```
icpx -fintel_fpga fpga_compile.cpp -Xssimulation -Xstarget=AGFB014R24A3EV -Xsghdl -o
fpga_compile.fpga_sim
```

- **Explicit board**

```
icpx -fintel_fpga fpga_compile.cpp -Xssimulation -Xstarget=intel_s10sx_pac:pac_s10 -o
fpga_compile.fpga_sim
```

FPGA Hardware Image

- **FPGA device family**

```
icpx -fintel_fpga fpga_compile.cpp -Xshardware -Xstarget=Arria10 -o fpga_compile.fpga
```

- **FPGA part number**

```
icpx -fintel_fpga fpga_compile.cpp -Xshardware -Xstarget=10AX115S2F45I1SG -o fpga_compile.fpga
```

- **Explicit board**

```
icpx -fintel_fpga fpga_compile.cpp -Xshardware -Xstarget=intel_s10sx_pac:pac_s10 -o
fpga_compile.fpga
```

NOTE When you specify `-o` flag, the output directory is named based on what you specify with the `-o` flag. For example, if you specify `-o fpga_compile.fpga`, the output directory would be `fpga_compile.prj`. However, when you do not specify the `-o` flag, the output directory is always `a.prj`.

Warning The output of an `icpx` compile command overwrites the output of previous compiles that used the same output name. Therefore, Intel® recommends using unique output names (specified with `-o`). This is especially important for FPGA compilation since a lost hardware image may take hours to regenerate.

In addition to the compiler flags demonstrated by the example commands, there are flags to provide additional compilation controls. The following section briefly describes those flags.

Other SYCL* FPGA Flags Supported by the Compiler

The Intel® oneAPI DPC++/C++ Compiler offers several options that allow you to customize the kernel compilation process. The following table summarizes other options supported by the compiler:

Other Supported FPGA Flags

Flag	Description
<code>-fsycl-help=fpga</code>	Prints out FPGA-specific options for the <code>icpx</code> command.
<code>-fsycl-link=early</code> <code>-fsycl-link=image</code>	<code>-fsycl-link=early</code> is synonymous with <code>-fsycl-link</code>. Both instruct the compiler to stop after creating the FPGA Optimization Report and RTL IP core. <code>-fsycl-link=image</code> is used in the device link compilation flow to instruct the compiler to generate the FPGA hardware image. Refer to the Fast Recompile for FPGA section for additional information.
<code>-fsycl-device-code-split[=value]</code>	The <code>-fsycl-device-code-split[=value]</code> option informs the compiler how to separate your design into device code modules. This option supports the following modes: <code>auto</code>: This is the default mode and is the same as the <code>-fsycl-device-code-split</code> option without any value. The compiler uses a heuristic to select the best way of splitting device code. <code>off</code>: Creates a single module for all kernels. <code>per_kernel</code>: Creates a separate device code module for each kernel. Each device code module contains a kernel and dependencies, such as called functions and user variables. <code>per_source</code>: Creates a separate device code module for each source (translation unit). Each device code module contains a bunch of kernels grouped on a per-source basis and all their dependencies, such as all used variables and called functions, including the <code>SYCL_EXTERNAL</code> macro-marked functions from other translation units. Attention Each split must not share device resources, such as memory, across it. Furthermore, kernel pipes must have their source and sink within the same split. For additional information about this option, refer to the fsycl-device-code-split topic in <i>Intel® oneAPI DPC++/C++ Compiler Developer Guide and Reference</i>.
<code>-reuse-exe=<exe_file></code>	Instructs the compiler to extract the compiled FPGA hardware or simulation image from the existing executable if the kernel has not changed and package it into the new executable, saving the device or simulation compilation time. This option is not applicable when compiling for emulation. Refer to the Fast Recompile for FPGA section for additional information.
<code>-Xsv</code>	FPGA backend generates a verbose output describing the progress of the compilation.

Flag	Description
<code>-Xsghdl[=<depth>]</code>	Causes the simulation flow to log signals to Siemens EDA (formerly Mentor Graphics) Questa* waveform files. Use the optional <code><depth></code> attribute to specify how many levels of hierarchy are logged. If you do not specify a value for the <code><depth></code> attribute, a depth of 1 is used by default.
<code>-Xsparallel=<num_threads></code>	Sets the degree of parallelism used in the FPGA bitstream compilation. The <code><num_threads></code> value specifies the number of parallel threads you want to use. The maximum recommended value is the number of available cores. Setting this flag is optional. The default behavior is for the Quartus® Prime software to compile in parallel on all available cores.
<code>-Xsseed=<value></code>	Sets the seed used by Quartus® Prime software when generating the FPGA bitstream. The value must be an unsigned integer, and by default, the value is 1.
<code>-Xsfast-compile</code>	Runs FPGA bitstream compilation with reduced effort. This option allows faster compile time but at the cost of reduced performance of the compiled FPGA hardware image. Use this flag only for faster development time. It is not intended for production-quality results. The <code>-Xsfast-compile</code> flag sets the Quartus® Prime Pro Edition software into the compile mode that is dominated by the Fast Functional Test. Warning When compiling your SYCL kernel using the <code>-Xsfast-compile</code> flag, you might see functional failures due to timing violations in your design. In such cases, either avoid using the <code>-Xsfast-compile</code> flag or try compiling your kernel with different seeds.

For more information about FPGA optimization flags, refer to [Optimization Flags](#).

FPGA Workflows in IDEs

The oneAPI tools integrate with third-party integrated development environments (IDEs) on Linux (Eclipse*) and Windows (Visual Studio*) to provide a seamless GUI experience for software development. See [FPGA Workflows on Third-Party IDEs for Intel® oneAPI Toolkits](#) for more details.

For FPGA development with Visual Studio Code, refer to [FPGA Development for Intel oneAPI Toolkits with Visual Studio* Code](#).

Getting Started with the Intel® oneAPI DPC++/C++ Compiler for Intel® FPGA Development

4

Installing the Intel® oneAPI FPGA Development Environment

For instructions on installing your Intel® oneAPI FPGA Development Environment, refer to [Installing the Intel® oneAPI FPGA Development Environment](#) in the Altera documentation library.

After you install the development environment, initialize the environment before you start developing your SYCL* application.

Initializing the Intel® oneAPI FPGA Development Environment

Before you start developing your SYCL* application, initialize the development environment by setting the required environment variables required for compiling, emulating, and generating the FPGA optimization report with a setvars or oneapi-vars script.

For instructions on running these scripts, refer to [Use the setvars and oneapi-vars Scripts with Linux*](#) in the *Intel® oneAPI Programming Guide*.

Simulation Prerequisites

To use the FPGA simulation flow, you must download the following prerequisite software:

- **Quartus® Prime Pro Edition software:** Download this package from the [FPGA Software Download Center](#) download page.
- **Compatible simulation software (Questa*-Intel® FPGA Edition and Questa*-Intel® FPGA Starter Edition):** Obtain them from the [FPGA Software Download Center](#).

NOTE

- The Questa*-Intel® FPGA Edition requires a license. However, Questa*-Intel® FPGA Starter Edition is free but requires a zero-cost license. For additional details, refer to the Licensing chapter of the [Intel FPGA Software Installation and Licensing](#).
- You can also use your licensed version of Siemens* EDA ModelSim* SE or Siemens* EDA Questa Advanced Simulator software. For information about all ModelSim* and Questa* software versions that your Quartus® Prime Pro Edition software supports, refer to the [EDA Interface Information](#) section of the *Quartus® Prime Pro Edition: <version_number> Software and Device Support Release Notes*.
- On Linux systems, you must install [Red Hat* development tools](#) to work with Questa*-Intel® FPGA Edition and Questa*-Intel® FPGA Starter Edition software.

Installing the Questa*-Intel FPGA Edition Software

For installation instructions, refer to the [Questa*-Intel® FPGA Edition Quick-Start: Quartus® Prime Pro Edition guide](#).

NOTE From within the oneAPI environment, you can determine the Quartus® Prime software installation location by inspecting the `QUARTUS_ROOTDIR_OVERRIDE` environment variable.

Set Up the Simulation Environment

You must add directories containing the Quartus® Prime and Questa* simulation software binaries to your `PATH` environment variable.

NOTE Commands listed in this topic assume that you have installed the Questa* simulation software alongside the Quartus® Prime Pro Edition software, as mentioned in the [Simulation Prerequisites](#). If you installed the Questa* simulation software elsewhere, you must modify the `PATH` environment variable appropriately.

For Quartus® Prime Software (Simulation only)

For simulation, you must explicitly add the Quartus® Prime software binary directory to your `PATH` environment variable using the following command:

- **Linux**

```
$ export PATH=$PATH:<quartus_installdir>/quartus/bin
```

- **Windows**

```
set "PATH=%PATH%;<quartus_installdir>\quartus\bin64"
```

Additionally, you must also set the `OCL_ICD_FILENAMES` variable to specify the Installable Client Driver (ICD) to load.

```
set "OCL_ICD_FILENAMES=%OCL_ICD_FILENAMES%;alteraicl_icd.dll"
```

For Questa*-Intel® FPGA Starter Edition Software

For the free Questa*-Intel® FPGA Starter Edition software, run the following command:

- **Linux**

```
$ export PATH=$PATH:<quartus_installdir>/questa_fse/bin
```

- **Windows**

```
set "PATH=%PATH%;<quartus_installdir>\questa_fse\win64"
```

For Questa*-Intel® FPGA Edition Software

For the licensed Questa*-Intel® FPGA Edition software, run the following command:

- **Linux**

```
$ export PATH=$PATH:<quartus_installdir>/questa_fe/bin
```

- **Windows**

```
set "PATH=%PATH%;<quartus_installdir>\questa_fe\win64"
```

You should now be able to successfully compile for simulation.

FPGA Development for Intel® oneAPI Toolkits with Visual Studio* Code

You can integrate the Intel® oneAPI Base toolkit with Visual Studio* Code (VS Code*) to support a seamless software development environment.

For instructions, refer to [FPGA Development with Visual Studio* Code](#) in the Altera documentation library.

Defining a Kernel for FPGAs

In SYCL*, a *queue* connects a host program to a single device. Host programs submit tasks to a device via the queue and can monitor the queue for task completion.

Your kernel (that is, the code that you want to run on the FPGA device) is defined within the command that submits the task for processing. For single work-item kernels, the command is `queue-name.singletask`. The format changes slightly depending on the coding style you use:

- Lambda coding style:

```
queue-name.single_task<kernel_name>([=] () {
    // Kernel code goes here
})
```

- Functor coding style:

```
queue-name.single_task<kernel_name>(Kernel functor object goes here)
```

In the SYCL HLS flow, your kernel code is compiled to an RTL IP core and your host program is the testbench for the IP core.

For more information about SYCL queues and kernels and programming in DPC++, refer to the following publications:

- SYCL* 2020 Specification by the Khronos* Group. (<https://registry.khronos.org/SYCL/specs/sycl-2020/html/sycl-2020.html>)

The SYCL* specification provides detailed descriptions of SYCL concepts and application programming interfaces (APIs).

- *Data Parallel C++: Programming Accelerated Systems Using C++ and SYCL** (<https://link.springer.com/book/10.1007/978-1-4842-9691-2>)

This free book teaches you how to accelerate C++ programs using data parallel programming that targets various device types, including FPGA devices.

Suggested Kernel Coding Styles

For creating your kernel, use one of the following recommended general coding styles:

- **Functor Coding Style Example:** With the functor coding style, you can write your kernel code out-of-line from the host code. The style is typically preferred for the SYCL* HLS flow. In the SYCL* HLS flow, your host code becomes the testbench for your IP core.

The functor coding style clearly communicates what the inputs to your kernel are and you can easily annotate kernel arguments.

- **Lambda Coding Style Example:** The lambda coding style is typically used for kernels in the FPGA acceleration flow.

In the following examples, the kernel code and the code for its enqueueing is highlighted.

Functor Coding Style Example

```
#include <iostream>

// oneAPI headers
#include <sycl/ext/intel/fpga_extensions.hpp>
#include <sycl/sycl.hpp>

// Forward declare the kernel name in the global scope. This is an FPGA best
```

```
// practice that reduces name mangling in the optimization reports.
class VectorAddID;

struct VectorAdd {
    int *const vec_a_in;
    int *const vec_b_in;
    int *const vec_c_out;
    int len;

    void operator()() const {
        for (int idx = 0; idx < len; idx++) {
            int a_val = vec_a_in[idx];
            int b_val = vec_b_in[idx];
            int sum = a_val + b_val;
            vec_c_out[idx] = sum;
        }
    }
};

constexpr int kVectSize = 256;

int main() {
    bool passed = true;
    try {
        // Use compile-time macros to select either:
        // - the FPGA emulator device (CPU emulation of the FPGA)
        // - the FPGA device (a real FPGA)
        // - the simulator device
#ifdef FPGA_SIMULATOR
        auto selector = sycl::ext::intel::fpga_simulator_selector_v;
#elif FPGA_HARDWARE
        auto selector = sycl::ext::intel::fpga_selector_v;
#else // #if FPGA_EMULATOR
        auto selector = sycl::ext::intel::fpga_emulator_selector_v;
#endif

        // create the device queue
        sycl::queue q(selector);

        // make sure the device supports USM host allocations
        auto device = q.get_device();

        std::cout << "Running on device: "
                    << device.get_info<sycl::info::device::name>().c_str()
                    << std::endl;

        if (!device.has(sycl::aspect::usm_host_allocations)) {
            std::terminate();
        }

        // declare arrays and fill them
        // allocate in shared memory so the kernel can see them
        int *vec_a = sycl::malloc_shared<int>(kVectSize, q);
        int *vec_b = sycl::malloc_shared<int>(kVectSize, q);
        int *vec_c = sycl::malloc_shared<int>(kVectSize, q);
        assert(vec_a);
        assert(vec_b);
        assert(vec_c);
    }
}
```

```

for (int i = 0; i < kVectSize; i++) {
    vec_a[i] = i;
    vec_b[i] = (kVectSize - i);
}

std::cout << "add two vectors of size " << kVectSize << std::endl;

q.single_task<VectorAddID>(VectorAdd(vec_a, vec_b, vec_c, kVectSize))
    .wait();

// verify that vec_c is correct
for (int i = 0; i < kVectSize; i++) {
    int expected = vec_a[i] + vec_b[i];
    if (vec_c[i] != expected) {
        std::cout << "idx=" << i << ": result " << vec_c[i] << ", expected ("
            << expected << ") A=" << vec_a[i] << " + B=" << vec_b[i]
            << std::endl;
        passed = false;
    }
}

std::cout << (passed ? "PASSED" : "FAILED") << std::endl;

sycl::free(vec_a, q);
sycl::free(vec_b, q);
sycl::free(vec_c, q);
} catch (sycl::exception const &e) {
    // Catches exceptions in the host code.
    std::cerr << "Caught a SYCL host exception:\n" << e.what() << "\n";

    // Most likely the runtime couldn't find FPGA hardware!
    if (e.code().value() == CL_DEVICE_NOT_FOUND) {
        std::cerr << "If you are targeting an FPGA, please ensure that your "
            "system has a correctly configured FPGA board.\n";
        std::cerr << "Run sys_check in the oneAPI root directory to verify.\n";
        std::cerr << "If you are targeting the FPGA emulator, compile with "
            "-DFPGA_EMULATOR.\n";
    }
    std::terminate();
}
return passed ? EXIT_SUCCESS : EXIT_FAILURE;
}

```

Lambda Coding Style Example

```

#include <iostream>
#include <vector>

// oneAPI headers
#include <sycl/sycl.hpp>
#include <sycl/ext/intel/fpga_extensions.hpp>

using namespace sycl;

// Forward declare the kernel name in the global scope. This is an FPGA best
// practice that reduces name mangling in the optimization reports.
class VectorAddID;

```

```

void VectorAdd(const int *vec_a_in, const int *vec_b_in, int *vec_c_out,
               int len) {
    for (int idx = 0; idx < len; idx++) {
        int a_val = vec_a_in[idx];
        int b_val = vec_b_in[idx];
        int sum = a_val + b_val;
        vec_c_out[idx] = sum;
    }
}

constexpr int kVectSize = 256;

int main() {
    bool passed = true;
    try {
        // Use compile-time macros to select either:
        // - the FPGA emulator device (CPU emulation of the FPGA)
        // - the FPGA device (a real FPGA)
        // - the simulator device
#ifdef FPGA_SIMULATOR
        auto selector = sycl::ext::intel::fpga_simulator_selector_v;
#elif FPGA_HARDWARE
        auto selector = sycl::ext::intel::fpga_selector_v;
#else // #if FPGA_EMULATOR
        auto selector = sycl::ext::intel::fpga_emulator_selector_v;
#endif

        // create the device queue
        sycl::queue q(selector);

        // make sure the device supports USM host allocations
        auto device = q.get_device();

        std::cout << "Running on device: "
                   << device.get_info<sycl::info::device::name>().c_str()
                   << std::endl;

        if (!device.has(sycl::aspect::usm_host_allocations)) {
            std::terminate();
        }

        // declare arrays and fill them
        // allocate in shared memory so the kernel can see them
        int *vec_a = malloc_shared<int>(kVectSize, q);
        int *vec_b = malloc_shared<int>(kVectSize, q);
        int *vec_c = malloc_shared<int>(kVectSize, q);
        assert(vec_a);
        assert(vec_b);
        assert(vec_c);
        for (int i = 0; i < kVectSize; i++) {
            vec_a[i] = i;
            vec_b[i] = (kVectSize - i);
        }

        std::cout << "add two vectors of size " << kVectSize << std::endl;

        q.single_task<VectorAddID>([=]() {
            VectorAdd(vec_a, vec_b, vec_c, kVectSize);
        });
    }
}

```

```

    })
    .wait();

    // verify that vec_c is correct
    for (int i = 0; i < kVectSize; i++) {
        int expected = vec_a[i] + vec_b[i];
        if (vec_c[i] != expected) {
            std::cout << "idx=" << i << ": result " << vec_c[i] << ", expected ("
                << expected << ") A=" << vec_a[i] << " + B=" << vec_b[i]
                << std::endl;
            passed = false;
        }
    }

    std::cout << (passed ? "PASSED" : "FAILED") << std::endl;

    free(vec_a, q);
    free(vec_b, q);
    free(vec_c, q);
} catch (sycl::exception const &e) {
    // Catches exceptions in the host code.
    std::cerr << "Caught a SYCL host exception:\n" << e.what() << "\n";

    // Most likely the runtime couldn't find FPGA hardware!
    if (e.code().value() == CL_DEVICE_NOT_FOUND) {
        std::cerr << "If you are targeting an FPGA, please ensure that your "
            "system has a correctly configured FPGA board.\n";
        std::cerr << "Run sys_check in the oneAPI root directory to verify.\n";
        std::cerr << "If you are targeting the FPGA emulator, compile with "
            "-DFPGA_EMULATOR.\n";
    }
    std::terminate();
}
return passed ? EXIT_SUCCESS : EXIT_FAILURE;
}

```

Single Work-Item Kernels

When a kernel describes a single work item, the SYCL* host can execute the kernel as a single task. The SYCL* specification version 1.2.1 describes this mode of operation as task-parallel programming. A task is equivalent to a kernel executed with one workgroup that contains one work item. For more information, refer to [Concepts of Hardware Design](#) and [Mapping Parallelism Models to Hardware](#).

Single Work-item Kernel Design Guidelines

If your kernels contain loop structures, follow the Intel®-recommended guidelines to construct the kernels in a way that allows the Intel® oneAPI DPC++/C++ Compiler to analyze them effectively. Well-structured loops are particularly important to aid the compiler in generating a pipeline parallel datapath for loops.

Avoid Pointer Aliasing

If your kernels have pointer arguments, you can improve the throughput of the design if the Intel® oneAPI DPC++/C++ Compiler can prove those arguments never point to the same memory location. It is possible to provide the compiler with information about pointer arguments in kernels. For more information, refer to [Ignoring Dependencies Between Accessor Arguments](#).

Construct "Well-Formed" Loops

A *well-formed* loop has an exit condition that compares against an integer bound and has a simple induction increment. Including *well-formed* loops in your kernel improves performance because the Intel® oneAPI DPC++/C++ Compiler can analyze these loops efficiently.

The following example is a *well-formed* loop:

```
for (i = 0; i < N; i++) {
    //statements
}
```

NOTE

Well-formed nested loops also contribute to maximizing kernel performance.

The following example is a *well-formed* nested loop structure:

```
for (i = 0; i < N; i++) {
    //statements
    for(j = 0; j < M; j++) {
        //statements
    }
}
```

Minimize Loop-Carried Dependencies

The following loop structure creates a loop-carried dependence because each loop iteration reads data written by the previous iteration:

```
for (int i = 0; i < N; i++) {
    A[i] = A[i - 1] + i;
}
```

As a result, each read operation cannot proceed until the write operation from the previous iteration completes. The presence of loop-carried dependencies decreases the extent of pipeline parallelism that the Intel® oneAPI DPC++/C++ Compiler can achieve, which reduces kernel performance.

The Intel® oneAPI DPC++/C++ Compiler performs a static memory dependence analysis on loops to determine the extent of parallelism that it can achieve. In some cases, the Intel® oneAPI DPC++/C++ Compiler might assume loop-carried dependence:

- Between two array accesses and as a result, extract less pipeline parallelism.
- If it cannot resolve the dependencies at compilation time because of unknown variables or complex indexing expressions.

To minimize loop-carried dependencies, follow these guidelines whenever possible:

- **Avoid pointer arithmetic.** Compiler output is suboptimal when the kernel accesses arrays by dereferencing pointer values derived from arithmetic operations. For example, avoid accessing an array in the following manner:

```
for (int i = 0; i < N; i++) {
    int t = *(A++);
    *A = t;
}
```

- **Introduce simple, affine array indexes.** Avoid the following types of complex array indexes because the Intel® oneAPI DPC++/C++ Compiler cannot analyze them effectively, which might lead to suboptimal compiler output:
 - Non-constants in array indexes. For example, `A[K + i]`, where `i` is the loop index variable and `K` is an unknown variable.

- Multiple index variables in the same subscript location. For example, $A[i + 2 \times j]$, where i and j are loop index variables for a double nested loop.

NOTE

The Intel® oneAPI DPC++/C++ Compiler can analyze the array index $A[i][j]$ effectively because the index variables are in different subscripts.

Avoid Complex Loop Exit Conditions

The Intel® oneAPI DPC++/C++ Compiler evaluates exit conditions to determine if subsequent loop iterations can enter the loop pipeline. Occasionally, the Intel® oneAPI DPC++/C++ Compiler requires memory accesses or complex operations to evaluate the exit condition. In these cases, subsequent iterations cannot launch until the evaluation completes, decreasing the overall loop performance.

Convert Nested Loops into a Single Loop

To maximize performance, combine nested loops into a single form whenever possible. Restructuring nested loops into a single loop reduces hardware footprint and computational overhead between loop iterations.

The following code examples illustrate the conversion of a nested loop into a single loop:

Conversion of a Nested Loop into a Single Loop

Nested Loop	Converted Single Loop
<pre>for (i = 0; i < N; i++) { //statements for (j = 0; j < M; j++) { //statements } //statements }</pre>	<pre>for (i = 0; i < N*M; i++) { //statements }</pre>

Avoid Conditional Loops

To maximize performance, avoid declaring conditional loops. Conditional loops are tuples of loops that are declared within conditional statements such that one and only one of the loops is expected to be reached. These loops cannot be efficiently parallelized and result in a serialized implementation.

The following code examples illustrate the conversion of conditional loops to a more optimal implementation:

Conversion of a Conditional Loop to an Optimized Loop

Conditional Loops	Converted Loop
<pre>if (condition) { for (int i = 0; i < m; i++) { // statements } } else { for (int i = 0; i < m; i++) { // statements } }</pre>	<pre>for (int i = 0; i < m; i++) { if (condition) { // statements } else { // statements } }</pre>

Declare Variables in the Deepest Scope Possible

To reduce hardware resources necessary for implementing a variable, declare the variable prior to its use in a loop. Declaring variables in the deepest scope possible minimizes data dependencies and hardware use because the Intel® oneAPI DPC++/C++ Compiler does not need to preserve the variable data across loops that do not use variables.

Consider the following example:

```
int a[N];
for (int i = 0; i < m; ++i) {
    int b[N];
    for (int j = 0; j < n; ++j) {
        // statements
    }
}
```

The array `a` requires more resources to implement than the array `b` because array `a` is declared at a broader scope. To reduce hardware use, declare array `a` outside the inner loop unless it is necessary to maintain the data through iterations of the outer loop, as shown in the following:

```
for (int i = 0; i < m; ++i) {
    int a[N];
    int b[N];
    for (int j = 0; j < n; ++j) {
        // statements
    }
}
```

Tip

Overwriting all values of a variable in the deepest scope possible also reduces resources necessary to present the variable.

Single-Cycle Floating-Point Accumulator for Single Work-Item Kernels

Single work-item kernels that perform accumulation in a loop can leverage the single-cycle floating-point accumulator feature of the Intel® oneAPI DPC++/C++ Compiler. The compiler searches for these kernel instances and attempts to map an accumulation that executes in a loop into the accumulator structure.

The compiler supports an accumulator that adds or subtracts a value. To leverage this feature, describe the accumulation in a way that allows the compiler to infer the accumulator, which must be part of a loop, must have an initial value of 0, and cannot be conditional.

NOTE

The dedicated floating-point accumulator hardware is available only on Arria® 10 devices.

Strategies for Inferring the Accumulator

To leverage the single cycle floating-point accumulator feature, you can modify the accumulator description in your kernel code to improve efficiency or work around programming restrictions.

Describe an Accumulator Using Multiple Loops

Consider a case where you want to describe an accumulator using multiple loops, with some of the loops being unrolled:

```
float acc = 0.0f;
for (i = 0; i < k; i++) {
    #pragma unroll
    for (j = 0; j < 16; j++)
        acc += (x[i+j]*y[i+j]);
}
```

With fast math enabled by default, the Intel® oneAPI DPC++/C++ Compiler automatically rearranges operations in a way that exposes the accumulation.

Modify a Multi-Loop Accumulator Description

If you want an accumulator to be inferred even when using `-fp-model=precise`, rewrite your code to expose the accumulation.

For the code example above, rewrite it in the following manner:

```
float acc = 0.0f;
for (i = 0; i < k; i++) {
    float my_dot = 0.0f;
    #pragma unroll
    for (j = 0; j < 16; j++)
        my_dot += (x[i+j]*y[i+j]);
    acc += my_dot;
}
```

Modify an Accumulator Description Containing a Variable or Non-Zero Initial Value

Consider a situation where you might want to apply an offset to a description of an accumulator that begins with a non-zero value:

```
float acc = array[0];
for (i = 0; i < k; i++) {
    acc += x[i];
}
```

Because the accumulator hardware does not support variable or non-zero initial values in a description, you must rewrite the description.

```
float acc = 0.0f;
for (i = 0; i < k; i++) {
    acc += x[i];
}
acc += array[0];
```

Rewriting the description in the above manner enables the kernel to use an accumulator in a loop. The loop structure is then followed by an increment of `array[0]`.

Debugging and Verifying Your Design

6

To debug and verify the functional correctness of your design, you have the following options:

- Emulate your design through the Intel® FPGA Emulation Platform for OpenCL™ software and use a debugger like GDB or the Windows* Visual C++ debugger.
- Simulate your kernel in simulation software like Questa*-Intel® FPGA Edition. Simulation lets you evaluate how your kernel performs on the FPGA device without having to compile your design to hardware.

For details, refer to [Emulate and Debug Your Design](#).

During simulation, you can also capture signal waveforms to further debug your kernel.

For details, refer to [Evaluate Your Kernel Through Simulation](#).

Depending on whether you are targeting the FPGA emulator, simulator, or hardware, you must use the correct SYCL* device selector in the host code. You can also use the FPGA hardware device selector for simulation. To see how you can use a selector to specify the target device at compile time, refer to [Device Selectors for FPGA](#).

Device Selectors for FPGA

Depending on whether you are targeting the FPGA emulator, simulator, or hardware, you must use the correct SYCL* device selector in the host code. You can use the FPGA hardware device selector for simulation also. The following host code snippet demonstrates how you can use a selector to specify the target device at compile time:

```
// FPGA device selectors are defined in this utility header, along with
// all FPGA extensions such as pipes and fpga_reg
#include <sycl/ext/intel/fpga_extensions.hpp>

int main() {
    // Select either:
    // - the FPGA simulator
    // - the FPGA device (a real FPGA)
    // - the FPGA emulator device (CPU emulation of the FPGA)
    #if FPGA_SIMULATOR
        auto selector = sycl::ext::intel::fpga_simulator_selector_v;
    #elif FPGA_HARDWARE
        auto selector = sycl::ext::intel::fpga_selector_v;
    #else // #if FPGA_EMULATOR
        auto selector = sycl::ext::intel::fpga_emulator_selector_v;
    #endif
    queue q(selector);
    ...
}
```

NOTE

- The FPGA emulator and the FPGA are different target devices. Intel® recommends using a preprocessor `define` to choose between the emulator and FPGA selectors. This makes it easy to switch between targets using only command-line flags. For example, you can compile the above code snippet for the FPGA emulator by passing the flag `-DFPGA_EMULATOR` to the `icpx` command.
- Since FPGAs support only the [ahead-of-time compilation method](#), dynamic selectors (such as the `default_selector`) are less useful than explicit selectors when targeting FPGAs.

Caution When targeting the FPGA emulator or FPGA hardware, you must pass correct compiler flags and use the correct device selector in the host code. Otherwise, you might experience run time failures. To get started with compiling SYCL* code for FPGA, refer to the FPGA Compile Sample on [GitHub](#).

`printf` Command Restrictions

In the SYCL* HLS flow, the `sycl::oneapi::experimental::printf()` function is currently supported only for emulation.

To ensure that `printf` command is used appropriately, wrap the command in a precompiler guard using `#if FPGA_EMULATOR/#endif` directives. For example:

```
#if FPGA_EMULATOR
    printf("Debug message\n");
#endif
```

Emulate and Debug Your Design

Verify the functionality of your design by compiling your kernel and host code to an x86-64 FPGA emulation executable to run in the Intel® FPGA Emulation Platform for OpenCL™ software. This process is sometimes referred to as debugging through emulation.

Compiling your design to an x86-64 executable is faster than generating and simulating your design in simulation software like Questa*-Intel® FPGA Edition. Shorter compilation time allows you to debug and refine your kernel quickly.

No additional software is required to emulate your kernel, and no modifications to your host code are required.

The Intel FPGA Emulation Platform for OpenCL software (also referred to as the emulator or the FPGA emulator) is installed as part of the [Intel® oneAPI Base Toolkit](#). It assesses the functionality of your kernel. The emulator supports 64-bit Windows and Linux operating systems. On Linux systems, the GNU C Library (glibc) version 2.15 or later is required.

NOTE

- You cannot use the execution time of an emulated design to estimate its execution time on an FPGA. Furthermore, running an emulated design is not a substitute for natively running a functionally equivalent C/C++ implementation on an x86-64 host.
- Emulation does not support cross-compilation to ARM® processors. To run emulation on a design that targets an ARM SoC device, emulate on a non-SoC board (for example, `intel_a10gx_pac` or `intel_s10sx_pac`). When satisfied with the emulation results, you can target your design on an SoC board for subsequent optimization steps.
- To enable debugging of kernel code, optimizations are disabled by default for the FPGA emulator. This can lead to sub-optimal execution speed when emulating kernel code. You can pass the `-g0` flag to the `icpx` compile command to disable debugging and enable optimizations. This enables faster emulator execution.
- When targeting the FPGA emulator device, use the `-O2` compiler flag to turn on optimizations and speed up the emulation. To turn off optimizations (for example, to facilitate debugging), pass `-O0`. With Windows Visual C++ debugger, specify `/Od`.
- To enable native debuggers to debug your device code, use the following `icpx` command flags:
 - Windows Visual C++ debugger: `/DEBUG /Od`
 - GDB: `-g -O0`
- For information about debugging with Intel® Distribution for GDB*, refer to the following:
 - [Debugging with Intel® Distribution for GDB* on Linux* OS Host](#)
 - [Get Started with Intel® Distribution for GDB* on Linux* OS Host](#)
 - [Get Started with Intel® Distribution for GDB* on Windows* OS Host](#)

Refer to the following topics for additional information:

- [Emulator Environment Variables](#)
- [Emulate Applications with a Pipe That Reads or Writes to an I/O Pipe](#)
- [Compile and Emulate Your Design](#)
- [Limitations of the Emulator](#)
- [Discrepancies in Hardware and Emulator Results](#)
- [Emulator Known Issues](#)

Emulator Environment Variables

The following table lists environment variables that you can use to modify the behavior of the emulator:

Emulator Environment Variables

Environment Variable	Description
<code>CL_CONFIG_CPU_EMULATE_DEVICES</code>	Controls the number of identical emulator devices provided by the emulator platform. If not set, a single emulator device is available. Therefore, set this variable only if you want to emulate multiple devices.
<code>DPCPP_CPU_NUM_CUS</code>	Indicates a maximum number of threads that the emulator can use. The default value is 32, and the maximum value is 255. Each thread can run a single kernel. If the application requires several kernels to be executing simultaneously, you must set the <code>DPCPP_CPU_NUM_CUS</code> environment variable appropriately to the number of kernels used or a higher value.
<code>CL_CONFIG_CPU_FORCE_LOCAL_MEM_SIZE</code>	Set the amount of available local memory with units. For example: 8MB, 256KB, or 1024B.
<code>CL_CONFIG_CPU_FORCE_PRIVATE_MEM_SIZE</code>	Set the amount of available private memory with units. For example: 8MB, 256KB, or 1024B.

Environment Variable	Description
	NOTE On Windows, the FPGA emulator can fail silently by running out of memory. Set the value of <code>CL_CONFIG_CPU_FORCE_PRIVATE_MEM_SIZE</code> to ensure that the kernel has sufficient private memory. To detect a lack of memory at run time, surround the kernel invocation in a try-catch block. For example: <pre>try { q.single_task<>(); } catch (sycl::exception& e) { std::cerr << e << std::endl; }</pre>
<code>CL_CONFIG_CHANNEL_DEPTH_EMULATION_MODE</code>	When you compile your kernel for emulation, the pipe depth is different from the pipe depth generated when your kernel is compiled for hardware. You can change this behavior with the <code>CL_CONFIG_CHANNEL_DEPTH_EMULATION_MODE</code> environment variable. For details, see Emulate Pipe Depth.

Emulate Pipe Depth

When you compile your kernel for emulation, the default pipe depth is different from the default pipe depth generated when your kernel is compiled for hardware. You can change this behavior when you compile your kernel for emulation with the `CL_CONFIG_CHANNEL_DEPTH_EMULATION_MODE` environment variable.

Important For kernels that include pipes, you must set the `CL_CONFIG_CHANNEL_DEPTH_EMULATION_MODE` environment variable before running the host program.

The `CL_CONFIG_CHANNEL_DEPTH_EMULATION_MODE` environment variable accepts the following values:

CL_CONFIG_CHANNEL_DEPTH_EMULATION_MODE values

Environment Variable	Description
<code>ignoreddepth</code>	All pipes are given a pipe depth chosen to provide the fastest execution time for your kernel emulation. Any explicitly set pipe depth attribute is ignored.
<code>default</code>	Pipes with an explicit depth attribute have their specified depth. Pipes without a specified depth are given a default pipe depth that is chosen to provide the fastest execution time for your kernel emulation.
<code>strict</code>	All pipe depths in the emulation are given a depth that matches the depth given for the FPGA compilation. If the specified depth is not given, the depth will be 1. This value is used by default if the <code>CL_CONFIG_CHANNEL_DEPTH_EMULATION_MODE</code> environment variable is not set.

Compile and Emulate Your Design

To compile and emulate your FPGA kernel design, perform the following steps:

1. Modify the host part of your program to declare the `sycl::ext::intel::fpga_emulator_selector_v` device selector. Use this selector when instantiating a device queue for enqueueing your FPGA device kernel. For more information, refer to [Device Selectors for FPGA](#).
2. Compile your design by including the `-fintel_fpga` option in your `icpx` command to generate an executable.
3. Run the resulting executable:

- For Windows:

1. Define the number of emulated devices by invoking the following command:

```
set CL_CONFIG_CPU_EMULATE_DEVICES=<number_of_devices>
```

2. Run the executable.

3. Invoke the following command to unset the variable:

```
set CL_CONFIG_CPU_EMULATE_DEVICES=
```

- For Linux, invoke the following command:

```
env CL_CONFIG_CPU_EMULATE_DEVICES=<number_of_devices> <executable_filename>
```

This command specifies the number of identical emulation devices that the emulator must provide.

Tip If you want to use only one emulator device, you need not set the `CL_CONFIG_CPU_EMULATE_DEVICES` environment variable.

NOTE

- The emulator is built with GCC 7.4.0 as part of the Intel® oneAPI DPC++/C++ Compiler. When running the executable for an emulated FPGA device, the version of `libstdc++.so` must be at least that of GCC 7.4.0. In other words, the `LD_LIBRARY_PATH` environment variable must ensure that the correct version of `libstdc++.so` is found.

If the correct version of `libstdc++.so` is not found, the call to `clGetPlatformIDs` function fails to load the FPGA emulator platform and returns `CL_PLATFORM_NOT_FOUND_KHR` (error code -1001). Depending on which version of `libstdc++.so` is found, the call to `clGetPlatformIDs` may succeed, but a later call to the `clCreateContext` function may fail with `CL_DEVICE_NOT_AVAILABLE` (error code -2).

If the `LD_LIBRARY_PATH` does not point to a compatible `libstdc++.so`, use the following syntax to invoke the host program:

```
env LD_LIBRARY_PATH=<path to compatible libstdc++.so>:$LD_LIBRARY_PATH <executable>
[executable arguments]
```

Limitations of the Emulator

The Intel® FPGA Emulation Platform for OpenCL™ software has the following limitations:

- **Concurrent execution**

Modeling of concurrent kernel executions has limitations. During execution, the emulator is not guaranteed to run interacting work items in parallel. Therefore, some concurrent execution behaviors, such as different kernels accessing global memory without a barrier for synchronization, might generate inconsistent emulation results between executions.

- **Same address space execution**

The emulator executes the host runtime and kernels in the same address space. Certain pointer or array use in your host application might cause the kernel program to fail and vice versa. Example uses include indexing externally allocated memory and writing to random pointers.

To analyze your program, you may use memory leak detection tools, such as Valgrind. However, the host might encounter a fatal error caused by out-of-bounds write operations in your kernel and vice versa.

- **Conditional pipe operations**

Emulation of pipe behavior has limitations, especially for conditional pipe operations where the kernel does not call the pipe operation in every loop iteration. In these cases, the emulator might execute pipe operations in a different order than on the hardware.

- **GCC version**

You must run the emulator host programs on Linux with a version of `libstdc++.so` from GCC 7.4.0 or later. You can achieve this either by installing GCC 7.4.0 or later on your system or setting the `LD_LIBRARY_PATH` environment variable such that a compatible `libstdc++.so` is identified.

Troubleshooting Discrepancies in Hardware and Emulator Results

When you emulate a kernel, your kernel might produce results different from the kernel compiled for hardware. You can further debug your kernel before you compile for hardware by running your kernel through [simulation](#).

Warning These discrepancies usually occur when the Intel® FPGA Emulation Platform for OpenCL™ is unable to model some aspects of the hardware computation accurately or when your program relies on undefined behavior.

The most common reasons for differences in emulator and hardware results are as follows:

- Your kernel code is using the [ivdep Attribute](#). The emulator does not model your kernel when the `ivdep` attribute breaks a true dependence. During a full hardware or simulator compilation, you observe this as an incorrect result.
- Your kernel code relies on uninitialized data. Examples of uninitialized data include uninitialized variables and uninitialized or partially initialized global buffers, and arrays.
- Your kernel code behavior depends on the precise results of floating-point operations. The emulator uses floating-point computation hardware of the CPU, whereas the hardware run uses floating-point cores implemented as FPGA cores.

NOTE The SYCL* standard allows one or more least significant bits of floating-point computations to differ between platforms while still being considered correct on both such platforms.

- Your kernel code behavior depends on the order of pipe accesses in different kernels. The emulation of pipe behavior has limitations, especially for conditional pipe operations where the kernel does not call the pipe operation in every loop iteration. In such cases, the emulator might execute pipe operations in an order different from that of the hardware. For instance, if you have two kernels connected by pipes, and each kernel contains a loop containing a `read()` or `write()` function that does not happen every loop iteration (for example, if it is gated by an if-statement), the emulator might interleave the `read()` or `write()` calls differently than the hardware.
- Your kernel or host code is accessing global memory buffers out-of-bounds.

NOTE

- Uninitialized memory read and write behaviors are platform-dependent. Verify the sizes of your global memory buffers when using all addresses within kernels.
 - You can use software memory leak detection tools, such as Valgrind, on the emulated version of your kernel to analyze memory-related problems. The absence of warnings from such tools does not mean the absence of issues. It only means that the tool could not detect any problem. In such a scenario, Intel recommends manual verification of your kernel or host code.
-

- Your kernel code is accessing local variables out-of-bounds. For example, accessing a local array out-of-bounds or accessing a variable after it has gone out of scope.

NOTE In software terms, these issues are stack corruption issues because accessing variables out of bounds usually affects unrelated variables located close to the variable being accessed on a stack. Emulated kernels are implemented as regular CPU functions and have an actual stack that can be corrupted. When targeting hardware, no stack exists. Hence, the stack corruption issues are guaranteed to manifest differently. When you suspect a stack corruption, use memory leak analyzer tools, such as Valgrind. However, stack-related issues are usually difficult to identify. Intel recommends manual verification of your kernel code to debug a stack-related issue.

- Your kernel code uses shifts that are larger than the type being shifted. For example, shifting a 64-bit integer by 65 bits. According to the SYCL specification version 1.0, the behavior of such shifts is undefined.
- When you compile your kernel for emulation, the default pipe depth is different from the default pipe depth generated when your kernel is compiled for hardware. This difference in pipe depths might lead to scenarios where execution on the hardware hangs while kernel emulation works without any issue. Refer to [Emulate Pipe Depth](#) for information about fixing the pipe depth difference.
- In terms of ordering the printed lines, the output of the `cout stream` function might be ordered differently on the emulator and hardware. This is because, in the hardware, `cout stream` data is stored in a global memory buffer and flushed from the buffer only when the kernel execution is complete or when the buffer is full. In the emulator, the `cout stream` function uses the `x86 stdout`.
- The hardware and emulator might produce different results if you perform an unaligned load/store through upcasting of types. A load/store of this type is undefined in the C99 specification.
- When debugging unknown behaviors that differ between emulation and simulation/hardware, Intel recommends using the `-Weverything` diagnostic command option for emulation. The `-Weverything` option turns on all warnings allowing you to utilize available diagnostics and expose risky coding patterns, which you might be inadvertently using in your design.

Emulator Known Issues

A few known issues might affect your use of the emulator. Review these issues to avoid possible problems when using the emulator.

Discrepancy in the Results for Task Sequence Functions

For task sequence functions that have multiple `async()` calls before the first `get()` call, the order the `get()` calls return results may not be the order in which the `async()` calls were called. To overcome this issue, use the simulator to verify that correct results are being returned in this scenario.

For more information about task sequence functions, refer to [Asynchronous Parallelism Within Kernels \(task_sequence\)](#).

Compiler Diagnostics

Some compiler diagnostics are not yet implemented for the emulator.

CL_OUT_OF_RESOURCES Error Returned When Launching a Kernel

This can occur when a kernel uses more `__private` or `__local` memory than the emulator supports by default.

Once you have determined the amount of memory needed, try setting larger values for the `CL_CONFIG_CPU_FORCE_PRIVATE_MEM_SIZE` or the `CL_CONFIG_CPU_FORCE_LOCAL_MEM_SIZE` environment variable, as described in [Emulator Environment Variables](#).

NOTE On Windows, the FPGA emulator can silently fail by running out of memory. As a workaround to catch this error, write your kernel code using the try-catch syntax.

FPGA Runtime Compatibility With Emulation Binaries

The oneAPI FPGA runtime does not support emulation binaries built using an earlier version of oneAPI. You must recompile emulation binaries with the current oneAPI release.

libstdc++.so and GCC Version

The emulator is built with GCC 7.4.0 as part of the Intel® oneAPI DPC++/C++ Compiler. When running the executable for an emulated FPGA device, the version of `libstdc++.so` must be at least that of GCC 7.4.0. In other words, the `LD_LIBRARY_PATH` environment variable must ensure that the correct version of `libstdc++.so` is found.

If the correct version of `libstdc++.so` is not found, the call to `clGetPlatformIDs` function fails to load the FPGA emulator platform and returns `CL_PLATFORM_NOT_FOUND_KHR` (error code -1001). Depending on which version of `libstdc++.so` is found, the call to `clGetPlatformIDs` may succeed, but a later call to the `clCreateContext` function may fail with `CL_DEVICE_NOT_AVAILABLE` (error code -2).

If the `LD_LIBRARY_PATH` does not point to a compatible `libstdc++.so`, use the following syntax to invoke the host program:

```
env LD_LIBRARY_PATH=<path to compatible libstdc++.so>:$LD_LIBRARY_PATH <executable> [executable arguments]
```

Evaluate Your Kernel Through Simulation

The Questa*-Intel® FPGA Edition and Questa*-Intel® FPGA Starter Edition software assess the functionality of your kernel.

The simulator flow generates a simulation binary file that runs on the host. The hardware portion of your code is evaluated in an RTL simulator, and the host portion is executed natively on the processor. This feature allows you to simulate the functionality of your kernel and iterate on your design without needing to compile your kernel to hardware and run on the FPGA each time.

NOTE The performance of the simulator is very slow when compared to that of hardware. To reduce simulation times, use a smaller data set for testing.

Use the simulator when you want an insight into the dynamic performance of your kernel and more information about the functional correctness of your kernel than emulation or the reporting tools provide.

Verifying the functionality of your design in this way is sometimes called debugging through simulation.

To verify the design functionality from your design simulation, use the following debugging techniques:

- Run the executable that the compiler generates by targeting the FPGA device.
- Write variable values to output pipes or `mm_host` interfaces at certain points in your code.
- Review the waveforms generated when running your design.

The compiler does not log signals by default when you compile your design. To enable signal logging in simulation, refer to [Debug During Verification](#).

The simulator is cycle accurate and bit-accurate. It has a netlist identical to the generated hardware and can provide full waveforms for debugging. View the waveforms with Siemens* EDA (formerly Mentor Graphics) Questa* software.

Debug During Verification

By default, the compiler instructs the simulator not to log any signals because logging signals slows the simulation, and waveform files can be extremely large. However, you can configure the compiler to save these waveforms for debugging purposes.

To enable signal logging in the simulator, invoke the `icpx` command with the `-Xsghdl` option command as follows:

```
icpx -fintelfpga -Xssimulation -Xstarget=<family_or_part_number> -Xsghdl <input_files>
```

NOTE After you compile your component and testbench with the `-Xsghdl` option, run the resulting executable to run the simulation and generate the waveform.

While the simulation runs, or after it finishes, open the `vsim.wlf` file inside the current project directory to view the waveform.

To view the waveform:

1. In the Questa® simulator, open the `vsim.wlf` file inside the `<project_name>.prj` directory.
2. Right-click the **<RTL_IP_component_name>_inst** block and select **Add Wave**.

You can now view the top-level component signals: `start`, `done`, `ready_in`, `ready_out`, parameters, and outputs. Use the waveform to see how the component interacts with its interfaces.

Tip When you view the simulation waveform in the Questa® simulator, the simulation clock period is set to a default value of 1000 picoseconds (ps). To synchronize the Time axis to show one cycle per tick mark, change the time resolution from picoseconds (ps) to nanoseconds (ns):

1. Right-click the timeline and select **Grid, Timeline & Cursor Control**.
 2. Under **Timeline Configuration**, set the **Time** units to **ns**.
-

Compile a Kernel for Simulation

Before performing simulation, you must ensure that you have installed the Quartus® Prime Pro Edition software on your system. For more information, refer to the [Intel® oneAPI Toolkits Installation Guide](#) and [Intel® FPGA development flow](#) webpage.

To compile a kernel for simulation, include the `-Xssimulation` option in your `icpx` command as shown in the following:

```
icpx -fintelfpga -Xssimulation fpga_compile.cpp
```

To enable collecting the waveform during the simulation, include the `-Xsghdl[=<depth>]` option in your `icpx` command, where the optional `<depth>` attribute specifies how many levels of hierarchy are logged. If you do not specify a value for the `<depth>` attribute, a depth of 1 is used by default. To log all waveforms, specify a depth of 0 (`-Xsghdl=0`).

When simulating on Windows systems, you need the Microsoft linker and additional compilation time libraries. Verify the following settings:

- The `PATH` environment variable setting must include the path to the `LINK.EXE` file in Microsoft Visual Studio.
- `LIB` environment variable setting includes the path to the Microsoft compile-time libraries. The compile-time libraries are available with Microsoft Visual Studio.

Simulate Your Kernel

The simulation runtime creates a simulation device based on the automatically discovered or specified `board_spec.xml/ipinterfaces.xml` file. You can apply two simulation environment variables to control the `.xml` file searching mechanism.

- `CL_CONTEXT_MPSIM_DEVICE_INTELFPGA`: When you set this environment variable to 1, the simulation runtime attempts to automatically search for a `board_spec.xml/ipinterfaces.xml` file to use, based on the following rules:

- If there is only one `.prj` directory in the current working directory, the compiler uses the `board_spec.xml/ipinterfaces.xml` file in that directory.
- If there are multiple `.prj` directories in the current working directory, the compiler uses the `board_spec.xml/ipinterfaces.xml` file in the directory with the name matching the current executable name. When comparing the name, the compiler strips the extensions (`.exe`, `.bin`, `.elf`, and `.out` for Linux and `.exe` for Windows).

NOTE When the automatic search fails, the runtime emits a `CL_DEVICE_NOT_AVAILABLE` error.

- `INTELFPGA_SIM_DEVICE_SPEC_DIR`: Use this environment variable to set the path to the directory containing the target `.xml` file when the automatic search mechanism fails or when the automatically found `.xml` file is not desired. This environment variable forces the value of the `CL_CONTEXT_MPSIM_DEVICE_INTELFPGA` variable to 1 so that you can avoid specifying both variables to run a design in simulation.

If you want to use the simulation flow and view the waveforms generated during simulation, you must have either the Siemens EDA* Questa* Simulator or ModelSim* SE installed and available.

To run your SYCL* library through the simulator:

1. Set the `CL_CONTEXT_MPSIM_DEVICE_INTELFPGA` or `INTELFPGA_SIM_DEVICE_SPEC_DIR` environment variable to enable the simulation device:

- **Linux**

```
export CL_CONTEXT_MPSIM_DEVICE_INTELFPGA=1
or
export INTELFPGA_SIM_DEVICE_SPEC_DIR=<prj dir>
```

- **Windows**

```
set CL_CONTEXT_MPSIM_DEVICE_INTELFPGA=1
or
set INTELFPGA_SIM_DEVICE_SPEC_DIR=<prj dir>
```

NOTE When you set any of the simulation environment variables, only the simulation devices are available. That is, access to physical boards is disabled. To unset the environment variable, run the following command:

- **Linux**

```
unset CL_CONTEXT_MPSIM_DEVICE_INTELFPGA
or
unset INTELFPGA_SIM_DEVICE_SPEC_DIR
```

- **Windows**

```
set CL_CONTEXT_MPSIM_DEVICE_INTELFPGA=
or
set INTELFPGA_SIM_DEVICE_SPEC_DIR=
```

You might need to set `CL_CONTEXT_COMPILER_MODE_INTELFPGA=3` if the host program cannot find the simulator device.

2. Run your host program. On Linux systems, you can use GDB or Eclipse to debug your host. If necessary, you can inspect the simulation waveforms for your kernel code to verify the functionality of the generated hardware.

If you compiled with the `-Xsghdl` flag, running your compiled program produces a waveform file (`vsim.wlf`) that you can view in the Questa*-Intel® FPGA Edition software as your host code executes. The `vsim.wlf` file appears in the `.prj` directory.

Viewing Simulation Waveforms

By default, the Intel oneAPI DPC++/C++ Compiler instructs the simulator not to log any signal because logging signals slows the simulation, and the waveform files can be enormous. However, you can configure the compiler to save these waveforms for debugging purposes.

To enable signal logging in the simulator, invoke the `icpx` command with the `-Xsghdl` option, as follows:

```
icpx -fintelpga -Xssimulation -Xsghdl[=<depth>] <input files> -o <project_name>
```

Specify the `<depth>` attribute to indicate the number of hierarchy levels logged. A depth value of 1 logs only the top-level signals. A depth of 1 is used as the default if you do not specify the `<depth>` attribute. To log all waveforms, specify a depth of 0 (`-Xsghdl=0`).

After running the simulation, you can view the generated waveform files by invoking the appropriate script as follows:

- **Linux**

```
bash <project_directory>/view_waveforms.sh
```

- **Windows**

```
<project_directory>\view_waveforms.cmd
```

NOTE The `<project_directory>` is commonly `<project_name>.prj`, where `<project_name>` is the name specified with the `-o` argument to the `icpx` command.

Troubleshooting Simulator Issues

Review this section to troubleshoot simulator problems that you might have when attempting to run a simulation.

Windows Compilation or Run Fails

On Windows, simulation might fail at compilation time or run time if you are running from a directory with a very long path. Use the `-o` compiler option to output your compilation results to a shorter path.

A `socket=-11` Error Is Logged to `transcript.log`

If you receive the following error message, you might be mixing resources from multiple simulators, such as Questa*-Intel FPGA Edition and ModelSim* SE:

```
Message: "src/hls_cosim_ipc_socket.cpp:202: void IPCSocketMaster::connect():  
Assertion `sockfd != -1 && "IPCSocketMaster::connect() call to accept() failed"' failed."
```

An example of mixing simulator resources is compiling a device with ModelSim* SE and running the host program in Questa*-Intel FPGA Starter Edition.

Compatibility with Questa*-Intel FPGA Starter Edition Software

Questa*-Intel FPGA Starter Edition software has limitations on design size that prevent it from simulating large designs. When trying to launch a simulation using Questa*-Intel FPGA Starter Edition software, you may encounter the following error message:

```
Error: The simulator's process ended unexpectedly.
```

Instead, simulate the designs with Questa*-Intel FPGA Edition or ModelSim* SE software.

Environment Variable Not Set

When you forget setting the `CL_CONTEXT_MPSIM_DEVICE_INTELFPGA=1` environment variable, you might encounter an error message.

On Linux

```
terminate called after throwing an instance of 'sycl::_V1::runtime_error'
what(): No device of requested type available. Please check https://www.intel.com/content/www/us/en/developer/articles/system-requirements/intel-oneapi-dpcpp-system-requirements.html -1 (PI_ERROR_DEVICE_NOT_FOUND)
Aborted (core dumped)
```

On Windows

The simulator might crash and you might see the following message in the debugger:

```
Unhandled exception at 0x00007FFD92DFDE4E (ucrtbase.dll) in <your exe>.exe: Fatal program exit requested.
```

To overcome this issue, define the `CL_CONTEXT_MPSIM_DEVICE_INTELFPGA=1` environment variable.

Analyzing Your Design

7

When compiling an FPGA hardware image, the Intel® oneAPI DPC++/C++ Compiler provides object files, the FPGA Optimization Report, an FPGA image object, and executables as checkpoints to allow you to inspect errors and modify your source code without performing a full compilation on each iteration.

At the FPGA Optimization Report image object checkpoint, you can view the optimization report that the compiler generates. At the FPGA image object checkpoint, the compiler generates a complete FPGA image.

For detailed information about the FPGA compilation, refer to the [FPGA Flow Terminology](#).

Review the FPGA Optimization Report

The FPGA optimization report provides suggestions on how you can modify kernels to increase performance. For compiler commands to generate the report, refer to the [FPGA Compilation Flags](#).

Navigate to the `<project_dir>/reports/` directory and review one of the following files of your application to determine whether the estimated kernel performance data is acceptable. Both options display the same report:

- `report.html`: You can view this file using Internet browsers, such as Microsoft* Edge, Google* Chrome, or Mozilla* Firefox.
- `<design_name>.zip`: Use the Intel® oneAPI FPGA Reports tool `report.html` as described in the following sections to view the optimization reports:
 - [Launching the Intel® oneAPI FPGA Reports Tool](#)
 - [Loading Your Design Optimization Report](#)
 - [Comparing Reports](#)
 - [Filtering Nodes in Report Tables \(Linux only\)](#)

Launching the Intel® oneAPI FPGA Reports Tool

The Intel oneAPI FPGA Reports Tool is a generic `report.html` file from which you can open the `<design_name>.zip` file for any design. Use the Intel oneAPI FPGA Reports Tool to navigate between multiple reports quickly and easily, while needing to bookmark only one `report.html` file in your chosen Internet browser.

The Intel oneAPI FPGA Reports tool is installed as part of the FPGA Support Package for the Intel® oneAPI DPC++/C++ Compiler.

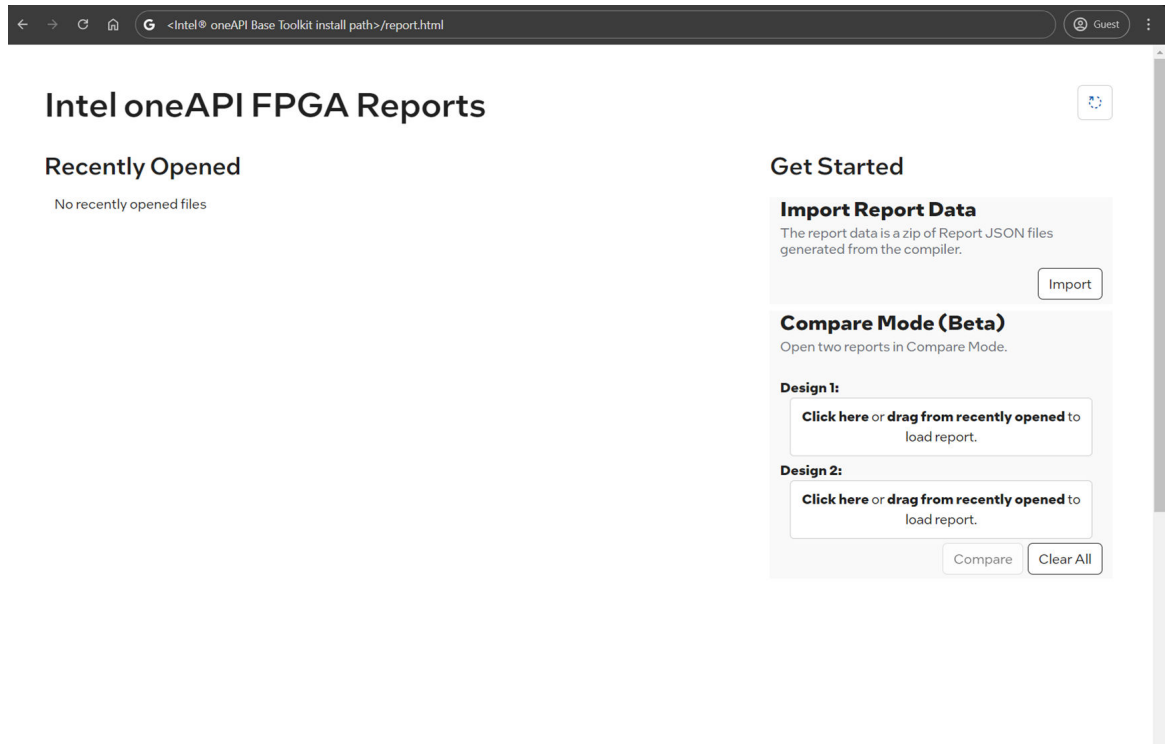
Run the `oneAPI setvars` (or `oneapi-vars`) script to have the `fpga_report` command added to your `PATH` environment variable.

On a Linux system, invoke the `fpga_report` command on the command-line interface. The command reports the path to the Intel oneAPI FPGA Reports Tool `report.html` file, which you can then open in a supported web browser.

On a Windows system, invoke the `fpga_report.exe` command from the Intel® oneAPI Base Toolkit install path. The command opens the Intel oneAPI FPGA Reports Tool `report.html` file in your default web browser.

The initial page of the Intel oneAPI FPGA Reports Tool `report.html` file displays a menu as shown in the following image:

Intel® oneAPI FPGA Reports Tool



Loading Your Design Optimization Report

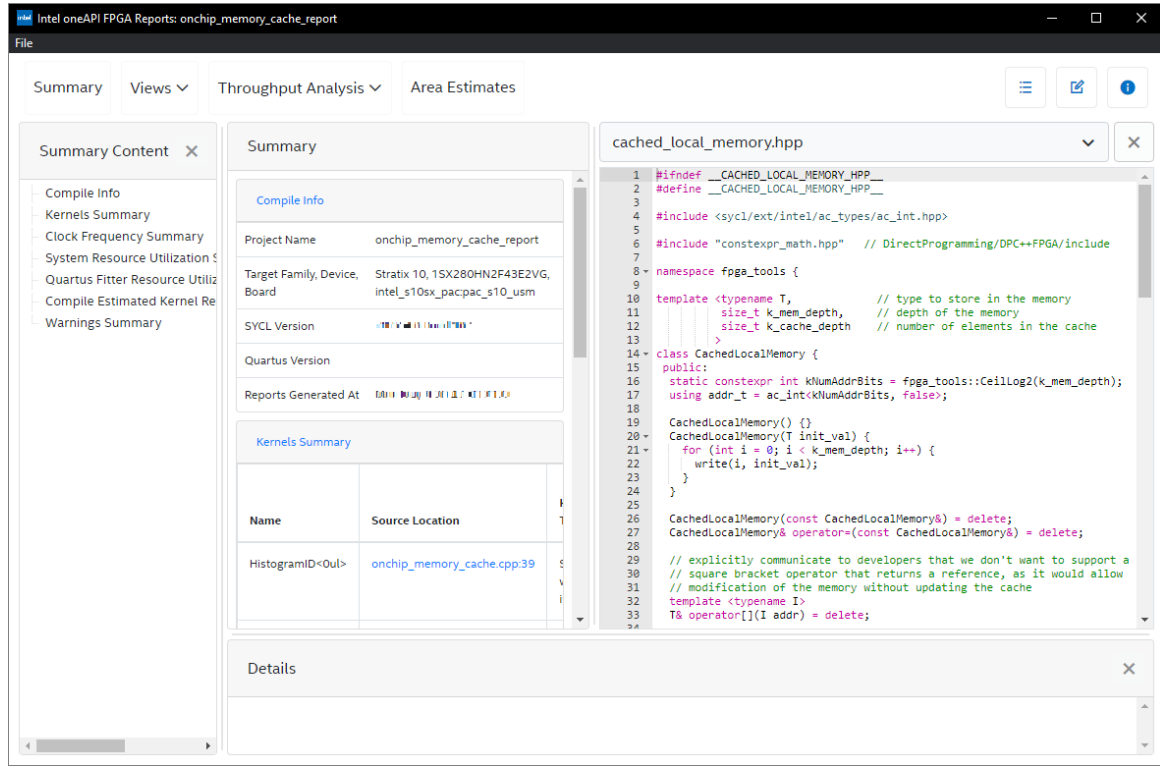
Load a design optimization report by importing it into the Intel® oneAPI FPGA Reports interface as follows:

1. Go to **Get Started** > **Import Report Data** and click **Import**.
2. Navigate to the `<project_dir>/reports/` directory.
3. Select the `<design_name>.zip` file.
4. Click **Open**. The report loads in the Intel® oneAPI FPGA Reports interface.

To open a report that you have opened previously, click the report under **Recently Opened**. This list is browser-specific, so reports opened in one browser are not shown in the **Recently Opened** list in a different browser.

Restriction Mozilla* Firefox does not support the **Recently Opened** section.

Sample Optimization Report in the Intel® oneAPI FPGA Reports Tool



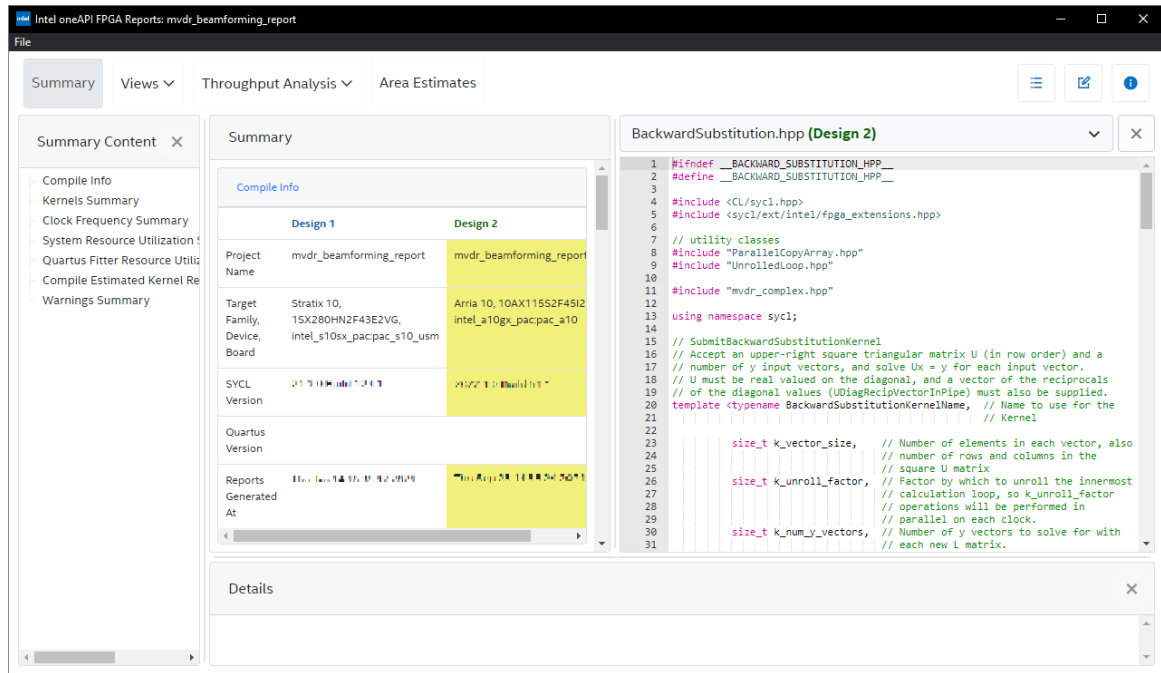
Comparing Reports

Use the Intel® oneAPI FPGA Reports tool to compare two optimization reports of the same design or different designs as follows:

1. Under **Compare Reports > Design 1**, click **Click here**, navigate to the <project_dir>/reports directory and select the <design_name>.zip file of the first design. You can also drag files from the **Recently Opened** list to the file field.
2. Under **Compare Reports > Design 2**, click **Click here**, navigate to the <project_dir>/reports directory and select the <design_name>.zip file of the second design. You can also drag files from the **Recently Opened** list to the file field.
3. Click **Compare**.

The selected reports are launched in comparison mode as shown in the following image:

Comparison Mode in the Intel® oneAPI FPGA Reports Tool



NOTE

Currently, only the Summary View and Loop Analysis have a comparison mode. For views not supported in the comparative mode, a view for each design is generated and you can identify the designs by the **Design <number>** text in the view name.

Filtering Nodes in Report Tables (Linux only)

Restriction File nodes in report tables is available only for designs that were compiled on a Linux* based operating system.

For table views in the Intel® oneAPI FPGA Reports, you can filter nodes in the table views to hide information about data that is outside of your design such as header files and third-party libraries. With a filtered view of your design, you can focus on your design with fewer distractions.

For large designs, you can filter nodes to reduce clutter in the report tables and improve the load time of report tables.

You filter nodes by choosing what source files to hide in the reports. Hiding a source file in the reports hides all the nodes that result from the hidden source file.

Data that is found in tables that provide summaries about the entire design (such as area summaries) do not remove data from hidden nodes. The summaries show information about the entire design regardless of any node filtering applied.

Remember Only nodes seen in table views will be hidden. Node Filtering does not affect the graphical views such as System Viewer or Schedule Viewer.

To filter nodes in the table views of the reports:

1. From the Summary of the report, open the Report Settings page by clicking the settings icon in the top-right corner of the view:

The screenshot shows the 'Summary' report view. On the left is a 'Summary Content' sidebar with a list of report sections. The main area displays 'Compile Info' and 'Kernels Summary'. The 'Kernels Summary' table lists three kernels: Beamformer, InputDemo, and CalcWeights. In the top-right corner of the report view, there is a settings icon (a gear) which is highlighted by a red arrow.

2. On the Report Settings page, go to the Hide Source Files section.

The screenshot shows the 'Report Settings' page. The 'Hide Source File(s)' section is active. It contains a text field for 'Enter filepath regex' and a search icon. Below this is a tree view of files to be hidden. The 'Header Files' section is expanded, showing three files: exception_handler.hpp, fft2d.hpp, and twiddle_factors.hpp. The 'Source Files' section is also expanded, showing one file: fft2d_demo.cpp.

3. In the Hide Source Files section, select the source files for the nodes that you want to hide in the table views. You can scroll this list of files or use the search field to filter the list file names that match a regular expression.

Summary Views ▾ Throughput Analysis ▾ Area Estimates

Report Settings

Hide Source File(s)

Files selected in the below tree list will result in nodes originating from said files to be hidden from table views.
NOTE: Area & resource summaries are not affected by filtered files.

fft2d/  

- ▼ ☒ Header Files
 - ☒ fft2d.hpp (referencedesigns/fft2d/src/fft2d.hpp)
 - ☒ twiddle_factors.hpp (referencedesigns/fft2d/src/twiddle_factors.hpp)
- ▼ ☐ Source Files
 - ☐ fft2d_demo.cpp (referencedesigns/fft2d/src/fft2d_demo.cpp)

The Hide Source Files section also provides categories of source files. Select a category to hide nodes from all the source files in that category.

Filtered nodes can show up in navigation lists, but selecting them displays a message that indicates that the source file for the node is hidden:

Summary Views ▾ Throughput Analysis ▾ Area Estimates

Loop List

- ▼ Kernel System
 - ▼ Kernel: FetchKernel (fft2d.hpp:383)
 - ▼ TestFFT_FetchKernel.B1 (fft2d.hpp:397)
 - ▼ TestFFT_FetchKernel.B3 (fft2d.hpp:400)
 - Fully unrolled loop (fft2d.hpp:427)
 - ▼ TestFFT_FetchKernel.B5 (fft2d.hpp:433)
 - ▼ Fully unrolled loop (fft2d.hpp:441)
 - Fully unrolled loop (fft2d.hpp:345)
 - ▼ Kernel: TransposeKernel (fft2d.hpp:510)
 - ▼ TestFFT_TransposeKernel.B1 (fft2d.hpp:538)
 - ▼ TestFFT_TransposeKernel.B3 (fft2d.hpp:540)
 - Fully unrolled loop (fft2d.hpp:544)
 - ▼ TestFFT_TransposeKernel.B4 (fft2d.hpp:549)

Loop Analysis

No data is available for this table due to hidden files or missing data.

Use the [setvars](#) and [oneapi-vars](#) Scripts with Linux*

Use the [setvars](#) and [oneapi-vars](#) Scripts with Windows*

Loop Analysis

The `report.html` file contains information about all loops in your design and their unroll statuses. The Loop Analysis report helps you examine whether the Intel® oneAPI DPC++/C++ Compiler can maximize your kernel's throughput.

To view the Loop Analysis report, click **Throughput Analysis > Loop Analysis**. The purpose of this view is to show estimates of performance indicators (such as II) and potential performance bottlenecks. For each loop, you can identify the following using the report:

- Whether the loop is pipelined
- Whether the loop uses a hyper-optimized loop structure
- Any pragma or attribute applied to the loop

- II of the loop

NOTE

- Loop Analysis report does not report anything about loops for NDRange kernels.
- FPGA optimization reports support user-defined loop labels and replace the system-generated loop labels.

The left-hand **Loops List** pane of the Loop Analysis report displays the following types of loops the compiler detects in your design and indicates any transformations the compiler may have applied. These transformations may be applied automatically or due to a source-code annotation, such as a pragma or an attribute.

- Fused loops (see [Fuse Loops to Reduce Overhead and Improve Performance](#))
- Fused subloops
- Coalesced loops
- Fully unrolled loops (see [Unroll Loops](#))
- Partially unrolled loops

Key Performance Metrics

The Loop Analysis report captures the following key performance metrics on all blocks:

- **Source Location:** Indicates the loop location in the source code.
- **Pipelined:** Indicates whether a loop is pipelined. Pipelining allows for many data items to be processed concurrently (in the same clock cycle) while efficiently using of the hardware in the datapath by keeping it occupied.
- **II:** Shows the sustainable initiation interval (II) of the loop. Processing data in loops is an additional source of pipeline parallelism. When you pipeline a loop, the next iteration of the loop begins before previous iterations complete. You can determine the number of clock cycles between iterations by the number of clock cycles you require to resolve any dependencies between iterations. You can refer to this number as the initiation interval (II) of the loop. The Intel® oneAPI DPC++/C++ Compiler automatically identifies these dependencies and builds hardware to resolve these dependencies while minimizing the II.
- **Estimated f_{MAX} :** Shows the estimated maximum clock frequency at which the loop operates. You can also reference it as "Scheduled f_{MAX} ." The f_{MAX} is the maximum rate at which the outputs of registers are updated. If the estimated f_{MAX} is below the target frequency, then the estimated f_{MAX} appears in red color and a question mark with a tooltip displaying the target frequency displays.

The physical propagation delay of the signal between two consecutive registers limits the clock speed. This propagation delay is a function of the complexity of the Boolean logic in the path. The path with the most logic (and the highest delay) limits the speed of the entire circuit, and you can refer to this path as the critical path.

The f_{MAX} is calculated as the inverse of the critical path delay. High f_{MAX} is desirable because it correlates directly with high performance in the absence of other bottlenecks. The compiler attempts to optimize for different objectives for the estimated f_{MAX} depending on whether the f_{MAX} target is set and whether the `#pragma II` is set for each of the loops. The f_{MAX} target is a strong suggestion, and the compiler does not error out if it cannot achieve this f_{MAX} , whereas the `#pragma II` triggers an error if the compiler is not able to achieve the requested II. The f_{MAX} achieved for each block of code is shown in the Loop Analysis report. This behavior is outlined in the following table:

Explicitly specify f_{MAX} ?	Explicitly specify II?	Compiler Behavior
No	No	Use heuristic to achieve best f_{MAX} /II trade-off.
No	Yes	Best effort to achieve the II for the corresponding loop (may not achieve the best possible f_{MAX}).

Explicitly specify f_{MAX} ?	Explicitly specify II?	Compiler Behavior
Yes	No	Best effort to achieve f_{MAX} specified (may not achieve the best possible II).
Yes	Yes	Best effort to achieve the f_{MAX} specified at the given II. The compiler errors out if it cannot achieve the requested II.

NOTE Intel® recommends that if you are using an f_{MAX} target in the command line or for a kernel, use `#pragma II = <N>` for performance-critical loops in your design.

- **Latency:** Shows the number of clock cycles a loop takes to complete one or more instructions. Typically, you want to have low latency. However, lowering latency often results in decreased f_{MAX} .
- **Speculated Iterations:** Shows the loop speculation. Loop speculation is an optimization technique that enables more efficient loop pipelining by allowing future iterations to be initiated before determining whether the loop was exited already.
- **Max Interleaving Iterations:** Indicates the number of interleaved invocations of an inner loop that can be executed simultaneously. For more information, refer to [max_interleaving Attribute](#).

Loop Analysis Example

The following is a SYCL* kernel example that includes three loops:

```
cgh.single_task<class example>([=] () {
    #pragma unroll
    for (int i = 0; i < 10; i++) {
        acc_data[i] += i;
    }
    #pragma unroll 1
    for (int k = 0; k < N; k++) {
        #pragma unroll 5
        for (int j = 0; j < N; j++) {
            acc_data[j] = j + k;
        }
    }
});
```

The Loop Analysis report of this design example highlights the unrolling strategy for the different kinds of loops defined in the code.

The Intel® oneAPI DPC++/C++ Compiler implements the following loop unrolling strategies based on the source code:

- Fully unrolls the first inner loop (line 3) because of the `#pragma unroll` specification.
- Does not unroll the second loop (line 7), which is an outer loop because of the `#pragma unroll 1` specification.
- Unrolls the third loop (line 9, an inner loop of the second loop) five times because of the `#pragma unroll 5` specification.

For more examples, refer to [Loops](#) section.

Loop Pragma and Attributes

You can use the Loop Analysis report to help determine where to deploy one or more of the following pragmas or attributes on your loops:

- [disable_loop_pipelining Attribute](#)

- [initiation_interval](#) Attribute
- [ivdep](#) Attribute
- [loop_coalesce](#) Attribute
- [max_concurrency](#) Attribute
- [max_interleaving](#) Attribute
- [speculated_iterations](#) Attribute
- [unroll](#) Pragma
- [Loop Fuse Functions and nofusion](#) Attribute
- [max_reinvocation_delay](#) Attribute (Beta)

Bottlenecks Viewer

The Bottlenecks viewer, when used with the [Loop Analysis](#) and [Schedule Viewer](#), provides information about the throughput bottleneck(s) in your design. This viewer lists all loops that result in a bottleneck for the currently selected system, kernel, or task. You can select these loops to view more details about the bottleneck in the Details pane. For more information about the concept of bottlenecks, refer to [Remove Loop Bottlenecks](#).

The Bottlenecks viewer identifies the following categories of bottlenecks:

- F_{MAX} reduced, II increased, or both
- Compiler applied bottlenecks ([private copies](#) set to 1 on local memory)
- Bottlenecks due to pragmas or attributes you apply on a loop
- Concurrency limiter bottlenecks

For example, consider a simple loop with memory dependency from the [Triangular Loops](#) example on GitHub:

```
for (int y = x + 1; y < n; y++) {
    local_buf[y] = local_buf[y] + SomethingComplicated(local_buf[x]);
}
```

In the Loop Analysis viewer, this loop is identified as having an II of 30. When you select this loop in the Loop Analysis report, you can find the following message in the Details pane:

```
Compiler failed to schedule this loop with smaller II due to memory dependency:
  From: Load Operation (triangular_loop.cpp: 78)
  To: Store Operation (triangular_loop.cpp: 78)
Compiler failed to schedule this loop with smaller II due to memory dependency:
  From: Load Operation (triangular_loop.cpp: 78)
```

In the Bottlenecks viewer, you can then select the loop to display more information in the Details pane, which you can use to investigate why and what caused this bottleneck. For additional information about the bottlenecks, refer to [System Viewer](#) and [Schedule Viewer](#) reports. The System Viewer provides information about the isolated failing path and bottleneck type. The Schedule Viewer displays the bottleneck path for the variable.

Area Estimates

This view gives you an estimate of how many device resources your design needs. The view can be useful if you are trying to ensure that your design fits onto the FPGA device, or you are trying to keep the design below a certain resource footprint.

The report provides the following information:

- Detailed area breakdown of the whole DPC++ system, mapped to your source code where possible.
- Architectural details to give insight into the generated hardware and offers actionable suggestions to resolve potential inefficiencies.

To view the Area Estimates report, click **Area Estimates**.

As you can observe in the following figure, the report is divided into three levels of hierarchy:

- **System area:** Used by all kernels, pipes, interconnects, and board logic.
- **Kernel area:** Used by a specific kernel, including resources by the logic inside the kernel, as well as resources used by local memories in the kernel and the workgroup dispatch logic that is detecting when it can send new workgroups into the kernel.
- **Block area:** Used by a specific block within a kernel. A block represents a branch-free section of your source code (for example, a loop body). To view the area, use information from the source code lines associated with a block and expand the report entry for that block.

Each line in the table is a sum of the resources used by itself and all of its children.

Area Estimates Report Hierarchy

Area Estimates						
Notation file:X > file:Y indicates a function call on line X was inlined using code on line Y.						
Collapse All Expand All						
	ALUTs	FFs	RAMs	MLABs	DSPs	Details
Static Partition Board interface Kernel System	179950 (0%)	358572 (21%)	492 (18%)	0 (0%)	123 (8%)	
	179950	358572	492	0	123	Platform i...
Global interconnect	5034	9568	52	0	0	For 1 glob...
System description ROM	0	67	2	0	0	Contains i...
kernelCompute Function overhead Private Variable: \n - 'i' (max_replicates.cpp:50) Private Variable: \n - 'i' (max_replicates.cpp:56) Private Variable: \n - 'result' (max_replicates.cpp:54) max_replicates.cpp:46 (mem)	2174 (<1%)	5169 (<1%)	15 (<1%)	47 (<1%)	0 (0%)	1 compute ...
	1072	2225	0	10	0	Kernel dis...
	14	47	0	0	0	Register,\...
	7	36	0	0	0	Register,\...
	7	36	0	0	0	Register,\...
	0	0	2	0	0	Stall-free...
kernelCompute.B0 kernelCompute.B2 kernelCompute.B3 kernelCompute.B4	8 (<1%)	2 (<1%)	0 (0%)	0 (0%)	0 (0%)	
	435 (<1%)	501 (<1%)	13 (<1%)	5 (<1%)	0 (0%)	
	234 (<1%)	186 (<1%)	0 (0%)	1 (<1%)	0 (0%)	
	397 (<1%)	2136 (<1%)	0 (0%)	31 (<1%)	0 (0%)	

NOTE

The area use data are estimates that the Intel® oneAPI DPC++/C++ Compiler generates. These estimates might differ from the final area utilization results.

Messages in the Area Estimates Report

After you compile your DPC++ application, review the Area Estimates report that the Intel® oneAPI DPC++/C++ Compiler generates. In addition to summarizing the application's resource use, the Area Estimates report offers suggestions on modifying your design to improve efficiency. Refer to the following sections that describe various messages reported in the Area Estimates report.

Board Interface

The Area Estimates report identifies the amount of logic that the Intel® oneAPI DPC++/C++ Compiler generates for the Custom Platform or board interface. The board interface is the static region of the device that facilitates communication with external interfaces such as PCIe®. The Custom Platform specifies the size of the board interface.

Function Overhead

The Area Estimates report identifies the amount of logic that the Intel® oneAPI DPC++/C++ Compiler generates for dispatching kernels.

State

The Area Estimates report identifies the number of resources your design uses for live values and control logic. To reduce the reported area consumption under State, modify your design as follows:

- Decrease the size of local variables.
- Decrease the scope of local variables by localizing them whenever possible.
- Decrease the number of nested loops in the kernel.

Feedback

The Area Estimates report specifies the resources that your design uses for loop-carried dependencies.

To reduce the reported area consumption under Feedback, decrease the number and size of loop-carried variables in your design.

Messages for Private Variable Storage

The Area Estimates report provides information on the implementation of private memory based on your DPC++ design. For single work-item kernels, the Intel® oneAPI DPC++/C++ Compiler implements private memory differently, depending on the types of variable. The Intel® oneAPI DPC++/C++ Compiler implements scalars and small arrays in registers of various configurations (for example, plain registers, shift registers, and barrel shifter). The Intel® oneAPI DPC++/C++ Compiler implements larger arrays in block RAM.

The following table lists messages and notes of different private variable storage types:

Additional Information About Area Estimates Report Message

Message	Notes
Implementation of Private Memory Using On-Chip Block RAM	
Private memory implemented in on-chip block RAM.	The block RAM implementation creates a system that is similar to local memory for NDRange kernels.
Implementation of Private Memory Using On-Chip Block ROM	
—	For each use of an on-chip block ROM, the Intel® oneAPI DPC++/C++ Compiler creates another instance of the same ROM. There is no explicit annotation for private variables that the Intel® oneAPI DPC++/C++ Compiler implements in on-chip block ROM.
Implementation of Private Memory Using Registers	

Message	Notes
Implemented using registers of the following size: • <X> registers of width <Y> bits and depth <Z>. • Depth was increased by a factor of <N> due to a loop initiation interval of <M>. • Each register is implemented in a RAM-based FIFO and consumes <U> RAMs.	Reports that the Intel® oneAPI DPC++/C++ Compiler implements a private variable in registers. The Intel® oneAPI DPC++/C++ Compiler might implement a private variable in many registers. This message provides a list of the registers with their specific widths and depths.
Implementation of Private Memory Using Shift Registers	
Implemented as a shift register with <N> or fewer tap points. This is a very efficient storage type. Implemented using registers of the following sizes: • <X> register(s) of width <Y> bits and depth <Z>. • Depth was increased by a factor of <N> due to a loop initiation interval of <M>. • Each register is implemented in a RAM-based FIFO and consumes <U> RAMs.	Reports that the Intel® oneAPI DPC++/C++ Compiler implements a private variable in shift registers. This message provides a list of shift registers with their specific widths and depths. The Intel® oneAPI DPC++/C++ Compiler might break a single array into several smaller shift registers depending on its tap points. NOTE The compiler might overestimate the number of tap points.
Implementation of Private Memory Using Barrel Shifters with Registers	
Implemented as a barrel shifter with registers due to dynamic indexing. This is a high overhead storage type. If possible, change to compile-time known indexing. The area cost of accessing this variable is shown on the lines where the accesses occur. Implemented using registers of the following size: • <X> registers of width <Y> bits and depth <Z>. • Depth was increased by a factor of <N> due to a loop initiation interval of <M>. • Each register is implemented in a RAM-based FIFO and consumes <U> RAMs.	Reports that the Intel® oneAPI DPC++/C++ Compiler implements a private variable in a barrel shifter with registers because of dynamic indexing. This row in the report does not specify the full area use of the private variable. The report shows additional area use information on the lines where the variable is accessed.

NOTE

- The Area Estimates report annotates memory information on the line of code that declares or uses private memory, depending on its implementation.
 - When the Intel® oneAPI DPC++/C++ Compiler implements private memory in on-chip block RAM, the Area Estimates report displays relevant local-memory-specific messages to private memory systems.
-

System Viewer

The FPGA Optimization Report provides a graph that visualizes the structure of the generated verilog, which includes sizes and types of loads and stores, stalls, latencies, load and store information between kernels and different memories, pipes connected between kernels, and loops.

The System Viewer (see [Figure 41: Kernel System View of the System Viewer Report](#)) shows an abstracted netlist of your DPC++ system in a hierarchical graphical report consisting of system, global memory, block, and cluster views. It allows you to review information such as sizes, types, dependencies, and schedules of instructions, FIFO created by feedback nodes, FIFO created for capacity balancing in stallable regions, properties of interfaces such as pipe and memory interface, and view variables with loop-carried dependencies.

To view the System Viewer, click **Views > System Viewer**.

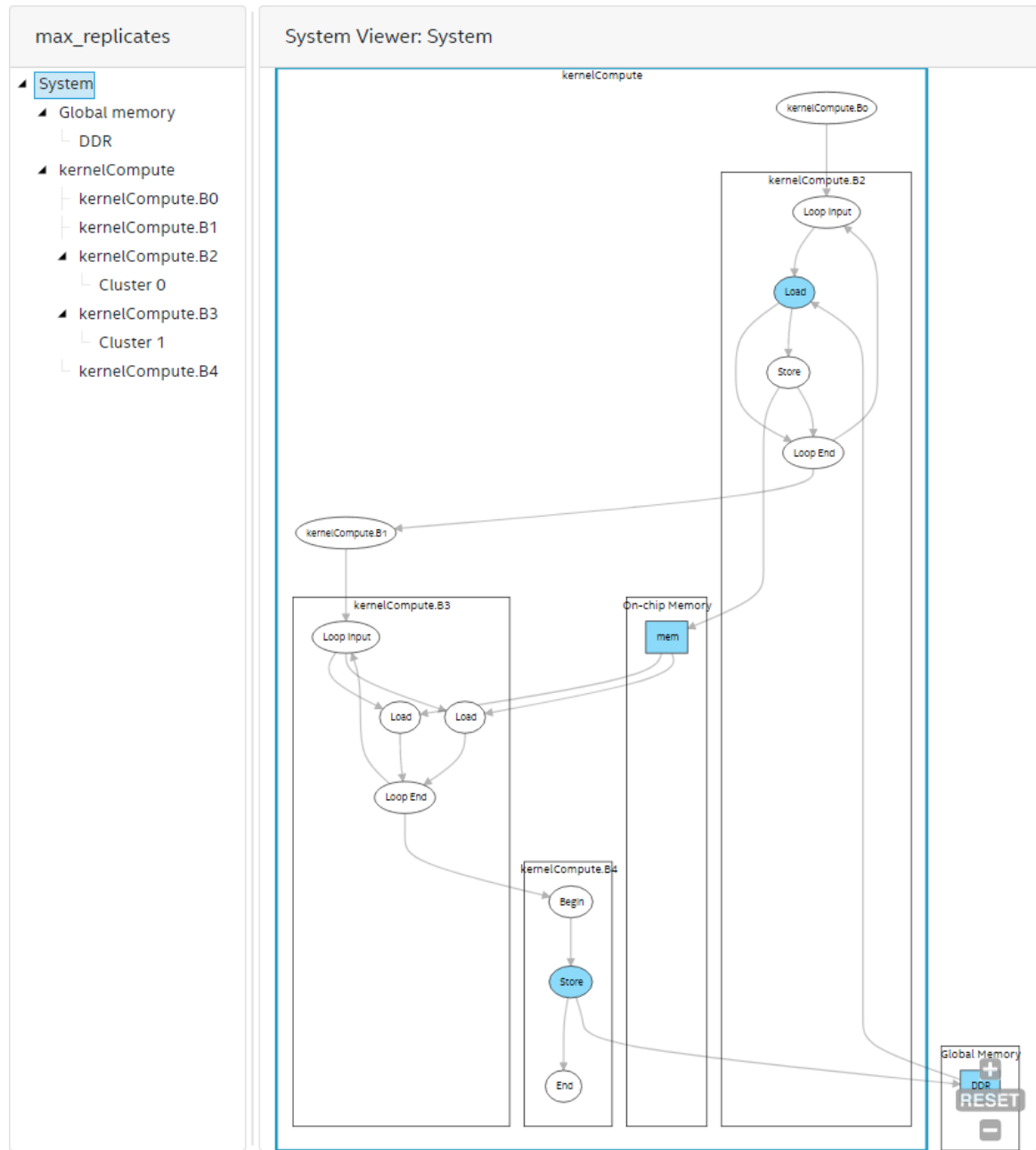
You can interact with the System Viewer in the following ways:

- Use the mouse wheel to zoom in and out within the System Viewer.
- Navigate through the following hierarchical views in the left-hand pane:
 - System
 - Global memory
 - Block
 - Cluster
- Click a node to display its location in the source code in the Code pane and node details in the Details pane.

System View

Use the system view of the System Viewer report to view various kernels in your system. The system view illustrates connections between your kernels and connections from kernels to memories. In addition, the system view shows the connection of blocks within a kernel and highlights blocks with a high initiation interval (II).

Kernel System View of the System Viewer Report



Global Memory View

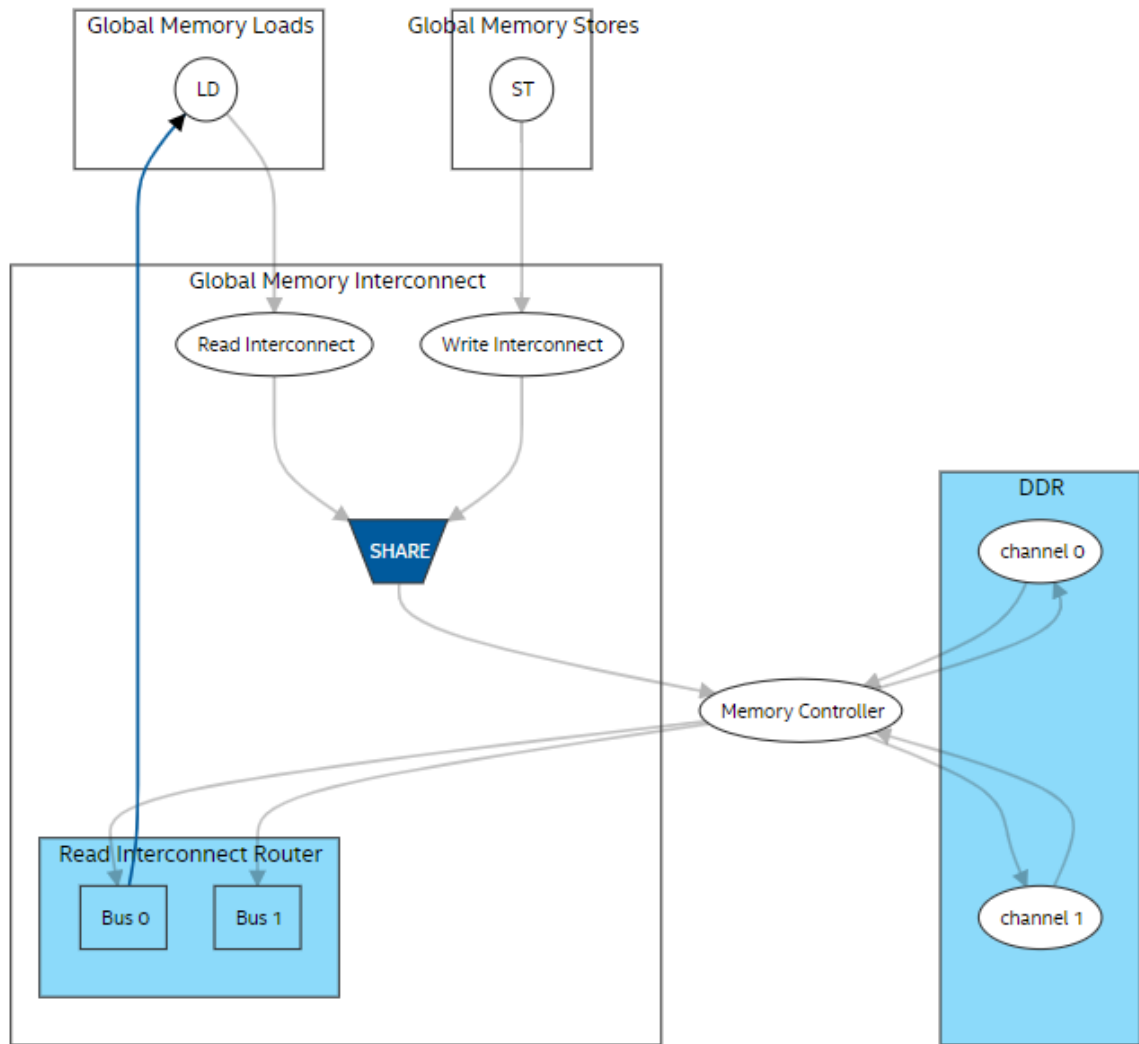
The global memory view of the System Viewer provides a list of all global memories in the design. The global memory view shows the following:

- Connectivity within the system showing data flow direction between global memory and kernels.

- Memory throughput bottlenecks.
- Status of the compiler flags, such as `-Xsnum-reorder` and `-Xsforce-single-store-ring`.
- Global **load-store unit (LSU)** types.
- Type of write/read interconnects.
- Number of write rings.
- Number and connectivity of read-router buses.

The following image is an example of the global memory view in the System Viewer:

Graphical Representation of the Global Memory in the System Viewer



In Figure 42: Graphical Representation of the Global Memory in the System Viewer:

- When you select stores or loads, you can view respective lines in the source code and details about the LSU type and LSU-level bandwidth.
- For the write interconnect block, you can view the interconnect style, number of writes to the global memory, status of the `-Xsforce-single-store-ring` compiler flag, and number of store rings.
- For the read interconnect block, you can view the interconnect style and number of reads from the global memory.
- For the read interconnect router block, you can view the status of the `-Xsnum-reorder` flag, total number of buses, and all connections between buses and load LSUs. Buses in this block provide read data from the memory to load LSUs.

- For the global memory (DDR in [Figure 42: Graphical Representation of the Global Memory in the System Viewer](#)), you can view the status of interleaving, interleaving size, number of channels, maximum bandwidth the BSP can deliver, and channel width.
- For the memory controller block, you can view the maximum bandwidth the BSP can deliver, sum of the load/store throughput, and read/write bandwidth. For additional information about how global memory bandwidth use is calculated, refer to [Global Memory Bandwidth Use Calculation](#) in this guide. It describes the formulas used in calculating the bandwidth.
- LSUs using USM pointers show up twice in both host and device global memory views as they can access both memories.

Block View

The block view of the System Viewer provides a more granular system view of the kernel. This view shows the following:

- Fine-grained details within kernels (including instructions, dependencies, and schedule of the instructions) of the generated datapath of computations. The Intel® oneAPI DPC++/C++ Compiler encapsulates maximum instructions in clusters for better quality of results (QoR). The System Viewer shows clusters, instructions outside clusters, and their connections.
- Linking from the instruction back to the source line by clicking the instruction node.
- Various information about the instructions, such as data width, node's schedule information in the start cycle and latency, are provided if applicable.

NOTE

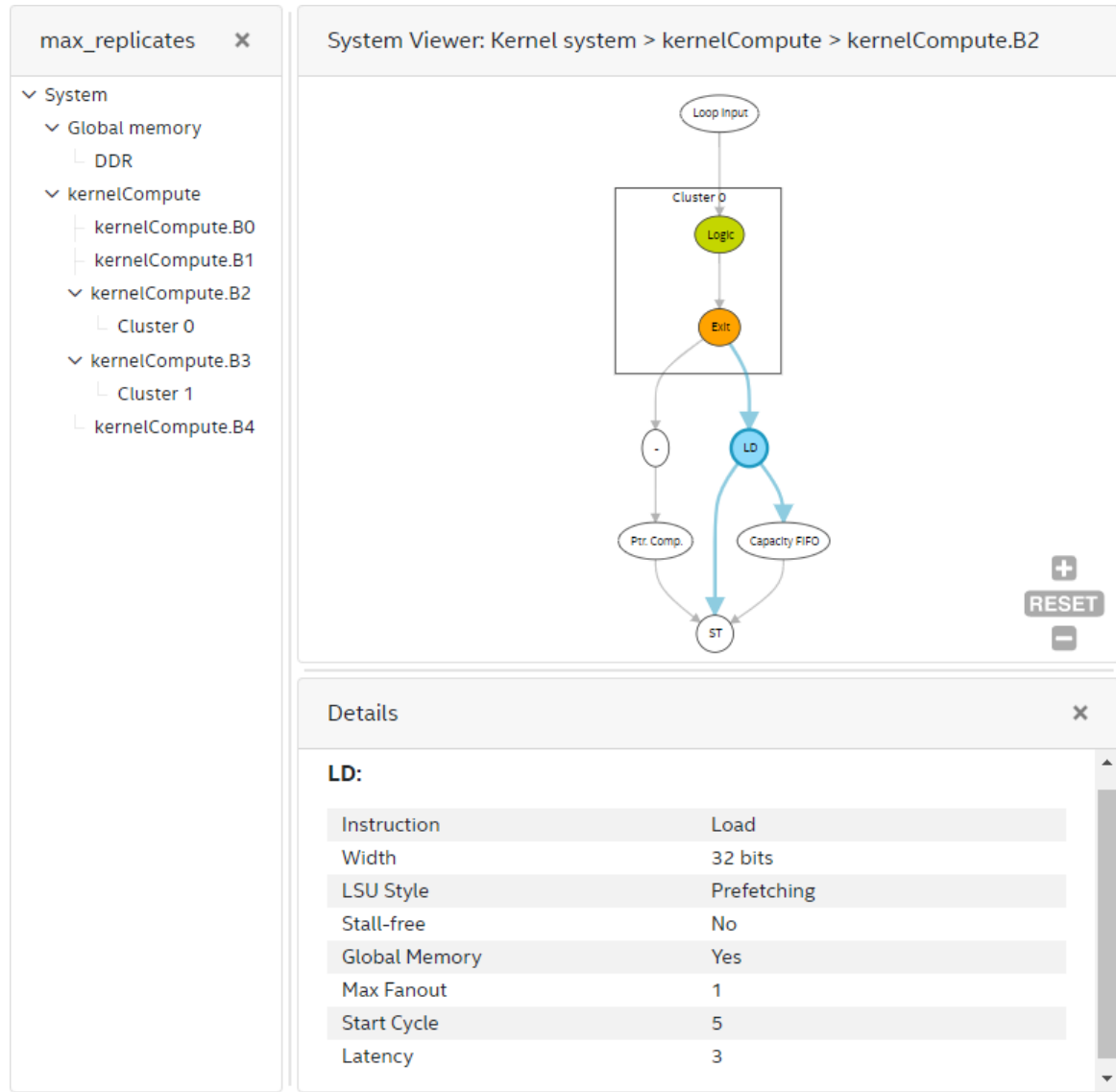
The schedule information is relative to the start of each block. Because the Intel® oneAPI DPC++/C++ Compiler cannot statically infer the trip counts of blocks, if your design consists of multiple blocks, the compiler cannot compute the absolute schedule information by considering the trip counts. Moreover, the schedule information provides estimated values from empirical measurements for the stallable instructions (such as pipe RD/WR or memory LD/ST). The real schedule is likely to be different, and you must verify it with a hardware, or a simulation run.

If your design has loops, the Intel® oneAPI DPC++/C++ Compiler encapsulates the loop control logic into loop orchestration nodes and the initial condition of the loops into loop input nodes and their connection to the datapath.

Inside a block, there are often pipe RD/WR or memory LD/ST nodes connecting to computation nodes or clusters. You can click on the computation nodes and view the Details pane (or hover over the nodes) to see specific instructions and the bit width. You can click on the RD/WR or LD/ST nodes to see information such as instruction type, width, depth, LSU type, stall-free global memory, scheduled start cycle, estimated latency, and schedule of a pipe or an LSU from the Details pane. For stallable nodes, the latency value provided is an estimate. Perform a simulation or hardware run for more accurate latency values.

If your design has clusters, a cluster has a FIFO in its exit node to store any pipelined data in-flight. You can click on the cluster exit node to find the exit FIFO width and depth attribute. The cluster exit FIFO size is also available in the cluster view of the System Viewer.

Block View of the System Viewer Report



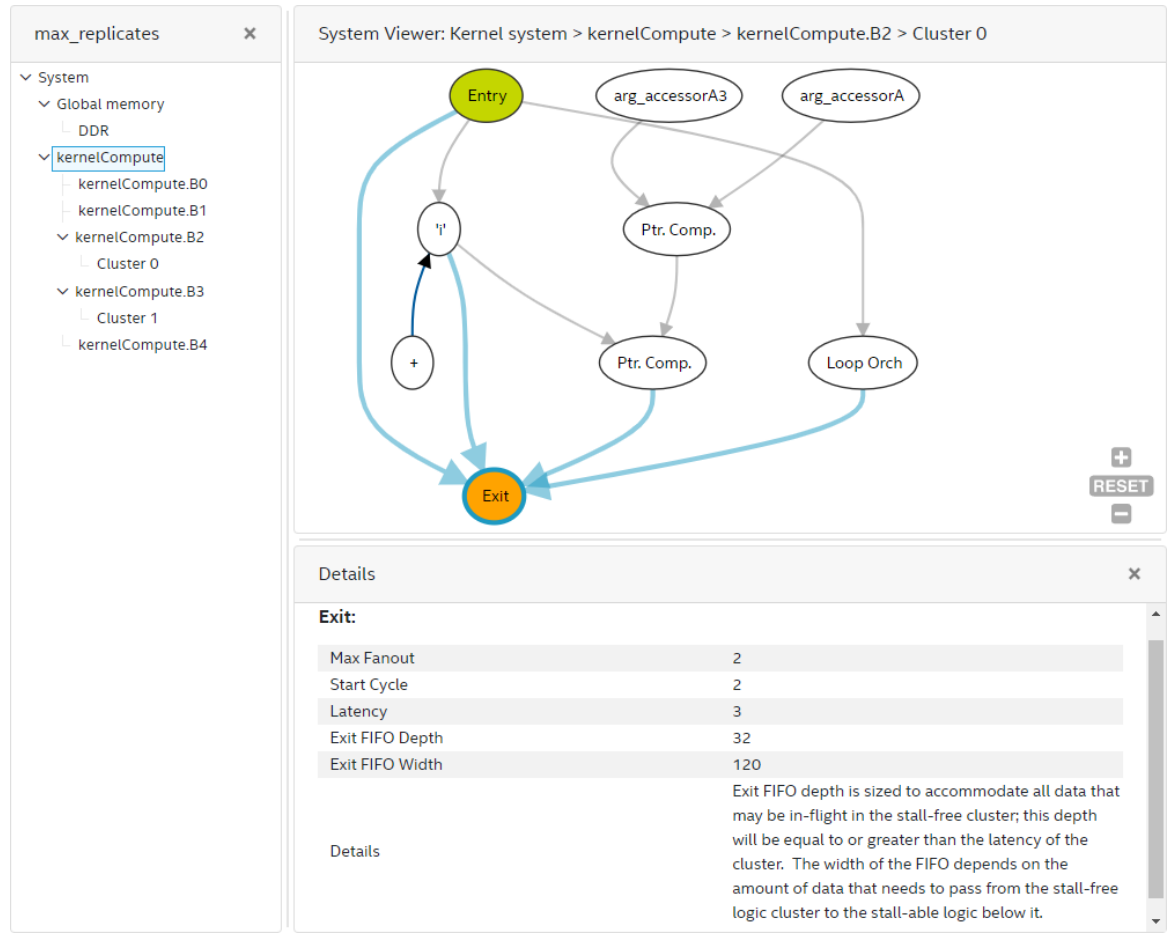
Cluster View

The cluster view of the System Viewer provides more granular graph views of the system or kernel. It helps in viewing clusters inside a block, and it shows all variables inside a cluster that have a loop-carried dependency. This view shows the following:

- Fine-grained details within clusters (including instructions and dependencies of the instructions) of the generated datapath of computations.
- Linking from the instruction back to the source line by clicking the instruction node.
- Various information about the instructions, such as data width, node's schedule information in start cycle, and latency, are provided if applicable.

A cluster starts with an entry node and ends with an exit node. The cluster exit node has a FIFO of depth greater than or equal to the latency of the cluster to store any data in-flight. You can find the size of the cluster exit FIFO by clicking on the exit node. The cluster exit FIFO size information is also available in the block view of the System Viewer when you click on the exit node.

Cluster View of the System Viewer



A cluster has a FIFO in its exit node to store any pipelined data in-flight. You can click on the cluster exit node to find the exit FIFO width and depth attribute. The cluster exit FIFO size is also available in the cluster view of the System Viewer.

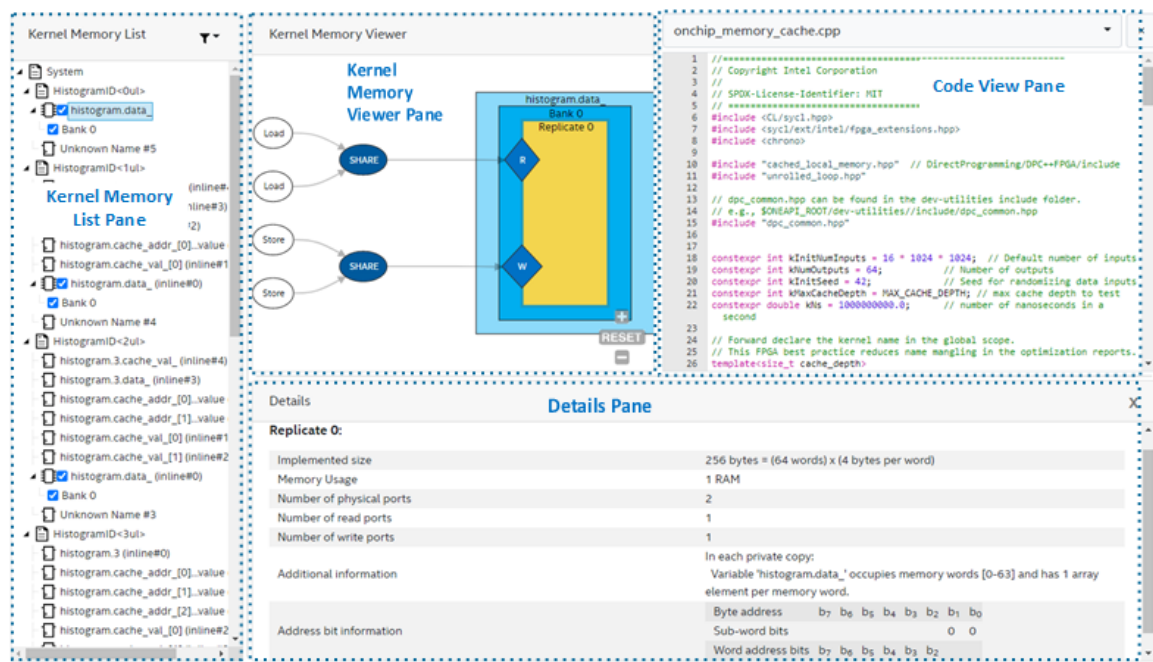
Besides computation nodes, when your design contains loops, you can see loop orchestration nodes and variable nodes along with their Feedback nodes. The compiler generates the loop orchestration logic for loops in your design. Loop orchestration nodes represents this logic in the cluster view of the System Viewer. A variable node corresponds to a variable that has a loop-carried dependency in your design. A variable node goes through various computation logic and finally feeds to a Feedback node that connects back to the variable node. This back edge means that the variable is passed to the next iteration after the new value is evaluated. Scan for loop-carried variables that have a long latency to the Feedback nodes as they can be the II bottlenecks. You can cross-check by referring to the Loop Analysis report for more information about the II bottleneck. The Feedback node has a FIFO to store any data in-flight for the loop and is sized to $d \cdot II$ where d is the dependency distance, and II is the initiation interval. You can find the cluster exit FIFO size by clicking on the feedback node and looking at the Details pane or the pop-up box.

The dependency distance is the number of iterations between successive load/store nodes that depend on each other.

Data movement is a bottleneck in many algorithms. The Kernel Memory Viewer shows you how the Intel® oneAPI DPC++/C++ Compiler interprets the data connections and synthesizes memory for your kernel. Use the Kernel Memory Viewer to help identify data movement bottlenecks in your kernel design.

To analyze your SYCL* system, click **Views** > **Kernel Memory Viewer**.

Kernel Memory Viewer Layout



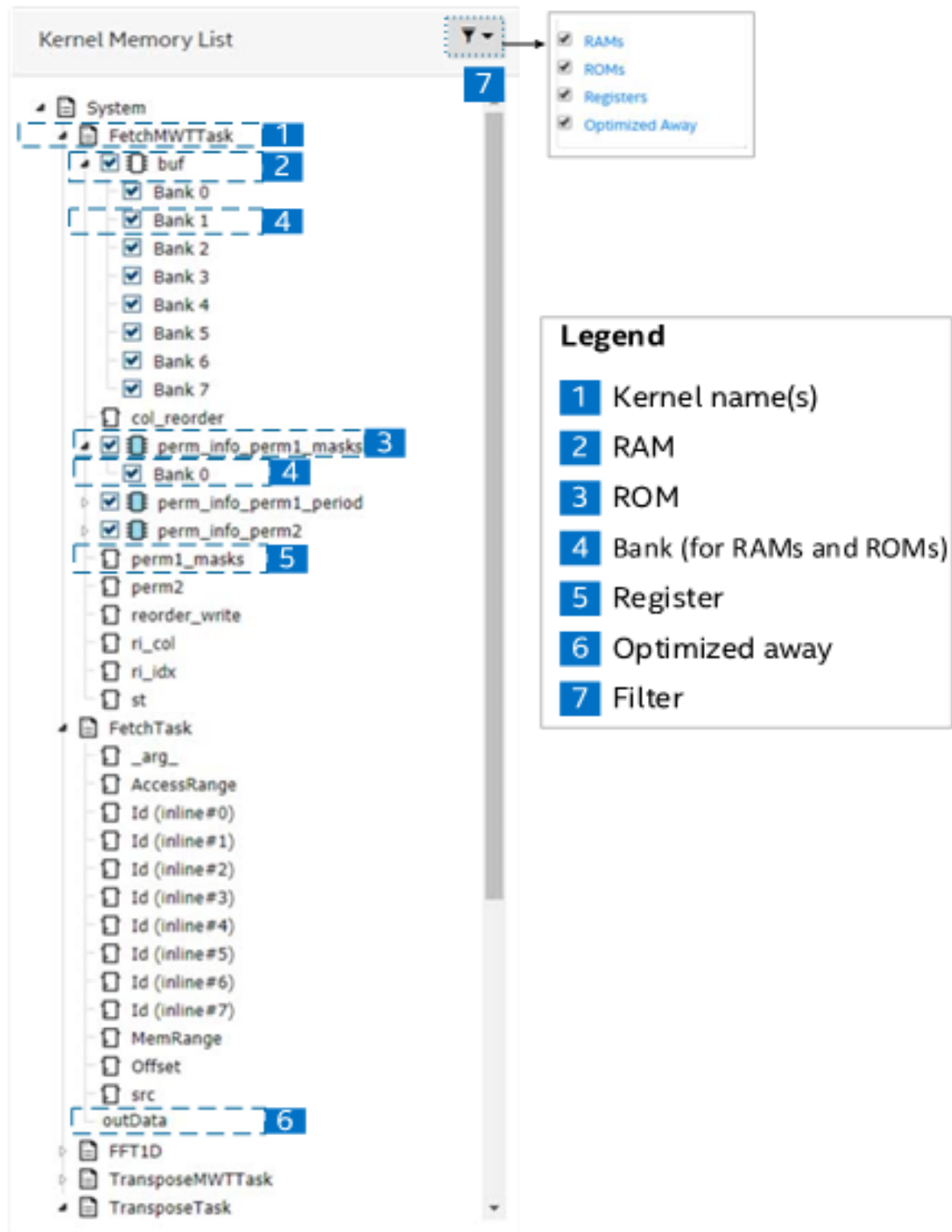
Kernel Memory Viewer Panes

Pane	Description
Kernel Memory List	Lists all memories present in your design. When you select a memory name, you can view its graphical representation in the Kernel Memory Viewer pane.
Kernel Memory Viewer	Shows a graphical representation of the memory system or memory bank selected in the Kernel Memory List pane.
Code View	Shows the source code file for which the reports are generated.
Details	Shows the details of the memory system or memory bank selected in the Kernel Memory List pane.

Kernel Memory List

The Kernel Memory List pane displays a kernel hierarchy with memories synthesized (RAMs, ROMs, and registers) and optimized away in that kernel.

Features and Details of the Kernel Memory List Pane



The following table describes each numbered feature highlighted in the above image:

Kernel Memory List Pane Icons and Labels

No.	Icon or Label	Name	Description
1		Kernel name	You can expand or collapse the list of memories in your kernel. Memories that do not belong to any kernel are displayed under (Other) .
2		RAM	A RAM is a memory that has at least one write to it. The name of the RAM memory is the same as its name in your design. When you select a memory name, you can view a logical representation of the RAM in the Kernel Memory Viewer pane. By default, only the first bank of the memory system is displayed. To select banks that you want the Kernel Memory Viewer pane to display: • Expand the memory name. • Clear the memory name check box to collapse all memory banks in the view. • Select the memory name check box to show all memory banks in the view.
3		ROM	A ROM is a read-only memory. The name of the ROM memory is same as its name in your design. When you select a memory name, you can view a logical representation of the ROM in the Kernel Memory Viewer pane. By default, only the first bank of the memory system is displayed. To select banks that you want the Kernel Memory Viewer pane to display: • Expand the memory name. • Clear the memory name check box to collapse all memory banks in the view. • Select the memory name check box to show all memory banks in the view.
4	Bank #num	Bank	A memory bank is always associated with a RAM or a ROM. Each bank is named Bank #num, where #num is the memory bank's ID starting from 0. • Click on the bank name to display the bank view in the Kernel Memory Viewer pane, which displays a graphical representation of the bank with all its replicates and private copies. This view can help you focus on specific memory banks of a complex memory design. • Clear the memory bank name check-box to collapse the bank in the logical representation of the memory. • Select the memory bank name check-box to display the bank in the logical representation of the memory.
5		Register	A register is a kernel variable carried through the pipeline in registers (rather than being stored in a RAM or ROM). The name of the register is the same as its name in your design.

No.	Icon or Label	Name	Description
			A register variable is implemented either exclusively in FFs or in a combination of FFs and RAM-based FIFOs.
6	Text label	Optimized Away	A kernel variable can be optimized away because it is unused in your design, or compiler optimizations have transformed all uses of the variable such that it is unnecessary. The name of the optimized away variable is the same as its name in your design.
7		Filter	Use the Kernel Memory List filter to selectively view the list of RAMs, ROMs, registers, and optimized away variables in your design. When you clear the checkbox associated with an item in the filter, you hide all occurrences of that kind of item in the Kernel Memory List. Filter your Memory List to help you focus on a specific type of memory in your design.

Kernel Memory Viewer

In the Kernel Memory Viewer pane, you can view connections between loads and stores to specific logical ports on the banks in a memory system. You can also view the number of replicates and private copies created per bank for your memory system. You can see the following types of nodes in the Kernel Memory Viewer pane, depending on the kernel memory system and your selection in the Kernel Memory List pane:

Node Types Observed in the Kernel Memory Viewer Pane

Node Type	Description
Memory node	The memory system for a given variable in your design.
Bank node	A bank in the memory system. A memory system contains at least one bank. A memory bank can connect to one or more port nodes.
Replication node	A replication node shows memory bank replicates created to support multiple accesses to a local memory efficiently. A bank contains at least one replicate. You can view replicate nodes when you view a memory bank by clicking its name in the Kernel Memory List pane.
Private-copy node	A private-copy node shows private copies within a replicate created to allow simultaneous execution of multiple loop iterations. A replicate contains at least one private copy. You can view private-copy nodes when you view a memory bank by clicking its name in the Kernel Memory List pane.
Port node	Each read or write access to local memory is mapped to a port. The logical port for a bank. There are three types of ports: • R: A read-only port • W: A write-only port • RW: A read and write port
LSU node	A store (ST) or load (LD) node connected to the memory through port nodes.
Arbitration node	An arbitration (ARB) node shows that LSUs compete for access to a shared port node, which can lead to stalls.

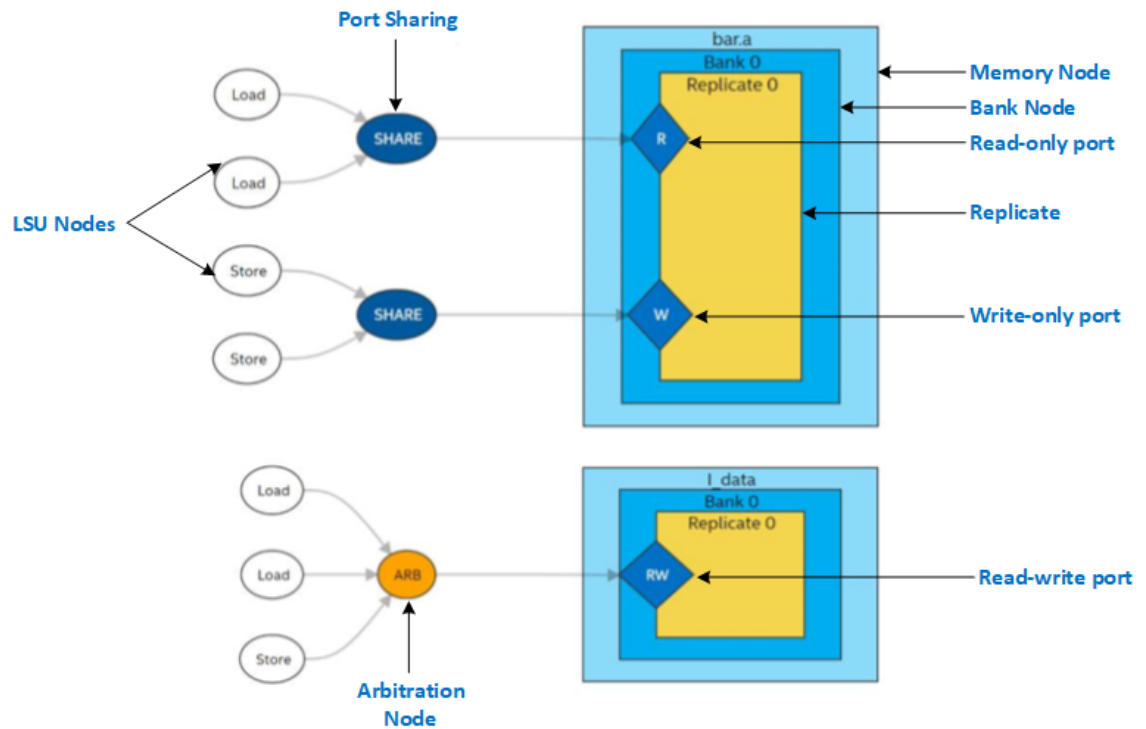
Node Type	Description
Port-sharing node	A port-sharing node (SHARE) shows that LSUs have mutually exclusive access to a shared port node, so the load-store units are free from stalls.

Within the graphical representation of a memory in the Kernel Memory Viewer pane, you can perform the following:

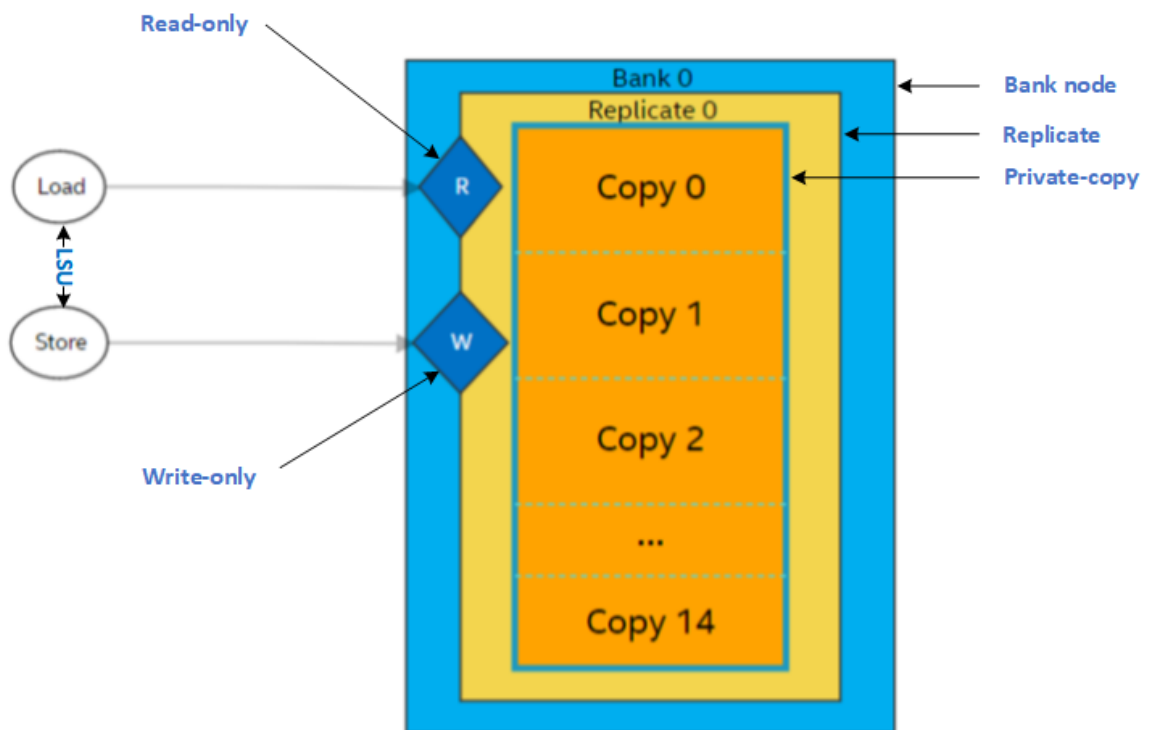
- Hover over any node to view the attributes of that node.
- Hover over an LSU node to highlight the path from the LSU node to all ports to which the LSU connects.
- Hover over a port node to highlight the path from the port node to all LSUs that read or write to the port node.
- Click a node to select it and display the node attributes in the Details pane.

The following images illustrate examples of what you see in the Kernel Memory Viewer:

Logical Representation of a Memory in the Kernel Memory Viewer Pane



Bank View of a Memory Bank in the Kernel Memory Viewer Pane



Code View

The Code View pane displays your source code. When you click on a memory or a bank in the Kernel Memory Viewer pane, the code view pane highlights the line of your code where you declared the memory.

Details

The Details pane shows the attributes of the node selected in the Kernel Memory Viewer pane. For example, when you select a memory in a kernel, the Details pane displays the following information:

- Width and depths of memory banks
- Memory layout
- Address-bit mapping
- Memory attributes that you specified in your source code

The content of the Details pane persists until you select a different node in the Kernel Memory Viewer pane.

Schedule Viewer

The Schedule Viewer displays a static view of the scheduled cycle and latency of a clustered group of instructions in your design. Use this report to view loop bottlenecks such as f_{MAX}/II bottlenecks, memory dependency, and occupancy limiter.

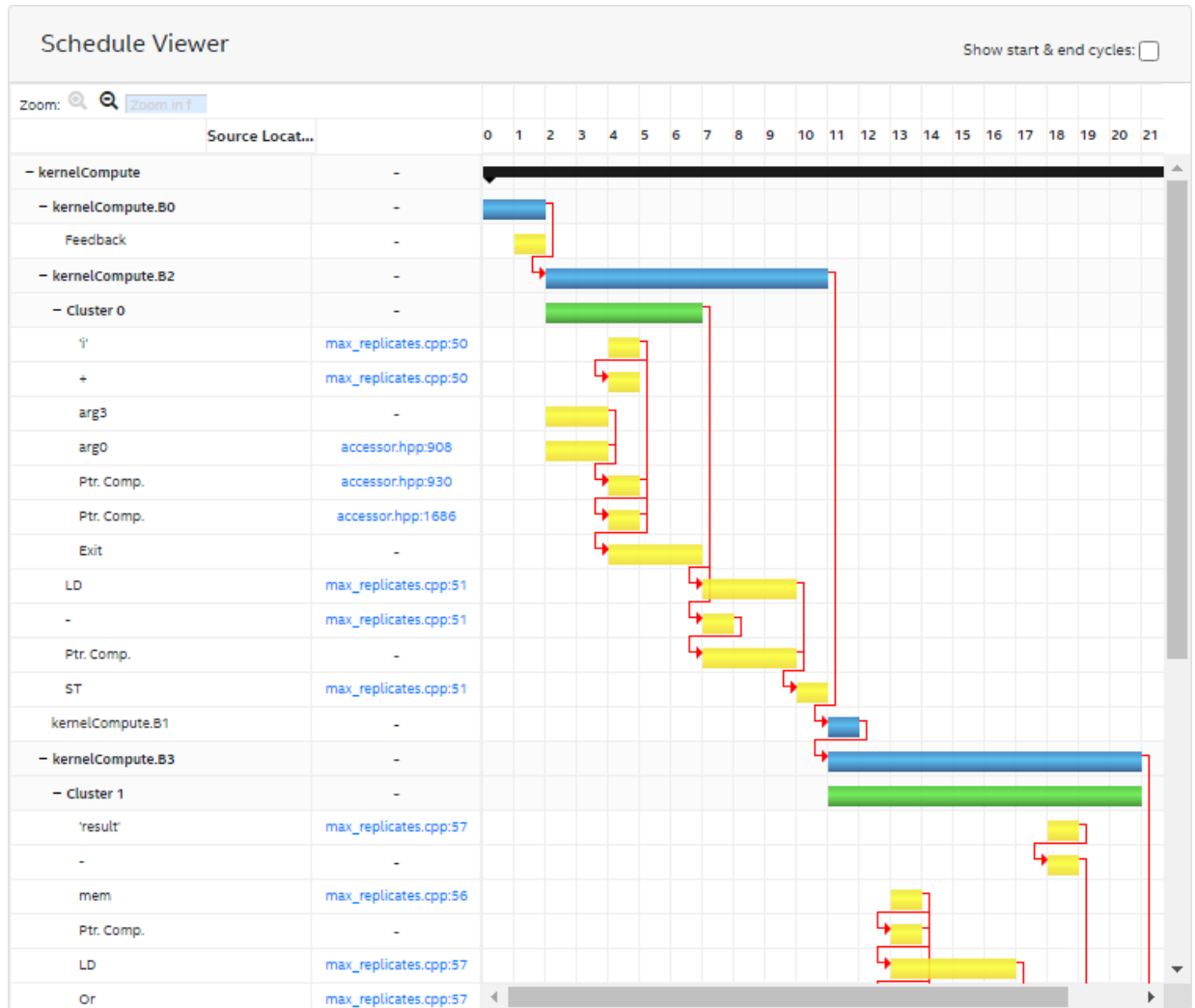
To view the Schedule Viewer, click **Views > Schedule Viewer**.

In the Schedule Viewer:

- Columns depict the clock cycles.
- Rows display a list of kernels, blocks, clusters, and instructions ranked by the order of execution.
- The red arrows are dependency lines for each block, cluster, or instruction. The arrows show how each block, cluster, or instruction is dependent on other blocks, clusters, or instructions. Hovering over a node (bar) highlights its outgoing dependency lines.
- Each row represents a node and its start and end cycle.

- The bars are color-coded. Black indicates a kernel, blue indicates a block, green indicates a cluster, and yellow indicates an instruction.

Schedule Viewer



Access FPGA Optimization Reports in JSON Format

In addition to the `report.html` file, SYCL* also provides the FPGA Optimization Report data in JSON files.

The JSON files containing the report data are available in the `<project_dir>/reports/resources/json` directory. The directory provides the following `.json` files:

JSON Files in the `<project_dir>/reports/resources/json` Directory

File	Description
<code>area.json</code>	Area Estimates
<code>block.json</code>	Block view of the System Viewer
<code>bottleneck.json</code>	Bottleneck view of the Loop Analysis report
<code>gmw.json</code>	Global memory view of the System Viewer

File	Description
info.json	Summary of project name, compilation command, versions, and timestamps
loops.json	Navigation tree of the Loop Analysis report
loops_attr.json	Loop Analysis
mav.json	System view of the System Viewer
new_lmv.json	Kernel Memory Viewer
pipeline.json	Cluster view of the System Viewer
quartus.json	Quartus® Prime compilation summary
schedule.json	Schedule Viewer
summary.json	Kernel compilation name mapping
tree.json	Navigation tree of the System Viewer
warnings.json	Compilation warning messages

Quartus (Static) Summary

After you generate the FPGA image, sections that summarize the compilation results are populated in the `report.html`. The following sections appear on the **Summary** page:

- **Clock Frequency Summary > Quartus Achieved Clock Frequency:** Shows the maximum clock frequencies that you can achieve for the design.
- **Quartus Fit Resource Utilization Summary:** Shows the total area utilization both for the entire design and for each kernel individually. There is no breakdown of area information by source line.

Compile the entire design in the same folder. The **Summary** includes everything, as shown in the following report:

Clock Frequency Summary			
	Quartus Achieved Clock Frequency (MHz)	Compile Target Frequency (MHz)	Compile Estimated Frequency (MHz)
Kernel clock	475.74	480.00	480.00

System Resource Utilization Summary							
Quartus Fitter Resource Utilization Summary							
Name	Source Location	ALM	ALUT	REG	MLAB	RAM	DSP
Quartus Fitter: Full Design (All Components)		81036.8	0	240457	1294	466	619
mvdr_beamforming_fpga_di		81036.8		240457	1294	466	619
Device		487200	974400	1948800	0	7110	4510

Compile Estimated Kernel Resource Utilization Summary							
Name	Source Location	ALM	ALUT	REG	MLAB	RAM	DSP
Global Interconnect		207	415	716	0	6	0
System description ROM		1	2	71	0	2	0
Pipe resources		93	187	5206	8	87	0
DataProducerID		1709	3418	10915	29	25	0
DiagReciprocal<Oul>		335	670	621	8	3	2.5
DataOutConsumerID		605	1210	3046	17	0	0
SinThetaProducerID		373	747	764	13	0	0
ForwardSubstitution<Oul>		9961	19923	49201	625	34	62
BackwardSubstitution<Oul>		11525	23051	57122	665	34	62
SteeringVectorGenerator<Oul>		2614	5229	3821	26	10	10
Transpose<Oul>		406	812	617	1	28	0
Compile Estimated: Kernel System		52165.5	104331	207568	1888	590	614

Timing Failures

If your FPGA compile fails to meet timing requirements, the Intel® oneAPI DPC++/C++ Compiler deletes the generated FPGA image, prints an error message, and returns an error code. This means that the generated FPGA image did not meet all timing constraints. The best solution is usually to recompile with a different seed (see `-Xsseed=<value>` in [Other SYCL* FPGA Flags Supported by the Compiler](#)). However, some rare designs

where the FPGA is extremely full might require sweeping several seeds to find one that passes the timing checks. If your design has chronic timing failures and you cannot resolve with seed sweeps, consult your BSP vendor.

When a timing failure happens, the compiler generates a `*.failing_clocks.rpt` file. The path to this file and file name are dependent on your BSP. This `.rpt` file lists which clocks in your design had failing paths and the magnitude of failures.

Intel® FPGA Dynamic Profiler for DPC++

Multiplatform Binary Kernels Only The Intel® FPGA Dynamic Profiler for DPC++ can be used only with multiarchitecture binary kernels. It does not work with RTL IP core kernels.

The Intel® FPGA dynamic profiler for DPC++ uses performance counters to collect kernel performance data during the design's execution. This data can be viewed using the [Intel® VTune™ Profiler](#).

Measure Kernel Performance

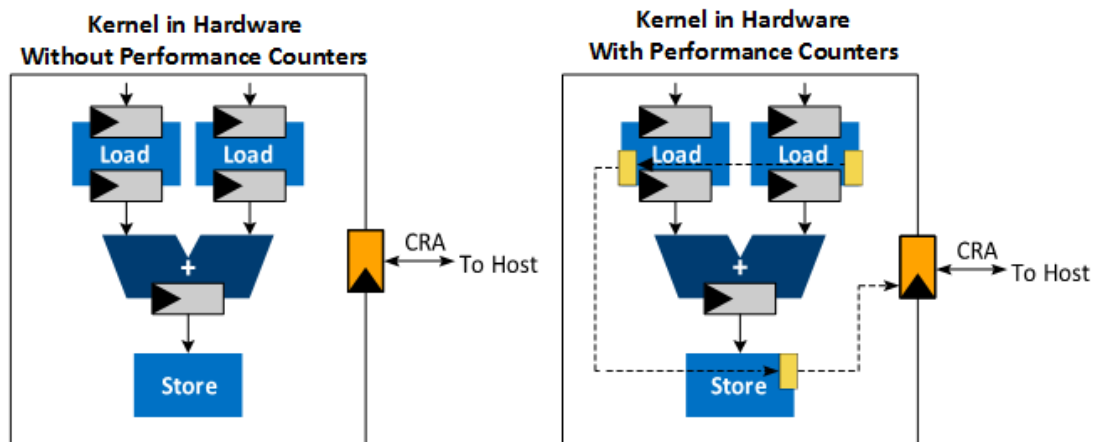
The Profiler instruments and connects performance counters in a daisy chain throughout the pipeline generated for the kernel program. The host then reads data collected by these counters. For example, in PCI Express® (PCIe®)-based systems, the host reads the Profiler data over the PCIe interface.

Consider the following SYCL example code:

```
// Vector Add Kernel
h.single_task<VectorAdd>([=] () {
    for (int i = 0; i < kSize; ++i) {
        r[i] = a[i] + b[i];
    }
});
```

The profiler instruments and connects performance counters in a daisy chain throughout the pipeline generated for the kernel as shown in [Figure 50: Intel® FPGA Dynamic Profiler for DPC++: Performance Counters Instrumentation](#). The host then reads the data collected by these counters. For example, in PCI Express® (PCIe®)-based systems, the host reads the data via the PCIe control register access (CRA) or control and status register (CSR) port.

Intel® FPGA Dynamic Profiler for DPC++: Performance Counters Instrumentation



Applications that use many pipes or memory accesses might stall frequently to enable the completion of memory transfers. The dynamic profiler collects various performance metrics such as stall, occupancy, idle, and bandwidth data at these points in the pipeline to help identify memory or pipe operations that create stalls.

Instrument the Kernel Pipeline with Performance Counters (-Xsprofile)

Enable profiling during design compilation to add profiling counters to the SYCL kernel pipeline. To instrument the SYCL code with performance counters, add the `-Xsprofile` flag to your compiler command.

NOTE

- Instrumenting the design with performance counters increases hardware resource utilization (that is, increases FPGA area use) and typically decreases performance.
- Ensure that all kernel names are unique so that the dynamic profiler interprets the results correctly.

Caution

In large designs, the overhead from profiling can cause f_{MAX} degradation. It may also prevent the design from fitting on the chip due to the area overhead of the profiling counters.

Obtain Profiling Data During Run Time

You can obtain profiling data during run time in one of the following ways:

- Use the Profiler Runtime Wrapper from the command line to obtain the data. This data can later be imported into the Intel® VTune™ Profiler as described in [Import FPGA Data collected with Profiler Runtime Wrapper](#). For more information about the Profiler Runtime Wrapper, refer to [Invoke the Profiler Runtime Wrapper to Obtain Profiling Data](#).
- Run your host application in Intel® VTune™ Profiler using the **CPU/FPGA Interaction** view. For more information about how to configure and run your host application, refer to [CPU/FPGA Interaction Analysis](#) and [Use Intel® VTune™ Profiler](#).

Invoke the Profiler Runtime Wrapper to Obtain Profiling Data

After compiling your SYCL* program using the Intel® oneAPI DPC++/C++ Compiler, you can profile your FPGA design using the Profiler Runtime Wrapper. The Profiler Runtime Wrapper calls your executable and collects profile information at a given sample rate. The performance counter data is saved in a `profile.mon` monitor description file that the Profiler Runtime Wrapper post-processes and outputs into a readable `profile.json` file. You are encouraged to use the `profile.json` for further data processing instead of the `profile.mon` file. However, both are available for use after host execution completes.

To invoke the Profiler Runtime Wrapper, execute the following command:

```
aocl profile [options] /path/to/executable [executable options]
```

where:

- `[options]` are any additional flags you want to pass to the wrapper. Refer to `aocl profile -help` for a list of options and their uses.
- `/path/to/executable` is the path to the executable generated by the compiler.
- `[executable options]` are any options or arguments that need to be passed along to the executable.

Caution

Because of slow network disk accesses, running the host application from a networked directory might introduce delays between kernel executions. These delays might increase the overall execution time of the host application. In addition, they might introduce delays during kernel executions while the runtime stores profile output data to disk.

Split the Execution and Data Post-Processing

By default, the Profiler Runtime Wrapper automatically runs a post-processing step on your `profile.mon` monitor file to produce a readable `profile.json` file. In some situations, the post-processing step may take longer than expected. Because of this, you can choose to separate the execution and data post-processing steps into two separate manual steps. To do this, use the `--no-json` and `--no-run <path to profile.mon file>` Profiler Runtime Wrapper options.

- The `--no-json` flag only runs your executable and produces a `profile.mon` monitor file without post-processing it.
- The `--no-run <path to profile.mon file>` flag does not invoke your executable and instead just calls the post-processing step on the supplied `profile.mon` file.

Temporal Performance Collection

During the run of your host application, the Profiler collects performance counter data at a given sample rate `n`. After `n` cycles, the Profiler collects the performance counter data and outputs it to the `profile.mon` monitor file.

- You can control the rate at which the Profiler counters are sampled by setting the Profiler Runtime Wrapper's `-period` flag. The specified period is the minimum number of kernel pipeline clock cycles between profiling samples. If you do not set a period, the default behavior is to profile as often as possible.

Caution

For particularly large or long-running designs, the amount of data generated by the default temporal period might result in very large `profile.mon` and `profile.json` files. To reduce this file size, increase the sampling period or turn off temporal profiling.

- To turn off temporal profiling and instead collect performance data only once a kernel has finished executing, you can set the Profiler Runtime Wrapper's `-no-temporal` flag.

NOTE

If you collect the performance data only at the end of execution, the data is an average representation of the kernel's overall execution.

Use Intel® VTune™ Profiler

To view performance data, you can upload your `profile.json` file to the **CPU/FPGA Interaction View** in the Intel® VTune™ Profiler. For more information about how to upload the file and open the correct views, refer to [CPU/FPGA Interaction Analysis \(Preview\)](#) in the *Intel® VTune™ Profiler User Guide*.

You can use the **CPU/FPGA Interaction View** in the Intel® VTune™ Profiler to determine performance information about your design in various graphical representations. You can view the following:

- Summarized or average data about your SYCL* kernels.
- A graphical representation of the overall kernel program execution process, including both host and device side events.

- Detailed statistics about memory and pipe accesses in both a source view format and timeline format.

The following tables describe types of performance data and information available in the **CPU/FPGA Interaction View** in the Intel® VTune™ Profiler:

Types of Performance Data

Column	Description	Access Type
Attributes	Memory or pipe attributes information such as memory type (local or global), corresponding memory system (DDR or quad data rate (QDR)), and read or write access.	All memory and pipe accesses
Stall%	Percentage of time the memory or pipe access is causing pipeline stalls. It is a measure of the ability of the memory or pipe access to fulfill an access request.	All memory and pipe accesses
Occupancy%	Percentage of the overall profiled period when a valid work-item executes the memory or pipe instruction.	All memory and pipe accesses
Bandwidth	Average memory bandwidth that the memory access uses and its overall efficiency. For each global memory access, FPGA resources are assigned to acquire data from the global memory system. However, the amount of data a kernel program uses might be less than the acquired data. The overall efficiency is the percentage of total bytes acquired from the global memory system that the kernel program uses.	Global memory accesses

See Also

[Analyzing CPU and FPGA Arria® 10 GX Interaction](#)

[Profiling an FPGA-driven SYCL* Application](#)

Interpret Performance Counter Data

Profiling information helps in identifying poor memory or pipe behaviors that lead to unsatisfactory kernel performance.

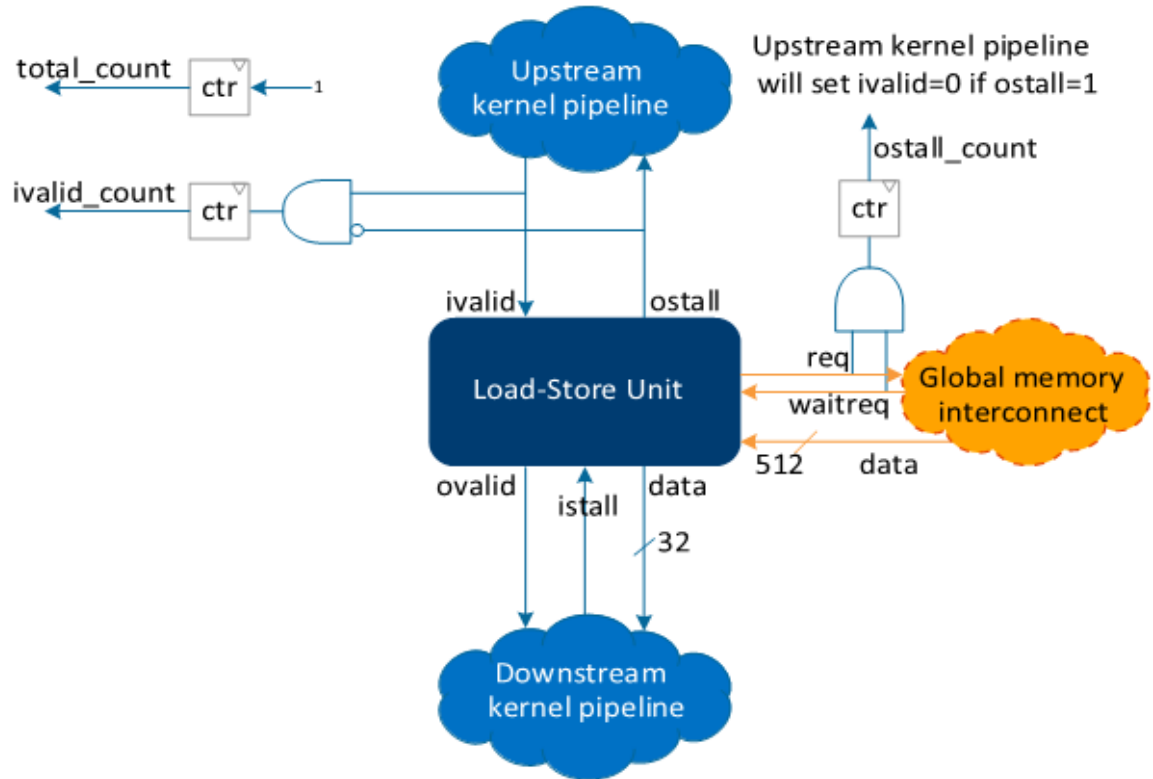
The following sections explain the Profiler metrics that the Profiler reports record.

Stall, Occupancy, Bandwidth

The **CPU/FPGA Interaction View** shows stall percentage, occupancy percentage, and average memory bandwidth for specific lines of kernel code. For definitions of stall, occupancy, data transfer size, and bandwidth, refer to [Table 17: Types of Performance Data](#).

SYCL* generates a pipeline architecture where work-items traverse through the pipeline stages sequentially. For more information, refer to [Pipelining](#).

Simplified Representation of a Kernel Pipeline Instrumented with Performance Counters



The following are simplified equations that describe how the Profiler calculates stall, occupancy, and bandwidth:

$$\text{Stall} = \frac{\text{ostall_count}}{\text{total_count}} \times 100\%$$

$$\text{Occupancy} = \frac{\text{ivalid_count}}{\text{total_count}} \times 100\%$$

$$\text{Bandwidth} = \frac{\text{data_width} \times \text{ivalid_count}}{\text{kernel_time}}$$

NOTE

`ivalid_count` in the bandwidth equation also includes the `predicate=true` input to the load-store unit.

Ideal kernel pipeline conditions:

- Stall percentage equals 0%
- Occupancy percentage equals 100%
- Bandwidth equals the board's bandwidth

For a given location in the kernel pipeline, if the sum of the stall percentage and the occupancy percentage approximately equals 100%, the Profiler identifies the location as the stall source. If the stall percentage is low, the Profiler identifies the location as the victim of the stall.

The Profiler reports a high occupancy percentage if the compiler generates a highly efficient pipeline from your kernel, where work-items or iterations are moving through the pipeline stages without stalling. The following notes may be helpful when interpreting profiling data:

- If all LSUs are accessed the same number of times, they have the same occupancy value.
- If work-items cannot enter the pipeline consecutively, bubbles are inserted into the pipeline.
- In loop pipelining, loop-carried dependencies also form bubbles in the pipeline because of bubbles that exist between iterations. For more information about bubbles, refer to Occupancy.
- If an LSU is accessed less frequently than other LSUs, such as when an LSU is outside a loop that contains other LSUs, this LSU has a lower occupancy value than the other LSUs. This rule also applies to pipes.

Stalling Pipes

Pipes provide a point-to-point communication link between two kernels.

Stalls occur if there is an imbalance between the read and write sides of the pipe or if the read and write kernels are not running concurrently.

For example, if the kernel that reads is not launched concurrently with the kernel that writes, or if the read operations occur much slower than the write operations, the Profiler identifies a stall for the

`ProducerToConsumerPipe::write` call in the write kernel.

Reduce Area Resource Use While Profiling

Due to various performance counters being added to the pipeline, introducing profiling into your design can result in a large amount of area resource use. This may be inconvenient for particularly large designs as adding profiling performance counters might result in *no fit* errors.

To reduce the amount of area resources that profiling takes up, you can choose to profile with shared performance counters. This profiling mode allows counters to be shared by various signals over multiple design runs to reduce the number of performance counters added to the design. During run time, the [Profiler Runtime Wrapper](#) runs the host application four times, where, for each run, the counters count a different signal.

NOTE

You must invoke the [Profiler Runtime Wrapper](#) only once.

To turn on the shared performance counters profiling mode, perform these steps:

1. Include the `-Xsprofile-shared-counters` flag along with the `-Xsprofile` flag during your compile.
2. Include the `-shared-counters` flag when running your design with the Profiler Runtime Wrapper.

The `-shared-counters` flag tells the profiler to run your design multiple times to collect the profiling data from everywhere requested. If you do not include the `-shared-counters` flag, your design is run only once, which provides a limited set of profiler data (no data for everything after the first shared signal).

Caution

The shared performance counters profiling mode works well only for kernels and designs that are deterministic. Because the host application and design are run multiple times to collect all of the data, non-deterministic designs result in shared data that is difficult to combine, and it may be difficult to determine where design problems occur temporally.

Profiler Analyses of Example SYCL* Design Scenarios

Understanding the problems and solutions presented in example SYCL design scenarios might help you leverage the Profiler metrics of your design to optimize its performance.

High Stall Percentage

A high stall percentage implies that the memory or pipe instruction cannot fulfill the access request because of contention for memory bandwidth or pipe buffer space.

Memory instructions stall often whenever bandwidth usage is inefficient or if a large amount of data transfer is necessary during the execution of your application. Inefficient memory accesses lead to suboptimal bandwidth utilization. In such cases, analyze your kernel memory access for possible improvements.

Pipe instructions stall whenever there is a strong imbalance between read and write accesses to the pipe. Imbalances might be caused by pipe reads or writes operating at different rates.

For example, if you find that the stall percentage of a write pipe call is high, check to see if the occupancy and activity of the read pipe call are low. If they are, the kernel's speed controlling the read pipe call is too slow compared to the kernel controlling the write pipe call, leading to a performance bottleneck.

Low Occupancy Percentage

A low occupancy percentage implies that a work-item is accessing the load and store operations or the pipe infrequently. This behavior is expected for load and store operations or pipes that are in non-critical loops. However, if the memory or pipe instruction is in critical portions of the kernel code and the occupancy or activity percentage is low, it implies that a performance bottleneck exists because work-items or loop iterations are not being issued in the hardware.

Consider the following code example:

```
queue_event = deviceQueue.submit([&](handler &cgh) {
    cgh.single_task<class LowOccupancyPercentageEx>([=]() {
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < 1000; j++){
                Pipe1::write(data1);
            }
            for (int k = 0; k < 3; k++) {
                Pipe2::write(data2);
            }
        }
    });
});
```

Assuming all loops are pipelined, the first inner loop with a trip count of 1000 is the critical loop. The second inner loop with a trip count of three is executed infrequently. As a result, you can expect that the occupancy and activity percentages for `Pipe1` are high and for `Pipe2` are low.

In addition, occupancy percentage might be low if you define a small work-group size, the kernel might not receive sufficient work-items. This is problematic because the pipeline is generally empty for the duration of kernel execution, which leads to poor performance.

High Stall and High Occupancy Percentages

A load and store operation or pipe with a high stall percentage is the cause of the kernel pipeline stall.

NOTE

An ideal kernel pipeline condition has a stall percentage of 0% and an occupancy percentage of 100%.

Usually, the sum of stall and occupancy percentages approximately equals 100%. If a load and store operation or pipe has a high stall percentage, it means that the load and store operation or pipe has the ability to execute every cycle but is generating stalls.

Solutions for stalling global load and store operations:

- Use local memory to cache data.
- Reduce the number of times you read the data.
- Improve global memory accesses:
 - Change the access pattern for more global-memory-friendly addressing (for example, change from stride accessing to sequential accessing).
 - Compile your kernel with the `-Xsno-interleaving=default` compiler command option and separate the read and write buffers into different DDR banks.

- Have fewer but wider global memory accesses.
- Acquire an accelerator board with more bandwidth (for example, a board with three DDRs instead of two DDRs).

Solution for stalling local load and store operations:

- Review the Memory Viewer report to verify the local memory configuration and modify the configuration to make it stall-free.

Solutions for stalling pipes:

- Fix stalls on the other side of the pipe. For example, if a pipe read stalls, it means that the writer to the pipe is not writing data into the pipe fast enough, and you must adjust it.
- If there are pipe loops in your design, specify the pipe depth.

No Stalls, Low Occupancy Percentage, and Low Bandwidth

Loop-carried dependencies might create a bottleneck in your design that causes an LSU to have a low occupancy percentage and a low bandwidth.

NOTE

An ideal kernel pipeline has a bandwidth that equals the board's available bandwidth.

Example SYCL Kernel and Profiler Analysis

Source		Device Metrics			
		Stalls (%)	Occupancy (%)	Data Transfer Size	Average Bandwidth, GB/s
44	template<int unroll1, int unroll2> class ND;				
45	template<int UNROLL_FACTOR1, int UNROLL_FACTOR2>				
46	void nd(const std::vector<float>& VA, int n) {				
47	event queue_event;				
48	cl::sycl::queue deviceQueue = cl::sycl::queue(fpga_selector{});				
49	buffer<float, 1> bufferA(VA.data(), n);				
50	queue_event = deviceQueue.submit([&{handler& cgh) {				
51	auto dst = bufferA.get_access<sycl::write>(cgh);				
52	auto n_items = n;				
53	cgh.single_task<class ND<UNROLL_FACTOR1, UNROLL_FACTOR2> >{				
54	{=}{() {				
55	for (int i = 0; i < n_items; i++){				
56	int res = i;				
57	for(int k = 0; k < 20; k += UNROLL_FACTOR1){				
58	unroller<0, UNROLL_FACTOR1>::step([&](int i) {				
59	res += 1;				
60	res ^= i;				
61	});				
62	}				
63	for(int k = 0; k < 20; k += UNROLL_FACTOR2){				
64	unroller<0, UNROLL_FACTOR2>::step([&](int i) {				
65	res += 1;				
66	res ^= -i;				
67	});				
68	}				
69	dst[i] = res;				
70	}				
71	});				
72	});				
73	}				
74	}				
75	}				
76	}				
77	}				
78	}				
79	}				

UNROLL_FACTOR1 = 5
> Has 4 Iterations

UNROLL_FACTOR2 = 1
> Has 20 Iterations

0.0% 3.8% 3 KB 0.028

In this example, `dst[i]` is executed once every 20 iterations of the `FACTOR2` loop and once every four iterations of the `FACTOR1` loop. Therefore, `FACTOR2` loop is the source of the bottleneck.

Solutions for resolving loop bottlenecks:

- Unroll the `FACTOR1` and `FACTOR2` loops evenly. Simply unrolling `FACTOR1` loop further does not resolve the bottleneck.
- Vectorize your kernel to allow multiple work-items to execute during each loop iteration.

No Stalls, High Occupancy Percentage, and Low Bandwidth

The structure of a kernel design might prevent it from leveraging all the available bandwidth that the accelerator board can offer.

Example SYCL Kernel and Profiler Analysis

Source		Device Metrics			
		Stalls (%)	Occupancy (%)	Data Transferred	
				Data Transfer Size	Average Bandwidth, GB/s
43	template <typename T, size_t N>				
44	void simple_vadd(const std::array<T, N>& VA, const std::array<T, N>& VB,				
45	std::array<T, N>& VC) {				
46	cl::sycl::queue deviceQueue = cl::sycl::queue(fpga_selector());				
47	cl::sycl::device device = deviceQueue.get_device();				
48	std::cout << "Device name: " << device.get_info<cl::sycl::info::device::name>().get_value();				
49					
50	cl::sycl::range<1> numItems(N);				
51	cl::sycl::buffer<T, 1> bufferA(VA.data(), numItems);				
52	cl::sycl::buffer<T, 1> bufferB(VB.data(), numItems);				
53	cl::sycl::buffer<T, 1> bufferC(VC.data(), numItems);				
54					
55	deviceQueue.submit([&cl::sycl::handler& cgh) {				
56	auto accessorA = bufferA.template get_access<sycl_read>(cgh);				
57	auto accessorB = bufferB.template get_access<sycl_read>(cgh);				
58	auto accessorC = bufferC.template get_access<sycl_write>(cgh);				
59					
60	cgh.parallel_for<class SimpleVaddT>(numItems,				
61	[&](cl::sycl::id<1> wiID) {				
62	accessorC[wiID] = accessorA[wiID] + accessorB[wiID];	0.5%	0.6%	482 KB	3.528
63	});				
64	});				
65	}				

In this example, the accelerator board can provide a bandwidth of 25.6 gigabytes per second (GB/s). However, the `vector_add` kernel is requesting (2 reads + 1 write) x 4 bytes x 294 MHz = 12 bytes/cycle x 294 MHz = 3.528 GB/s, which is 14% of the available bandwidth. To increase the bandwidth, increase the number of tasks performed in each clock cycle.

Solutions for low bandwidth:

- Automatically or manually, vectorize the kernel to make wider requests.
- Unroll the innermost loop to make more requests per clock cycle.
- Delegate some of the tasks to another kernel.

High Stall and Low Occupancy Percentages

There might be situations where a global store operation might have a high stall percentage (for example, 30%) and very low occupancy percentage (for example, 0.01%). If such a store operation happens once every 10000 cycles of computation, the efficiency of this store is likely not a cause for concern.

Limitations

The Intel® FPGA Dynamic Profiler for DPC++ has some limitations:

- Profile data is not persistent across SYCL* programs or multiple devices.
- The Profiler is unique to each SYCL program and device, meaning, each program on each device has its own profiler information. If your host swaps a new kernel program in and out of the FPGA, the Profiler does not save any data.

- Profile data is not saved on a device for later profiling.
- All profiling data is read to the host during execution and is only stored on the device long enough to be read on the next readback. Any reprogramming of new designs or restarting the same design results in new profiling data, erasing any previous data that may have existed.
- Instrumenting the design with performance counters increases hardware resource utilization (that is, FPGA area use) and typically decreases performance.

Optimizing Your Kernel

Debugging and testing your kernel as well as reviewing the FPGA Optimization Report can provide areas where you can try to improve the performance of your kernel. In general, the methods you use to improve the performance of your kernels should achieve the following results:

- Increase the number of parallel operations.
- Increase the memory bandwidth of the implementation.
- Increase the number of operations per clock cycle that kernels can perform in hardware.

Areas of optimization are covered in separate chapters as follows:

- [RTL IP Core Kernel Interfaces](#) (RTL IP components only)

Your RTL IP core can have a variety of interfaces: from basic wires to streaming and memory-mapped host interfaces.

Each interface type has different benefits. However, the system that surrounds your component might limit your choices. Keep your requirements in mind when determining the optimal interface for your component.

- [Loops](#)

Review the techniques in this chapter to optimize your loops to boost the performance of your component. Try to eliminate any dependencies in your loops that prevent the compiler from optimizing them.

You can also provide explicit guidance to the compiler for optimizing loops by using the available loop pragmas and loop attributes.

- [Pipes](#)

Using global memory to communicate data between your kernels can constrain the performance of your design. Pipes provide a mechanism for passing data between kernels and synchronizing kernels with high efficiency and low latency. Pipes allow kernels to use on-device FIFO buffers to communicate directly with each other.

Host pipes enable a similar communication method between your kernel and host.

- [Data Types and Arithmetic Operations](#)

The data types in your kernel and possible conversions or casting that they might undergo can significantly affect the performance and FPGA area usage of your kernel.

After you optimize the algorithm bottlenecks of your design, you can fine-tune some data types in your component by using arbitrary precision data types to shrink data widths, which reduces FPGA area utilization.

Because C++ automatically promotes smaller data types such as `short` or `char` to 32 bits for operations such as addition or bit-shifting, you must use the arbitrary precision data types if you want to create narrow data paths in your kernel.

- [Parallelism](#)

The Intel® oneAPI DPC++/C++ Compiler provides the following forms of parallelism for your kernel:

- *NDRange kernels* tell your kernel to operate in parallel instances over a work-item index-space. This mode of operation is useful if your kernel describes multiple concurrent threads operating in a data-parallel manner.
- The `task_sequence` class provides a way to define operations that you want to run asynchronously from the main flow of your kernel. This class is helpful when you want to express coarse-grained thread-level parallelism in your kernel.

- [Memories and Memory Operations](#)

The Intel® oneAPI DPC++/C++ Compiler infers efficient memory architectures (like memory width, number of banks and ports) in a kernel by adapting the architecture to the memory access patterns of your kernel. Review this section to learn how you can get the best memory architecture for your component from the compiler. In most cases, you can optimize the memory architecture by modifying the access pattern. However, the compiler also gives you some explicit control over the memory architecture.

- [Libraries](#)

With libraries, you can reuse functions without knowing the underlying hardware design or implementation details. Libraries let you take advantage of optimized designs developed by others. You can also develop your own libraries for reuse or sharing with others.

Optimizing Your Host Application

9

Multiplatform Binaries Only The information presented here applies only to the host application part of multiarchitecture binaries. It does not apply to the testbench application for RTL IP core kernels.

This section shows ways you can write your host to improve your application's throughput.

Throughput

For most designs, you can achieve high performance by optimizing the throughput of the hardware for a given algorithm. This section describes techniques to improve the design's throughput.

Resource Use

You may want to improve throughput first. However, you must also consider the use of resources (ALMs, DSPs, and so on) in your design when targeting FPGAs, for the following reasons:

- To obtain a design to fit on an FPGA image.
- To be able to use multiple copies of the design to increase throughput.
- To make room for other functionality to be implemented in the same FPGA image.

System-level Profiling Using the Intercept Layer for OpenCL™ Applications

The Intercept Layer for OpenCL™ Applications is an open-source tool that you can use to profile oneAPI designs at a system-level. Although it is not part of the Intel® oneAPI Base Toolkit installation, it is freely available on GitHub*.

This tool serves the following purpose:

- Intercept host calls before they reach the device to gather performance data and log host calls.
- Provide data to visualize the calls through time and separate them into queued, submitted, and execution sections to better understand the execution.
- Identify gaps (using visualization) in the runtime that may be leading to inefficient execution and throughput drops.

NOTE

The Intercept Layer for OpenCL™ Applications tool has a different purpose than the Intel® FPGA Dynamic Profiler for DPC++, which provides information about the kernels themselves and helps optimize the hardware. Together, you can use these tools to optimize both host and device-side execution.

The Intercept Layer has different options for capturing different aspects of the host run, and these options are described in its documentation. Call-logging and device timeline features are used to print information about the calls made by the host during execution.

You can view visualizations of this data in the following methods:

- Use JSON files generated by the Intercept Layer for OpenCL Applications that contain device timeline information. You can open these JSON files in the Google* Chrome trace event profiling tool, which provides a visualization of the data.
- Use the Intercept Layer for OpenCL Applications python script that parses the timeline information into a Microsoft* Excel file, where it is presented both in a table format and in a bar graph.

Use the visualized data to identify gaps in the runtime where events are waiting for something else to finish executing. While it is not possible to eliminate all the gaps, you might be able to eliminate gaps caused by dependencies that can be avoided.

For example, [double-buffering](#) allows host-data processing and host transfers to the device-side buffer to occur in parallel with the kernel execution on the FPGA device. This parallelization is useful when the host performs any combination of the following actions between consecutive kernel runs:

- Preprocessing
- Postprocessing
- Writes to the device buffer

By running host and device actions in parallel, execution gaps between kernels are removed as they no longer have to wait for the host to finish its operation. You can clearly see the benefits of double-buffering with the visualizations provided by the Intercept Layer output.

NOTE

For additional information, refer to the FPGA tutorial sample "Using the Intercept Layer for OpenCL™ Applications to Identify Optimization Opportunities" at [GitHub](#).

Set Up the Intercept Layer for OpenCL™ Applications

The Intercept Layer for OpenCL™ Applications is available on GitHub* at <https://github.com/intel/opencl-intercept-layer>

To set up the Intercept Layer for OpenCL Applications, perform the following steps:

1. Download Intercept Layer for OpenCL Applications version 2.2.1 or later from GitHub* at the following URL:
<https://github.com/intel/opencl-intercept-layer>
2. Build the Intercept Layer according to the instructions provided in [How to Build the Intercept Layer for OpenCL™ Applications](#).
3. Ensure that you have set `ENABLE_CLILOADER=1` when running `cmake` command. For example, run `cmake -DENABLE_CLILOADER=1 ...`
4. Run the `make` command in the build directory. This step builds the `cliloader` loader utility.

The `cliloader` executable should now exist in the `<path to opencl-intercept-layer-master download>/<build dir>/cliloader/` directory.

5. Add the directory to your `PATH` environment variable if you want to run multiple designs using `cliloader`.

You can now pass your executables to `cliloader` to run them with the intercept layer. For details about the `cliloader` loader utility, see [cliloader: A Intercept Layer for OpenCL™ Applications Loader](#).

6. Set `cliloader` and other Intercept Layer options.

If you run multiple designs with the same options, set up a `clintercept.conf` file in your home directory. You can also set the options as environment variables by prefixing the option name with `CLI_`. For example, the `DllName` option can be set through the `CLI_DllName` environment variable. For a list of options, see *Controls* in [How to Use the Intercept Layer for OpenCL Applications](#).

Option/Variable	Description
<code>DllName=\$CMAKE_ROOT/linux/lib/libOpenCL.so</code>	The intercept layer must know where <code>libOpenCL.so</code> file from the original oneAPI build is.

Option/Variable	Description
DevicePerformanceTiming=1 and DevicePerformanceTimelineLogging=1	These options print out runtime timeline information in the output of the executable run.
ChromePerformanceTiming=1, ChromeCallLogging=1, ChromePerformanceTimingInStages=1	These variables set up the chrome tracer output and ensure the output has Queued, Submitted, and Execution stages.

These instructions set up the `cliloader` executable, which provides some flexibility by allowing for more control over when the layer is used or not used. If you prefer a local installation (for a single design) or a global installation (always ON for all designs), follow the instructions at [How to Install the Intercept Layer for OpenCL Applications](#).

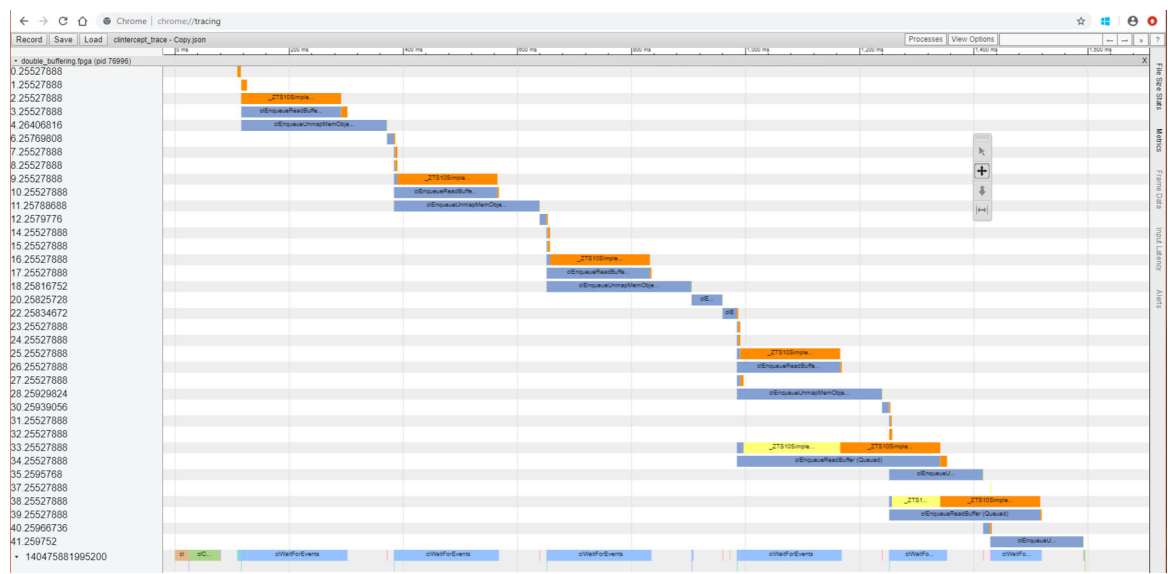
When you run the host executable with `cliloader <executable> [executable args]` command, the `stderr` output contains lines as shown in the following example:

```
Device Timeline for clEnqueueWriteBuffer (enqueue 1) = 63267241140401 ns (queued),
63267241149579 ns (submit), 63267241194205 ns (start), 63267242905519 ns (end)
```

These lines give the timeline information about a variety of oneAPI runtime calls. After the host executable finishes running, there is also a summary of the performance information for the run. After the executable runs, the data collected is placed in the `CLIntercept_Dump` directory, which is in the home directory by default. Its location can be adjusted using the `DumpDir=<directory where you want the output files>` `cliloader` option. The `CLIntercept_Dump` directory contains a file called `clintercept_trace.json`. You can load this JSON file in the Google* Chrome trace event profiling tool (`chrome://tracing/`) to visualize the timeline data collected by the run.

The following is a sample visualization of timeline data:

OpenCL Intercept Layer Full Example Trace



This visualization shows different calls executed through time. The X-axis is time, with the scale shown near the top of the page. The Y-axis shows different calls that are split up in several ways.

The left side (Y-axis) has two different types of numbers:

- Numbers that contain a decimal point.
 - The part of the number before the decimal point orders the calls approximately by start time.
 - The part of the number after the decimal point represents the queue number the call was made in.
- Numbers that do not contain a decimal point. These numbers represent the thread ID of the thread being run on in the operating system.

The colors in the trace represent different stages of execution:

- Blue during the queued stage.
- Yellow during the submitted stage.
- Orange for the execution stage.

Identify gaps between consecutive execution stages and kernel runs to identify possible areas for optimization.

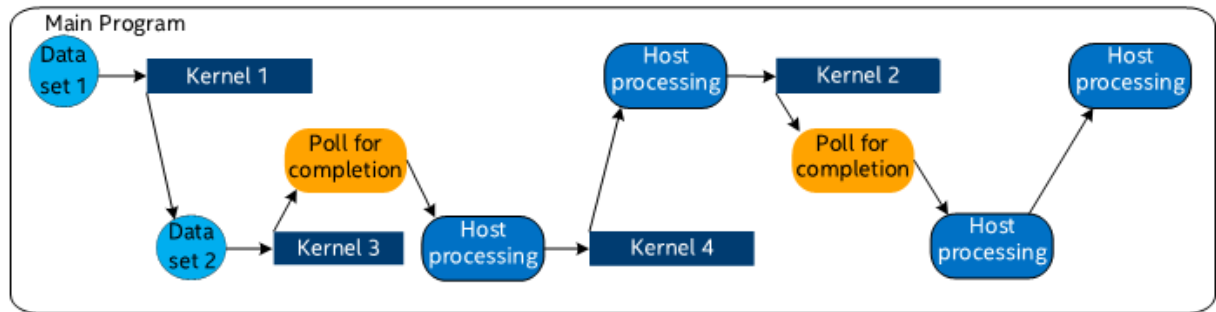
For an example use of Intercept Layer for OpenCL Applications, see [Analyzing Buffering Using the Intercept Layer for OpenCL™ Applications](#).

Multithreaded Host Application

When there are parallel, independent datapaths and the host must process data between kernel executions, consider using a multithreaded host application.

The following figure illustrates how a single-threaded host application might process parallel, independent datapaths between kernel executions:

Kernel Execution by a Single-Threaded Host Application



The SYCL* runtime is thread safe and supports multithreaded applications. Thus, you can perform tasks on the host in parallel threads while still allowing those threads to access the SYCL APIs in a thread-safe way.

Multithreaded host applications are supported when only device code interacts with pipes (sometimes referred to as "interkernel pipes"). If host code interacts with pipes (sometimes referred to as "host pipes"), the multithreaded host applications is unsupported.

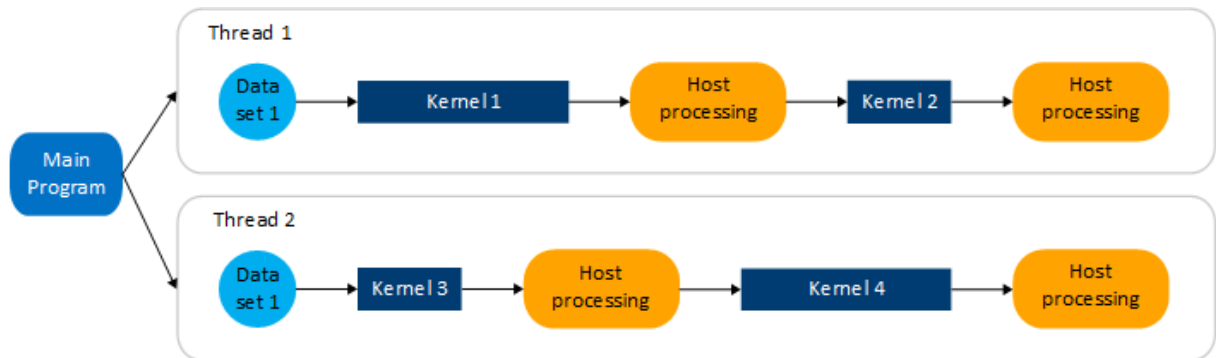
NOTE

A SYCL system that has FPGAs installed does not support multiprocess execution. Creating a context opens the device associated with the context and locks it for that process. No other process may use that device. Some queries about the device through `device.get_info<>()` also opens up the device and locks it to that process since the runtime needs to query the actual device to obtain that information. The following are examples of queries that lock the device:

- `is_endian_little`
- `global_mem_size`
- `local_mem_size`
- `max_constant_buffer_size`
- `max_mem_alloc_size`
- `vendor`
- `name`
- `is_available`

The following figure illustrates how a multithreaded host application processes parallel, independent datapaths between kernel executions:

Kernel Execution by a Multithreaded Host Application in a Thread-Safe Runtime Environment

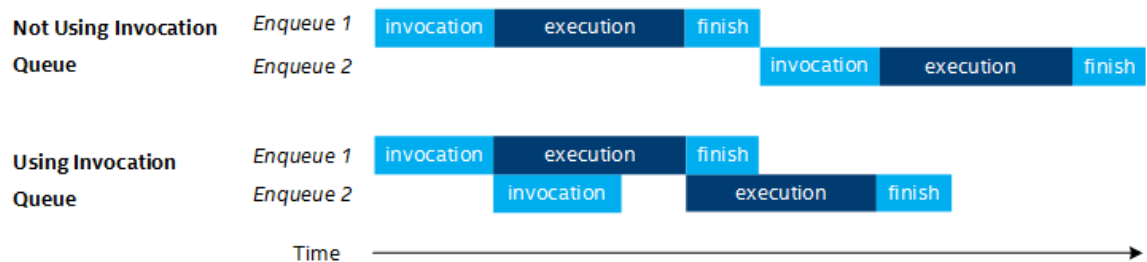


Utilizing Hardware Kernel Invocation Queue

Kernel invocation queue is a first-in first-out (FIFO) buffer used by the SYCL* runtime to store arguments for multiple kernel invocations on the device. Once the kernel finishes execution, the invocation queue allows the next invocation of the kernel to start immediately after. SYCL kernels are built with invocation queue to enable immediate launch of the next invocation.

As illustrated in the following figure, when the invocation queue is used, system and SYCL runtime environment overheads (from responding to the finish and sending in the next set of invocation arguments) are overlapped with the kernel executions. This allows kernels to execute continuously, maximizing the system level throughput.

Kernel Execution with and without Invocation Queue



SYCL kernel invocations are queued in hardware when the same SYCL kernel function is already running on the device, and the following are true:

- SYCL kernel was not compiled with hardware kernel invocation buffer disabled (`-Xsno-hardware-kernel-invocation-queue`). See [Disable Hardware Kernel Invocation Queue \(-Xsno-hardware-kernel-invocation-queue\)](#)
- SYCL kernel was not compiled with performance counters (`-Xsprofile`). See [Instrument the Kernel Pipeline with Performance Counters \(-Xsprofile\)](#)
- Any host to device synchronization operation (such as, host accessor, buffer destruction, and so on) is done between sequential kernel enqueues that requires first enqueue to finish.

Consider the following definitions of `simple_kernel()` and `check_output()` functions:

`simple_kernel()`

```
void simple_kernel(queue &deviceQueue,
                  buffer<cl_float, 1> &bufferA,
                  buffer<cl_float, 1> &bufferC)
```

```
{
    deviceQueue.submit([&](handler& cgh) {
        accessor accessorA(bufferA, cgh, read_only);
        accessor accessorC(bufferC, cgh, read_write, no_init);

        cgh.single_task<class SimpleAdd>([=]() {
            for (int i = 0; i < N; i++) {
                accessorC[i] = accessorA[i] + accessorA[i];
            }
        });
    });
}
```

check_output()

```
void check_output(buffer<cl_float, 1> &outBuffer) {
    accessor output_buf_acc(outBuffer, read_only);
    ...
    // Check output
    ...
}
```

Based on the function definitions of [simple_kernel\(\)](#) and [check_output\(\)](#), consider the following example code snippet where the kernel enqueue can be queued on the hardware kernel invocation queue:

```
// Example 1
main()
{
    ...
    simple_kernel(device_queue, bufferA, bufferC);
    simple_kernel(device_queue, bufferX, bufferZ);
    check_output(bufferC);
    check_output(bufferZ);
    ...
}
```

As soon as the first enqueue of `SimpleAdd` kernel is running, the second enqueue can be queued since they have no dependency.

Now, consider the following example code where kernel invocation cannot be queued on hardware:

```
// Example 2
main()
{
    ...
    simple_kernel(device_queue, bufferA, bufferC);
    check_output(bufferC);
    simple_kernel(device_queue, bufferX, bufferZ);
    check_output(bufferZ);
    ...
}
```

Creating the `output_buf_acc` accessor for the output buffer in [check_output\(\)](#) function is a synchronization point that blocks the SYCL runtime until the first enqueue of `SimpleAdd` kernel is complete.

Double Buffering Host Utilizing Kernel Invocation Queue

Double buffering in SYCL* host application allows SYCL runtime environment to coalesce memory transfers and kernel execution.

In an application where the FPGA kernel is executed multiple times, the host must perform the following processing and buffer transfers before each kernel invocation.

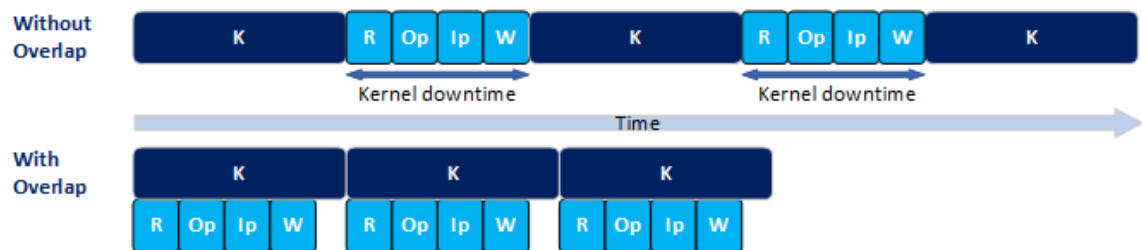
1. The output data from the *previous* invocation must be transferred from device to host and then processed by the host. Examples of this processing include:
 - Copying the data to another location
 - Rearranging the data
 - Verifying it in some way
2. The input data for the *next* invocation must be processed by the host and then transferred to the device. Examples of this processing include:
 - Copying the data from another location
 - Rearranging the data for kernel consumption
 - Generating the data in some way

Without double buffering, host processing and buffer transfers occur between kernel executions. Therefore, there is a gap in time between kernel executions, which you can refer as *kernel downtime* (See [Figure 58: Double Buffering Host](#)). If these operations overlap with kernel execution, the kernels can execute back-to-back with minimal downtime, thereby increasing overall application performance.

Determine When is Double Buffering Possible

Consider the following illustration:

Double Buffering Host



Following are the definitions of the required variables:

- R : Time to transfer the kernel's output buffer from device to host.
- Op : Host-side processing time of kernel output data (output processing)
- Ip : Host-side processing time for kernel input data (input processing)
- W : Time to transfer the kernel's input buffer from host to device.
- K : Kernel execution time

In general, R , Op , Ip , and W operations must all complete before the next kernel is launched. To maximize performance, while one kernel is executing on the device, these operations should execute simultaneously on the host and operate on a second set of buffer locations. They should complete before the current kernel completes, thus allowing the next kernel to be launched immediately with no downtime. In general, to maximize performance, the host must launch a new kernel every K .

This leads to the following constraint:

$$R + Op + Ip + W \leq K, \text{ to minimize kernel downtime}$$

If the above constraint is not satisfied, a performance improvement may still be observed because of some overlap (perhaps not complete overlap) is still possible.

Measure the Impact of Double Buffering

You must get a sense of the kernel downtime to identify the degree to which this technique can help improve performance.

This can be done by querying the total kernel execution time from the runtime and comparing it to the overall application execution time. In an application where kernels execute with minimal downtime, these two numbers are close. However, if kernels have a lot of downtime, overall execution time notably exceeds kernel execution time.

Hardware Kernel Invocation Queue While Double Buffering Example

To utilize hardware kernel invocation queue while double buffering, write your host code as shown in the following code snippet:

```
main()
{ ...
  initialize_input(input_buf[0]);
  initialize_input(input_buf[1]);
  simple_kernel(device_queue, input_buf[0], output_buf[0]);
  for (int i = 1; i < TIMES; i++) {
    simple_kernel(device_queue,
                  input_buf[i%2],
                  output_buf[i%2]); // Launch the next kernel
    // Process output from previous kernel.
    // This will block on kernel completion.
    check_output(output_buf[(i-1)%2]);
    // Generate input for the next kernel.
    initialize_input(input_buf[(i-1)%2]);
  }
  ...
}
```

The following is the example function definition for `initialize_input()`:

```
void initialize_input (buffer<cl_float, 1> &inBuffer){
  accessor buf_acc(inBuffer, write_only, no_init);
  for (int i = 0; i < N; i++) {
    buf_acc[i] = rand();
  }
}
```

NOTE

For additional information, refer to the FPGA tutorial sample "Double Buffering" on [GitHub](#).

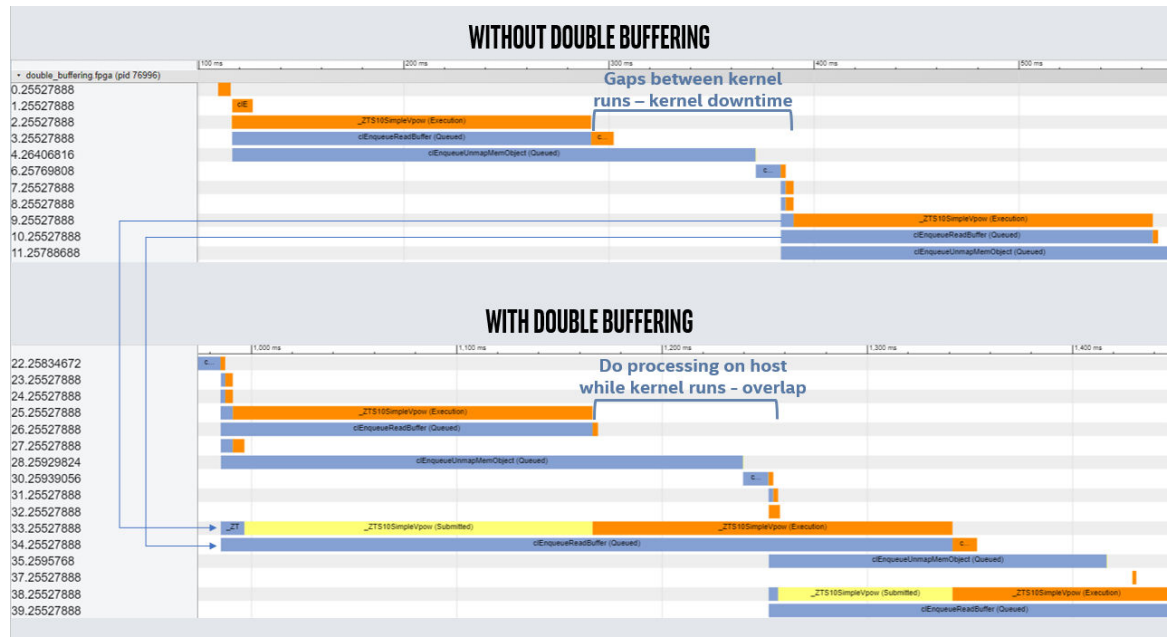
Analyzing Buffering Using the Intercept Layer for OpenCL™ Applications

The Intercept Layer for OpenCL™ Applications is an open-source tool that you can use to profile oneAPI designs at a system-level. Although it is not part of the Intel® oneAPI Base Toolkit installation, it is freely available on GitHub*. For more information, refer to [System-level Profiling Using the Intercept Layer for OpenCL™ Applications](#).

The [double-buffering optimization](#) can help minimize or remove gaps between consecutive kernels as they wait for host processing to finish. These gaps are minimized or removed by having the host perform processing operations on a second set of buffers while the kernel executes. With this execution order, the host processing is done by the time the next kernel can run, so kernel execution is not held up waiting for the host. For additional information, refer to the FPGA tutorial sample "Double Buffering" on [GitHub](#).

In this example, the first three kernels are run without the double-buffer optimization, and the next three are run with it. The kernels were run on an Intel® Programmable Acceleration Card with Intel® Arria® 10 GX FPGA when the intercept layer data was collected. The change made by this optimization can be clearly seen in the Intercept Layer for OpenCL™ Applications trace:

OpenCL Intercept Layer - With and Without Double Buffering



Here, the kernel runs named `_ZTS10SimpleVpow` can be recognized as the bars with the largest execution time (the large orange bars). Double buffering removes the gaps between the kernel executions that can be seen in the top trace image. This optimization improves the throughput of the design.

The Intercept Layer for OpenCL™ Applications makes the need for and benefits of the optimization clear. Use the Intercept Layer tool on your designs to identify scenarios where you can apply double buffering (and other optimizations).

N-Way Buffering to Overlap Kernel Execution

N-way buffering is a generalization of the [double buffering](#) optimization technique. This system-level optimization enables kernel execution to occur in parallel with host-side processing and buffer transfers between host and device, improving application performance. N-way buffering can achieve this overlap even when the host-processing time exceeds kernel execution time.

In an application where the FPGA kernel is executed multiple times, the host must perform the following processing and buffer transfers before each kernel invocation.

1. The output data from the *previous* invocation must be transferred from the device to the host and then processed by the host. Examples of this processing include the following:
 - Copying the data to another location
 - Rearranging the data
 - Verifying it in some way
2. The input data for the *next* invocation must be processed by the host and then transferred to the device. Examples of this processing include:
 - Copying the data from another location
 - Rearranging the data for kernel consumption
 - Generating the data in some way

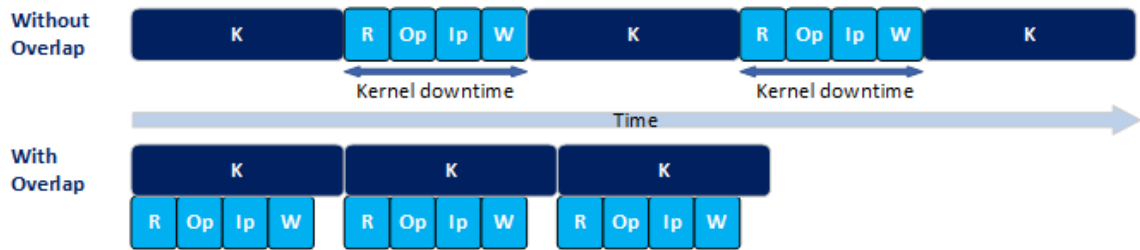
Without the N-way buffering, host processing and buffer transfers occur between kernel executions. Therefore, there is a gap in time between kernel executions, which you can refer as *kernel downtime* (See [Figure 60: N-Way Buffering Host](#)). If these operations overlap with kernel execution, the kernels can execute back-to-back with minimal downtime, thereby increasing the overall application performance.

Determine When is N-Way Buffering Possible

N-way buffering is frequently referred to as *double buffering* in the most common case where $N=2$.

Consider the following illustration:

N-Way Buffering Host



The following are the definitions of the required variables:

- R : Time to transfer the kernel's output buffer from device to host.
- Op : Host-side processing time of kernel output data (output processing).
- Ip : Host-side processing time for kernel input data (input processing).
- W : Time to transfer the kernel's input buffer from host to device.
- K : Kernel execution time
- N : The number of buffer sets used.
- C : The number of host-side CPU cores.

In general, R , Op , Ip , and W operations must all complete before the next kernel is launched. To maximize performance, while one kernel is executing on the device, these operations must run in parallel and operate on a separate set of buffer locations. They must complete before the current kernel completes, thus allowing the next kernel to be launched immediately with no downtime. In general, to maximize performance, the host must launch a new kernel every K .

If these host-side operations are executed serially, this leads to the following constraint:

$$R + Op + Ip + W \leq K, \text{ to minimize kernel downtime}$$

In the above example, if the constraint is satisfied, the application requires two sets of buffers. In this case, $N=2$. However, the above constraint may not be satisfied in some applications if host-processing takes longer than the kernel execution time.

NOTE

A performance improvement may still be observed because kernel downtime may still be reduced (though perhaps not maximally reduced).

In this case, to further improve performance, reduce host-processing time through multithreading. Instead of executing the above operations serially, perform the input and output-processing operations in parallel using two threads, leading to the following constraint:

$$\text{Max } (R+Op, Ip+W) \leq K$$

$$R + W \leq K, \text{ to minimize kernel downtime}$$

If the above constraint is still unsatisfied, the technique can be extended beyond two sets of buffers to N sets of buffers to help improve the degree of overlap. In this case, the constraint becomes:

```
Max (R + Op, Ip + W) <= (N-1)*K  
R + W <= K, to minimize kernel downtime.
```

The idea of N -way buffering is to prepare N sets of kernel input buffers, launch N kernels, and when the first kernel completes, begin the subsequent host-side operations. These operations may take a long time (longer than K), but they do not cause kernel downtime because an additional $N-1$ kernels have already been queued and can launch immediately. By the time these first N kernels complete, the aforementioned host-side operations would have also completed, and the $N+1$ kernel can be launched with no downtime. As additional kernels complete, corresponding host-side operations are launched on the host, using multiple threads in a parallel fashion. Although the host operations take longer than K , if N is chosen correctly, they complete with a period of K , which is required to ensure you can launch a new kernel every K . To reiterate, this scheme requires multi-threaded host-operations because the host must perform processing for up to N kernels in parallel to keep up.

Use the above formula to calculate the N required to minimize downtime. However, there are some practical limits:

- N sets of buffers are required on both the host and device. Therefore, both must have the capacity for this many buffers.
- If the input and output processing operations are launched in separate threads, then $(N-1)*2$ cores are required so that C can become the limiting factor.

Measure the Impact of N-Way Buffering

You must get a sense of the kernel downtime to identify the degree to which this technique can help improve performance.

You can do this by querying the total kernel execution time from the runtime and comparing it to the overall application execution time. In an application where kernels execute with minimal downtime, these two numbers are close. However, if kernels have a lot of downtime, overall execution time notably exceeds the kernel execution time.

NOTE

For additional information, refer to the FPGA tutorial sample "N-Way Buffering" on [GitHub](#).

Prepinning Memory

You must consider how the transfer of data from the host to the device occurs when optimizing kernel memory accesses. For designs that have longer data transfer times than the compute time, the data transfer time may be a bottleneck. On devices supporting greater than a PCIe Gen3 x 8 transfer rates, prepinning the memory that is on the host prior to its transfer allows for it to transfer at a higher bandwidth.

For example, the following code snippet shows how to copy the prepped memory to the device global memory when using a board with PCIe Gen3 x16 transfer rate. The memory transfer rate with prepinning achieves approximately 12 GB/s in half-duplex and 21 GB/s in full-duplex.

```
intel::fpga_selector device_selector;  
auto device_queue = queue(device_selector);  
int* data = malloc_host<int>(1024, device_queue);  
... // initialize the data  
int* data_device = malloc_device<int>(1024, device_queue);  
device_queue.template copy<int>(data_device, data, 1024);
```

Restriction

- Most BSPs implement the [Unified Shared Memory \(USM\)](#) call `malloc_host()` using prepinned memory. Hence, a prepinned memory is available only on devices that support USM host allocation.
- SYCL USM host allocations are only supported by some BSPs. Check with your BSP vendor to see if they support SYCL USM host allocations.

The OFS Intel® oneAPI Accelerator Support Package (ASP) supports USM. For details, refer <https://github.com/OFS/oneapi-asp/releases>.

Pinned memory is a scarce resource on the system (limited by the physical RAM available on your system), so carefully consider which buffers you want to pin to avoid exceeding the system limit. In addition, pinning itself is an expensive operation, so for optimal performance, ensure that the creation of pinned buffers takes place outside the main compute loop.

Simple Host-Device Streaming

You can take advantage of SYCL [Unified Shared Memory](#) host allocation and zero-copy host memory to implement a streaming host-device design with low latency and high throughput.

Tip If your BSP supports host pipes, use them instead of simple host-device streaming. For information about host pipes, refer to [Host Pipes](#).

Tip

Before you begin learning how to stream data, review [Pipes](#) and [Zero-Copy Memory Access](#) concepts.

NOTE SYCL USM host allocations are only supported by some BSPs. Check with your BSP vendor to see if they support SYCL USM host allocations.

The OFS Intel® oneAPI Accelerator Support Package (ASP) supports USM. For details, refer <https://github.com/OFS/oneapi-asp/releases>.

To understand the concept, consider the following design example (available on GitHub):

- An offload design that maximizes throughput with no optimization for latency. See `DoWorkOffload` in [simple_host_streaming.cpp](#).
- A single-kernel design that uses the methods described below to achieve a much lower latency while maintaining throughput. See `DoWorkSingleKernel` in [simple_host_streaming.cpp](#) and [single_kernel.hpp](#).
- A multi-kernel design that uses the methods described below to achieve a much lower latency while maintaining throughput. See `DoWorkMultiKernel` in [simple_host_streaming.cpp](#) and [multi_kernel.hpp](#).

Offload Processing

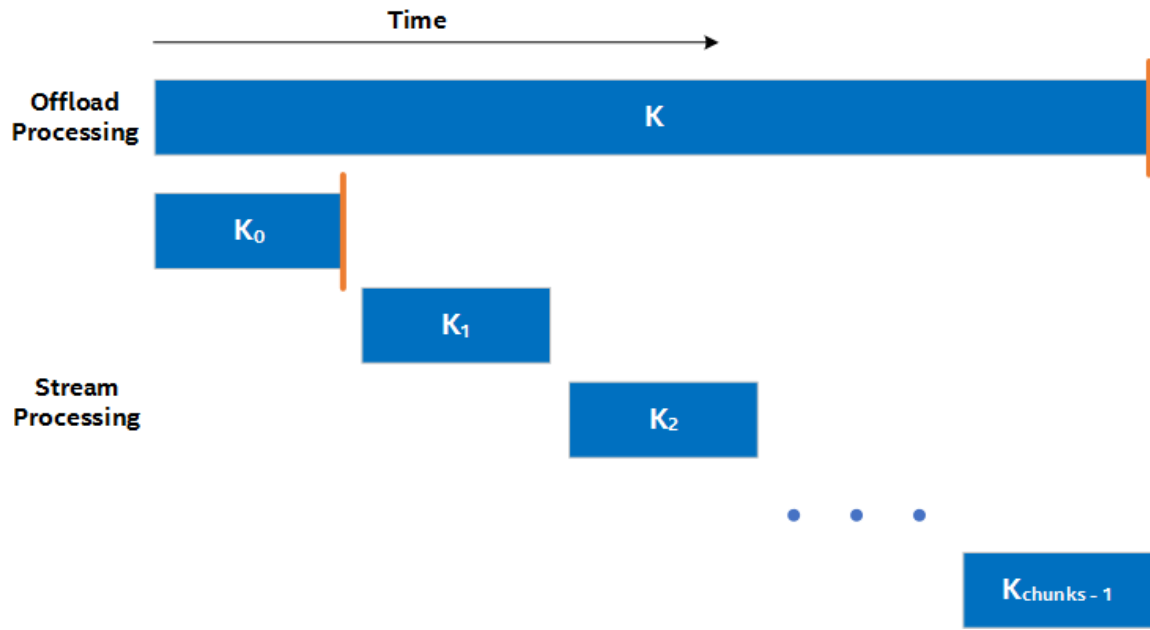
Typical SYCL designs perform offload processing. All of the input data is prepared by the CPU, and then transferred to an FPGA device. Kernels are started on the device to process the data. When the kernels finish, the CPU copies the output data from the FPGA back to its memory. Memory synchronization is achieved on the host after the device kernel signals its completion.

Offload processing achieves excellent throughput when the memory transfers and kernel computation are performed on large data sets, as the CPU's kernel management overhead is minimized. You can conceal data transfer overhead using [double buffering](#) or [N-way buffering](#) to maximize kernel throughput. However, a significant shortcoming of this design pattern is latency. The coarse-grain synchronization of waiting for the entire set of data to be processed results in a latency that is equal to the processing time of the entire data.

Host-device Streaming Processing

The method for achieving lower latency between the host and device is to break data set into smaller chunks and, instead of enqueueing a single long-running kernel, launch a set of shorter-running kernels. Together, these shorter-running kernels process the data in smaller batches. As memory synchronization occurs upon kernel completion, this strategy makes the output data available to the CPU in a more granular way. This is illustrated in the following figure, where the orange lines illustrate the time when the first set of data is available in the host:

Host-device Streaming Processing



In the streaming version, the first piece of data is available in the host earlier than in the offload version. To understand how much earlier, consider that you have `total_size` elements of data to process and you break the computation into `chunks` chunks of size `chunk_size=total_size/chunks` (as is the case in [Figure 61: Host-device Streaming Processing](#)). Then, in an ideal situation, the streaming design achieves a latency that is `chunks` times better than the offload version.

Setting the `chunk_size`

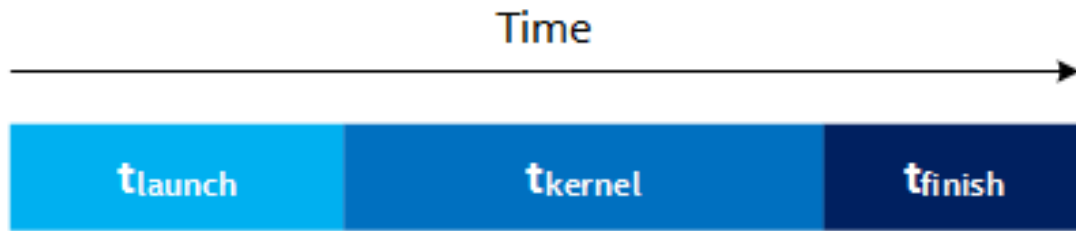
You might wonder if you can set the `chunk_size` to 1 (for example, `chunks=total_size`) to minimize the latency. In the figure above, you may notice small gaps between the kernels in the streaming design (for example, between K_0 and K_1). This is caused by the overhead of launching kernels and detecting kernel completion on the host. These gaps increase the total processing time and therefore, decrease the throughput of the design (that is, when compared to the offload design, it takes more time to process the same amount of data). If these gaps are negligible, then the throughput is negligibly affected.

In the streaming design, the choice of the `chunk_size` is thus a tradeoff between latency (a smaller chunk size results in a smaller latency) and throughput (a smaller chunk size increases the relevance of the inter-kernel latency).

Lower Bounds on Latency

Lowering the `chunk_size` can reduce latency, sometimes at the expense of throughput. However, even if you are not concerned with throughput, there still exists a lower-bound on the latency of a kernel. Review the following figure:

Lower Bounds on Latency



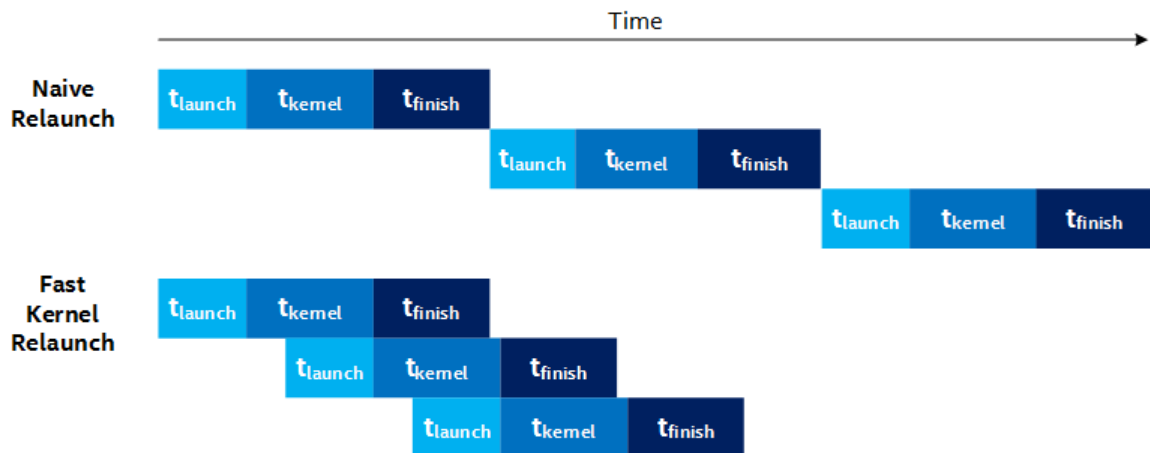
In the [Figure 62: Lower Bounds on Latency](#):

- t_{launch} is the time for a kernel launch signal to go from the host to the device.
- t_{kernel} is the time for the kernel to execute on the device.
- t_{finish} is the time for the finished signal to go from the device to the host.

Even if you set `chunk_size` to 0 (for example, launch a kernel that does nothing) and therefore $t_{\text{kernel}} \approx 0$, the latency is still $t_{\text{launch}} + t_{\text{finish}}$. In other words, the lower bound on kernel latency is the time needed for the *start* signal to go from the host to the device and for the *finished* signal to go from the device to the host.

In the [Setting the `chunk\_size`](#) section above, you learned how gaps between kernel invocations can degrade throughput. In the [Figure 62: Lower Bounds on Latency](#), there appears to be a minimum $t_{\text{launch}} + t_{\text{finish}}$ gap between kernel invocations. This is illustrated as the Naive Relaunch timeline in the following figure:

Naive Relaunch Timeline

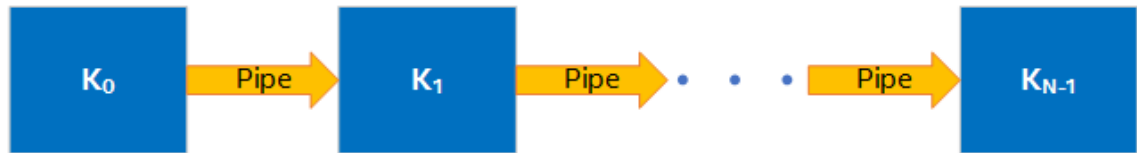


Fortunately, this overhead is circumvented by an automatic runtime kernel launch scheme that buffers kernel arguments on the device before the previous kernel finishes. This enables kernels to queue on the device and to begin execution without waiting for the previous kernel's *finished* to propagate back to the host. You can refer to this as *Fast Kernel Relaunch*, and it is also illustrated in the [Figure 63: Naive Relaunch Timeline](#). Fast kernel relaunch reduces the gap between kernel invocations and allows you to achieve lower latency while maintaining throughput.

Multiple Kernel Pipeline

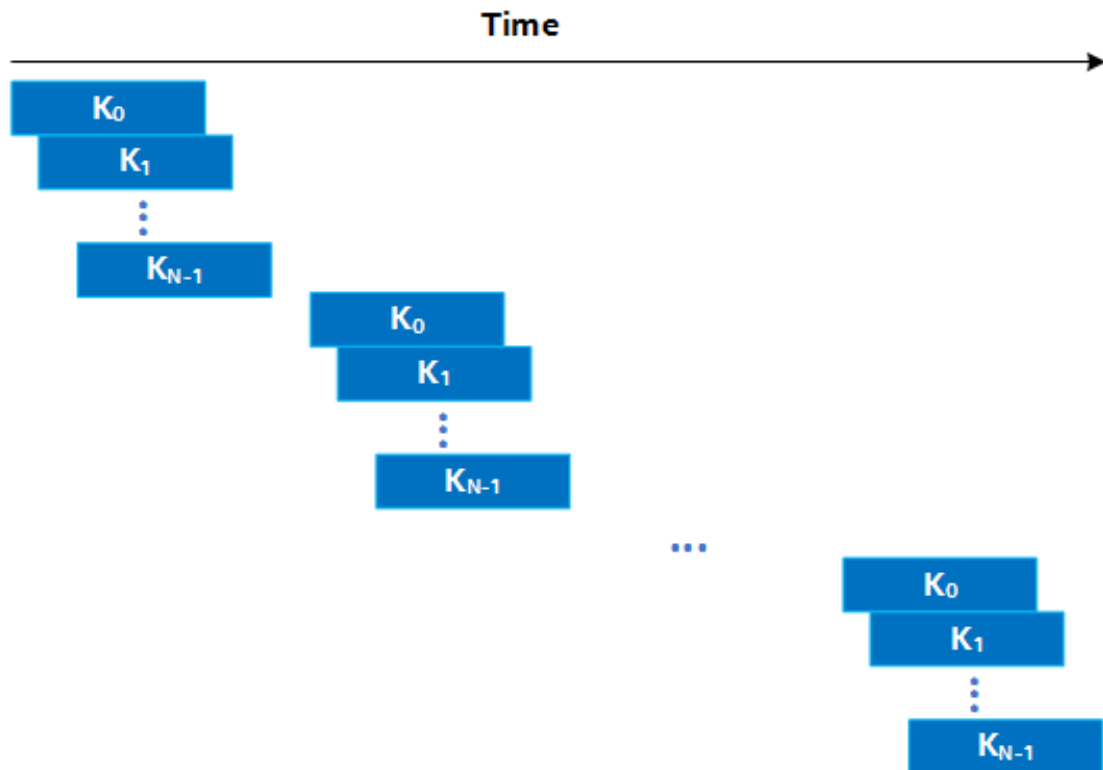
More complicated FPGA designs often instantiate multiple kernels connected by SYCL pipes (for examples, see [FPGA Reference Designs on GitHub](#)). Suppose you have a kernel system of N kernels connected by pipes, as shown in the following figure:

Multiple Kernels Connected by Pipes



With the goal of achieving lower latency, you use the [Host-device Streaming Processing](#) technique to launch multiple invocations of your N kernels to process chunks of data. This gives you a timeline as shown in the following figure:

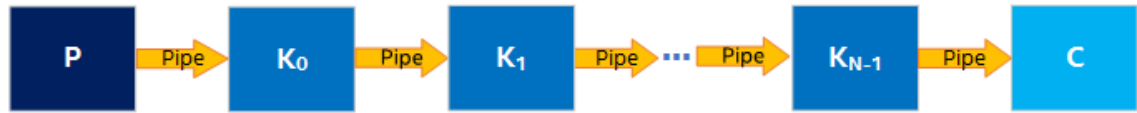
Multiple Invocation of Kernels to Process Chunks of Data



Notice the gaps between the start times of the N kernels for a single chunk. This is the t_{launch} time discussed in the [Host-device Streaming Processing](#). However, the multi-kernel design introduces a potential new lower bound on the latency for a single chunk because processing a single chunk of data requires launching N kernels, which takes $N \times t_{\text{launch}}$. If N (the number of kernels in your system) is sufficiently large, this limits your achievable latency.

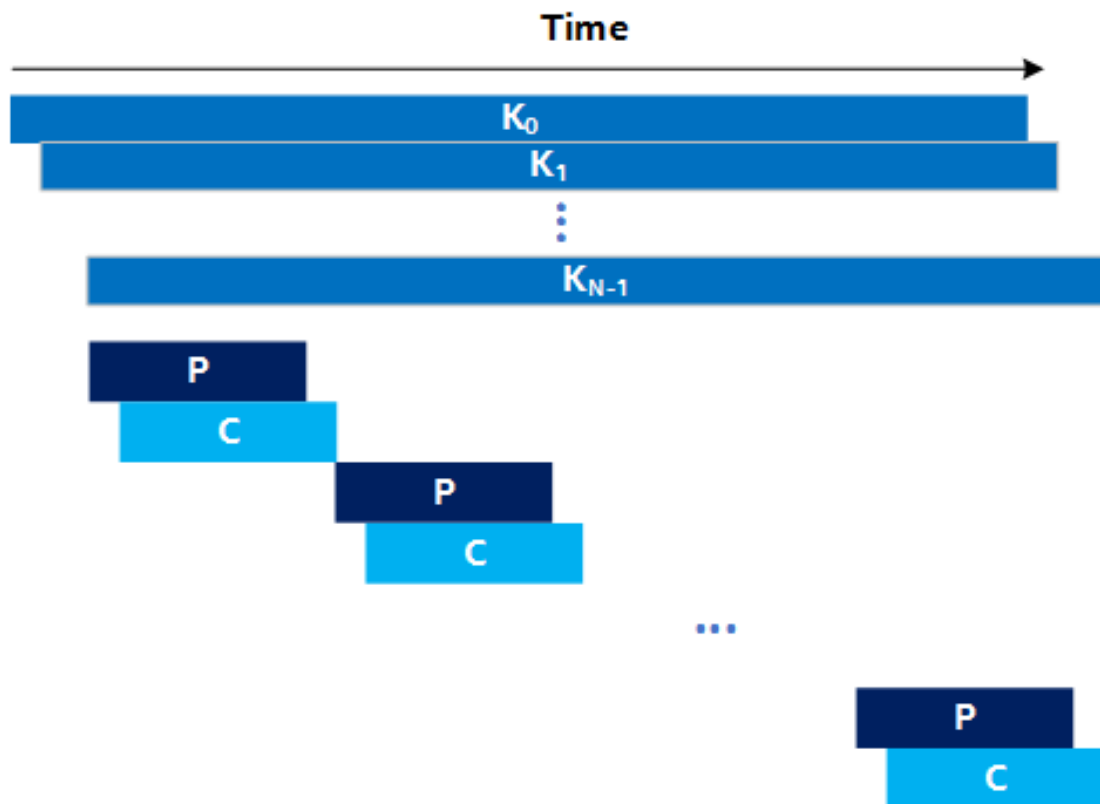
For designs with $N > 2$, Intel recommends a different approach. The idea is to enqueue your system of N kernels once, and to introduce Producer (P) and Consumer (C) kernels to handle the production and consumption of data from and to the host, respectively. This method is illustrated in the following figure:

Enqueuing a System of Kernels



The Producer streams data from the host and presents it to the kernel system through a SYCL pipe. The output of the kernel system is consumed by the Consumer and written back into host memory. To achieve low latency, you still process the data in chunks, but instead of having to enqueue N kernels for each chunk, you must only enqueue a single Producer and Consumer kernel per chunk. This enables you to reduce the lower bound on the latency to $\text{MAX}(2 \times t_{\text{launch}}, t_{\text{launch}} + t_{\text{finish}})$. Observe that this lower bound does not depend on the number of kernels in your system (N).

Use this method only when $N > 2$. FPGA area is sacrificed to implement the Producer, Consumer, and their pipes to achieve lower overall processing latency.



Limitations

Fundamentally, the ability to stream data between the host and device is built around SYCL USM host allocations. The underlying problem is how to efficiently synchronize between the host and device to signal that some data is ready to be processed, or has been processed. In other words, how does the host signal to the device that some data is ready to be processed? Conversely, how does the device signal to the host that some data is done being processed?

One method to achieve this signaling is to use the start of a kernel to signal to the device that data is ready to be processed, and the end of a kernel to signal to the host that data has been processed. This is the approach taken in the examples discussed above. However, this method has two notable drawbacks. First, the latency to start and end kernels is high. To maintain high throughput, you must size the `chunk_size` sufficiently large to hide the inter-kernel latency, resulting in a latency increase. Second, the programming model to achieve this performance is non-trivial as you must intelligently manage the SYCL device queue.

NOTE

For additional information, refer to the FPGA tutorial sample “Host-Device Streaming using USM” on [GitHub](#).

Buffered Host-Device Streaming

In this topic, you learn how to optimize a full system design that streams data from the host to the device and back to the host and create heterogeneous designs that can achieve high throughput. The techniques described in this topic are not specific to a CPU-FPGA system, and you can apply them to GPUs, multi-core CPUs, and other processing units as well.

Tip If your BSP supports host pipes, use them instead of buffered host-device streaming. For information about host pipes, refer to [Host Pipes](#).

Important

Prior to learning about buffered host-device streaming concept, you must review the following concepts:

- [Simple Host-Device Streaming](#)
 - [Double Buffering Host Utilizing Kernel Invocation Queue](#)
 - [N-Way Buffering to Overlap Kernel Execution](#)
 - [Multithreaded C++ Programming](#)
-

To understand the concept of buffered host-device streaming, consider an example design where a *Producer* (running on the CPU) produces data into [USM](#) host allocations, a *Kernel* (running on the FPGA) processes this data and produces output into host allocations, and a *Consumer* (running on the CPU) consumes the data. Data is shared between the host and FPGA device via host pointers (pointers to USM host allocations).

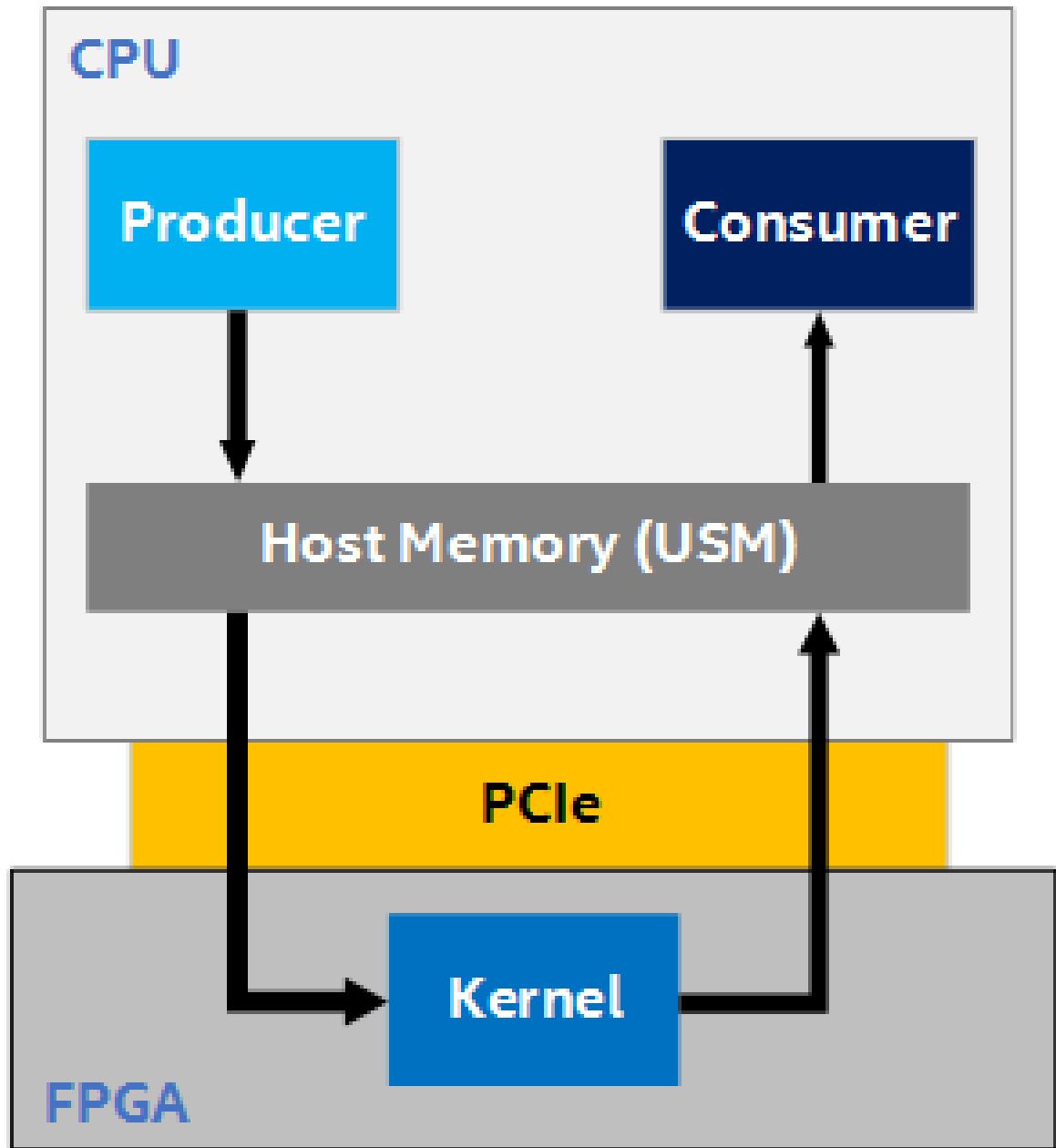
Example Design with a Producer, Kernel, and Consumer



In heterogeneous systems, it is important to understand how different compute architectures are connected (that is, how data is transferred from one to another) to better reason about and address performance bottlenecks in the system. The following figure is a slightly more detailed illustration of the processing

pipeline in the example design, which shows the data flow between the Producer, Kernel, and Consumer. It illustrates that the Producer and Consumer operations both execute on the CPU and communicate with the FPGA kernel over a PCIe link (PCIe Gen3 x16).

Detailed Illustration of the Processing Pipeline



Roofline Analysis

When designing for throughput, you must first estimate the maximum throughput the system is capable of. This back-of-the-envelope calculation is called *roofline analysis*. To calculate the maximum achievable throughput of the Producer-Kernel-Consumer design shown in [Figure 68: Detailed Illustration of the Processing Pipeline](#), assume that the Producer and Consumer have infinite bandwidth (that is, they can

produce or consume data at an infinite rate, respectively). Then, the bottleneck in the system is the FPGA kernel. Assume that the kernel has an f_{MAX} of 400MHz and processes 64 bytes per cycle. The following is kernel's maximum steady-state throughput:

$$400 \text{ MHz} * 64 \text{ bytes/cycle} = 25.6 \text{ GB/s} \approx 26 \text{ GB/s}.$$

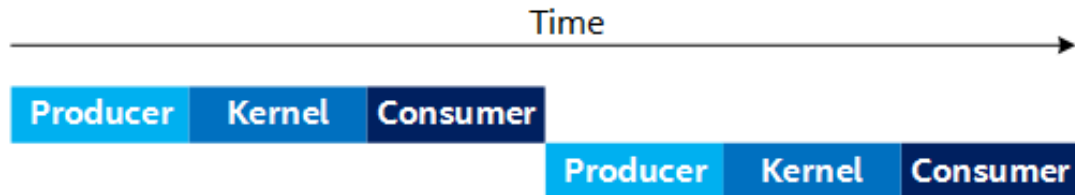
In reality, the rate of data transfer to and from the kernel depends on the bandwidth of the PCIe link. In [Figure 68: Detailed Illustration of the Processing Pipeline](#), you can observe that this depends on the bandwidth of the PCIe link. It has been experimentally measured that the PCIe Gen3 x16 link has a total bandwidth of $\sim 22 \text{ GB/s}$ (11 GB/s bidirectional). This means that, while the kernel can process data at 26 GB/s, you can only send data to it and receive from it at a rate of 11 GB/s. The system bottleneck is the PCIe link between the CPU and FPGA. It is common for the bottleneck in a heterogeneous system to be the inter-device link, rather than any single device.

In this analysis so far, you assumed that the Producer and Consumer had infinite bandwidth. You can now perform a roofline analysis on the entire system by assuming a finite bandwidth for the Producer and Consumer. For the entire Producer-Kernel-Consumer design, the maximum possible throughput is the minimum of the Producer, Kernel, and Consumer throughput, and the bandwidth of the link (in this case, PCIe) connecting the CPU and FPGA (a chain is only as strong as the weakest link).

Optimizing the Design for Throughput

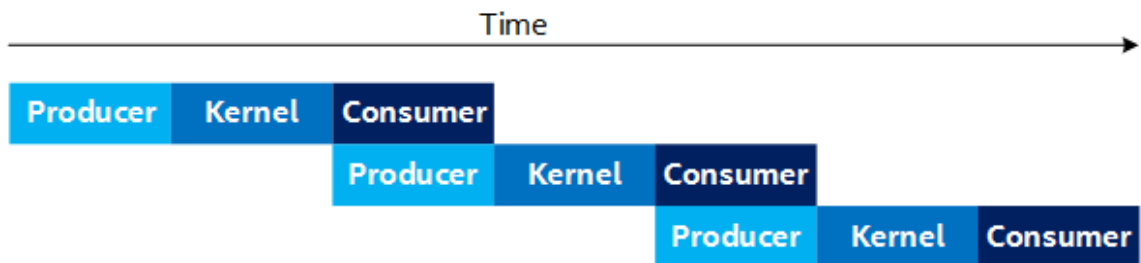
A naive approach to this design is for the Producer, Kernel, and Consumer to run sequentially that is depicted by the following timing diagram. This naive approach underperforms because operations, which could run in parallel, are run sequentially. For example, in the following figure, the Producer could be producing the second set of data, while the Consumer is consuming the first set of data:

Timing Diagram



To improve the performance, you can create a separate CPU thread (called the Producer thread) that produces all of the data for the Kernel to process, and consumes later as shown in the following figure. This allows the Producer and Consumer to run in parallel (that is, the Producer thread produces the next set of data, while the Consumer consumes the current data).

Timing Diagram with Producer and Consumer Running in Parallel

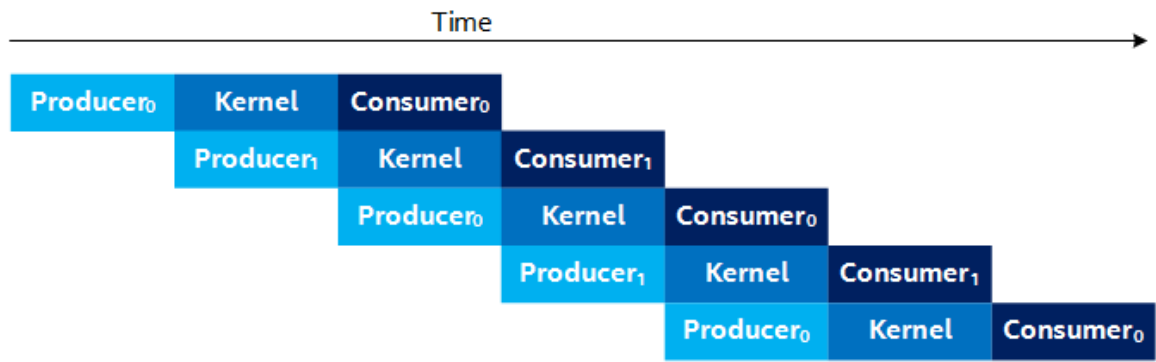


This approach improves the design's throughput, but does require more complicated thread synchronization logic to synchronize between the Producer thread and the main thread that is launching kernels and consuming its output.

Observe that in the [Figure 70: Timing Diagram with Producer and Consumer Running in Parallel](#), the Producer and Consumer are running simultaneously (in separate threads). You must consider the fact that both of these processes are running on the CPU in parallel, which affects their individual throughput capabilities. For example, running these processes in parallel can result in saturating the CPU's memory bandwidth, and therefore lower the overall throughput for the Producer and Consumer processes even if the threads run on different CPU cores.

By putting the Producer into its own thread, you can improve performance by producing and consuming data simultaneously. However, it is ideal if the Producer can produce the next set of data while the kernel is processing the current data. This allows the next kernel to launch as soon as the current kernel finishes, instead of waiting for the Producer to finish, as shown in [Figure 70: Timing Diagram with Producer and Consumer Running in Parallel](#). Unfortunately, you cannot produce data into the buffer that the kernel is currently processing, since it is still in use by the kernel. To address this, use multiple buffers (that is, [N-Way buffering](#)) of the input and output buffers, which results in a timing diagram, as shown in the following:

Timing Diagram With Multiple Buffers

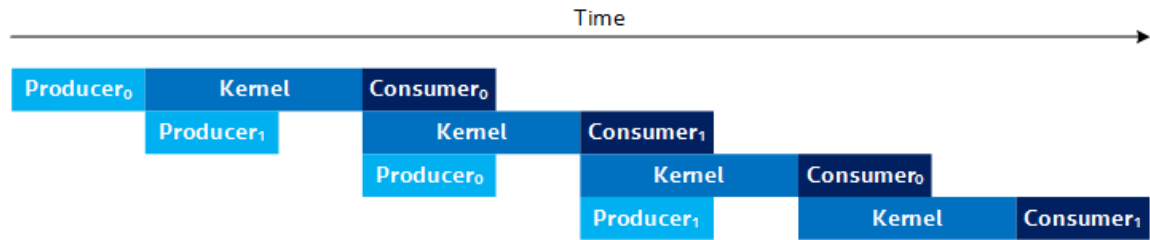


The subscript number on the Producer and Consumer (for example, Producer₀ and Consumer₀) represents the buffer index they are processing. The [Figure 71: Timing Diagram With Multiple Buffers](#) illustrates the case where you use two sets of buffers ([double-buffering](#)). Observe that in [Figure 71: Timing Diagram With Multiple Buffers](#), the Producer produces into buffer 1 (Producer₁) while the kernel is processing the data from buffer 0. Thus, by the time the kernel finishes processing buffer 0, it can start processing buffer 1 right away, so long as the Producer has finished producing the next buffer of data.

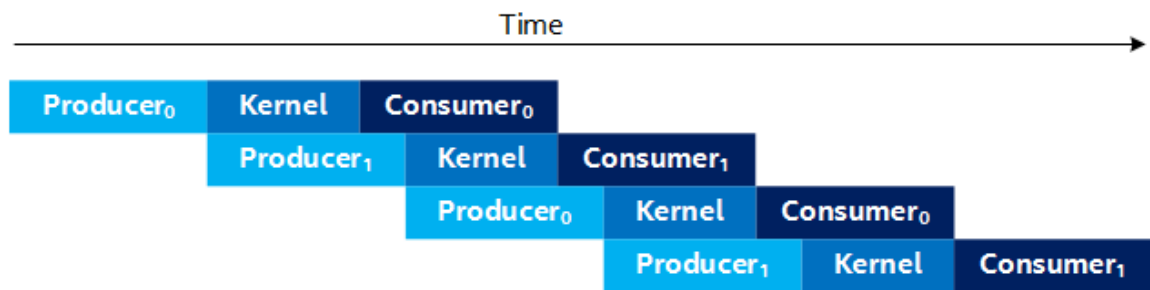
In the steady-state for [Figure 71: Timing Diagram With Multiple Buffers](#), the throughput of the design is bottlenecked by the throughput of the slowest stage (the Producer, Kernel, or Consumer), which matches the [Roofline Analysis](#) you did earlier. For example, the [Figure 72: Timing Diagram With Kernel as the Throughput](#)

Bottleneck shows an example timeline where the Kernel is the throughput bottleneck. The [Figure 73: Timing Diagram With Producer/Consumer as the Throughput Bottleneck](#) shows an example timeline where the Producer (or Consumer) is the bottleneck.

Timing Diagram With Kernel as the Throughput Bottleneck



Timing Diagram With Producer/Consumer as the Throughput Bottleneck



Streaming API

In the [Buffered Host-Device Streaming](#) example on GitHub, the techniques described in the previous section are implemented in two ways. The design is implemented directly using SYCL USM host allocations and C++ multithreading, and the kernel queue is managed intelligently. The code achieves high performance, but it might be difficult to understand and extend it to different designs. To address this, a convenient and performant API wrapper ([HostStreamer.hpp](#)) is created. The same design is implemented in [streaming_with_api.hpp](#) with similar performance and significantly less code that is much easier to understand.

Important

While the code that uses the `HostStreamer` API achieves similar performance to a direct implementation, it uses extra FPGA resources. The direct implementation has a single kernel (Kernel) that does all the processing. Using the API creates a Producer and Consumer kernel that access host allocations and produce/consume data to/from the processing kernel (`APIKernel` in [streaming_with_api.hpp](#)). These extra kernels (that are transparent) are the mechanism by which the API abstracts the production/consumption of data, but come at the cost of extra FPGA resources. However, when compiled for the Intel FPGA PAC D5005, these extra kernels result in less than 1% increase in FPGA resource utilization. Therefore, given the programming convenience they provide, the tradeoff is often worth it.

NOTE

For additional information, refer to the FPGA tutorial sample "Buffered Host-Device Streaming" on [GitHub](#).

Integrating Your Kernel into DSP Builder for Intel® FPGAs

10

Integrate your SYCL* kernel into DSP Builder for Intel® FPGAs to take advantage of MATLAB* and Simulink* features that can help you develop and test your SYCL* kernel.

To integrate your kernel into DSP Builder for Intel® FPGAs, use the DSP Builder SYCL* block.

For more information, including kernel requirements and restrictions when integrating into DSP Builder, refer to the ["SYCL" topic of the DSP Builder for Intel® FPGAs \(Advanced Blockset\): Handbook](#).

Integrating Your RTL IP Core Into a System

11

To integrate your RTL IP core into a system with the Quartus® Prime software, you must be familiar with Quartus® Prime software, including Platform Designer.

The Intel® oneAPI DPC++/C++ Compiler generates a project directory (`<project_name>.prj/`) and a set of IP files per IP core. You can control this with the `-fsycl-device-code-split=<off|per_source|per_kernel>` option as follows:

- The default behavior (`-fsycl-device-code-split` not specified or `-fsycl-device-code-split=off`) combines all kernels in the source file into a single IP.
- Specifying `-fsycl-device-code-split=per_kernel` gives you a separate project directory (`<project_name>.prj/`), FPGA Optimization Report, set of IP files for each kernel.

The `<project_name>.prj/` directory generated by the compiler contains all the files that you need to include your IP core in a Quartus® Prime project, including the following files:

- `top_<project_name>_di.ip`
An IP file that you can add to your Quartus® Prime projects.
- `<project_name>_di_hw.tcl`
A script that describes your IP core interfaces for Platform Designer.
- `<project_name>_di_inst.sv`
An example of how to instantiate the IP into other Verilog modules.

Synthesize Your RTL IP Core with Quartus® Prime Software

When you are satisfied with the predicted performance of your kernel, use Quartus® Prime software to synthesize your kernel. Synthesis also generates accurate area and performance (f_{MAX}) estimates for your design. However, your design is not expected to cleanly close timing in the Quartus® Prime reports.

You can expect to see timing closure warnings in the Quartus® Prime logs because the generated project targets a clock speed of 1000 MHz to achieve the best possible placement for your design. The f_{MAX} value presented in the FPGA Optimization Report estimates the maximum clock rate your component can cleanly close timing for.

After the Quartus® Prime compilation is completed, the summary section of the FPGA Optimization Report shows the area and performance data for your kernels. These estimates are more accurate than estimates generated when you compile your kernel for simulation only.

Typically, Quartus® Prime compilation times can take minutes to hours, depending on the size and complexity of your kernel.

To synthesize your kernel and generate quality of results (QoR) data, instruct the compiler to run the Quartus® Prime compilation flow automatically after synthesizing the components. Include the **-Xshardware** option in your **icpx** command:

```
icpx -fintelfpga -Xshardware -Xstarget="<FPGA device family or part number>"...
```

Encrypting RTL IP Cores

After synthesizing your IP cores, you might want to protect them with encryption before they are integrated into a larger system. You have the following options for encrypting your IP cores:

- Encryption with licensing

Your IP users can use the encrypted IP only in ways specified by the generated license. This license is compatible with the FlexLM licensing technology used by Quartus® Prime software.

Encryption with licensing requires a public key.

For details, refer to [Encrypting RTL IP Core With Licensing](#).

- Encryption without licensing

Your encrypted IP can be integrated freely using Quartus® Prime software. No licensing is enforced.

For details, refer to [Encrypting RTL IP Cores Without Licensing](#).

Encrypting RTL IP Cores Without Licensing

You can encrypt your RTL IP cores without licensing by using the `encrypt_1735` command provided by the Quartus® Prime Pro Edition design suite. The `encrypt_1735` command supports the IEEE 1735 v1 encryption standard.

After you have compiled your RTL IP core with the `icpx` command, encrypt your RTL IP core as follows:

1. Go to the folder that contains the generated SystemVerilog files. This folder is typically `<project_name>.prj/kernel_hdl`.
2. Encrypt all SystemVerilog file in the folder with the following command:

```
encrypt_1735 --quartus --language=systemverilog <filename>
```

3. Remove or backup any unencrypted files.
4. Change the file extension of the encrypted files from `.svp` to `.sv`.

The encrypted files can be integrated into a system with Platform Designer. For details, refer to [Add an RTL IP Core into a Platform Designer System](#).

For more information about encryption and the `encrypt_1735` command, refer to “[Support for the IEEE 1735 Encryption Standard](#)” in *Quartus® Prime Pro Edition User Guide: Getting Started*

Encrypting RTL IP Core With Licensing

If you are a member of the Intel® FPGA Design Solutions Network, you have access to tools to encrypt your IP design files and generate a license for it. Your IP users can use the encrypted IP only in ways specified by the generated license.

This license is compatible with the FlexLM licensing technology used by Quartus® Prime software.

If you have the Intel-provided IP encryption and licensing infrastructure installed, you can also generate encrypted IP with the Intel® oneAPI DCP++/C++ Compiler.

Your encrypted IP can then be used by your customers in Quartus® Prime software, licensed by the file that your users added to their Quartus® Prime license search path. For more details, refer to the documentation that Intel provided you when you joined the Intel® FPGA Design Solutions Network.

If you want to support simulation with your encrypted IP, you must create a separately-encrypted version of your IP for simulation. For simulation, an IEEE 1735 compliant encryption scheme is used.

To generate encrypted IP for use in Quartus® Prime software, use the following command:

```
icpx -fintel FPGA -Xshardware -Xstarget=<FPGA device or part number> \  
-Xsencryption-key=<key> -Xsencryption-id=<product_id> \  
-Xsencryption-release-date=<yyyy.mm><file.cpp> -o <file>.fpga
```

Important Before you run this command, you must create a license file for the IP and add the license file to your `$LM_LICENSE_FILE` environment variable.

To generate encrypted IP for use in simulation, use the following command:

```
icpx -fintelfpga -Xssimulation -Xstarget=<FPGA device or part number> \
  -DFPGA_SIMULATOR -I/$INTELFPGAOCSDKROOT/include -Xsencryption-key=<key> \
  -Xsencryption-id=<product_id> -Xsencryption-release-date=<yyyy.mm> \
  <file.cpp> -o <file>.fpga_sim
```

FPGA Compilation Flags for IP Encryption

Option name	Description
-Xsencryptionkey	Specifies the encryption key used to encrypt the source file. The key must be a 48-digit hexadecimal value.
-Xsencryption-id	Specifies the product ID for the IP. This ID must be a 4-digit hexadecimal value.
-Xsencryption-release-date	Sets the release date in the format <code>yyyy.mm</code> .
-Xsno-encryption	If you have created an alias to your <code>icpx</code> command that encrypts your IP, use this option on your alias command to temporarily disable encryption.

Add an RTL IP Core into a Platform Designer System

To use the RTL IP core generated by the Intel® oneAPI DPC++/C++ compiler in a Platform Designer system, you must first add the directory to the IP search path or the IP Catalog.

In Platform Designer, if your IP core does not appear in the IP Catalog, perform the following tasks:

1. In the Quartus® Prime software, click **Tools > Options**.
2. In the **Options** dialog box, under **Category**, expand **IP Settings** and click **IP Catalog Search Locations**.
3. In the **IP Catalog Search Locations** dialog box, add the path to the directory that contains the `_hw.tcl` file to IP Search Paths as `<project_name>.prj`.
4. In the **IP Catalog**, add your IP to the Platform Designer system by selecting it from the oneAPI project directory.

For more information about Platform Designer, refer to [Creating a System with Platform Designer](#) in *Quartus® Prime Pro Edition User Guide: Platform Designer*.

For an example of adding an IP core into a system with Platform Designer, see the [Platform Designer Sample tutorial on GitHub](#).

Add an RTL IP Core into a Quartus® Prime Project

To use the RTL IP core generated by the Intel® oneAPI DPC++/C++ compiler in a Quartus® Prime project, you must first add the `.ip` file to the project.

The `.ip` file contains information to add to all the necessary HDL files for the core. It also applies to any core-specific Quartus® Prime Settings File (`.qsf`) settings that are necessary for IP synthesis.

Follow these steps:

1. Create an Quartus® Prime Pro Edition project.
2. Open the Platform Designer and select your IP from the oneAPI folder.
For your IP to be in the oneAPI folder, either create the project in the same directory that contains the generated IP project or add the file path.
3. Create the rest of your Quartus® Prime software project.

For an example of how to instantiate the RTL IP core top-level module, examine the `<project_name>.prj/
<project_name>_di_inst.sv` file.

RTL IP Core Kernel Interfaces

RTL IP Core Kernels only The information presented here applies only to RTL IP core kernels only. It does not apply to multiarchitecture binary kernels.

Multiarchitecture Binary Kernels When developing a multiarchitecture binary, the compiler generates the kernel interfaces to interact with your BSP for you.

When you develop IP core kernels, the compiler generates interfaces for your larger system to interact with the IP core. An RTL IP core kernel has two basic interface types: an invocation interface and a data interface. A kernel can have multiple data interfaces but only one invocation interface.

Invocation Interfaces

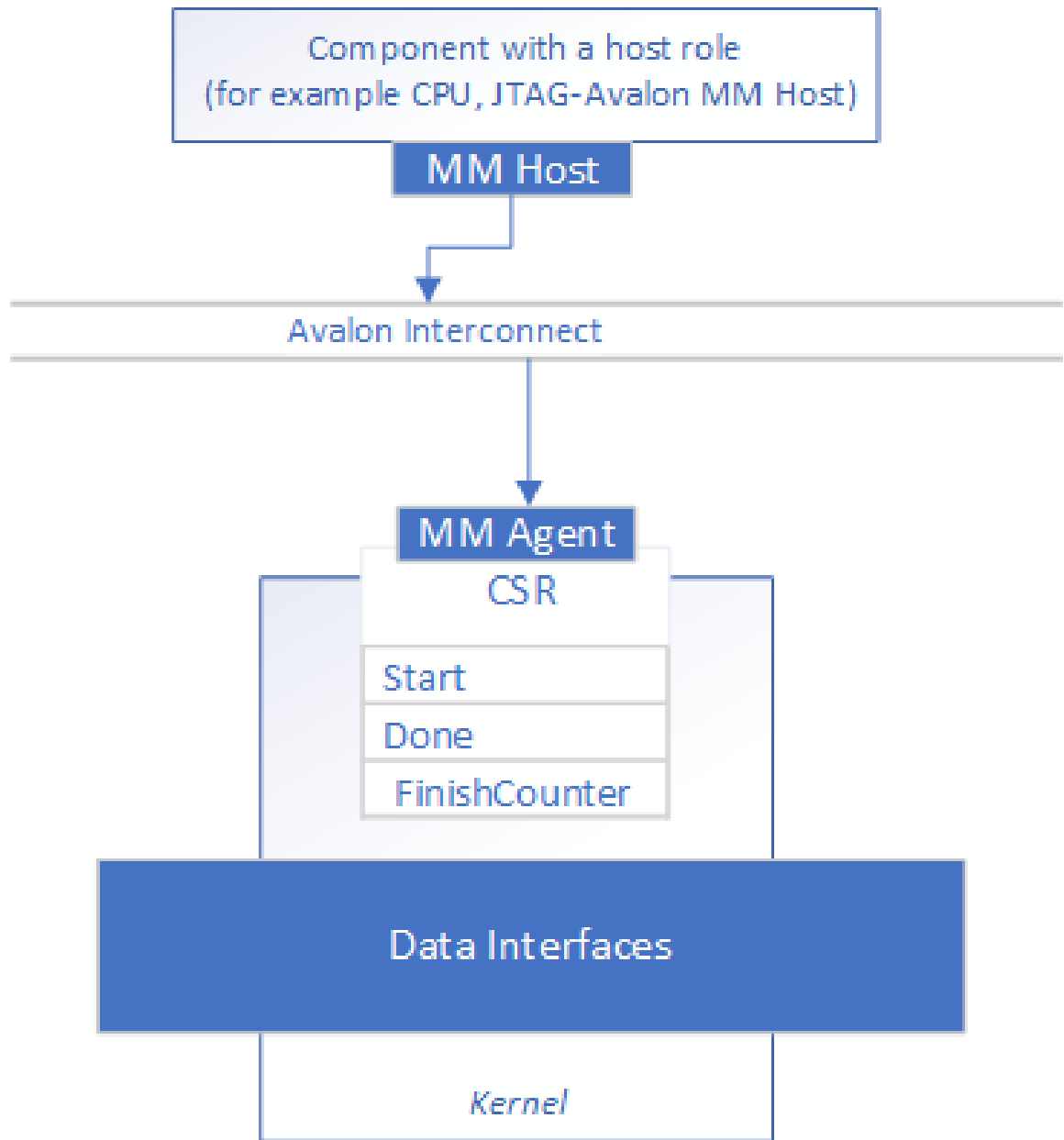
The *invocation interface* contains the handshake signals for invoking the kernel and the signals that indicate kernel completion.

By default, RTL IP core kernels generate a control-and-status register (CSR) agent interface for consuming inputs.

In an agent kernel, the `start`, `busy`, and `done` signals appear in the IP core CSR as a `start` register and a `status` register that includes the `busy`, and `done` status signals along with some other information. Because the signals are registered in the memory map, the handshaking protocol to invoke the kernel becomes a read of the `status` register to check if the kernel is ready before a write to the `start` register to invoke the kernel. The kernel sets its interrupt port high to signal kernel completions, and a read of the

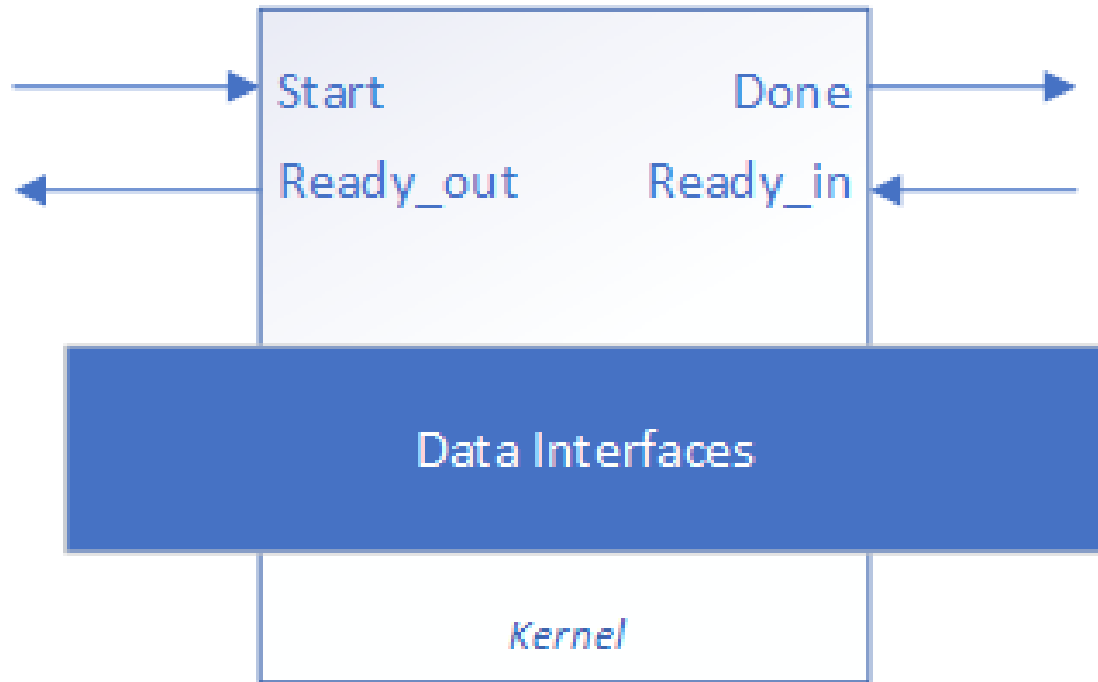
`finish_counter` register acknowledges the kernel completions and the clears the `finish_counter` register. For details about the agent kernel invocation interface, refer to [Memory-Mapped \(MM\) Agent Kernel Invocation Interface](#)

Memory-Mapped Agent Invocation Interface



In the invocation interface for a ready/valid handshaking kernel (sometimes referred to as a *streaming kernel*), the resulting IP core uses `start`, `ready_out`, `ready_in`, and `done` signals for handshaking. The IP core consumes the unstable arguments when the `start` signal is asserted and the `ready_out` signal is asserted. The IP core signals kernel completion by asserting the `done` signal. For details about the ready/valid handshaking kernel invocation interface, refer to [Ready/Valid Handshaking Kernel Invocation Interface](#).

Ready/Valid (Streaming) Invocation Interface



Tip As part of the invocation interface, the Intel® oneAPI DPC++/C++ Compiler also generates a `device_exception_bus` conduit. The `device_exception_bus` is a 64-bit conduit that is used to pass exception data in multiarchitecture binary kernel designs. It is exposed in the RTL IP core generated by the compiler, but you can ignore it.

Data Interfaces

A *data interface* is used to transfer data in and out of your component. Data interfaces are inferred from the kernel arguments and any annotations that you apply to them.

You can pass data into a kernel in the following ways:

- Kernel memory arguments
 - Using accessors
 - Using unified shared memory (USM) pointer
- Host pipes

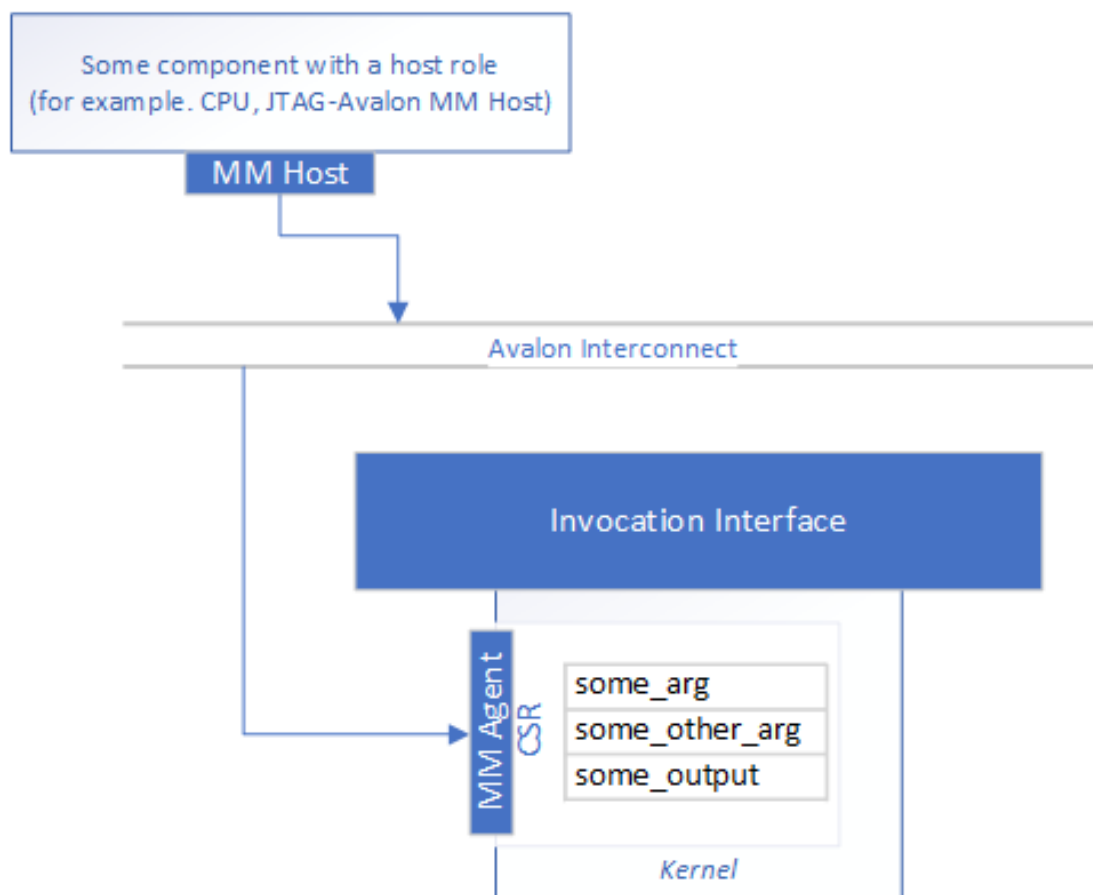
You can pass items by value as kernel arguments:

- Variables captured by the kernel lambda become kernel arguments.
- Data members of kernel functors become kernel arguments.

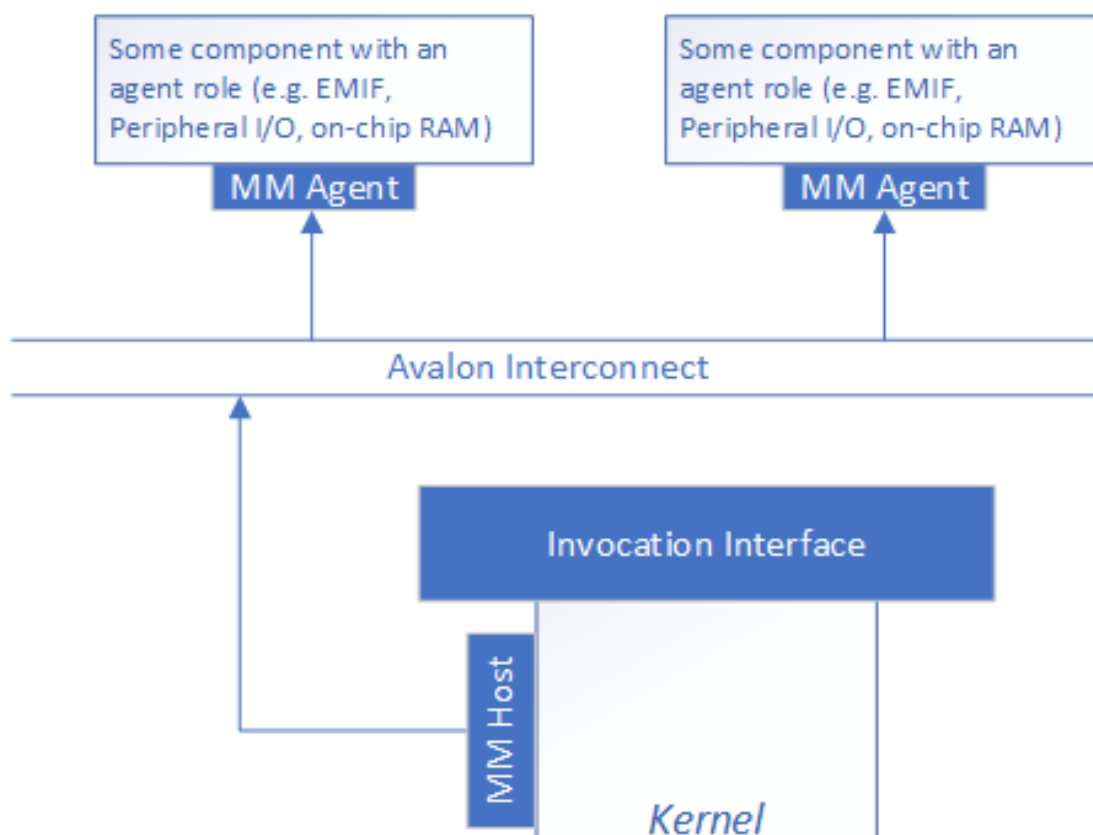
Using an accessor or a unified shared memory (USM) pointer creates an Avalon® memory mapped host interface on your IP.

Your IP can produce output only through an accessor, USM pointer, or a pipe. The kernel invocation interface cannot capture output from an IP core generated by the Intel® oneAPI DPC++/C++ Compiler.

Avalon® Memory-Mapped (MM) Agent Data Interface



Avalon® Memory-Mapped (MM) Host Data Interface



Kernel Argument Interfaces

By default, the kernel arguments are passed to your component through the same interface as the `start` signal (that is, either through the IP core CSR or through inputs synchronized to a `ready/valid` handshake). However, you can override this behavior. For example, you can select a register-mapped invocation interface with arguments passed through conduits, or you can select a streaming invocation interface with arguments passed through the register map.

The following code snippet demonstrates how to place the invocation interface in the CSR, and pass kernel arguments through conduit interfaces:

```
#include <sycl/sycl.hpp>
#include <sycl/ext/intel/fpga_extensions.hpp>

using namespace sycl;
using namespace ext::intel::experimental;
using namespace ext::oneapi::experimental;

struct MyIP {
    annotated_arg<int*, decltype(properties{conduit})> input_a, input_b, input_c;
    annotated_arg<int, decltype(properties{conduit})> n;
    void operator()() const {
        for (int i = 0; i < n; i++) {
            input_c[i] = input_a[i] + input_b[i];
        }
    }
};
```

The following code snippet demonstrates how to configure the kernel with a handshake invocation interface, and pass kernel arguments through the CSR:

```
#include <sycl/sycl.hpp>
#include <sycl/ext/intel/fpga_extensions.hpp>

using namespace sycl;
using namespace ext::intel::experimental;
using namespace ext::oneapi::experimental;

struct MyIP {
    annotated_arg<int*, decltype(properties{register_map})> input_a, input_b, input_c;
    annotated_arg<int, decltype(properties{register_map})> n;
    void operator()() const {
        for (int i = 0; i < n; i++) {
            input_c[i] = input_a[i] + input_b[i];
        }
        auto get(properties_tag) {
            return properties{streaming_interface<>};
        }
    }
};
```

Memory-Mapped (MM) Agent Kernel Invocation Interface

The compiler generates an invocation interface for your SYCL* kernel that you can use to control the kernel and pass in kernel arguments.

By default, the Intel® oneAPI DPC++/C++ Compiler generates a memory-mapped (MM) agent interface to control the kernel and pass in the kernel arguments.

Examples of MM Agent Kernel Invocation Interfaces

Functor Model MM Agent Kernel Invocation Interface Code Example

```
#include <sycl/sycl.hpp>

using namespace sycl;

struct MyFunctorIP {
    int *input_a, *input_b, *input_c;
    int n;

    void operator()() const {
        for (int i = 0; i < n; i++) {
            input_c[i] = input_a[i] + input_b[i];
        }
    }
};

...

q.single_task(MyFunctorIP{functor_input_A, functor_input_B, functor_input_C, kN}).wait();
```

Lambda Model MM Agent Kernel Invocation Interface Code Example

```
#include <sycl/sycl.hpp>

using namespace sycl;

class MyLambdaIP;

...

q.single_task<MyLambdaIP>([=] {
    for (int i = 0; i < n; i++) {
        lambda_input_C[i] = lambda_input_A[i] + lambda_input_B[i];
    }
}).wait();
```

Register Map Layout

Byte Address	Read/Write	Description
0x00	Read only	64-bit status register
0x08	Write only	64-bit start register
0x10		
0x18		
0x20		reserved
0x28		
0x30	Read only	finish counter
0x38		
0x40		reserved
0x48		
0x50		

Byte Address	Read/Write	Description
0x58		
0x60		
0x68		
0x70		
0x78		
0x80	Write only	kernel arguments and pipes that use <code>protocol_name::avalon_mm</code>
0x88	Write only	or <code>protocol_name::avalon_mm_uses_ready</code>
...	Write only	

NOTE You might observe that the simulator accesses some of the registers in the “reserved” region. These accesses are used for profiling purposes and are not functionally necessary.

Status Register (0x00)

Byte	7	6	5	4	3	2	1	0
Description	reserved				CSR Address Map Version*		kernel_status Bits	

The CSR address map version might change when you update the Intel® oneAPI DPC++/C++ Compiler. If the CSR address map version changes, review the generated CSR structure and any code that you that depends on the CSR structure.

kernel_status Bits

Bit	15	14:3	2	1	0
Description	running	reserved	busy	done	reserved

Where the behavior defined by the kernel_status bits is as follows:

- bit 15: running
Set to 1 if there is an invocation of the corresponding IP that started but not finished.
- bit 2: busy
This bit is set to 1 by the IP when it cannot accept any new input. New arguments can be written to the argument registers, but they overwrite any buffered arguments that have not been consumed. Any writes to the start bit in the `start` register are ignored.
If your kernel arguments do not change while the kernel is executing, you can disable this behavior to try to save some hardware area in your design. For more information, refer to [Stable Arguments](#).
- bit 1: done
This bit is set to 1 by the IP when an invocation of the IP finishes processing. This bit is cleared by reading the `finish_counter` register.

Start Register (0x08)

Bit	63:1	0
Description	reserved	start

Register Map Header File

The compiler generates a header file that provides the addresses of various registers in the agent memory map. A top-level header file named `register_map_offsets.h` is created in the `.prj/include` directory for each device image that you can include if you are interfacing with the SYCL* device image.

An additional header is generated for each of your kernels within the `.prj` directory. Each kernel has a `<kernel_name>_register_map.h` file associated with it. The file is located in the `.prj/include/kernel_headers` directory.

The `register_map_offsets.h` header file includes these `<kernel_name>_register_map.h` files and defines the address offsets for each kernel.

In most cases, include only the `register_map_offsets.h` file directly. However, you might still need to refer to the individual kernel `<kernel_name>_register_map.h` files for the macro definitions that you need to use in your application.

Example Register Map File

```
/* This header file describes the Register Map for the MyFunctorIP kernel */

/* Note that this header file should NOT be included directly! */
/* Please include the top-level header file register_map_offsets.h instead! */

#ifndef __MYFUNCTORIP_REGISTER_MAP_REGS_H__
#define __MYFUNCTORIP_REGISTER_MAP_REGS_H__

/* Status register contains all the control bits to control kernel execution */
/*****
/* Memory Map Summary
*****/

/*


| Address | Access | Register                  | Argument                                   | Description                                                                         |
|---------|--------|---------------------------|--------------------------------------------|-------------------------------------------------------------------------------------|
| 0x0     | R      | reg0[63:0]                | Status[63:0]                               | * Read the status bits that are described below                                     |
| 0x8     | W      | reg1[31:0]                | Start[31:0]                                | * Write 1 to initiate a kernel start                                                |
| 0x30    | R      | reg6[31:0]<br>reg6[63:32] | FinishCounter[31:0]<br>FinishCounter[31:0] | * Read to get number of kernel finishes, note that this register will clear on read |
| 0x80    | W      | reg16[63:0]               | arg_input_a[63:0]                          |                                                                                     |
| 0x88    | W      | reg17[63:0]               | arg_input_b[63:0]                          |                                                                                     |
| 0x90    | W      | reg18[63:0]               | arg_input_c[63:0]                          |                                                                                     |
| 0x98    | W      | reg19[31:0]               | arg_n[31:0]                                |                                                                                     |


*/
```

The status register (0x00) has the following layout:

Byte	7	6	5	4	3	2	1	0
Description	Reserved			CSR Address Map Version		kernel_status Bits		

If the structure of the generated CSR changes in future versions of the Intel® oneAPI DPC++/C++ Compiler, the CSR address map version will be updated. The current CSR address map version is 5. If the CSR address map version changes,

review the generated CSR structure and any code that depends on the CSR structure. The kernel_status bits have the following layout:

Bit	15	14:3	2	1	0
Description	running	reserved	busy	done	reserved

```

*/

#define CSR_VERSION_NUMBER (5)

/*****
/* Register Address Macros */
*****/

/* Status Register Bit Offsets (Bits) */
/* Note: Bits In Status Registers Are Marked As Read-Only or Read-Write
   Please Do Not Write To Read-Only Bits */
#ifndef __REGISTER_BITOFFSET_MACROS__
#define __REGISTER_BITOFFSET_MACROS__
#define KERNEL_REGISTER_MAP_DONE_OFFSET (1) // Read-only
#define KERNEL_REGISTER_MAP_BUSY_OFFSET (2) // Read-only
#define KERNEL_REGISTER_MAP_RUNNING_OFFSET (15) // Read-only
#endif

/* Status Register Bit Masks (Bits) */
#ifndef __REGISTER_BITMASK_MACROS__
#define __REGISTER_BITMASK_MACROS__
#define KERNEL_REGISTER_MAP_DONE_MASK (0x2)
#define KERNEL_REGISTER_MAP_BUSY_MASK (0x4)
#define KERNEL_REGISTER_MAP_RUNNING_MASK (0x8000)
#endif

/* Byte Addresses */
#define MYFUNCTORIP_REGISTER_MAP_STATUS_REG (0x0 + MYFUNCTORIP_REGISTER_MAP_OFFSET)
#define MYFUNCTORIP_REGISTER_MAP_START_REG (0x8 + MYFUNCTORIP_REGISTER_MAP_OFFSET)
#define MYFUNCTORIP_REGISTER_MAP_FINISHCOUNTER_REG (0x30 + MYFUNCTORIP_REGISTER_MAP_OFFSET)
#define MYFUNCTORIP_REGISTER_MAP_FINISHCOUNTER_REG (0x30 + MYFUNCTORIP_REGISTER_MAP_OFFSET)
#define MYFUNCTORIP_REGISTER_MAP_ARG_ARG_INPUT_A_REG (0x80 + MYFUNCTORIP_REGISTER_MAP_OFFSET)
#define MYFUNCTORIP_REGISTER_MAP_ARG_ARG_INPUT_B_REG (0x88 + MYFUNCTORIP_REGISTER_MAP_OFFSET)
#define MYFUNCTORIP_REGISTER_MAP_ARG_ARG_INPUT_C_REG (0x90 + MYFUNCTORIP_REGISTER_MAP_OFFSET)
#define MYFUNCTORIP_REGISTER_MAP_ARG_ARG_N_REG (0x98 + MYFUNCTORIP_REGISTER_MAP_OFFSET)

/* Argument Sizes (bytes) */
#define MYFUNCTORIP_REGISTER_MAP_ARG_ARG_INPUT_A_SIZE (8)
#define MYFUNCTORIP_REGISTER_MAP_ARG_ARG_INPUT_B_SIZE (8)
#define MYFUNCTORIP_REGISTER_MAP_ARG_ARG_INPUT_C_SIZE (8)
#define MYFUNCTORIP_REGISTER_MAP_ARG_ARG_N_SIZE (4)

/* Argument Masks */
#define MYFUNCTORIP_REGISTER_MAP_ARG_ARG_INPUT_A_MASK (0xffffffffffffffffULL)
#define MYFUNCTORIP_REGISTER_MAP_ARG_ARG_INPUT_B_MASK (0xffffffffffffffffULL)
#define MYFUNCTORIP_REGISTER_MAP_ARG_ARG_INPUT_C_MASK (0xffffffffffffffffULL)
#define MYFUNCTORIP_REGISTER_MAP_ARG_ARG_N_MASK (0xffffffffULL)

#endif /* __MYFUNCTORIP_REGISTER_MAP_REGS_H__ */

```

Ready/Valid Handshaking Kernel Invocation Interface

While the default kernel invocation interface is a memory-mapped (MM) agent interface, you can also have the Intel® oneAPI DPC++/C++ Compiler implement the kernel invocation interface with a ready/valid handshake, similar to what an Avalon® streaming interface uses. A kernels with a ready/valid handshake kernel invocation interface is often referred to as a *streaming kernel*.

Streaming kernels can also be pipelined. For details, refer to [Pipelined Kernels](#).

You can indicate that a kernel uses a "ready-valid" handshake interface as its invocation interface by using the `streaming_interface` kernel property.

The `streaming_interface` Kernel Property

Use the `streaming_interface` kernel property to request the compiler to implement the kernel invocation interface with a "ready-valid" handshake.

The `streaming_interface` kernel property supports the following template parameters:

- `accept_downstream_stall`

If the `streaming_interface` property with `accept_downstream_stall` (`streaming_interface<accept_downstream_stall>`) is specified on a kernel, the compiler generates a "ready-valid" kernel interface at both input and output such that the "ready-valid" handshaking happens both at kernel invocation and kernel completion.

A `streaming_interface_accept_downstream_stall` property is also provided for convenience.

- `remove_downstream_stall`

If the `streaming_interface` property with `remove_downstream_stall` (`streaming_interface<remove_downstream_stall>`) is specified on a kernel, the compiler generates a "ready-valid" kernel interface only at the input to the kernel. The "ready-valid" handshaking happens only at kernel invocation. There is no interface to gate the `done` signals emitted by the kernel. That is, there is no "back-pressure" on the kernel.

A `streaming_interface_remove_downstream_stall` property is also provided for convenience.

If no template parameter is provided, the `accept_downstream_stall` value is inferred.

To use the `streaming_interface` kernel property:

1. Include the following header file in your code:

```
sycl/ext/intel/fpga_extensions.hpp
```

2. Label your kernel with the `streaming_interface` property as follows:

- Functor Model:

1. Add a member function named `get` to the functor. Have the `get` function take an argument of type `properties_tag` and a return type `auto`.
2. Create a properties object in the new function with the `streaming_interface` property and return it.

Functor Model `streaming_interface` Kernel Property Code Example

```
#include <sycl/sycl.hpp>
#include <sycl/ext/intel/fpga_extensions.hpp>

using namespace sycl;
using namespace sycl::ext::intel::experimental;
using namespace sycl::ext::oneapi::experimental;
```

```

struct MyFunctorIP {
    int *input_a, *input_b, *input_c;
    int n;

    void operator()() const {
        for (int i = 0; i < n; i++) {
            input_c[i] = input_a[i] + input_b[i];
        }
    }

    auto get(properties_tag) {
        return properties{streaming_interface<>};
    }
};

...

q.single_task(MyFunctorIP{functor_input_A, functor_input_B, functor_input_C, kN}).wait();

```

- **Lambda Model:**

1. Pass a `properties` object that contains the `streaming_interface` property to your `q.single_task` call.

Lambda Model `streaming_interface` Kernel Property Code Example

```

#include <sycl/sycl.hpp>
#include <sycl/ext/intel/fpga_kernel_properties.hpp>

using namespace sycl;
using namespace sycl::ext::intel::experimental;
using namespace sycl::ext::oneapi::experimental;

class MyLambdaIP;

// Create a properties object containing the kernel invocation
// interface property
properties kernel_properties{streaming_interface<>};

...

q.single_task<MyLambdaIP>(kernel_properties, [=] {
    for (int i = 0; i < n; i++) {
        lambda_input_C[i] = lambda_input_A[i] + lambda_input_B[i];
    }
}).wait();

```

Limitations of Streaming Kernels

The following actions are not supported when using a streaming kernel:

- Using streaming kernels as SYCL NDRange kernels.
- Profiling of streaming kernels.

Memory-Mapped Host Interfaces

Each external memory interface is uniquely identified by a “buffer location” identifier. Buffer location identifiers can be applied to arguments annotated with the `annotated_arg<>` property and accessors. A pointer-type kernel argument with a specified buffer location is sometimes referred to as an *annotated pointer argument*. An *unannotated pointer argument* can be any of the of the following items:

- Accessor arguments without a specified buffer location
- Kernel arguments not specified as `annotated_arg` type with a `buffer_location` property

For each unique buffer location among the annotated pointer arguments of your kernels, the compiler infers one memory-mapped host interface. If there is no annotated pointer argument defined in all kernels, one global memory-mapped interface with default configuration is inferred (with buffer location 0).

You can attribute a buffer location to a kernel argument in the following ways:

- [Memory-Mapped Host Interfaces Using Unified Shared Memory Pointers and the `annotated\_arg` Class](#)

A kernel can interface with an external memory over an Avalon® Memory-mapped (MM) Host interface. You can specify the Avalon® MM host interface with the `annotated_arg` class.

For full details about the `annotated_arg` class, refer to [The `annotated\_arg` Template Class](#).

- [Memory-Mapped Host Interfaces Using Accessors](#)

Using accessors allows the runtime and BSP to manage the copying of the memory between the host and device.

Accessor kernel arguments result in four arguments for each accessor argument.

In most cases, the compiler encodes the virtual address space information automatically. However, in some cases, you might need to encode it yourself. For details, refer to [Addresses in Memory-Mapped Host Interfaces](#).

Addresses in Memory-Mapped Host Interfaces

Memory-mapped (MM) host interfaces issue byte addresses, not word addresses.

For example, if you dereference a pointer to a 4-byte wide data type, the address issued by the MM host interface is base address of the pointer plus the array index multiplied by 4.

Assuming a base address of 0x0000 for `mm_a`, the following code example results in 0x0018 on the interface because $0x0018 = 0x0006 \times 4$:

```
uint32_t value = mm_a[0x0006];
```

If `mm_a` has a base address of 0x1000, the resulting address is 0x1018 instead of 0x0018.

If you dereference a pointer to a 1-byte wide data type, the address issued by the MM Host interface is the base address of the pointer plus the array index multiplied by 1.

Assuming a base address of 0x0000 for `mm_a`, the following code example results in 0x0006 on the interface because $0x0006 = 0x0006 \times 1$:

```
uint8_t value = mm_a[0x0006];
```

If `mm_a` has a base address of 0x1000, the resulting address is 0x1006 instead of 0x0006.

Memory-Mapped Host Interfaces Using Unified Shared Memory Pointers and the `annotated_arg` Class

A kernel can interface with an external memory over an Avalon® Memory-mapped (MM) Host interface. You can specify the Avalon® MM host interface with the `annotated_arg` class

Describe a customized Avalon® MM host interface in your code by adding an `annotated_arg` object in your kernel functor definition or by capturing an `annotated_arg` object with a lambda kernel function.

Each `annotated_arg` argument of a kernel can be configured with a conduit or register map input type.

Each pointer type `annotated_arg` argument can be customized to different port configuration attaching to a specific buffer location. An Avalon® MM Host interface is created for each unique buffer location. Host interfaces that share the same buffer location are arbitrated on the same interface.

Learn how to configure memory-mapped host interfaces in your RTL IP kernel by reviewing the [Memory-Mapped Host Interfaces sample](#) from the [oneAPI Samples for FPGA GitHub repository](#).

The `annotated_arg` class has the following arguments. For full descriptions and details, refer to [The annotated_arg Template Class](#).

The annotated_arg Template Class Properties Summary

Template Object or Parameter	Description	Valid Values
Interface Type Properties		
<code>sycl::ext::intel::experimental::conduit</code>	Create a dedicated input conduit on the kernel.	N/A
<code>sycl::ext::intel::experimental::register_map</code>	Creates an input register map for the input instead of a dedicated input port.	N/A
<code>sycl::ext::intel::experimental::stable</code>	Specifies that the input to the kernel does not change between pipelined invocations of the kernel. The input can change after all active kernel invocations have finished.	N/A
Pointer Kernel Argument Properties		
<code>sycl::ext::intel::experimental::buffer_location</code>	The global memory identifier for the pointer interface. To use any of the other Avalon® MM Host interface properties, you must also specify the <code>buffer_location</code> property.	Non-negative integer value
<code>sycl::ext::intel::experimental::dwidth</code>	The width of the memory-mapped data bus in bits	8, 16, 32, 64, 128, 256, 512, or 1024 Default: 64
<code>sycl::ext::intel::experimental::awidth</code>	The width of the memory-mapped address bus in bits.	Integer 11 – 41 Default: 41
<code>sycl::ext::intel::experimental::latency</code>	The guaranteed latency from when a read command exits the kernel until the external memory returns valid read data. When a variable latency is specified by setting value of 0, a <code>waitrequest</code> signal is generated on the interface. When a fixed latency is specified by setting a non-zero value, a <code>waitrequest</code> signal is not generated on the interface.	Non-negative integer value Default: 0
<code>sycl::ext::intel::experimental::maxburst</code>	The maximum number of data transfers that can associate with a read or write transaction.	Integer 1 – 1024
<code>sycl::ext::oneapi::experimental::alignment</code>	The alignment of the base pointer address in bytes.	See description
<code>sycl::ext::intel::experimental::read_write_mode</code>	The port direction of the interface.	<code>read_write</code> , <code>read</code> , or <code>write</code> Default: <code>read_write</code>

You can use the `annotated_arg` class to annotate kernel arguments with either lambda kernels or functor kernels.

Functor Example

```
#include <sycl/sycl.hpp>
#include <sycl/ext/intel/fpga_extension.hpp>
using namespace sycl;
using namespace ext::intel::experimental;
using namespace ext::oneapi::experimental;
struct kernel {
    annotated_arg<int*, decltype(properties{ conduit,
                                             buffer_location<1>,
                                             dwidth<32>,
                                             awidth<16>,
                                             latency<0>,
                                             read_write_mode_readwrite
                                             })> mm_a;

    void operator() () const {
        *mm_a = 1;
    }
};
```

Lambda Example

```
#include <sycl/sycl.hpp>
#include <sycl/ext/intel/fpga_extension.hpp>
using namespace sycl;
using namespace ext::intel::experimental;
using namespace ext::oneapi::experimental;
class kernel;
void launch_kernel(queue& q, int* a) {
    auto mm_a = annotated_arg(a, conduit,
                              buffer_location<1>,
                              dwidth<32>,
                              awidth<16>,
                              latency<0>,
                              read_write_mode_readwrite);
    q.single_task<class kernel>([=] {
        *mm_a = 1;
    });
}
```

Buffer Locations in Memory-Mapped Host Interfaces

Each unique identifier specified by the `buffer_location<id>` property of a pointer kernel argument represents a unique Avalon® host interface for the kernel. Ensure that you avoid using the same buffer location *id* for interfaces with different properties.

The caller is responsible for ensuring the correct buffer location is specified; otherwise, functional failures might occur. If you specify a `buffer_location` property on your kernel argument, specify the same buffer location on the USM allocation API call that allocates memory on the test bench (that is, the host code).

Similarly, the caller is responsible for aligning the data to the set value for the `align` argument; otherwise, functional failures might occur. If you specify the `alignment<value>` property, ensure that you specify the same alignment value the following locations:

- The `sycl::ext::intel::experimental::alignment<value>` property

- The *alignment* argument of the `aligned_malloc_shared` template function in your host code. For example,

```
aligned_malloc_shared<int>(value, 1, q );
```

Memory-Mapped Interface Unified-Shared-Memory Virtual Address Space

The compiler encodes certain information regarding the virtual address space in the top bits of a 64-bit pointer address as follows:

Pointer-Address Bit-Range Descriptions

Bit Range	Description
40:0	Used for addressing within the memory system
63:41	Stores the virtual address space information that is derived from the buffer location

In some cases, the compiler cannot determine which buffer location a pointer corresponds to and it creates logic in the generated RTL that inspects the top bits of the pointer at runtime to detect the buffer location and route the memory transaction to the correct external memory interface.

You do not need to encode the buffer location information yourself in most cases. Exceptions are outlined in [Buffer Location Virtual Address Spaces](#).

The compiler automatically generates logic to embed this information from the buffer location specified on the pointer kernel argument in the source file.

Mixing Annotated and Unannotated Pointers in Your Kernel

The compiler infers Avalon host interfaces as follows:

- If annotated kernel arguments are present, one interface is inferred for each unique buffer location.
- If no annotated pointer kernel arguments are present, a single Avalon Host interface is inferred for all pointer arguments.

If your design has a mix of annotated kernel arguments and unannotated pointer arguments, the unannotated pointers can access **any** of the inferred Avalon host interfaces depending on the addresses passed to the kernel via the pointer arguments.

When your designs mix annotated and unannotated pointers, there are situations where the compiler is unable to determine if an error condition exists. In these situations, functional errors or hangs can appear when you try to simulate your design or use the generated RTL.

If you have a design that mixes annotated and unannotated pointers and your design fails in simulation, check for the following conditions:

- [Buffer Locations and Unannotated Pointers](#)
- [Buffer Location Virtual Address Spaces](#)

Buffer Locations and Unannotated Pointers

Ensure that your unannotated pointer accesses are to interfaces that support the access. For example, an unannotated pointer attempting a read access to a write-only interface is an error that the compiler cannot always catch.

Consider the following design with two kernels:

```
// Kernell: Write-only MM Host Interface
// Kernell defines a write-only memory-mapped host interface
struct Kernell {
    annotated_arg<int*, decltype(properties{buffer_location<1>, read_write_mode_write})> arg_a;
    annotated_arg<int*, decltype(properties{buffer_location<2>, read_write_mode_read})> arg_b;
```

```

...
void operator() () const {
    ...
}
};

//Kernel2: Unannotated Pointer
// Kernel2 uses an unannotated pointer and does a read access
struct Kernel2 {
    int* arg_a;
    ...
    void operator() () const {
        ... = *arg_a;
    }
};

```

The kernel `Kernel1` specifies two memory-mapped host interfaces. The compiler infers the two interfaces, and the unannotated pointer argument in kernel `Kernel2` can access either of them, depending on the memory addresses that are passed to it.

Because the `buffer_location<1>` memory has a write-only interface, passing an address for the `buffer_location<1>` memory to the unannotated pointer in kernel `Kernel2` is an illegal operation.

If your program has such mixed usage, ensure that correct addresses are passed via kernel arguments.

Buffer Location Virtual Address Spaces

If the kernel has at least one kernel argument where the buffer location is specified (annotated argument) and at least one argument where the buffer location is not specified (unannotated argument), then you must embed the buffer location information in the top bits of any unannotated kernel arguments.

Consider the following code example:

```

// This struct defines the IP that is generated
struct MyIPComponent{
    // struct members are kernel arguments
    int* a; // no buffer location specified
    annotated_arg<int*, decltype(properties{buffer_location<1>})> b; // buffer location 1
    annotated_arg<int*, decltype(properties{buffer_location<2>})> c; // buffer location 2
    annotated_arg<int*, decltype(properties{buffer_location<3>})> d; // buffer location 3

    // operator() () defines the device/IP code
    void operator() () const {
        *a *= 2;
        *b *= 2;
        *c *= 2;
        *d *= 2;
    }
};

```

In this example, the compiler does not know what external memory pointer `a` will point to, so the compiler creates logic to check the top bits of the pointer to determine, at run time, which buffer location to access. Therefore, you must set those top bits for argument `a` (but not for the other kernel arguments).

In such cases, if the unannotated pointer argument has a conduit interface then the port is 64 bits wide. And, if the interface is register-map based, all 64 bits are passed to the kernel.

Simulation Exception When you simulate your kernel you do not need to write any host code (even in this use case) to embed information in the pointer bits. The buffer locations are all taken care of by the runtime stack that allocates the pointers in the host code.

When the compiler can deduce which buffer location that a pointer argument points to (for example, when there is only one `mm_host` interface), the compiler embeds the buffer location automatically. Buffer locations need to be specified manually only when your kernels have a mix of annotated and unannotated kernel arguments.

The compiler infers one global memory (with inferred buffer location 0) when there are no annotated pointer kernel arguments in your entire design (that is, across all kernels). In the following code example, because there are only unannotated pointer arguments, the compiler infers only one global memory and so it can embed the correct information in the top bits of the pointer kernel arguments.

```
// This struct defines the IP that will be generated
struct MyIPComponent{
    // struct members are kernel arguments
    int* a; // no buffer location specified
    int* b; // no buffer location specified
    // no other annotated kernel argument is present

    // operator()() defines the device/IP code
    void operator()() const {
        *a = ...
        *b = ...
    }
};
```

Determining Virtual Address Space Information

If you need to embed the virtual address space information in the top bits of the pointer kernel arguments because the compiler cannot do so, get the information to embed from the HTML reports:

1. If you have not already done so, compile your kernel to obtain the HTML reports
2. Open the HTML reports.
3. Go to **Views > System Viewer**
4. In the left pane, expand **System** and then expand **Global memory**
Under **Global memory**, you see entries for all external memories for your kernel.
5. Click on a memory to display that memory in the **System Viewer** pane.
6. In the **System Viewer** pane, find the box that represents the memory and click the node inside the box.

This node represents the “interface” of that global memory.

7. In the **Details** pane, find the **Start Address** of the memory.

The top bits of your unannotated pointer argument must match the top bits of this start address if you want the pointer to access this buffer location.

The following screen capture shows an example of determining the top bit of the start address needed to address a buffer location 1.

The screenshot displays the Intel oneAPI System Viewer interface. On the left, a tree view shows the system hierarchy: System > Global memory > 1. The main window shows a diagram of 'Global memory 1' (a blue square with a circle inside) connected to a 'Global memory interface node' (a blue trapezoid labeled 'SHARE'). A 'RESET' button is visible on the right. Below the diagram, the 'Details' panel for 'Global memory 1' is shown, with the 'Start Address' field highlighted by a blue box. The 'Start Address' is 0x200000000000. To the right, a code editor shows a snippet of C++ code from 'test.cpp'.

Details	
Latency	1
ReadWrite Mode	Read Only
Maximum Burst	1
Wait Request	0
Start Address	0x200000000000

The annotated_arg Template Class

Use the `annotated_arg` template class to annotate your kernel arguments with properties that direct the compiler to customize the kernel argument interface.

The `annotated_arg` template class is provided in the `sycl.hpp` header file and uses the `sycl::ext::oneapi::experimental` namespace. To use the `annotated_arg` template class, use the following declarations in your code:

```
#include <sycl/sycl.hpp>
using namespace sycl::ext::intel::experimental;
using namespace sycl::ext::oneapi::experimental;
```

The `annotated_arg` template class has the following syntax:

```
annotated_arg<data_type, decltype(properties { (conduit | register_map),
    [stable,]
    [buffer_location<id>,]
    [dwidth<data_bus_width>,]
    [awidth<address_bus_wdith>,]
    [latency<value>,]
    [maxburst<value>,]
    [alignment<value>,]
    [read_write_mode<mode>] } ) >
```

When passing pointers to a SYCL* kernel with the `annotated_arg` template class, the pointer needs memory allocated using USM memory allocation APIs such as `malloc_shared` or `aligned_alloc_shared`.

Restriction

- For aggregate data types (like `struct`), the `annotated_arg` template class does not support the dot (`.`) and the arrow (`->`) member-access operators. You must use an explicit or implicit conversion to the underlying data type before accessing the members of the aggregate data type.
- The `annotated_arg` template class does not support using variable-precision data types (like `ac_int`) operations (like addition, multiplication...) directly. If you want to use the `annotated_arg` template class with a variable-precision data type, you must cast the `annotated_arg` object to the underlying data type object and perform operations on that object directly. For more details, refer to [Recommended Method for Annotating struct Data Types](#).

Recommended Method for Annotating struct Data Types

When you annotate `struct` data types, create a local variable that stores the kernel argument of `struct` type so that member functions and data members of the `struct` types can be accessed directly.

The following example code demonstrates how to annotate a kernel argument of `ac_int` data type:

```
using ac_int_type = ac_int<17, true>;

struct MyIP {
    annotated_arg<ac_int_type, decltype(properties{conduit})> a;
    annotated_arg<ac_int_type, decltype(properties{conduit})> b;
    annotated_arg<int*, decltype(properties{conduit})> c;
    void operator() () const {
        // Make local variables of the struct type before using them
        ac_int_type n1 = a;
        ac_int_type n2 = b;
        *c = n1 + n2;
    }
};
```

The following example code demonstrates how to annotate a kernel argument of `struct` data type:

```
struct DataT{
    float f1;
    float f2;
};

struct MyIP {
    annotated_arg<DataT, decltype(properties{conduit})> inp;
    annotated_arg<float*, decltype(properties{conduit})> sum;
    void operator() () const {
        // Make local variables of the struct type from the kernel argument
        // before using the kernel argument
        DataT data = inp;
        // This allows access to struct members with the 'dot' operator
        *c = data.f1 + data.f2;
    }
};
```

The annotated_arg Template Class Properties Summary

Template Object or Parameter	Description	Valid Values
Interface Type Properties		
<code>sycl::ext::intel::experimental::conduit</code>	Create a dedicated input conduit on the kernel.	N/A

Template Object or Parameter	Description	Valid Values
<code>sycl::ext::intel::experimental::register_map</code>	Creates an input register map for the input instead of a dedicated input port.	N/A
<code>sycl::ext::intel::experimental::stable</code>	Specifies that the input to the kernel does not change between pipelined invocations of the kernel. The input can change after all active kernel invocations have finished.	N/A
Pointer Kernel Argument Properties		
<code>sycl::ext::intel::experimental::buffer_location</code>	The global memory identifier for the pointer interface. To use any of the other Avalon® MM Host interface properties, you must also specify the <code>buffer_location</code> property.	Non-negative integer value
<code>sycl::ext::intel::experimental::dwidth</code>	The width of the memory-mapped data bus in bits	8, 16, 32, 64, 128, 256, 512, or 1024 Default: 64
<code>sycl::ext::intel::experimental::awidth</code>	The width of the memory-mapped address bus in bits.	Integer 11 – 41 Default: 41
<code>sycl::ext::intel::experimental::latency</code>	The guaranteed latency from when a read command exits the kernel until the external memory returns valid read data. When a variable latency is specified by setting value of 0, a <code>waitrequest</code> signal is generated on the interface. When a fixed latency is specified by setting a non-zero value, a <code>waitrequest</code> signal is not generated on the interface.	Non-negative integer value Default: 0
<code>sycl::ext::intel::experimental::maxburst</code>	The maximum number of data transfers that can associate with a read or write transaction.	Integer 1 – 1024
<code>sycl::ext::oneapi::experimental::alignment</code>	The alignment of the base pointer address in bytes.	See description
<code>sycl::ext::intel::experimental::read_write_mode</code>	The port direction of the interface.	<code>read_write</code> , <code>read</code> , or <code>write</code> Default: <code>read_write</code>

`sycl::ext::intel::experimental::conduit` Declaration

Syntax	<code>sycl::ext::intel::experimental::conduit</code>
Description	Directs the compiler to create a dedicated input conduit on the kernel for the kernel arguments. That input conduit is associated with the kernel start and busy signals. This parameter is mutually exclusive with the <code>sycl::ext::intel::experimental::register_map</code> declaration.

sycl::ext::intel::experimental::register_map Declaration

Syntax	<code>sycl::ext::intel::experimental::register_map</code>
Description	Directs the compiler to create a register map to store the input. This declaration is mutually exclusive with the <code>sycl::ext::intel::experimental::conduit</code> declaration.

sycl::ext::intel::experimental::stable Declaration

Syntax	<code>sycl::ext::intel::experimental::stable</code>
Description	While the SYCL* software model makes kernel arguments read-only, the RTL IP core generated by the compiler can be plugged into external systems where kernel arguments can change while the kernel executes. This property specifies that the input to the kernel does not change between pipelined invocations of the kernel. The input can still change after all active kernel invocations have finished. If the input changes while the pipelined kernel invocations are executing, the behavior is undefined.

sycl::ext::intel::experimental::buffer_location Property

Syntax	<code>template <unsigned id>sycl::ext::intel::experimental::buffer_location<id></code>
Valid Values	Non-negative integer value
Description	Specifies a global memory identifier for the pointer interface. You must specify this argument to use any of the other Avalon® MM Host interface properties. Pointers with this argument specified are often referred to as <i>annotated pointers</i> . Each unique ID results in a separate Avalon® MM Host interface on your RTP IP core. All hosts with the same address space are arbitrated within the kernel to a single interface. As such, these hosts must share the same template parameters that describe the interface. If this property is not specified, the pointer argument is considered to be an unannotated pointer. If you mix annotated pointers (that is, pointers with specified buffer locations) and unannotated pointers, ensure that you review Mixing Annotated and Unannotated Pointers in Your Kernel to understand the implications of mixing pointer types. If all pointers in your kernel are unannotated, the compiler infers an Avalon® MM Host interface with a buffer location ID of 0.

Important The caller is responsible for ensuring the correct buffer location is specified; otherwise, functional failures might occur. If you specify a `buffer_location` property on your kernel argument, specify the same buffer location on the USM allocation API call that allocates memory on the test bench (that is, the host code).

Restriction If you use annotated pointers, you cannot mix `sycl::accessor` and `annotated_arg` kernel arguments.

`sycl::ext::intel::experimental::dwidth` Property

Syntax	<pre>template <unsigned width>sycl::ext::intel::experimental::dwidth<width></pre>
Valid Values	8, 16, 32, 64, 128, 256, 512, or 1024
Default Value	64
Description	The width of the memory-mapped data bus in bits. To specify the <code>dwidth</code> property, you must also specify the <code>buffer_location</code> property.

`sycl::ext::intel::experimental::awidth` Property

Syntax	<pre>template <unsigned width>sycl::ext::intel::experimental::awidth<width></pre>
Valid Values	Integer value in the range 11-41
Default Value	41
Description	The width of the address bus. This value affects only the width of the Avalon® MM host interface.

Restriction The lowest-numbered buffer location must have a minimum address width of 11-bits (`awidth<11>`). There is no minimum width for higher-numbered buffer locations.

To specify the `awidth` property, you must also specify the `buffer_location` property.

`sycl::ext::intel::experimental::latency` Property

Syntax	<pre>template <unsigned value>sycl::ext::intel::experimental::latency<value></pre>
Valid Values	Non-negative integer value
Default Value	0
Description	The guaranteed latency from when a read command exits the kernel to when the external memory returns valid read data. When a variable latency is specified by setting value of 0, a <code>waitrequest</code> signal is generated on the interface. When a fixed latency is specified by setting a non-zero value, a <code>waitrequest</code> signal is not generated on the interface.

If this latency is variable (such as when accessing DRAM), set it to 0.

To specify the `latency` property, you must also specify the `buffer_location` property.

`sycl::ext::intel::experimental::maxburst` Property

Syntax	<pre>template <unsigned value>sycl::ext::intel::experimental::maxburst<value></pre>
Valid Values	Integer value in the range 1 – 1024
Default Value	1
Description	The maximum number of data transfers that can associate with a read or write transaction. This value controls the width of the <code>burstcount</code> signal. For fixed latency interfaces, this value must be set to 1. For more details, review information about burst signals and the <code>burstcount</code> signal role in "Avalon Memory-Mapped Interface Signal Roles" in <i>Avalon Interface Specifications</i>. To specify the <code>maxburst</code> property, you must also specify the <code>buffer_location</code> property and set the <code>latency</code> property to 0 (<code>latency<0></code>).

`sycl::ext::oneapi::experimental::alignment` Property

Syntax	<pre>template <unsigned value>sycl::ext::oneapi::experimental::alignment<value></pre>
Valid Values	Integer value greater than the alignment of the data type. The value must be a power of 2.
Default Value	1
Description	The alignment of the base pointer address in bytes. The compiler uses this information to determine how many simultaneous loads and stores this pointer can permit. For example, if you have a bus with 4 32-bit integers on it, you should use <code>sycl::ext::intel::experimental::dwidth<128></code> (bits) and <code>sycl::ext::intel::experimental::align<16></code> (bytes). This means that up to 16 contiguous bytes (or 4 32-bit integers) can be loaded or stored as a coalesced memory word per clock cycle.

Important The caller is responsible for aligning the data to the set value for the `align` argument; otherwise, functional failures might occur. If you specify this property, ensure that you specify the same alignment value the following locations:

- The `sycl::ext::oneapi::experimental::alignment<value>` property
- The *alignment* argument of the `aligned_malloc_shared` template function in your host code. For example,

```
aligned_malloc_shared<int>(value, 1, q );
```

`sycl::ext::intel::experimental::read_write_mode` Property

Syntax	<pre>template <unsigned mode>sycl::ext::intel::experimental::read_write_mode<value></pre>
Valid Values	<code>read_write</code> , <code>read</code> , or <code>write</code>
Default Value	<code>readwrite</code>
Description	The port direction of the memory interface associated with the input pointer. Only the relevant Avalon host signals are generated. The available modes are as follows: • <code>read_write</code>: The interface can used for read-write operations. You can also specify the <code>sycl::ext::intel::experimental::read_write_mode_readwrite</code> property to enable this mode. • <code>read</code>: The interface is read-only. It be used only for read operations. You can also specify the <code>sycl::ext::intel::experimental::read_write_mode_read</code> property to enable this mode. • <code>write</code>: The interface is write-only. It can be used only for write operations. You can also specify the <code>sycl::ext::intel::experimental::read_write_mode_write</code> property to enable this mode. To specify the <code>read_write_mode</code> property, you must also specify the <code>buffer_location</code> property.

Memory-Mapped Host Interfaces Using Accessors

The following example shows how to create a memory-mapped interface using the `buffer_location` property in SYCL*:

```
#include <sycl/sycl.hpp>
#include <iostream>
#include <sycl/ext/intel/fpga_extensions.hpp>
#include <vector>

using namespace sycl;

// Forward declare the kernel name in the global scope.
```

```

// This is an FPGA best practice that reduces name mangling in the
// optimization reports.
class SimpleVAdd;

// The members of the functor serve as inputs and outputs to your IP.
// The code inside the operator()() function describes your IP.
template <class AccA, class AccB, class AccC>
struct SimpleVAddKernel {
    AccA A;
    AccB B;
    AccC C;
    int count;

    void operator()() const {
        for (int i = 0; i < count; i++) {
            C[i] = A[i] + B[i];
        }
    }
};

constexpr int VECT_SIZE = 4;

int main() {

    #if FPGA_SIMULATOR
        auto selector = sycl::ext::intel::fpga_simulator_selector_v;
    #elif FPGA_HARDWARE
        auto selector = sycl::ext::intel::fpga_selector_v;
    #else // #if FPGA_EMULATOR
        auto selector = sycl::ext::intel::fpga_emulator_selector_v;
    #endif

    queue q(selector);

    int count = VECT_SIZE; // pass array size by value

    // declare arrays and fill them
    std::vector<int> VA;
    std::vector<int> VB;
    std::vector<int> VC(count);
    for (int i = 0; i < count; i++) {
        VA.push_back(i);
        VB.push_back(count - i);
    }

    std::cout << "add two vectors of size " << count << std::endl;

    buffer bufferA{VA};
    buffer bufferB{VB};
    buffer bufferC{VC};

    q.submit([&](handler &h) {
        accessor accessorA{bufferA, h, read_only};
        accessor accessorB{bufferB, h, read_only};
        accessor accessorC{bufferC, h, read_write, no_init};

        h.single_task<SimpleVAdd>(SimpleVAddKernel<decltype(accessorA), decltype(accessorB),
decltype(accessorC)>{accessorA, accessorB, accessorC, count});
    });
}

```

```
});

// verify that VC is correct
bool passed = true;
for (int i = 0; i < count; i++) {
    int expected = VA[i] + VB[i];
    std::cout << "idx=" << i << ": result " << VC[i] << ", expected (" << expected << ")
VA=" << VA[i] << " + VB=" << VB[i] << std::endl;
    if (VC[i] != expected) {
        passed = false;
    }
}

std::cout << (passed ? "PASSED" : "FAILED") << std::endl;
return passed ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

Structs in RTL IP Core Interfaces

Review the `<kernel_name>_interface_structs.sv` file in your `<project_name>.prj/` folder to see information about the padding and packed-ness of the implementation interfaces for the structs in your RTL IP core.

The `<kernel_name>_interface_structs.sv` file contains the Verilog-style definitions of the structs found on the RTL IP core interface.

IP core Reset Behavior

For your IP core, the reset assertion can be asynchronous but the reset deassertion must be synchronous.

The reset assertion and deassertion behavior can be generated from an asynchronous reset input by using reset synchronization. Add reset synchronization to your IP core with Platform Designer when you integrate your IP into a system.

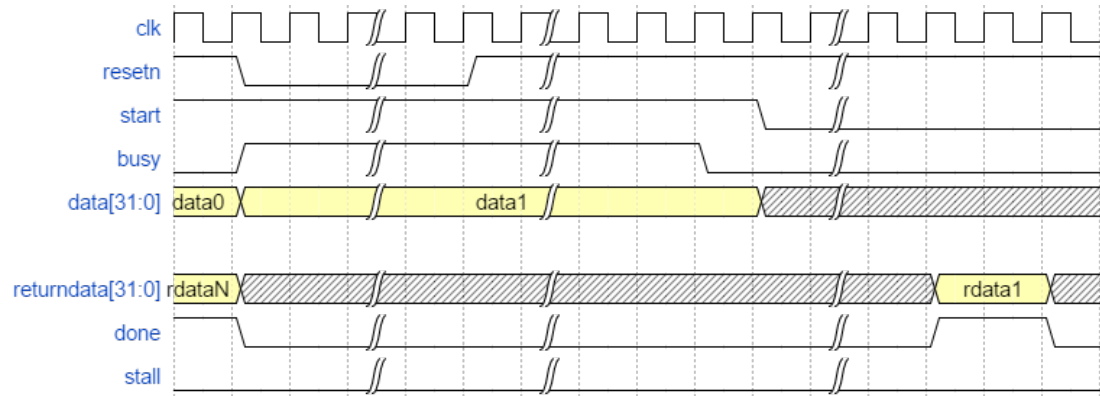
For information about integrating your IP core into a system, refer to [Integrating Your IP Into a System](#).

For an example of adding reset synchronization, refer to the [Platform Designer Sample example design on GitHub](#).

Synchronizers are implemented to minimize synchronization failures due to metastability in asynchronous signal transfers. For more information about metastability, refer to [“Managing Metastability with the Quartus® Prime Software” in Quartus® Prime Pro Edition User Guide: Design Recommendations](#).

When the reset is asserted, the IP core holds its busy signal high and its done signal low. After the reset is deasserted, the IP core holds its busy signal high until the IP core is ready to accept the next invocation. All IP core interfaces (agents, hosts, and streams) are ready only after the IP core busy signal is low.

Example Waveform Showing IP Core Reset Behavior



Loops

The Intel® oneAPI DPC++/C++ Compiler attempts to maximize the occupancy of the datapath of a loop within a task kernel by executing iterations in a pipeline parallel method. The following sections provide guidelines and describe techniques for writing loops in task kernels such that the Intel® oneAPI DPC++/C++ Compiler can best extract pipeline parallelism from these loops.

Refactor the Loop-Carried Data Dependency

Based on the feedback from the optimization report, you can restructure your program to reduce the critical path of a pipelined loop by reducing the distance between the first time you use a loop-updated variable in the loop body and the last time you define it within a single iteration.

Consider the following code:

```
constexpr int N = 128;
queue.submit([&](handler &cgh) {
    accessor A(A_buf, cgh, read_only);
    accessor B(B_buf, cgh, read_only);
    accessor Result(Result_buf, cgh, write_only);
    cgh.single_task<class unoptimized>([=]() {
        int sum = 0;
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                sum += A[i * N + j];
            }
            sum += B[i];
        }
        Result[0] = sum;
    });
});
```

- The report indicates that the Intel® oneAPI DPC++/C++ Compiler successfully infers pipelined execution for the outer loop, and a new loop iteration launches every other cycle.
- The Only a single loop iteration will execute... message in the Details pane indicates that the loop executes a single iteration at a time across the subloop because of the data dependency on the variable `sum`. This data dependency exists because each outer loop iteration requires the value of `sum` from the previous iteration to return before the inner loop can start executing. Moreover, the serialization is enforced by a critical path spanning from the first use of `sum` in the body of the loop `i` to the last definition of `sum` at the end of the body of loop `j`.
- The report also notifies you that the inner loop executes in a pipelined manner with no performance-limiting loop-carried dependencies.

NOTE

For recommendations on how to structure your single work-item kernel, refer to [Single Work-item Kernel Design Guidelines](#).

To optimize performance of this kernel, reduce the length of the critical path induced by the data dependency on variable `sum` so that the outer loop iterations do not execute serially across the subloop. Perform the following tasks to decouple the computations involving `sum` in the two loops:

1. Define a local variable (for example, `sum2`) for use in the inner loop only.
2. Use the local variable (`sum2`) to store the cumulative values of `A[i * N + j]` as the inner loop iterates.

3. In the outer loop, store the cumulative values of `B[i]` and `sum2` in the `sum` variable.

The following code illustrates the restructured optimized kernel snippet:

```
constexpr int N = 128;
queue.submit([&](handler &cgh) {
    accessor A(A_buf, cgh, read_only);
    accessor B(B_buf, cgh, read_only);
    accessor Result(Result_buf, cgh, write_only);
    cgh.single_task<class optimized>([=]() {
        int sum = 0;
        for (int i = 0; i < N; i++) {
            // Step 1: Definition
            int sum2 = 0;
            // Step 2: Accumulation of array A values for one outer
            // loop iteration
            for (int j = 0; j < N; j++) {
                sum2 += A[i * N + j];
            }
            // Step 3: Addition of array B value for an outer loop iteration
            sum += sum2;
            sum += B[i];
        }
        Result[0] = sum;
    });
});
```

Relax Loop-Carried Dependency

Example 1: Multiply Chain

Based on the feedback from the optimization report, you can relax a loop-carried dependency by allowing the compiler to infer a shift register and increase the dependence distance. Increase the dependence distance by increasing the number of loop iterations that occur between the generation of a loop-carried value and its use.

Consider the following code example:

```
constexpr int N = 128;
queue.submit([&](handler &cgh) {
    accessor A(A_buf, cgh, read_only);
    accessor result(result_buf, cgh, write_only);
    cgh.single_task<class unoptimized>([=]() {
        float mul = 1.0f;
        for (int i = 0; i < N; i++)
            mul *= A[i];

        result[0] = mul;
    });
});
```

The optimization report shows that the Intel® oneAPI DPC++/C++ Compiler infers pipelined execution for the loop successfully. However, the loop-carried dependency on the variable `mul` causes loop iterations to launch every six cycles. In this case, the floating-point multiplication operation on line 8 (that is, `mul *= A[i]`) contributes the largest delay to the computation of the variable `mul`. The latency through the floating point multiplier means you have to wait for the multiply to complete (6 cycles) before feeding the output back into the input with `A[i]`.

To relax the loop-carried data dependency, instead of using a single variable to store the multiplication results, operate on M copies of the variable, and use one copy every M iterations:

1. Declare multiple copies of the variable `mul` (for example, in an array called `mul_copies`).
2. Initialize all copies of `mul_copies`.
3. Use the last copy in the array in the multiplication operation.
4. Perform a shift operation to pass the last value of the array back to the beginning of the shift register.
5. Sum up all copies of `mul_copies` to `mul` and write the final value to the result.

The following code illustrates the restructured kernel:

```
constexpr int N = 128;
constexpr int M = 8;
queue.submit([&](handler &cgh) {
    accessor A(A_buf, cgh, read_only);
    accessor result(result_buf, cgh, write_only);
    cgh.single_task<class optimized>([=]() {
        float mul = 1.0f;

        // Step 1: Declare multiple copies of variable mul
        float mul_copies[M];

        // Step 2: Initialize all copies
        for (int i = 0; i < M; i++)
            mul_copies[i] = 1.0f;

        for (int i = 0; i < N; i++) {
            // Step 3: Perform multiplication on the last copy
            float cur = mul_copies[M-1] * A[i];

            // Step 4a: Shift copies
            #pragma unroll
            for (int j = M-1; j > 0; j--)
                mul_copies[j] = mul_copies[j-1];

            // Step 4b: Insert updated copy at the beginning
            mul_copies[0] = cur;
        }

        // Step 5: Perform reduction on copies
        #pragma unroll
        for (int i = 0; i < M; i++)
            mul *= mul_copies[i];

        result[0] = mul;
    });
});
```

Example 2: Accumulator

Similar to Example 1, this example also applies the same technique and demonstrates how to write single work-item kernels that carry out double precision floating-point operations efficiently by inferring a shift register.

Consider the following example:

```
queue.submit([&](handler &cgh) {
    accessor arr(arr_buf, cgh, read_only);
    accessor result(result_buf, cgh, write_only);
    accessor N(N_buf, cgh, read_only);
```



```

cgh.single_task<class unoptimized>([=]() {
    double temp_sum = 0;
    for (int i = 0; i < *N; ++i)
        temp_sum += arr[i];
    result[0] = temp_sum;
});
});

```

The kernel `unoptimized` is an accumulator that sums the elements of a double precision floating-point array `arr[i]`. For each loop iteration, the Intel® oneAPI DPC++/C++ Compiler takes 10 cycles to compute the result of the addition and then stores it in the variable `temp_sum`. Each loop iteration requires the value of `temp_sum` from the previous loop iteration, which creates a data dependency on `temp_sum`. To relax the data dependency, infer the array `arr[i]` as a shift register.

The following is the restructured kernel `optimized`:

```

//Shift register size must be statically determinable
constexpr int II_CYCLES = 12;
queue.submit([&](handler &cgh) {
    accessor arr(arr_buf, cgh, read_only);
    accessor result(result_buf, cgh, write_only);
    accessor N(N_buf, cgh, read_only);
    cgh.single_task<class optimized>([=]() {
        //Create shift register with II_CYCLE+1 elements
        double shift_reg[II_CYCLES+1];

        //Initialize all elements of the register to 0
        for (int i = 0; i < II_CYCLES + 1; i++) {
            shift_reg[i] = 0;
        }

        //Iterate through every element of input array
        for(int i = 0; i < N; ++i){
            //Load ith element into end of shift register
            //if N > II_CYCLE, add to shift_reg[0] to preserve values
            shift_reg[II_CYCLES] = shift_reg[0] + arr[i];

            #pragma unroll
            //Shift every element of shift register
            for(int j = 0; j < II_CYCLES; ++j)
            {
                shift_reg[j] = shift_reg[j + 1];
            }
        }

        //Sum every element of shift register
        double temp_sum = 0;

        #pragma unroll
        for(int i = 0; i < II_CYCLES; ++i)
        {
            temp_sum += shift_reg[i];
        }

        result[0] = temp_sum;
    });
});

```

Transfer Loop-Carried Dependency to Local Memory

Loop-carried dependencies can adversely affect the loop initiation interval or II (refer to [Pipelining Across Multiple Work Items](#)). For a loop-carried dependency that you cannot remove, improve the II by moving the array with the loop-carried dependency from global memory to local memory.

Consider the following example:

```
constexpr int N = 128;
queue.submit([&](handler &cgh) {
    accessor A(A_buf, cgh, read_write);
    cgh.single_task<class unoptimized>([=]() {
        for (int i = 0; i < N; i++)
            A[N-i] = A[i];
    });
});
```

Global memory accesses have long latencies. In this example, the loop-carried dependency on the array `A[i]` causes long latency. The optimization report reflects this latency with an II of 185. Perform the following tasks to reduce the II value by transferring the loop-carried dependency from global memory to local memory:

1. Copy the array with the loop-carried dependency to local memory. In this example, array `A[i]` becomes array `B[i]` in local memory.
2. Execute the loop with the loop-carried dependence on array `B[i]`.
3. Copy the array back to global memory.

When you transfer array `A[i]` to local memory and it becomes array `B[i]`, the loop-carried dependency is now on `B[i]`. Because local memory has a much lower latency than global memory, the II value improves.

Following is the restructured kernel optimized:

```
constexpr int N = 128;
queue.submit([&](handler &cgh) {
    accessor A(A_buf, cgh, read_write);
    cgh.single_task<class optimized>([=]() {
        int B[N];
        for (int i = 0; i < N; i++)
            B[i] = A[i];

        for (int i = 0; i < N; i++)
            B[N-i] = B[i];

        for (int i = 0; i < N; i++)
            A[i] = B[i];
    });
});
```

Minimize the Memory Dependencies for Loop Pipelining

Intel® oneAPI DPC++/C++ Compiler ensures that the memory accesses from the same thread respects the program order. When you compile an NDRange kernel, use barriers to synchronize memory accesses across threads in the same workgroup.

Loop dependencies might introduce bottlenecks for single work-item kernels due to latency associated with the memory accesses. The Intel® oneAPI DPC++/C++ Compiler defers a memory operation until a dependent memory operation completes. This could affect the loop initiation interval (II). The Intel® oneAPI DPC++/C++ Compiler indicates the memory dependencies in the optimization report.

To minimize the impact of memory dependencies for loop pipelining:

- Ensure that the Intel® oneAPI DPC++/C++ Compiler does not assume false dependencies.
- When the static memory dependence analysis fails to prove that dependency does not exist, the Intel® oneAPI DPC++/C++ Compiler assumes that a dependency exists and modifies the kernel execution to enforce the dependency. The impact of the dependency enforcement is lower if the memory system is stall-free.
 - Write-after-read operations with data dependency on a load-store unit can take just two clock cycles (II=2). Other stall-free scenarios can take up to seven clock cycles.
 - The Intel® oneAPI DPC++/C++ Compiler can fully resolve the read-after-write (control dependency) operation.
- Override the static memory dependence analysis by adding the line `[[intel::ivdep]]` before the loop in your kernel code if you are sure that it carries no dependencies. For more information, refer to [ivdep Attribute](#)

Unroll Loops

You can control the way the Intel® oneAPI DPC++/C++ Compiler translates SYCL kernel descriptions to hardware resources by unrolling loops. Loop unrolling decreases the number of iterations that the Intel® oneAPI DPC++/C++ Compiler executes at the expense of increased hardware resource consumption and increases performance. See also [Unroll Loops](#)

Consider the SYCL code for a parallel application in which each work-item is responsible for computing the accumulation of four elements in an array:

```
queue.submit([&](handler &cgh) {
    accessor x(x_buf, cgh, read_only);
    accessor sum(sum_buf, cgh, write_only);
    cgh.single_task<class unoptimized>([=]() {
        int accum = 0;
        for (size_t i = 0; i < 4; i++) {
            accum += x[i + get_global_id(0) * 4];
        }
        sum[get_global_id(0)] = accum;
    });
});
```

Observe the following three main operations that occur in this kernel:

- Load operations from input `x`
- Accumulation
- Store operations to output `sum`

The Intel® oneAPI DPC++/C++ Compiler arranges these operations in a pipeline according to the data flow semantics of the SYCL* kernel code. For example, the Intel® oneAPI DPC++/C++ Compiler implements loops by forwarding the results from the end of the pipeline to the top of the pipeline, depending on the loop exit condition.

The SYCL kernel performs one loop iteration of each work-item per clock cycle. With sufficient hardware resources, you can increase kernel performance by unrolling the loop, which decreases the number of iterations that the kernel executes. To unroll a loop, add a `#pragma unroll` directive to the main loop, as shown in the following code example:

NOTE

Loop unrolling significantly changes the structure of the compute unit that the Intel® oneAPI DPC++/C++ Compiler creates.

```
queue.submit([&](handler &cgh) {
    accessor x(x_buf, cgh, read_only);
    accessor sum(sum_buf, cgh, write_only);
    cgh.single_task<class unoptimized>([=]() {
```

```

int accum = 0;

#pragma unroll
for (size_t i = 0; i < 4; i++) {
    accum += x[i + get_global_id(0) * 4];
}
sum[get_global_id(0)] = accum;
});
});

```

In this example, the `#pragma unroll` directive causes the Intel® oneAPI DPC++/C++ Compiler to unroll four iterations of the loop completely. To accomplish the unrolling, the Intel® oneAPI DPC++/C++ Compiler expands the pipeline by tripling the number of addition operations and loading four times more data. With the removal of the loop, the compute unit assumes a feed-forward structure. As a result, the compute unit can store the `sum` elements in every clock cycle after the completion of the initial load operations and additions. The Intel® oneAPI DPC++/C++ Compiler further optimizes this kernel by coalescing the four load operations so that the compute unit can load all necessary input data to calculate a result in one load operation.

Factors to Consider for Loop Unrolling

- Avoid nested looping structures. Instead, implement a large single loop or unroll inner loops by adding the `#pragma unroll` directive whenever possible. For example, if you compile a kernel that has a heavily nested loop structure, wherein each loop includes a `#pragma unroll` directive, you might experience a long compilation time. The Intel® oneAPI DPC++/C++ Compiler might fail to meet scheduling because it cannot unroll this nested loop structure easily, resulting in a high II. In this case, the Intel® oneAPI DPC++/C++ Compiler issues the following error message along with the line number of the outermost loop:

```

Kernel <function> exceeded the Max II. The Kernel's resource usage is estimated to be
much larger than FPGA capacity. It will perform poorly
even if it fits. Reduce resource utilization of the kernel by reducing loop unroll
factors within it (if any) or otherwise reduce amount of
computation within the kernel.

```

- Unrolling the loop and coalescing load operations from global memory allow the hardware implementation of the kernel to perform more operations per clock cycle.
- The Intel® oneAPI DPC++/C++ Compiler might not be able to unroll a loop completely under the following circumstances:
 - You specify complete unrolling of a data-dependent loop with a very large number of iterations. Consequently, the hardware implementation of your kernel might not fit into the FPGA.
 - You specify complete unrolling and the loop bounds are not constants.
 - The loop consists of complex control flows (for example, a loop containing complex array indexes or exit conditions that are unknown at compilation time).

For the last two cases listed above, the Intel® oneAPI DPC++/C++ Compiler issues the following warning:

```

Full unrolling of the loop is requested but the loop bounds cannot be determined. The
loop is not unrolled.

```

To enable loop unrolling in these situations, specify the `#pragma unroll <N>` directive, where `<N>` is the unroll factor. The unroll factor limits the number of iterations that the Intel® oneAPI DPC++/C++ Compiler unrolls. Refer to [Single Work-item Kernel Design Guidelines](#) for tips on constructing well-structured loops.

Tip

For additional information, refer to the FPGA tutorial sample "Loop Unroll" on [GitHub](#).

Fuse Loops to Reduce Overhead and Improve Performance

Loop fusion is a compiler transformation in which two adjacent loops are merged into a single loop over the same index range. This transformation is typically applied to reduce loop overhead and improve run-time performance.

The following example shows the effects of fusing loops in a simple case:

Unfused Loops	Fused Loops
<pre>for (int i = 0 ; i < 10; i++) { for(int j = 0 ; j < 300 ; j++){ a[j] = localMem[j] + 3; } for(int k = 0 ; k < 300 ; k++){ b[k] = localMem[k] + 4; } }</pre>	<pre>for (int i = 0; i < 10; i++) { for(int j = 0 ; j < 300 ; j++){ int localMemVal = localMem[j]; a[j] = localMemVal + 3; b[j] = localMemVal + 4; } }</pre>

Loop control structures represent a significant overhead. By fusing two loops, the number of control structures needed for the loops is reduced from two to one, reducing this overhead. The main goal of reducing the number of control structures is to save FPGA area for your design while still maintaining (ideally increasing) kernel throughput.

Fusing outer loops introduces concurrency where there was previously none. Combining bodies of two adjacent loops (L_j and L_k) forms a single loop (L_f) with a loop body that spans the bodies of L_j and L_k . This combined loop body creates an opportunity for operations that were serialized across a given iteration of L_j and L_k to execute concurrently. In effect, the two loops now execute as one, reducing latency.

If inner loops are fused, concurrency is already achieved by pipelined execution of the outer loop iteration. In these cases, the concurrency effect of loop fusion is diminished.

Fusion Criteria

The compiler considers the fusion of two loops (L_j and L_k) to be valid if the loops meet the following criteria:

- Loops must be adjacent. That is, you cannot have a statement S_i with side-effects such that S_i executes after L_j and before L_k .
- Each loop must have a single-entry point and a single exit point. For example, loops that contain break statements are not considered for fusion.
- Loops must have no negative-distance dependencies. That is, for loops L_j and L_k where L_j is defined before L_k , iteration m of loop L_k does not depend on values calculated in iteration $m+n$ (where $n>0$) of loop L_j .

Automatic Loop Fusion

The Intel® oneAPI DPC++/C++ Compiler fuses loops with the same trip counts automatically if the compiler analysis of your kernel determines that fusing the loops is profitable.

Examples of where fusing loops is a valid transformation (based on the earlier criteria) but are not considered profitable by the compiler include the following situations:

- One of the two loops, but not both, is annotated with the `ivdep` attribute.
- One of the two loops, but not both, contains stall-free logic.

The [Loop Analysis report](#) indicates when loops were fused.

Tip

For additional information, refer to the FPGA tutorial sample "Loop Fusion" on [GitHub](#).

Optimize Loops With Loop Speculation

Loop speculation is an optimization technique that enables more efficient loop pipelining by allowing future iterations to be initiated before determining whether the loop was exited already. Consider the following simple loop example:

```
while (m*m*m < N) {
    m+=1;
}
```

Logically, the exit condition ($m*m*m < N$) for an iteration must be evaluated before determining whether you need to initiate another iteration or not. This means that, in the absence of speculation, the loop II cannot be lower than the number of cycles it takes to compute this exit condition. Speculated iterations are iterations that launch before the exit condition computation has completed. However, all operations with side-effects, such as stores to memory, are predicated by the exit condition. This means that operations with side-effects still wait for the exit condition to be computed. Loop speculation is beneficial when the exit condition is the bottleneck preventing from achieving a lower II. In the loop shown above, the exit condition contains two multiplications that cannot complete within a single clock cycle. However, loop speculation allows this loop to achieve II=1.

For example, for a given iteration i with exit condition E_i , the number of speculated iterations s is the number of iterations after i has been initiated but before E_i has been evaluated. By default, this number of speculated iterations is determined by the compiler on a per-loop basis, and can be found in the per-loop details of the [Loop Analysis](#) report.

The `speculated_iterations` attribute allows you to directly control the number of speculated iterations for a loop. If the exit condition calculation is the bottleneck to lowering II (as shown in the [Loop Analysis](#) report), increasing the number of speculated iterations may improve the II (this is not guaranteed as other bottlenecks may be uncovered). For more information, refer to [speculated_iterations Attribute](#).

Speculated iterations introduce some overhead in nested loops since a new invocation of a loop may not begin until all speculated iterations of its previous invocation have completed. In cases where a loop body with low latency is expected to be frequently invoked, (for example, an inner loop with a short trip count), use the `speculated_iterations` attribute to reduce the number of speculated iterations. You can estimate the amount of this overhead by multiplying the number of speculated iterations with II of the loop (as shown in the [Loop Analysis](#) report). Using the `speculated_iterations` attribute can reduce this overhead, but be aware that choosing an attribute value that is too low may increase the II (due to not having enough time to evaluate the exit condition).

Consider the following example:

```
while (m*m*m < N) {
    m+=1;
}
dst[0] = m;
```

In this example, the exit condition that has two multiplies and a compare is the bottleneck preventing II=1. The compiler's choice of four speculated iterations result in II=2 since the exit condition takes seven cycles (each multiply takes three cycles and the compare takes one cycle) and four speculated iterations times two-cycle II gives eight cycles to cover this evaluation.

```
[[intel::speculated_iterations(7)]]
while (m*m*m < N) {
    m+=1;
}
dst[0] = m;
```

Then, the speculated iterations are increased to seven to cover the seven-cycle exit condition calculation allowing us to achieve $II=1$.

```
[[intel::speculated_iterations(0)]]
while (m*m*m < N) {
    m+=1;
}
dst[0] = m;
```

By setting the `speculated_iterations` attribute to 0, you can verify that the II has increased to 7, which matches the exit condition bottleneck.

NOTE

For additional information, refer to the FPGA tutorial sample "Speculated Iterations" on [GitHub](#).

Remove Loop Bottlenecks

Bottlenecks in a loop means that one or more loop carried dependencies caused the compiler to either reduce the number of data items to be processed concurrently (in the same clock cycle) or reduce f_{MAX} . Bottlenecks occur only on single work-item kernels and are always created for loops.

Before analyzing the throughput of a simple loop, it is important to understand the concept of dynamic initiation interval. The initiation interval (II) is the statically determined number of cycles between successive iteration launches of a given loop invocation. However, the statically scheduled II may differ from the actual realized dynamic II when considering interleaving.

NOTE

Interleaving allows the iterations of more than one invocation of a loop to execute in parallel, provided that the static II of that loop is greater than 1. By default, the maximum amount of interleaving for a loop is equal to the static II of that loop.

In the presence of interleaving, the dynamic II of a loop can be approximated by the static II of the loop divided by the degree of interleaving, that is, by the number of concurrent invocations of the loop that are in flight.

Simple Loop Example

In a simple loop, the maximum number of data items that can be processed concurrently (also known as maximum concurrency) can be expressed as:

$\text{Concurrency}_{MAX} = (\text{Block latency} \times \text{Maximum interleaving iterations}) / \text{Initiation Interval}$

Consider a simple loop with memory dependency from the [Triangular Loops](#) example:

```
for (int y = x + 1; y < n; y++) {
    local_buf[y] = local_buf[y] + SomethingComplicated(local_buf[x]);
}
```

The Loop Analysis report displays the following for the simple loop:

Loop Analysis ☐ Show blocks

Name	Source Location	Pipelined	Block Scheduled II	Block Scheduled fMAX	Latency	Speculated Iterations	Max Interleaving Iterations	Brief Info
Task.B7	triangular_loop.cpp:77	Yes	30	240.00	36.000000	1	1	Memory dependency

The `for` loop on line 77 has a latency of 36, maximum interleaving iterations of 1, and initiation interval of 30. So, the maximum concurrency is ~ 1 ((Latency of 36 x Maximum interleaving iteration of 1) / II of 30). The bottleneck causing this low maximum concurrency results from a loop carried dependency caused by a memory dependency. This is reported in the [Bottlenecks Viewer](#) as shown in the following image:

The screenshot displays the Loop Analysis tool. The 'Loop List' on the left shows a hierarchy from System to Kernel: Task (triangular_loop.cpp) to Task.B7. The 'Loop Analysis' table shows Task.B7 with a latency of 36, block scheduled fMAX of 240.00, and a memory dependency. The 'Bottlenecks' section on the left shows a throughput bottleneck for 'local_buf (in Task.B7)'. The 'Details' panel on the right provides information about the variable on the critical loop feedback path, 'local_buf', including its declaration at 'triangular_loop.cpp:65', its dependency on 'Memory dependency', and its estimated fmax reduced to '240.0' due to an 'LD(triangular_loop.cpp:78)' instruction.

Another type of loop carried dependency is a data dependency, as shown in the following example:

```
int res = N;
for (int i = 0; i < N; i++) {
    res += 1;
    res ^= i;
}
```

Nested Loop Example

In a nested loop, the maximum concurrency is more difficult to compute. For example, the loop carried dependency in a nested loop does not necessarily affect the initiation interval of the outer loop. Additionally, a nested loop often requires the knowledge of the inner loop's trip count. Consider the [Triangular Loops](#) example:

```
void TriangularLoop(std::unique_ptr<queue>& q, buffer<uint32_t>& input_buf,
                   buffer<uint32_t>& output_buf, uint32_t n, event& e,
                   bool optimize) {

    e = q->submit([&](handler& h) {

        accessor input(input_buf, h, read_only);
        accessor output(output_buf, h, write_only, no_init);
```



```

h.single_task<Task>([=] () [[intel::kernel_args_restrict]] {

    const int real_iterations = (n * (n + 1) / 2 - 1);
    const int extra_dummy_iterations = (kM - 2) * (kM - 1) / 2;
    const int loop_bound = real_iterations + extra_dummy_iterations;

    uint32_t local_buf[kSize];

    for (uint32_t i = 0; i < kSize; i++) {
        local_buf[i] = input[i];
    }

    if (!optimize) { // Unoptimized loop.

        for (int x = 0; x < n; x++) {
            for (int y = x + 1; y < n; y++) {local_buf[y] = local_buf[y] +
SomethingComplicated(local_buf[x]);
            }
        }
    }
}

```

In this example, the bottleneck is resulted from a loop carried dependency caused by a memory dependency on the variable `local_buf`. The `local_buf` variable must finish loading in the loop on line 25 before the next outer loop (line 24) iteration is launched. Therefore, the maximum concurrency of the outer loop is 1. This information is reported in the details sections of the [Loop Analysis](#) and [Schedule Viewer](#) reports.

Addressing Bottlenecks

To address bottlenecks, primarily **consider restructuring your design code**.

After restructuring, consider applying the following loop attributes on arrays:

- `[[intel::initiation_interval(n)]]` (See [initiation_interval Attribute](#))
- `[[intel::ivdep(safelen)]]` (See [ivdep Attribute](#))
- `[[intel::max_concurrency(n)]]` (See [max_concurrency Attribute](#))
- `[[intel::private_copies(N)]]` (See [Memory Attributes](#))

Consider the previous [Simple Loop Example](#) where the concurrency is 1 and the initiation interval is 30. Applying the `[[intel::ivdep(M)]]` attribute, as shown in the following code snippet, comes at an expense of lowered predicted f_{MAX} from 240 MHz to 194.4 MHz. The original nested loop is manually coalesced or merged into a single loop. Since the loop is restructured to ensure that a minimum of M iterations is executed, the `[[intel::ivdep(M)]]` is used to inform the compiler that at least M iterations always separate any pair of dependent iterations. This new modified loop now has a maximum concurrency of 35 ($(\text{Latency of } 35 \times \text{Maximum interleaving iteration of } 1) / \text{II of } 1)$ and an initiation interval of 1.

```

// Indices to track the execution in the merged loop
int x = 0, y = 1;

// Total iterations of the merged loop
const int loop_bound = TotalIterations(M, n);

[[intel::ivdep(M)]]
for (int i = 0; i < loop_bound; i++) {

    // Determine if this is a real or dummy iteration
    bool compute = y > x;
    if (compute) {
        local_buf[y] = local_buf[y] + SomethingComplicated(local_buf[x]);
    }
}

```

```

y++;
if (y == n) {
    x++;
    y = Min(n - M, x + 1);
}
}

```

Improve f_{MAX}/II with Shannonization

Shannonization (named after [Claude Shannon](#)) is a loop optimization technique that improves the f_{MAX}/II of a design by precomputing operations in a loop and removing the operations from the critical path. To demonstrate shannonization, consider the following example, which counts the number of elements in an array *A* that are less than some runtime value *v*:

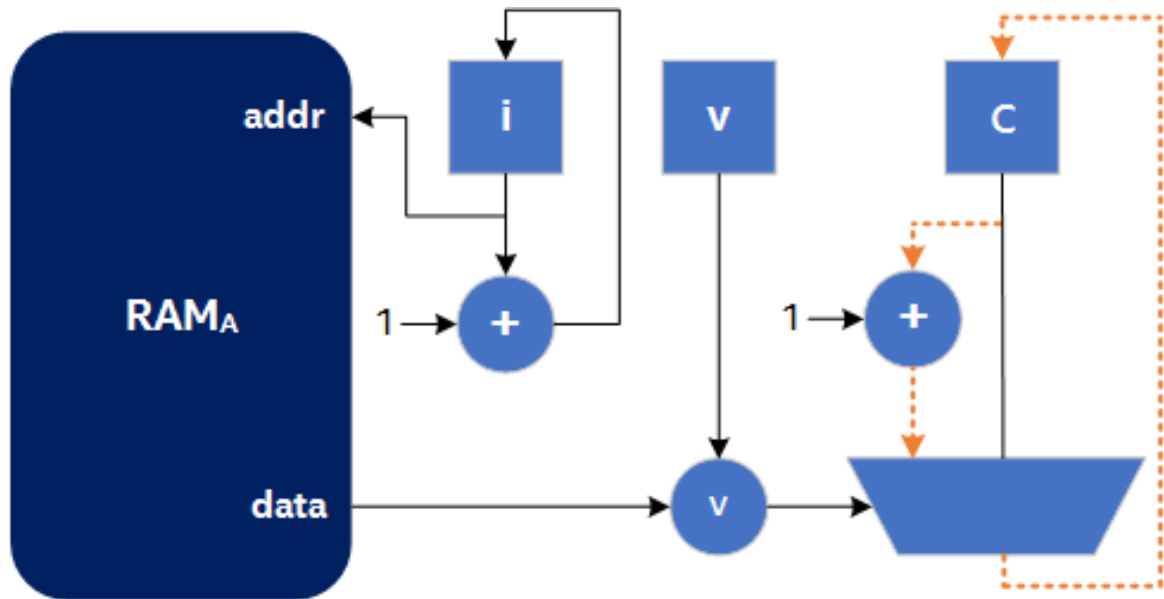
```

int A[SIZE] = { /*...*/ };
int v = /*some dynamic value*/
int c = 0;
for (int i = 0; i < SIZE; i++) {
    if (A[i] < v) {
        c++;
    }
}

```

A possible circuit diagram for this algorithm is shown in the following image, where the orange dotted line represents a possible critical path in the circuit:

Circuit Diagram for the Example Shannonization Algorithm



The goal of the shannonization optimization is to remove operations from the critical path. In this case, precompute the next value of *c* (fittingly named *c_{next}*) for a later iteration of the loop to use when required (that is, the next time *A*[*i*] < *v*). This optimization is shown in the following code sample:

```

int A[SIZE] = { /*...*/ };
int v = /*some dynamic value*/
int c = 0;
int c_next = 1;

```

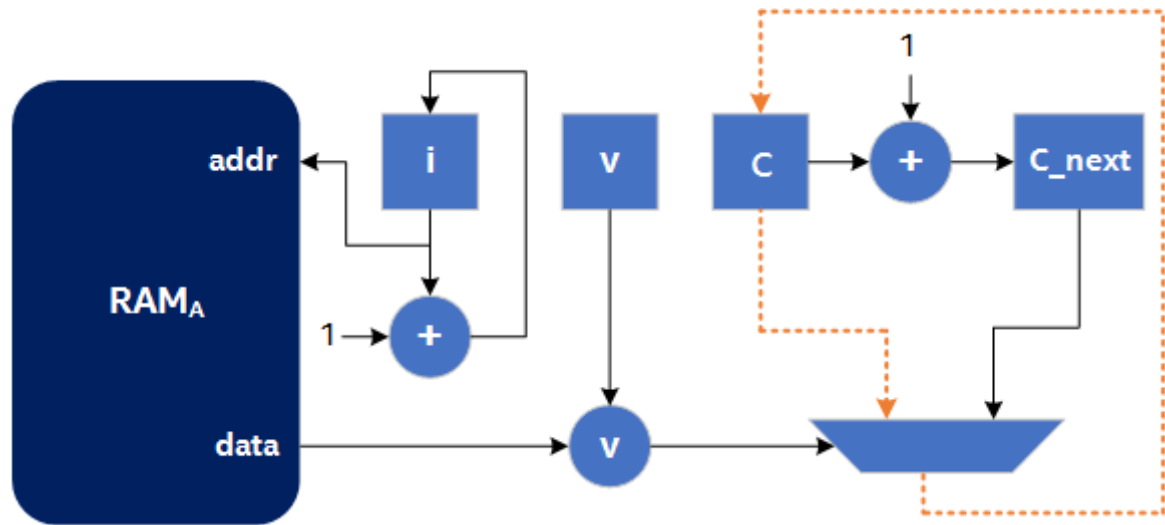
```

for (int i = 0; i < SIZE; i++) {
    if (A[i] < v) {
        // these operations can happen in parallel!
        c = c_next;
        c_next++;
    }
}

```

A possible circuit diagram for this optimized algorithm is shown in the following image, where the orange dotted line represents a possible critical path in the circuit:

Circuit Diagram for the Optimized Shannonization Algorithm



In the [Figure 88: Circuit Diagram for the Optimized Shannonization Algorithm](#), the + operation from the critical path is removed. This diagram assumes that the critical path delay through the multiplexer is higher than through the adder. This may not be the case, and the critical path could be from the `c` register to the `c_next` register through the adder, in which case, the multiplexer from the critical path is removed. Regardless of which operation has the longer critical path delay (the adder or the multiplexer), an operation from the critical path is removed by precomputing and storing the next value of `c`. This allows in reducing the critical path delay at the expense of area (in this case, a single 32-bit register).

NOTE

For additional information, refer to the FPGA tutorial sample "Shannonization" on [GitHub](#).

Optimize Inner Loop Throughput

In this topic, you learn how to optimize throughput of an inner loop with a low trip count. A *low* trip count is relative. For understanding the concept, consider *low* to be on the order of 100 or fewer iterations.

NOTE

Prior to reading the rest of this topic, you must review [Maximum Frequency \(fMAX\)](#) and [Optimize Loops With Loop Speculation](#).

Consider the following example code snippet:

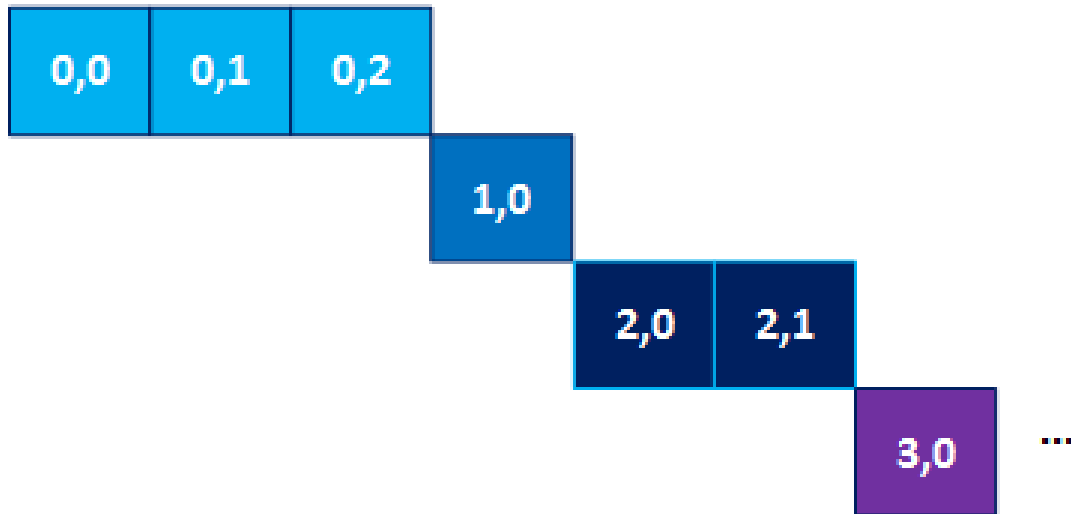
```
for (int i; i < kOuterLoopBound; i++) {
    int inner_loop_iterations = InputPipe::read() % kInnerLoopBound;
    for (int j; j < inner_loop_iterations; j++) {
        /* ... */
    }
}
```

Before you optimize the inner loops with low trip counts, assume the following:

- `kOuterLoopBound` is a number greater than one million.
- `kInnerLoopBound` is 3. Because the value of `InputPipe::read()` is dynamic, we know the value of `inner_loop_iterations` is in the range (0, 3).
- II of the inner loop is 1, which means that a new inner loop iteration can start every cycle. This means that the outer loop II is dynamic and depends on how many inner loop iterations the previous outer loop iteration must start.

A possible timing diagram for this loop structure is shown in the following figure, where the numbers in the squares are the values of `i` and `j`, respectively:

Timing Diagram for the Loop



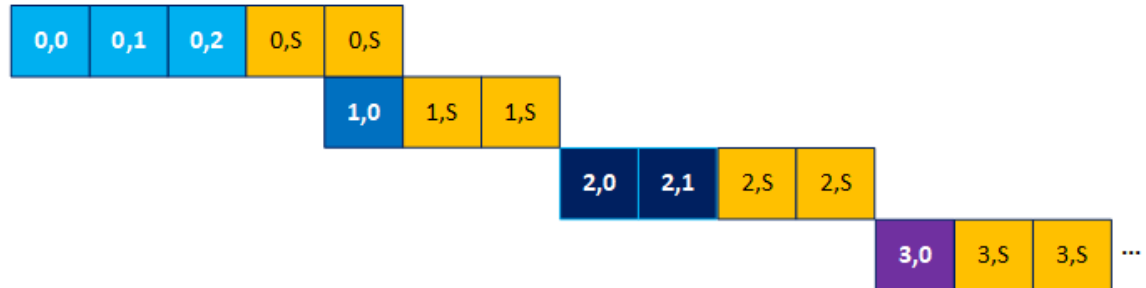
In general, the Intel® oneAPI DPC++/C++ Compiler optimizes loops for throughput with the assumption that the loop has a high trip count. These optimizations include (but are not limited to) speculating iterations and inserting pipeline registers in the circuit that starts loops. The next two sections describe how these optimizations can substantially decrease throughput and how you can disable them to improve your design when applied to inner loops with low trip counts.

Speculated Iterations

Loop Speculation enables loop iterations to be initiated before determining whether they should have been initiated. Speculated iterations are the iterations of a loop that launch before the exit condition computation has been completed. This is beneficial when the computation of the exit condition is preventing effective loop pipelining. However, when an inner loop has a low trip count, speculating iterations result in a relatively high proportion of invalid loop iterations.

For example, consider that the compiler speculated two inner loop iterations for the code above. In that case, the timing diagram appears as shown in the following diagram, where the orange blocks with the S denote an invalid speculated iteration:

Timing Diagram with Invalid Speculated Iteration



In this case where the inner loop iteration count is in the range $(0, 3)$, speculating two iterations can cause up to a 3x reduction in the design's throughput. This happens when each outer loop iteration launches one inner loop iteration (`inner_loop_iterations` is always 1), but two iterations are speculated. For this reason, Intel® advises to force the compiler to not speculate iterations for inner loops with known small trip counts using the `[[intel::speculated_iterations(0)]]` attribute.

Dynamic Trip Counts

As mentioned earlier, the compiler's default behavior is to optimize loops for throughput. However, as you saw in the previous section, loops with low trip counts have unique throughput characteristics that lead to the compiler making different optimizations. The compiler attempts its best to determine if a loop has a high or low trip count and optimizes accordingly. However, in some circumstances, you may need to provide it with more information to make a better decision.

In the previous section, this additional information was the `speculated_iterations` attribute. However, it is not just speculated iterations that cause delays in the launching of inner loops. The compiler uses other heuristics. For example, the compiler may attempt to improve the f_{MAX} of a loop circuit by adding a pipeline register on the circuit path that starts a loop, which results in a one-cycle delay in starting the loop. For outer loops with large trip counts, this one cycle delay is negligible. However, for inner loops with small trip counts, this one cycle delay can cause throughput degradation. Like the speculated iteration case discussed in the previous section, this one cycle delay can result in up to a 2x reduction in the design's throughput.

If the inner loop bounds are known to the compiler, it decides whether to turn on/off this delay register depending on the (known) trip count. However, in the code snippet above, the inner loop's trip count is not a constant (`inner_loop_iterations` is a random number at runtime). In cases like this, Intel recommends explicitly bounding the trip count of the inner loop. This is illustrated in the example code snippet in the following, where `j < kInnerLoopBound` exit condition is added to the inner loop. This gives the compiler more explicit information about the loop's trip count and allows it to optimize accordingly.

```
for (int i; i < kOuterLoopBound; i++) {
    int inner_loop_iterations = InputPipe::read() % kInnerLoopBound;
    for (int j; j < inner_loop_iterations && j < kInnerLoopBound; j++) {
        /* ... */
    }
}
```

NOTE

For additional information, refer to the FPGA tutorial sample "Optimize Inner Loop" on [GitHub](#).

Improve Loop Performance by Caching Data in On-Chip Memory

In SYCL* task kernels for FPGA, the main objective is to achieve an initiation interval (II) of 1 on performance-critical loops. This means that a new loop iteration is launched on every clock cycle, thereby maximizing the loop's throughput. When the loop contains a loop-carried variable implemented in on-chip memory, the Intel® oneAPI DPC++/C++ Compiler often cannot achieve II=1 because the memory access takes more than one clock cycle. If the updated memory location is necessary on the next loop iteration, the next iteration must be delayed to allow time for the update, hence $II > 1$.

The on-chip memory cache technique breaks this dependency by storing recently-accessed values in a cache capable of a one-cycle read-modify-write operation. The cache is implemented in FPGA registers rather than on-chip memory. By pulling memory accesses preferentially from the register cache, the loop-carried dependency is broken.

When is the On-chip Memory Cache Technique Applicable?

You can apply the on-chip memory cache technique in the following situations:

- **Failure to achieve II=1 because of a loop-carried memory dependency in on-chip memory**

The on-chip memory cache technique is applicable if the compiler could not pipeline a loop with II=1 because of an on-chip memory dependency. If the compiler could not achieve II=1 because of a global memory dependency, this technique does not apply as the access latencies are too great.

To check this for a given design, view the [Loop Analysis](#) report in the design's optimization report. The Loop Analysis report lists the II of all loops and explains why a lower II is not achievable. Check whether the reason given resembles the compiler failed to schedule this loop with smaller II due to memory dependency. The report describes the most critical loop feedback path during scheduling. Check whether this includes on-chip memory load/store operations on the critical path.

- **An II=1 loop with a load operation of latency 1**

The compiler is capable of reducing the latency of on-chip memory accesses to achieve II=1. In doing so, the compiler makes a trade-off by sacrificing f_{MAX} to improve the II.

In a design with II=1 critical loops but lower than the desired f_{MAX} , the on-chip memory cache technique might still be applicable. It can help recover f_{MAX} by enabling the compiler to achieve II=1 with a higher latency memory access. To check whether this is the case for a given design, view the [Kernel Memory Viewer](#) report in the design's optimization report. Select the desired on-chip memory from the Kernel Memory List, and mouse over the load operation `LD` to check its latency. If the latency of the load operation is 1, this is a clear sign that the compiler has attempted to sacrifice f_{MAX} to improve loop II.

Implement the On-chip Memory Cache Technique

Consider the FPGA design example in [onchip_memory_cache.cpp](#), which demonstrates the technique using a program that computes a histogram. The histogram operation accepts an input vector of values, separates the values into buckets, and counts the number of values per bucket. For each input value, an output bucket location is determined, and the count for the bucket is incremented. This count is stored in the on-chip memory, and the increment operation requires reading from memory, performing the increment, and storing the result. This read-modify-write operation is the critical path that can result in $II > 1$.

To reduce II, the idea is to store recently-accessed values in an FPGA register-implemented cache that is capable of a one-cycle read-modify-write operation. If the memory location required on a given iteration exists in the cache, it is pulled from there. The updated count is written back to both the cache and the on-chip memory. The `ivdep` attribute is added to inform the compiler that if a loop-carried variable (namely, the variable storing the histogram output) is required within `CACHE_DEPTH` iterations, it is guaranteed to be available right away.

Select the Cache Depth

While any value of `CACHE_DEPTH` results in functional hardware, the ideal value of `CACHE_DEPTH` requires some experimentation. The depth of the cache must roughly cover the latency of the on-chip memory access. To determine the correct value, Intel® recommends starting with a value of 2 and then increase it until both $II = 1$ and load latency > 1 . In the [onchip_memory_cache.cpp](#) example, a `CACHE_DEPTH` of 5 is necessary. It is important to find the minimal value of `CACHE_DEPTH` that results in a maximal performance increase. Unnecessarily large values of `CACHE_DEPTH` consume unnecessary FPGA resources and can reduce f_{MAX} . Therefore, at a `CACHE_DEPTH` that results in $II=1$ and load latency = 1, if further increases to `CACHE_DEPTH` show no improvement, do not increase `CACHE_DEPTH` any further.

NOTE

For additional information, refer to the FPGA tutorial sample “On-Chip Memory Cache” on [GitHub](#).

Pipes

Pipes are a first-in first-out (FIFO) buffer construct that provide links between elements of a design. The Intel® oneAPI DPC++/C++ Compiler provides the following types of pipes:

- [Host Pipes](#)

Host pipes connect a host and a device.

- [Cross-Kernel Pipes](#)

Cross-kernel pipes provide a mechanism for passing data between kernels and synchronizing kernels with high efficiency and low latency. Cross-kernel pipes allow kernels to communicate directly with each other using on-device FIFO buffers that are implemented using FPGA memory resources. The Intel® oneAPI DPC++/C++ Compiler supports concurrent kernel execution. Using cross-kernel pipes for data movement between concurrently executing kernels allows for data transfer without waiting for kernel completion, which can significantly increase the throughput of your design.

- [I/O Pipes](#)

An I/O pipe is a unidirectional (source or sink) connection to the hardware that may be connected to input or output features of an FPGA board. These features might include network interfaces, PCIe®, cameras, or other data capture or processing devices or protocols.

Host Pipes

Pipes that connect a host and a device are referred to as host pipes. Host pipe support is enabled by including the following include statement in your design:

```
#include <sycl/ext/intel/experimental/pipes.hpp>
```

The sections that follow provide more information about declaring and using host pipes.

Restriction For multiarchitecture binary kernels, the number of non-CSR host pipes in your design is limited by your BSP. RTL IP cores, which have no BSP, are not limited in this way.

Host Pipe Declaration

Each individual host pipe is a program scope class declaration of the templated pipe class.

Host Pipe Template Parameters

Template Parameter	Definition	Valid Values	Default Values
id	A unique type that identifies the host pipe.	type	None (must be specified)
type	The data type to be carried by the pipe.	type	None (must be specified)

Template Parameter	Definition	Valid Values	Default Values
	FPGA Acceleration Flow Restriction The data-type bit-width must match the width of one of the host pipes with matching direction specified in the <code><channels></code> element of the <code>board_spec.xml</code> file. For each <code><interface></code> element listed under <code><channels></code> in the <code>board_spec.xml</code> file, the <code>hostpipe</code> attribute indicates if the pipe is a host pipe, the <code>type</code> attribute indicates the pipe direction, and the <code>width</code> attribute indicates the physical pipe width.		
<code>min_capacity</code>	The minimum number of words in units of T size that the pipe must be able to store without any being read out. A minimum capacity is required in some algorithms to avoid deadlock or for performance tuning. The hardware implementation can include more capacity than this parameter, but not less.	Integer greater than or equal to 0	0
<code>properties</code>	The type of a properties class which contains an unordered list of zero or more properties that augment the semantics of the pipe. For property types and their descriptions, refer to Table 26: Host Pipe Template Properties.	See Table 26: Host Pipe Template Properties .	See Table 26: Host Pipe Template Properties .

The `properties` template parameter is the type of a SYCL properties class object, including an unordered list of any of the compile-time properties listed in the following table.

Unspecified properties take on the listed default value.

All of the listed properties are declared in the `sycl::ext::intel::experimental` name space. The properties class itself is declared in the `sycl::ext::oneapi::experimental` name space.

In RTL IP cores, each host pipe can be configured as a data interface. You can configure whether it appears as an Avalon streaming interface or whether it is accessed through your RTL IP core CSR using the `protocol` template parameter.

You can use other parameters to configure other details of the data interface. For details, refer to [Host Pipes RTL Interfaces](#).

FPGA Acceleration Flow Limitation The data-type bit-width must match the width of the one of the host pipe with a matching direction specified in the `<channels>` element of the `board_spec.xml` file.

In the `board_spec.xml` file, the `<channels>` element contains a set of `<interface>` elements where the `hostpipe` attribute indicates if a pipe is a host pipe, the `type` attribute indicates the pipe direction, and the `width` attribute indicates the bit-width of the pipe.

Host Pipe Template Properties

Template Property	Definition	Valid Values	Default Values
<code>ready_lateness<int></code>	The number of cycles between when the ready signal is de-asserted and when the pipe can no longer accept new inputs when using the <code>avalon_streaming</code> or <code>avalon_streaming_uses_ready</code> protocol.	Integer greater than or equal to 0	0

Template Property	Definition	Valid Values	Default Values
<code>bits_per_symbol<int></code>	Describes how the data is broken into symbols on the data bus. This value is used only in conjunction with Avalon Packet support. Data is broken down according to how the <code>first_symbol_in_high_order_bits</code> parameter is set.	Integer greater than or equal to 0 The value must divide evenly by the size of the data type	8
<code>uses_valid<bool></code>	Controls whether a valid signal is present on the pipe interface. If false, the upstream source must provide valid data on every cycle that ready is asserted. If set to false, the <code>min_capacity</code>, and <code>ready_latency</code> template parameters must be set to 0.	Boolean	true
<code>first_symbol_in_high_order_bits<bool></code>	Specifies whether the data symbols in the pipe are in big-endian byte order.	Boolean	false
<code>protocol</code>	Specifies the protocol for the pipe interface.	<code>protocol_name:avalon_streaming</code> - or <code>protocol_avalon_streaming</code> • <code>protocol_name:avalon_streaming_uses_ready</code> - or <code>protocol_avalon_streaming_uses_ready</code> • <code>protocol_name:avalon_mm</code> - or	<code>protocol_avalon_streaming_uses_ready</code>

Template Property	Definition	Valid Values	Default Values
		protocol_avalon_mm_use_ready or protocol_avalon_mm_use_read_y	

Example Host Pipe Declaration

The following code is an example declaration of a host pipe:

```
// define at the global scope to improve RTL readability
class MyPipeT;
class AnotherPipeT;

// properties
using MyPipePropertiesT =
    decltype(sycl::ext::oneapi::experimental::properties(
        sycl::ext::intel::experimental::bits_per_symbol<4>,
        sycl::ext::intel::experimental::first_symbol_in_high_order_bits<false>));

// a host pipe with alias
using MyPipeInstance = sycl::ext::intel::experimental::pipe<
    MyPipeT,      // An identifier for the pipe
    int,          // The type of data in the pipe
    8,           // The capacity of the pipe
    MyPipePropertiesT // properties
>;

// a second host pipe with alias
using AnotherPipeInstance = cl::sycl::ext::intel::prototype::pipe<
    AnotherPipeT,  // An identifier for the pipe
    float,         // The type of data in the pipe
    4              // The capacity of the pipe
    // no properties specified means all default property values used
>;
```

Both pipe declarations use an alias for the full templated pipe class name for convenience, as does the properties type declaration for the first pipe. The first pipe carries an `int` data type and has a `min_capacity` value of 8. It also uses two non-default property values for the `bits_per_symbol` property (4) and the `first_symbol_in_high_order_bits` property (false). The second pipe carries a `float` data type and has a `min_capacity` value of 4. By not specifying the `properties` template parameter, all properties listed in the [properties table](#) assume their default values for this pipe.

Host Pipe API

Host pipes expose read and write interfaces that allow a single element to be read or written in FIFO order to the pipe. These read and write interfaces are static class methods on the templated classes described later in this section and in [Host Pipe Declaration](#).

Blocking Write

The host pipe write interface writes a single element of the given data type (`int` in the examples that follow) to the host pipe. On the host side, this class method accepts a reference to a SYCL* device queue as its first argument and the element being written as its second argument.

```
queue q(...);
...
int data_element = ...;

// blocking write from host to pipe
MyPipeInstance::write(q, data_element);
...
```

In the kernel, writes to a host pipe accept a single argument, which is the element being written.

```
float data_element = ...;

// blocking write from device to pipe
AnotherPipeInstance::write(data_element);
```

Non-blocking Write

Non-blocking writes add a `bool` argument in both host and device APIs that is passed by reference and returns true in this argument if the write was successful, and false if it was unsuccessful.

On the host:

```
queue q(...);
...
int data_element = ...;

// variable to hold write success or failure
bool success = false;

// attempt non-blocking write from host to pipe until successful
while (!success) MyPipeInstance::write(q, data_element, success);
```

On the device:

```
float data_element = ...;

// variable to hold write success or failure
bool success = false;

// attempt non-blocking write from device to pipe until successful
while (!success) AnotherPipeInstance::write(data_element, success);
```

Blocking Read

The host pipe read interface reads a single element of a given data type from the host pipe. Like the write interface, the read interface on the host takes a SYCL* device queue as a parameter. The device read interface consists of the class method `read` call with no arguments.

On the host:

```
// blocking read in host code
float read_element = AnotherPipeInstance::read(q);
```

On the device:

```
// blocking read in device code
int read_element = FirstPipeInstance::read();
```

Non-blocking Read

Like non-blocking writes, non-blocking reads add a `bool` argument in both host and device APIs that is passed by reference and returns `true` in this argument if the read was successful and `false` if it was unsuccessful.

On the host:

```
// variable to hold read success or failure
bool success = false;

// attempt non-blocking read until successful in host code
float read_element;
while (!success) read_element = SecondPipeInstance::read(q, success);
```

On the device:

```
// variable to hold read success or failure
bool success = false;

// attempt non-blocking read until successful in device code
int read_element;
while (!success) read_element = FirstPipeInstance::read(success);
```

Host Pipe Connections

Host pipe connections for a particular host pipe are inferred by the compiler from the presence of read and write calls to that host pipe in your code.

A host pipe can be connected only between the host and a single kernel. You cannot call the same host pipe from different kernels.

Host pipes operate in only one direction. That is, host-to-kernel (only writes on the host, only reads on the device) or kernel-to-host (only writes on the device, only reads on the host).

Host code for a particular host pipe can contain either only all writes or only all reads to that pipe, and the corresponding kernel code for the same host pipe can consist only of the opposite transaction.

Kernel Invocation Order and Host Read/Writes

When you create a testbench for a oneAPI kernel that you intend to compile as an IP core, write all your data to the host pipe before invoking the kernel.

This order of operation helps ensure that the simulation gives you the most accurate estimate of your kernel performance.

By buffering all the writes to the pipe, the host can supply new data to the kernel every clock cycle when it begins running.

The following code example shows this recommended method. This method ensures that the host keeps up with the kernel in the simulation flow, and thus give a more accurate representation of the IP core performance.

```
queue q(...);
...
int data_element = ...;
// Host performs all writes to the host pipe
for (int i = 0; i < ITERATIONS; i++) {
    MyPipeInstance::write(q, data_element);
}
// Invoke the kernel which reads from the host pipe
q.single_task<class MyKernel>([=] () {
    ...
    for (int i = 0; i < ITERATIONS; i++) {
        MyPipeInstance::read();
    }
    ...
});
```

The following code example might cause the kernel to stall while it waits for the host to supply data, as the host produces data slower than the kernel can consume it. The simulation might not be representative of the maximum performance of the kernel.

```
queue q(...);
...
int data_element = ...;
// Invoke the kernel which reads from the host pipe
q.single_task<class MyKernel>([=] () {
    ...
    for (int i = 0; i < ITERATIONS; i++) {
        MyPipeInstance::read();
    }
    ...
});
// Host performs all writes to the host pipe
for (int i = 0; i < ITERATIONS; i++) {
    MyPipeInstance::write(q, data_element);
}
```

Host Pipes RTL Interfaces

This section provides a summary of interfacing with host pipes in your IP based on the choice of protocol.

Host pipes support Avalon® streaming and memory-mapped interfaces. Refer to the [Intel® Avalon Interface Specifications](#) for details about these protocols.

For Avalon® memory-mapped protocols, register addresses in the CRA are specified in the generated kernel header file in the project directory. Refer to [Register Map Header File](#) for further details on CRA agent registers.

`protocol_name::avalon_streaming_uses_ready`

This is the default protocol.

This protocol allows the sink to backpressure by deasserting the `ready` signal. The sink signifies that it is ready to consume data by asserting the `ready` signal. On the cycle where the sink asserts the `ready` signal, the source must wait for the `ready_latency` signal to cycle before responding with `valid` and `data` signals, where the property specifies the `ready_latency` in the host pipe declaration.

Host-to-Device Pipe

When the `uses_valid` property is set to `false` and the `ready` signal is asserted by the kernel and sampled by the host, the host must wait `ready_latency` cycles before the value on the data interface is sampled by the kernel and consumed.

When the `uses_valid` property is set to `true` and the `ready` signal is asserted by the kernel and sampled by the host, the host must wait `ready_latency` cycles before inserting the `valid` signal. The data interface is not sampled or consumed by the kernel until the `valid` signal is asserted.

Device-to-Host Pipe

When the `uses_valid` property is set to `true` and the host asserts the `ready` signal, the kernel replies with `valid=1` and qualified data (if available) `ready_latency` cycles after the corresponding `ready` was first asserted.

When the `uses_valid` property is set to `false` and the host asserts the `ready` signal, the kernel replies with qualified data `ready_latency` cycles after the corresponding `ready` was first asserted.

`protocol_name::avalon_streaming`

With this choice of protocol, the `ready` signal is not exposed by the host pipe, and the sink cannot backpressure.

The `valid` signal qualifies data transfer from source to sink per cycle when the `uses_valid` property is set to `true`. When the `uses_valid` property is set to `false`, the source implicitly provides a valid output on every cycle, and the sink assumes a valid input on every cycle.

Host-to-Device Pipe

When the `uses_valid` property is set to `false`, the kernel samples and processes the value on the host pipe data interfaces on each cycle.

When the `uses_valid` property is set to `true`, the kernel samples and processes the value on the host pipe data interface on each cycle that the `valid` signal is asserted.

Device-to-Host Pipe

When the `uses_valid` property is set to `false`, the host must sample and process values on the host pipe data interface every clock cycle. Failure to do so causes the data to be dropped.

When the `uses_valid` property is set to `true`, the host must sample and process values on the host pipe data interface every clock cycle that the `valid` signal is asserted. Failure to do so causes the data to be dropped.

`protocol_name::avalon_mm`

This protocol can be specified on both device-to-host and host-to-device pipes.

Device-to-Host Pipe

The CRA agent contains a data register for each pipe of this type. An implicit `ready` signal is held high, meaning that the sink (host) cannot backpressure. The data register contains the latest element written by the kernel into the pipe.

You cannot specify the `uses_valid` property on device-to-host pipes.

Host-to-Device Pipe

When the `uses_valid` property is `true`, the CRA agent contains a `valid` register in addition to the data register. The host writes a 1 to the `valid` register to indicate that the value in the data register is qualified. When the kernel has consumed this data, the kernel automatically clears the value in the `valid` register. A cleared `valid` register signifies that the host is free to write a new value into the data register.

When the `uses_valid` property is false, the CRA agent contains only a `data` register for this type of pipe. The host can write data into the pipe via the `data` register in any cycle.

protocol_name::avalon_mm_uses_ready

This protocol is only available for device-to-host host pipes. The `uses_valid` property cannot be specified on these pipes.

Device-to-Host Pipe

The CRA agent contains a `data` register and `ready` register for this type of pipe. Writing a 1 to the `ready` register signifies that the data at the head of the pipe can be written into the `data` register. After the `data` register has been successfully written, the `ready` register is automatically cleared to 0. At this point, writing another 1 to the `ready` register signifies that the next data element in the pipe can be written to the `data` register.

Avalon Streaming Sideband Signals

Important Support for Avalon streaming interface sideband signals has beta-level support. The implementation of sideband signal support might change in a future release.

Enable Avalon streaming sideband signal support by using the `StreamingBeat` struct provided by the `pipes_ext.hpp` header file. The `StreamingBeat` struct generates sideband signals only when used with a host pipe. Other types of pipes do not support sideband signals.

To use the `StreamingBeat` struct, include the `pipes_ext.hpp` header file:

```
$INTELFPGAOCSDKROOT/include/sycl/ext/intel/prototype/pipes_ext.hpp
```

Using the `StreamingBeat` struct with the `uses_packets` property set to `true` adds two additional 1-bit signals to the Avalon interface, `start_of_packet` (`sop`), and `end_of_packet` (`eop`).

Assert the `sop` signal when you send the first beat of the packet along with a `valid` signal assertion. Assert the `eop` signal when you send the last beat, along with a `valid` signal assertion. You can assert `sop` and `eop` signals in the same cycle for a single packet transfer transaction. The `sop` signal can also be asserted on the cycle immediately after the `eop` signal was asserted for the previous packet.

The third property for the `StreamingBeat` struct signifies `use_empty`. When `use_empty` is set to `true`, it

adds an `empty` signal that is $\text{ceil}(\log_2(\frac{\text{data_width}}{\text{bits_per_symbol}}))$ bits long. The `empty` signal indicates the number of symbols that are empty during the `eop` cycle.

Empty symbols are always the last symbols in the data. That is, the symbols carried by low-order bits when `first_symbol_in_high_order_bits` is `true`, or the high-order bits if `first_symbol_in_high_order_bits` is set to `false`.

You must set `use_empty` for all packet interfaces that carry more than one symbol of data that have a variable length packet format.

The following example uses the `StreamingBeat` struct with the `uses_packets` and `use_empty` properties both set to `true`. The size of the `PipeData` type is forced to a multiple of the `kBitsPerSymbol` value, which is passed to the associated host pipe declaration.

```
using PipeData = ac_int<kBitsPerSymbol * kSymbolsPerBeat, false>;
using StreamingBeatData = sycl::ext::intel::experimental::StreamingBeat<PipeData, true, true>;
```

When you define the host pipe, set the `data` type to the `StreamingBeatData` struct:

```
using H2DPipe = sycl::ext::intel::experimental::pipe<H2DPipeID, StreamingBeatData>;
```


The following code example instantiates a `StreamingBeat` struct and writes to the pipe (from the host):

```
bool sop = true;
bool eop = false;
int empty = 0;
PipeData data = ...
StreamingBeatData in_beat(data, sop, eop, empty);
H2DPipe::write(q, in_beat);
```

The following code example reads from the pipe and extracts the sideband signals (from device):

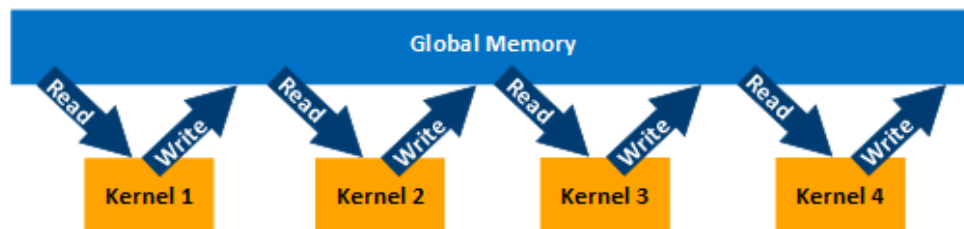
```
StreamingBeat in_beat = H2DPipe::read();
PipeData in_data = in_beat.data;
bool sop = in_beat.sop;
bool eop = in_beat.eop;
int empty = in_beat.empty;
```

Pipes Extension

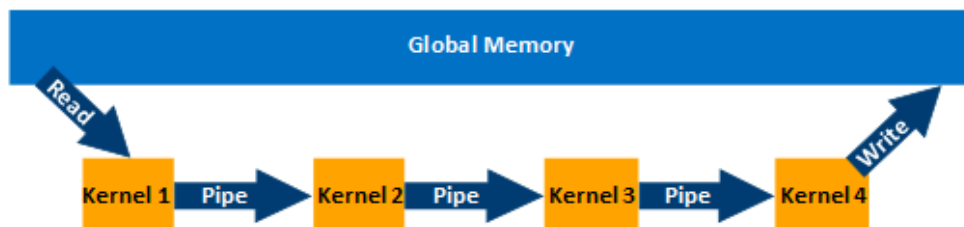
Using global memory to communicate data between your kernels can constrain the performance of your design. DPC++ pipes provide a mechanism for passing data between kernels and synchronizing kernels with high efficiency and low latency. DPC++ pipes allow kernels to use on-device FIFO buffers to communicate directly with each other. The memory model of pipes allows them to be used for inter-kernel communication without waiting for kernel completion or involvement of the host processor, as shown in the following figure:

Using SYCL* Pipes to Decouple Data Movement between Concurrently Executing Kernels

Global Memory Access Pattern Before Intel oneAPI DPC++/C++ Compiler Pipe Implementation



Global Memory Access Pattern After Intel oneAPI DPC++/C++ Compiler Pipe Implementation



Key Properties of a Pipe

The following are the key properties of a pipe:

Key Properties

Property	Description
FIFO ordering	Data is only accessible (readable) in FIFO order. NOTE There is no concept of a memory address or pointer (that is, there is no random access).
Capacity	Number of outstanding words or packets that can be written to an initially empty pipe before reading anything from it.

Accessing Pipes

Data is written to a pipe through an API that commits a single word or packet (of data type contained in the pipe), and that word or packet is later returned by an API reading data from the pipe. The API accessing the pipe can be a blocking or non-blocking type. Blocking calls wait until there is available a capacity to commit data, or until data is available to be read. Non-blocking calls do not wait. They return with a status to indicate whether their operation is successful or not.

The `pipe` Class and its Use

The `pipe` API exposed by the FPGA implementation is equivalent to the following class declaration:

```
template <class name,
          class dataT,
          size_t min_capacity = 0>
class pipe {
public:
    // Blocking
    static dataT read();
    static void write(dataT data);
    // Non-blocking
    static dataT read(bool &success_code);
    static void write(dataT data, bool &success_code);
}
```

The following table describes the `template` parameters:

Template Parameters

Parameter	Description
<code>name</code>	The type that is the basis of a pipe identification. It is typically a user-defined class, in a user namespace. Forward declaration of the type is enough, and the type need not be defined.
<code>dataT</code>	The type of data packet contained within a pipe. This is the data type that is read during a successful pipe <code>read()</code> operation, or written during a successful pipe <code>write()</code> operation. The type must have a standard layout and be trivially copyable.
<code>min_capacity</code>	User-defined minimum number of words (in units of <code>dataT</code>) that the pipe must be able to store without any being read out. The compiler may create a pipe with a larger capacity due to performance considerations.

The `pipe` class exposes static methods for writing a data word to a pipe and reading a data word from a pipe. The reads and writes can be blocking or non-blocking depending on the parameters you pass to the `read()` and/or `write()` function.

NOTE

A data word in this context is the data type that the pipe contains (`dataT` pipe template argument).

Example Code Using Blocking Inter-Kernel Pipes

When writing code with SYCL* pipes, use of the C++ type alias mechanism (`using`) is highly encouraged to avoid errors where slightly different pipe types inadvertently lead to unique pipes. The following code sample shows how to use pipes with blocking accessors to transfer data between two kernels:

```
#include <sycl/sycl.hpp>
using namespace sycl;
constexpr int N = 3;
// Specialize a pipe type
using my_pipe = ext::intel::pipe<class some_pipe, int, 8>;
void producer(const std::array<int, N> &src) {
    queue q;
    // Launch the producer kernel
    buffer<int> src_buf = {std::begin(src), std::end(src)};
    q.submit([&](handler &cgh) {
        // Get read access to src array
        accessor rd_src_buf(src_buf, cgh, read_only);
        cgh.single_task<class producer>([=]() {
            for (int i = 0; i < N; i++) {
                // Blocking write an int to the pipe
                my_pipe::write(rd_src_buf[i]);
            }
        });
    });
}

void consumer(std::array<int, N> &dst) {
    queue q;
    // Launch the consumer kernel
    buffer<int> dst_buf = {std::begin(dst), std::end(dst)};
    q.submit([&](handler &cgh) {
        // Get write access to dst array
        accessor wr_dst_buf(dst_buf, cgh, write_only);
        cgh.single_task<class consumer>([=]() {
            for (int i = 0; i < N; i++) {
                // Blocking read an int from the pipe
                wr_dst_buf[i] = my_pipe::read();
            }
        });
    });
}
```

The pipe data packet is of type `int` and the pipe has a depth of 8, as specified by the template parameters of `my_pipe` type. The pipe `read()` call blocks only when the pipe is empty, and the pipe `write()` call blocks only when the pipe is full.

NOTE

The SYCL specification does not guarantee concurrent kernel execution. However, the Intel® oneAPI DPC++/C++ Compiler guarantees that kernels submitted to the same FPGA device have a concurrent forward progress guarantee under the following conditions:

- The kernels must not have event or accessor dependencies, including transitive dependencies.
- The kernels must be submitted to separate queues or, if they are submitted to the same queue it must not have the `in_order` property.
- The kernels must be contained in the same device image.

Example Code Using Non-Blocking Inter-Kernel Pipes

The code samples ([Sample 1](#) and [Sample 2](#)) in this section illustrate how to use pipes with non-blocking writes and reads to transfer data between two concurrently running kernels:

```
//Sample 1
// The Producer kernel reads data from a SYCL buffer and writes it to
// a pipe. This transfers the input data from the host to the Consumer kernel
// that is running concurrently.
event Producer(queue &q, buffer<int, 1> &input_buffer) {
    std::cout << "Enqueuing producer...\n";

    auto e = q.submit([&](handler &h) {
        accessor input_accessor(input_buffer, h, read_only);
        size_t num_elements = input_buffer.size();

        h.single_task<ProducerTutorial>([=]() {
            for (size_t i = 0; i < num_elements; ++i) {
                ProducerToConsumerPipe::write(input_accessor[i], valid);
            }
        });
    });

    return e;
}
```

For both pipes, the data packet is of type `int`. The pipes are different because the first `template` parameter is different. The non-blocking pipe `write()` and `read()` calls do not block. They respectively return a boolean value that indicates whether data is written successfully to the pipe (that is, the pipe is not full) or if the data is read successfully from the pipe (that is, the pipe is not empty).

Perform non-blocking pipe writes to facilitate applications where writes to a full FIFO buffer should not cause the kernel to stall until a slot in the FIFO buffer becomes free. Consider a scenario where your application has one data producer with two identical workers that consume the data. Assume the time each worker takes to process a message varies depending on the contents of the data. In this case, there might be a situation where one worker is busy while the other is free. A non-blocking write can facilitate work distribution such that both workers are busy. Like a non-blocking write, perform non-blocking reads to facilitate applications where data is not always available, and other operations need not wait for the data to become available.

NOTE

You can mix blocking and non-blocking accessors for writing or reading data to or from pipes. For example, you can write data to a pipe using a blocking pipe `write()` call and read it from the other end using a non-blocking pipe `read()` call, and vice versa.

```
//Sample 2
#include <sycl/sycl.hpp>
using namespace sycl;
constexpr size_t N = 16;
// Specialize the two pipe types, differentiated based on their first template
// parameter
using pipe1 = ext::intel::pipe<class some_pipe, int>;
using pipe2 = ext::intel::pipe<class other_pipe, int>;
// the producer kernels are not shown
void consumer(const std::array<int, N> &dst) {
    queue q;
    // Launch the consumer kernel
    buffer<int> dst_buf = {std::begin(dst), std::end(dst)};
    q.submit([&](handler &cgh) {
        // Get write access to src array
        accessor wr_dst_buf(dst_buf, cgh, write_only);
        cgh.single_task<class consumer>([=]() {
            int = 0;
            while (i < N) {
                bool valid0 = false, valid1 = false;
                auto data0 = pipe1::read(valid0);
                auto data1 = pipe2::read(valid1);
                if (valid0) {
                    wr_dst_buf[i++] = process(data0);
                }
                if (valid1) {
                    wr_dst_buf[i++] = process(data1);
                }
            }
        });
    });
}
```

NOTE

For additional information, refer to FPGA tutorial sample “Pipes” on [GitHub](#).

I/O Pipes

An I/O pipe is a unidirectional (source or sink) connection to the hardware that may be connected to input or output features of an FPGA board. These features might include network interfaces, PCIe®, cameras, or other data capture or processing devices or protocols.

A source I/O device provides data that a SYCL kernel can read whereas a sink I/O device accepts data written by a SYCL kernel and sends it to the hardware device. A common use of I/O pipes is to interface to ethernet connections that interface directly with the FPGA. The source reads from the network and the sink writes to the network. This allows a SYCL kernel to process data from the network and resend it back to the network.

For testing purposes, you can use files on the disk as input or output devices, allowing you to debug using emulation or simulation for faster compilation.

I/O Pipes Implementation

To implement I/O pipes, follow these steps:

1. Consult your board vendor documentation before you implement I/O pipes in your kernel program.
2. Use a struct with a numeric id to define a `kernel_readable_io_pipe` or `kernel_writable_io_pipe` type to declare an I/O pipe to interface with hardware peripherals. These definitions are typically provided by a board vendor.
 - The numeric `id` value is the 0-origin index into the interfaces in the channels section of the `board_spec.xml` for the device.
 - The `chan_id` argument is necessary for simulation and it is the name of the I/O interface listed in the `board_spec.xml` file as shown in the following:

```
<channels>
  <interface name="board" port="c1" type="streamsource" width="32" chan_id="c1"/>
  <interface name="board" port="c2" type="streamsink" width="32" chan_id="c2"/>
</channels>
```

NOTE

Only channels marked type `streamsource` or `streamsink` are used for indexing.

3. Implement the interface to hardware I/O pipes or files (emulator or simulator) by mapping the `id` variable, as shown in the following:

```
// Specialize a pipe type
struct read_io_pipe {
    static constexpr unsigned id = 0;
};

struct write_io_pipe {
    static constexpr unsigned id = 1;
};

// id 0 -> file name or channel name: "c1" for hardware, "0" for emulator, "c1" for simulation.
using read_iopipe = ext::intel::kernel_readable_io_pipe<read_io_pipe, unsigned, 4>;

// id 1 -> file name or channel name: "c2" for hardware, "1" for emulator, "c2" for simulation.
using write_iopipe = ext::intel::kernel_writable_io_pipe<write_io_pipe, unsigned, 3>;
```

where:

Interface	Description
For Hardware	<code>id N</code> is mapped to the channel (not file) in the associated hardware. For example: • <code>id 0</code> is mapped to the <code>chan_id</code> in the first interface defined. • <code>id 1</code> is mapped to the <code>chan_id</code> in the second interface defined, and so on. If there is no matching channel, an error is generated.
For Emulator	<code>id N</code> is mapped to a file named <code>N</code>, which means <code>id 0</code> is file <code>"0"</code>. This file is read or written by reading or writing to the associated I/O pipe.
For Simulator	<code>id N</code> is mapped to <code>chan_id</code> names in the <code>board_spec.xml</code> file supplied with the BSP (See channels). For example: • <code>id 0</code> is mapped to the <code>chan_id</code> in the first interface defined. • <code>id 1</code> is mapped to the <code>chan_id</code> in the second interface defined, and so on. If there is no matching channel, an error is generated.

The I/O Pipe Classes and Their Use

The I/O pipe APIs exposed by the FPGA implementations is equivalent to the following class declarations:

```
template <class name,
          class dataT,
          size_t min_capacity = 0>
class kernel_readable_io_pipe {
public:
    static dataT read(); // Blocking
    static dataT read(bool &success_code); // Non-blocking
};

template <class name,
          class dataT,
          size_t min_capacity = 0>
class kernel_writeable_io_pipe {
public:
    static void write(dataT data); // Blocking
    static void write(dataT data, bool &success_code); // Non-blocking
}
```

The following table describes the `template` parameters:

Template Parameters

Parameter	Description
<code>name</code>	The type that is the basis of an I/O pipe identification. It may be provided by the device vendor or user defined. The type must contain a <code>static constexpr unsigned</code> expression with name <code>id</code> , which is used to determine the hardware device referenced by the I/O pipe.
<code>dataT</code>	The type of data packet contained within an I/O pipe. This is the data type that is read during a successful pipe <code>read()</code> operation, or written during a successful pipe <code>write()</code> operation. The type must have a standard layout and be trivially copyable.
<code>min_capacity</code>	User-defined minimum number of words (in units of <code>dataT</code>) that an I/O pipe must be able to store without any being read out. The compiler may create an I/O pipe with a larger capacity due to performance considerations.

NOTE

A data word in this context is the data type that the pipe contains (`dataT` pipe template argument).

Example Code for I/O Pipes

Here is an example that includes the definitions above:

```
// "Built-in pipes" provide interfaces with hardware peripherals
// These definitions are typically provided by a device vendor and
// made available to developers for use.
namespace example_platform {
    template <unsigned ID>
    struct ethernet_pipe_id {
        static constexpr unsigned id = ID;
    };
};
```

```
using ethernet_read_pipe = kernel_readable_io_pipe<ethernet_pipe_id<0>, int, 0>;
using ethernet_write_pipe = kernel_writeable_io_pipe<ethernet_pipe_id<1>, int, 0>;
}
```

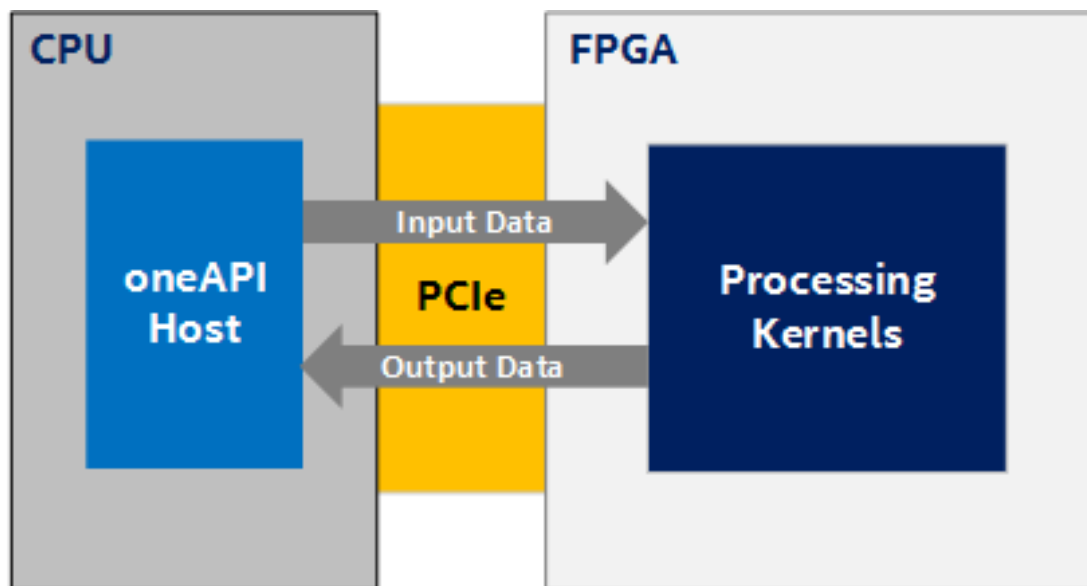
NOTE

For additional information, refer to the FPGA tutorial sample "IO Streaming with SYCL* IO Pipes" on [GitHub](#).

Using I/O Pipes to Stream Data

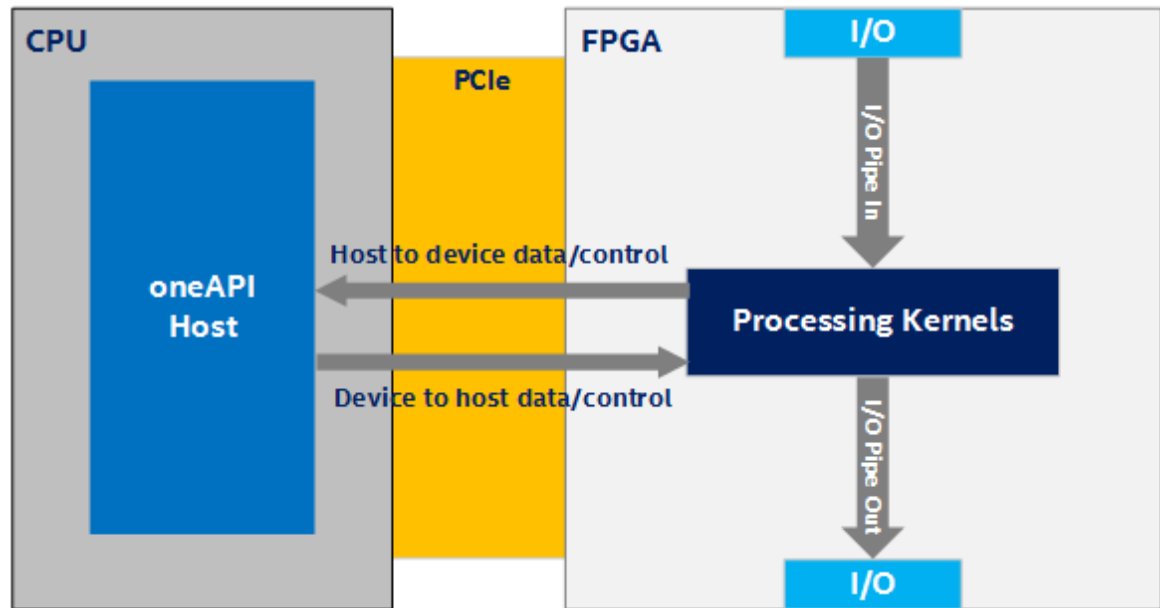
In many of the design examples, an FPGA device is treated as an accelerator, as illustrated in the following diagram, where the main computation (and therefore the data to be computed) resides on the host (CPU) and you accelerate some compute-intensive task using kernels on the device. The host moves the data to the device, performs the calculation, and moves the data back:

FPGA Device Used as an Accelerator



However, a key feature of FPGAs is their rich input-output (I/O) capabilities (for example, Ethernet). Taking advantage of these capabilities within the oneAPI programming environment requires a different programming model than the accelerator model as described above. In the model illustrated in the following figure, consider a kernel (or kernels) where some of the kernels are connected to the FPGA's I/O via I/O pipes and the main data flow is through the FPGA device rather than from CPU to FPGA and back:

Data Flow Through the FPGA Device Using I/O Pipes



In the [Figure 93: Data Flow Through the FPGA Device Using I/O Pipes](#), there are four possible directions of data flow:

- I/O to device
- Device to I/O
- Device to host
- Host to device

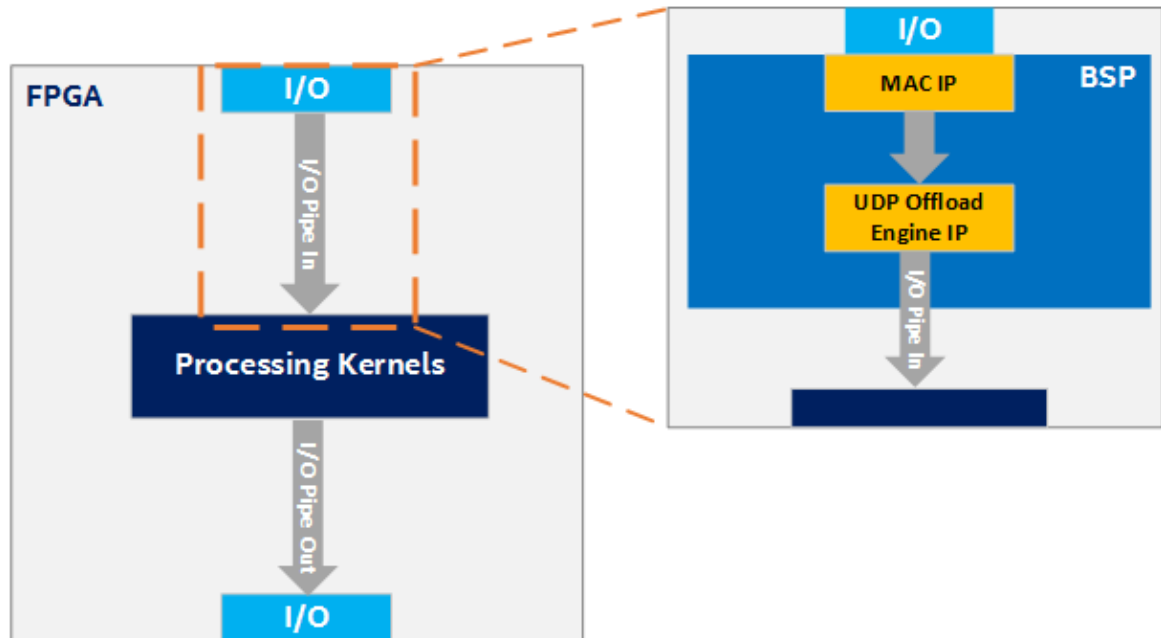
The direction and amount of data flowing in the system is application dependent. The main source of data is I/O. Data streams into the device from I/O (I/O to device) and out of the device to I/O (device to I/O). However, the host and device exchange low-bandwidth control signals using a host-to-device and device-to-host connection (or side channel).

I/O pipes have the same interface API as inter-kernel pipes. This abstraction is implemented in the Board Support Package (BSP) and provides a much simpler programming model for accessing the FPGA I/O. See [The I/O Pipe Classes and Their Use](#).

Consider an example scenario where you want the kernel to be able to receive UDP packets through the FPGA's Ethernet interface. Implementing the necessary hardware to process the data coming from the Ethernet pins in oneAPI would be both extremely difficult and inefficient. Moreover, there are already many RTL solutions for doing this. So, instead, you can implement this low-level logic with RTL in the BSP. This example is illustrated in the following figure, where the BSP connects the Ethernet I/O pins to a MAC IP, the MAC IP to a UDP offload engine IP, and finally the UDP offload engine IP to an I/O pipe. This I/O pipe can

then be simply connected to the processing kernels. The details of these connections are abstracted from the oneAPI kernel developer. The following figure shows only an input I/O pipe. The process is similar for an output I/O pipe, but the data flows in the opposite direction.

I/O Input Pipe



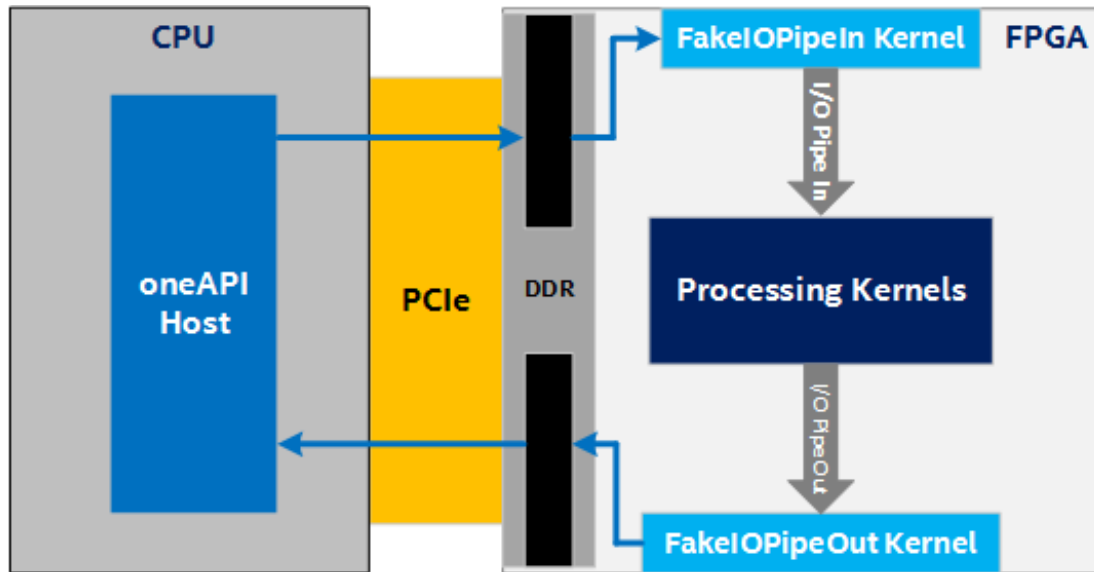
Faking I/O Pipes

Unfortunately, designs that use these I/O pipes are confined to BSPs that support that specific I/O pipe, which makes prototyping and testing the processing kernels difficult. To address these concerns, a fake I/O pipes library is available. With this library, you can create kernels that behave like I/O pipes without having a BSP that actually supports them. This allows you to start prototyping and testing your processing kernels without a BSP that supports I/O pipes.

There are currently two options for faking IO pipes:

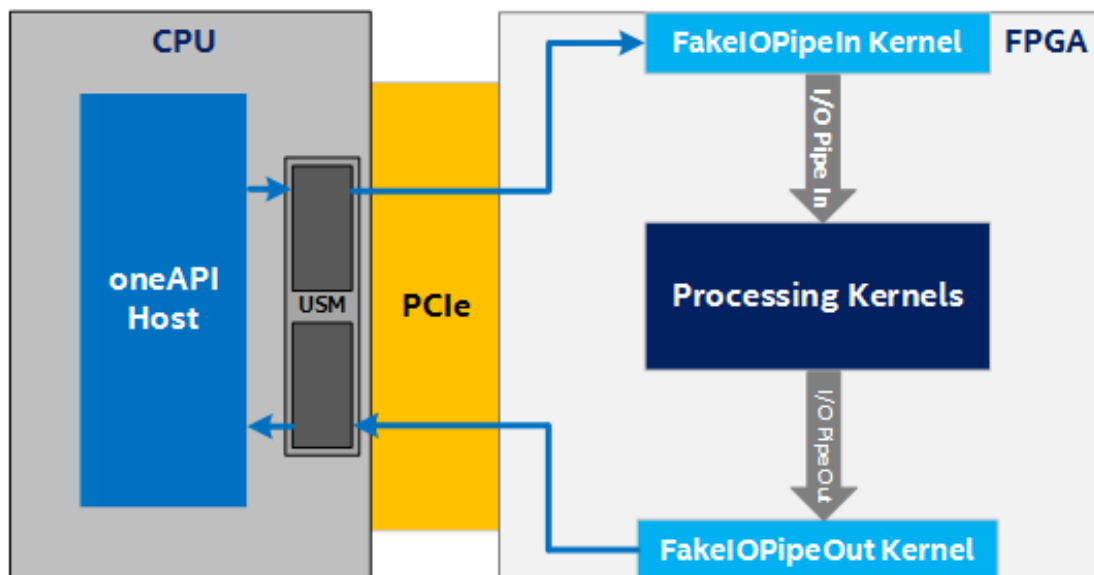
- **Device memory allocations:** This option requires no special BSP support, as shown in the following figure:

Device Memory Allocations



- **Unified Shared Memory (USM) host allocations:** This option requires USM support in the BSP as shown in the following figure. See also [Unified Shared Memory](#) topic in the open-access book *Data Parallel C++: Programming Accelerated Systems Using C++ and SYCL**.

Unified Shared Memory (USM) Host Allocations



Emulate Applications with a Pipe That Reads or Writes to an I/O Pipe

FPGA Acceleration Flow Only The information presented here applies only to multiarchitecture binary kernels in the FPGA acceleration flow. It does not apply for RTL IP core kernels in the SYCL* HLS flow.

The Intel® FPGA Emulation Platform for OpenCL™ software emulates kernel-to-kernel pipes. However, it does not support interacting directly with the hardware I/O pipes on your target board. Nevertheless, you can emulate the behavior of I/O pipes using the following procedures:

For Input I/O Pipes

1. Store input data to be transferred to the pipe in a file with a name matching the `id` specialization of the pipe. Consider the following example:

```
// Specialize a pipe type
struct read_io_pipe {
    static constexpr unsigned id = 0;
};
using read_iopipe = sycl::ext::intel::kernel_readable_io_pipe<read_io_pipe, unsigned, 4>;
```

2. Create a file named 0.
3. Store the test input data in the file 0.

For Output I/O Pipes

Output data is automatically written to a file with a name matching the `id` specialization of the output pipe.

Characteristics of Pipes

Pipes in your FPGA program have characteristics as described in this section.

Data Persistence in Pipes

Data written to a pipe using a pipe `write()` call remains in the pipe as long as the kernel program remains loaded on the FPGA device. In other words, the data written to a pipe is persistent across work-groups and NDRange invocations. Data written by a work item to a pipe remains in that pipe until another work item reads from it. In addition, the sequence of data in a pipe always follows FIFO ordering, and the order is independent of the work item that performs the write or read operation.

Data in pipes is not persistent across multiple or different invocations of kernel programs that lead to FPGA device reprogramming, or between context, program, device, or platform releases, even if the compiler performs optimizations that avoid reprogramming operations on a device. For example, if you run a host program twice using the same FPGA image, or if a host program releases and reacquires a context, the data in the pipe may or may not persist across the operation. FPGA device reset operations might happen behind the scenes on object releases that may purge data in any pipes.

Work-Item Order in NDRange kernels

In an NDRange kernel, the order in which work items and work groups access a pipe is not deterministic. Kernels should not make assumptions about the order in which the data from different work items is written to or read from a pipe.

Restrictions of Pipes

The following table summarizes restrictions of pipes:

Restrictions of Pipes

Restriction	Description
Multiple Pipe Call Sites	A kernel can read from the same pipe multiple times. However, multiple kernels cannot read from the same pipe. Similarly, a kernel can write to the same pipe multiple times, but multiple kernels cannot write to the same pipe.
Feedback and Feed-Forward Pipes	Within a single kernel, you should either read from a pipe or write to a pipe. Writing and reading to the same pipe within a single kernel may lead to poor performance.
Emulation Support	The FPGA emulator supports emulation of kernels that contain pipes. For better conformity between emulation and hardware, provide a non-zero pipe capacity while specializing a pipe type. For more information about the Emulator, refer to Emulate and Debug Your Design. NOTE If the same kernel is invoked more than once, the FPGA emulator may attempt to execute kernels concurrently, resulting in a data race if both invocations attempt to read or write from the same pipe. This issue affects only emulation. In the hardware flow, multiple invocations of the same kernel are executed serially. To work around this issue, add a call to the <code>sycl::queue::wait()</code> function between executions of the same kernel.

Guidelines for Designing Pipes

Consider the following best practices when designing pipes:

- Determine whether you should split the design into multiple kernels connected by pipes.
- Aggregate data on pipes only when all the data is used at the same point in the kernel.
- Do not use non-blocking pipes if you are using a looping structure waiting for the data, that is, avoid the following coding pattern for non-blocking pipe accessors:

```
bool success = false;
while (!success) {
    my_pipe::write(rd_src_buf[i], success); // can be a non-blocking read too
}
```

NOTE

Whenever you use the above code pattern, use the corresponding blocking accessor instead because it is more efficient in hardware.

The `atomic_fence` Function and Pipes

Attention

This topic assumes that you already have an understanding of the `atomic_fence` function described in the SYCL specification. If you are new to it, then before you proceed, read about the `atomic_fence` function in the [SYCL* 2020 Specification](#).

When running kernels in parallel, you might want multiple kernels to collaboratively access a shared memory. SYCL* provides the `atomic_fence` function as a synchronization construct to reason about the order of memory instructions accessing the shared memory. The `atomic_fence` function controls the reordering of memory load and store operations (subject to the associated memory order and memory scope) when paired with synchronization through an atomic object. Pipe read and write operations behave as if they are SYCL-relaxed atomic load and store operations. When paired with `atomic_fence` functions to establish a synchronizes-with relationship, pipe operations can provide guarantee on side-effect visibility in memory, as defined by the SYCL memory model. For additional information about the `atomic_fence` function, refer to the [SYCL* 2020 Specification](#).

Caution

The current `atomic_fence` function for FPGA uses an overly conservative implementation and is still preliminary.

- The implementation guarantees only functional correctness and not the maximum performance because the `atomic_fence` function currently enforces more memory ordering than it requires. If you do not use the `atomic_fence` function with a correct `memory_order` parameter, then you might see unexpected behavior in your program when the `atomic_fence` function handles memory ordering properly in a future release.
- The implementation does not support the `memory_scope::system` constraint. The broadest scope supported for FPGA is the `memory_scope::device` constraint.

Example Code for Using the `atomic_fence` Function and Blocking Inter-Kernel Pipes

The following code sample shows how to use the `atomic_fence` function with a blocking inter-kernel pipe to synchronize the load and store to a shared device memory between a producer and a consumer:

```
#include <sycl/sycl.hpp>
using namespace sycl;
using my_pipe = ext::intel::pipe<class some_pipe, int>;
constexpr int READY = 1;

int produce_data(int data);
int consume_data(int data);

event Producer(queue&q, int *shared_ptr, size_t size) {
    return q.submit([&](handler& h) {
        h.single_task<class ProducerKernel>([=]() [[intel::kernel_args_restrict]] {
            // create a device pointer to explicitly inform the compiler the
            // pointer resides in the device's address space
            device_ptr<int> shared_ptr_d(shared_ptr);

            // produce data
            for (size_t i = 0; i < size; i++) {
                shared_ptr_d[i] = produce_data(i);
            }
        });
    });
}
```

```

        // use atomic_fence to ensure memory ordering
        atomic_fence(memory_order::seq_cst, memory_scope::device);
        // notify the consumer to start data processing
        my_pipe::write(READY);
    });
}
}

event Consumer(queue & q, int* shared_ptr, size_t size, int *output_ptr) {
    return q.submit([&](handler& h) {
        h.single_task<class ConsumerKernel>([=]() [[intel::kernel_args_restrict]] {
            // create device pointers to explicitly inform the compiler these
            // pointer reside in the device's address space
            device_ptr<int> shared_ptr_d(shared_ptr);
            device_ptr<int> out_ptr_d(output_ptr);

            // wait on the blocking pipe_read until notified by the producer
            int ready = my_pipe::read();

            // use atomic_fence to ensure memory ordering
            atomic_fence(memory_order::seq_cst, memory_scope::device);

            // consume data and write to output memory address
            for(int i = 0; i < size; i++) {
                out_ptr_d[i] = consume_data(shared_ptr_d[i]);
            }
        });
    });
}
}

```

In the above example, the consumer loads data produced by the producer. To prevent a scenario where the consumer loads the shared device memory before the producer finishes storing to it, a blocking pipe is used to synchronize between the two kernels. The consumer's pipe read does not return until it sees the `READY` written by the producer. In this example, the `atomic_fence` functions in the producer and consumer prevent the shared memory read and write from being reordered with the pipe instructions. They also form a release-acquire ordering, which ensures that by the time the consumer sees the pipe read returns, the producer's write operation to the shared device memory is also visible to the consumer.

NOTE

The shared device memory is created using a USM device allocation that allows the two kernels to be running in parallel even though they both access the shared device memory simultaneously.

Data Types and Arithmetic Operations

15

Data Type Selection Considerations

Select the appropriate data type to optimize the FPGA area use by your SYCL* application:

- Select the most appropriate data type for your application. For example, do not define your variable as `float` if the data type `short` is sufficient.
- Ensure that both sides of an arithmetic expression belong to the same data type. Consider an example where one side of an arithmetic expression is a floating-point value and the other side is an integer. The mismatched data types cause the Intel® oneAPI DPC++/C++ Compiler to create implicit conversion operators, which can become expensive if they are present in large numbers.
- Take advantage of padding if it exists in your data structures. For example, if you only need `float3` data type, which has the same size as `float4`, you may change the data type to `float4` to make use of the extra dimension to carry an unrelated value.

Arithmetic Operation Considerations

Select the appropriate arithmetic operation for your SYCL application to avoid excessive FPGA area use.

- Introduce floating-point arithmetic operations only when necessary.
- The Intel® oneAPI DPC++/C++ Compiler defaults floating-point constants to double data type. Add an `f` designation to the constant to make it a single precision floating-point operation. For example, the arithmetic operation `sin(1.0)` represents a double precision floating-point sine function. The arithmetic operation `sin(1.0f)` represents a single precision floating-point sine function.
- If you do not require full precision result for a complex function, compute simpler arithmetic operations to approximate the result. Consider the following example scenarios:
 - Instead of computing the function `pow(x, n)` where `n` is a small value, approximate the result by performing repeated squaring operations because they require much less hardware resources and area.
 - Ensure you are aware of the original and approximated area uses because in some cases, computing a result via approximation might result in excess area use. For example, the `sqrt` function is not resource-intensive. Other than a rough approximation, replacing the `sqrt` function with arithmetic operations that the host must compute at runtime might result in larger area use.
 - If your kernel performs a complex arithmetic operation with a constant that the Intel® oneAPI DPC++/C++ Compiler computes at compilation time (for example, `log(PI/2.0)`), perform the arithmetic operation on the host instead and pass the result as an argument to the kernel at runtime.

Restriction

Currently, SYCL implementation of math functions is not supported on FPGAs.

Optimize Floating-point Operation

Starting with the oneAPI 2021.2 release, fast math is enabled by default, allowing the Intel® oneAPI DPC++/C++ Compiler to make various out-of-box floating point math (`float` or `double`) optimizations. With these optimizations enabled, you might observe different bitwise results when compared to results from the oneAPI 2021.1 release or from GCC. The tradeoff is done to improve performance and area of your design.

Automatic dot product inference and floating-point contraction for double precision math are two key noticeable FPGA optimizations that save a large amount of FPGA area and improve performance/latency. To return to the same precision as the oneAPI 2021.1 release or GCC, use the following compiler options:

- For Linux: `-no-fma -fp-model=precise`
- For Windows: `/Qfma- /fp:precise`

For more information about these options, refer to `-fp-model`, `fp` and `fma`, `Qfma` topics in the *Intel® oneAPI DPC++/C++ Compiler Developer Guide and Reference*.

For floating-point operations, you can manually direct the Intel® oneAPI DPC++/C++ Compiler to perform optimizations that create more efficient pipeline structures in hardware and reduce the overall hardware use. These optimizations can cause small differences in floating-point results. You can also apply the `fp contract` and `fp reassociate` floating-point pragmas to handle kernel's arithmetic and floating-point operations at finer granularity. For more information about the pragmas, refer to [Floating-Point Pragmas](#).

Minimize the Use of Expensive Functions

Some functions are expensive to implement in FPGAs. Expensive functions might decrease kernel performance or require a large amount of hardware to implement.

The following functions are expensive:

- Integer division and modulo (remainder) operators
- Most floating-point operators except addition, multiplication, absolute value, and comparison. For more information about optimizing floating-point operations, refer to the [Optimize Floating-point Operation](#) section.
- Atomic operations. For more information, refer to the [Memory Model and Atomics](#) in *Data Parallel C++: Programming Accelerated Systems Using C++ and SYCL** book and [Atomic Operations](#) topic in the SYCL* 2020 specification.

In contrast, inexpensive functions have minimal effects on kernel performance, and their implementation consumes minimal hardware.

The following functions are inexpensive:

- Binary logic operations such as AND, NAND, OR, NOR, XOR, and XNOR
- Logical operations with one constant argument
- Shift by constant
- Integer multiplication and division by a constant that is a power of two

If an expensive function produces a new piece of data for every work item in a work group, it is beneficial to code it in a kernel.

On the contrary, the following code example depicts a case of an expensive floating-point operation (division) executed by every work item in the NDRange:

```
// this function is used in kernel code
void myKernel (accessor<int,
                access::mode::read,
                access::target::global_buffer> a,
                accessor<int,
                access::mode::read,
                access::target::global_buffer> b,
                sycl::id<1> wiID,
                const float c,
                const float d)
{
    //inefficient since each work-item must calculate c divided by d
    b[wiID] = a[wiID] * (c / d);
}
```

The result of this calculation is always the same. To avoid this redundant and hardware resource-intensive operation, perform the calculation in the host application and then pass the result to the kernel as an argument for all work items in the NDRange to use. The modified code is shown in the following:

```
void myKernel (accessor<int,
               access::mode::read,
               access::target::global_buffer> a,
               accessor<int,
               access::mode::read,
               access::target::global_buffer> b,
               sycl::id<1> wiID,
               const float c_divided_by_d)
{
    /*host calculates c divided by d once and passes it into
    kernel to avoid redundant expensive calculations*/
    b[wiID] = a[wiID] * c_divided_by_d;
}
```

The Intel® oneAPI DPC++/C++ Compiler consolidates operations that are not work-item-dependent across the entire NDRange into a single operation. It then shares the result across all work items. In the first code example, the Intel® oneAPI DPC++/C++ Compiler creates a single divider block shared by all work-items because division of *c* by *d* remains constant across all work-items. This optimization helps minimize the amount of redundant hardware.

However, implementing an integer division requires a significant amount of hardware resources. In this case, it is beneficial to offload the division operation to the host processor and then pass the result as an argument to the kernel to conserve hardware resources.

Variable-Precision Data Type Support

The Intel® oneAPI DPC++/C++ Compiler supports a range of FPGA-optimized variable-precision (sometimes referred to as *arbitrary-precision*) data types that are defined in header files, which you can include in your designs. Some of these header files are based on the Algorithmic C (AC) data types provided by Siemens EDA* (formerly *Mentor Graphics*) under the Apache 2.0 license. For more information about the Algorithmic C data types, refer to <https://cdrdv2.intel.com/v1/dl/getContent/728986>.

Data Types Supported by the Intel® oneAPI DPC++/C++ Compiler

Data Type	Header File	Description
ac_int	<sycl/ext/intel/ac_types/ac_int.hpp>	Arbitrary-precision integer support.
ac_fixed	<sycl/ext/intel/ac_types/ ac_fixed.hpp> <sycl/ext/intel/ac_types/ ac_fixed_math.hpp>	Arbitrary-precision fixed-point number support. Supports some non-standard math functions for arbitrary-precision fixed-point data types.
ac_complex	<sycl/ext/intel/ac_types/ ac_complex.hpp>	Complex number support.
ap_float	<sycl/ext/intel/ac_types/ ap_float.hpp> <sycl/ext/intel/ac_types/ ap_float_math.hpp>	Arbitrary-precision floating-point number support. Supports commonly used exponential, logarithmic, power, and trigonometric functions with ap_float data type.

NOTE

Ensure that the AC data type headers are included after <sycl/sycl.hpp> and <sycl/ext/intel/fpga_extensions.hpp> header files.

Compilation Flags

Compilation Flags for Data Types

AC Data Type	icpxCommand Flags	Description
Any	Linux: <code>-qactypes</code> Windows: <code>/Qactypes</code>	Use these flags to include <code>ac_types</code> header files on the include path and link against AC type libraries required for the host device execution support.
	<code>-DFPGA_EMULATOR</code>	Use this flag when you use any AC data type and want to compile programs for emulation.
<code>ap_float</code>	Linux: <code>-fp-model=precise -no-fma</code> Windows: <code>/fp:precise /Qfma-</code>	Use these flags to ensure that floating-point operations are accurate. For more information about these flags, refer to <code>-fp-model</code> , <code>fp</code> and <code>fma</code> , <code>Qfma</code> topics in the <i>Intel® oneAPI DPC++/C++ Compiler Developer Guide and Reference</i> .

Advantages and Limitations of Variable Precision Data Types

Advantages

The variable precision data types have the following advantages over the use of standard C/C++ data types:

- You can achieve narrower data paths and processing elements for various operations in the circuit.
- The data types ensure that all operations are carried out in a size guaranteed not to lose any data. However, you can still lose data if you store data in a location where the data type is too narrow in size.

Limitations

AC Data Types

The AC data types have the following limitations:

- Multipliers are limited to generating 512-bit results.
- Dividers for `ac_int` data types are limited to a maximum of 64-bit unsigned or 63-bit signed.
- You must initialize all AC data type variables before accessing them. Accessing the AC data types before initialization is an undefined behavior and leads to unexpected results. Assigning each bit explicitly using the `[]` operator or `set_slc` function does not count as initializing an AC data type. The default constructor of AC data types does not initialize the variable; other constructors initialize them.
- Dividers for `ac_fixed` data types are limited to a maximum of 64-bits (unsigned or signed).
- Creation of `ac_fixed` variables larger than 32 bits are supported only with the use of the `bit_fill` utility function.

For example:

```
// Creating an ac_fixed with value set to 4294967298, which is larger than 2^32.

// Unsupported
ac_fixed<64, 64, false> v1 = ac_fixed<64, 64, false>(4294967298);

// Supported
// 4294967298 is 0b10000000000000000000000000000010 in binary
// Express that as two 32-bit numbers and use the bit_fill utility function.
const int vec_inp[2] = {0x00000001, 0x00000002};
ac_fixed<64, 64, false> bit_fill_res;
bit_fill_res.bit_fill<2>(vec_inp);
```

- The AC data types are not supported on the Red Hat Enterprise Linux* (RHEL) 7 operating system for emulation due to a bug in the `glibc` version bundled with RHEL 7.
- You cannot template the `ac_complex` data type with the `ap_float` data type.
- When using the `bit_fill_hex()` function inside a kernel, pass the input string to the kernel through a `char` buffer and not as a `string` buffer. In addition, hardware and simulation compile flows do not support using a `string` literal or passing the string directly to the function. The following are the supported and unsupported code patterns:

Supported Patterns

```
// Supported Pattern 1: Passing string as a char sycl::buffer to the kernel
ac_int<140, false> supported_example1(queue &q) {
    ac_int<140, false> a;
    std::string hex_string{"0x177632EE7E265080BD54FF0CE7EF42C12"};
    constexpr int N = 36; // size of hex_string

    buffer<ac_int<140, false>, 1> in1(&a, 1);
    // Note: the N + 1 ensures that the null byte
    //terminating the char array buffer is copied
    buffer<char, 1> in2(hex_string.c_str(), range<1>(N + 1));

    q.submit([&](handler &h) {
        accessor x(in1, h, read_write);
        accessor y(in2, h, read_only);
        h.single_task<class D>([=] {
            x[0].bit_fill_hex(&y[0]);
        });
    });
    q.wait();
    return a;
}
```

```
// Supported Pattern 2: Create a char array with the string literal.
ac_int<140, false> supported_example2(queue &q) {
    ac_int<140, false> a;
    buffer<ac_int<140, false>, 1> in1(&a, 1);

    q.submit([&](handler &h) {
        accessor x(in1, h, read_write);
        h.single_task<class D>([=] {
            char str[36] = "0x177632EE7E265080BD54FF0CE7EF42C12";
            x[0].bit_fill_hex(str);
        });
    });
    q.wait();
    return a;
}
```

Unsupported Patterns

```
// Unsupported Pattern 1 - Using a string Literal, will result in compilation error
ac_int<140, false> unsupported_example1(queue &q) {
    {
        ac_int<140, false> a;
        buffer<ac_int<140, false>, 1> a_buff(&a, 1);

        q.submit([&](handler &h) {
            accessor a_acc {a_buff, h, write_only, no_init};
            h.single_task<class A>([=]() {
```

```

        a_acc[0].bit_fill_hex("1141e98e8c51b7ac7ad387d7f8ee4f1b9");
    });
});
q.wait_and_throw();
return a;
}
}

// Unsupported Pattern 2 - Passing the string to the kernel in a string sycl::buffer
ac_int<140, false> unsupported_example2(queue& q) {
{
    std::string str{"1141e98e8c51b7ac7ad387d7f8ee4f1b9"};
    ac_int<140, false> a;
    buffer<std::string, 1> str_buff(&str, 1);
    buffer<ac_int<140, false>, 1> a_buff(&a, 1);

    q.submit([&](handler &h) {
        accessor str_acc {str_buff, h, read_only};
        accessor a_acc {a_buff, h, write_only, no_init};

        h.single_task<class B>([=]() {
            a_acc[0].bit_fill_hex(str_acc[0].c_str());
        });
    });
    q.wait_and_throw();
    return a;
}
}

```

ap_float Data Type

The `ap_float` data type has the following limitations:

- While the floating-point optimization of converting into constants is performed for `float` and `double` data types, it is not performed for the `ap_float` data type.
- A limited set of math functions is supported. For details, see [Math Functions Supported by ap_float Data Type](#).
- Constant initialization works only with the round-towards-zero (`RZERO`) rounding mode.
- For emulation, the `ap_float` math library is not supported on the Red Hat Enterprise Linux* (RHEL) 7 operating system.
- When computing A^B using `ap_float`'s `ihc_pown` function, if B is an unsigned type T of size N bits and is equal to the maximum unsigned value, redefine B to be of size $N+1$ bits. Otherwise, results will be incorrect. For example:

```

// Sample Code:
ap_float<8, 7> a = 2;
ac_int<4, false> b = 15; // max value that this ac_int can hold
... = ihc_pown(a, b); // !!! Will produce incorrect result

```

```

// Workaround:
ap_float<8, 7> a = 2;
ac_int<5, false> b = 15; // Workaround
... = ihc_pown(a, b); // Will produce correct result

```

Declare and Use the AC Data Types

Refer to the following topics for more information about how to declare and use various AC data types:

- [ac_int](#)

- `ac_fixed`
- `ac_complex`
- `ap_float`

Declare the `ac_int` Data Type

Perform the following steps to declare the `ac_int` data type:

1. Include the `ac_int.hpp` header file as follows:

```
#include <sycl/ext/intel/ac_types/ac_int.hpp>
```

2. Declare your `ac_int` variables in one of the following ways:

- Template-based declaration:

```
ac_int<N, true> var_name; //Signed N-bit integer
```

```
ac_int<N, false> var_name; //Unsigned N-bit integer
```

- Predefined types up to 63 bits:

```
ac_intN::intN var_name; //Signed N-bit integer
```

```
ac_intN::uintN var_name; //Unsigned N-bit integer
```

Where, N is the total length of the integer in bits.

The `ac_int` data type has several API calls. For more information about the Algorithmic C data types, refer to <https://cdrdv2.intel.com/v1/dl/getContent/728986>.

Restriction

If you want to initialize an `ac_int` variable to a value larger than 64 bits, you must use the `bit_fill` or `bit_fill_hex` utility function. For details, refer to the section *Methods to Fill Bits* in the documentation provided in the [Algorithmic C \(AC\) Datatypes PDF file](#).

The following code example shows the use of the `bit_fill` or `bit_fill_hex` utility functions:

```
typedef ac_int<80,false> i80_t;
i80_t x;
x.bit_fill_hex("a9876543210fedcba987"); // member function
x = ac::bit_fill_hex<i80_t>("a9876543210fedcba987"); // global function
int vec[] = { 0xa987, 0x6543210f, 0xedcba987 };
x.bit_fill(vec); // member function
x = bit_fill<i80_t>(vec); // global function
// inlining the constant array
x.bit_fill( (int [3]) { 0xa987,0x6543210f,0xedcba987 } ); // member function
x = bit_fill<i80_t>( (int [3]) { 0xa987,0x6543210f,0xedcba987 } ); // global function
```

Debugging the `ac_int` Data Type Use

The `ac_int.hpp` header file provides tools to help you check `ac_int` data type operations and assignments for overflow.

Restriction

Currently, you can debug `ac_int` data type operations and assignments for overflow only in the emulation flow and only for non-kernel code.

NOTE

- When you use `DEBUG_AC_INT_WARNING` and `DEBUG_AC_INT_ERROR` macros, you cannot declare `constexpr ac_int` variables or `constexpr ac_int` arrays.

Macro	Description
<code>DEBUG_AC_INT_WARNING</code>	Emits a warning for each detected overflow.
<code>DEBUG_AC_INT_ERROR</code>	Emits a message for the first overflow that is detected and then exits the code with an error.

- Within your code, you must declare the macros before you include the `ac_int.hpp` header file.

Explicit Conversion Functions

The following table lists the functions to convert to C signed and unsigned integer types `int`, `long` and `Slong` for the `ac_int` data type:

Function	Return Type
<code>to_int()</code>	<code>int</code>
<code>to_uint()</code>	<code>unsigned int</code>
<code>to_long()</code>	<code>long</code>
<code>to_ulong()</code>	<code>unsigned long</code>
<code>to_int64()</code>	<code>Slong</code>
<code>to_uint64()</code>	<code>Ulong</code>
<code>to_double()</code>	<code>double</code>

NOTE

For additional information, refer to the FPGA tutorial sample "AC Int" on [GitHub](#).

Declare the `ac_fixed` Data Type

Perform the following steps to declare the `ac_fixed` data type:

1. Include the `ac_fixed.hpp` header file as follows:

```
#include <sycl/ext/intel/ac_types/ac_fixed.hpp>
```

Declare your `ac_fixed` variables as follows:

```
ac_fixed<W, I, S, Q, O> var_name;
```

The following table describes the template parameters:

Template Parameter	Description
<i>W</i>	The total length of the fixed-point number in bits.
<i>I</i>	The number of bits used to represent the integer value of the fixed-point number. The difference of $N-I$ determines how many bits represent the fractional part of the fixed-point number.
<i>S</i>	A Boolean value that indicates the signedness of the of the data type: <code>true</code> indicates a signed fixed-point number <code>false</code> indicates an unsigned fixed-point number

Template Parameter	Description
Q	The quantization mode that determines how to handle values where the generated precision (number of decimal places) exceeds the number of bits available in the variable to represent the fractional part of the number. For a list of quantization modes and their descriptions, refer to the <i>Quantization and Overflow</i> section in https://github.com/hlslibs/ac_types/blob/v3.7/pdftdocs/ac_datatypes_ref.pdf .
O	The overflow mode that determines how to handle values where the generated value has more bits than the number of bits available in the variable. For a list of overflow modes and their descriptions, refer to the <i>Quantization and Overflow</i> section in https://github.com/hlslibs/ac_types/blob/v3.7/pdftdocs/ac_datatypes_ref.pdf .

For a list of supported operators and their return types, refer to the *Arbitrary-Length Bit-Accurate Integer and Fixed-Point Datatypes* chapter in https://github.com/hlslibs/ac_types/blob/v3.7/pdftdocs/ac_datatypes_ref.pdf.

Explicit Conversion Functions

The following table lists the functions to convert to C signed and unsigned integer types `int`, `long` and `Slong` for the `ac_fixed` data type:

Function	Return Type
<code>to_int()</code>	<code>int</code>
<code>to_uint()</code>	<code>unsigned int</code>
<code>to_long()</code>	<code>long</code>
<code>to_ulong()</code>	<code>unsigned long</code>
<code>to_int64()</code>	<code>Slong</code>
<code>to_uint64()</code>	<code>Ulong</code>
<code>to_double()</code>	<code>double</code>
<code>to_float()</code>	<code>float</code>
<code>to_ac_int()</code>	<code>ac_int<max(I,1), S></code> , where <code>I</code> represents the integer width and <code>S</code> is a bool parameter representing the signedness of the <code>ac_int</code> data type.

Math Functions Provided by the `ac_fixed_math.hpp` Header File

Include the header file as follows:

```
#include <sycl/ext/intel/ac_types/ac_fixed_math.hpp>
```

The `ac_fixed_math.hpp` header file provides support for the following arbitrary precision fixed-point (`ac_fixed`) data type functions:

- `sqrt_fixed`
- `reciprocal_fixed`
- `reciprocal_sqrt_fixed`
- `sin_fixed`
- `cos_fixed`
- `sincos_fixed`
- `sinpi_fixed`
- `cospi_fixed`
- `sincospi_fixed`

- `log_fixed`
- `exp_fixed`

For details about input type restrictions, input value limits, and output type propagation rules, review the comments in the `ac_fixed_math.hpp` header file.

Important

Due to the differences in the internal math implementations, the results from the `ac_fixed` data type operations might differ between simulation and emulation. The maximum difference is within a few units in the last place (ULPs).

Declare the `ac_complex` Data Type

Perform the following steps to declare the `ac_complex` data type:

1. Include the `ac_complex.hpp` header file as follows:

```
#include <sycl/ext/intel/ac_types/ac_complex.hpp>
```

2. Declare your `ac_complex` variables according to the data type of your complex number.

The underlying data type can be `ac_int`, `ac_fixed`, `ap_float`, and standard-C integer or floating-point data types. The following are some examples of declaring `ac_complex` variables:

```
ac_complex<ac_int<5,true> > i(2, 1);
```

```
ac_complex<ac_fixed<8,3,false> f(1, 5);
```

For a list of supported operators and their return types, refer to *Complex Datatype* chapter in <https://cdrdv2.intel.com/v1/dl/getContent/728986>.

Declare the `ap_float` Data Type

The `ap_float.hpp` header file provides support for arbitrary-precision floating-point numbers. The floating-point representation for `ap_float` data types adopts the same data layout as the IEEE 754 floating-point representation.

An `ap_float` variable carries an explicit sign bit and an arbitrary number of bits for the exponent and mantissa. Due to the differences in the internal math implementations and rounding errors, the results from `ap_float` operations might not always be bit-accurate compared to those produced by C++ native floating-point types with the same exponent and mantissa bit widths.

Perform the following steps to declare the `ap_float` data type:

1. Include the `ap_float.hpp` header file as follows:

```
#include <sycl/ext/intel/ac_types/ap_float.hpp>
```

2. Declare your `ap_float` variables as follows:

```
ihc::ap_float<exponent_width, mantissa_width[,rounding_mode]>
```

Where, the template attributes are defined as follows:

- **`exponent_width, mantissa_width`**: The bit-width of the exponent and mantissa of the floating-point variable.

The `ap_float.hpp` header file also provides aliases to declare `bfloat16` and `bfloat19` data types directly. The `ap_float` data type supports the following `exponent_width, mantissa_width` combinations:

Exponent- and Mantissa-Width Combinations Supported by the `ap_float` Data Type

5, 10	8, 7	8, 10	8, 17	8, 23
8, 26	10, 35	11, 44	11, 52	15, 63

Some of these width combinations map to some commonly used floating-point formats, as listed in the following table:

Exponent- and Mantissa-Width Setting for Various Floating-point Formats

Floating-point Format	exponent_width, mantissa_width Setting
IEEE 754 half-precision (<i>binary16</i>)	5, 10
bfloat16	8, 7
bfloat19	8, 10
IEEE 754 single-precision (<i>binary32</i>)	8, 23
IEEE 754 double-precision (<i>binary64</i>)	11, 52
80-bit extended precision	15, 63

- **rounding_mode**: Optional parameter to specify the IEEE 754 rounding mode used when converting between data types. Set the rounding mode with one of the following values:

Rounding Mode Values

Rounding Mode	Description
<code>ihc::fp_config::FP_Round::RNE</code>	Round to the nearest, tie break to even. This rounding mode is more accurate (0.5 ULP) but requires more FPGA area.
<code>ihc::fp_config::FP_Round::RZERO</code>	Round towards zero. This rounding mode is less accurate (1 ULP) and requires less FPGA area.

NOTE

If you do not set the `rounding_mode` parameter, the `ihc::FP_Round::RNE` rounding mode is used by default.

Math Functions Supported by `ap_float` Data Type

The `ap_float` data type supports all overloaded math operators and a limited set of the math functions provided by the Intel® oneAPI DPC++/C++ Compiler. For some math operators, you can control the output's precision by using templated versions of the functions.

Important

Due to the differences in the internal math implementations and rounding errors, the results from `ap_float` operations might not always be bit-accurate when compared to those produced by C++ native floating-point types with the same exponent and mantissa-bit widths. However, these results are validated against the infinitely accurate results.

The following additional math functions are supported through the `ap_float_math.hpp` header file. All functions are in the `ihc` namespace.

- ¹ You can also declare this format directly as a `ihc::bfloat16` data type.
- ² You can also declare this format directly as a `ihc::bfloat19` data type.
- ³ You can also declare this format directly as a `ihc::FPsingle` data type.
- ⁴ You can also declare this format directly as a `ihc::FPdouble` data type.
- ⁵ Not a bit-to-bit mapping. The integer part (bit 63) of the 80-bit extended precision value is dropped when converting it to `ap_float<15,63>`.

Math Functions Provided by the `ap_float_math.hpp` header file

Function Type	Math Function	Comment
Exponential and logarithmic functions	• <code>ihc_exp</code>	Supported only for <code>ap_float</code> data types with exponent width less than or equal to 15 bits and mantissa width less than or equal to 63 bits.
	• <code>ihc_exp2</code>	
	• <code>ihc_exp10</code>	
	• <code>ihc_log</code>	
	• <code>ihc_log2</code>	
	• <code>ihc_log10</code>	
	• <code>ihc_expm1</code>	Supported only for <code>ap_float</code> data types with exponent width less than or equal to 11 bits and mantissa width less than or equal to 52 bits.
	• <code>ihc_log1p</code>	
	• <code>ihc_recip</code>	
	• <code>ihc_rsqrt</code>	
Advanced functions	• <code>ihc_sqrt</code>	
	• <code>ihc_cbrt</code>	
	• <code>ihc_hypot</code>	
Power functions	• <code>ihc_pow</code>	
	• <code>ihc_pown</code>	
	• <code>ihc_powr</code>	
Trigonometric functions	• <code>ihc_sin</code>	
	• <code>ihc_sinpi</code>	
	• <code>ihc_cos</code>	
	• <code>ihc_cospi</code>	
	• <code>ihc_sincos</code>	
	• <code>ihc_sincospi</code>	
	• <code>ihc_asin</code>	
	• <code>ihc_asinpi</code>	
	• <code>ihc_acos</code>	
	• <code>ihc_acospi</code>	
	• <code>ihc_atan</code>	
	• <code>ihc_atanpi</code>	
	• <code>ihc_atan2</code>	

Conversion Rules for the `ap_float` Data Type

You can convert between different sizes of `ap_float` data types through assignment or by using the `convert_to()` function. For example,

```
using namespace ihc;
ap_float<8, 23> myFloat = ...;
ap_float<5, 10> myFloat2 = myFloat; // use rounding rules defined by ap_float type

// use rounding rules defined in convert_to() function call
ap_float <5, 10> myFloat3 = myFloat.convert_to<5, 10, ihc::fp_config::FP_Round::RZERO>();
```

For two `ap_float` variables in a binary operation, the `ap_float` variable with the larger exponent bit-width is considered to be the *larger* variable. If two variables have the same exponent bit width, the variable with the larger mantissa bit-width is considered to be the *larger* variable. The operands are then unified to the *larger* type before the binary operation occurs.

To convert between `ap_float` and other data types:

- To convert between native data types (for example, `float`, `double`) and `ap_float` data types, assign to or from the types. Type conversion in an assignment occurs according to the rules mentioned in [Table 37: Default Conversion Rules for `ap\_float` Variables](#).
- To convert between `ac_int` data types and `ap_float` data types:
 - To convert from `ac_int` data types to `ap_float`, use explicit conversion. The type conversion occurs according to the rules mentioned in [Table 37: Default Conversion Rules for `ap\_float` Variables](#).
 - To convert from `ap_float` data types to `ac_int`, use the `to_ac_int<W,S>` function, where `W` is the bit-width, and `S` is a true/false value indicating if the `ac_int` variable is signed. For example,

```
using namespace ihc;
ac_int<5,true> myAcInt1 = ...;
auto myFloat = ap_float<8,23>(myAcInt1);
auto myAcInt2 = myFloat.to_ac_int<5,true>();
```

- To convert between `ac_fixed` data types and `ap_float`, cast to or from the data types. Type conversion occurs according to the rules mentioned in [Table 37: Default Conversion Rules for `ap\_float` Variables](#). For example,

```
using namespace ihc;
ac_fixed<8,7> myAcFixed = ...;
// use rounding rules defined by ap_float type
auto myFloat2 = ap_float<5,10,fp_config::FP_Round::RZERO>(myAcFixed);
// use rounding & overflow rules defined by ac_fixed type
auto myAcFixed2 = ac_fixed<10,0,true,AC_RND,AC_SAT>(myFloat2);
```

The Intel® oneAPI DPC++/C++ Compiler also provides some operations that leave the precision of input types untouched and provide control over the output precision. For more details, refer to [Operations with Explicit Precision Controls](#).

Default Conversion Rules for `ap_float` Variables

Data Type	From <code>ap_float</code> To Data Type	From Data Type To <code>ap_float</code>
<code>ap_float</code> with higher representable range	Keep exponent equivalent. The mantissa is rounded according to the rounding mode of the target <code>ap_float</code> (with the higher representable range).	$\pm\text{Inf}$ if the source of the conversion is out of the representable range. Otherwise, keep exponent equivalent. The mantissa is rounded according to the rounding mode of the target <code>ap_float</code> (with the smaller representable range). Important If the input number is in the subnormal range of the target <code>ap_float</code> number, the output value is flushed to zero.
<code>float</code>	Convert original <code>ap_float</code> to <code>ap_float<8, 23></code> with the previous <code>ap_float</code> rule, and then bit cast to <code>float</code> .	Bit-cast <code>float</code> to <code>ap_float<8, 23></code> , and then convert to target <code>ap_float</code> precision using the <code>ap_float</code> to <code>ap_float</code> rules described previously.
<code>double</code>	Convert original <code>ap_float</code> to <code>ap_float<11, 52></code> with earlier <code>ap_float</code> rule, and then bit cast to <code>double</code> .	Bit-cast <code>double</code> to <code>ap_float<11, 52></code> , and then convert to the target <code>ap_float</code> precision using the <code>ap_float</code> to <code>ap_float</code> rules described earlier.
C++ native integer types	Truncate towards zero. Converting from <code>ap_float</code> that is larger than the range of integer type is an undefined behavior.	Round to the nearest, tie breaks to even. If the integer value is too large, the <code>ap_float</code> value saturates to plus infinity.

Data Type	From <code>ap_float</code> To Data Type	From Data Type To <code>ap_float</code>
<code>ac_int</code>	Truncate towards zero. Converting from <code>ap_float</code> that is larger than the range of integer type is an undefined behavior. <code>ac_int</code> width in bits: $W \leq 64$	Round to the nearest, tie breaks to even. If the integer value is too large, the <code>ap_float</code> value saturates to $+\infty$ (plus infinity). <code>ac_int</code> width in bits: $W \leq 64$
<code>ac_fixed</code>	Rounding and overflow mode depend on the type parameters of the target <code>ac_fixed</code> Converting Inf/NaN to <code>ac_fixed</code> is an undefined behavior	Rounding mode depends on rounding mode type parameter of the target <code>ap_float</code> <code>ac_fixed</code> width in bits: $W \leq 64$

Notes

- Avoid assigning the result of the `convert_to` function to another `ap_float` variable. If the left-hand side of the assignment has a different exponent or mantissa widths than the ones specified in the `convert_to` function on the right-hand side, another conversion can occur.
- Converting between floating-point data types and integer or fixed-point data types on FPGA devices can be expensive in terms of area.

Operations with Explicit Precision Controls

The following operations leave the precision of the input `ap_float`-type variables untouched and allow you to control the output precision:

Rounding Mode Control For `ap_float` to `ap_float` Conversions**Syntax**

```
convert_to<output_exponent_width, output_mantissa_width, rounding_mode>
```

Description

Use this method to override the rounding mode set for an `ap_float` variable when converting the variable to a different precision. By default, `ap_float` to `ap_float` conversions use the rounding mode that you specified when you declared the variable.

Multiplication**Syntax**

```
ihc::ap_float<output_exponent_width, output_mantissa_width>::mul <[accuracy_setting],  
[subnormal_setting]> (ap_float_a, ap_float_b)
```

Description

This math function supplements the basic multiplication operation performed by the multiplication (*) operator. Multiplies `ap_float_a` and `ap_float_b` without changing the input types and outputs an `ap_float` at the specified precision. The optional parameters are defined as follows:

- **subnormal_setting**: Optional parameter to specify whether input and output numbers are flushed to zero when carrying out basic binary operations explicitly. Set this parameter with one of the following values:

subnormal_setting Parameter Values

subnormal_setting Values	Description
<code>ihc::fp_config::FP_Subnormal::ON</code>	Input and output numbers in the subnormal range are preserved. The target FPGA device must have subnormal support. Subnormal support might require more FPGA area.
<code>ihc::fp_config::FP_Subnormal::OFF</code>	Input or output numbers in the subnormal range are flushed to zero.
<code>ihc::fp_config::FP_Subnormal::AUTO</code>	With this setting, the Intel® oneAPI DPC++/C++ Compiler enables subnormal support only under the following conditions: • The target FPGA device directly supports it. • It does not incur any extra FPGA area overhead.

If you do not set the `subnormal_setting` parameter, the `ihc::fp_config::FP_Subnormal::AUTO` subnormal setting is used by default.

- **accuracy_setting:** Optional parameter that influences trade-offs between the accuracy of the result due to different rounding decisions in the intermediary calculations and the FPGA area utilized by the generated hardware. Floating-point operations with less accurate results typically use fewer logic elements. For example, a divider with a high accuracy might use 20% more FPGA area than a divider with low accuracy. The low accuracy divider has a higher error bound [1 unit of least precision (ULP)] than a high accuracy divider (0.5 ULP).

Set this parameter with one of the following values:

accuracy_setting Parameter Values

accuracy_setting Values	Description
<code>ihc::fp_config::FP_Accuracy::HIGH</code>	Uses the high precision version of the floating-point math operations. This is the default setting.
<code>ihc::fp_config::FP_Accuracy::LOW</code>	Allows the compiler to use a higher error bound to save on-chip area.

If you do not set the `accuracy_setting` parameter, `ihc::fp_config::FP_Accuracy::HIGH` accuracy setting is used by default.

Addition/Subtraction/Division**Syntax**

```
ihc::ap_float<output_exponent_width, output_mantissa_width>::add <[optional parameters]>
(ap_float_a, ap_float_b)
```

```
ihc::ap_float<output_exponent_width, output_mantissa_width>::sub <[optional parameters]>
(ap_float_a, ap_float_b)
```

```
ihc::ap_float<output_exponent_width, output_mantissa_width>::div <[optional parameters]>
(ap_float_a, ap_float_b)
```

Description

These math functions supplement the basic math operations performed by the addition/subtraction/division (+ / - //) operators. Adds/subtracts/divides `ap_float_a` and `ap_float_b` by first casting `ap_float_a` and `ap_float_b` to the specified `ap_float` precision. The operation and output are at the specified precision.

You can also specify the optional parameters `accuracy_setting` and `subnormal_setting` described earlier.

NOTE

If your SYCL code performs computations on constant/literal native floating-point values, the compiler can sometimes combine them at compile time and save area. This is a compiler optimization technique called *constant folding* or *constant propagation*. This optimization does not work for `ap_float` even when the operands are constant. You should compute your constant arithmetic in native types or precompute manually.

Comparison Operators

Comparison operators (`>`, `<`, `==`, `!=`, `>=`, `<=`) are subject to the conversion rules described in [Conversion Rules for the `ap\_float` Data Type](#).

The `==` and `!=` operators impose a bit-wise comparison of the casted values.

Comparisons with NaN always return `false`.

Additional `ap_float` Functions

The `ap_float` data type also provides the following additional functions:

Additional `ap_float` Functions

Function	Description
Getters and Setters	
<code>ap_float::get_exponent</code>	Gets/sets the exponent value of the <code>ap_float</code> variable.
<code>ap_float::set_exponent</code>	
<code>ap_float::get_mantissa</code>	Gets/sets the mantissa value of the <code>ap_float</code> variable.
<code>ap_float::set_mantissa</code>	
<code>ap_float::get_sign</code>	Gets/sets the sign bit of the <code>ap_float</code> variable.
<code>ap_float::set_sign</code>	
Special Constants	
<code>ap_float<e,m>::nan()</code>	Assigns the <code>ap_float</code> variable a value of NaN.
<code>ap_float<e,m>::pos_inf()</code>	Assigns the <code>ap_float</code> variable a value of $+\infty$.
<code>ap_float<e,m>::neg_inf()</code>	Assigns the <code>ap_float</code> variable a value of $-\infty$.
Value Queries	
<code>ap_float::is_nan()</code>	Returns <code>true</code> if the value of the <code>ap_float</code> variable is NaN.
<code>ap_float::is_inf()</code>	Returns <code>true</code> if the value of the <code>ap_float</code> variable is $\pm\infty$.
<code>ap_float::is_zero()</code>	Returns <code>true</code> if the value of the <code>ap_float</code> variable is zero.
Special Functions	
<code>ap_float::next_after(next_val)</code>	Returns the next representable value towards <code>next_val</code> .

Additional Data Types Provided by the `ap_float.hpp` Header File

The `ap_float.hpp` header file provides some aliases for certain data types that you can use instead of explicitly declaring an `ap_float` data type.

Data Type	Description
<code>ihc::bfloat16</code>	A 16-bit floating-point number with an 8-bit exponent and a 7-bit mantissa (equivalent to declaring <code>ihc::ap_float<8, 7></code>).

Data Type	Description
<code>ihc::bfloat19</code>	On Agilex™ 7 devices, dot product operations that involve the <code>bfloat16</code> (or <code>ihc::ap_float<8, 7></code>) data type are mapped to FP16 DSPs. On other device families, dot product operations are mapped to ALMs and fixed-point 18-bit DSPs. On all device families, all other math functions are mapped to ALMs and fixed-point 18-bit DSPs. A 19-bit floating-point number with an 8-bit exponent and a 10-bit mantissa (equivalent to declaring <code>ihc::ap_float<8, 10></code>). On Agilex™ 7 devices, dot product operations that involve the <code>bfloat19</code> (or <code>ihc::ap_float<8, 10></code>) data type are mapped to FP19 DSPs. On other device families, dot product operations are mapped to ALMs and fixed-point 18-bit DSPs. On all device families, all other math functions are mapped to ALMs and fixed-point 18-bit DSPs.
<code>ihc::FPsingle</code>	A 32-bit floating-point number with an 8-bit exponent, a 23-bit mantissa, and a "Round to the nearest, tie break to even" rounding mode (equivalent to declaring <code>ihc::ap_float<8, 23, fp_config::FP_Round::RNE></code>). Math operations with <code>FPsingle</code> map to the same hardware as the native <code>float</code> data type when the native <code>float</code> data type uses IEEE-754 round nearest ties to even (RNE) mode. For information about controlling the rounding mode for native floating-point data types, refer to Modify the Rounding Mode of Floating-point Operations (-Xsrounding=<rounding_type>).
<code>ihc::FPdouble</code>	A 64-bit floating-point number with an 11-bit exponent, a 52-bit mantissa, and a "Round to the nearest, tie break to even" rounding mode (equivalent to declaring <code>ihc::ap_float<11, 52, fp_config::FP_Round::RNE></code>). When subnormal support is on, math operations with <code>FPdouble</code> map to the same hardware as the native <code>double</code> data type. For information about controlling the subnormal settings of <code>FPdouble</code> (and other <code>ap_float</code> data types), refer to Operations with Explicit Precision Controls.

Quality of Results and the `ap_float` Data Type

Replacing C++ native `float` and `double` data types with the `ihc::FPsingle` and `ihc::FPdouble` data types can result in different quality of results (QoR) for your design.

Use the techniques in the following sections to influence the QoR that you get when using floating-point data types.

Tuning Accuracy and Subnormal Settings with Explicit Operations

If you notice a difference in QoR when using `ihc::FPsingle` instead of `float`, set `FP_Subnormal` to `OFF` and set `FP_Accuracy` to `LOW`.

If you notice a difference in QoR when using `ihc::FPdouble` instead of `double`, set `FP_Subnormal` to `ON` and set `FP_Accuracy` to `HIGH`.

Important These settings might not eliminate a QoR gap between a C++ native data type and its `ap_float` equivalent, but they should get the QoR as close to the native data type as possible. Fast math, which is turned on by default, applies to the C++ native floating-point data types but not to `ap_float` data types. When fast math is enabled, the QoR gap between a C++ native data type and its `ap_float` equivalent can still be large.

For example, consider the changes to the following code snippet that uses C++ native data types:

```
float a_times_b(float a, float b)
{
    float ab = a * b;
    return ab;
}
```

You can change this code to use `ap_float` controls as follows:

```
ihc::FPsingle a_times_b(ihc::FPsingle a, ihc::FPsingle b)
{
    constexpr auto AccLOW = ihc::fp_config::FP_Accuracy::LOW;
    constexpr auto SubON = ihc::fp_config::FP_Subnormal::ON;
    auto ab = ihc::FPsingle::mul<AccLOW, SubOFF>(a, b);
    return ab;
}
```

Cast Constant Multiplication Operations to Native Data Types

For single-precision data types (`ihc::FPsingle` and `ihc::ap_float<8,23>`) and double-precision data types (`ihc::FPdouble` and `ihc::ap_float<11,52>`), casting constant multiplications to native data types can help with QoR.

For example, consider the following multiplication code snippet using C++ native data types:

```
float four_times_a(float a)
{
    float fourA = 4.0f * a;
    return fourA;
}
```

By using the earlier technique, you can convert that code snippet in the following code:

```
ihc::FPsingle four_times_a(ihc::FPsingle a)
{
    constexpr auto AccLOW = ihc::fp_config::FP_Accuracy::LOW;
    constexpr auto SubON = ihc::fp_config::FP_Subnormal::ON;
    auto fourA = ihc::FPsingle::mul<AccLOW, SubOFF>(
        ihc::FPsingle(4.0f), a);
    return fourA;
}
```

By casting the multiplication operation to native data types, you can improve the QoR of that code:

```
ihc::FPsingle four_times_a(ihc::FPsingle a)
{
    auto fourA = ihc::FPsingle(4.0f * ((float)a));
    return fourA;
}
```

Parallelism

Pipelined Kernels

By default, SYCL* task kernels are not pipelined. A kernel blocks further kernel invocations from starting until the current invocation has completed.

However, kernels with a ready/valid handshaking invocation interface can be pipelined. For information about kernels with a ready/valid handshaking invocation interface, refer to [Ready/Valid Handshaking Kernel Invocation Interface](#).

You can pipeline kernels by using the `pipelined` kernel property.

Important

Separate invocations of a kernel are **independent**. The independence of separate kernel invocation means that you cannot make assumptions about memory ordering or memory dependencies between kernel invocations.

Ensure that you use synchronization mechanisms such as the `.wait()` function or [SYCL* atomic operations](#) to avoid race conditions.

If you want to guarantee sequential equivalence, write your kernel with a `while(1)` loop in the kernel body instead of using a pipelined kernel.

This restriction can affect you if you are migrating code from the Intel® HLS Compiler to SCYL* code. Repeatedly-invoked kernel code that would have worked correctly when compiled by the Intel® HLS Compiler might result in undefined behavior in SYCL* code and might not function as you expect.

The `pipelined` Kernel Property

To pipeline a streaming kernel, add the `pipelined<N>` property to your streaming kernel.

The `pipelined<N>` property takes the following values for *N*:

Property	Description
<code>pipelined<-1></code>	The compiler generates hardware that allows kernel invocations to execute in a pipelined fashion, while attempting to achieve the lowest possible II (initiation interval) at the targeted f_{MAX} .
<code>pipelined<></code>	
	This is the default behavior if no value for <i>N</i> is specified.
<code>pipelined<N></code>	Where <i>N</i> represents the desired II value.
where <i>N</i> > 0	The compiler generates hardware that allows kernel invocations to execute in a pipelined fashion, while attempting to achieve the specified II (<i>N</i>) at the targeted f_{MAX} .
<code>pipelined<0></code>	The compiler does not generate hardware to allow kernel invocations to execute in a pipelined fashion.
	This is the equivalent of not specifying the <code>pipelined</code> kernel property.

To use the `pipelined` kernel property:

1. Include the following header file in your code:

```
sycl/ext/intel/fpga_extensions.hpp
```

2. Label your kernel with the `pipelined` property as follows:

- **Functor Model:**
 1. Add a member function named `get` to the functor. Have the `get` function take an argument of type `properties_tag` and a return type `auto`.
 2. Create a `properties` object in the new function with the `streaming_interface` and `pipelined` properties and return it.

Functor Model `pipelined` Kernel Property Code Example

```
#include <sycl/sycl.hpp>
#include <sycl/ext/intel/fpga_extensions.hpp>

using namespace sycl;
using namespace sycl::ext::intel::experimental;
using namespace sycl::ext::oneapi::experimental;

struct MyFunctorIP {
    int *input_a, *input_b, *input_c;

    void operator()() const {
        *input_c = *input_a + *input_b
    }
}

auto get(properties_tag) {
    return properties{streaming_interface<>, pipelined<>};
}

/* To exercise the pipelined nature of the kernel in simulation,
you must queue up multiple instances of the functions before you
call the wait() function. The following code example shows how to
exercise a pipelined kernel: */

for (int i = 0; i < kN; i++) {
    q.single_task(MyFunctorIP{functor_input_A, functor_input_B, functor_input_C});
}
q.wait();
```

- **Lambda Model:**
 1. Pass a `properties` object that contains the `streaming_interface` and `pipelined` properties to your `q.single_task` call.

Lambda Model `pipelined` Kernel Property Code Example

```
#include <sycl/sycl.hpp>
#include <sycl/ext/intel/fpga_extensions.hpp>

using namespace sycl;
using namespace sycl::ext::intel::experimental;
using namespace sycl::ext::oneapi::experimental;

class MyLambdaIP;

// Create a properties object containing the kernel invocation
// interface property
properties kernel_properties{streaming_interface_remove_downstream_stall, pipelined<>};
```

```
...

/* To exercise the pipelined nature of the kernel in simulation,
you must queue up multiple instances of the functions before you
call the wait() function. The following code example shows how to
exercise a pipelined kernel: */

for (int i = 0; i < kN; i++) {
    q.single_task<MyLambdaIP>(kernel_properties, [=] {
        lambda_input_C[i] = lambda_input_A[i] + lambda_input_B[i];
    });
}
q.wait();
```

Stable Arguments

By default, the Intel® oneAPI DPC++/C++ Compiler assumes that the values of kernel arguments change during kernel executions.

For pipelined kernels, you can mark a kernel argument as stable if the kernel and kernel argument satisfy the following conditions:

- The kernel argument does not change during the execution of any invocation of the kernel.
- The kernel is not launched with a different kernel argument value when a different invocation of this kernel is still running.

Declare a streaming (conduit) kernel argument to be stable with the `stable` property

Changing the value of a stable kernel argument results in undefined behavior.

You might save some FPGA area in your kernel design when you declare a streaming (conduit) kernel argument as stable.

If all the kernel arguments do not change while the kernel is executing, you can include the `-Xsno-hardware-kernel-invocation-queue` option in your `icpx` command.

Changing the value of a kernel argument on a kernel compiled with the `-Xsno-hardware-kernel-invocation-queue` option results in undefined behavior.

NDRange Kernels

FPGA Acceleration Flow Only The information presented here applies only to multiarchitecture binary kernels in the FPGA acceleration flow. It does not apply for RTL IP core kernels in the SYCL* HLS flow.

If your program naturally tends to describe multiple concurrent threads operating in a data-parallel manner, specify your kernel to operate in parallel instances over a work-item index-space (NDRange).

Avoid Work-Item ID-Dependent Backward Branching

The Intel® oneAPI DPC++/C++ Compiler collapses conditional statements into single bits that indicate when a particular functional unit becomes active. The Intel® oneAPI DPC++/C++ Compiler eliminates simple control flow paths that do not involve looping structures, resulting in a flat control structure and more efficient hardware use.

Avoid including any work-item ID-dependent backward branching (that is, branching that occurs in a loop) in your kernel because it degrades performance.

For example, the following code fragment illustrates branching that involves work-item ID such as `get_global_id` or `get_local_id`:

```
for (size_t i = 0; i < get_global_id(0); i++)
{
    // statements
}
```

Asynchronous Parallelism Within Kernels (`task_sequence`)

Your kernel design might contain operations that you want to run asynchronously from the main flow of your kernel. The Intel oneAPI DPC++/C++ Compiler allows you to define these asynchronous activities in task functions and to asynchronously launch parallel invocations of these task functions through object instances of the `task_sequence` class. To enable the `task_sequence` class, include the following `task_sequence` header file in your source code:

```
#include <sycl/ext/intel/experimental/task_sequence.hpp>
```

The `task_sequence` class is a templated class in the `sycl::ext::intel::experimental` namespace. The template parameters include a reference to the task function to be associated with the class and optional parameters specifying the depth of the queues for launching the tasks and holding their results. Instantiated objects of a parameterized instance of `task_sequence` represent the FPGA hardware implementing the associated task function and task queues. You can control the amount of replication or hardware reuse by the number of objects you declare.

The `task_sequence` class objects are helpful in situations where you want to express coarse-grained thread-level parallelism. For example:

- Improving the performance of operations like executing loops in parallel.
- Reducing FPGA area utilization by sharing an expensive compute block with different parts of your kernel.

Learn more about using the `task_sequence` class with the following tutorials on GitHub:

- [Hardware reuse](#)
- [Parallel loops](#)

`task_sequence` Template Parameters

Template Parameter	Description
<code>auto &f typename ReturnT, typename... ArgsT, ReturnT (&f) (ArgsT...)</code>	A callable object <code>f</code> that defines the asynchronous task to be associated with the <code>task_sequence</code>. The callable object <code>f</code> must meet the following requirements: • The object <code>f</code> must be statically resolvable at compile time, which means it is not a function pointer. • The object <code>f</code> must not be an overloaded function. • The return type (<code>ReturnT</code>) and argument types (<code>ArgsT...</code>) of object <code>f</code> must be resolvable and fixed.
<code>uint32_t invocation_capacity</code>	The size of the hardware queue instantiated for <code>async()</code> function calls. This parameter value corresponds to the minimum number of outstanding <code>async()</code> function calls to be supported. When the outstanding number of <code>async()</code> function calls reach this value, further calls may block until the number of outstanding calls is reduced to the <code>invocation_capacity</code>. The default value of this parameter is 1.
<code>uint32_t response_capacity</code>	The size of the hardware queue instantiated to hold task function results. This parameter value corresponds to the maximum number of outstanding <code>async()</code> calls such that all outstanding tasks are guaranteed to make forward progress. Further <code>async()</code> calls may block until the number of outstanding calls reduce to the <code>response_capacity</code>. The default value of this parameter is 1.

task_sequence Function APIs

Function API	Description
<code>void async(ArgsT... Args)</code>	Asynchronously calls <code>f</code> with arguments <code>Args</code> . It increments the number of outstanding tasks by 1.
<code>ReturnT get()</code>	Synchronously retrieves the result of an asynchronous call. Results are retrieved in FIFO order of their <code>async()</code> invocations. It decrements the number of outstanding tasks by 1.
<code>~task_sequence()</code>	Destructor for the <code>task_sequence</code> class. It implicitly invokes the <code>get()</code> function on all outstanding invocations launched through the <code>async()</code> function call.

Task Functions

Task functions for the `task_sequence` class are defined as a C++ function. A reference to this C++ function is used as the first template parameter to a specific parameterization of the `task_sequence` class. More specifically, the template parameter is an auto reference to a callable `f` that defines the asynchronous task to be associated with the `task_sequence` class. The requirement for an auto reference amounts to a requirement that `f` be statically resolvable at compile time. For example, `f` cannot be a function pointer. Furthermore, the return type and argument types of `f` must be resolvable and fixed for each parameterization of the `task_sequence` class.

Task Invocation Interface

You can invoke the task function `f` with the `async()` function, which accepts the same arguments as `f` (both in terms of argument type and order), and stores the return value in a FIFO queue upon completion of `f`. The `async()` function call is non-blocking, and it returns before the asynchronous `f` invocation completes executing and potentially before `f` even begins executing, as the return type from the `async()` function provides no implicit information on the execution status of `f`.

The `get()` function retrieves the oldest result from this logical FIFO queue and blocks (waits) until a result is available if no result is available immediately upon the call to the `get()` function. The return type of the `get()` function is the same as the return type of the task function `f`.

You can invoke both functions only on the device on which you have declared a `task_sequence` object. Calling `async()` or `get()` function on a different device results in undefined behavior.

In the optimization report (`report.html`), `async()` and `get()` function calls appear as blocking pipe write and pipe read operations.

The following example uses the task function `mult` to parameterize the `task_sequence` class with two object declarations, `task1` and `task2`. These objects asynchronously call `mult()` function twice using the `async()` function on each object, with their results collected by calls to the `get()` function on each object.

```
int mult(int a, int b) {
    return a * b;
}
int mult_and_add(int a1, int b1, int a2, int b2) {
    task_sequence<mult> task1, task2;
    task1.async(a1, b1);
    task2.async(a2, b2);
    return task1.get() + task2.get();
}
```

NOTE

The arguments to the `async()` function match the arguments to the `mult()` function (that is `(int, int)`). The return type to the `get()` function matches the return type to the `mult()` function (that is `(int)`).

Scope, Lifetime, and Reuse of `task_sequence` Objects

`task_sequence` objects must follow these guidelines:

- Object declarations of a parameterized `task_sequence` class must be local, which means global declarations and dynamic allocations are not allowed.
- `task_sequence` objects must not have their lifetime extended beyond the scope in which they are declared. It is undefined behavior if their lifetime is extended. Both `move` and `copy` constructors for the `task_sequence` class are therefore deleted.
- Each `task_sequence` class object represents a specific instantiation of FPGA hardware to perform the task function operation.
- Launching tasks via the `async()` function calls on the same object results in the reuse of that object's hardware. Thus, you can control the reuse or replication of FPGA hardware by the number of `task_sequence` objects you declare. Since object lifetime is confined to the scope in which the `task_sequence` object is created, carefully declare your object in the scope in which you intend to perform its reuse.
- A task function associated with a `task_sequence` can contain `task_sequence` declarations. In such cases, each object instantiation of the base `task_sequence` class results in new object declaration of the contained objects. In the following example, `baseTask` and `childTask` are task functions that parameterize the `task_sequence` class. In the kernel code, two `task_sequence<baseTask>` objects are declared, which means that two hardware instances implementing `baseTask` are instantiated. Since `baseTask` includes two declarations of `task_sequence<childTask>`, each `task_sequence<baseTask>` object instantiates two `task_sequence<childTask>` objects and therefore two hardware instantiations of `childTask`. Since there are two `task_sequence<baseTask>` objects in this code, there are a total of four `task_sequence<childTask>` objects and four hardware instantiations of `childTask`.

```
void childTask() {
    // useful computation here
    ...
}

void baseTask() {
    task_sequence<childTask> child1, child2;
    // useful computation here, including async() and get() on child1, child2
    ...
}

// in kernel code
{
    task_sequence<baseTask> base1, base2;
    ...
}
```

- Before exiting scope, `task_sequence` objects must retire all outstanding `async()` function invocations. This is guaranteed by the `task_sequence` destructor, which calls the `get()` function several times matching the count of outstanding `async()` functions calls.

Adding Capacity When Launching Task Functions

Consider specifying the `invocation_capacity` parameter of the `task_sequence` class if you observe stall patterns in your simulation waveforms that indicate an imbalance between the following:

- Any backpressure introduced by the task function.
- How often you invoke the task function using the `async()` function.

Adding Capacity When Collecting Task Functions

Consider specifying the `response_capacity` parameter of the `task_sequence` class if you observe stall patterns in your design waveforms that indicate a difference in the following:

- The cadence of data production in the task function.
- The cadence of collecting task results via the `get()` function.

task_sequence Use Cases

Using a `task_sequence` in your kernel enables a variety of design structures that you can implement. Common uses for the `task_sequence` class include executing multiple loops in parallel, as described in the following section.

Executing Multiple Loops in Parallel

Using the `task_sequence` class, you can run sequential loops in a pipelined manner within the context of the loop nest.

For example, in the following code sample, the first and second loops cannot be executed in parallel by the same invocation of the kernel they are contained in:

```
// first loop
for (int i = 0; i < n; i++) {
    // Do something
}
// second loop
for (int i = 0; i < m; i++) {
    // Do something else
}

// program scope
void firstLoop() {
    for (int i = 0; i < n; i++) {
        // Do something
    }
}
void secondLoop() {
    for (int i = 0; i < m; i++) {
        // Do something else
    }
}

// in kernel code
using namespace sycl::ext::intel::experimental;
task_sequence<firstLoop> firstTask;
task_sequence<secondLoop> secondTask;
firstTask.async();
secondTask.async();
firstTask.get();
secondTask.get();
```

Tip

For additional information, refer to the [Parallel Loops sample on GitHub](#).

Memories and Memory Operations

17

The `device_global` Extension (Beta)

The `device_global` class introduces device-scoped memory allocations into SYCL that can be accessed within a kernel using C++ global variable syntax. These memory allocations have unique instances per SYCL device. Like global variables, the `device_global` class is declared at a namespace scope and visible to all kernels within that scope.

Notice

In the current oneAPI release, the `device_global` class supports only a few features on FPGAs that are described in this topic. Subsequent releases might include more features of this class. For detailed information about the `device_global` API, refer to the [SYCL `device\_global` Language Specification](#).

The `device_global` class is a class template that is parameterized by the underlying allocation type and a list of optional properties (see [Table 43: Properties of the `device\_global` Class](#)). The type of the allocation also encodes the size of the allocation for potentially multidimensional array types.

Example: Declaring a `device_global` Object

The following is an example of the `device_global` object declaration that consists of an `int` array annotated with `device_image_scope` and `host_access_none` properties:

```
#include <sycl/sycl.hpp>

namespace exp = sycl::ext::oneapi::experimental;

using FPGAProperties = decltype(exp::properties(
    exp::device_image_scope, exp::host_access_none));
exp::device_global<int[10], FPGAProperties> val;
```

In the above example, the `device_image_scope` property limits the lifetime of the `val` variable. The `host_access_none` property asserts that the host code does not copy to or from the `device_global`. These properties are further described in [Table 43: Properties of the `device\_global` Class](#).

NOTE

The current implementation does not support the `device_global` declaration without `device_image_scope` and `host_access_none` properties.

Accessing a `device_global` Object

The following is an example of accessing a `device_global` object:

```
int main () {
    sycl::queue q;
    q.single_task([=] {
        val[0] = 42;
        ...
    }).wait();
}
```

```
int val_0;
q.copy(val, &val_0, 1 /*count*/, 0 /*startIndex*/).wait();
}
```

In most cases, you can directly access the underlying data through overloads of common operators. Host code needs to enqueue copy operations onto the queue to access the variable.

Initializing the `device_global` Class

For data types that have a `constexpr` constructor, you can constant-initialize a `device_global` object. To constant-initialize the `device_global` object, include the `-std=c++20` compiler command option.

```
device_global<int, decltype(properties(device_image_scope, host_access_none))> DGInt{3};
```

Properties of the `device_global` Class

The following table describes several compile-time-constant properties that the `device_global` class supports:

Properties of the `device_global` Class

Property	Description
<code>device_image_scope</code>	Mandatory property for FPGAs. If you set this property, the <code>device_global</code> memory lifetime begins when you program the FPGA binary image on an FPGA and ends when you program a different FPGA binary image on that FPGA.
<code>host_access</code>	Dictates how the host code accesses the <code>device_global</code> class and enables compiler optimizations. The following values are supported: <code>host_access_none</code>: Asserts that the host code never copies to or from the variable. For an FPGA device, no external ports are exposed. <code>host_access_read</code>: The user asserts that the host code may copy from (read) the variable, but it will never copy to (write) it. For an FPGA device, only a read port is exposed on the device image. <code>host_access_write</code>: The user asserts that the host code may copy to (write) the variable, but it never copies from (read) it. For an FPGA device, only a write port is exposed on the device image. <code>host_access_read_write</code>: The user provides no assertions, and the host code may either copy to (write) or copy from (read) the variable. This is the default if the property is omitted. For an FPGA device, a read/write port is exposed on the device image. FPGA acceleration Only <code>host_access_none</code> is supported for multiarchitecture binaries SYCL HLS All values are supported.

NOTE

For additional information, refer to the FPGA tutorial sample “Device Global” on [GitHub](#).

Hardware Implementation

Global variables are not captured implicitly by the lambda, so the compiler need not pass `device_global` variables as kernel arguments to the generated IP core.

A `device_global` variable with a `device_image_scope` property is implemented in on-chip memory, which carries all the on-chip memory advantages, as described in [Perform Kernel Computations Using Local or Private Memory](#).

If you use a compiler-generated IP core in your hardware system, the values stored in a `device_global` variable can be reset to their initial values (reinitialized) only by reprogramming the device image on to the FPGA.

Host Access

Each `device_global` variable that allows for host (external) access exposes a dedicated Avalon® memory-mapped interface at the RTL module boundary. Accesses from this interface are not stallable and are also given priority over any arbitrated with any stallable kernel (internal) accesses. Addressing for this variable starts at 0x0 and goes up to the size of the variable.

FPGA acceleration If a BSP does not support dedicated Avalon® memory-mapped interfaces for accessing `device_global` variables then host access on that BSP is not supported.

The `annotated_ptr` Template Class (Beta)

Use the `annotated_ptr` template class to annotate variable pointers in your kernel with a specific buffer location. This annotation directs the compiler to constrain memory accesses and build more efficient FPGA hardware.

The `annotated_ptr` template class is provided in the `sycl.hpp` header file. To use the `annotated_ptr` template class, use the following declarations in your code:

```
#include <sycl/sycl.hpp>
using namespace sycl::ext::intel::experimental;
using namespace sycl::ext::oneapi::experimental;
```

The `annotated_ptr` template class has the following syntax:

```
annotated_ptr<data type, decltype(properties { buffer_location<id> } ) >
```

Where the *data type* indicates the data type of the underlying pointer and *id* in `buffer_location<id>` corresponds to a memory location defined as follows, depending on your FPGA development flow:

- **SYCL* HLS Flow** The memory location must be a buffer location that you defined in the kernel interface with the `annotated_arg` template class. For details about defining buffer location in a kernel interface, refer to `sycl::ext::intel::experimental::buffer_location` in [The annotated_arg Template Class](#).
- **FPGA Acceleration Flow** The memory location must be one of the global memories defined in `board_spec.xml` file of your FPGA platform. To identify the index of the `buffer_location<index>` property, refer to [Partitioning Buffers Across Different Memory Types \(Heterogeneous Memory\)](#).

Important If the buffer location ID specified with the `annotated_ptr` does not match the actual buffer location of the pointer at run time the behavior is undefined. Ensure that the pointers that are being annotated inside the kernel are allocated with the matching buffer location via USM memory allocation APIs such as `malloc_shared` or `aligned_alloc_shared`.

Construction of an `annotated_ptr` Instance

Before using an instance of `annotated_ptr`, it must be explicitly constructed in one of the following ways:

```
datatype *p = ...;
annotated_ptr<datatype, decltype(properties { buffer_location<id> } )> ap {p};
```

```
datatype *p = ...;
annotated_ptr ap { p, properties { buffer_location<id> } };
```

Example of Using the `annotated_ptr` Template Class

The following example demonstrates how to constrain external memory accesses via pointers inside a kernel. The kernel in the example takes two `annotated_arg` arguments:

- `pX` with buffer location 1
- `Y` with buffer location 2

These `annotated_arg` arguments direct the compiler to build an interconnect that allows accessing the data in the following ways:

- Data that the `pX` pointer points to is accessed only through memory-mapped host interface 1 (`buffer_location<1>`).
- Data that the `Y` pointer points to is accessed only through memory-mapped host interface 2 (`buffer_location<2>`).

For more details on the `annotated_arg` class, refer to [Memory-Mapped Host Interfaces Using Unified Shared Memory Pointers and the `annotated\_arg` Class](#).

When dereferencing the `pX` pointer to get another pointer `temp`, the buffer location for the `temp` pointer is ambiguous. Use the `annotated_ptr` class to specify the buffer location for the `temp` pointer, so that the compiler needs to build an interconnect only between the load unit and memory-mapped host interface 1 when reading the data that the `temp` pointer points to.

```
struct MyIP {
    annotated_arg<int **, decltype(properties{buffer_location<1>})> pX;
    annotated_arg<int *, decltype(properties{buffer_location<2>})> Y;

    void operator() () const {
        for (int i = 0; i < N; i++) {
            int *temp = pX[i]; // The buffer location of temp is unknown

            // Use annotated_ptr to constrain accesses via temp to buffer location 1
            annotated_ptr<int, decltype(properties{buffer_location<1>})> X{temp};
            Y[i] = X[i];
        }
    }
};
```

Memory Accesses

Memory access efficiency often dictates the overall performance of your SYCL* kernel. Refer to [Memory Types](#) for an introduction to memory accesses.

The pipeline parallel nature of SYCL execution on FPGA means that memory loads and stores in your SYCL code compete for access to memory resources (global, local, and private memories). If your SYCL kernel performs a large number of memory accesses, the compiler must generate arbitration logic to share the available memory bandwidth between memory access sites in your kernel's datapath. If the bandwidth demanded by the datapath exceeds what the memory and arbitration logic can provide, the datapath stalls. This degrades the kernel's throughput because the compute pipeline must wait for a memory access before resuming.

When optimizing your design, it is important to understand whether your kernel's throughput is limited by memory accesses (a memory-bound kernel) or by the structure of the kernel datapath (a compute-bound kernel). These situations require different optimization techniques. The following sections discuss memory access optimization in detail.

Consider the following when developing your SYCL code:

- The maximum computation bandwidth of an FPGA is much larger than the available global memory bandwidth.
- The available global memory bandwidth is much smaller than the local and private memory bandwidth.
- Minimize the number of global memory accesses.

Load-Store Units

The Intel® oneAPI DPC++/C++ Compiler generates several types of load-store units (LSUs). For some types of LSU, the compiler might modify the LSU behavior and properties depending on the memory access pattern and other memory attributes. Refer to the following topics for more information:

Load-Store Unit Styles

The Intel® oneAPI DPC++/C++ Compiler generates different styles of load-store units (LSUs) based on:

- Inferred memory access pattern
- Types of memory available on the target platform
- Whether the memory accesses are to the local or global memory

The Intel® oneAPI DPC++/C++ Compiler can generate the following styles of LSUs:

- [Burst-Coalesced Load-Store Units](#)
- [Prefetching Load-Store Units](#)
- [Pipelined Load-Store Units](#)

Burst-Coalesced Load-Store Units

A burst-coalesced LSU is the default LSU style instantiated by the compiler for accessing global memory. It buffers contiguous memory requests until the largest possible burst can be made. For noncontiguous memory requests, a burst-coalesced LSU flushes the buffer between requests.

While a burst-coalesced LSU provides efficient, variable-latency access to global memory, a burst-coalesced LSU requires a considerable amount of FPGA resources.

The following example code results in the compiler instantiating burst-coalesced LSUs:

```
cgh.single_task<class Kernel>([=] {  
  
    int x = input_accessor[RandomIndex]; //burst-coalesced  
    output_accessor[0] = x;  
});
```

Depending on the memory access pattern and other attributes, the compiler might modify a burst-coalesced LSU in the following ways:

- [Cached](#)
- [Write-Acknowledge \(write-ack\)](#)

- **Nonaligned**

Prefetching Load-Store Units

A prefetching LSU instantiates a FIFO that burst-reads large memory blocks to keep the FIFO full of valid data based on the previous address and assumes contiguous reads. Noncontiguous reads are supported, but a penalty is incurred to flush and refill the FIFO. A prefetching LSU is inferred only for nonvolatile global pointers.

The following example code results in the compiler instantiating prefetching LSUs to access global memory:

```
cgh.single_task<class Kernel>([=] {
    int x = 1;
    for (int i = 0; i < VectorSize; i++) {
        x = x + input_accessor[i]; //prefetching
    }
    output_accessor[0] = x;
});
```

Pipelined Load-Store Units

A pipelined LSU is used for accessing local memory. Memory requests are submitted immediately after they are received. Memory accesses are pipelined, so multiple requests can be in flight at a time. If there is no arbitration between the LSU and the local memory, a pipelined never-stall LSU is created.

```
cgh.single_task<class Kernel>([=] {

    const unsigned LMEM_SIZE = 128;
    int lmem[LMEM_SIZE];
    for (int i = 0; i < LMEM_SIZE; i++) {
        lmem[i] = i * 100; //pipelined
    }
    output_accessor[0] = lmem[input_accessor[0]];
});
```

The compiler might modify a local-pipelined LSU as a never-stall LSU. For more details, refer to [Never-stall](#).

The Intel® oneAPI DPC++/C++ Compiler may also infer a pipelined LSU for global memory accesses that can be proven to be infrequent. The compiler uses a pipelined LSU for such accesses because a pipelined LSU is smaller than other LSU styles. While a pipelined LSU might have lower throughput, this throughput tradeoff is acceptable because memory accesses are infrequent.

Load-Store Unit Modifiers

Depending on the memory access pattern in your kernel, the compiler modifies some LSUs.

Cached

Burst-coalesced LSUs might sometimes include a cache. A cache is created when the memory access pattern is data-dependent or appears to be repetitive. The cache cannot be shared with other loads even if the loads want the same data. The cache is flushed on kernel start and consumes more hardware resources than an equivalent LSU without a cache. The cache is inferred only for non-volatile global pointers.

Write-Acknowledge (write-ack)

Burst-coalesced store LSUs sometimes require a write-acknowledgment signal when data dependencies exist. LSUs with a write-acknowledge signal require additional hardware resources. Throughput might be reduced if multiple write-acknowledge LSUs access the same memory.

Nonaligned

When a burst-coalesced LSU can access memory that is not aligned to the external memory word size, a nonaligned LSU is created. Additional hardware resources are required to implement a nonaligned LSU. The throughput of a nonaligned LSU might be reduced if it receives many unaligned requests.

Never-stall

If a pipelined LSU is connected to a local memory without arbitration, a never-stall LSU is created because all accesses to the memory take a fixed number of cycles that are known to the compiler.

Load-Store Unit Controls

The Intel® oneAPI DPC++/C++ Compiler allows you to control the LSU that is generated for [global memory accesses](#) via a set of templated options in the `ext::intel::lsu` class. The `ext::intel::lsu` class has two member functions, `load()` and `store()`, which allow loading from and storing to a global pointer, respectively. See also [Annotating Unified Shared Memory Pointers](#).

NOTE Not all LSU control options listed below work with both `load()` and `store()` member functions. Not all styles and types are supported on every target device. Thus, the use of LSU control options results in a specific type of LSU.

LSU Control Options Available in the `ext::intel::lsu` Class

Control	Syntax	Value	Default Value	Supported Functionality
Burst-Coalesce	<code>ext::intel::burst_coalesce</code>	B is a boolean	False	Load and store
Static Coalescing	<code>ext::intel::statically_coalesce</code>	B is a boolean	True	Load and store
Prefetch	<code>ext::intel::prefetch</code>	B is a boolean	false	Only loads
Cache	<code>ext::intel::cache<N></code>	N is an integer greater than or equal to 0.	0	Only loads
Pipeline	No option provided	-	Defaults above result in this.	Load and store

Currently, not every combination of LSU controls is supported by the compiler. The following rules apply:

- For store, the `ext::intel::cache` control must be 0 and the `ext::intel::prefetch` control must be false.
- For load, if the `ext::intel::cache` control is set to a value greater than 0, then the `ext::intel::burst_coalesce` control must be set to true.
- For load, exactly one of `ext::intel::prefetch` and `ext::intel::burst_coalesce` control options are allowed to be true.
- For load, exactly one of `ext::intel::prefetch` and `ext::intel::cache` control options are allowed to be true.

Burst-Coalesce

The burst-coalesce control helps the Intel® oneAPI DPC++/C++ Compiler infer a burst-coalesced LSU. It can be combined with other attributes such as `cache`. For more details about this LSU type, refer to [Burst-Coalesced Load-Store Units](#).

The following is an example of burst-coalesce LSU control:

```
using BurstCoalescedLSU =
    ext::intel::lsu<ext::intel::burst_coalesce<true>,
        ext::intel::statically_coalesce<false>>;
BurstCoalescedLSU::store(output_ptr, X);
```

Static Coalescing

The static-coalescing control helps the Intel® oneAPI DPC++/C++ Compiler turn on or off static coalescing for global memory accesses. Static coalescing of memory accesses is the default behavior of the compiler, so this feature can be used to turn off static coalescing. It can be combined with other attributes such as *burst_coalesce*. For more details about this memory optimization modifier, refer to [Static Memory Coalescing](#).

The following is an example that shows how to use the LSU control to turn off static-coalescing LSU control:

```
using NonStaticCoalescedLSU =
    ext::intel::lsu<ext::intel::burst_coalesce<true>,
        ext::intel::statically_coalesce<false>>;

int X = NonStaticCoalescedLSU::load(input_ptr);
```

Prefetch

The prefetch control helps the Intel® oneAPI DPC++/C++ Compiler infer a prefetch style LSU. The compiler honors this control only if it is physically possible. If there is a risk that an LSU is not functionally correct, it is not inferred. This control also works only for load accesses. For more details about this LSU type, refer to [Prefetching Load-Store Units](#).

The following is an example of prefetch LSU control:

```
using PrefetchingLSU =
    ext::intel::lsu<ext::intel::prefetch<true>,
        ext::intel::statically_coalesce<false>>;

int X = PrefetchingLSU::load(input_ptr);
```

Cache

The cache control helps the Intel® oneAPI DPC++/C++ Compiler turn on or off caching behavior and set the size of the cache. This LSU modifier can be useful for *parallel_for* kernels. The cache control can be combined with other attributes such as *burst_coalesce*. For more details about this LSU modifier, refer to [Cached](#).

The following is an example of cache LSU control:

```
using CachingLSU =
    ext::intel::lsu<ext::intel::burst_coalesce<true>,
        ext::intel::cache<1024>,
        ext::intel::statically_coalesce<false>>;

int X = CachingLSU::load(input_ptr);
```

Pipeline

The pipeline control helps the Intel® oneAPI DPC++/C++ Compiler infer the LSU style of pipeline. It can be combined with other attributes such as *static coalesce*. For more details about this LSU type, refer to [Pipelined Load-Store Units](#).

For example:

```
using PipelinedLSU = ext::intel::lsu<>;
PipelinedLSU::store(output_ptr,X);
```

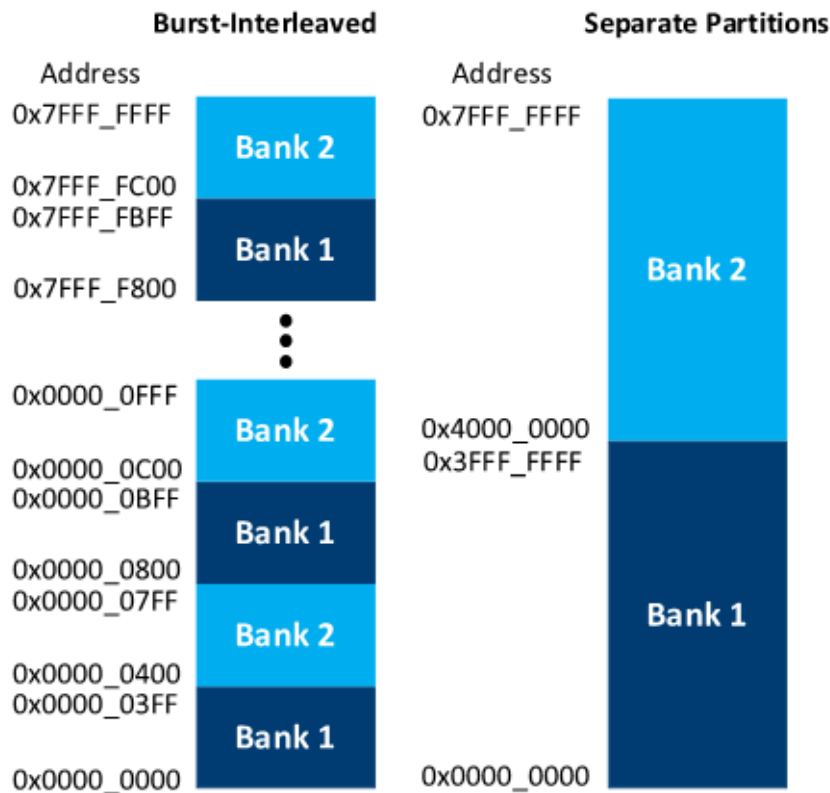
Global Memory Accesses Optimization

The Intel® oneAPI DPC++/C++ Compiler uses SDRAM as global memory. By default, the compiler configures global memory in a burst-interleaved configuration. The Intel® oneAPI DPC++/C++ Compiler interleaves global memory across each of the external memory banks.

In most circumstances, the default burst-interleaved configuration leads to the best load balancing between memory banks. However, in some cases, you might want to partition the banks manually as two non-interleaved (and contiguous) memory regions to achieve better load balancing.

The following figure illustrates the difference in memory mapping patterns between burst-interleaved and non-interleaved memory partitions:

Global Memory Partitions



Global Memory Bandwidth Use Calculation

To ensure the global memory bandwidth listed in the board specification file is utilized completely, calculating the kernel bandwidth use is beneficial. The [FPGA optimization report](#) also displays the kernel bandwidth values in the [global memory view of the System Viewer](#). The following formulas explain how you can calculate this value on a per-LSU basis:

Formulas for Calculating Kernel Bandwidth Use

$$\begin{aligned} \text{LSU bandwidth} &= \min(BW_1, BW_2, BW_3) \text{ MB/s} \\ BW_1 &= KWIDTH * FMAX \\ BW_2 &= MWIDTH * FMAX \\ BW_3 &= \frac{\text{MaxBandwidth} * \text{NUM_INTERLEAVING_CHANNELS}}{\text{NUM_CHANNELS}} \end{aligned}$$

The LSU bandwidth equation is the minimum of three bottlenecks you need to calculate the use of global memory bandwidth. The remaining equations represent three bottlenecks that can limit the LSU bandwidth. These formulas represent the theoretical maximum bandwidth an LSU may consume, ignoring all other LSUs. The actual bandwidth depends on the LSU's access pattern and the interconnect's arbitration between all LSUs. To get an estimate of the overall bandwidth, a sum of the LSU bandwidths is available in the controller of the [global memory view of the System Viewer](#).

The following table describes the variables used in the above equations:

Variables Used in Calculating Kernel Bandwidth

Variable	Description
KWIDTH	Byte-width of the LSU on the kernel. In the <code>report.html</code> file, it is referred to as WIDTH.
MWIDTH	Byte-width of the LSU facing the external memory. In the <code>report.html</code> file, it is referred to as the <code><Memory Name>_Width</code> .
FMAX	Clock speed of the kernel in MHz. In the <code>report.html</code> file, you can identify this as the design's clock speed.
MaxBandwidth	Maximum bandwidth (measured in MB/s) the global memory can achieve. You can find this in the <code>board_spec.xml</code> file for the specific global memory.
NUM_CHANNELS	Number of interfaces an external memory has. You can find this by counting the number of interfaces listed in the <code>board_spec.xml</code> file under that memory.
NUM_INTERLEAVING_CHANNELS	When interleaving is enabled, this is the number of channels. Otherwise, this value is 1.
BW ₁	Bottleneck at the kernel boundary. Therefore, BW ₁ uses only kernel values, which means, values you can change by optimizing the design. If this is limiting the overall bandwidth use than it indicates, changing your design can improve the bottleneck at the kernel boundary.
BW ₂	Bottleneck at the memory interface to the kernel. Therefore, BW ₂ uses the size of the memory interface and the f_{MAX} , which means either improving f_{MAX} of your design or switching to a board with a wider memory interface can improve the bandwidth use.
BW ₃	Bottleneck in the external memory. Therefore, BW ₃ uses external memory properties exclusively, and if this is limiting your design, you have utilized the board bandwidth completely.

Manual Partition of Global Memory

You can partition the memory manually so that each buffer occupies a different memory bank.

The default burst-interleaved configuration of the global memory prevents load imbalance by ensuring that memory accesses do not favor one external memory bank over another. However, you have the option to control the memory bandwidth across a group of buffers by partitioning your data manually.

The Intel® oneAPI DPC++/C++ Compiler cannot burst-interleave across different memory types. To manually partition a specific type of global memory, compile your kernels with the `-Xsno-interleaving=<global_memory_name>` flag to configure each bank of a certain memory type as non-interleaved banks.

If your kernel accesses two buffers of equal size in memory, you can distribute your data to both memory banks simultaneously regardless of dynamic scheduling between the loads. This optimization step might increase your apparent memory bandwidth.

If your kernel accesses heterogeneous global memory types, include the `-Xsno-interleaving=<global_memory_name>` option in the `icpxcommand` for each memory type that you want to partition manually.

For more information, refer to [Disable Burst-Interleaving of Global Memory \(-Xsno-interleaving\)](#).

Partitioning Buffers Across Different Memory Types (Heterogeneous Memory)

The board support package for your FPGA board can assemble a global memory space consisting of different memory technologies (for example, DRAM or SRAM). The board support package designates one such memory (which might consist of multiple interfaces) as the default memory. All buffers reside in this heterogeneous memory.

To use the heterogeneous memory, perform the following steps to modify the code in your source file:

1. Determine the names of global memory types available on your FPGA board using one of the following methods:
 - Refer to the board vendor's documentation for information.
 - Identify the index of the global memory type in the `board_spec.xml` file of your Custom Platform. The index starts at 0, which indicates the global memory that is marked as "default". If no interface is marked as "default", then 0 indicates the first global memory listed in the `board_spec.xml` file.

The index then follows the order in which the global memory appears in the `board_spec.xml` file. For example, if there is no memory marked as "default", then the first global memory type in the `board_spec.xml` file has an index 0, the second has an index 1, and so on. Another example: if the second global memory is marked as "default" then 0 index refers to the second memory, index 1 refers to the first memory and so on.

For more information, refer to [global_mem in oneAPI Accelerator Support Package \(ASP\) Reference Manual: Open FPGA Stack](#).

2. To instruct the host to allocate buffer to a specific global memory type, insert the `buffer_location<index>` property in the accessor property list.

For example:

```
ext::oneapi::accessor_property_list PL{ext::intel::buffer_location<2>};
accessor acc(buffer, cgh, read_only, PL);
```

Notice

Using the `buffer_location<index>` property on BSPs with heterogeneous memory support in combination with [read-only cache](#) is an untested functionality and Intel does not guarantee its support.

If you do not specify the `buffer_location` property, the host allocates the buffer to the default memory type automatically. To determine the default memory type, consult the documentation provided by your board vendor. Alternatively, in the `board_spec.xml` file of your Custom Platform, search for the memory type that is defined first or has the attribute `default=1` assigned to it. For more information, refer to [oneAPI Accelerator Support Package\(ASP\) Reference Manual: Open FPGA Stack](#).

NOTE

Streams support is limited if the target FPGA is used in an FPGA board with heterogeneous memory. With SYCL streams, all work items write to the same stream buffer in parallel, so atomics are leveraged to ensure race conditions. Atomics support for boards that have heterogeneous memory is limited. For more information about atomics and streams, refer to the SYCL* 2020 specification available at <https://registry.khronos.org/SYCL/specs/sycl-2020/html/sycl-2020.html>. For debugging kernels, use `sycl::ext::oneapi::experimental::printf` instead of SYCL streams, which can lead to performance issues or hangs.

Partitioning Buffers Across Memory Channels of the Same Memory Type

By default, the Intel® oneAPI DPC++/C++ Compiler configures each global memory type in a burst-interleaved manner. Usually, the burst-interleaving configuration leads to the best load balancing between the memory channels. However, there might be situations where it is more efficient to partition the memory into non-interleaved regions. For additional information, refer to [Global Memory Accesses Optimization](#).

The [Global Memory Partitions](#) diagram in [Global Memory Accesses Optimization](#) illustrates the differences between burst-interleaved and non-interleaved memory partitions.

To manually partition some or all of the available global memory types, perform the following tasks:

1. Create a buffer with `property::buffer::mem_channel` specifying channel ID in its `property_list`.
 - Specify `property::buffer::mem_channel` with value 1 to allocate the buffer in the lowest available memory channel (default).
 - Specify `property::buffer::mem_channel` with value 2 or greater to allocate the buffer in the higher available memory channel.

Here is an example buffer definition:

```
range<1> num_of_items{N};
buffer<T, 1> bufferA(VA.data(), num_of_items,
  {property::buffer::mem_channel{1}});
buffer<T, 1> bufferB(VB.data(), num_of_items,
  {property::buffer::mem_channel{2}});
buffer<T, 1> bufferC(VC.data(), num_of_items,
  {property::buffer::mem_channel{3}});
```

2. Compile your design kernel using the `-Xsno-interleaving=<global_memory_name>>` flag to configure the memory channels of the specified memory type as separate addresses. For more information about the use of the `-Xsno-interleaving=<global_memory_name>` flag, refer to the [Disable Burst-Interleaving of Global Memory \(-Xsno-interleaving\)](#) section.

Important If you do not specify the `-Xsno-interleaving` option in your compiler command and have specified the `mem_channel` property on buffers in your kernel, the compiler assumes that a `-Xsno-interleaving=default` option was specified in the compiler command. This behavior might change in a future release, so ensure that you always explicitly set the `-Xsno-interleaving=<global_memory_name>>` option in the compiler command when using the `mem_channel` property.

Caution

- Do not set more than one memory channel property on a buffer.
- If the memory channel specified is not available on the target board, the buffer is placed in the first memory channel.

Ignoring Dependencies Between Accessor Arguments

You can direct the Intel® oneAPI DPC++/C++ Compiler to ignore dependencies between accessor arguments in a SYCL* kernel in one of the following methods. Both methods allow the compiler to analyze dependencies between kernel memory operations more accurately, which can result in higher performance.

Method 1: Add the `[[intel::kernel_args_restrict]]` Attribute to Your Kernel

To direct the Intel® oneAPI DPC++/C++ Compiler to ignore dependencies between accessor arguments and USM pointers used in a kernel, add the `[[intel::kernel_args_restrict]]` attribute to your kernel. The use of the `[[intel::kernel_args_restrict]]` attribute is an assurance to the compiler that the accessor arguments and USM pointers used by the kernel do not alias with each other. This is an unchecked assertion by the programmer and results in an undefined behavior if violated.

Example

```
device_queue.submit([&](handler& cgh) {
    // create accessors from global memory
    accessor in_accessor(in_buf, cgh, read_only);
    accessor out_accessor(out_buf, cgh, write_only);

    // run the task (note the use of the attribute here)
    cgh.single_task<KernelArgsRestrict>([=]() [[intel::kernel_args_restrict]] {
        for (int i = 0; i < N; i++) {
            out_accessor[i] = in_accessor[i];
        }
    });
});
```

Tip

For additional information, refer to the FPGA tutorial sample "Kernel Args Restrict" on [GitHub](#).

Method 2: Add the `no_alias` Property to an Accessor's Property List

The `no_alias` property notifies the Intel® oneAPI DPC++/C++ Compiler that all modifications to the memory locations accessed (directly or indirectly) by an accessor during kernel execution is done through the same accessor (directly or indirectly) and not by any other accessor or USM pointer in the kernel. This is an unchecked assertion by the programmer and results in an undefined behavior if it is violated. Effectively, applying `no_alias` to all accessors of a kernel is equivalent to applying the `[[intel::kernel_args_restrict]]` attribute to the kernel unless the kernel uses USM. You cannot apply the `no_alias` property on a USM pointer.

Example

```
ext::oneapi::accessor_property_list PL{ext::oneapi::no_alias};
accessor acc(buffer, cgh, PL);
```

Contiguous Memory Accesses

The Intel® oneAPI DPC++/C++ Compiler attempts to dynamically coalesce accesses to adjacent memory locations to improve global memory efficiency. This is effective if consecutive work items access consecutive memory locations in a given load or store operation. The same is true in a `single_task` invocation if consecutive loop iterations access consecutive memory locations.

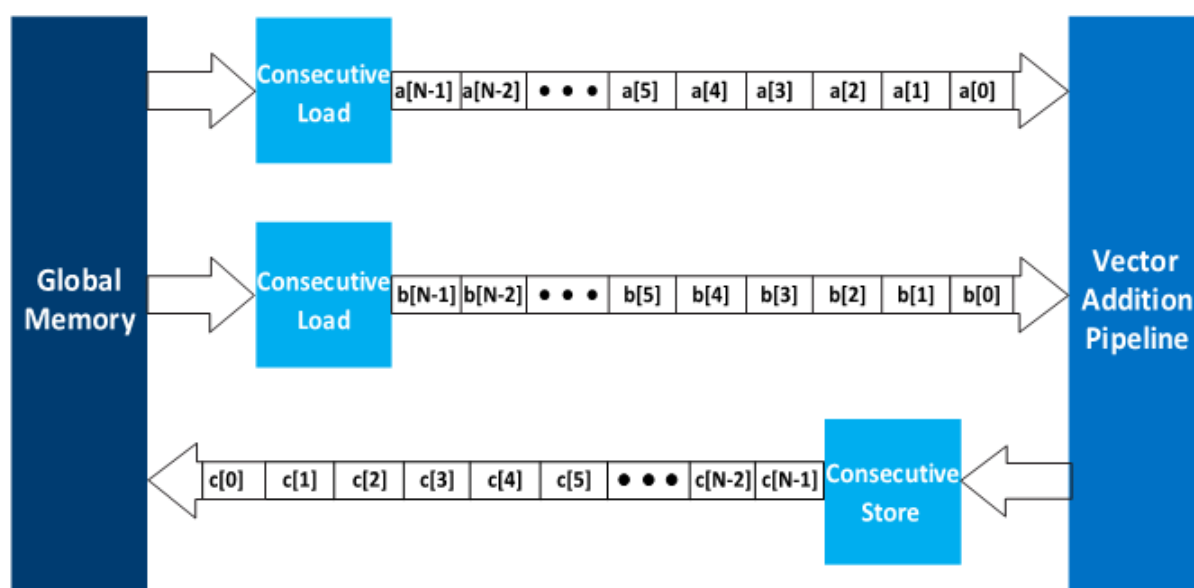
Consider the following code example:

```
q.submit([&](handler &cgh) {
    accessor a(a_buf, cgh, read_only);
    accessor b(b_buf, cgh, read_only);
    accessor c(c_buf, cgh, write_only, no_init);
    cgh.parallel_for<class SimpleVadd>(N, [=](id<1> ID) {
        c[ID] = a[ID] + b[ID];
    });
});
```

The load operation from the accessor `a` uses an index that is a direct function of the work-item global ID. By basing the accessor index on the work-item global ID, the Intel® oneAPI DPC++/C++ Compiler can ensure contiguous load operations. These load operations retrieve the data sequentially from the input array and send the read data to the pipeline as required. Contiguous store operations then store elements of the result that exits the computation pipeline in sequential locations within global memory.

The following figure illustrates an example of the contiguous memory access optimization:

Contiguous Memory Access



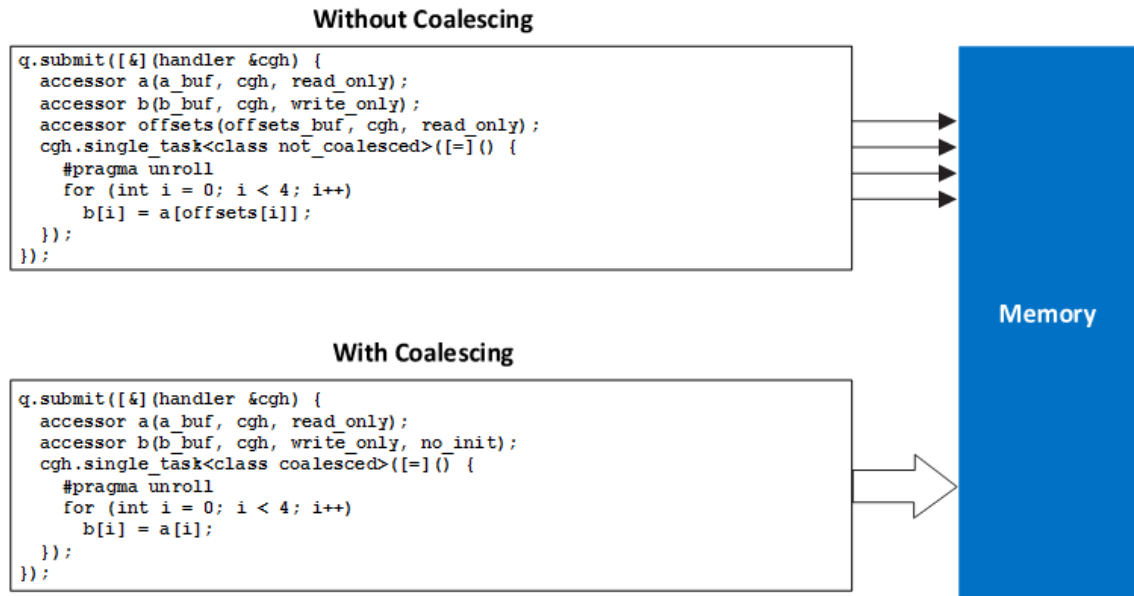
Contiguous load and store operations improve memory access efficiency because they lead to increased access speeds and reduced hardware resource needs. The data travels in and out of the computational portion of the pipeline concurrently, allowing overlaps between computation and memory accesses. Where possible, use work-item IDs that index accesses to arrays in global memory to maximize memory bandwidth efficiency.

Static Memory Coalescing

Static memory coalescing is an Intel® oneAPI DPC++/C++ Compiler optimization step that merges contiguous accesses to global memory into a single wide access. A similar optimization is applied to on-chip memory.

The figure below shows a common case where kernel performance might benefit from static memory coalescing:

Static Memory Coalescing



Consider the following vectorized kernel:

```
q.submit([&](handler &cgh) {
    accessor a(a_buf, cgh, read_only);
    accessor b(b_buf, cgh, write_only, no_init);
    cgh.single_task<class coalesced>([=]() {
        #pragma unroll
        for (int i = 0; i < 4; i++)
            b[i] = a[i];
    });
});
```

The kernel performs four load operations from buffer `a` that access consecutive locations in memory. Instead of performing four memory accesses to competing locations, the compiler coalesces the four loads into a single, wider vector load. This optimization reduces the number of accesses to a memory system and potentially leads to better memory access patterns.

Although the compiler performs static memory coalescing automatically, you should use wide vector loads and stores in your SYCL* code whenever possible to ensure efficient memory accesses.

To allow static memory coalescing, you must write your code in such a way that the compiler can identify a sequential access pattern during compilation. The original kernel code shown in the figure above can benefit from static memory coalescing because all indexes into buffers `a` and `b` increment with offsets that are known at compilation time. In contrast, the following code does not allow static memory coalescing to occur:

```
q.submit([&](handler &cgh) {
    accessor a(a_buf, cgh, read_only);
    accessor b(b_buf, cgh, write_only);
    accessor offsets(offsets_buf, cgh, read_only);
    cgh.single_task<class not_coalesced>([=]() {
        #pragma unroll
        for (int i = 0; i < 4; i++)
            b[i] = a[offsets[i]];
    });
});
```


The value `offsets[i]` is unknown at compilation time. As a result, the Intel® oneAPI DPC++/C++ Compiler cannot statically coalesce the read accesses to buffer `a`.

For more information, refer to [Local and Private Memory Accesses Optimization](#).

Perform Kernel Computations Using Local or Private Memory

To optimize memory access efficiency, minimize the number of global memory accesses by performing kernel computations in local or private memory.

To minimize global memory accesses, it is often best to preload data from a group of computations from global memory to a local or private memory. Perform kernel computations on the preloaded data and write the results back to the global memory.

Preload Data into Local Memory or Private Memory

When you structure your kernel code, if your global memory accesses are not sequential, consider refactoring your code to access global memory sequentially while buffering that data in local or private memory before using the data for computation. This can be beneficial for performance since the Intel® oneAPI DPC++/C++ Compiler implements local and private memory on-chip whereas global memory is off-chip for most platforms. On-chip memory is smaller than off-chip memory, but it significantly has higher bandwidth and much lower latency. Additionally, on-chip memory is more effective with random access memory patterns than off-chip. For more information, refer to [Memory Accesses](#) and [Memory Attributes](#).

Store Variables and Arrays in Private Memory

The Intel® oneAPI DPC++/C++ Compiler implements private memory using FPGA registers in the kernel datapath, block RAMs, or MLABs.

Aggregate data types are also implemented in the registers if array-access indices are compile-time constants. Typically, private memory is useful for storing single variables or small arrays. Otherwise, the compiler uses block RAMs or MLABs.

Registers are ample hardware resources in FPGAs, and you should use them with private memory instead of other memory types whenever possible. If a variable is implemented in registers, it can be accessed in parallel across the datapath, as each stage of the pipeline will have its own copy of the data.

For more information on the implementation of private memory using registers, refer to [Inferring a Shift Register](#).

Local and Private Memory Accesses Optimization

To optimize local memory access efficiency, consider the following guidelines:

- Implementing certain optimization techniques, such as loop unrolling, might lead to more concurrent memory accesses.
- Increasing the number of memory accesses can complicate memory systems and degrade performance.
- If you have function scope local data (defined in the body of a `parallel_for_work_group lambda` or by using `group_local_memory_for_overwrite`), the Intel® oneAPI DPC++/C++ Compiler statically sizes the local data that you define within a function body at compilation time.
- For accessors in the local space, the host assigns their memory sizes dynamically at runtime. However, the compiler must set these physical memory sizes at compilation time. By default, that size is 16 kB.
 - The host can request a smaller data size than has been physically allocated at compilation time, but never a larger size.
- When accessing local memory, use the simplest address calculations possible and avoid pointer math operations that are not mandatory.
 - Intel® recommends this coding style to reduce FPGA resource utilization and increase local memory efficiency by allowing the Intel® oneAPI DPC++/C++ Compiler to make better inferences about access patterns through static code analysis. For information about the benefits of static coalescing, refer to

Static Memory Coalescing. Complex address calculations and pointer math operations can prevent the Intel® oneAPI DPC++/C++ Compiler from identifying the memory system accessed by a given load or store, leading to increased area use and decreased runtime performance.

- Avoid storing pointers to memory whenever possible. Stored pointers often prevent static compiler analysis from determining the data sets accessed, when pointers are subsequently retrieved from memory. Storing pointers to memory usually leads to suboptimal area and performance results.
- Create local array elements that are a power of two bytes to allow the Intel® oneAPI DPC++/C++ Compiler to provide an efficient memory configuration.
- Whenever possible, the Intel® oneAPI DPC++/C++ Compiler automatically pads the elements of the local memory to be a power of two to provide a more efficient memory configuration. For example, if you have a `struct` containing three chars, the Intel® oneAPI DPC++/C++ Compiler pads it to four bytes, instead of creating a narrower and deeper memory with multiple accesses (that is, a 1-byte wide memory configuration). However, there are cases where the Intel® oneAPI DPC++/C++ Compiler might not pad the memory, such as when the kernel accesses local memory indirectly through pointer arithmetic.
- To determine if the Intel® oneAPI DPC++/C++ Compiler has padded the local memory, review the memory dimensions in the Memory Viewer. If the Intel® oneAPI DPC++/C++ Compiler fails to pad the local memory, it prints a warning message in the Area Analysis of System report.

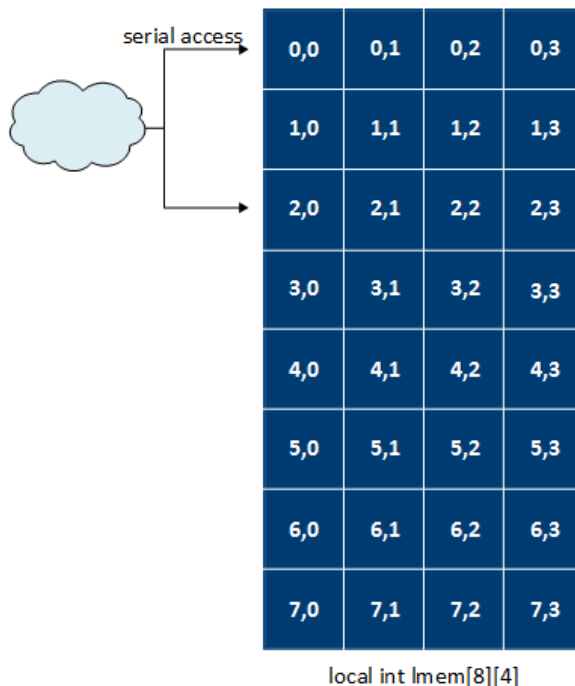
Local Memory Banks

Specifying the `numbanks(N)` and `bankwidth(M)` memory attributes allow you to configure the local memory banks for parallel memory accesses. The banking geometry described by these attributes determines which elements of the local memory system your kernel can access in parallel.

The following code example depicts an 8 x 4 local memory system that is implemented in a single bank. As a result, no two elements in the system can be accessed in parallel.

```
[[intel::fpga_memory]] int lmem[8][4];
#pragma unroll
for(int i = 0; i<4; i+=2) {
    lmem[i][x] = ...;
}
```

Serial Accesses to an 8 x 4 Local Memory System



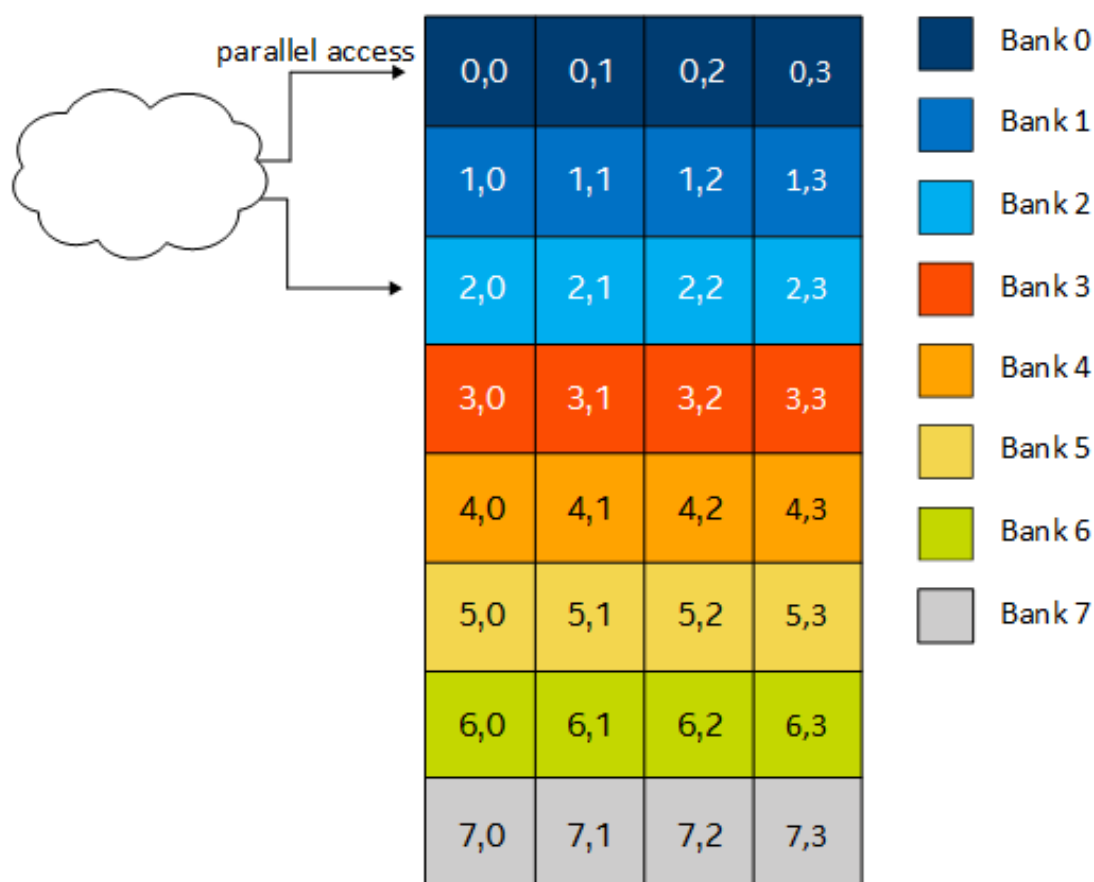
To improve performance, you can add `numbanks(N)` and `bankwidth(M)` in your code to define the number of memory banks and the bank widths in bytes. The following code implements eight memory banks, each 16-bytes wide. This memory bank configuration enables parallel memory accesses to the 8 x 4 array.

```
[[intel::numbanks(8), intel::bankwidth(16)]] int lmem[8][4];
#pragma unroll
for (int i = 0; i < 4; i+=2) {
    lmem[i][x & 0x3] = ...;
}
```

NOTE

To enable parallel access, you must mask the dynamic access on the lower array index. Masking the dynamic access on the lower array index informs the Intel® oneAPI DPC++/C++ Compiler that `x` does not exceed the lower index bounds.

Parallel Access to an 8 x 4 Local Memory System with Eight 16-Byte-Wide Memory Banks



```
[[intel::fpga_memory]] int lmem[8][4]
```

By specifying different values for the `numbanks(N)` and `bankwidth(M)` attributes, you can change the parallel access pattern. The following code implements four memory banks, each being four bytes wide:

NOTE

This memory bank configuration enables parallel memory accesses to the 8 x 4 array.

```
[[intel::numbanks(4), intel::bankwidth(4)]] int lmem[8][4];  
#pragma unroll  
for (int i = 0; i < 4; i+=2) {  
    lmem[x][i] = ...;  
}
```

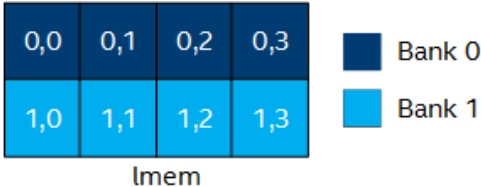
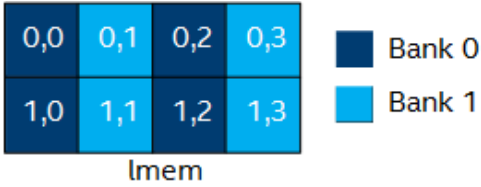
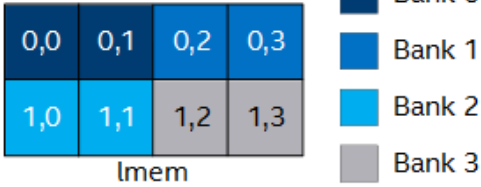
Parallel Access to an 8 x 4 Local Memory System with Four 4-Byte-Wide Memory Banks

Optimize the Geometric Configuration of Local Memory Banks Based on Array Index

By default, the Intel® oneAPI DPC++/C++ Compiler attempts to improve performance by automatically banking a local memory system. The compiler includes advanced features that allow you to customize the banking geometry of your local memory system. To configure the geometry of local memory banks, include `numbanks(N)` and `bankwidth(M)` kernel attributes in your SYCL* kernel.

The table provides code examples illustrating how the bank geometry changes based on the values you assign to `numbanks` and `bankwidth`. The first and last rows of this table illustrate how to bank memory on the upper and lower indexes of a 2D array, respectively.

Effects of `numbanks` and `bankwidth` on the Bank Geometry of 2 x 4 Local Memory System

Code Example	Bank Geometry
<pre>[[intel::numbanks(2), intel::bankwidth(16)]] int lmem[2][4];</pre>	
<pre>[[intel::numbanks(2), intel::bankwidth(8)]] int lmem[2][4];</pre>	
<pre>[[intel::numbanks(2), intel::bankwidth(4)]] int lmem[2][4];</pre>	
<pre>[[intel::numbanks(4), intel::bankwidth(8)]] int lmem[2][4];</pre>	
<pre>[[intel::numbanks(4), intel::bankwidth(4)]] int lmem[2][4];</pre>	

Annotating Unified Shared Memory Pointers

When using [Unified Shared Memory \(USM\)](#) to perform a host allocation or a device allocation, Intel® recommends annotating raw USM pointers inside the kernel before accessing the pointers using the `host_ptr` or `device_ptr` object. The `host_ptr` and `device_ptr` objects are instances of the `multi_ptr` class in SYCL that provides constructors for address space qualified pointers.

Using `host_ptr` and `device_ptr` objects allow the compiler to perform better alias analysis, which typically leads to better throughput and smaller silicon area for your design. Also, host or device annotated pointers allow the compiler to infer simpler RTL, because [Load-Store Units \(LSUs\)](#) that want to access the USM pointers must be connected only to the host memory or only to the device memory, respectively. Without the annotations, the compiler is compelled to connect LSUs to both memories because the location of the pointer is unknown at compile time.

For example, when using the `malloc_device` function to define a pointer `Ptr`, construct a `device_ptr` object using the pointer `Ptr` inside the kernel, and access the `device_ptr` object directly instead of accessing the pointer `Ptr`:

```
T* Ptr = malloc_device<T>(1024, Queue);
...
cgh.single_task<class DeviceAnnotation>([=]() {
    Ptr[0] = 42; // load-store unit connected to both device and host memories
    device_ptr<T> DevicePtr(Ptr);
    DevicePtr[1] = 43; // load-store unit connected only to the device memory
});
```

Similarly, when using the `malloc_host` function to define a pointer `Ptr`, construct a `host_ptr` object using the pointer `Ptr` inside the kernel and access the `host_ptr` object directly instead of accessing `Ptr`:

```
T* Ptr = malloc_host<T>(1024, Queue);
...
cgh.single_task<class HostAnnotation>([=]() {
    Ptr[0] = 42; // load-store unit connected to both device and host memories
    host_ptr<T> HostPtr(Ptr);
    HostPtr[1] = 43; // load-store unit connected only to the host memory
});
```

Caution

Use annotations consistently and match them with the type of the runtime allocation used. Mismatches and inconsistencies when using the address space annotations are considered undefined behavior and may lead to incorrect results or hardware hangs.

Unified Shared Memory (USM)

Zero-Copy Memory Access

Prior to the implementation of restricted USM, you had to access host data from the device using one of the following methods:

- Through SYCL buffers
- By copying data between the host and the device using explicit USM

With host allocations on devices supporting USM host allocation, a kernel can directly access data over PCIe (no copying required). By using host allocations in designs that have infrequent random accesses to large pieces of data, you can improve throughput and latency of the design as a large piece of data no longer requires copying in full to the device. For detailed explanation of this concept, refer to the [Zero-copy Data Transfer](#) tutorial on GitHub.

SYCL* buffers created with host allocations set as `hostData` in the constructor still result in data copy from the host to the device memory.

Annotating Unified Shared Memory Pointers

Additional Recommendations

Optimizing memory accesses in your kernels can improve overall kernel performance. Consider implementing the following techniques for optimizing memory accesses:

- Avoid designing systems where one kernel writes an intermediate result to global memory and another kernel reads this data back from global memory. Instead, implement a SYCL* pipe (described in [Pipes](#)) between the producer and consumer kernels for direct data transfer. Alternatively, you can merge both kernels into a single larger kernel and use helper functions to logically separate the two original kernels.
- The Intel® oneAPI DPC++/C++ Compiler implements local memory in FPGAs differently than in GPUs. If your kernel contains code to avoid GPU-specific local memory bank conflicts, remove that code because the compiler generates hardware that avoids local memory bank conflicts automatically whenever possible.

Kernel Variable Accesses

This section shows techniques you can use to optimize local and private variables in kernels.

Inferring a Shift Register

The shift register design pattern is a very important design pattern for efficient implementation of many applications on the FPGA. However, the implementation of a shift register design pattern might seem counter-intuitive at first.

Consider the following code example:

```
using InPipe = ext::intel::pipe<class PipeIn, int, 4>;
using OutPipe = ext::intel::pipe<class PipeOut, int, 4>;

#define SIZE 512
//Shift register size must be statically determinable
// this function is used in kernel
void foo()
{
    int shift_reg[SIZE];
    //The key is that the array size is a compile time constant
    // Initialization loop
    #pragma unroll
    for (int i = 0; i < SIZE; i++)
    {
        //All elements of the array should be initialized to the same value
        shift_reg[i] = 0;
    }
    while(1)
    {
        // Fully unrolling the shifting loop produces constant accesses
        #pragma unroll
        for (int j = 0; j < SIZE-1; j++)
        {
            shift_reg[j] = shift_reg[j + 1];
        }

        shift_reg[SIZE - 1] = InPipe::read();
        // Using fixed access points of the shift register
        int res = (shift_reg[0] + shift_reg[1]) / 2;

        // 'out' pipe will have running average of the input pipe
    }
}
```

```
    OutPipe::write(res);  
  }  
}
```

In each clock cycle, the kernel shifts a new value into the array. By placing this shift register into a block RAM, the Intel® oneAPI DPC++/C++ Compiler can efficiently handle multiple access points into the array. The shift register design pattern is ideal for implementing filters (for example, image filters like a Sobel filter or time-delay filters like a finite impulse response (FIR) filter).

When implementing a shift register in your kernel code, remember the following key points:

- Unroll the shifting loop so that it can access every element of the array.
- All access points must have constant data accesses. For example, if you write a calculation in nested loops using multiple access points, unroll these loops to establish the constant access points.
- Initialize all elements of the array to the same value. Alternatively, you may leave the elements uninitialized if you do not require a specific initial value.

Memory Access Considerations

Intel® recommends the following kernel programming strategies that can improve memory access efficiency and reduce area use of your kernel:

- Minimize the number of access points to external memory to reduce area. The compiler infers an LSU for each access point in your kernel, which consumes area.

If possible, structure your kernel such that it reads its input from one location, processes the data internally, and then writes the output to another location.

- Instead of relying on local or global memory accesses, structure your kernel as a single work-item with shift register inference whenever possible.

Libraries

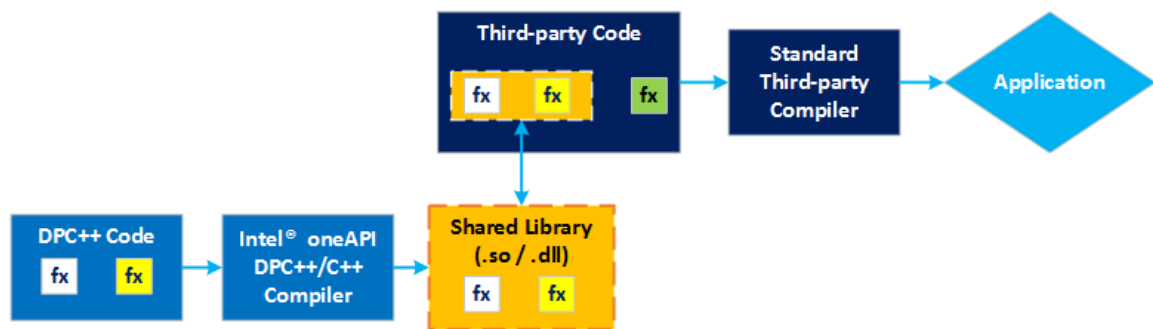
18

With libraries, you can reuse functions without knowing the underlying hardware design or implementation details. Libraries can be created from SYCL* code or from RTL code.

Use SYCL Shared Library With Third-Party Applications

Use the Intel® oneAPI DPC++/C++ Compiler to compile your SYCL code to a C-standard shared library (.so file on Linux and .dll file on Windows). You can then call this library from other third-party code to access a broad base of accelerated functions from your preferred programming language.

SYCL Functions Packaged into a Shared Library File For Use in Third-party Applications



To use a shared library with a third-party application, perform these steps:

1. [Define the Shared Library Interface.](#)
2. Generate the Library File in [Linux](#) or [Windows](#).
3. [Use the Shared Library.](#)

Define the Shared Library Interface

Intel® recommends defining an interface between the C-standard shared library and your SYCL code. The interface must include functions you want to export and how those functions interface with your SYCL code. Prefix the functions that you want to include in the shared library with `extern "C"`.

NOTE If you do not prefix with `extern "C"`, then the functions appear with mangled names in the shared library.

Consider the following example code of the `vector_add` function:

```
extern "C" int vector_add(int *a, int *b, int **c, size_t vector_len) {
    // Create device selector for the device of your interest.
    #if FPGA_EMULATOR
        // Intel extension: FPGA emulator selector on systems without FPGA card.
        auto selector = sycl::ext::intel::fpga_emulator_selector_v;
    #elif FPGA_SIMULATOR
        // Intel extension: FPGA simulator selector on systems without FPGA card.
        auto selector = sycl::ext::intel::fpga_simulator_selector_v;
    #elif FPGA_HARDWARE
        // Intel extension: FPGA selector on systems with FPGA card.
    #endif
}
```

```

    auto selector = sycl::ext::intel::fpga_selector_v;
#else
    // The default device selector will select the most performant device.
    auto selector = default_selector_v;
#endif

try {
    queue q(d_selector, exception_handler);
    // SYCL code interface:
    // Vector addition in SYCL
    VectorAddKernel(q, a, b, c, vector_len);
} catch (exception const &e) {
    std::cout << "An exception is caught for vector add.\n";
    return -1;
}
return 0;
}

```

Generate the Shared Library File in Linux

If you are using a Linux system, then perform these steps to generate the shared library file:

1. Compile the device code separately.

```
icpx -fPIC -fintel-fpga -fsycl-link=image [kernel src files] -o <hw image name> -Xshardware
```

Where:

- **fPIC**: Determines whether the compiler generates position-independent code for the host portion of the device image. Option **-fPIC** specifies full symbol preemption. Global symbol definitions and global symbol references get default (preemptable) visibility unless explicitly specified otherwise. You must use this option when building shared objects. You can also specify this option as **-fpic**.

NOTE **PIC** is required so that pointers in the shared library reference global addresses and not local addresses.

- **fintel-fpga**: Targets FPGA devices.
- **fsycl-link=image**: Informs the Intel® oneAPI DPC++/C++ Compiler to partially link device binaries for use with FPGA.
- **Xshardware**: Compiles for hardware instead of the emulator.

2. Compile the host code separately.

```
icpx -fPIC -fintel-fpga <host src files> -o <host image name> -c -DFPGA=1
```

Where:

- **DFPGA=1**: Sets a compiler macro, **FPGA**, equal to 1. It is used in the device selector to change between target devices (requires corresponding host code to support this). This is optional as you can also set your device selector to **FPGA**. For more information, refer to [Device Selectors for FPGA](#).

3. Link the host and device images and create the binary.

```
icpx -fPIC -fintel-fpga -shared <host image name> <hw image name> -o lib<library name>.so
```

Where:

- **shared**: Outputs a shared library (**.so** file).

- Output file name: Prefix with `lib` for the GCC type of compilers. For additional information, see [Shared libraries with GCC on Linux](#). For example:

```
gcc -Wall -fPIC -L. -o out.a -l<library name>.so
```

NOTE Instead of the above multi-step process, you can also perform a single-step compilation to generate the shared library. However, you must perform a full compile if you want to build the executable for testing purposes (for example, `a.out`) or if you make changes in the SYCL code or C interface.

Generate the Shared Library File in Windows

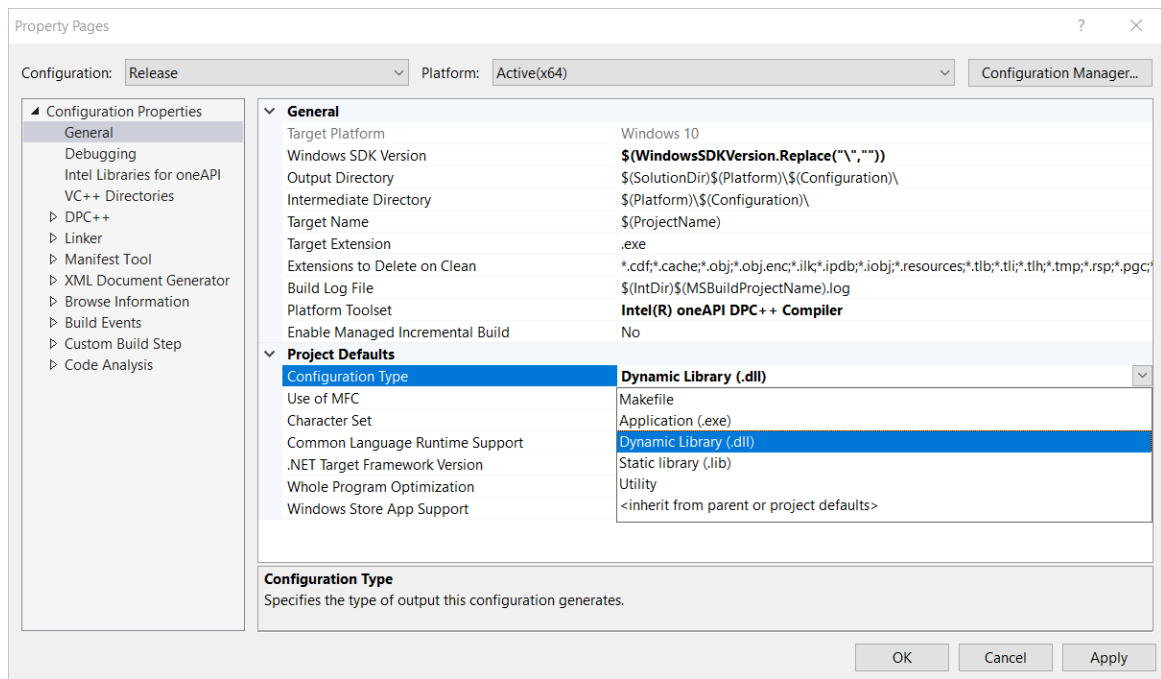
If you are using a Windows system, then perform these steps to generate the library file:

NOTE

- Intel® recommends creating a new configuration in the same project properties. If you want to build the application, you can avoid changing the configuration type for your project.
- Creating a Windows library with the default Intel® oneAPI Base Toolkit is supported only for FPGA emulation. For custom platforms, contact your board vendor for Windows support for FPGA hardware compiles.

1. In Microsoft Visual Studio*, navigate to **Project > Properties**. The **Property Pages** dialog is displayed for your project.
2. Under the **Configuration Properties > General > Project Defaults > Configuration Type** option, select **Dynamic Library (`.dll`)** from the drop-down list.

Project Properties Dialog



3. Click **OK** to close the dialog.

The project automatically builds to create a dynamic library (`.dll`)

Use the Shared Library

These steps may vary depending on the language or compiler you decide to use. Consult the specifications for your desired language for more details. See [Shared libraries with GCC on Linux](#) for an example.

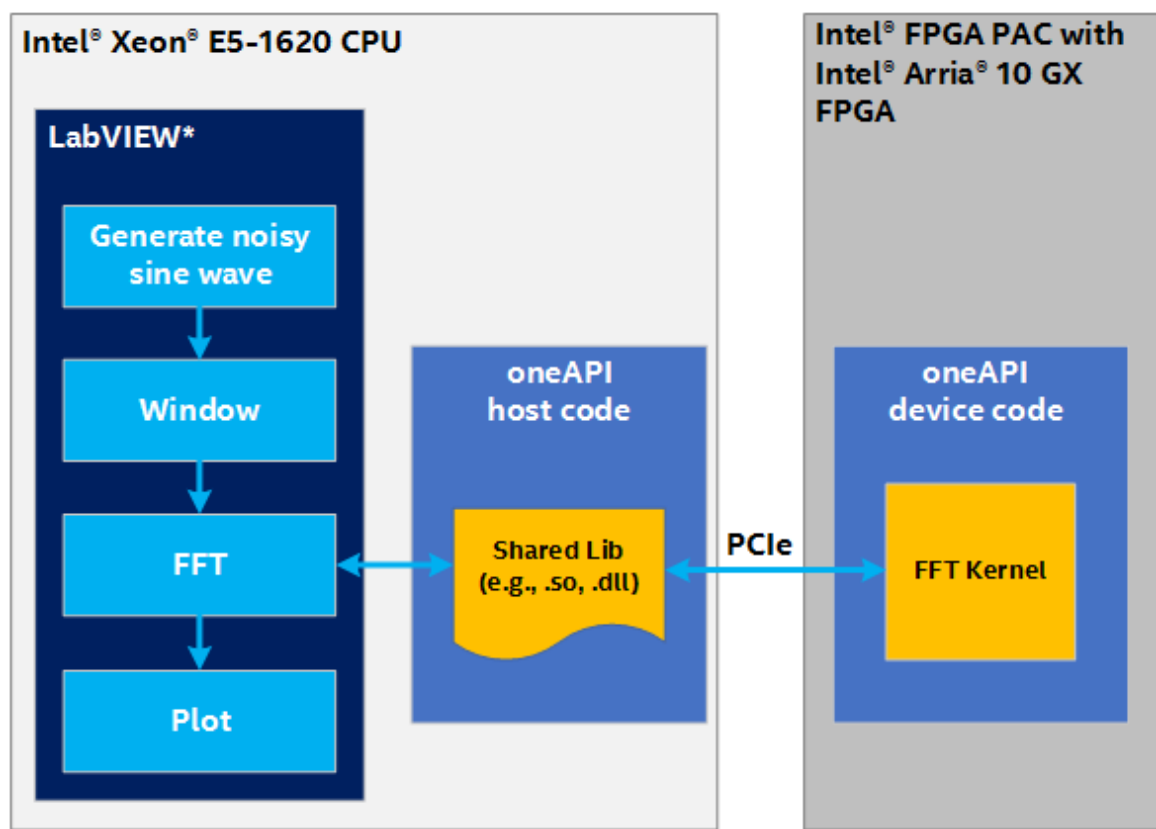
Generally, follow these steps to use the shared library:

1. >Use the shared library function call in your third-party host code.
2. Link your host code with the shared library during the compilation.
3. Ensure that the library file is discoverable. For example:

```
export LD_LIBRARY_PATH=<lib file location>:$LD_LIBRARY_PATH
```

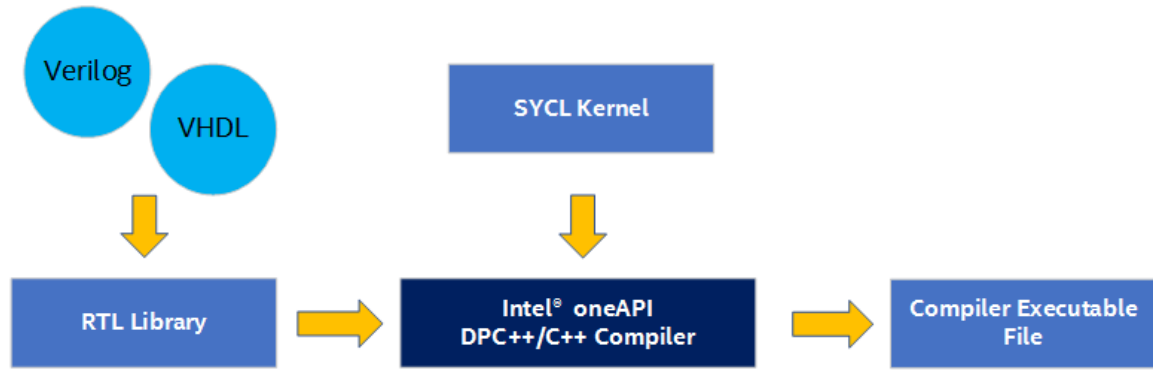
The following is an example illustration of using the shared library:

Example Use of the Shared Library



Use of RTL Libraries for FPGA

An RTL library is a file that contains one or more functions. You can create an RTL library file using register transfer level (RTL) code. You can then include this library file and use the functions inside your SYCL* kernels.



To generate libraries that you can use with SYCL, you need to create the following files:

Generating RTL Libraries for Use with SYCL for FPGA

File or Component	Description
RTL Library Files	
RTL source files	Verilog, System Verilog, or VHDL files that define the RTL component. Additional files such as Quartus® Prime IP File (.qip), Synopsys Design Constraints File (.sdc), and Tcl Script File (.tcl) are not allowed. For information about the file syntax, see Object Manifest File Syntax of an RTL Library .
XML File (.xml)	Describes the properties of the RTL component. The Intel® oneAPI DPC++/C++ Compiler uses these properties to integrate the RTL component into the SYCL pipeline. For information about the XML attributes, see XML Elements for ATTRIBUTES .
Header file (.hpp)	A header file containing valid SYCL kernel language and declares the signatures of functions implemented by the RTL component.
Emulation model file (SYCL-based)	Provides a C++ model for the RTL component that is used only for emulation. Full hardware compilations use the RTL source files.
SYCL Functions	
SYCL source files (.cpp)	Contains definitions of the SYCL functions. These functions are used during emulation and full hardware compilations.
Header file (.hpp)	A header file describing the functions to be called from SYCL in the SYCL syntax.

The format of the library files is determined by which operating system you compile your source code on, with additional sections that carry additional library information.

- On Linux* platforms, a library is a .a archive file that contains .o object files.
- On Windows* platforms, a library is a .lib archive file that contains .obj object files.

You can call the functions in the library from your kernel without the need to know the hardware design or the implementation details of the underlying functions in the library. Add the library to the `icpx` command line when you compile your kernel.

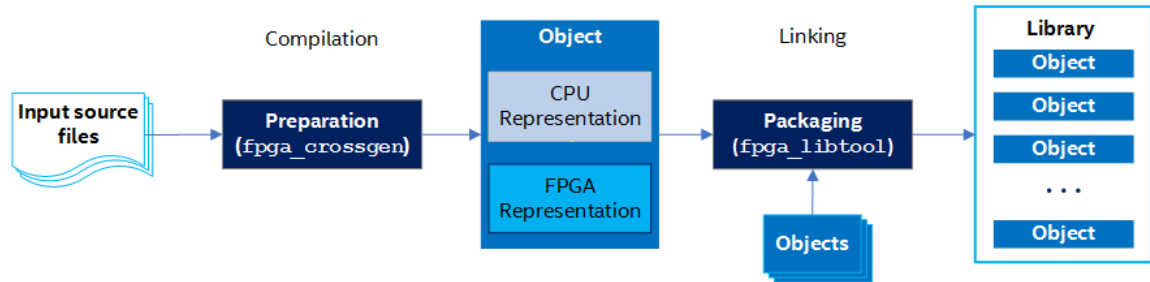
Creating a library is a two-step process:

1. Each object file is created from an input source file using the `fpga_crossgen` command.
 - An object file is effectively an intermediate representation of your source code with both a CPU representation and an FPGA representation of your code.
 - An object can be targeted for use with only one Intel® high-level design product. If you want to target more than one high-level design product, you must generate a separate object for each target product.

- Object files are combined into a library file using the `fpga_libtool` command. Objects created from different types of source code can be combined into a library, provided all objects target the same high-level design product.

A library is automatically assigned a toolchain version number and can be used only with the targeted high-level design product with the same version number.

Library Toolchain Creation Process



Create Library Objects From Source Code

You can create a library from object files from your source code. A SYCL-based object file includes code for CPU and hardware execution of CPU capturing for use in host and emulation of the kernel.

Create an Object File From Source Code

Use the `fpga_crossgen` command to create library objects from your source code. An object created from your source code contains information required both for emulating the functions in the object and synthesizing the hardware for the object functions.

The `fpga_crossgen` command creates one object file from one input source file. The object created can be used only in libraries that target the same high-level design tool. Also, objects are versioned. Each object is assigned a compiler version number and be used only with high-level design tools with the same version number.

Create a library object using the following command:

```
fpga_crossgen <rtl_spec>.xml --cpp_model <emulation_model>.cpp -o <object_file>
```

The following table describes the parameters:

`fpga_crossgen` Command Parameters

Parameter	Description
<code><rtl_spec></code>	An XML file name that specifies the details about your RTL library.
<code>-o</code>	Optional flag. This option helps you specify an object file name. If you do not specify this option, the object file name defaults to be the same name as the source code file name but with an object file suffix (<code>.o</code> or <code>.obj</code>).

Example command:

```
fpga_crossgen lib_rtl_spec.xml --cpp_model lib_rtl_model.cpp -o lib_rtl.o
```

Packaging Object Files into a Library File

Gather the object files into a library file so that others can incorporate the library into their projects and call the functions that are contained in the objects in the library. To package object files into a library, use the `fpga_libtool` command.

Before you package object files into a library, ensure that you have the path information for all of the object files that you want to include in the library.

All objects you want to package into a library must have the same version number. The `fpga_libtool` command creates libraries encapsulated in operating-system-specific archive files (`.a` on Linux* and `.lib` on Windows*). You cannot use libraries created on one operating system with an Intel® high-level design product running on a different operating system.

Create a library file using the following command:

```
fpga_libtool file1 file2 ... fileN --create <library_name>
```

The command parameters are defined as follows:

Library File Command Parameters

Parameter	Description
<code>file1 file2 ... fileN</code>	You can specify one or more object files to include in the library.
<code>--create <library_name></code>	Allows you to specify the name of the library archive file. Specify the file extension of the library file as <code>.a</code> for Linux-platform libraries.

Example command:

```
fpga_libtool lib_rtl.o --create lib.a
```

where, the command packages objects created from RTL source code into a SYCL library called `lib.a`.

NOTE For additional information, access the code sample on [Github](#).

Using Static Libraries

You can include static libraries in your compiler command along with your source files, as shown in the following command:

```
icpx -fintelfpga main.cpp lib.a
```

NOTE For the functions you implemented in RTL to be usable, you must declare them in your source code so that the compiler can dynamically link the functions. For example:

```
SYCL_EXTERNAL extern "C" void foo()
```

Object Manifest File Syntax of an RTL Library

This section provides the syntax of a simple object manifest file for an RTL library that implements double-precision square root function. The RTL library is implemented in VHDL with a Verilog wrapper.

The following object manifest file is for an RTL library named `my_fp_sqrt_double` (line 2) that implements a SYCL* helper function named `my_sqrtfd` (line 2):

```
<RTL_SPEC>
  <FUNCTION name="my_sqrtfd" module="my_fp_sqrt_double">
    <ATTRIBUTES>
      <IS_STALL_FREE value="yes"/>
      <IS_FIXED_LATENCY value="yes"/>
      <EXPECTED_LATENCY value="31"/>
      <CAPACITY value="1"/>
      <HAS_SIDE_EFFECTS value="no"/>
      <ALLOW_MERGING value="yes"/>
    </ATTRIBUTES>
    <INTERFACE>
      <AVALON port="clock" type="clock"/>
      <AVALON port="resetn" type="resetn"/>
      <AVALON port="ivalid" type="ivalid"/>
      <AVALON port="iready" type="iready"/>
      <AVALON port="ovalid" type="ovalid"/>
      <AVALON port="oready" type="oready"/>
      <INPUT port="datain" width="64"/>
      <OUTPUT port="dataout" width="64"/>
    </INTERFACE>
    <C_MODEL>
      <FILE name="c_model.cl" />
    </C_MODEL>
    <REQUIREMENTS>
      <FILE name="my_fp_sqrt_double_s5.v" />
      <FILE name="fp_sqrt_double_s5.vhd" />
    </REQUIREMENTS>
    <RESOURCES>
      <ALUTS value="2057"/>
      <FFS value="3098"/>
      <RAMS value="15"/>
      <MLABS value="43"/>
      <DSPS value="1.5"/>
    </RESOURCES>
  </FUNCTION>
</RTL_SPEC>
```

Elements and Attributes in the Object Manifest File

XML Element	Description
RTL_SPEC	Top-level element in the object manifest file. There can only be one such top-level element in the file. In the above example, the name <code>RTL_SPEC</code> is historic and carries no file-specific meaning.
FUNCTION	Element that defines the SYCL function that the RTL library implements. The <code>name</code> attribute within the <code>FUNCTION</code> element specifies the function's name. You can have multiple <code>FUNCTION</code> elements, each declaring a different function that you can call from the SYCL kernel. The same RTL library can implement multiple functions by specifying different parameters.

XML Element	Description
ATTRIBUTES	Element containing other XML elements that describe various characteristics (for example, latency) of the RTL library. The example RTL library takes one <code>PARAMETER</code> setting named <code>WIDTH</code>, which has a value of 32. Refer to XML Elements for ATTRIBUTES section below for more details about other ATTRIBUTES-specific elements. NOTE If you create multiple SYCL helper functions for different libraries, or use the same RTL library with different <code>PARAMETER</code> settings, you must create a separate <code>FUNCTION</code> element for each function.
INTERFACE	Element containing other XML elements that describe the RTL library's interface. The example object manifest file shows the Avalon® streaming interface signals that every RTL library must provide (that is, <code>clock</code>, <code>resetn</code>, <code>ivalid</code>, <code>iready</code>, <code>ovalid</code>, and <code>oready</code>). The <code>resetn</code> signal is active low. Its synchronicity depends on the target device: • Arria® 10, Cyclone® V, and Cyclone® 10 GX: The <code>resetn</code> signal is asynchronous to the clock signal. • Agilex™ 7 and Stratix® 10: The <code>resetn</code> signal is synchronous to the clock signal. For more information about reset signal timing, see Agilex 7 and Stratix 10 Design-Specific Reset Requirements for Stall-Free and Stallable RTL Libraries. NOTE The signal names must match the ones specified in the <code>.xml</code> file. An error occurs during library creation if a signal name is inconsistent.
C_MODEL	Element specifying one or more files that implement SYCL model for the function. The model is used only during emulation. However, the <code>C_MODEL</code> element and the associated file(s) must be present when you create the library file.
REQUIREMENTS	Element specifying one or more RTL resource files (that is, <code>.v</code>, <code>.sv</code>, <code>.vhd</code>, <code>.hex</code>, and <code>.mif</code>). The specified paths to these files are relative to the location of the object manifest file. Each RTL resource file becomes part of the associated Platform Designer component that corresponds to the entire SYCL system. NOTE The SYCL library feature does not support <code>.qip</code> files. The Intel® oneAPI DPC++/C++ Compiler error occurs if you compile a SYCL kernel while using a library that includes an unsupported resource file type.

XML Element	Description
RESOURCES	Optional element specifying the FPGA resources that the RTL library uses. If you do not specify this element, the FPGA resources that the RTL library uses defaults to zero.

XML Elements for ATTRIBUTES

In the object manifest file of the RTL library, there are XML elements under `ATTRIBUTES` that you can specify to set the library's characteristics.

Attention Except for `IS_STALL_FREE` and `EXPECTED_LATENCY`, all elements have safe values. If you are unsure which value you should specify for an attribute, set it to the safe value. Compiling your kernel with a library that uses safe values results in functional hardware. However, the hardware might be larger than the actual size.

XML Elements Associated with the ATTRIBUTES Element in the Object Manifest File of an RTL Library

XML Element	Description
IS_STALL_FREE	Instructs the compiler properly handle all stall and valid signals. In this case, the compiler can save some area by not generating stall logic around your RTL library. Set <code>IS_STALL_FREE</code> to "yes" to indicate that the library neither generates stalls internally nor can it properly handle incoming stalls. The library simply ignores its stall input. If you set <code>IS_STALL_FREE</code> to "no", the library must properly handle all stall and valid signals. NOTE If you set <code>IS_STALL_FREE</code> to "yes", you must also set <code>IS_FIXED_LATENCY</code> to "yes". Also, if the RTL library has an internal state, it must properly handle <code>invalid=0</code> inputs. An incorrect <code>IS_STALL_FREE</code> setting leads to incorrect results in hardware.
IS_FIXED_LATENCY	Indicates whether the RTL library has a fixed latency. Set <code>IS_FIXED_LATENCY</code> to "yes" if the RTL library always takes a known number of clock cycles to compute its output. The value you assign to the <code>EXPECTED_LATENCY</code> element specifies the number of clock cycles. The safe value for <code>IS_FIXED_LATENCY</code> is "no". When you set <code>IS_FIXED_LATENCY="no"</code>, the <code>EXPECTED_LATENCY</code> value must be at least 1.

XML Element	Description
EXPECTED_LATENCY	NOTE For a given library, you may set <code>IS_FIXED_LATENCY</code> to "yes" and <code>IS_STALL_FREE</code> to "no". Such a library produces its output in a fixed number of clock cycles and handles stall signals properly. Specifies the expected latency of the RTL library. If you set <code>IS_FIXED_LATENCY</code> to "yes", the <code>EXPECTED_LATENCY</code> value indicates the number of pipeline stages inside the library. In this case, you must set this value to be the exact latency of the library. Otherwise, the compiler generates incorrect hardware. For a library with variable latency, the compiler balances the pipeline around this library to the <code>EXPECTED_LATENCY</code> value that you specify. For libraries that can stall and require use of signals such as <code>iready</code>, set the <code>EXPECTED_LATENCY</code> value to at least 1. The specified value and the actual latency might differ, which might affect the number of stalls inside the pipeline. However, the resulting hardware is correct.
CAPACITY	Specifies the number of multiple inputs that the library can process simultaneously. You must specify a value for <code>CAPACITY</code> if you also set <code>IS_STALL_FREE</code>="no" and <code>IS_FIXED_LATENCY</code>="no". Otherwise, you do not need to specify a value for <code>CAPACITY</code>. If <code>CAPACITY</code> is strictly less than <code>EXPECTED_LATENCY</code>, the compiler automatically inserts capacity-balancing FIFO buffers after this library when necessary. The safe value for <code>CAPACITY</code> is 1.
HAS_SIDE_EFFECTS	Indicates whether the RTL library has side effects. Libraries that have internal states or communicate with external memories are examples of libraries with side effects. Set <code>HAS_SIDE_EFFECTS</code> to "yes" to indicate that the library has side effects. Specifying <code>HAS_SIDE_EFFECTS</code> to "yes" ensures that optimization efforts do not remove calls to libraries with side effects.

XML Element	Description
ALLOW_MERGING	Stall-free libraries with side effects (that is, <code>IS_STALL_FREE="yes"</code> and <code>HAS_SIDE_EFFECTS="yes"</code>) must properly handle <code>invalid=0</code> input cases because the library might receive invalid data occasionally. The safe value for <code>HAS_SIDE_EFFECTS</code> is "yes". Instructs the compiler to merge multiple instances of the RTL library. Set <code>ALLOW_MERGING</code> to "yes" to allow merging of multiple instances of the library. Intel® recommends setting <code>ALLOW_MERGING</code> to "yes". The safe value for <code>ALLOW_MERGING</code> is "no". NOTE Marking the library with <code>HAS_SIDE_EFFECTS="yes"</code> does not prevent merging.
PARAMETER	Specifies the value of an RTL library parameter. <code>PARAMETER</code> attributes: <code>name</code>: Specifies the name of the RTL library parameter. <code>value</code>: Specifies a decimal numeric value for the parameter. <code>type</code>: Specifies a system parameter the value of which is used as the RTL library parameter value. You can use the <code>bspaddresswidth</code> parameter that specifies the Avalon® memory bus width required to address the memory range configured for SYCL global memory in the board support package. NOTE You can specify the value for an RTL library parameter using either a <code>value</code> or a <code>type</code> attribute.

XML Elements for INTERFACE

In the object manifest file of the RTL library within a SYCL library, there are XML elements under `INTERFACE` that you can define to specify aspects of the RTL library's interface (for example, Avalon® streaming interface).

Mandatory XML Elements Associated with the `INTERFACE` Element in the Object Manifest File of an RTL Library

XML Element	Description
INPUT	Specifies the RTL library input parameter.

XML Element	Description
	INPUT attributes: • <code>port</code>: Specifies the RTL library port name. • <code>width</code>: Specifies the port width in bits. The input parameters are concatenated to form the input stream. NOTE Aggregate data structures, such as structs and arrays, are not supported as input parameters.
OUTPUT	Specifies the RTL library output parameter. OUTPUT attributes: • <code>port</code>: Specifies the port name of the RTL library. • <code>width</code>: Specifies the port width in bits. The return value from the input stream is sent out via the output parameter on the output stream. NOTE Aggregate data structures, such as structs and arrays, are not supported as input parameters.

XML Elements for RESOURCES

In the object manifest file of the RTL library within a SYCL library, there are optional elements under `RESOURCES` that you can define to specify the FPGA resource utilization of the library. If you do not specify a particular element, it has a default value of zero.

Additional XML Elements to Support External Memory Access

XML Element	Description
ALUTS	Specifies the number of combinational adaptive look-up tables (ALUTs) that the library uses.
FFS	Specifies the number of dedicated logic registers that the library uses.
RAMS	Specifies the number of block RAMs that the library uses.
DSPS	Specifies the number of digital signal processing (DSP) blocks that the library uses.
MLABS	Specifies the number of memory logic arrays (MLABs) that the library uses. This value is equal to the number of adaptive logic modules (ALMs) that is used for memory divided by 10 because each MLAB consumes 10 ALMs.

Restrictions and Limitations in RTL Support

When creating your RTL module for use inside SYCL kernels, ensure that the RTL module operates within the following restrictions:

- An RTL module must use a single input Avalon® streaming interface. A single pair of ready and valid logic must control all the inputs. You have the option to provide the necessary Avalon® streaming interface ports but declare the RTL module as stall-free. In this case, you do not have to implement proper stall behavior because the Intel® oneAPI DPC++/C++ Compiler creates a wrapper for your module. Refer to [Object Manifest File Syntax of an RTL Library](#) for additional information.

NOTE You must handle `invalid` signals properly if your RTL module has an internal state. Refer to [Stall-Free RTL](#) for more information.

- The RTL module must work correctly regardless of the kernel clock frequency.
- RTL modules cannot connect to external I/O signals. All input and output signals must come from a SYCL kernel.
- An RTL module must have a `clock` port, a `resetn` port, and Avalon® streaming interface input and output ports (that is, `invalid`, `ovvalid`, `iready`, `oready`). Name the ports as specified here.
- RTL modules cannot act as stand-alone SYCL kernels. RTL modules can only be helper functions and be integrated into a SYCL kernel during kernel compilation.
- Every function call corresponding to RTL module instantiation is independent of other instantiations. There is no hardware sharing.
- Do not incorporate kernel code into a SYCL library file. Incorporating kernel code into the library file causes the offline compiler to issue an error message. You may incorporate helper functions into the library file.
- An RTL component must receive all its inputs at the same time. A single `invalid` input signifies that all inputs contain valid data.
- You can only set RTL module parameters in the `<RTL module description file name>.xml` specification file and not in the SYCL kernel source file. To use the same RTL module with multiple parameters, create a separate `FUNCTION` tag for each parameter combination.
- You can only pass data inputs to an RTL module by value via the SYCL kernel code. Do not pass data inputs to an RTL module via pass-by-reference, structs, or channels. In the case of channel data, pass the extracted scalar data.

NOTE Passing data inputs to an RTL module via pass-by-reference or structs causes a fatal error in the offline compiler.

- The debugger (for example, GDB for Linux) cannot step into a library function during emulation if the library is built without the debug information. However, irrespective of whether the library is built with or without the debug data, optimization and area reports are not mapped to the individual code line numbers inside a library.
- Names of RTL module source files cannot conflict with the file names of Intel® oneAPI DPC++/C++ Compiler IP. Both the RTL module source files and the compiler IP files are stored in the `<kernel file name>/system/synthesis/submodules` directory. Naming conflicts cause existing compiler IP files in the directory to be overwritten by the RTL module source files.
- The compiler does not support `.qip` files. You must manually parse nested `.qip` files to create a flat list of RTL files.

Tip It is challenging to debug an RTL module that works correctly on its own but works incorrectly as part of a SYCL kernel. Double-check all parameters under the [XML Elements for ATTRIBUTES](#) in the `<RTL object manifest file name>.xml` file.

Agilex™ 7 and Stratix® 10 Design-Specific Reset Requirements for Stall-Free and Stallable RTL Libraries

When creating an RTL library for Agilex™ 7 and Stratix® 10 designs, ensure that the library satisfies specific logic reset requirements.

Reset Requirements for Stall-Free RTL Libraries

A stall-free RTL library is a fixed-latency library for which the Intel® oneAPI DPC++/C++ Compiler can optimize away stall logic. It accepts valid data, and does not have a `ready_in` signal at the end.

- When creating a stall-free RTL library for an Stratix® 10 design, use synchronous clear signals only.
- After **deassertion** of the reset signal to the stall-free RTL library, the library must be operational within 15 clock cycles. If the reset signal is pipelined within the library, this requirement limits the reset pipelining to no more than 15 stages.

Reset Requirements for Stallable RTL Libraries

A stallable RTL library has a variable latency, and it relies on backpressured input and output interfaces to function correctly.

- When creating a stallable RTL library for an Agilex™ 7 and Stratix® 10 designs, use synchronous clear signals only.
- After assertion of the reset signal to the stallable RTL library, the library must deassert its `oready` and `ovalid` interface signals within 40 clock cycles.
- After deassertion of the reset signal to the stallable RTL library, the library must be fully operational within 40 clock cycles. The library signals its readiness by asserting the `oready` interface signal.

Stall-Free RTL

The compiler can optimize hardware resource usage and performance by not placing stall logic around an RTL module with fixed latency.

If you have an RTL module with a fixed latency that you want integrated into your pipeline without surrounding stall logic, ensure that you set attributes in the object manifest file (`.xml`) as follows:

1. Specify a value for the `EXPECTED_LATENCY` attribute (under the `FUNCTION` element) so that the latency equals the number of pipeline stages in the module.

Important An inaccurate `EXPECTED_LATENCY` value causes the RTL module to be out of sync with the rest of the pipeline, and can lead to functionally incorrect results.

2. Set the `IS_STALL_FREE` attribute under the `FUNCTION` element to "yes".

This setting instructs the compiler to avoid placing stall logic around the RTL module. This setting also tells the compiler that the RTL module produces a result after the number cycles specified in the `EXPECTED_LATENCY` attribute after accepting input values. The stall free logic produces a result every cycle but the result is delayed by the number cycles specified in the `EXPECTED_LATENCY` attribute.

For RTL modules with a fixed latency, the output signals (`ovalid` and `oready`) can have constant high values, and the input ready signal (`iready`) can be ignored.

A stall-free RTL module might receive an invalid input signal (`ivalid` is low). In this case, the module must produce invalid data on the output `EXPECTED_LATENCY` cycles after the cycle in which the input was invalid. For a stall-free RTL module without an internal state, you might find it convenient to propagate the invalid input through the module. If the module has an internal state, that state should not be affected by data inputs that are not accompanied by `ivalid = 1`.

Additional FPGA Acceleration Flow Considerations

19

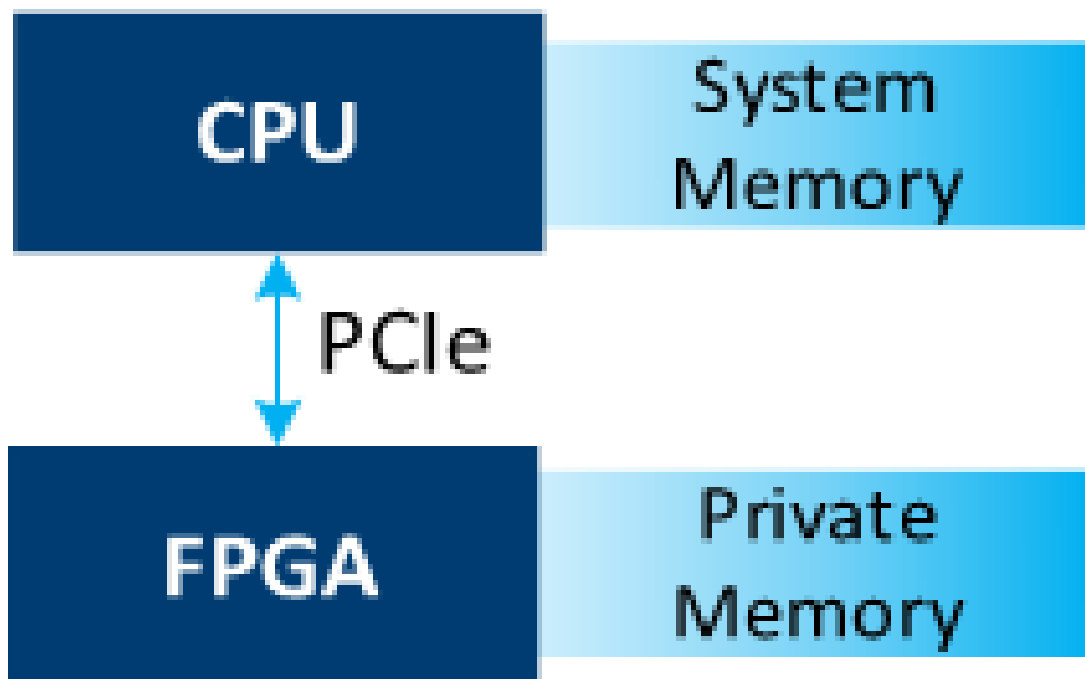
FPGA-CPU Interaction

One of the main influences on the overall performance of an FPGA design is how kernels executing on the FPGA interact with the host on the CPU.

Host and Kernel Interaction

FPGA devices typically communicate with the host (CPU) via [PCIe](#).

FPGA Device Communication with the Host



This is an important factor influencing the performance of SYCL* programs targeting FPGAs. Furthermore, the first time you run a particular SYCL program, you must configure the FPGA with its hardware bitstream, and this may require several seconds.

Data Transfer

Typically, the FPGA board has its own private [double data rate \(DDR\)](#) memory on which it primarily operates. The CPU must bulk transfer or [direct memory access \(DMA\)](#) all data that the kernel needs to access into the FPGA's local DDR memory. After the kernel completes its operations, results must be transferred over DMA back to the CPU. The transfer speed is bound by the PCIe link itself and the efficiency of the DMA solution.

The following are the techniques to manage these data transfer times:

- SYCL allows buffers to be tagged as read-only or write-only, which eliminates some unnecessary transfers.

- Improve the overall system efficiency by maximizing the number of concurrent operations. Since PCIe supports simultaneous transfers in opposite directions and PCIe transfers do not interfere with kernel execution, you can apply techniques such as double buffering. Refer to [Double Buffering Host Utilizing Kernel Invocation Queue](#) and the [double_buffering](#) tutorial on GitHub for additional information about these techniques.
- Improve data transfer throughput by prepinning system memory on board variants that support Restricted USM. Refer to [Prepinning Memory](#) for additional information.

Configuration Time

You must program the hardware bitstream on the FPGA device in a process called configuration. Configuration is a lengthy operation requiring several seconds of communication with the FPGA device. The SYCL* runtime manages configuration for you automatically. The runtime decides when the configuration occurs. For example, the configuration might be triggered when a kernel is first launched, but subsequent launches of the same kernel may not trigger configuration since the bitstream has not changed. Therefore, during development, time the execution of the kernel after the FPGA has been configured. For example, by performing a warm-up execution of the kernel before timing kernel execution. You must remove this warm-up execution in the production code.

In addition to the configuration done by the runtime, the FPGA board must be initialized with the BSP board variant targeted by the kernel. This initialization is required if the system containing the FPGA board is rebooted or power-cycled. For more information about BSP initialization, refer to [Managing an FPGA Board](#).

Multiple Kernel Invocations

If a SYCL program submits the same kernel to a SYCL queue multiple times (for example, by calling `single_task` within a loop), only one kernel invocation is active at a time. Each subsequent invocation of the kernel waits for the previous run of the kernel to complete.

FPGA Boards and Board Support Packages (BSPs)

The multiarchitecture binary generated by the FPGA acceleration flow requires additional hardware and software to run. This section provides an overview of these dependencies when you use the Intel® oneAPI DPC++/C++ Compiler to build and run all-in-one FPGA acceleration designs.

The following terms are often mentioned when discussing the multiarchitecture binaries and the FPGA acceleration flow:

- [Boards](#)
- [Board Support Packages \(BSPs\)](#)
- [Board Variants](#)
- [FPGA Client Driver \(FCD\)](#)

Boards

Like a GPU, an FPGA is an integrated circuit that must be mounted onto a card or a board to interface with a server or a desktop computer. In addition to the FPGA, the board provides memory, power, and thermal management, and physical interfaces to allow the FPGA to communicate with other devices.

Board Support Packages (BSPs)

A BSP consists of software layers and an FPGA hardware scaffold design that makes it possible to target the FPGA through the Intel® oneAPI DPC++/C++ Compiler. The functionality described in your SYCL device code is inserted into this scaffold to produce the final FPGA device image that is part of the multiarchitecture binary.

An important part of the BSP is the `board_spec.xml` file that describes the hardware interface of the board.

Generating the FPGA Optimization Reports for your board requires a BSP.

The [FPGA Support Package for Intel® oneAPI DPC++/C++ Compiler](#) webpage includes links to board vendors to help you choose a board and a BSP.

The Open FPGA stack (OFS) is the recommended tool for developing BSPs to use with oneAPI. OFS is an open-source hardware and software framework. OFS-based BSPs for oneAPI typically use the OFS FPGA Interface Manager (FIM) for the targeted board and the OFS oneAPI Accelerator Support Package (oneAPI ASP).

For more information about OFS and how to develop a custom BSP with OFS, refer to <https://ofs.github.io>.

For more information about the oneAPI ASP, refer to [oneAPI Accelerator Support Package \(ASP\): Getting Started User Guide](#) and [oneAPI Accelerator Support Package\(ASP\) Reference Manual: Open FPGA Stack](#).

You can have multiple BSPs installed to support multiple FPGA boards in your system. The BSPs are managed by the [FPGA client driver \(FCD\)](#).

Board Variants

A BSP can provide multiple board variants that support different functionality. For example, a BSP could contain two variants that differ in their support for Unified Shared Memory (USM). For additional information about USM, refer to the [Unified Shared Memory](#) and [USM Interfaces](#) topics in the SYCL* Reference Documentation.

A board can be supported by more than one BSP, and a BSP might support more than one board variant.

NOTE When running an executable on an FPGA board, you must ensure that you have initialized the FPGA board for the board variant that the executable is targeting. For information about initializing an FPGA board, refer to [Managing an FPGA Board](#).

FPGA Client Driver (FCD)

An FPGA client driver (FCD) helps the oneAPI runtime discover a boards. The installation process for a BSP installs a new FCD.

When you have multiple boards installed on the same system, the FCD helps your program to discover the installed boards and run your application on the desired board with its corresponding BSP.

On Linux* systems, the default installation directory for FCDs is `/opt/Intel/OpenCLFPGA/oneAPI/Boards`, which might require root privileges. You can override the default FCD installation directory by setting the `ACL_BOARD_VENDOR_PATH` environment variable to the FCD installation directory.

Installable Client Driver (ICD)

The oneAPI runtime stack can include multiple OpenCL™ implementations, each provides support for its corresponding SYCL* platform (such as GPU or FPGA). The FPGA Support Package for the Intel® oneAPI DPC++/C++ Compiler ships an OpenCL™ Installable Client Driver (ICD) Loader from the Khronos Group™ to allow the host program to enumerate and select between multiple SYCL platforms. With the ICD loader you can have a GPU and an FPGA accelerator installed in the same host.

For more information about the ICD, refer to [The OpenCL™ Installable Client Driver \(ICD\)](#).

Comparing FCD and ICD

The ICD and FCD serve different functions:

- The ICD allows you to enumerate and choose between different runtime implementations in your host program. The ICD is required when you run your multiarchitecture binary in emulation, simulation, or on hardware.

- The FCD allows you to enumerate and choose between different BSPs in your host program. The FCD is required only when you run your multiarchitecture binary on hardware.

The FPGA Board Utility Commands

The FPGA Support Package for the Intel® oneAPI DPC++/C++ Compiler includes utility commands that you can use to manage your FPGA board.

Use these utilities for the following tasks:

- Installing/uninstalling a BSP
- Listing the detected FPGA boards
- Initializing an FPGA board
- Running FPGA board and system configuration diagnostics

FPGA Board Utility Commands

The FPGA board utility commands can be categorized as follows:

- **Board Management Utilities**

These commands get your board ready to be recognized and accessed by the oneAPI runtime, and they provide information about the boards that the runtime can detect.

- **Advanced Utilities**

These commands directly program an FPGA device or the board flash memory. You likely rarely need these commands when developing and running a typical multiarchitecture binary application.

Board Management Utilities

Command	Description
<code>aocl install</code>	Installs the BSP, including the FPGA client driver (FCD). Important Do not move BSP files to a different directory after you install it. If you want to move the installation location of BSP files: 1. Uninstall the BSP with the <code>aocl uninstall</code> utility. 2. Move the BSP files to their new location. 3. Reinstall the BSP from the new location with the <code>aocl install</code> utility.
<code>aocl list-devices</code>	Lists the board names for detected FPGA boards. These board names are sometimes required for other FPGA utility commands. If you have an FPGA board physically connected to your system but have not yet installed its BSP with the <code>aocl install</code> command, it will not be listed in the <code>aocl list-devices</code> command output.
<code>aocl diagnose</code>	Provides a list of detected boards, their statuses, board descriptions, and some diagnostic information about the boards. If you specify a board name when running the <code>aocl diagnose</code> command, the command runs more detailed diagnostic tests on your board. Refer to your board vendor BSP documentation for details about the information provided from these diagnostic tests.
<code>aocl initialize</code>	Prepares the FPGA board to run the workload. You must initialize a board before you run a multiarchitecture binary application.

Command	Description
<code>aocl uninstall</code>	Uninstalls the BSP for a board and removes its FCD.

Advanced Utilities

Command	Description
<code>aocl program</code>	Programs the FPGA device with FPGA device image that you extracted from your multiarchitecture binary file. Remember Typically, this operation is handled by the runtime and you do not need to program the FPGA device explicitly with this command. You might consider explicitly programming the FPGA device when you want to avoid the overhead of programming the device.
<code>aocl flash</code>	For BSPs that support programming the board flash memory, you can use this command to load the board flash memory with a specified startup configuration.

Managing an FPGA Board

Managing an FPGA board typically involves the following tasks:

- [Installing a board support package \(BSP\)](#)
- [Listing detected FPGA boards](#)
- [Initializing an FPGA board](#)
- [Getting information about an FPGA board and running diagnostic tests on the board](#)
- [Uninstalling a BSP](#)

Installing a BSP (`aocl install` Command)

Installing a BSP with the `aocl install` command installs the device drivers that make the FPGA board accessible to the host operating system, and it installs the FPGA client driver (FCD) for your board to make the board accessible to the oneAPI runtime.

Before you install a BSP, you must have the FPGA board connected to your host system.

To run the `aocl install` command, you must know the directory that contains the BSP files for your board. Optionally, you can set the `AOCL_BOARD_PACKAGE_ROOT` environment variable to point this directory. If you have the `AOCL_BOARD_PACKAGE_ROOT` environment variable, you can omit the path to the BSP from any FPGA utility commands that require that path.

`aocl install` Command Use Cases

Use Case	Command
Install a BSP	<pre>aocl install <BSP_root_folder></pre> where <code><BSP_root_folder></code> is the full path of the directory that contains the BSP.
Install the FCD for the board only	<pre>aocl install <BSP_root_folder> -fcd-only</pre> where <code><BSP_root_folder></code> is the full path of the directory that contains the BSP. For this use case, the FPGA board device drivers must be already be installed. This use case typically occurs if you have previous run the <code>aocl uninstall -fcd-only</code> command.

After you have run the `aocl install` command, you can use the `aocl list-devices` and `aocl diagnose` commands to confirm that the BSP installation was successful.

Listing Detected FPGA Boards (aocl list-devices Command)

Use the `aocl list-devices` command to show a list of FPGA boards that are detected on your system. The board device name shown under the **Device Name** section in the output of this command is needed for board-specific FPGA utility commands such as `aocl initialize`.

The command generates output similar to the following:

```

-----
Device Name:
aocl0

BSP Install Location:
<board_package_path>

Vendor: Intel(R) Corporation

Phys Dev Name    Status    Information
<phys_dev_name> Passed    <descriptive_board_name>
                                PCIe dev_id = <dev_id>, bus:slot.func = 01:00.00, Gen3 x8
                                FPGA temperature = 32.6016 degrees C.

DIAGNOSTIC_PASSED
-----

```

Initializing an FPGA Board (aocl initialize Command)

Initializing an FPGA board with the `aocl initialize` command lets the oneAPI SYCL* runtime know that the FPGA board (and board variant) is available. Your multiarchitecture binary will fail to run it if the target board is not initialized.

You must initialize an FPGA board after any of the following events:

- The FPGA board host system was power cycled.
- A non-SYCL* workload was run on the FPGA device.
- The multiarchitecture binary was recompiled to a different board variant (as specified by the `-Xs target` compiler command option)

To initialize a board, you must know the FPGA board device name (from the `aocl list-devices` command), and the name of the board variant as specified by the `-Xs target` compiler command option when the multiarchitecture binary was compiled. Whoever compiled the multiarchitecture binary can provide you with the board variant to specify.

Initialize a board with the following command:

```
aocl initialize <board_device_name><board_variant>
```

Running the Board-Related Diagnostic Utility (aocl diagnose Command)

Running the board-related diagnostic utilities checks and reports your OpenCL* installable client driver (ICD) and your FPGA client driver (FCD) configuration. For more information about the OpenCL* installable client driver (ICD) and its role in developing your multiarchitecture binary, refer to [The OpenCL™ Installable Client Driver \(ICD\)](#).

The utility can report information about the FPGA boards that are discoverable by the host operating system.

The diagnostic utility has the following use modes:

Default mode	In default mode, the utility runs ICD and FCD diagnostics and lists the FPGA boards that are discoverable by the host operating system. Run the utility in default mode with the following command:
--------------	---

```
aocl diagnose
```

If your system is configured correctly, the command output includes the following messages:

- ICD diagnostics passed
- DIAGNOSTIC_PASSED

Client driver-only mode

In client driver-only mode, the utility runs only the ICD and FCD diagnostics. Run the utility in client driver-only mode with the following command:

```
aocl diagnose -icd-only
```

If your system is configured correctly, the command output includes the following messages:

- ICD diagnostics passed

You can successfully run the utility in client driver-only mode without any FPGA boards present on the host system.

Board diagnostic mode

In board diagnostic mode, the utility runs a diagnostic routine provided by your FPGA acceleration board vendor. To run the diagnostic routine, the FPGA board must be discoverable by the host operating system. Run the utility in board diagnostic mode with the following command:

```
aocl diagnose <board_device_name>
```

where **<board_device_name>** is the board device name as reported under the **Device Name** section of either the `aocl list-devices` command output or the default mode `aocl diagnose` command output.

Uninstalling a BSP (`aocl uninstall` Command)

Uninstalling a BSP removes the FCD for the board and uninstalls the device drivers for the board. You can also choose to uninstall only the FCD.

`aocl uninstall` Command Use Cases

Use Case	Command
Uninstall a BSP	<code>aocl uninstall <BSP_root_folder></code> where <BSP_root_folder> is the full directory path of the BSP that you are uninstalling.
Uninstall the FCD for the board only	<code>aocl uninstall <BSP_root_folder> -fcd-only</code>
For this use case, the FPGA board device drivers remain installed.	where <BSP_root_folder> is the full directory path of the BSP that you are uninstalling.
Use the <code>aocl install -fcd-install</code> command to reinstall the FCD.	

Advanced Board Management

Advanced board management involves programming the FPGA device or programming the flash memory on board (when supported by the BSP).

Programming the FPGA Device (`aocl program` Command)

Typically, the oneAPI SYCL* runtime programs the FPGA device on the board when you run your multiarchitecture binary. However, if you want to avoid reprogramming time overhead when the multiarchitecture binary is invoked, you can program the FPGA device before you run the binary.

Before you can program the FPGA device, you must have completed the following tasks:

- Extract the FPGA device image (.aocx file) from the multiarchitecture binary. For instructions on how to extract the file, refer to [Extracting the FPGA Device Image \(.aocx\) File from a Multiarchitecture Binary File](#).
- Determine the name of the FPGA board that you want to program. Use the `aocl list-devices` or `aocl diagnose` commands to get a list of boards recognized by the host system.

To program the FPGA device in advance, run the following command:

```
aocl program <board_device_name><extracted_.aocx_filename>
```

Programming the Flash Memory (aocl flash Command)

Directly programming the flash memory can be helpful if the SYCL* runtime is updated and the runtime is incompatible with the hardware configuration file programmed in the flash memory. In this case, if the board cannot be reprogrammed after rebooting the board, you can program the flash memory directly.

Before you program the flash memory, you have the following things:

- FPGA hardware startup configuration file (.aocx) to load into flash memory.
- The name of the FPGA board that you want to program. Use the `aocl list-devices` or `aocl diagnose` commands to get a list of devices detected by the host system.

To program the board flash memory, run the following command:

```
aocl flash <board_device_name><startup_configuration_.aocx_file>
```

Extracting the FPGA Device Image (.aocx) File from a Multiarchitecture Binary File

To extract the configuration file, use the `clang-offload-extract` command:

1. Determine the path to the `clang-offload-extract` command with the following command:

```
icpx --print-prog-name=clang-offload-extract
```

This command returns the full path to the `clang-offload-extract` command.

2. Extract the FPGA hardware configuration file from the multiarchitecture binary with the following command:

```
<full_command_path>/clang-offload-extract --output <output_file_name>.aocx <sycl_executable>
```

Where:

- `<full_command_path>` is the full path to the `clang-offload-extract` command that you determined earlier.
- `<output_file_name>` is the file name that you want to give to the FPGA hardware configuration file.
- `<sycl_executable>` is the file name of the multiarchitecture binary file that contains the FPGA hardware configuration file that you want to extract.

Restriction You can extract .aocx files only from multiarchitecture binary files compiled for a single FPGA target device.

The OpenCL™ Installable Client Driver (ICD)

The oneAPI runtime stack can include multiple OpenCL™ implementations, each provides support for its corresponding SYCL* platform (such as GPU or FPGA). The FPGA Support Package for the Intel® oneAPI DPC++/C++ Compiler ships an OpenCL™ Installable Client Driver (ICD) Loader from the Khronos Group™ to allow the host program to enumerate and select between multiple SYCL platforms. With the ICD loader you can have a GPU and an FPGA accelerator installed in the same host.

When you compile your multiarchitecture binary, the Intel® oneAPI DPC++/C++ Compiler links the host application to the ICD Loader library automatically. For more information about the OpenCL™ ICD Loader and how it searches for the available OpenCL implementations, refer to <https://github.com/KhronosGroup/OpenCL-ICD-Loader>.

Important The ICD and FCD serve different functions:

- The ICD allows you to enumerate and choose between different runtime implementation in your host program. The ICD is required when you run your multiarchitecture binary in emulation, simulation, or on hardware.
- The FCD allows you to enumerate and choose between different BSPs in your host program. The FCD is required only when you run your multiarchitecture binary on hardware.

Verifying ICD Configuration for FPGA Targets

The FPGA Support Package for the Intel® oneAPI DPC++/C++ Compiler provides the OpenCL implementation for the FPGA platform in the following library files:

- **Linux:** `libalteracl.so`, found under `<oneapi_root>/compiler/latest/opt/oclfpga/host/linux64/lib`
- **Windows:** `alteracl.dll`, found under `<oneapi_root>/compiler/latest/opt/oclfpga/host/windows64/bin`

To confirm that the ICD Loader can find the OpenCL implementation for the FPGA platform, do the following:

Linux	Ensure that the file <code>/etc/OpenCL/vendors/Altera.icd</code> exists in the system and that the file contains the text <code>libalteracl.so</code> .
Windows	Open the <code>regedit</code> with administrator privilege and go to the Windows registry key HKEY_LOCAL_MACHINE\SOFTWARE\Khronos\OpenCL\Vendors . The value of Name should be <code><oneapi_root>/compiler/<oneapi_version>/opt/oclfpga/host/windows64/bin/alteracl.dll</code> . The Type should be DWORD , and the Data should be 00000000 . For example, the registry key should resemble the following: HKEY_LOCAL_MACHINE\SOFTWARE\Khronos\OpenCL\Vendors "<oneapi_root>/compiler/<oneapi_version>/opt/oclfpga/host/windows64/bin/alteracl.dll"=dword:00000000

Overriding Default Environment Variables

Use the following environment variables to override some of the ICD loader default behavior.

Environment Variable	Behavior	Example Format
OCL_ICD_FILENAMES	Specifies a list of additional OpenCL implementations to load. The libraries here are enumerated before any other ones discovered via default mechanisms.	export OCL_ICD_FILENAMES=libVendorA.so:libVendorB.so set OCL_ICD_FILENAMES=vendor_a.dll;vendor_b.dll
OCL_ICD_VENDORS	(Linux only) Specifies a directory to scan for <code>.icd</code> files to enumerate in place of the default <code>/etc/OpenCL/vendors</code> .	export OCL_ICD_VENDORS=/my/local/icd/search/path

Targeting Multiple Identical FPGA Devices

The Intel® oneAPI DPC++/C++ Compiler supports targeting multiple identical FPGA devices from a single host CPU. This allows to improve your design's throughput by parallelizing the execution of your program on multiple FPGAs.

Intel® recommends creating a single context with multiple device queues because, with multi-context, buffers at the OpenCL layer must be copied between contexts, which introduces overhead and impacts overall performance. However, you can use multi-context if your design is simple and the overhead does not affect the overall performance.

Follow one of the following methods to target multiple FPGA devices:

Create a Single Context with Multiple Device Queues

Perform the following steps to target multiple FPGA devices with a single context:

1. Create a single SYCL* context to encapsulate a collection of FPGA devices of the same platform.

```
context ctxt(deviceList, &m_exception_handler);
```

2. Create a SYCL queue for each FPGA device.

```
std::vector<queue> queueList;
for (unsigned int i = 0; i < ctxt.get_devices().size(); i++) {
    queue newQueue(ctxt, ctxt.get_devices()[i], &m_exception_handler);
    queueList.push_back(newQueue);
}
```

3. Submit either the same or different device codes to all available FPGA devices. If you want to target a subset of all available devices, then you must first perform device selection to filter out unwanted devices.

```
for (unsigned int i = 0; i < queueList.size(); i++) {
    queueList[i].submit([&](handler& cgh) {...});
}
```

Create a Context For Each Device Queue (Multi-Context)

Perform the following steps to target multiple FPGA devices with multiple contexts:

1. Obtain a list of all available FPGA devices. Optionally, you can select a device based on the device member or device property. For device properties such as device name, use the member function `get_info()` const with the desired device property.

```
std::vector<device> deviceList = device::get_devices();
```

2. Create a SYCL queue for each FPGA device.

```
std::vector<queue> queueList;
for (unsigned int i = 0; i < deviceList.size(); i++) {
    queue newQueue(deviceList[i], &m_exception_handler);
    queueList.push_back(newQueue);
}
```

3. Submit either the same or different device codes to all available FPGA devices. If you want to target a subset of all available devices, then you must first perform device selection to filter out unwanted devices.

```
for (unsigned int i = 0; i < queueList.size(); i++) {
    queueList[i].submit([&](handler& cgh) {...});
}
```

Limitations

Consider the following limitations when targeting multiple FPGA devices:

- All FPGA devices use the same FPGA bitstream.
- All FPGA devices used must be of the same FPGA card (same `-Xtarget target`)

Targeting Multiple Platforms

To compile a design that targets multiple target device types (using different device selectors), you can run the following commands:

Emulation Compile

For compiling your SYCL* code for the FPGA emulator target, execute the following commands:

```
Linux
icpx jit_kernel.cpp -c -o jit_kernel.o
icpx -fintelfpga -fsycl-link=image fpga_kernel.cpp -o fpga_kernel.a
icpx -fintelfpga main.cpp jit_kernel.o fpga_kernel.a
```

```
Windows
icx-cl jit_kernel.cpp -c -o jit_kernel.o
icx-cl -fintelfpga -fsycl-link=image fpga_kernel.cpp -o fpga_kernel.lib
icx-cl -fintelfpga main.cpp jit_kernel.o fpga_kernel.lib
```

The design uses libraries and includes an FPGA kernel (AOT flow) and a CPU kernel (JIT flow).

Specifically, there should be a main function residing in the `main.cpp` file and two kernels for both CPU (`jit_kernel.cpp`) and FPGA (`fpga_kernel.cpp`).

Sample `jit_kernel.cpp` file:

```
sycl::cpu_selector device_selector;
queue deviceQueue(device_selector);
deviceQueue.submit([&](handler &cgh) {
    // CPU Kernel function
});
```

Sample `fpga_kernel.cpp` file:

```
#if FPGA_SIMULATOR
    auto selector = sycl::ext::intel::fpga_simulator_selector_v;
#elif FPGA_HARDWARE
    auto selector = sycl::ext::intel::fpga_selector_v;
#else // #if FPGA_EMULATOR
    auto selector = sycl::ext::intel::fpga_emulator_selector_v;
#endif
queue deviceQueue(device_selector);
deviceQueue.submit([&](handler &cgh) {
    // FPGA Kernel Function
});
```

FPGA Hardware Compile

To compile for the FPGA hardware target, add the `-Xshardware` flag and remove the `-DFPGA_EMULATOR` flag, as follows:

```
# For Linux:
icpx jit_kernel.cpp -c -o jit_kernel.o

//Hardware compilation command. Takes a long time to complete.
icpx -fintelfpga -fsycl-link=image -Xshardware fpga_kernel.cpp -o fpga_kernel.a

icpx -fintelfpga main.cpp jit_kernel.o fpga_kernel.a

# For Windows:
icx-cl jit_kernel.cpp -c -o jit_kernel.o

//Hardware compilation command. Takes a long time to complete.
icx-cl -fintelfpga -fsycl-link=image -Xshardware fpga_kernel.cpp -o fpga_kernel.lib

icx-cl -fintelfpga main.cpp jit_kernel.o fpga_kernel.lib
```

Split Kernel into Multiple FPGA Images (Linux only)

Use this feature of the Intel® oneAPI DPC++/C++ Compiler when you want to split your FPGA compilation into different FPGA images. This feature is particularly useful when your design does not fit on a single FPGA. You can use it to split your very large design into multiple smaller images, which you can use to partially reconfigure your FPGA device.

You can split your design using one of the following approaches, each giving you different benefits:

- Dynamic Linking Flow
- Dynamic Loading Flow

Between the two flows, dynamic linking is easier to implement than dynamic loading. However, dynamic linking can require more memory on the host device as all of the device images must be loaded into memory. Dynamic loading addresses these limitations but introduces the need for some extra source-level changes. The following comparison table highlights the differences between the flows:

Dynamic Linking vs. Dynamic Loading Flow

	Dynamic Linking	Dynamic Loading
Can dynamically change FPGA Image at runtime?	Yes	Yes
Defining the type and number of FPGA images	At compile time	At runtime
Host-program memory footprint	All FPGA images are stored in memory at runtime.	Only explicitly loaded FPGA images are stored in memory.
Calling host code	Call function in the dynamic library directly.	Explicitly load the dynamic library and functions to call.

Dynamic Linking Flow

This flow allows you to split your design into different source files and map them into a separate FPGA image. Intel® recommends this flow for designs with a small number of FPGA images.

To use this flow, perform the following steps:

1. Split your source code such that for each FPGA image you want, you create a separate `.cpp` file that submits various kernels. Separate the host code into one or more `.cpp` files that can then interface with functions in the kernel files.

Consider that you now have the following three files:

- `main.cpp` containing your host code. For example:

```
// main.cpp
int main() {
    queue queueA;
    add(queueA);
    mul(queueA);
}
```

- `vector_add.cpp` containing a function that submits the `vector_add` kernel. For example:

```
// vector_add.cpp
extern "C"{
    void add(queue queueA) {
        queue.submit(
            // Kernel Code
        );
    }
}
```

- `vector_mul.cpp` containing a function that submits the `vector_mul` kernel. For example:

```
// vector_mul.cpp
extern "C"{
    void mul(queue queueA) {
        queue.submit(
            // Kernel Code
        );
    }
}
```

2. Compile the source files using the following commands:

```
icpx -fPIC -fintelfpga -c vector_add.cpp -o vector_add.o
icpx -fPIC -fintelfpga -c vector_mul.cpp -o vector_mul.o

// FPGA image compiles take a long time to complete
icpx -fPIC -shared -fintelfpga vector_add.o -o vector_add.so \
    -Xshardware -Xstarget=<bsp:board_variant>
icpx -fPIC -shared -fintelfpga vector_mul.o -o vector_mul.so \
    -Xshardware -Xstarget=<bsp:board_variant>

// Final link step
icpx -o main.exe main.cpp vector_add.so vector_mul.so
```

With this flow, the long FPGA compile steps are split into separate commands that you can potentially run on different systems or only when you change the files.

Dynamic Loading Flow

Use this flow to avoid loading all of the different FPGA images into memory at once. Similar to dynamic linking flow, this flow also requires you to split your code. However, for this flow, you must load the `.so` (shared object) files in the host program. The advantage of this flow is that you can load large FPGA image files dynamically as necessary instead of linking all image files at compile time.

To use this flow, perform the following steps:

1. Split your source code in the same manner as done in [step 1 of the dynamic linking flow](#).

2. Modify the main.cpp file to appear as follows:

```
// main.cpp
#include <dlfcn.h>

int main() {
    queue queueA;
    bool runAdd, runMul;
    // Assuming runAdd and runMul are set dynamically at runtime
    if (runAdd) {
        auto add_lib = dlopen("./vector_add.so", RTLD_NOW);
        auto add = (void (*)(queue))dlsym(add_lib, "add");
        add(queueA);
    }
    if (runMul) {
        auto mul_lib = dlopen("./vector_mul.so", RTLD_NOW);
        auto mul = (void (*)(queue))dlsym(mul_lib, "mul");
        mul(queueA);
    }
}
```

3. Compile the source files using the following commands:

NOTE You do not have to link the .so files at compile time because they are loaded dynamically at runtime.

```
icpx -fPIC -fintel FPGA -c vector_add.cpp -o vector_add.o
icpx -fPIC -fintel FPGA -c vector_mul.cpp -o vector_mul.o

// FPGA Image compiles take a long time to complete
icpx -fPIC -shared -fintel FPGA vector_add.o -o vector_add.so \
    -Xshardware -Xstarget=<bsp:board_variant>
icpx -fPIC -shared -fintel FPGA vector_mul.o -o vector_mul.so \
    -Xshardware -Xstarget=<bsp:board_variant>

icpx -o main.exe main.cpp
// Before running the design, add the path containing the .so files to LD_LIBRARY_PATH
// e.g., export LD_LIBRARY_PATH=./:$LD_LIBRARY_PATH
```

With this approach, you can arbitrarily load many .so files at runtime. This is useful when you have a large library of FPGA images, and you want to select a subset of files from it.

FPGA Optimization Flags, Attributes, Pragmas, and Extensions

20

This chapter describes a list of compiler optimization flags, attributes, pragmas, and extensions that allow you to customize the kernel compilation process.

Optimization Flags

This section describes the FPGA optimization flags supported by the Intel® oneAPI DPC++/C++ Compiler. Refer to the sub-topics for detailed information about the supported flags.

Specify Schedule f_{MAX} Target for Kernels (-Xsclock=<clock target>)

The schedule f_{MAX} target determines the pipelining effort the scheduler attempts during the scheduling process.

You can direct the Intel® oneAPI DPC++/C++ Compiler to globally compile all kernels with `-Xsclock=<clock target>` in Hz/KHz/MHz/GHz or s/ms/us/ns/ps> option in the `icpx` command.

Example

```
icpx -fintel-fpga -Xshardware -Xsclock=375MHz source_file.cpp
```

NOTE

The schedule target f_{MAX} determines the pipelining effort during compilation. Compile to hardware to get the actual f_{MAX} value.

Create a 2xclock Interface (-Xsuse-2xclock)

The Intel® oneAPI DPC++/C++ Compiler explicitly creates a 2xclock interface for the given design with the `-Xsuse-2xclock` compiler option.

Example

```
icpx -fintel-fpga -Xshardware -Xsuse-2xclock source_file.cpp
```

However, consider the following scenarios:

- If your board specification specifies a 2xclock value, the `-Xsuse-2xclock` has no effect.
- If your design uses the `[[intel::doublepump]]` memory attribute without the `-Xsuse-2xclock` option, the compiler still creates a 2xclock interface but emits a compiler warning.
- If you are compiling your design for the FPGA acceleration flow, BSPs always have a 2xclock value.
- If you are compiling your design for SYCL* HLS flow, specify the `-Xsuse-2xclock` option to include a 2xclock value in your design.

Disable Burst-Interleaving of Global Memory (`-Xsno-interleaving`)

The Intel® oneAPI DPC++/C++ Compiler cannot [burst-interleave global memory](#) across different memory types. You can disable burst-interleaving for all global memory banks of the same type and manage them manually by including the `-Xsno-interleaving=<global_memory_name>` option in your compiler command.

Manual partitioning of memory buffers overrides the default burst-interleaved configuration of global memory.

Caution

The `-Xsno-interleaving` option requires a global memory type parameter. If you do not specify a memory type, the Intel® oneAPI DPC++/C++ Compiler issues an error message.

- To direct the Intel® oneAPI DPC++/C++ Compiler to disable burst-interleaving for the default global memory, invoke the following command:

```
icpx -fintelfpga -Xshardware -Xsno-interleaving=default source_file.cpp
```

- Your accelerator board might include multiple global memory types. To identify the default global memory type, refer to your board vendor's documentation for your custom platform or the `board_spec.xml` file. The `board_spec.xml` file includes an entry for the global memory. If there is only one memory, that is the default. If there is more than one memory, the `board_spec.xml` file specifies the default. You can identify the global memory name with the `name` attribute of the `<global_mem>` XML element within the `board_spec.xml` file.
- For a heterogeneous memory system, to direct the Intel® oneAPI DPC++/C++ Compiler to disable burst-interleaving of a specific global memory type, perform the following tasks:
 - Consult the `board_spec.xml` file of your Custom Platform for the names of the available global memory types. For example, double data rate (DDR) and quad data rate (QDR).
 - To disable burst-interleaving for one of the memory types (for example, DDR), invoke:

```
icpx -fintelfpga -Xshardware -Xsno-interleaving=DDR source_file.cpp
```

The Intel® oneAPI DPC++/C++ Compiler enables manual partitioning for the DDR memory bank and configures the other memory bank in a burst-interleaved fashion.

- To disable burst-interleaving for more than one type of global memory buffers, include a `-Xsno-interleaving=<global_memory_name>` option for each global memory type. For example, to disable burst-interleaving for both DDR and QDR, invoke the following command:

```
icpx -fintelfpga -Xshardware -Xsno-interleaving=DDR -Xsno-interleaving=QDR source_file.cpp
```

Caution

Do not pass a buffer as a kernel argument that associates it with multiple memory technologies.

Force Ring Interconnect for Global Memory (`-Xsglobal-ring`)

The Intel® oneAPI DPC++/C++ Compiler attempts to choose an optimal [global memory interconnect topology](#) based on various characteristics of the design.

To override the compiler's choice and force a ring topology, use the `-Xsglobal-ring` option in your `icpx` command. This can improve your kernel f_{MAX} . In particular, designs that target board support packages with four or more banks of global memory may see an f_{MAX} benefit from this option.

Example

```
icpx -fintelfpga -Xshardware -Xsglobal-ring <source_file>.cpp
```

Force a Single Store Ring to Reduce Area (`-Xsforce-single-store-ring`)

Force a Single Store Ring to Reduce Area (-Xsforce-single-store-ring)

When the Intel® oneAPI DPC++/C++ Compiler implements a ring topology for the global memory interconnect (either by automatic choice or by forcing the ring through `-Xsglobal-ring`), it widens the interconnect by default to allow more writes to occur in parallel. This allows for saturation of the global memory throughput using write-only traffic.

The `-Xsforce-single-store-ring` option allows you to save area if you do not require that much write bandwidth. To narrow the interconnect in order to save area while limiting write-only throughput to one bank's worth, use the `-Xsforce-single-store-ring` option in your `icpx` command.

Example

```
icpx -fintelfpga -Xshardware -Xsforce-single-store-ring <source_file>.cpp
```

NOTE

You can use `-Xsforce-single-store-ring` and `-Xsglobal-ring` command options independently. A combination of their use is also functional.

There is here is some dependence between the two options. For example, using the `-Xsforce-single-store-ring` option has no effect if the compiler does not use the ring interconnect. So, if you want a ring interconnect and want to force a single-store ring, use the `-Xsglobal-ring` option to force a ring interconnect. Then, use the `-Xsforce-single-store-ring` option to make that ring a single-store ring.

Force Ring Interconnect for Global Memory (-Xsglobal-ring)

Force Fewer Read Data Reorder Units to Reduce Area (-Xsnum-reorder)

When the Intel® oneAPI DPC++/C++ Compiler implements a ring topology for the global memory interconnect (either by automatic choice or by forcing the ring through `-Xsglobal-ring`), it widens the interconnect by default to allow more reads to occur in parallel. This allows for saturation of the global memory throughput using read-only traffic.

To narrow the interconnect in order to save area while reducing read-only throughput, use the `-Xsnum-reorder=N` option in your `icpx` command, where `N` is the number of bank's worth of read bandwidth you desire. For example, if on a two-bank BSP, you require only one bank's worth of read bandwidth, set `-Xsnum-reorder=1`.

Example

```
icpx -fintelfpga -Xshardware -Xsnum-reorder=1 <source_file>.cpp
```

Disable Hardware Kernel Invocation Queue (-Xsno-hardware-kernel-invocation-queue)

To direct the Intel® oneAPI DPC++/C++ Compiler to reduce kernel area use by removing kernel invocation queue in SYCL* kernel, include the `-Xsno-hardware-kernel-invocation-queue` option in your `icpx` command.

Example

```
icpx -fintelfpga -Xshardware -Xsno-hardware-kernel-invocation-queue <source_file>.cpp
```

Caution

Using this option may result in longer kernel execution time as the kernel invocation queue allows SYCL runtime environment to queue kernel launches in accelerator so that the accelerator can start execution on the next invocation as soon as previous invocation of the same kernel is complete.

NOTE

Use the `-Xsno-hardware-kernel-invocation-queue` option only when your kernel execution time is much greater than the system and SYCL runtime environment overhead hidden by the kernel invocation queue (20-100us), or if the Intel® oneAPI DPC++/C++ Compiler has difficulty fitting your kernel.

Refer to [Utilizing Hardware Kernel Invocation Queue](#) for more information about how to utilize the kernel invocation queue.

Modify the Handshaking Protocol Between Clusters (-Xshyper-optimized-handshaking)

To modify the [handshaking protocol between clusters](#), use the `-Xshyper-optimized-handshaking=<auto|off|on>` option in your `icpx` command. The `-Xshyper-optimized-handshaking` option can be set to one of the following values:

- **auto**: The Intel® oneAPI DPC++/C++ Compiler chooses whether to enable or disable the optimization. This is the default behavior for all flows except the [minimum latency optimization flow](#).
- **on**: The compiler enables the optimization if possible. Use this value to achieve a higher f_{MAX} . When you enable the optimization, the compiler adds pipeline registers to the handshaking paths of the [stallable nodes](#). As a result, you observe higher f_{MAX} at the cost of increased area and latency.
- **off**: The compiler attempts to optimize for lower latency at the potential cost of lower f_{MAX} . Disabling hyper-optimized handshaking might also decrease area. This is useful for smaller designs where you are willing to give up f_{MAX} for lower latency and area. This is the default for the [minimum latency optimization flow](#).

Examples

```
icpx -fintelfpga -Xshardware -Xshyper-optimized-handshaking=auto <source_file>.cpp
```

```
icpx -fintelfpga -Xshardware -Xshyper-optimized-handshaking=off <source_file>.cpp
```

```
icpx -fintelfpga -Xshardware -Xshyper-optimized-handshaking=on <source_file>.cpp
```

NOTE

The `-Xshyper-optimized-handshaking` option applies only to designs targeting Agilex™ 7 and Stratix® 10 devices. If you use this option on other target devices, the compiler fails and produces an error. This option applies only when running the report or hardware flow.

[Clustering the Datapath](#)
[Handshaking Between Clusters](#)
[Minimum Latency Flow](#)

Disable Automatic Fusion of Loops (-Xsdisable-auto-loop-fusion)

Include the `-Xsdisable-auto-loop-fusion` flag in your `icpx` command to direct the Intel® oneAPI DPC++/C++ Compiler to disable automatic loop fusion when compiling your design.

Example

```
icpx -fintelfpga -Xshardware -Xsdisable-auto-loop-fusion <source_file>.cpp
```

For more information about automatic loop fusion, refer to [Automatic Loop Fusion](#)

Fuse Adjacent Loops With Unequal Trip Counts (-Xsenable-unequal-tc-fusion)

Use the `-Xsenable-unequal-tc-fusion` flag in your `icpx` command to direct the Intel® oneAPI DPC++/C++ Compiler to fuse adjacent loops with different trip counts into a single loop without affecting either loop's functionality.

Example

```
icpx -fintel-fpga -Xshardware -Xsenable-unequal-tc-fusion <source_file>.cpp
```

For more information about fusing loops, refer to [Fuse Loops to Reduce Overhead and Improve Performance](#).

Pipeline Loops in Non-task Kernels (-Xsauto-pipeline)

To direct the Intel® oneAPI DPC++/C++ Compiler to compile your design and pipeline loops in non-task (`parallel_for`) kernels, include the `-Xsauto-pipeline` option in your `icpx` command. The host program invokes non-task kernels through the kernel execution function `parallel_for`, `parallel_for_work_item`, or `parallel_for_work_group`.

Example

```
icpx -fintel-fpga -Xshardware -Xsauto-pipeline <source_file>.cpp
```

With the `-Xsauto-pipeline` option, the compiler attempts to pipeline the loops in your design, but the pipelining is not guaranteed. If you do not include the `-Xsauto-pipeline` option, the compiler does not pipeline the loops in `parallel_for` kernels. However, it executes different work items in parallel.

NOTE

The `-Xsauto-pipeline` option might improve or degrade performance depending on the memory access pattern in your design.

- If the auto-pipelining is successful, the [Loop Analysis](#) report displays the message Auto-pipelined `parallel_for` and `parallel_for` rewritten as a pipelined `single_task` (Details pane). The compiler-generated loops appear marked as Compiler generated auto-pipeline loop in the report.
- If the compiler chooses not to auto-pipeline the loops, the [Loop Analysis](#) report displays a message for the kernel. The reasons for not auto-pipelining a loop can be one of the following:
 - A barrier in the function is not at the top-level function scope.
 - Kernel uses a local or private memory.
 - Kernel uses a volatile or atomic memory, or channels.

Tip

If you do not want the compiler to pipeline some infrequently used loops while allowing other loops to be auto-pipelined, use the `[[intel::disable_loop_pipelining]]` loop directive on specific loops when using the `-Xsauto-pipeline` option. This loop directive disables the loop pipelining.

Control Semantics of Floating-Point Operations (-fp-model=<value>)

Include the `-fp-model=<value>` option in your `icpx` command to direct the Intel® oneAPI DPC++/C++ Compiler to control the semantics of floating-point operations.

NOTE

You can also use the [Floating-Point Pragmas](#) to influence the floating-point operations.

Example

```
icpx -fintel-fpga -Xshardware -fp-model=<value> <source_file>.cpp
```

where, the *<value>* can be one of the following values:

Value	Description
fast [=1 2]	Default value. Enables more aggressive optimizations on floating-point operations.
NOTE There is currently no difference between <code>fast=1</code> and <code>fast=2</code> .	
precise	Disables optimizations that are not value-safe on floating-point data.

For additional information about this flag, refer to `fp-model`, `fp` topic in the *Intel® oneAPI DPC++/C++ Compiler Developer Guide and Reference*.

Modify the Rounding Mode of Floating-point Operations (-Xsrounding=<rounding_type>)

To modify the rounding mode of floating-point elementary operations in your design, use the `-Xsrounding=<rounding_type>` option in your `icpx` command. You can set the `-Xsrounding` option to one of the following values:

- ieee**: All elementary operations (+, -, *, /) of both single-precision and double-precision floating-point use IEEE-754 round nearest ties to even (RNE) mode, which has a 0.5 unit of least precision (ULP) error at most.
- faithful**: All elementary operations of double-precision floating-point and multiplication and division of single-precision floating-point use faithful rounding mode, which has a 1 ULP error at most. This rounding mode leads to more efficient hardware at the expense of numerical variation in results. Addition and subtraction of single-precision floating-point still have to use IEEE-754 RNE rounding mode.

The following tables summarize the rounding modes:

Single-precision Floating-point

	Addition	Subtraction	Multiplication	Division
Default	IEEE-754 RNE	IEEE-754 RNE	IEEE-754 RNE	Faithful
- <code>Xsrounding=ieee</code>	IEEE-754 RNE	IEEE-754 RNE	IEEE-754 RNE	IEEE-754 RNE
- <code>Xsrounding=faitful</code>	IEEE-754 RNE	IEEE-754 RNE	Faithful	Faithful

Double-precision Floating-point

	Addition	Subtraction	Multiplication	Division
Default	IEEE-754 RNE	IEEE-754 RNE	IEEE-754 RNE	IEEE-754 RNE
- <code>Xsrounding=ieee</code>	IEEE-754 RNE	IEEE-754 RNE	IEEE-754 RNE	IEEE-754 RNE

	Addition	Subtraction	Multiplication	Division
-	Faithful	Faithful	Faithful	Faithful
Xsrounding=fait hful				

Examples

```
icpx -fintel-fpga -Xshardware -Xsrounding=ieee <source_file>.cpp
```

```
icpx -fintel-fpga -Xshardware -Xsrounding=faithful <source_file>.cpp
```

Global Control of Exit FIFO Latency of Stall-free Clusters (-Xssfc-exit-fifo-type=<value>)

Use the `-Xssfc-exit-fifo-type=<value>` flag in the `icpx` command to direct the Intel® oneAPI DPC++/C++ Compiler to globally compile all stall-free clusters in kernels with a specified exit FIFO type. This flag supports the following arguments:

- **default:** Infers the mid-speed FIFO (implemented with MLABs or M20Ks) for a minimum latency of three cycles.
- **zero-latency:** Combinational path around the default FIFO for a minimum latency of zero cycles.
- **low-latency:** Registered path around the default FIFO for a minimum latency of one cycle.

Caution

Depending on the specified exit FIFO type and resulting hardware implementation, f_{MAX} or FPGA area use might be affected negatively.

Example

```
icpx -fintel-fpga -Xshardware -Xssfc-exit-fifo-type=zero-latency <source_file>.cpp
```

Clustering the Datapath**Enable the Read-Only Cache for Read-Only Accessors (-Xsread-only-cache-size=<N>)**

If your kernel accesses a read-only accessor that is guaranteed not to alias with other accessors and USM pointers, consider enabling the read-only cache using the `-Xsread-only-cache-size=<N>` flag in your `icpx` command. You should use a read-only cache for high-bandwidth table lookups that is constant throughout the kernel execution. The read-only cache is optimized for high cache-hit performance.

Example

```
icpx -fintel-fpga -Xshardware -Xsread-only-cache-size=<N><source_file>.cpp
```

The compiler implements the read-only cache using on-chip memory blocks and privatizes it per kernel. Each kernel receives a version of the cache that serves all reads in the kernel from read-only no-alias accessors. The compiler replicates each private cache as many times as necessary to expose extra read ports. The size of each replicate is at least `<N>` bytes as specified by the `-Xsread-only-cache-size=<N>` flag.

NOTE

- Unlike global memory accesses that have extra hardware for tolerating long memory latencies, the read-only cache suffers significant performance penalties for cache misses. If the buffer being accessed in your kernel code cannot fit in the cache, you might achieve better performance without enabling the cache. The cached data is discarded (invalidated) from the read-only cache every time the kernel is launched.
- Currently, omitting the read-only cache for only a subset of your read-only accessors in your design is unsupported. If your design has multiple read-only no-alias accessors, you can either enable caching for all of them using the global `-Xsread-only-cache-size=<N>` flag or disable caching for all of them by removing the flag.

Consider the following example code snippet:

```
q.submit([&](handler &h) {
    accessor sqrt_lut(sqrt_lut_buf, h, read_only,
                     ext::oneapi::accessor_property_list{no_alias});
    accessor indices(indices_buf, h, read_write,
                     ext::oneapi::accessor_property_list{no_alias, no_init});
    accessor output(output_buf, h, write_only,
                     ext::oneapi::accessor_property_list{no_alias, no_init});

    h.single_task<class Test>([=]() {
        for (int i = 0; i < kNumInputs; ++i) {
            output[i] = sqrt_lut[indices[i]];
        }
    });
});
```

Compile the above code using the following command:

```
icpx -fintelpga -Xshardware -Xsread-only-cache-size=2048 <source_file>.cpp
```

The compiler creates a read-only cache of size 2048 bytes that serves the single read from `sqrt_lut`. If the cache is sized correctly to match the size of `sqrt_lut_buf`, then the cache improves the design throughput, especially because the read accesses are random.

Control Hardware Implementation of the Supported Data Types and Math Operations (-Xsdsp-mode=<option>)

The Intel® oneAPI DPC++/C++ Compiler allows you to control (at global and local scope) whether the implementation of the [supported data type and math functions](#) uses [DSPs](#) or soft logic using [ALMs](#). You can find this implementation in the Details pane of the corresponding math instruction nodes in the [System Viewer](#) of the FPGA optimization report (`report.html`).

Global-scope Control

Include the `-Xsdsp-mode=[default|prefer-dsp|prefer-softlogic]` flag in your `icpx` command to control the hardware implementation of the supported data types and math operations of all kernels in your source code.

Example

```
icpx -fintelpga -Xshardware -Xsdsp-mode=<option> <source_file>.cpp
```

where the `<option>` is one of the following values:

Option	Description
default	Default option if you do not pass this command-line flag manually. The compiler determines the implementation based on the data type and math operation.
prefer-dsp	Prefer math operations to be implemented in DSPs. If a math operation is implemented by DSPs by default, you see no difference in resource utilization or area. Otherwise, you will notice a decrease in the use of soft-logic resources and an increase in the use of DSPs.
prefer-softlogic	Prefer math operations to be implemented in soft-logic. If a math operation is implemented without DSPs by default, you see no difference in resource utilization or area. Otherwise, you will notice a decrease in the use of DSPs and an increase in the use of soft-logic resources.

Local-scope Control

You can control DSP usage of the supported math operations on a local scope by using the library function `sycl::ext::intel::math_dsp_control(<Preference::<enum>, Propagate::<enum>>)` defined in the `fpga_dsp_control.hpp` header file, which is included in the `fpga_extensions.hpp` header file. This library function provides control at the block level within a single kernel. A reference-capturing lambda expression is passed as the argument to this library function. Inside the lambda expression, the implementation preference of math operations that support DSP control is determined by two template arguments.

The `<Preference::<enum>>` argument takes up an `enum` data type with one of the following values:

Value	Description
<code>sycl::ext::intel::Preference::DSP</code>	Prefer math operations to be implemented in DSPs. Its behavior on a math operation is equivalent to the global control <code>-Xsdsp-mode=prefer-dsp</code> . NOTE The <code>sycl::ext::intel::Preference::DSP</code> option is automatically applied if you do not specify the template argument <code>Preference</code> manually.
<code>sycl::ext::intel::Preference::Softlogic</code>	Prefer math operations to be implemented in soft logic. Its behavior on a math operation is equivalent to the global control <code>-Xsdsp-mode=prefer-softlogic</code> .
<code>sycl::ext::intel::Preference::Compiler_default</code>	Compiler determines the implementation based on the data type and math operation. Its behavior on a math operation is equivalent to the global control <code>-Xsdsp-mode=default</code> .

NOTE

Local control overrides global control on a controlled math operation. For example:

```
// icpx -Xsdsp-mode=prefer-softlogic ...
using namespace sycl;
cgh.single_task<class LocalControl>([=]() {
    out_acc[0] = in_acc[0] + in_acc[1]; // Addition implemented in soft-logic.
    ext::intel::math_dsp_control<ext::intel::Preference::DSP>([&] {
        out_acc[1] = in_acc[0] + in_acc[1]; // Addition implemented in DSP.
    });
});
```

The `<Propagate::<enum>>` argument is a boolean value that determines the propagation of the `Preference::<enum>` value to function calls in the lambda expression as follows:

Value	Description
<code>sycl::ext::intel::Propagate::On</code>	DSP control recursively applies to controllable math operations in all function calls in the lambda expression function. This value is the default and it is applied if you do not specify the argument <code><Propagate::<enum>></code> .
<code>sycl::ext::intel::Propagate::Off</code>	DSP control applies only to the controllable math operations directly inside the lambda expression. Math operations in function calls inside the lambda expression are not affected by this DSP control.

Example

```
sycl::ext::intel::math_dsp_control<sycl::ext::intel::Preference::Softlogic,
                                sycl::ext::intel::Propagate::On>([&]{
    a += 1.23f;
});
```

NOTE

A nested `math_dsp_control<>()` call is controlled only by its own `Preference` argument. The `Preference` of the parent `math_dsp_control<>()` function does not affect the nested `math_dsp_control<>()` function, even if the parent has the argument `Propagate::On`. For example:

```
using namespace sycl;
cgh.single_task<class LocalControl>([=]() {
    ext::intel::math_dsp_control<ext::intel::Preference::DSP,
                                ext::intel::Propagate::On>([&] {
        out_acc[0] = in_acc[0] + in_acc[1]; // Addition implemented in DSP.
        ext::intel::math_dsp_control<ext::intel::Preference::Softlogic>([&] {
            // Addition implemented in soft-logic. Preference::DSP on the parent
            // math_dsp_control<>() call does not affect this math_dsp_control<>().
            out_acc[1] = in_acc[0] + in_acc[1];
        });
    });
});
```

Supported Data Type and Math Operations

Data types	Math Operations
<code>float</code>	Addition, subtraction, multiplication
<code>ap_float<8, 23></code>	Addition, subtraction, multiplication
<code>int</code>	Multiplication
<code>ac_int</code>	Multiplication
<code>ac_fixed</code>	Multiplication

NOTE

For additional information, refer to the FPGA tutorial sample "DSP Control" on [GitHub](#).

Generate Register Map Wrapper (-Xsregister-map-wrapper-type)

Attention Only the SYCL* HLS flow supports this compiler option.

The Intel® oneAPI DPC++/C++ Compiler generates a ring-like wrapper structure to connect all register map interfaces for different kernels inside an RTL IP core. You can direct the compiler to generate different types of the wrapper by including the `-Xsregister-map-wrapper-type=<default|high-fmax|low-latency>` option in the `icpx` command, as shown in the following example:

Example

```
icpx -fintelfpga -Xshardware -Xsregister-map-wrapper-type=<default|high-fmax|low-latency>
source_file.cpp
```

Where:

- **-Xsregister-map-wrapper-type=high-fmax:** The ring wrapper contains pipeline stages to curtail it from being the $f_{\max}$ bottleneck of the IP core. The number of pipeline stages varies and depends on the number of kernels in the IP core.
- **-Xsregister-map-wrapper-type=low-latency:** The ring wrapper contains combinational logic only and does not introduce extra latency for the Avalon® Memory-Mapped signals between the IP core boundary and the kernel.
- **-Xsregister-map-wrapper-type=default:** When you set it to default or omit this compiler option, the compiler automatically infers the ring wrapper type. This compiler option does not change the signals on the register map interfaces in any manner.

Caution

- If you attempt to use this option in the full-system flow, the compiler issues a warning and ignores the option. The compiler still generates the ring wrapper, but the wrapper type used in the full-system flow may differ from the default wrapper type used in the SYCL* HLS flow.
 - If any kernel in the IP core contains streaming invocation interfaces and register map kernel arguments, and you specify the `high-fmax` version of the register map wrapper, the compiler returns an error message indicating this combination is not supported.
-

Allow Wide Memory Initialization (-Xsallow-wide-device-globals)

By default, the Intel® oneAPI DPC++/C++ Compiler uses a conservative memory width (maximum 1024 bits) when compiling your kernel to prevent possible simulation errors.

Direct the compiler to skip these conservative choices and use a more optimal internal word width with the `-Xsallow-wide-device-globals` option of the `icpx` command as shown in the following example:

Example

```
icpx -fintelfpga -Xsallow-wide-device-globals source_file.cpp
```

Important Due to a known simulation issue with the Embedded Memory IP cores provided by Quartus® Prime software, using the `-Xsallow-wide-device-globals` option might generate functional failures in simulation that do not appear when compiling to hardware.

Optimization Targets

This section describes the FPGA optimization targets the Intel® oneAPI DPC++/C++ Compiler supports. It covers multiple FPGA performance metrics, such as latency, throughput, and area. The current release supports minimum latency and maximum throughput without area optimization heuristics optimization targets. In future oneAPI releases, Intel plans to add support for more optimization targets.

NOTE

The manual controls described in the optimization targets that follow are beneficial in overriding one or more of the underlying controls without affecting other underlying controls that the `-Xsoptimize=<target>` compiler flag implies.

Tip

For additional information, refer to the FPGA tutorial sample “Optimization Targets” on [GitHub](#).

Minimum Latency Flow

The minimum latency flow attempts to minimize your kernel latency at the cost of decreased f_{MAX} . Use this flow to optimize latency-sensitive designs.

To compile your design with the minimum latency flow, pass the `-Xsoptimize=latency` flag to the `icpx` command, as shown in the following example:

```
icpx -fintel FPGA -Xshardware -Xsoptimize=latency <source_file>.cpp
```

The following table lists the underlying controls that are enabled by the minimum latency flow, as well as their equivalent user controls. You can use these same user controls to manually override the underlying controls:

Description	Equivalent User Control
Disable hyper-optimized handshaking on Agilex® 7 and Stratix® 10 devices	<code>-Xshyper-optimized-handshaking=off</code>
Use zero-latency stall-free clusters exit FIFO	<code>-Xssfc-exit-fifo-type=zero-latency</code>
Disable loop speculation	<code>[[intel::speculated_iterations(0)]]</code> <code>[[intel::max_reinvocation_delay(1)]]</code>
Set reinvocation delay on all loops to 1 thereby allowing new loop invocations to begin immediately after a previous loop invocation has completed	<code>[[intel::max_reinvocation_delay(1)]]</code>

Minimum Area Flow

The minimum area flow attempts to minimize your kernel area at the cost of decreased f_{MAX} . Use this flow to optimize area-sensitive designs.

To compile your design with the minimum area flow, pass the `-Xsoptimize=area` flag to the `icpx` command, as shown in the following example:

```
icpx -fintel FPGA -Xshardware -Xsoptimize=area <source_file>.cpp
```

The following table lists underlying controls enabled by the minimum area flow, as well as their equivalent user controls. You can use these same user control to manually override the underlying controls:

Description	Equivalent User Control
Disable hyper-optimized handshaking on Agilex [®] 7 and Stratix [®] 10 devices	-Xshyper-optimized-handshaking=off
Do not create replicates	[[intel::max_replicates(1)]]
Do not create private copies	[[intel::private_copies(1)]]
Set RAM stitching to use minimum number of RAMs	[[intel::force_pow2_depth(0)]]

Tip Consider these other best practices for achieving low area:

- Use [stall-enable clusters](#) if you see stall-free clusters with larger exit FIFOs in the reports.
- [Disable loop pipelining](#) if loop-carried dependencies result in an initiation interval (II) that is equal or close to the latency of a given iteration.
- [Recompile with a lower \$f_{\text{MAX}}\$ target](#) if the achieved f_{MAX} was lower than the default family f_{MAX} or your originally targeted f_{MAX} .
- [Disable interleaving with \[\[intel::max_interleaving\(1\)\]\]](#) if the loop has an II greater than 1.
- [Narrow the width of data by using variable-precision data types](#).

Balanced Throughput-Area Trade-Offs Flow

This flow attempts to balance throughput-area trade-offs. Specifically, the compiler might disable throughput-area trade-off heuristics that increase the throughput at the cost of area in this flow.

To compile your design with the maximum throughput without area optimization heuristics flow, pass the `-Xsoptimize=throughput-area-balanced` flag to the `icpx` command, as shown in the following example:

```
icpx -fintel fpga -Xshardware -Xsoptimize=throughput-area-balanced <source_file>.cpp
```

The following table lists the underlying controls that are enabled by the minimum latency flow, as well as their equivalent user controls. You can use these same user controls to manually override the underlying controls:

Description	Equivalent User Control
Do not create banks	[[intel::numbanks(1)]]
Do not create replicates (Create only one bank per memory)	[[intel::max_replicates(1)]]
Do not create private copies	[[intel::private_copies(1)]]

Kernel Variables

This section describes mechanisms available to manipulate datapath for kernel variables.

fpga_reg

The Intel[®] oneAPI DPC++/C++ Compiler provides FPGA extension `fpga_reg()` that you can include in your kernel code.

The `fpga_reg()` function directs the compiler to insert at least one register between the operand and the return value of the function call. In general, it is not necessary to include the `fpga_reg()` function in your kernel code to achieve desired performance.

NOTE

Intel® strongly recommends that you use the `fpga_reg()` function only if you are experienced in using the Quartus® Prime Pro Edition software performing advanced optimization for a specific target device. You must have sufficient knowledge about the placement of portions of the datapath on an FPGA device.

Syntax

```
T fpga_reg(T op)
```

Where, `T` may be any sized type, such as standard SYCL* device data types, or a user-defined `struct` containing SYCL types.

Example

Consider the following example:

```
#include <sycl/ext/intel/fpga_extensions.hpp>
...
r[k] = ext::intel::fpga_reg(a[k]) + b[k];
...
```

Use the `fpga_reg()` function to perform the following:

- Break critical paths between spatially distant portions of a datapath, such as between processing elements of a large systolic array.
- Reduce the pressure on placement and routing efforts caused by spatially distinct portions of the kernel implementation.

The `fpga_reg()` function directs the compiler to insert at least one hardware pipelining register on the signal path that assigns the operand to the return value. This built-in function operates as an assignment in the SYCL programming language where the operand is assigned to the return value. The assignment has no implicit semantic or functional meaning beyond a standard-C assignment. Functionally, you can consider the `fpga_reg()` function being always optimized away by the compiler.

Tip

For additional information, refer to the FPGA tutorial sample "FPGA register" on [GitHub](#).

NOTE

The compiler does not provide feedback on where you should insert the `fpga_reg()` function calls in your code. Use the Quartus® Prime Pro Edition software to determine where you should insert the calls to address specific aspects of performance.

You may introduce nested `fpga_reg()` function calls in your kernel code to increase the minimum number of registers that the compiler inserts on the assignment path. Because each function call guarantees the insertion of at least one register stage, the number of calls provides a lower limit on the number of registers.

Kernel Attributes

This section describes the FPGA kernel attributes supported by the Intel® oneAPI DPC++/C++ Compiler. Refer to the sub-topics for detailed information about the supported attributes.

Specify Schedule f_{MAX} Target for Kernels (`scheduler_target_fmax_mhz`)

The schedule f_{max} target determines the pipelining effort the scheduler attempts during the scheduling process.

You can use one or both of the following options to specify the kernel specific f_{max} target:

- By using the `[[intel::scheduler_target_fmax_mhz(N)]]` source-level attribute.
- By directing the Intel® oneAPI DPC++/C++ Compiler to globally compile all kernels `-Xsclock=<clock target in Hz/KHz/MHz/GHz or s/ms/us/ns/ps>` option in the `icpx` command.

If you use both the command-line option and source-level attribute, the kernel attribute takes the priority. Consider the following example:

```
cgh.single_task<class mykernel1>([=] {
    ...
});

cgh.single_task<class mykernel2>([=
]() [[intel::scheduler_target_fmax_mhz(200)]] {
    ...
});
```

In you direct the compiler to compile the above code with `-Xsclock=300MHz` in the `icpx` command, the compiler schedules kernel `mykernel1` at 300 MHz and kernel `mykernel2` at 200 MHz.

The schedule target f_{max} determines the pipelining effort during compilation. Refer to the [Quartus compilation summary](#) in the `report.html` file to get the actual f_{max} value.

Specify a Workgroup Size (`max_work_group_size/reqd_work_group_size`)

Specify a maximum or the required [workgroup](#) size whenever possible. The Intel® oneAPI DPC++/C++ Compiler relies on this specification to optimize hardware use of the SYCL* kernel without involving excess logic.

- If you do not specify the `[[intel::max_work_group_size(Z, Y, X)]]` or `[[sycl::reqd_work_group_size(Z, Y, X)]]` attribute in your kernel, the workgroup size assumes a default value depending on compilation time and runtime constraints.
- If your kernel contains a barrier, the Intel® oneAPI DPC++/C++ Compiler sets a default maximum scalarized work-group size of 128 work-items.
- If your kernel does not query any SYCL intrinsic that allow different threads to behave differently (that is, local or global thread IDs, or work-group ID), the Intel® oneAPI DPC++/C++ Compiler infers a single-threaded execution mode and sets the maximum work-group size to (1, 1, 1). In this case, the SYCL runtime also enforces a global enqueue size of (1, 1, 1), and loop pipelining optimizations are enabled within the Intel® oneAPI DPC++/C++ Compiler.

Deprecation Notice

The `[[cl::reqd_work_group_size(Z, Y, X)]]` attribute is deprecated. Use the `[[sycl::reqd_work_group_size(Z, Y, X)]]` attribute.

To specify the work-group size, modify your kernel code in the following manner:

- To specify the maximum number of work-items that the compiler provisions for a work-group in a kernel, insert the `[[intel::max_work_group_size(Z, Y, X)]]` attribute in your kernel source code.

For example:

```
constexpr unsigned MAX_WG_SIZE = 4;
...
cgh.parallel_for<class kernelCompute>(
```

```
nd_range<1>(range<1>(N), range<1>(wg_size)),
[=] (nd_item<id> it)
[[intel::max_work_group_size(1, 1, MAX_WG_SIZE)]] {
    auto gid = it.get_global_id(0);
    accessorRes[gid] = accessorIdx[gid] * 2;
});
```

- To specify the required number of work-items that the Intel® oneAPI DPC++/C++ Compiler provisions for a work-group in a kernel, insert the `[[sycl::reqd_work_group_size(Z, Y, X)]]` attribute in your kernel source code.

For example:

```
constexpr unsigned REQD_WG_SIZE = 4;
...
cgh.parallel_for<class kernelCompute>(
    nd_range<1>(range<1>(N), range<1>(wg_size)),
    [=] (nd_item<id> it)
    [[sycl::reqd_work_group_size(1, 1, REQD_WG_SIZE)]] {
        auto gid = it.get_global_id(0);
        accessorRes[gid] = accessorIdx[gid] * 2;
    });
```

Specify Number of SIMD Work Items (`num_simd_work_items`)

You have the option to increase the data-processing efficiency of a SYCL kernel by executing multiple work-items in a single instruction multiple data (SIMD) manner without manually vectorizing your kernel code.

Specify the number of work-items within a work-group that the Intel® oneAPI DPC++/C++ Compiler should execute in a SIMD or vectorized manner.

Deprecation Notice

The `[[cl::reqd_work_group_size(Z, Y, X)]]` attribute is deprecated. Use the `[[sycl::reqd_work_group_size(Z, Y, X)]]` attribute.

To specify the number of SIMD work-items in a work-group, insert the `[[intel::num_simd_work_items(N)]]` attribute in the kernel source code. The supported values for size *N* are 2, 4, 8, and 16. Other sizes are accepted, but ignored (no vectorization occurs).

Consider the following example:

```
cgh.parallel_for<class kernelComputeSIMD>(
    nd_range<1>(range<1>(N), range<1>(REQD_WORK_GROUP_SIZE)),
    [=] (nd_item<id> it)
    [[intel::num_simd_work_items(NUM_SIMD_WORK_ITEMS),
    sycl::reqd_work_group_size(1, 1, REQD_WORK_GROUP_SIZE)]] {
        auto gid = it.get_global_id(0);
        accessorRes[gid] = sycl::sqrt(accessorIdx[gid]);
    }
```

NOTE

Introduce the `[[intel::num_simd_work_items(N)]]` attribute in conjunction with the `[[sycl::reqd_work_group_size(Z, Y, X)]]` attribute. The `[[intel::num_simd_work_items(N)]]` attribute you specify must evenly divide the last argument that you specify to the `req_work_group_size` attribute.

For additional information about `[[sycl::reqd_work_group_size(Z, Y, X)]]` attribute, refer to [Specify a Workgroup Size \(max_work_group_size/reqd_work_group_size\)](#).

Omit Hardware that Generates and Dispatches Kernel IDs (`max_global_work_dim`)

This attribute is deprecated. The compiler automatically adds this attribute for any single-task kernel, so adding this attribute explicitly is no longer required.

The `[[intel::max_global_work_dim(0)]]` kernel attribute instructs the Intel® oneAPI DPC++/C++ Compiler to omit logic that generates and dispatches global, local, and group IDs into the compiled kernel.

Semantically, the `[[intel::max_global_work_dim(0)]]` kernel attribute specifies that the global work dimension of the kernel is zero. Setting this kernel attribute means that the kernel does not use any global, local, or group IDs. The presence of this attribute in the kernel code serves as a guarantee to the compiler that the kernel is a single work-item kernel.

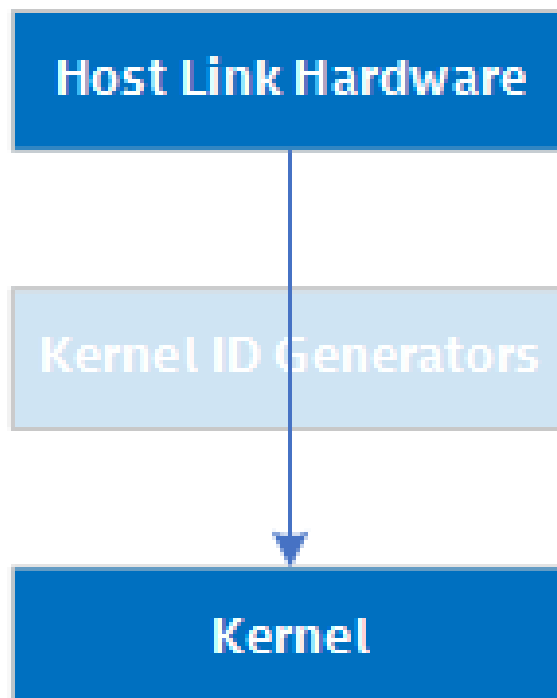
When compiling the following kernel, the compiler generates interface hardware as illustrated in [Figure 111: Compiler-generated Interface Hardware for a Kernel with the `\[\[intel::max\_global\_work\_dim\(0\)\]\]` Attribute](#):

```
cgh.single_task<class kernelComputeAsTask>(  
  [=] ()  
  [[intel::max_global_work_dim(0)]] {  
    for (unsigned i = 0; i < SIZE; i++) {  
      accessorRes[i] = accessorIdx[i] * 2;  
    }  
  });
```

NOTE

The `[[intel::max_global_work_dim(0)]]` attribute must be run as a task and not as a `parallel_for` function.

Compiler-generated Interface Hardware for a Kernel with the `[[intel::max_global_work_dim(0)]]` Attribute



If your current kernel implementation has multiple work-items but does not use global, local, or group IDs, you can use the `[[intel::max_global_work_dim(0)]]` kernel attribute if you modify the kernel code accordingly:

1. Wrap the kernel body in a `for` loop that iterates as many times as the number of work-items.
2. Use `cgh.single_task<kernelName>` to invoke the device code.

Omit Hardware that Supports Global Work Offsets (`no_global_work_offset`)

The `[[intel::no_global_work_offset(1)]]` kernel attribute instructs the Intel® oneAPI DPC++/C++ Compiler to omit generating hardware required to support global work offsets. This may improve area and/or throughput for all device kernels that are invoked using `parallel_for` without the `workItemOffset` parameter and that do not use `ndrange` objects with offsets.

Reduce Kernel Area and Latency (`use_stall_enable_clusters`)

The `[[intel::use_stall_enable_clusters]]` attribute enables you to direct the Intel® oneAPI DPC++/C++ Compiler to reduce the area and latency of your kernel. Reducing the latency does not have a large effect on loops that are pipelined, unless the number of iterations of the loop is very small.

Computations in an FPGA kernel are normally grouped into the following cluster types:

- **Stall-Free Clusters (SFC):** Allows simplification of signals within a cluster, but the FIFO queue at the end of the cluster is used to save intermediate results if the computation must stall. For more information about SFCs, refer to [Clustering the Datapath](#).
- **Stall-Enable Clusters (SEC):** Saves area and cycles by removing the FIFO queue and passing the stall signals to each part of the computation. These extra signals may cause the f_{MAX} to reduce. For more information, refer to [Clustering the Datapath](#).

Caution

If you specify the `[[intel::use_stall_enable_clusters]]` attribute on one or more kernels, the compiler might reduce the f_{MAX} of the generated FPGA bitstream, which may reduce performance on all kernels.

Hyper-Optimized Handshaking Restriction

The `[[intel::use_stall_enable_clusters]]` attribute prevents the use of hyper-optimized handshaking on designs that target Stratix® 10 and later FPGA architectures.

Example

```
h.single_task<class KernelComputeStallFree>( [=]() [[intel::use_stall_enable_clusters]] {  
    // The computations in this device kernel uses Stall Enable Clusters  
    Work(accessor_vec_a, accessor_vec_b, accessor_res);  
});
```

The compiler uses stall-enable clusters for the kernel when possible. Some computations might not be stallable, so the compiler places them in a stall-free cluster even if a stall-enable cluster was requested.

NOTE

For more information, refer to the FPGA tutorial sample “Stall Enable Clusters” on [GitHub](#).

Memory Attributes

The following table lists attributes that allow fine-grained control on how you can implement a variable (usually an array) in on-chip memory. The attribute immediately precedes the variable declaration.

FPGA Memory Attributes

Attribute	Syntax	Description
bank_bits	<code>[[intel::bank_bits($b_0, b_1, \dots, b_n$)]]</code>	Specifies that the local memory addresses should use bits ($b_0, b_1, \dots, b_n$) for bank selection, where, ($b_0, b_1, \dots, b_n$) are indicated in terms of word-addressing and not byte-addressing. As a result, the number of banks is equal to $2^{\text{number of bank bits}}$. The bits of the local memory address not included in ($b_0, b_1, \dots, b_n$) are used for word selection in each bank.
bankwidth	<code>[[intel::bankwidth(N)]]</code>	Specifies that the memory system implementing the local variable must have banks that are N bytes wide, where N is a power-of-2 integer value greater than zero.
doublepump	<code>[[intel::doublepump]]</code>	Specifies that the memory system implementing the local variable must operate at twice the clock frequency of the kernel accessing it. This allows twice as many memory accesses per kernel clock cycle but may reduce the maximum kernel clock frequency.
force_pow2_depth	<code>[[intel::force_pow2_depth(N)]]</code>	Specifies that the memory implementing the variable or array has a power-of-2 depth. This attribute is enabled if N is 1, and disabled if N is 0.
max_replicates	<code>[[intel::max_replicates(N)]]</code>	Specifies that the memory implementing the local variable or array has no more than the specified number of replicates to enable simultaneous reads from the datapath.
fpga_memory	<code>[[intel::fpga_memory(<impl_type>)]]</code>	Specifies that the compiler must implement the local variable in a memory system. You may pass an optional string argument to specify the memory implementation type. Specify <impl_type> as either a <code>BLOCK_RAM</code> or <code>MLAB</code> to implement the memory using memory blocks (for example, M20K) or memory logic array blocks (MLABs), respectively.
merge	<code>[[intel::merge(<key>, <direction>)]]</code>	Merges of two or more variables or arrays defined in the same scope in a width-wise or depth-wise manner. All variables with the same key string are merged into the same memory system. The string <direction> can be either <code>width</code> or <code>depth</code> .
numbanks	<code>[[intel::numbanks(N)]]</code>	Specifies that the memory system implementing the local variable must have N banks, where N is a power-of-2 integer value greater than zero.
private_copies	<code>[[intel::private_copies(N)]]</code>	Specifies that the memory has a defined number of copies to allow simultaneous iterations of a loop at any given time. When you declare an array in the scope of a loop body, the array is private to that iteration of the loop. However, concurrency is limited if all iterations of the loop must share the same physical memory. Specifying

Attribute	Syntax	Description
		<code>private_copies(N)</code> allows <code>N</code> iterations of the loop to execute concurrently, each with its own private copy of the array. A larger value of <code>N</code> may expose more opportunities for parallel execution, at a cost of higher on-chip memory utilization. <code>private_copies(N)</code> interacts with <code>max_concurrency</code> attribute applied to the loop. For more information, refer to the max_concurrency Attribute and FPGA tutorial sample "Private Copies" on GitHub.
<code>fpga_register</code>	<code>[[intel::fpga_register]]</code>	Specifies that the variable must be carried through the pipeline in registers. The compiler may implement a register variable either exclusively in flip-flops (FFs), or in a combination of FFs and RAM-based FIFOs. You can also apply this attribute to device_global variables. However, the device_global variables must be used in a single kernel and they must not have any host accesses. The <code>fpga_register</code> memory attribute is functionally equivalent to the fpga_datapath class template. Restriction You cannot apply this attribute to device_global struct/class member variables. The compiler ignores the <code>fpga_register</code> attribute when it is applied to device_global struct/class member variables.
<code>simple_dual_port</code>	<code>[[intel::simple_dual_port]]</code>	Specifies that the memory implementing the variable or array should have read-only and write-only ports rather than read or write ports. Specifying <code>simple_dual_port</code> forces the compiler to configure the memory's underlying hardware resources in simple dual port mode, which can have area benefits in some corner cases.
<code>singlepump</code>	<code>[[intel::singlepump]]</code>	Specifies that the memory system implementing the local variable must operate at the same clock frequency as the kernel accessing it.

Tip

For additional information, refer to the FPGA tutorial sample "Memory Attributes" on [GitHub](#).

Struct Data Types and Memory Attributes

You can apply memory attributes to the member variables in a `struct` variable within the `struct` declaration. If you also apply memory attributes to the object instantiation of a `struct` variable, the attributes on the instantiation override the attributes from the declaration.

Consider the following code example where memory attributes are applied to both a declaration and instantiation:

```
struct State {
    [[intel::fpga_memory]] int array[100];
    [[intel::fpga_register]] int reg[4];
};

cgh.single_task<class test>([=] {
    struct State S1;
    [[intel::fpga_memory]] struct State S2;
    // some uses
});
```

In this example code, the compiler splits `S1` into two variables, `S1.array[100]` (implemented in the memory) and `S1.reg[4]` (implemented in registers). However, the compiler ignores attributes applied at the `struct` declaration for object `S2` and does not split it since the `S2` object has the `[[intel::fpga_memory]]` attribute applied to it.

Loop Directives

The following directives apply to a loop that immediately follows the pragma or an attribute directive. No statements can be placed between the pragma or attribute directive and the loop (for, while, do) to which the directive applies, with the exception of other loop directives.

disable_loop_pipelining Attribute

If loop-carried dependencies result in an initiation interval (II) that is equal or close to the latency of a given iteration (effectively inducing serial execution of the pipelined loop), disable pipelining of the loop to generate a simpler datapath and reduce area utilization. Use the `disable_loop_pipelining` attribute to direct the Intel® oneAPI DPC++/C++ Compiler to disable pipelining of a loop. When you apply this attribute, the compiler generates a simple sequential loop datapath. This attribute applies to single work-item kernels (that is, single-threaded kernels) in which loops are pipelined. For more information about single work-item kernels and associated concepts, refer to [Single Work-Item Kernels](#).

Syntax

```
[[intel::disable_loop_pipelining]]
```

Unless otherwise specified, the compiler always attempts to generate a pipelined loop datapath where possible. When generating a pipelined circuit, resources of the loop must be duplicated to execute multiple iterations simultaneously, leading to an increased silicon area utilization. In cases where loop pipelining does not result in an improvement in throughput, avoid the area overhead by applying the `disable_loop_pipelining` attribute to the loop, as shown in the following example code snippet:

Example

```
[[intel::disable_loop_pipelining]]
for (int i = 1; i < N; i++) {
    int j = a[i-1];
    // Memory dependency induces a high-latency loop feedback path
    a[i] = foo(j)
}
```

In the above example, the compiler fails to schedule the loop with a small II due to memory dependency (as reported in the Details pane of the Loop Analysis report). In such cases, loop pipelining is unlikely to be beneficial.

initiation_interval Attribute

The initiation interval, or II, is the number of clock cycles between the launch of successive loop iterations.

Use the `initiation_interval` attribute to direct the Intel® oneAPI DPC++/C++ Compiler to attempt to set the II for the loop that follows the attribute declaration. If the Intel® oneAPI DPC++/C++ Compiler cannot achieve the specified II for the loop, then the compilation errors out.

Syntax

```
[[intel::initiation_interval(n)]]
```

The `initiation_interval` attribute applies to pipelined loops in single task kernels. Refer to [Pipelining](#) for information about loop pipelining.

The attribute parameter `n` is required and must be a positive constant expression of integer type. The parameter specifies a minimum number of clock cycles to wait between the beginnings of execution of successive loop iterations.

The higher the II value, the longer the wait before the subsequent loop iteration starts executing. Refer to [Loop Analysis](#) for information about II and compiler reports that provide you with details on the performance implications of II on a specific loop.

NOTE

The `initiation_interval` attribute should only be applied to a pipelined loop in a single work-item (task) kernel.

If the throughput of a loop is important to the overall throughput of your kernel, you can use the `initiation_interval` attribute to force an II value of 1 even though this may result in lower f_{MAX} .

For some loops in your kernel, specifying a higher II value with the `initiation_interval` attribute than the value the compiler chooses by default can increase the maximum operating frequency (f_{MAX}) of your kernel without a decrease in throughput.

A loop is a good candidate to have a higher `initiation_interval` than the default if the loop meets the following conditions:

- The loop is not critical to the throughput of your kernel.
- The running time of the loop is small compared to other loops it might contain.

Example

Consider a case where your kernel has two distinct pipelineable loops:

- A short-running initialization loop that has a loop-carried dependence
- A long-running loop that does the bulk of your processing.

In this case, the compiler does not know that the initialization loop has a much smaller impact on the overall throughput of your design. If possible, the compiler attempts to pipeline both loops with an II of 1.

Because the initialization loop has a loop-carried dependence, it does have a feedback path in the generated hardware. To achieve an II with such a feedback path, some clock frequency might be forfeited. Depending on the feedback path in the main loop, the rest of your design could have run at a higher operating frequency.

If you specify `[[intel::initiation_interval(2)]]` on the initialization loop, then you are informing the compiler that it can be less aggressive in optimizing II for this loop. Less aggressive optimization allows the compiler to pipeline the path limiting the f_{MAX} and allow your overall kernel design to achieve a higher f_{MAX} .

The initialization loop takes longer to run with its new II. However, the decrease in the running time of the long-running loop due to higher f_{MAX} compensates for the increased length in running time of the initialization loop.

Tip

For additional information, refer to the FPGA tutorial sample “Loop Initiation Interval” on [GitHub](#).

ivdep Attribute

Include the `ivdep` attribute in your single task kernel to direct the Intel® oneAPI DPC++/C++ Compiler to ignore memory aliasing dependencies carried by the loop that the attribute is applied to. This attribute applies only to the loop it is applied to, and not to any of the future loops that might appear as a result of the `[[intel::loop_coalesce(N)]]` attribute.

Syntax

```
[[intel::ivdep]]  
[[intel::ivdep(safelen)]]  
[[intel::ivdep(array)]]  
[[intel::ivdep(array, safelen)]]  
[[intel::ivdep(safelen, array)]]
```

Caution

Applying the `ivdep` attribute incorrectly results in functionally incorrect hardware and potential functional differences between the hardware run and emulation. The `ivdep` attribute is ignored in emulation.

During compilation, the Intel® oneAPI DPC++/C++ Compiler creates hardware that ensures load and store instructions operate within dependency constraints. An example of a dependency constraint is that dependent load and store instructions must execute in order. The presence of the `ivdep` attribute instructs the Intel® oneAPI DPC++/C++ Compiler to remove the extra hardware between load and store instructions in the loop that immediately follows the attribute declaration in the kernel code. Removing the extra hardware may reduce logic utilization and lower the II value.

You can provide more information about loop dependencies by specifying a `safelen` parameter to the attribute by adding an integer type C++ constant expression argument to the attribute. The `safelen` parameter specifies the maximum number of consecutive loop iterations without loop-carried dependencies. For example, `[[intel::ivdep(32)]]` indicates to the compiler that there are at least 32 iterations of the loop before loop-carried dependencies are introduced. That is, while the `[[intel::ivdep]]` attribute guarantees to the compiler that there are no implicit memory dependencies between any iteration of this loop, `[[intel::ivdep(32)]]` guarantees that there does not exist a loop-carried dependence with a dependence distance less than 32. For example, if an iteration reads from memory, the preceding 31 iterations and succeeding 31 iterations are guaranteed not to write to the same memory location.

NOTE A `safelen` setting of 1 or 0 is allowed as a convenience, but it has no effect on the loop and the compiler generates a warning.

To specify that accesses to a particular memory array inside a loop do not cause loop-carried dependencies, add the array parameter to the attribute by specifying the array variable name as an argument to the attribute. The array specified by the `ivdep` attribute must be a local or private memory array, or a pointer variable that points to a global, local, or private memory storage. The array specified by the `ivdep` attribute can also be an array or a pointer member of a `struct`.

Important

The `ivdep` attribute applies to memory aliasing dependencies and not memory ordering dependencies.

For example, the `ivdep` attribute in the following code does not cause the compiler to ignore dependencies.

```
atomic_ref<int, memory_order::acquire, memory_scope_device> B_ref(B);
atomic_ref<int, memory_order::release, memory_scope_device> A_ref(A);
[[intel::ivdep]]
for (int i = 0; i < N; ++i) {
    // Assume A and B don't alias
    A_ref[i] = ...;
    ... = B_ref[i];
}
```

Because the atomic store to `A` has release semantics (that is, the store must happen after any memory operation that comes before it), and the atomic load from `B` has acquire semantics (that is, the load must happen before any memory operation that comes after it) then the `ivdep` attribute will not cause the compiler to ignore the dependence between the load on iteration i and the store on iteration $i+1$.

Examples

```
// No loop-carried dependencies for accesses to arrays A and B
[[intel::ivdep]]
for (int i = 0; i < N; i++) {
    A[i] = A[i - X[i]];
    B[i] = B[i - Y[i]];
}

// No loop-carried dependencies for accesses to array A
// Compiler inserts hardware that reinforces dependency constraints for B
[[intel::ivdep(A)]]
for (int i = 0; i < N; i++) {
    A[i] = A[i - X[i]];
    B[i] = B[i - Y[i]];
}

// No loop-carried dependencies for array A inside struct
[[intel::ivdep(S.A)]]
for (int i = 0; i < N; i++) {
    S.A[i] = S.A[i - X[i]];
}

// No loop-carried dependencies for array A inside the struct pointed by S
[[intel::ivdep(S->X[2][3].A)]]
for (int i = 0; i < N; i++) {
    S->X[2][3].A[i] = S.A[i - X[i]];
}
```

Restriction Do not apply the `ivdep` attribute to a variable that is used in a lambda function or a variable that is passed as a function argument. The `ivdep` attribute is not propagated to the lambda function or function argument. Because the attribute is not propagated, its effects are ignored and can result in functional failures in your kernel.

```
// The ivdep directive will not be applied to usages of accessorA inside the lambda foo or
the function bar
[[intel::ivdep(accessorA, safelen)]]
for (;;) {
    auto foo = [=]() { ...; // Uses of accessorA }
    bar(accessorA, ...);
}
```

Caution

When specifying the array name an `ivdep` attribute must apply to, the compiler still claims there is a memory dependency on that array. Consider the following example code:

```
[[intel::ivdep( kSafeLen, histogram.data_ )]]
for( uint32_t n = 0; n < kInitNumInputs; ++n ) {
    // Compute the Histogram index to increment
    uint32_t hist_group = input[n] % kNumOutputs;
    auto hist_count = histogram.read( hist_group );
    hist_count++;
    histogram.write( hist_group, hist_count );
}
```

This `for` loop returns an II of 2 and reports memory dependency on the `histogram.data_` array, to which the compiler must apply the `ivdep` attribute. `histogram.data_` is accessed in the `histogram.write()` and `histogram.read()` function calls, which prevents the `ivdep` directive from working.

Loop Analysis
☐ Show blocks

Name	Source Location	Pipelined	Block Scheduled II	Block Scheduled fMAX	Latency	Speculated Iterations	Max Interleaving Iterations	Brief Info
HistogramID<5ul>.B4	onchip_memory_cache.cpp:52	Yes	2	480.00	882.000000	2	1	Memory dependency

Details
X

Variable on critical loop carried feedback path: `histogram.data_`
Declared at: [onchip_memory_cache.cpp: 44](#)
Dependency: Memory dependency
Estimated fmax reduced to: 480.0
Loop feedback path that raised II and/or lowered fMAX:

- LD([cached_local_memory.hpp: 54](#))
- ST([cached_local_memory.hpp: 47](#))
- +([onchip_memory_cache.cpp: 56](#))
- Select([onchip_memory_cache.cpp: 44](#))

You can apply `[[intel::ivdep(safelen, array)]]` only to accesses of that specific array by name in the code that is lexicographically in the loop, which means the array is not traced through function calls.

Caution

if a loop with the `[[intel::ivdep]]` or `[[intel::ivdep(safelen)]]` attributes has an array declared within the scope of the loop itself, the effects of the `ivdep` apply to the loads and stores of that array.

It is currently undefined behavior to add an `ivdep` attribute to a loop that includes a locally-defined array if the same indices of that array are accessed in consecutive loop iterations. Even though such a program does not technically have loop-carried dependencies, this is a current limitation of the compiler.

Consider the following example:

```
[[intel::ivdep]]
for (int i = 0; i < N; ++i) {
    int A[2];
    A[0] = i;
    A[1] = i+1;
    ...
    B[idx[i]] = A[i%2];
}
```

Although `A` is local to the loop, because we generate one physical memory for `A`, we need to make sure that the writes to `A` in iteration i happen after the read from `A` in iteration $i-1$. Removing that dependence may lead to undefined behavior.

For this example, if the accesses to `B` do not have loop-carried dependencies, it would be legal to replace `[[intel::ivdep]]` with `[[intel::ivdep(B)]]`.

For additional information, refer to [“Loop ivdep Sample” on GitHub](#).

loop_coalesce Attribute

Use the `loop_coalesce` attribute to direct the Intel® oneAPI DPC++/C++ Compiler to coalesce nested loops into a single loop without affecting the loop functionality. Coalescing loops can help reduce your kernel area usage by directing the compiler to reduce the overhead needed for loop control.

NOTE

If you want to use the [ivdep attribute](#) to ignore loop-carried dependencies, apply it to the loop that causes dependencies and not to any of the future loops that might appear as a result of the `[[intel::loop_coalesce(N)]]` attribute.

Syntax

```
[[intel::loop_coalesce(N)]]
```

where, the integer argument `N` specifies the nested loop levels you want the compiler to attempt to coalesce.

For example, consider the following set of nested loops:

```
for (A)
  for (B)
    for (C)
      for (D)
        for (E)
```

If you place the `loop_coalesce` attribute before loop (A), then the loop nesting level for these loops is defined as:

- Loop (A) has a loop nesting level of 1.
- Loop (B) has a loop nesting level of 2.
- Loop (C) has a loop nesting level of 3.
- Loop (D) has a loop nesting level of 4.
- Loop (E) has a loop nesting level of 3.

Depending on the loop nesting level that you specify, the compiler attempts to coalesce loops differently:

- If you specify `[[intel::loop_coalesce(1)]]` on loop (A), the compiler does not attempt to coalesce any of the nested loops.
- If you specify `[[intel::loop_coalesce(2)]]` on loop (A), the compiler attempts to coalesce loops (A) and (B).
- If you specify `[[intel::loop_coalesce(3)]]` on loop (A), the compiler attempts to coalesce loops (A), (B), (C), and (E).
- If you specify `[[intel::loop_coalesce(4)]]` on loop (A), the compiler attempts to coalesce all of the loops [loop (A) - loop (E)].

Coalescing nested loops also reduce the latency of the component, which could further reduce your kernel area usage. However, in some cases, coalescing loops might lengthen the critical loop initiation interval path, so coalescing loops might not be suitable for all kernels.

For `parallel_for` kernels, the compiler automatically attempts to coalesce loops even if they are not annotated by the `[[intel::loop_coalesce(N)]]` attribute. Coalescing loops in `parallel_for` kernels usually improves throughput as well as reducing kernel area use. You can use the `[[intel::loop_coalesce(N)]]` attribute to prevent the automatic coalescing of loops in `parallel_for` kernels.

NOTE If you specify `[[intel::loop_coalesce(1)]]` for a loop in an `parallel_for` kernel, you prevent automatic loop coalescing for that loop.

Example

The following simple example shows how the compiler coalesces two loops into a single loop.

Consider a simple nested loop written as follows:

```
[[intel::loop_coalesce(2)]]
for (int i = 0; i < N; i++)
    for (int j = 0; j < M; j++)
        sum[i][j] += i+j;
```

The compiler coalesces the two loops together so that they run as if they were a single loop written as follows:

```
int i = 0;
int j = 0;
while(i < N){

    sum[i][j] += i+j;
    j++;

    if (j == M){
        j = 0;
        i++;
    }
}
```

NOTE

For additional information, refer to the FPGA tutorial sample “loop_coalesce” on [GitHub](#).

max_concurrency Attribute

Use the `max_concurrency` attribute to limit the concurrency of a loop in your kernel. The concurrency of a loop is how many iterations of that loop can be in progress at one time. By default, the Intel® oneAPI DPC++/C++ Compiler tries to maximize the concurrency of loops so that your kernel runs at peak throughput.

Syntax

```
[[intel::max_concurrency(n)]]
```

The `max_concurrency` attribute applies to pipelined loops in single task kernels. Refer to [Pipelining](#) for information about loop pipelining.

The `max_concurrency` attribute enables you to control the on-chip memory resources required to pipeline your loop. To achieve simultaneous execution of loop iterations, the Intel® oneAPI DPC++/C++ Compiler must create copies of any memory that is private to a single iteration. These copies are called private copies. The greater the permitted concurrency, the more private copies the compiler must create.

The attribute parameter `n` is required and must be a non-negative constant expression of integer type. The parameter directs the compiler to restrict the loop’s concurrency to `n` simultaneous iterations.

The kernel’s `report.html` ([Review the FPGA Optimization Report](#)) provides the following information pertaining to loop concurrency:

- **Maximum concurrency that the Intel® oneAPI DPC++/C++ Compiler has chosen:** This information is available in the [Loop Analysis](#) report and [Kernel Memory Viewer](#).
 - In the Loops Analysis report, a message in the **Details** pane reports as the maximum number of simultaneous executions has been limited to `n`.

NOTE

The value of unsigned `N` can be greater than or equal to zero. A value of `N = 0` indicates unlimited concurrency.

- In the Memory Viewer, the bank view of your local memory graphically shows the number of private copies.
- **Impact to memory usage:** This information is available in the [Area Estimates](#) report. A message in the Details pane reports that the Intel® oneAPI DPC++/C++ Compiler has created `N` independent copies of the memory to enable simultaneous execution of `N` loop iterations.

If you want to exchange some performance for physical memory savings, apply

`[[intel::max_concurrency(n)]]` to the loop, as shown in the following code snippet:

```
[[intel::max_concurrency(1)]]
for (int i = 0; i < N; i++) {
    int arr[M];
    // Doing work on arr
}
```

When you apply this attribute, the Intel® oneAPI DPC++/C++ Compiler limits the number of simultaneously-executed loop iterations to `n`. The number of private copies of loop-scooped memories is also restricted to `n`.

You can also control the number of private copies (created for a local memory and accessed within a loop) by using `[[intel::private_copies(N)]]`. If a local memory with `[[intel::private_copies(N)]]` is accessed with a loop that has `[[intel::max_concurrency(M)]]` attribute, the Intel® oneAPI DPC++/C++ Compiler limits the number of simultaneously-executed loop iterations to `min(M, N)`. For

more information about `[[intel::private_copies(N)]]`, refer to [FPGA Memory Attributes](#). For additional information about using `[[intel::private_copies(N)]]`, refer to the FPGA tutorial sample “Private Copies” on [GitHub](#).

max_interleaving Attribute

Use the `max_interleaving` attribute to maximize the throughput and hardware resource occupancy of pipelined inner loops in a loop nest by issuing new inner loop iterations as frequently as possible (minimizing the loop initiation interval). When the compiler cannot achieve a loop II of 1 for an inner loop, the compiler configures the loop nest to interleave iterations of one invocation of the inner loop with iterations of other invocations of the inner loop.

Syntax

```
[[intel::max_interleaving(n)]]
```

The Intel® oneAPI DPC++/C++ Compiler restricts the annotated (inner) loop to be invoked at most `n` times per outer loop iteration. When this attribute is specified with `n=0`, the compiler allows the pipeline to contain a number of simultaneous invocations of the annotated loop equal to the loop initiation interval (II) of that loop. For example, an annotated inner loop with an II of 2 can have iterations from two invocations in the pipeline at a time. This behavior is the default behavior for the compiler if you do not specify the `max_interleaving` attribute.

As an example, consider the loop nest in the following code snippet:

```
int a[N];
// ...
// Loop j is pipelined with ii=1
for (int j = 0; j < M; j++) {
    // Loop i is pipelined with ii=2
    for (int i = 0; i < N; i++) {
        a[i] += foo(i);
    }
}
```

In this example, the inner `i` loop is pipelined with a loop II of 2. Under normal pipelining, this means that the inner loop hardware only achieves 50% utilization since one `i` iteration is initiated every other cycle. To take advantage of these idle cycles, the compiler interleaves a second invocation of the `i` loop from the next iteration of the outer `j` loop. Here, a loop invocation means to start pipelined execution of a loop body. In this example, since the `i` loop resides inside the `j` loop, and the `j` loop has a trip count of `M`, the `i` loop is invoked `M` times. Since the `j` loop is an outermost loop, it is invoked once. The following table illustrates the difference between normal pipelined execution of the `i` loop and interleaved execution for this example where `N=5`:

Difference Between Normal Pipelined Execution and Interleaved Execution

Cycle	Pipelined	Interleaved
0	(0,0)	(0,0)
1	---	(1,0)
2	(0,1)	(0,1)
3	---	(1,1)
4	(0,2)	(0,2)
5	---	(1,2)
6	(0,3)	(0,3)
7	---	(1,3)
8	(0,4)	(0,4)
9	---	(1,4)
10	(1,0)	(2,0)
11	---	(3,0)
12	(1,1)	(2,1)
13	---	(3,1)

Cycle	Pipelined	Interleaved
14	(1,2)	(2,2)
15	---	(3,2)
16	(1,3)	(2,3)
17	---	(3,3)
18	(1,4)	(2,4)
19	---	(3,4)

The table shows the values (j, i) for each inner loop iteration that is initiated at each cycle. At cycle 0, both modes of execution initiate the $(0,0)^{\text{th}}$ iteration of the i loop. Under normal pipelined execution, no i loop iteration is initiated at cycle 1. Under interleaved execution, the $(1,0)^{\text{th}}$ iteration of the innermost loop, that is, the first iteration of the next $(j=1)$ invocation of the i loop is initiated. By cycle 10, interleaved execution has initiated all of the iterations of both the $j=0$ invocation of the i loop and the $j=1$ invocation of the i loop. This represents twice the efficiency of the normal pipelined execution.

In some cases, you may decide that the performance benefit from interleaving is not equal to the area cost associated with enabling interleaving. In these cases, you may want to limit or restrict the amount of interleaving to reduce FPGA area utilization. To limit the number of interleaved invocations of an inner loop that can be executed simultaneously, annotate the inner loop with the `[[intel::max_interleaving(n)]]` attribute. The annotated loop must be contained inside another pipelined loop. The required parameter (n) specifies an upper bound on the degree of interleaving allowed, that is, how many invocations of the containing loop can execute the annotated loop at a given time.

Specify the `[[intel::max_interleaving(n)]]` attribute in one of the following ways:

- `[[intel::max_interleaving(1)]]`

The compiler restricts the annotated (inner) loop to be invoked only once per outer loop iteration. That is, all iterations of the inner loop travel the pipeline before the next invocation of the inner loop can occur.

- `[[intel::max_interleaving(0)]]`

The compiler allows the pipeline to contain a number of simultaneous invocations of the inner loop equal to the loop initiation interval (II) of the inner loop. For example, an inner loop with an II of 2 can have iterations from two invocations in the pipeline at a time. This behavior is the default behavior for the compiler if you do not specify the `[[intel::max_interleaving(n)]]` attribute.

Example

In the following code snippet, the compiler restricts the pipelined execution of the i loop. A new invocation of the i loop corresponds only to the subsequent iteration of the j loop.

```
// Loop j is pipelined with ii=1
for (int j = 0; j < M; j++) {
    int a[N];
    // Loop i is pipelined with ii=2
    [[intel::max_interleaving(1)]]
    for (int i = 1; i < N; i++) {
        a[i] = foo(i)
    }
    ...
}
```

NOTE

For additional information, refer to the FPGA tutorial sample "Max Interleaving" on [GitHub](#).

speculated_iterations Attribute

Use the `speculated_iterations` attribute to direct the Intel® oneAPI DPC++/C++ Compiler to improve the performance of pipelined loops. The `speculated_iterations` attribute is applied to loops and hence, it must appear directly before the loop (the same place as other loop attributes). For more information, refer to [Optimize Loops With Loop Speculation](#).

Syntax

```
[[intel::speculated_iterations(N)]]
```

Where, the integer argument *N* specifies the permissible number of iterations to speculate.

The Intel® oneAPI DPC++/C++ Compiler generates hardware to run *N* extra iterations of the loop while ensuring the extra iterations do not affect anything. This allows either reducing the II of the loop or increasing the f_{max} . The deciding factor is how quickly the exit condition of the loop is calculated. If the calculation takes many cycles, it is better to have larger `speculated_iterations`.

NOTE

Extra iterations increase the time before the next invocation of the loop can begin. This may be a factor if the actual number of iterations of the loop is very small (less than 5 to 10 or similar). In this case, specify the *N* value as 0 to allow subsequent loop iterations to start immediately but at the cost of a larger II to allow more time to evaluate the exit condition. Refer to the [Loop Analysis](#) report to identify whether the exit condition is a bottleneck for II.

Example

```
[[intel::speculated_iterations(1)]]
while (m*m*m < N) {
    m += 1;
}
dst[0] = m;
```

The loop in this example will have one speculated iteration.

NOTE

For additional information, refer to the FPGA tutorial sample “Speculated Iterations” on [GitHub](#).

unroll Pragma

Loop unrolling involves replicating a loop body multiple times and reducing the trip count of a loop. Unroll loops to reduce or eliminate loop control overhead on the FPGA. In cases where there are no loop-carried dependencies and the Intel® oneAPI DPC++/C++ Compiler can perform loop iterations in parallel, unrolling loops can also reduce latency and overhead.

Important

Unrolling of nested loops with large bounds might generate huge number of instructions that could lead to very long compile times.

The compiler might unroll simple loops even if a pragma does not annotate them. To direct the compiler to unroll a loop, or to explicitly not unroll a loop, insert an `unroll` kernel pragma in the kernel code preceding a loop you want to unroll. To specify an unroll factor *N*, use the optional unroll factor specifier `#pragma unroll <N>`. For more information, see *Determining the Correct Unroll Factor* section in [Unrolling Loops FPGA tutorial](#).

Syntax

```
#pragma unroll  
  
#pragma unroll N
```

If you specify the unroll factor *N*, the factor must be a positive constant expression of integer type. If you omit the unroll factor *N*, the loop is unrolled fully.

Examples

The following is an example of full loop unrolling:

```
// Before unrolling loop  
#pragma unroll  
for(i = 0 ; i < 5; i++){  
    a[i] += 1;  
}  
  
// After fully unrolling the loop by a factor of 5,  
// the loop is flattened. There is no loop after unrolling.  
a[0] += 1;  
a[1] += 1;  
a[2] += 1;  
a[3] += 1;  
a[4] += 1;
```

You can observe that a full unroll is a special case where the unroll factor is equal to the number of loop iterations.

The following is an example of partial loop unrolling:

```
// Before unrolling loop  
#pragma unroll 4  
for(i = 0 ; i < 20; i++){  
    a[i] += 1;  
}  
  
// After the loop is unrolled by a factor of 4,  
// the loop has five (20 / 4) iterations.  
for(i = 0 ; i < 5; i++){  
    a[i * 4] += 1;  
    a[i * 4 + 1] += 1;  
    a[i * 4 + 2] += 1;  
    a[i * 4 + 3] += 1;  
}
```

In the partial unroll example, each loop iteration in the unrolled loop is equivalent to four iterations. The Intel® oneAPI DPC++/C++ Compiler instantiates four adders instead of one adder. Because there is no data dependency between iterations in the loop (which is true in this case), the compiler executes four adds in parallel.

Tip

For additional information, refer to the FPGA tutorial sample "Loop Unroll" on [GitHub](#).

NOTE

- Provide an `unroll` factor whenever possible. To specify an unroll factor `N`, insert the `#pragma unroll <N>` directive before a loop in your kernel code. The Intel® oneAPI DPC++/C++ Compiler attempts to unroll the loop at most `<N>` times. Consider the following code fragment. By assigning a value of 2 as the unroll factor, you direct the compiler to unroll the loop twice.

```
#pragma unroll 2
for(size_t k = 0; k < 4; k++)
{
    mac += data_in[(gid * 4) + k] * coeff[k];
}
```

For more information, see *Determining the Correct Unroll Factor* in the FPGA tutorial sample “Loop Unroll” on [GitHub](#).

- To unroll a loop fully, you may omit the unroll factor by simply inserting the `#pragma unroll` directive before a loop in your kernel code. The compiler attempts to unroll the loop fully if it understands the trip count and issues a warning if it cannot execute the unroll request.

Loop Fuse Functions and `nofusion` Attribute

This topic describes the loop fuse functions and `nofusion` attribute that the Intel® oneAPI DPC++/C++ Compiler supports.

Loop Fuse Functions

The loop fuse functions are declared in the `sycl/ext/intel/fpga_loop_fuse.hpp` header file, which is invoked by the `sycl/ext/intel/fpga_extensions.hpp` header file. Apply these loop functions to a block of code to indicate to the compiler that it must fuse adjacent loops in the code block overriding the compiler profitability or safety analysis of the fusion. Fusing adjacent loops reduces the amount of loop control overhead in your kernel that reduces the FPGA area used and increases the performance by executing both loops as one (fused) loop. For additional information, see [Fuse Loops to Reduce Overhead and Improve Performance](#)

The compiler supports the following loop fuse functions:

- `sycl::ext::intel::fpga_loop_fuse<v>(f)`: Directs the compiler to fuse loops within the function `f` and up to a depth of `v >= 1` without affecting the functionality of either loop, overriding the compiler profitability analysis of fusing the loops. By default, `v = 1`, which is equivalent to indicating that the compiler should consider only the adjacent top-level loops for fusing. For example:

```
[=] () { //Kernel
    sycl::ext::intel::fpga_loop_fuse<1>([&] {
        L1: for(...) {}
        L2: for(...) {
            L3: for(...) {}
            L4: for(...) {
                L5: for(...) {}
                L6: for(...) {}
            }
        }
    });
}
```

By default (`v = 1`), only loops `L1` and `L2` are initially considered for fusing. At a depth of `v = 2`, the compiler considers `L1-L2` and `L3-L4` loop pairs for fusing.

The compiler automatically considers fusing adjacent loops with equal trip counts when the loops meet the [Automatic Loop Fusion](#) criteria or when fusion is deemed safe and profitable. You can use the `sycl::ext::intel::fpga_loop_fuse<v>(f)` function to inform the compiler to consider fusing adjacent loops with different trip counts, which is considered to be unprofitable by default.. With the loop fuse function applied to a block of code, the compiler always attempts to fuse adjacent loops (with equal or different trip counts) in the block whenever the compiler determines that it is safe to fuse the loops. Two loops are considered safe to merge if they meet the [Fusion Criteria](#). The following example shows the effects of fusing loops with unequal trip counts:

Unfused Loops

```
[=]() { //Kernel
    sycl::ext::intel::fpga_loop_fuse([&]
    {
        for (int i = 0; i < N; i++) {
            // Loop Body 1
        }
        for (int j = 0; j < M; j++) {
            // Loop Body 2
        }
    });
}
```

Fused Loops

```
for (int f = 0; f < max(M,N); f++) {
    if (f < N) {
        // Loop Body 1
    }
    if (f < M) {
        // Loop Body 2
    }
}
```

A fused loop can itself be considered for fusing with other loops. For example, in the following code, L1 and L2 are initially considered for fusing. That resulting fused loop can then be considered for fusing with L4.

```
[=]() { //Kernel
    sycl::ext::intel::fpga_loop_fuse([&] {
        L1: for(...) {}
        L2: for(...) {
            L3: for(...) {}
        }
        L4: for(...) {
        }
    });
}
```

- **`sycl::ext::intel::fpga_loop_fuse_independent<v>(f)`**: Directs the compiler to fuse loops within the function `f` up to a depth `v` ≥ 1 while overriding fusion-safety checks. Here, `v = 1` by default. When you use this function, you are guaranteeing to the compiler that fusing pairs of loops affected by the loop fuse function is safe. That is, there are no negative distance dependencies between the pairs of loops. If it is not safe, you might get functional errors in your kernel. Besides this difference, the `sycl::ext::intel::fpga_loop_fuse<v>(f)` and `sycl::ext::intel::fpga_loop_fuse_independent<v>(f)` functions behave identically.

Function Calls in Loop Fuse Code Blocks

If a function call occurs in a code block annotated with a loop fuse function and inlining that function call contains a loop, the resulting loop can be a candidate for loop fusion.

Nested Loop Fusion Functions

When you nest loop fusion functions, you might create overlapping sets of candidate loops. Consider the following example:

```
[=]() { //Kernel
    sycl::ext::intel::fpga_loop_fuse_independent<2>([&] {
        L1: for(...) {}
        L2: for(...) {
```

```

sycl::ext::intel::fpga_loop_fuse<2>([&] {
    L3: for(...) {}
    L4: for(...) {
        L5: for(...) {}
        L6: for(...) {}
    }
});
}
});
}

```

In this example, the compiler considers the following loop pairs for fusion: L1-L2, L3-L4, and L5-L6. In addition, the compiler overrides the compiler negative distance dependency analysis of L1-L2 and L3-L4 loop pairs.

nofusion Attribute

You can exempt a loop from being fused with an adjacent loop by annotating the loop with the `nofusion` loop attribute. This attribute prevents the annotated loop from being automatically or fused when it is subject to the loop fusion functions.

Syntax

```
[[intel::nofusion]]
```

Example

For example, the following code samples have the same effect. If one loop in a pair is annotated with the `nofusion` attribute, the other loop has no other loop to fuse with.

<pre> [[intel::nofusion]] L1: for (int j = 0; j < N; ++j){ data[j] += Q; } L2: for (int i = 0; i < N; ++i) { output[i] = Q * data[i]; } </pre>	<pre> L1: for (int j = 0; j < N; ++j){ data[j] += Q; } [[intel::nofusion]] L2: for (int i = 0; i < N; ++i) { output[i] = Q * data[i]; } </pre>
--	--

In the following example, the compiler does not apply the loop fusion transformation to the loops:

```

for (int x = 0; x < N; x++) { // loop 1
    arr1_acc[x] = x;
}

[[intel::nofusion]]
for (int x = 0; x < N; x++) { // loop 2
    arr2_acc[x] = x;
}

```

Because you have applied the `nofusion` attribute to loop 2, the compiler cannot fuse loop 1 with loop 2.

Tip

For additional information, refer to the FPGA tutorial sample “Loop Fusion” on [GitHub](#).

See Also

[Fuse Loops to Reduce Overhead and Improve Performance](#)

[Fuse Adjacent Loops With Unequal Trip Counts \(-Xsenable-unequal-tc-fusion\)](#)

max_reinvocation_delay Attribute (Beta)

The loop reinvocation delay is the delay between launching the last iteration of a loop invocation and launching the first iteration of the next loop invocation. For interleaved loops, the loop reinvocation delay is redefined as the delay between launching the last iteration of a particular loop and launching the first iteration of the next loop invocation to enter the loop, which may not be the next loop invocation in the program order. This is due to interleaved loops having multiple invocations in the loop body, so the redefinition refers to when the loop body is fully occupied and when a loop invocation is exiting the loop body while a new loop invocation is ready to enter the loop body.

Use the `max_reinvocation_delay` attribute to direct the Intel® oneAPI DPC++/C++ Compiler to implement the loop that follows the attribute declaration with the specified maximum reinvocation delay value. This attribute restricts the maximum allowed delay between loop invocations for the implemented loop (without any stalls) and assumes a new invocation is ready to start executing.

Syntax

```
[[intel::max_reinvocation_delay(N)]]
```

The attribute parameter *N* is required and must be a positive constant expression of integer type. This parameter specifies the maximum number of clock cycles that you are willing to allow between launching the last iteration of a loop invocation and launching the first iteration of the next loop invocation. The higher the maximum reinvocation delay allowed, the longer the wait before the next loop invocation can start executing. However, the compiler can optimize the loop better if a high reinvocation delay is allowed in one of the following ways:

- Enable [loop speculation](#).
- Pipeline the [loop orchestration](#) hardware that can increase f_{MAX} .

NOTE

- The extra latency between invocations for a loop can be a significant factor in overall performance if the trip count of the loop is very small. In this case, specify the *N* value as 1 to allow subsequent loop invocations to start immediately, but at the cost of larger II and/or lower f_{MAX} to allow time to evaluate the exit condition.

```
for(int i = 0; i < maxI; i++) {
    int m = ...
    [[intel::max_reinvocation_delay(1)]]
    while(m*m*m < N) {
        m += 1;
    }
    dst[i] = m;
}
```

The inner loop in this example is implemented such that the first iteration of the $(i+1)^{th}$ invocation of the outer loop launches one cycle after the last iteration of the i^{th} invocation of the outer loop.

- The `max_reinvocation_delay` is a prototype loop attribute, and it currently supports only a value of $N=1$.
- For hyper-optimized loops, the minimum reinvocation delay must be equal to the loop II. The compiler may be unable to implement the loop with an appropriate II for the currently supported values of `max_reinvocation_delay` attribute.

Floating-Point Pragmas

Use `fp contract` and `fp reassociate` pragmas to influence the intermediate rounding and conversions of floating-point operations and the ordering of arithmetic operations in your kernel at finer granularity than the Intel® oneAPI DPC++/C++ Compiler flags.

Notice

Starting with the oneAPI 2021.2 release, fast math is enabled by default, allowing the Intel® oneAPI DPC++/C++ Compiler to make various out-of-box floating point math (`float` or `double`) optimizations. With these optimizations enabled, you might observe different bitwise results when compared to results from the oneAPI 2021.1 release or from GCC. The tradeoff is done to improve performance and area of your design. Automatic dot product inference and floating-point contraction for double precision math are two key noticeable FPGA optimizations that save a large amount of FPGA area and improve performance/latency. To return to the same precision as the oneAPI 2021.1 release or GCC, use the following compiler options:

- For Linux: `-no-fma -fp-model=precise`
- For Windows: `/Qfma- /fp:precise`

For more information about these options, refer to `-fp-model`, `fp` and `fma`, `Qfma` topics in the *Intel® oneAPI DPC++/C++ Compiler Developer Guide and Reference*.

NOTE

- You can place `fp contract` and `fp reassociate` pragma statements inside any compound statement (brace-enclosed sequences of statements), outside of all functions (file scope), or at the start of a function within the curly braces. For example:

```
{
    #pragma clang fp reassociate(on)
    float temp1 = 0.0f, temp2 = 0.0f;
    temp1 = accessorA[0] + accessorB[0];
    temp2 = accessorC[0] + accessorD[0];
    accessorRES[0] += temp1 * temp2;
}
```

- The `-fp-model=<value>` command option and the floating-point pragmas do not affect performance and area of designs implemented with `ap_float` data types.

fp contract Pragma

The `fp contract` pragma controls whether the compiler can skip intermediate rounding and conversions mainly between double precision arithmetic operations. If multiple occurrences of this pragma affect the same scope of your code, the pragma with the narrowest scope takes precedence.

Syntax

```
#pragma clang fp contract(fast|off|on)
```

where:

State	Description
<code>fast</code>	Allows the fusing of multiply and add instructions into an FMA, but it might violate the language standard. With fast math enabled by default, the <code>fp contract</code> pragma is set to <code>fast</code> by default. This allows the compiler to skip intermediate rounding and conversions mainly between double-precision arithmetic operations.
<code>off</code>	Prohibits the compiler from fusing multiple floating-point operations.
<code>on</code>	Fuses floating-point operations (multiply and add) within the same statement to form FMAs.

fp reassociate Pragma

The `fp reassociate` pragma controls the relaxing of the order of floating-point arithmetic operations within the code block that this pragma is applied to. With reordering, the compiler can optimize the hardware structure, which improves the performance of your kernel. If multiple occurrences of this pragma affect the same scope of your code, the pragma with the narrowest scope takes precedence.

Syntax

```
#pragma clang fp reassociate(on|off)
```

Where:

State	Description
on	Enables the compiler to reorder floating-point operations to improve performance and on-chip area. With fast math enabled by default, the <code>fp reassociate</code> pragma is set to on by default.
off	Prohibits the compiler from reordering floating-point operations to improve performance and on-chip area.

Latency Controls (Beta)

The Intel® oneAPI DPC++/C++ Compiler allows you to set latency constraints between operations with side effects, such as pipes and LSUs, which are visible outside the kernel. Specifically, you can apply latency controls to pipe read/write and LSU load/store.

For stallable operations, the scheduler considers only the inherent latency of the operation without making any assumption about the actual stall time. The compiler strives to achieve the latency constraints. If the requested latency cannot be achieved, even when ignoring the possibility of stalls, the compiler errors out.

Use Model

Caution

The APIs described in this section are experimental. Future versions of latency controls might change these APIs in ways that are incompatible with the version described here.

Latency control APIs are provided on `read()` and `write()` member functions of the `sycl::ext::intel::experimental::pipe` class, and on `load()` and `store()` member functions of the `sycl::ext::intel::experimental::lsu` class. Other than the latency controls support, the experimental pipe and LSU are identical to `sycl::ext::intel::pipe` and `sycl::ext::intel::lsu`, and the `<sycl/ext/intel/fpga_extensions.hpp>` header file provides experimental pipe and LSU. These member functions (`read()`, `write()`, `load()`, and `store()`) can accept a property list instance (`sycl::ext::oneapi::experimental::properties`) as a function argument, which can contain the following latency-control properties:

- `sycl::ext::intel::experimental::latency_anchor_id<N>`, where *N* is a signed integer:

A label that you can associate with pipes and LSU functions listed above. You can then reference this label using the `latency_constraint` property to define relative latency constraints. You can refer functions with this property as *labeled functions*.

- `sycl::ext::intel::experimental::latency_constraint<A, B, C>`:

A constraint that you can associate with pipes and LSU functions listed above. It provides a latency constraint between the function associated with the constraint and a different labeled function. You can refer functions with this property as *constrained functions*. The `latency_constraint` property has the following parameters:

- **A (signed integer)**: The label of the labeled function that is constrained relative to the constrained function.
- **B (enum value)**: The constraint type that can be one of the following:
 - `latency_control_type::exact` (exact latency)
 - `latency_control_type::max` (maximum latency)
 - `latency_control_type::min` (minimum latency)
- **C (signed integer)**: The relative clock cycle difference between the labeled function and constrained function that the constraint must infer, subject to the type of constraint (`exact`, `max`, or `min`).

Latency Controls and Stall-free Loops

In general, latency anchors and constraints may not span a loop because the number of cycles the loop takes to execute cannot be determined at compile time. Hence, it is not possible to satisfy the latency constraint.

An exception to this rule is a *stall-free loop*, which has a compile-time constant integer number of iterations and does not contain any stallable statements. These may include arithmetic operations and loads/stores to local stall-free memory. Stall-free loops execute in a compile-time known number of cycles.

Example of a stall-free loop:

```
float Value = <anything>;
float Result = 0;
for (int i = 0; i < 5; i++) // compile time integer number of iterations
    Result *= (i+Value);    // arithmetic calculation that cannot stall
// Use Result
```

Example 1: Using Latency Controls on Pipes

The following example shows how to use latency controls on pipes. The example uses a function behaving both as a labeled function and a constrained function:

```
using Pipe1 = sycl::ext::intel::experimental::pipe<class PipeClass1, int, 8>;
using Pipe2 = sycl::ext::intel::experimental::pipe<class PipeClass2, int, 8>;
using Pipe3 = sycl::ext::intel::experimental::pipe<class PipeClass2, int, 8>;
...
// In kernel:
// The following read has a label 0.
int value = Pipe1::read(sycl::ext::oneapi::experimental::properties(
    sycl::ext::intel::experimental::latency_anchor_id<0>));

// The following write occurs exactly 2 cycles after the label-0 function, i.e.,
// the read above. Also, it has a label 1.
Pipe2::write(
    value,
    sycl::ext::oneapi::experimental::properties(
        sycl::ext::intel::experimental::latency_anchor_id<1>,
        sycl::ext::intel::experimental::latency_constraint<
            0, sycl::ext::intel::experimental::latency_control_type::exact,
            2>));

// The following write occurs at least 2 cycles after the label-1 function,
// i.e., the write above.
Pipe3::write(
    value,
```

```
sycl::ext::oneapi::experimental::properties(
    sycl::ext::intel::experimental::latency_constraint<
        1, sycl::ext::intel::experimental::latency_control_type::min, 2>));
```

Example 2: Using Latency Controls on LSUs

The following example shows how to use latency controls on LSUs. It uses a negative relative cycle number in the `latency_constraint` property, which means that the constrained function is scheduled before the associated labeled function:

```
using BurstCoalescedLSU = sycl::ext::intel::experimental::lsu<
    sycl::ext::intel::experimental::burst_coalesce<false>,
    sycl::ext::intel::experimental::statically_coalesce<false>>;
...
// In kernel:
// The following load occurs at most 5 cycles before the label-2 function,
// i.e., the store below.
int value = BurstCoalescedLSU::load(
    input_ptr,
    sycl::ext::oneapi::experimental::properties(
        sycl::ext::intel::experimental::latency_constraint<
            2, sycl::ext::intel::experimental::latency_control_type::max, -5>));

// The following store has a label 2.
BurstCoalescedLSU::store(
    output_ptr, value,
    sycl::ext::oneapi::experimental::properties(
        sycl::ext::intel::experimental::latency_anchor_id<2>));
```

Using Latency Controls on LSUs With a Stall-free Loop

The following example shows how to use latency controls on LSUs with a stall-free loop. It uses a negative relative cycle number in the `latency_constraint` property, which means that the constrained function is scheduled before the associated labeled function:

```
using BurstCoalescedLSU = sycl::ext::intel::experimental::lsu<
    sycl::ext::intel::experimental::burst_coalesce<false>,
    sycl::ext::intel::experimental::statically_coalesce<false>>;
...
// In kernel:
// The following load occurs at most 25 cycles before the label-2 function,
// i.e., the store below.
int value = BurstCoalescedLSU::load(
    input_ptr,
    sycl::ext::oneapi::experimental::properties(
        sycl::ext::intel::experimental::latency_constraint<
            2, sycl::ext::intel::experimental::latency_control_type::max, -25>));

float Result = 0;
for (int i = 0; i < 5; i++)
    Result *= (Result-Value);
// The following store has a label 2.
BurstCoalescedLSU::store(
    output_ptr, Result,
    sycl::ext::oneapi::experimental::properties(
        sycl::ext::intel::experimental::latency_anchor_id<2>));
```

Rules and Limitations

- `latency_anchor_id` must be a non-negative number.
- `latency_anchor_id` must be a unique number within the whole design.
- The labeled function and constrained function of a constraint must meet one of the following conditions:
 - Both functions are in the same block but not in any cluster
 - Both functions are in the same cluster.

The compiler attempts to achieve latency constraints. However, it errors out if some constraints cannot be satisfied. For example, if one constraint specifies function A must be scheduled after function B, while another constraint specifies function B must be scheduled after function A, then that set of constraints is unsatisfiable.

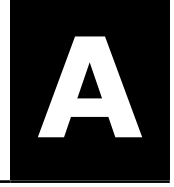
Tip

For additional information, refer to the FPGA tutorial sample “Latency Control” on [GitHub](#).

See Also

[Pipe API](#)

[Load-Store Unit Controls](#)



Quick Reference

This section provides a quick reference list of all FPGA-specific attributes, pragmas, and variables.

Algorithmic C Data Types

The following table summarizes the algorithmic C (AC) data types:

Algorithmic C Data Types Supported by the Intel® oneAPI DPC++/C++ Compiler

Data Type	Header File	Variable Declaration	Description
ac_int	<sycl/ext/intel/ ac_types/ ac_int.hpp>	- Template-based declaration: <pre>ac_int<N, true> var_name; //Signed N-bit integer ac_int<N, false> var_name; //Unsigned N-bit integer</pre> - Predefined types up to 63 bits: <pre>ac_intN::intN var_name; // Signed N-bit integer ac_intN::uintN var_name; //Unsigned N-bit integer</pre>	Arbitrary-precision integer support
ac_fixed	<sycl/ext/intel/ ac_types/ ac_fixed.hpp>	<pre>ac_fixed<W, I, S, Q, O> var_name;</pre> For a description of the template parameters, refer to Declare the ac_fixed Data Type.	Arbitrary-precision fixed-point number support. For math functions supported by this data type, refer to Math Functions Provided by the ac_fixed_math.hpp Header File .
ac_complex	<sycl/ext/intel/ ac_types/ ac_complex.hpp>	Declare according to the data type of your complex number.	Complex number support
ap_float	<sycl/ext/intel/ ac_types/ ap_float.hpp>	<pre>ihc::ap_float<exponent_width , mantissa_width[,rounding_mod e]></pre>	Arbitrary-precision floating-point number support. For math functions supported by this data type, refer to Math Functions Provided by ap_float_math.hpp Header File

Compilation Flags

Compilation Flags for Data Types

AC Data Type	SYCL Command Flags	Description
Any	Linux: -qactypes Windows: /Qactypes	Use these flags to include ac_types header files on the include path and link against AC type libraries required for the host device execution support.

AC Data Type	SYCL Command Flags	Description
ap_float	-DFPGA_EMULATOR • Linux: -fp-model=precise -no-fma • Windows: /fp:precise / Qfma-	Compiles programs with an AC data type for emulation. Use these flags to ensure that floating-point operations are accurate.

Explicit Conversion Functions

The following table lists the functions to convert to C signed and unsigned integer types `int`, `long` and `slong` for the `ac_int` and `ac_fixed` data types:

AC Data Type	Supported Conversion Functions
<code>ac_int</code>	<code>to_int()</code> , <code>to_uint()</code> , <code>to_long()</code> , <code>to_ulong()</code> , <code>to_int64()</code> , <code>to_uint64()</code> , <code>to_double()</code>
<code>ac_fixed</code>	<code>to_int()</code> , <code>to_uint()</code> , <code>to_long()</code> , <code>to_ulong()</code> , <code>to_int64()</code> , <code>to_uint64()</code> , <code>to_double()</code> , <code>to_float()</code> , <code>to_ac_int()</code>

Math Functions Provided by the `ac_fixed_math.hpp` Header File

The `ac_fixed_math.hpp` header file adds support for the following non-standard math functions for the arbitrary precision fixed-point (`ac_fixed`) data type:

- `sqrt_fixed`
- `reciprocal_fixed`
- `reciprocal_sqrt_fixed`
- `sin_fixed`
- `cos_fixed`
- `sincos_fixed`
- `sinpi_fixed`
- `cospi_fixed`
- `sincospi_fixed`
- `log_fixed`
- `exp_fixed`

Math Functions Provided by `ap_float_math.hpp` Header File

The `ap_float_math.hpp` header file adds support for the following arbitrary precision fixed-point (`ap_float`) data type functions:

Math Functions Provided by `ap_float_math.hpp` Header File

Function Type	Math Functions	Comment
Exponential and logarithmic functions	• <code>ln</code> • <code>log₂</code>, <code>log₁₀</code> • <code>e^x</code>, <code>2^x</code>, <code>10^x</code>	Supported only for <code>ap_float</code> data types with exponent width less than or equal to 15 bits and mantissa width less than or equal to 63 bits.
	• <code>ln(1+x)</code> • <code>e^x-1</code>	Supported only for <code>ap_float</code> data types with exponent width less than or equal to 11 bits and mantissa width less than or equal to 52 bits.
Advanced functions	• <code>reciprocal</code>	

Function Type	Math Functions	Comment
Power functions	- reciprocal_sqrt - sqrt - cube root - hypot (hypotenuse)	
	- pow - powr - pown	
	- sin, cos, sincos - sinpi, cospi - asin, asinpi - acos, acospi - atan, atanpi, atan2	

Floating Point Pragmas

The following table summarizes the floating point pragmas:

Pragma	Description	Example
<code>fp contract(off fast)</code>	Controls whether the compiler can skip intermediate rounding and conversions mainly between double precision arithmetic operations.	<pre>{ #pragma clang fp contract(fast) float temp1 = 0.0f, temp2 = 0.0f; temp1 = accessorA[0] + accessorB[0]; temp2 = accessorC[0] + accessorD[0]; accessorRES[0] += temp1 * temp2; }</pre>
<code>fp reassociate(on off)</code>	Controls the relaxing of the order of floating-point arithmetic operations within the code block that this pragma is applied to.	<pre>{ #pragma clang fp reassociate(on) float temp1 = 0.0f, temp2 = 0.0f; temp1 = accessorA[0] + accessorB[0]; temp2 = accessorC[0] + accessorD[0]; accessorRES[0] += temp1 * temp2; }</pre>

FPGA Accessor Properties

The following table summarizes FPGA accessor properties:

FPGA Accessor Properties

Property	Description	Example
<code>buffer_location<index></code>	Instructs the host to allocate a buffer to a specific global memory type. It identifies the index of the global memory type in the <code>board_spec.xml</code> file of your Custom Platform. If there is no global memory marked with "default=1" in the <code>board_spec.xml</code> file, then the index starts at 0 and follows the order in which the global memory appears in	<pre>ext::oneapi::accessor_p property_list PL{ext::intel::buffer_l ocation<2>}; accessor accessor(buffer, cgh, read_only, PL);</pre>

Property	Description	Example
	the <code>board_spec.xml</code> file. Otherwise, index 0 refers to the default memory and indices 1 and onwards follow the order in which the remaining global memories appear in the <code>board_spec.xml</code> file.	
	If you do not specify the <code>buffer_location</code> property, the host allocates the buffer to the default memory type automatically.	
<code>no_alias</code>	Notifies the compiler that all modifications to the memory locations accessed (directly or indirectly) by an accessor during kernel execution is done through the same accessor (directly or indirectly) and not by any other accessor or USM pointer in the kernel. This is an unchecked assertion by the programmer and results in an undefined behavior if it is violated.	<pre>ext::oneapi::accessor_p property_list PL{ext::oneapi::no_alias}; accessor accessor(buffer, cgh, read_only, PL);</pre>
<code>no_offset</code>	Notifies the compiler that the accessor will never contain an offset. This can enable the compiler to make assumptions about the alignment of the accessor that it could not make otherwise.	<pre>ext::oneapi::accessor_p property_list PL{ext::oneapi::property::no_offset}; accessor accessor(buffer, cgh, read_only, PL);</pre>

FPGA Extensions

The following sections summarize FPGA extensions supported:

The `device_global` Extension

`device_global` Extension

FPGA Extension	Description	Example
<code>sycl::ext::oneapi::experimental::device_global()</code>	Introduces device-scoped memory allocations into SYCL that can be accessed within a kernel using C++ global variable syntax. These memory allocations have unique instances per SYCL device.	<pre>namespace exp = sycl::ext::oneapi::experimental; using FPGAProperties = decltype(exp::properties(exp::device_image_scope, exp::host_access_none)); exp::device_global<int, FPGAProperties> val;</pre>

The `fpga_datapath` Extension

`fpga_datapath` Extension

FPGA Extension	Description	Example
<code>sycl::ext::intel::experimental::fpga_datapath<T></code>	The <code>fpga_datapath</code> extension is a class template templated on a type <code>T</code>, that represents an object of type <code>T</code>. It is a request to the compiler to implement that object in the datapath instead of an off-datapath memory. The <code>fpga_datapath</code> template has reference wrapper-like semantics, and is implicitly convertible to the wrapped type. Because C++ doesn't allow for overloading of the dot operator, a <code>get()</code> member of <code>fpga_datapath</code> allows the extractions of a reference to which the usual dot operator can be applied. The <code>fpga_datapath</code> class template is functionally equivalent to the <code>fpga_register memory</code> attribute.	<pre>struct MyClass { bool x; }; namespace intexp = sycl::ext::intel::experimental; sycl::queue q; q.single_task([=] { intexp::fpga_datapath<int[4]> fd1{1, 3, 5, 7}; intexp::fpga_datapath<MyClass> fd2; fm2.get().x = fm1[0]; });</pre>

The `fpga_reg` Extension

`fpga_reg` Extension

FPGA Extension	Description	Example
<code>ext::intel::fpga_reg()</code>	Helps the compiler infer at least one pipelining register in the datapath.	<pre>#include <sycl/ext/intel/ fpga_extensions.hpp> r[k] = ext::intel::fpga_reg(a[k]) + b[k];</pre>

The `task_sequence` Extension

The following table summarizes the `task_sequence` template parameters and function APIs:

`task_sequence` Template Parameters

Template Parameter	Description
<code>auto &f typename ReturnT, typename... ArgsT, ReturnT (&f)(ArgsT...)</code>	Callable <code>f</code> that defines the asynchronous task to be associated with the <code>task_sequence</code> .
<code>uint32_t invocation_capacity</code>	The size of the hardware queue instantiated for <code>async()</code> function calls.
<code>uint32_t response_capacity</code>	The size of the hardware queue instantiated to hold task function results.

`task_sequence` Function APIs

Function API	Description
<code>void async(ArgsT... Args)</code>	Asynchronously calls <code>f</code> with <code>Args</code> . Increments the number of outstanding tasks by 1.

Function API	Description
ReturnT get()	Synchronously retrieves the result of an asynchronous call.
~task_sequence()	Destructor for task_sequence.

FPGA Kernel Attributes

The following table summarizes kernel attributes:

FPGA Kernel Attributes

Attribute	Description	Example
<code>[[intel::scheduler_target_fmax_mhz(N)]]</code>	Determines the pipelining effort the scheduler attempts during the scheduling process.	<pre>[[intel::scheduler_target_fmax_mhz(SCHEDULER_TARGET_FMAX)]] { for (unsigned i = 0; i < SIZE; i++) { accessorRes[0] += accessorIdx[i] * 2; } });</pre>
<code>[[intel::max_work_group_size(Z, Y, X)]]</code>	Specifies a maximum or the required work-group size for optimizing hardware use of the SYCL kernel without involving excess logic.	<pre>[[intel::max_work_group_size(1, 1,MAX_WG_SIZE)]] { accessorRes[wiID] = accessorIdx[wiID] * 2; });</pre>
<code>[[intel::max_global_work_dim(0)]]</code>	This attribute is deprecated. The compiler automatically adds this attribute for any single-task kernel, so adding this attribute explicitly is no longer required. Omits logic that generates and dispatches global, local, and group IDs into the compiled kernel.	<pre>[[intel::max_global_work_dim(0)]] { for (unsigned i = 0; i < SIZE; i++) { accessorRes[i] = accessorIdx[i] * 2; } }</pre>
<code>[[intel::num_simd_work_items(N)]]</code>	Specifies the number of work items within a work group that the compiler executes in a SIMD or vectorized manner.	<pre>[[intel::num_simd_work_items(NUM_SIMD_WORK_ITEMS), cl::reqd_work_group_size(1,1,REQD_WORK_GROUP_SIZE)]] { accessorRes[wiID] = sqrt(accessorIdx[wiID]); });</pre>
<code>[[intel::no_global_work_offset(1)]]</code>	Omits generating hardware required to support global work offsets.	<pre>[[intel::no_global_work_offset(1)]] { accessorRes[wiID] = accessorIdx[wiID] * 2; }</pre>

Attribute	Description	Example
<code>[[intel::kernel_args_restrict]]</code>	Ignores the dependencies between accessor arguments in a SYCL* kernel.	<pre>[[intel::kernel_args_restrict]] { for (unsigned i = 0; i < size; i++) { out_accessor[i] = in_accessor[i]; } };</pre>
<code>[[intel::use_stall_enable_clusters]]</code>	Reduces the area and latency of your kernel.	<pre>h.single_task<class KernelComputeStallFree>([=] () [[intel::use_stall_enable_clusters]] { // The computations in this device kernel // uses Stall Enable Clusters Work(accessor_vec_a, accessor_vec_b, accessor_res); }); });</pre>

FPGA Local Memory Function

The following table summarizes the local memory function:

Local Memory Function

Function	Description
<pre>template <typename T, typename Group> multi_ptr<T, Group::address_space> group_local_memory_for_overwrite (Group g)</pre>	Constructs an object of type <code>T</code> in an address space accessible by all work items in the workgroup <code>g</code>, using the default initialization. The object is initialized upon or before the first call to the <code>group_local_memory_for_overwrite</code> function. The storage for the object is allocated upon or before the first call to the <code>group_local_memory_for_overwrite</code> function, and deallocated when all work items in the workgroup have completed executing the kernel. All arguments in <code>args</code> must be the same for all work items in the group. Group must be <code>sycl::group</code>, and <code>T</code> must be trivially destructible.

Latency Control Properties (Beta)

The following table summarizes the latency control properties:

Latency Control Properties

Argument	Description	Example
<code>sycl::ext::intel::experimental::latency_anchor_id<N></code>	A label that you can associate with pipes and LSU functions	<pre>// The following write occurs exactly 2 cycles after the label-0 function, i.e., // the read above. Also, it has a label 1. Pipe2::write(</pre>

Argument	Description	Example
<code>sycl::ext::intel::experimental::latency_constraint<A, B, C></code>	A constraint that you can associate with pipes and LSU functions	<pre> value, sycl::ext::oneapi::experimental::properties(sycl::ext::intel::experimental::latency_anchor_id<1>, sycl::ext::intel::experimental::latency_constraint< 0, sycl::ext::intel::experimental::latency_control_type::exact, 2>)); </pre>

NOTE

Latency control APIs are provided on `read()` and `write()` member functions of the `sycl::ext::intel::experimental::pipe` class, and on `load()` and `store()` member functions of the `sycl::ext::intel::experimental::lsu` class. Other than the latency controls support, the experimental pipe and LSU are identical to `sycl::ext::intel::pipe` and `sycl::ext::intel::lsu`, and the `<sycl/ext/intel/fpga_extensions.hpp>` header file provides experimental pipe and LSU. These member functions (`read()`, `write()`, `load()`, and `store()`) can accept a property list instance (`sycl::ext::oneapi::experimental::properties`) as a function argument.

For detailed information about the variables, refer to [Latency Controls \(Beta\)](#).

FPGA LSU Controls

The following table summarizes the load-store unit (LSU) control:

LSU Controls

Syntax	Description	Example
<code>ext::intel::lsu<...>;</code>	Allows you to specify LSU attributes to control the LSU inferred by the compiler. No attributes attempt to infer a pipelined LSU. For example: <code>ext::intel::burst_coalesce<true></code> <code>ext::intel::statically_coalesce<false></code> <code>ext::intel::prefetch<true></code> <code>ext::intel::cache<1024></code>	<pre> #include <sycl/ext/intel/fpga_extensions.hpp> using namespace sycl ... using BurstCoalescedLSU = ext::intel::lsu<ext::intel::burst_coalesce<true>, ext::intel::statically_coalesce<false>>; BurstCoalescedLSU::store(output_ptr, X); </pre>

FPGA Loop Directives

The following table summarizes loop directives:

FPGA Loop Directives

Directive (Pragma, Attribute, or Function)	Description	Example
<code>disable_loop_pipelining</code>	Directs the Intel® oneAPI DPC++/C++ Compiler to disable pipelining of a loop.	<pre>[[intel::disable_loop_pipelining]] for (int i = 1; i < N; i++) { int j = a[i-1]; // Memory dependency induces // a high-latency loop feedback path a[i] = foo(j) }</pre>
<code>initiation_interval</code>	Forces a loop to have a loop initialization interval (II) of a specified value.	<pre>// ii set to 5 [[intel::initiation_interval(5)]] for (int i = 0; i < N; ++i){ }</pre>
<code>ivdep</code>	Ignores memory dependencies between iterations of this loop. Applying the <code>ivdep</code> attribute to a variable that is used in a lambda function or a variable that is passed as a function argument can result in functional failures in your kernel.	<pre>// ivdep loop [[intel::ivdep]] for (...) {} //ivdep safelen [[intel::ivdep(safelen)]] for (;;) {} // ivdep accessor [[intel::ivdep(accessorA)]] for (;;) {} //ivdep array safelen [[intel::ivdep(accessorA, safelen)]] for (;;) {}</pre>
<code>loop_coalesce</code>	Coalesces nested loops into a single loop without affecting the loop functionality.	<pre>[[intel::loop_coalesce(2)]] for (int i = 0; i < N; i++) for (int j = 0;</pre>

Directive (Pragma, Attribute, or Function)	Description	Example
		<pre>j < M; j++) sum[i][j] += i +j;</pre>
<code>max_concurrency</code>	Limits the number of iterations of a loop that can simultaneously execute at any time.	<pre>//max concurrency set to 1 [[intel::max_concu rrency(1)]] for (int i = 0; i < c; ++i){ }</pre>
<code>max_interleaving</code>	Maximizes the throughput and hardware resource occupancy of pipelined inner loops in a loop nest.	<pre>// Loop j is pipelined with ii=1 for (int j = 0; j < M; j++) { int a[N]; // Loop i is pipelined with ii=2 [[intel::max_inter leaving(1)]] for (int i = 1; i < N; i++) { a[i] = foo(i) } ... }</pre>
<code>speculated_iterations</code>	Improves the performance of pipelined loops.	<pre>[[intel::speculate d_iterations(1)]] while (m*m*m < N) { m += 1; } dst[0] = m;</pre>
<code>unroll</code>	Unrolls a loop in the kernel code.	<pre>// unroll factor N set to 2 #pragma unroll 2 for(size_t k = 0; k < 4; k++){ mac += data_in[(gid * 4) + k] * coeff[k]; }</pre>
<code>nofusion</code>	Prevents the compiler from fusing the annotated loop with any of the adjacent loops.	<pre>for (int x = 0; x < N; x++) { al_acc[x] = x;</pre>

Directive (Pragma, Attribute, or Function)	Description	Example
		<pre> } [[intel::nofusion]] for (int x = 0; x < N; x++) { a2_acc[x] = x; } </pre>
<code>sycl::ext::intel::fpga_loop_fuse<v>(f)</code>	Fuses loops within the function <code>f</code> up to a depth of <code>v >= 1</code> , where <code>v = 1</code> by default.	<pre> [=] () [[intel::kernel_arguments_restrict]] { sycl::ext::intel:: fpga_loop_fuse<v>{ for (int x = 0; x < N; x++) { for (int y = 0; y < N; y++) { for (int z = 0; z < N; z+ +) { a1_acc[x][y][z] = 0; } } } for (int x = 0; x < N + 1; x+ +) { for (int y = 0; y < N + 1; y+ +) { for (int z = 0; z < N + 1; z++) { a2_acc[x][y][z] = 0; } } } } </pre>
<code>sycl::ext::intel::fpga_loop_fuse<v><v>(f)</code>	Fuses loops within the function <code>f</code> up to a depth <code>v >= 1</code> while overriding fusion-safety checks. Here, <code>v = 1</code> by default.	<pre> [=] () { //Kernel sycl::ext::intel:: fpga_loop_fuse_inde pendent([&] { for(int x = 0; x < N; x++){ </pre>

Directive (Pragma, Attribute, or Function)	Description	Example
		<pre> a3_acc[x] = x; } for(int x = 0; x < N + 1; x++) { a4_acc[x] = x; } }); } </pre>

FPGA Memory Attributes

The following table summarizes memory attributes:

FPGA Memory Attributes

Attribute	Description	Example
bank_bits	Specifies that the local memory addresses should use bits for bank selection.	<pre> // Array is implemented with 4 banks // where // bits 6 and 5 of the memory word // address // are used to select between the banks [[intel::bank_bits(6,5)]] int array[128]; </pre>
bankwidth	Specifies that the memory implementing the variable or array must have memory banks of a defined width.	<pre> // Each memory bank is 8 bytes (64- // bits) wide [[intel::bankwidth(8)]] int array[128]; </pre>
doublepump	Specifies that the memory implementing the variable, or an array must be clocked at twice the rate as the kernel accessing it.	<pre> // Array is implemented in a memory that // operates at // twice the clock frequency of the // kernel [[intel::doublepump, bankwidth(128)]] int array[128]; </pre>
force_pow2_depth	Specifies that the memory implementing the variable or array has a power-of-2 depth.	<pre> // array1 is implemented in a memory // with depth 1536 [[intel::force_pow2_depth(0)]] int array1[1536]; </pre>
max_replicates	Specifies that the memory implementing the variable, or an array has no more than the specified number of replicates to enable simultaneous accesses from the datapath.	<pre> // Array is implemented in a memory with // maximum four // replicates [[intel::max_replicates(4)]] int array[128]; </pre>

Attribute	Description	Example
<code>fpga_memory</code>	Forces a variable or an array to be implemented as an embedded memory.	<pre>// Array is implemented in memory (MLAB/ M20K), // the actual implementation is automatically decided // by the compiler [[intel::fpga_memory]] int array1[128]; // Array is implemented in M20K [[intel::fpga_memory("BLOCK_RAM")]] int array2[64]; // Array is implemented in MLAB [[intel::fpga_memory("MLAB")]] int array3[64];</pre>
<code>merge</code>	Allows merging of two or more variables or arrays defined in the same scope with respect to width or depth.	<pre>// Both arrays are merged width-wise and implemented // in the same memory system [[intel::merge("mem", "width")]] short arrayA[128]; [[intel::merge("mem", "width")]] short arrayB[128];</pre>
<code>numbanks</code>	Specifies that the memory implementing the variable or array must have a defined number of memory banks.	<pre>// Array is implemented with 2 banks [[intel::numbanks(2)]] int array[128];</pre>
<code>private_copies</code>	Specifies that the memory implementing the variable, or an array has no more than the specified number of independent copies to enable concurrent thread or loop iteration accesses.	<pre>// Array is implemented in a memory with two // private copies [[intel::private_copies(2)]] int array[128];</pre>
<code>fpga_register</code>	Forces a variable or an array to be carried through the pipeline in registers. The <code>fpga_register</code> memory attribute is functionally equivalent to the <code>fpga_datapath</code> class template .	<pre>// Array is implemented in register [[intel::fpga_register]] int array[128];</pre>
<code>simple_dual_port</code>	Specifies that the memory implementing the variable or array should have no port that serves both reads and writes.	<pre>// Array is implemented in a memory such that no // single port serves both a read and a write [[intel::simple_dual_port]] int array[128];</pre>
<code>singlepump</code>	Specifies that the memory implementing the variable or array must be clocked at the same rate as the kernel accessing it.	<pre>// Array is implemented in a memory that operates // at the same clock frequency as the kernel [[intel::singlepump]] int array[128];</pre>

FPGA Optimization Flags

The following table summarizes FPGA optimization flags:

FPGA Optimization Flags

Flags	Description	Example
-Xsclock=<clock target in Hz/KHz/MHz/GHz or s/ms/us/ns/ps>	Schedules f_{MAX} target for kernels.	icpx -fintelfpga -Xshardware -Xsclock=<clock target> <source_file>.cpp
-Xsuse-2xclock	Explicitly creates a 2xclock interface for the given design.	icpx -fintelfpga -Xshardware -Xsuse-2xclock source_file.cpp
-Xsno-interleaving=<global_memory_name>	Disables burst-interleaving for all global memory banks of the same type and manages them manually	icpx -fintelfpga -Xshardware <source_file>.cpp-Xsno-interleaving=DDR
-Xsglobal-ring	Forces ring interconnect for global memory.	icpx -fintelfpga -Xshardware -Xsglobal-ring <source_file>.cpp
-Xsforce-single-store-ring	Narrows the interconnect to save area while limiting write-only throughput to one bank's worth.	icpx -fintelfpga -Xshardware -Xsforce-single-store-ring <source_file>.cpp
-Xsnum-reorder	Narrows the interconnect to save area while reducing read-only throughput.	icpx -fintelfpga -Xshardware -Xsnum-reorder=1 <source_file>.cpp
-Xsno-hardware-kernel-invocation-queue	Reduces kernel area use by removing kernel invocation queue in SYCL* kernel.	icpx -fintelfpga -Xshardware -Xsno-hardware-kernel-invocation-queue <source_file>.cpp
-Xshyper-optimized-handshaking=<auto off>	Modifies the handshaking protocol used in certain areas of the design	icpx -fintelfpga -Xshardware -Xshyper-optimized-handshaking=auto <source_file>.cpp
		icpx -fintelfpga -Xshardware -Xshyper-optimized-handshaking=off <source_file>.cpp
-Xsdisable-auto-loop-fusion	Disables the automatic fusion of loops when compiling the design.	icpx -fintelfpga -Xshardware -Xsdisable-auto-loop-fusion <source_file>.cpp
-Xsenable-unequal-tc-fusion	Fuses adjacent loops with unequal trip counts into a single loop without affecting either loop's functionality.	icpx -fintelfpga -Xshardware -Xsenable-unequal-tc-fusion <source_file>.cpp
-Xsauto-pipeline	Pipelines loops in non-task (parallel_for) kernels.	icpx -fintelfpga -Xshardware -Xsauto-pipeline <source_file>.cpp
-fp-model=<value>	Controls the semantics of floating-point operations.	icpx -fintelfpga -Xshardware -fp-model=<value> <source_file>.cpp
-Xsrounding=<rounding_type>	Modifies the rounding mode of floating-point elementary operations in your design.	icpx -fintelfpga -Xshardware -Xsrounding=ieee <source_file>.cpp

Flags	Description	Example
		<code>icpx -fintelfpga -Xshardware -Xsrounding=faithful <source_file>.cpp</code>
<code>-Xssfc-exit-fifo-type= <default zero-latency low-latency></code>	Globally controls exit FIFO latency of stall-free clusters using the specified exit FIFO type.	<code>icpx -fintelfpga -Xshardware -Xssfc-exit-fifo-type=zero-latency <source_file>.cpp</code>
<code>-Xsread-only-cache-size=<N></code>	Enables read-only cache and sets its size to <N> bytes.	<code>icpx -fintelfpga -Xshardware -Xsread-only-cache-size=<N><source_file>.cpp</code>
<code>-Xsdsp-mode=[default prefer-dsp prefer-softlogic]</code>	Controls the hardware implementation of the supported data types and math functions of all kernels in your source code.	<code>icpx -fintelfpga -Xshardware -Xsdsp-mode=<option> <source_file>.cpp</code>
<code>-Xsregister-map-wrapper-type</code>	Generates a ring-like wrapper structure to connect all register map interfaces for different kernels inside an IP core.	<code>icpx -fintelfpga -Xshardware -Xsregister-map-wrapper-type=<default high-fmax low-latency> source_file.cpp</code>

Pipe API

The following table summarizes the pipe API:

Pipe API

API	Description	Example
<code>ext::intel::pipe</code>	Use pipes to transfer data between kernels directly using on-device FIFO buffers. Pipes can either be non-blocking or blocking.	<pre>using my_pipe = ext::intel::pipe<class some_pipe, int>; ... my_pipe::write(rd_src_buf[i]); ... auto data = my_pipe::read(); ...</pre>



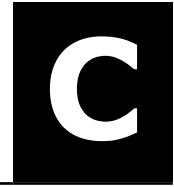
Additional Information

For additional information, refer to the following resources:

Resource	Description
Intel® oneAPI Base Toolkit	Main landing page of the Intel® oneAPI Base Toolkit, which includes the Intel® oneAPI DPC++/C++ Compiler and provides tools and libraries for developing high-performance, data-centric applications across diverse architectures.
FPGA Support Package for the Intel® oneAPI DPC++/C++ Compiler	Main landing page for the FPGA Support Package for the Intel® oneAPI DPC++/C++ Compiler.
FPGA Tutorials on Github*	Refer to these tutorials for more in-depth instructions about how to use the FPGA tutorials.
FPGA oneAPI Training	Training site with webinars and quick videos.
Data Parallel C++: Programming Accelerated Systems Using C++ and SYCL	A third-party open-access book to learn how to accelerate C++ programs using data parallelism. It is full of practical advice, detailed explanations, and code examples to illustrate key topics.
Migrating OpenCL FPGA Designs to SYCL*	Provides guidelines to migrate your OpenCL FPGA designs to SYCL.
Get Started with Intel® Distribution for GDB* on Linux* OS Host	Provides instructions for using Intel® Distribution for GDB* for debugging SYCL and OpenCL™ applications.
Intel® oneAPI DPC++/C++ Compiler Developer Guide and Reference	Provides information about the Intel® oneAPI DPC++/C++ Compiler (icx-cl/icpx) and runtime environment.
Intel® oneAPI Programming Guide	Describes the oneAPI programming model in detail, including a brief overview of the FPGA flows.
Intel® VTune™ Profiler User Guide	Provides a comprehensive overview of the product functionality, tuning methodologies, workflows, and instructions to use Intel® VTune Profiler performance analysis tool.
Analyzing CPU and FPGA (Arria® 10 GX) Interaction	Provides instructions for configuring your platform to analyze an interaction of your CPU and FPGA using Arria® 10 GX FPGA as an example.
Profiling an FPGA-driven SYCL Application	Provides instructions for profiling an FPGA-driven SYCL application.
oneAPI Accelerator Support Package (ASP): Getting Started User Guide	Describes how to get up and running with the Intel® oneAPI Base Toolkit (Base Kit) and the Open FPGA Stack (OFS).
oneAPI Accelerator Support Package (ASP) Reference Manual: Open FPGA Stack	Provides information on the hardware & software components in the oneAPI Open FPGA Stack (OFS) ASP.

Resource	Description
Quartus® Prime Software User Guides	Provides links to various Quartus® Prime user guides, which cover specific topics to help you see your design through to completion.

Document Revision History for the Intel® oneAPI DPC++/C++ Compiler Handbook for Intel FPGAs



Date	Release Version	Changes
May 2025	2025.0	- Revised <i>Installing the Intel® oneAPI FPGA Development Environment</i>. - Revised <i>FPGA Development for Intel® oneAPI Toolkits with Visual Studio* Code</i> - Changed links to the oneAPI FPGA tutorials. These tutorials are now posted at the Altera Developer Site on GitHub.
October 2024	2025.0	Maintenance release
June 2024	2024.2	- Revised code examples in <i>Optimize Inner Loop Throughput</i> - Added "Filtering Nodes in Report Tables" to <i>Review the FPGA Optimization Report</i> - Revised <i>Reduce Kernel Area and Latency</i> (<i>use_stall_enable_clusters</i>). - Revised <i>Declare the ap_float Data Type</i>. - Revised <i>Host Pipes RTL Interfaces</i>. - Revised <i>Partitioning Buffers Across Memory Channels of the Same Memory Type</i>. - Added the <i>no_offset</i> property to <i>FPGA Accessor Properties</i>. - Revised <i>The pipe Class and its Use</i>. - Revised <i>latency</i> property description in <i>The annotated_arg Template Class</i> and <i>Memory-Mapped Host Interfaces Using Unified Shared Memory Pointers and the annotated_arg Class</i> - Revised <i>fpga_register</i> description in <i>Memory Attributes</i> - Added <i>Structs in RTL IP Core Interfaces</i> - Revised <i>Pipelined Kernels</i>. - Revised <i>ivdep Attribute</i>. - Removed references to the <code>--target sycl</code> and <code>--source sycl</code> options of the <code>fpga_crossgen</code> and <code>fpga_libtool</code> commands. These options are no longer required and have been removed from the commands. - Replaced occurrences of <code><project_name>_di_inst.v</code> with <code><project_name>_di_inst.sv</code>. - Added <i>Allow Wide Memory Initialization (-Xsallow-wide-device-globals)</i>. - Revised references to <code>icpx -fsycl -fintel-fpga</code> commands to <code>icpx -fintel-fpga</code>. The <code>-fintel-fpga</code> option includes the <code>-fsycl</code> option, so specifying the <code>-fsycl</code> option is redundant. - Added <i>Managing an FPGA Board</i>. - Added <i>Linking Your Host Application to the Khronos ICD Loader Library</i>.

Date	Release Version	Changes
May 2024	2024.1	Fix broken links
March 2024	2024.1	- Changed the title of this document to <i>Intel® oneAPI DPC++/C++ Compiler Handbook for Intel FPGAs</i>. - Added <i>The annotated_ptr Template Class</i>. - Added <i>Getting Started with Intel® oneAPI FPGA Development</i> chapter. This chapter includes the following sections: <i>Installing the Intel® oneAPI FPGA Development Environment</i> <i>FPGA Development for Intel® oneAPI Toolkits with Visual Studio* Code</i> This section includes content that was previously provided in FPGA Development for Intel® oneAPI Toolkits with Visual Studio Code on Linux*. This section also includes new topics <i>Enable Code Completion in Visual Studio Code</i> and <i>Debugging Your Kernel in Visual Studio Code</i>. - Revised <i>Minimum Latency Flow</i>. - Revised <i>Balanced Throughput-Area Trade-Offs Flow</i>. - Added <i>Minimum Area Flow</i>. - Revised information about encrypting your RTL IP core. Refer to <i>Encrypting RTL IP Core</i> for details. - Added <i>Allow Wide Memory Initialization (-Xsallow-wide-mif)</i> optimization target. - Added <i>Integrating Your Kernel into DSP Builder for Intel FPGAs</i>. - Removed references the following macros: - mmhost - register_map_mmhost - conduit_mmhost - streaming_interface - streaming_pipelined_interface - Deprecated the <code>--emulation_model</code> option of the <code>fpga_crossgen</code>. Use the <code>--cpp_model</code> option instead. - Renamed <code>register_map_offsets.hpp</code> and <code><kernel_name>_register_map.hpp</code> files. Starting with oneAPI 2024.1, these files are <code>register_map_offsets.h</code> and <code><kernel_name>_register_map.h</code>, respectively. - Revised <i>FPGA BSPs and Boards</i>. - Revised <i>FPGA Compilation Flags</i> - Changed the default compilation target when you do not specify the <code>-Xs target</code> compiler option. If you do not specify the option, the compiler assumes <code>-Xs target=Agilex7</code>. Previously, the default target was <code>-Xs target=intel_al0gx_pac:pac_al0</code>. - Revised <i>The device_global Extension (Beta)</i>. - Revised <i>Declare the ac_fixed Data Type</i>. - Moved <i>System-level Profiling Using the Intercept Layer for OpenCL™ Applications</i> to <i>Optimizing Your Host Application</i> section. - Rebrand Intel Agilex® 7 as Agilex™ 7 - Rebrand Intel® Arria® 10 as Arria® 10

Date	Release Version	Changes
		- Rebrand Intel® Cyclone® 10 GX as Cyclone® 10 GX - Rebrand Intel® Cyclone® V as Cyclone® V - Rebrand Intel® Stratix® 10 as Stratix® 10 - Rebrand Intel® Quartus® Prime as Quartus® Prime

Document Revision History for the Intel® oneAPI FPGA Handbook

This document was previous title *Intel® oneAPI FPGA Handbook*.

Date	Release Version	Changes
January 2024	2024.0	Revised <i>Memory-Mapped (MM) Agent Kernel Invocation Interface</i>. The topic previous listed bit 0 of the Start register as "reserved". Bit 0 is now correctly labeled "start".
November 2023	2024.0	- Created <i>Intel® oneAPI FPGA Handbook</i> with content from <i>FPGA Optimization Guide for Intel® oneAPI Toolkits</i> and <i>Intel® oneAPI Programming Guide</i>. - Deprecated the following macros: <code>mmhost</code> <code>register_map_mmhost</code> <code>conduit_mmhost</code> <code>streaming_interface</code> <code>streaming_pipelined_interface</code> These macros will be removed in a future release. - Updated <i>Conversion Rules for the <code>ap_float</code> Data Type</i>. - Added <i>Extracting the FPGA Hardware Configuration (.aocx) File from a Multiarchitecture Binary File</i>. - Updated <i>Agent IP Component Kernels</i>. - Revised <i>Host Pipes RTL Interfaces</i> to indicate the restriction on simulation for host pipes with <code>protocol_name::avalon_mm</code> is lifted. - Updated <i>The device_global Extension (Beta)</i> - Revised <i>Strategies for Inferring the Accumulator</i>. - Renamed <i>Maximum Throughput Without Area Optimization Heuristics Flow</i> to <i>Balanced Throughput-Area Trade-Offs Flow</i> and revised the topic. - Revised <i>ivdep Attribute</i> - Added information about the <code>annotated_arg</code> class as follows: - Added <i>Memory-Mapped Host Interfaces Using the <code>annotated_arg</code> Class</i> - Added <i>Avalon® Memory-Mapped Host Interfaces</i> - Revised <i>Memory-Mapped Host Interfaces</i> - Revised <i>Memory-Mapped Host Interfaces Using Unified Shared Memory</i> - Revised <i>Memory-Mapped Host Interfaces Using Accessors</i> - Deprecated the <code>[[intel::max_global_work_dim(0)]]</code> kernel attribute in the following topics:

Date	Release Version	Changes
		• Omit Hardware that Generates and Dispatches Kernel IDs • FPGA Kernel Attributes • Renamed <i>Agent IP Component Kernels</i> to <i>Avalon® Memory-Mapped (MM) Agent Kernel Invocation Interface</i> and revised the topic. • Renamed <i>Streaming IP Component Kernels</i> to <i>Ready/Valid Handshaking Kernel Invocation Interface</i> and revised the topic. • Revised <i>Pipelined Kernels</i>

Document Revision History for the FPGA Optimization Guide for Intel® oneAPI Toolkits

Most of the content in the *Intel® oneAPI FPGA Handbook* originally appeared in the *FPGA Optimization Guide for Intel® oneAPI Toolkits*.

Date	Release Version	Changes
July 2023	2023.2	• Revised the note about SYCL streams support and mentioned about using <code>sycl::ext::oneapi::experimental::printf</code> for debugging kernels. • Updated all occurrences of <code>-Xsno-interleaving=<global_memory_type></code> to <code>-Xsno-interleaving=<global_memory_name></code>. • Revised the information in the following topics: • <i>Optimization Targets</i> • <i>Minimum Latency Flow</i> • <i>Loop Analysis</i> • <i>Quartus (Static) Summary</i> • <i>Pipes</i> • <i>Simple Host-Device Streaming</i> • <i>Buffered Host-Device Streaming</i> • Added the following new topics: • <i>Generate Register Map Wrapper (-Xsregister-map-wrapper-type)</i> • <i>Maximum Throughput Without Area Optimization Heuristics Flow</i> • <i>Create a 2xclock Interface (-Xsuse-2xclock)</i> • <i>Host Pipes</i> • <i>Host Pipe Declaration</i> • <i>Host Pipe API</i> • <i>Host Pipes RTL Interfaces</i>
March 2023	2023.1	• Added <code>max_reinvocation_delay</code> loop attribute. • Updated the limitations of AC data types in <i>Advantages and Limitations of Arbitrary Precision Data Types</i>. • Added another option to the <code>-Xshyper-optimized-handshaking</code> compiler flag in <i>Modify the Handshaking Protocol Between Clusters (-Xshyper-optimized-handshaking)</i>. • Updated the values of supported data types and math operations in <i>Control Hardware Implementation of the Supported Data Types and Math Operations</i>. • Rebranded Intel® Agilex™ as Intel Agilex® 7. • Added <code>device_globals</code> extension. • Added <i>Minimum Latency Flow</i>.

Date	Release Version	Changes
		- Added <i>Optimization Levels</i>. - Revised the information in <i>Perform Kernel Computations Using Local or Private Memory</i>. - Reorganized all extensions under <i>FPGA Extensions</i>. - Added examples of declaring <code>ac_complex</code> variables in <i>Declare the ac_complex Data Type</i>.
December 2022	2023.0	- Revised all references to <code>#include <CL/...></code> as <code>#include <sycl/...></code>. - Revised all references to <code>using namespace cl::sycl;</code> as <code>using namespace sycl;</code>. - Revised all references to dpcpp compiler driver to <code>icpx -fsycl</code>. - Added a note about avoiding adding the result of the <code>convert_to</code> function to another <code>ap_float</code> variable in <i>Conversion Rules for ap_float</i>. - Added a note about constant propagation optimization technique in <i>Operations with Explicit Precision Controls</i>. - Added additional about values supported by <code>N</code> for the <code>[[intel::num_simd_work_items(N)]]</code> attribute in <i>Specify Number of SIMD Work-Items</i>. - Revised <i>Latency Controls</i> topic to include a section on using latency controls with stall-free loop. - Added information about <code>ihc::FPsingle</code> and <code>ihc::FPdouble</code> to <i>Declare the ap_float Data Type and Additional Data Types Provided by the ap_float.hpp Header File</i>. - Added new sections on explicit conversion functions in <i>Declare the ac_fixed Data Type</i> and <i>Declare the ac_int Data Type</i> topics. - Removed deprecation notice for <code>hls_float</code> data type renamed to <code>ap_float</code>. - Added additional information about using the Intel® oneAPI FPGA Reports tool in <i>Review the Optimization Report (report.html)</i>.
September 2022	2022.3	- Added a note about setting the <code>safelen</code> parameter with 0 or 1 in <i>ivdep Attribute</i>. - Added a note about applying the <code>ivdep</code> attribute to an array in <i>ivdep Attribute</i>. - Minor update to example code in <i>max_interleaving Attribute</i>. - Updated all images and made moderate updates in all topics in the <i>Analyze the FPGA Early Image</i> section. - Made minor updates in Pipes topic. - Updated <i>Latency Controls (Beta)</i> topic completely. - Added a note about using the <code>buffer_location<index></code> on BSPs with heterogeneous memory support in combination with read-only cache in <i>Partitioning Buffers Across Different Memory Types (Heterogeneous Memory)</i>. - Revised the guidance in <i>Timing Failures</i>. - Made minor revisions in <i>Zero-Copy Memory Access</i> and <i>Prepinning Memory</i> topics. - Added the following new topics: <i>System of Tasks (task_sequence) Extension</i> <i>Task Functions</i> <i>task_sequence Use Cases</i>

Date	Release Version	Changes
April 2022	2022.2	- Changed the document title <i>Intel® oneAPI DPC++ FPGA Optimization Guide</i> to <i>FPGA Optimization Guide for Intel® oneAPI Toolkits</i>. - Replaced all general references to DPC++ with SYCL. - Added new loop functions to the existing list in <i>FPGA Loop Directives</i>. - Added a new limitation and removed some existing limitations in <i>Advantages and Limitations of Arbitrary Precision Data Types</i>. - Replaced all references to https://hlslibs.org/ with a reference to the documentation at https://github.com/hlslibs/ac_types/blob/v3.7/pdfdocs/ac_datatypes_ref.pdf. - Made a minor update to the description in <i>The pipe Class and its Use</i>. - Modified all occurrences of <code>[[cl::reqd_work_group_size(Z, Y, X)]]</code> to <code>[[sycl::reqd_work_group_size(Z, Y, X)]]</code>. <code>[[cl::reqd_work_group_size(Z, Y, X)]]</code> is now deprecated. - Added the following new topics: <i>Control Hardware Implementation of the Supported Data Types and Math Functions</i> (<code>-Xsdsp-mode=<option></code>) <i>Latency Controls</i>
December 2021	2022.1	- Added new attributes to the existing list in <i>FPGA Loop Directives</i> and <i>FPGA Optimization Flags</i>. - Added the following new topics: <i>Global Control of Exit FIFO Latency of Stall-free Clusters</i> (<code>-Xssfc-exit-fifo-type=<value></code>) <i>nofusion Attribute</i> <i>Enable the Read-Only Cache for Read-Only Accessors</i> (<code>-Xsread-only-cache-size=<N></code>) - Renamed <code>hls_float</code> data type to <code>ap_float</code>. Added support for the header files <code><sycl/ext/intel/ac_types/ap_float.hpp></code> and <code><sycl/ext/intel/ac_types/ap_float_math.hpp></code>. In a future release, the header files <code><sycl/ext/intel/ac_types/hls_float.hpp></code> and <code><sycl/ext/intel/ac_types/hls_float_math.hpp></code> will be removed. - Updated few limitations in <i>Advantages and Limitations of Arbitrary Precision Data Types</i>.
September 2021	2021.4	- Changed all occurrences of the namespace <code>sycl::ONEAPI::</code> to <code>sycl::ext::oneapi::</code>. - Changed all occurrences of the namespace <code>sycl::INTEL::</code> to <code>sycl::ext::intel::</code>. - Changed all occurrences of the header <code>#include <CL/sycl/INTEL/...></code> to <code>#include <sycl/ext/intel/...></code>. - Changed <code>clang++</code> to <code>dpcpp</code>. - Changed all occurrences of <code>[[intel::ii(n)]]</code> to <code>[[intel::initiation_interval(n)]]</code>. - Minor update about the types of FIFOs reported in the <i>System Viewer</i>. - Added a note about multi-process execution in the installed FPGAs in <i>Multi-Threaded Host Application</i>. - Added an additional value on to <code>#pragma fp contract(fast/off)</code>.

Date	Release Version	Changes
		- Updated the code snippets in <i>Specify Schedule F_{MAX} Target for Kernels and Memory Attributes</i>. - Modified the flag description for AC type in <i>Variable-Precision Integer and Floating-Point Support</i>. - Modified the list of limitations in <i>Advantages and Limitations of Arbitrary Precision Data Types</i>. - Added information about debugging in <i>Declare the <code>ac_int</code> Data Type</i>. - Added support for simulation flow. - Changed all occurrences of Mentor Graphics* to Siemens EDA* (formerly Mentor Graphics) . - Changed the <code>noinit</code> attribute to <code>no_init</code>. - Modified the description of <code>[[intel::kernel_args_restrict]]</code> attribute in <i>Global Memory Accesses Optimization</i>. - Replaced the flag <code>-Xsffpcontract=<value></code> with <code>-fp-model=<value></code> and updated its description in <i>Reducing Floating-Point Rounding Operations</i>. - Removed few limitations of shift register in <i>Kernel Variable Accesses</i>. - Added the following new topics: <i>Modify the Rounding Mode of Floating-point Operations (-Xsrounding=<rounding_type>)</i> <i>Reduce Kernel Area and Latency</i> (<code>[[intel::use_stall_enable_clusters]]</code> attribute) <i>Timing Failures</i>
July 2021	2021.3	Bug fixes.
June 2021	2021.3	- Added more information about streaming data using I/O pipes and faking I/O pipes in <i>I/O Pipes</i>. - Added description about the local memory function in <i>Kernel Memory</i>. - Added a new section to describe the global memory bandwidth use calculation in <i>Global Memory Accesses Optimization</i>. - In <i>System Viewer</i>, added description about the global memory view, changed the Graph Viewer report name to System Viewer, updated all images and made minor updates in the remaining part of the topic. - Updated the description and image in <i>Schedule Viewer</i>. - Updated most of the code snippets to reflect the latest coding practices. - Made minor updates to the description of <i>ivdep Attribute</i> and <i>loop_coalesce Attribute</i>. - Added MLABs to the description in <i>Perform Kernel Computations Using Local or Private Memory</i>. - Modified the description in <i>Strategies for Inferring the Accumulator</i> to remove references to the <code>-Xsfp-relaxed</code> flag. - Updated the compilation options in <i>Variable-Precision Integer and Floating-Point Support</i>. - Updated the limitations in <i>Advantages and Limitations of Arbitrary Precision Data Types</i>. - Merged <i>Ignoring Dependencies Between Accessor Arguments</i> topic with <i>Global Memory Accesses Optimization</i>. - Updated the list of attributes in <i>FPGA Kernel Attributes</i>. - Added the following new topics:

Date	Release Version	Changes
		• <i>Shannonization to Improve F_{MAX}/II</i> • <i>Simple Host-device Streaming</i> • <i>Buffered Host-device Streaming</i> • <i>N-Way Buffering</i> • <i>Optimize Inner Loop Throughput</i> • <i>Improve Loop Performance by Caching On-Chip Memory</i> • <i>FPGA Local Memory Function</i> • <i>Pipelining Loops in Non-task Kernels (<code>-Xsauto-pipeline</code>)</i> • <i>Pipe and Atomic Fence</i> • <i>Reducing Floating-Point Rounding Operations (<code>-Xsfpc-contract=fast</code>)</i> • <i>FPGA Accessor Properties</i> • Removed the following topics because the respective flags are now deprecated: • <i>Relax the Order of Floating-Point Operations (<code>-Xsfp-relaxed</code>)</i> • <i>Reduce Floating-Point Rounding Operations (<code>-Xsfpc</code>)</i> • Removed <i>Tree Balancing and Rounding Operations</i> topic from <i>Optimize Floating-point Operation</i>.
March 2021	2021.2	• Updated the topic <i>Cluster the Datapath</i> completely including the diagrams. • Updated the dependency graph in <i>Mapping Source Code Instructions to Hardware</i>. • Updated <i>Hyper-Optimized Handshaking Data Flow</i> diagram in <i>Handshaking Between Clusters</i> topic. • Made minor updates in <i>Executing Independent Operations Simultaneously</i>. • Updated the topic <i>Pipelining</i> completed including the diagrams. • Added a note about the enablement of fast math operations for floating point operations in <i>Data Types and Operations</i>. • Updated the topic <i>Access HLD FPGA Reports in JSON Format</i> to include details about the Bottlenecks viewer. • Made minor update to a note in <i>Specify Number of SIMD Work-Items</i> • Added the following new topics: • <i>Floating Point Optimizations</i> • <i>Floating Point Pragmas</i> • <i>Specify Schedule F_{MAX} Target for Kernels</i> • <i>Bottlenecks Viewer</i> • <i>Loop Bottlenecks</i> • <i>Variable-Precision Integer and Floating-Point Support</i> • <i>Advantages and Limitations of Arbitrary Precision Data Types</i> • <i>Declare and Use the AC Data Types</i> • <i>Declare the <code>ac_int</code> Data Type</i> • <i>Declare the <code>ac_fixed</code> Data Type</i> • <i>Declare the <code>ac_complex</code> Data Type</i> • <i>Declare the <code>hls_float</code> Data Type</i> • <i>Conversion Rules for <code>hls_float</code></i> • <i>Operations with Explicit Precision Controls</i> • <i>Comparison Operators</i> • <i>Additional <code>hls_float</code> Functions</i> • <i>Additional Data Types Provided by <code>hls_float.hpp</code></i>
December 2020	2021.1	First major release.

Notices and Disclaimers

Intel technologies may require enabled hardware, software or service activation.

No product or component can be absolutely secure.

Your costs and results may vary.

© Intel Corporation. Intel, the Intel logo, and other Intel marks are trademarks of Intel Corporation or its subsidiaries. Other names and brands may be claimed as the property of others.

Product and Performance Information

Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Notice revision #20201201

Unless stated otherwise, the code examples in this document are provided to you under an MIT license, the terms of which are as follows:

Copyright Intel Corporation

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.